

The $\text{\texttt{sTeX}}$ 4 Package Collection*

Michael Kohlhase, Dennis Müller
FAU Erlangen-Nürnberg
<http://kwarc.info/>

2026-08-08

Contents

1	Introduction & Setup	8
1.1	What is $\text{\texttt{sTeX}}$?	8
1.2	The $\text{\texttt{sTeX}}$ package	9
1.3	$\text{\texttt{Math}}$ archives and the MathHub Directory	9
1.4	Setting Up the $\text{\texttt{F\&Mj}}$ IDE	9
I	Tutorial	11
2	The Basics	12
2.1	Text symbols	15
2.1.1	Using $\text{\texttt{Modules}}$ & Search in the $\text{\texttt{IDE}}$	15
2.2	$\text{\texttt{Symbol}}$ References	17
2.3	$\text{\texttt{Modules}}$ and Simple $\text{\texttt{Symbol}}$ Declarations	20
2.4	Documenting $\text{\texttt{Symbols}}$	23
2.5	Sectioning and Reusing Document Fragments	25
2.6	Building and Exporting $\text{\texttt{HTML}}$	27
3	Mathematical Concepts	30
3.1	Simple $\text{\texttt{Symbol}}$ Declarations	30
3.1.1	$\text{\texttt{Semantic Macros}}$ and $\text{\texttt{Notations}}$	30
3.1.2	$\text{\texttt{Types}}$ and $\text{\texttt{Variables}}$	33
3.1.3	$\text{\texttt{Flexary Macros}}$ and $\text{\texttt{Argument Modes}}$	35
3.1.4	$\text{\texttt{Precedences}}$	37
3.1.5	Finishing Equality	38
3.1.6	Variable Sequences	38
3.2	Statements	38
3.2.1	Definitions	39
	$\text{\texttt{Semantic Macros}}$ in Text Mode	40
	Definientia	41

*Version 4.0.1 (last revised 2026-08-08)

	Using <code>Symbols</code> Without <code>Semantic Macros</code> and Exporting Code in <code>Modules</code>	42
3.2.2	Assertions	43
3.2.3	Proofs	45
3.3	Mathematical Structures	45
3.3.1	Declaring and Using <code>Structures</code>	45
	Instantiating <code>Structures</code>	46
3.3.2	Extending <code>Structures</code> and Axioms	48
	Conservative Extensions	49
3.3.3	Nesting <code>Structures</code> and <code>\this</code>	50
3.4	Complex Inheritance and Theory Morphisms	52
3.4.1	Glueing <code>Structures</code> Together	55
3.4.2	Realizations	56
4	Extensions for Education	59
4.1	Slides and Course Notes	59
4.1.1	Introduction	59
4.1.2	Package Options	60
4.1.3	Notes and Slides	60
4.1.4	Managing and Customizing Themes	61
4.1.5	Frame Images	62
4.1.6	Ending Documents Prematurely	63
4.1.7	Global Document Variables	63
4.1.8	Excursions	63
4.2	Problems and Exercises	64
4.2.1	Background	64
4.2.2	The Package, Options, and Configuration	65
4.2.3	(Open) Problems	66
4.2.4	Solutions and Answer Classes	67
4.2.5	Grading Support	68
4.2.6	Structured Problems	69
4.2.7	Single/Multiple Choice Blocks	70
4.2.8	Filling-In Concrete Solutions	73
4.2.9	Including Problems	75
4.2.10	Testing and Spacing	75
4.3	Homework Assignments and Exams	76
4.3.1	Introduction	76
4.3.2	Package Options	76
4.3.3	Assignments	76
4.3.4	Including Assignments	76
4.3.5	Typesetting Exams	77
II	User Manual	78

5	Basics	79
5.1	Package and Class Options	79
5.2	Math Archives and the MathHub Directory	80
5.2.1	The Structure of Math Archives	81
5.2.2	MANIFEST.MF-Files	81
5.3	The <code>lib</code> -Directory	82
5.4	Basic Macros	83
6	Document Features	84
6.1	Document Fragments	84
6.2	Using and Referencing Document Fragments	85
6.3	Cross-Document References	85
7	Modules and Symbols	88
7.1	Modules	88
7.1.1	Signature Modules , Languages, and Multilinguality	89
7.2	Symbol Declarations	89
7.2.1	Returns	90
7.3	Referencing Symbols	90
7.4	Notations and Semantic Macros	92
7.4.1	Precedences and Bracketing	93
7.4.2	Notations for Argument Sequences	94
7.4.3	Semantic Macros	94
7.5	Simple Inheritance	95
7.6	Variables and Sequences	97
7.7	Structures	98
7.7.1	Semantic Macros for Structures	99
8	Statements	101
8.1	More on Definitions	102
8.2	More on Assertions	103
9	Customizing Typesetting	104
9.1	Highlighting Symbol References	104
9.2	Styling Environments and Macros	105
9.3	Custom CSS for Environments	107
10	Additional Packages	108
10.1	NotesSlides Manual	108
10.1.1	Introduction	108
10.1.2	Package Options	108
10.1.3	Notes and Slides	109
10.1.4	Customizing Header and Footer Lines	110
10.1.5	Frame Images	110
10.1.6	Ending Documents Prematurely	111
10.1.7	Global Document Variables	111
10.1.8	Excursions	112
10.2	Problem Manual	113
10.2.1	Introduction	113
10.2.2	Problems and Solutions	113

10.2.3	Markup for Added-Value Services	115
	Multiple Choice Blocks	115
	Filling-In Concrete Solutions	116
10.2.4	Including Problems	117
10.2.5	Testing and Spacing	118
10.3	HWExam Manual	118
10.3.1	Introduction	118
10.3.2	Package Options	118
10.3.3	Assignments	119
10.3.4	Including Assignments	119
10.3.5	Typesetting Exams	119
10.4	Tikzinput Manual	120
III	Documentation	122
11	Utilities	123
11.1	kpsewhich and Environment Variables	123
11.2	Logging	124
11.3	File Paths	124
11.4	Group-like Behaviours	125
11.5	Key Handling	126
11.6	Languages	127
11.7	Styling	127
11.8	Persisting Dependencies	127
12	HTML Output	128
13	Math Archives	132
14	URIs	134
14.1	Document URIs	134
14.2	Module URIs	135
14.3	Symbol URIs	136
15	Documents	138
15.1	Sectioning	139
15.2	Inter-Document References	140
15.3	Inputting From MathHub Resources	140
16	Modules	142
16.1	SMS Mode	143
16.2	Inheritance and Morphisms	145
16.3	Theory Morphisms	145

17 Symbols	147
17.1 Declarations	147
17.2 Retrieval	148
17.3 Variables	148
17.3.1 Variable Sequences	149
17.4 Expressions	149
17.4.1 Term Annotations	150
17.4.2 Symbol Arguments	151
17.4.3 Notation components	151
17.4.4 Symbol References	152
17.5 Checking	152
18 Notations	153
18.1 Automated Bracketing	154
19 Structures	156
20 Statements	157
21 Proofs	158
22 Metatheory	160
23 Others	161
24 Additional Packages	162
24.1 NotesSlides Documentation	162
24.2 Problem Documentation	162
24.3 HWExam Documentation	162
24.4 Tikzinput Documentation	162
IV Implementation	163
25 Setting up	164
26 Utilities	166
26.1 kpsewhich and Environment Variables	167
26.2 Logging	168
26.3 File Paths	169
26.4 Group-like Behaviours	173
26.5 Key Handling	174
26.6 Languages	176
26.7 Styling	177
26.8 Persisting Dependencies	179
26.9 Auxiliary Methods	181
27 HTML Output	182
28 Math Archives	187

29 URIs	194
29.1 Document URIs	194
29.2 Module URIs	197
29.3 Symbol URIs	201
30 Documents	204
30.1 Sectioning	206
30.2 Inter-Document References	211
30.3 Inputting From MathHub Resources	216
31 Modules	223
31.1 SMS Mode	231
31.2 Inheritance and Morphisms	235
31.3 Theory Morphisms	240
32 Symbols	255
32.1 Declarations	255
32.2 Retrieval	263
32.3 Variables	267
32.3.1 Variable Sequences	270
32.4 Expressions	272
32.4.1 Term Annotations	293
32.4.2 Symbol Arguments	295
32.4.3 Notation components	301
32.4.4 Symbol References	303
32.5 Checking	307
33 Notations	310
33.1 Automated Bracketing	325
34 Structures	327
35 Statements	333
36 Proofs	340
37 Metatheory	353
38 Others	356
39 Additional Packages	358
39.1 Implementation: The notesslides Package	358
39.1.1 Class and Package Options	358
39.1.2 Notes and Slides	362
39.1.3 Environment and Macro Patches	366
39.1.4 Styling Across Notes/Slides	367
39.1.5 Beamer Compatibility	368
39.1.6 TODO Excursions	369
39.2 Implementation: The problem Package	371
39.2.1 Package Options	371
39.2.2 Problems and Solutions	373

39.3	Implementation: The hwexam Package	391
39.3.1	Package Options	391
39.3.2	QR Codes	392
39.3.3	Assignments	398
39.3.4	Leftovers	402
39.4	Tikzinput Implementation	403

Index 405



sTeX is – by now – relatively stable and ready to use for the general public. However, it is also actively being developed further and subject to ongoing research. Some of the features described in here might not fully work as expected, some are still experimental, there might occasionally be cryptic error messages, and in general bugs are expected.

We welcome all kinds of issues you might encounter at <https://github.com/slatex/sTeX>.

*If you have questions or problems with **sTeX**, you can talk to us directly at <https://matrix.to/#/#stex:fau.de>.*

Chapter 1

Introduction & Setup

1.1 What is $\text{\textcolor{teal}{s}TeX}$?

$\text{\textcolor{teal}{s}TeX}$ is a system for generating human-oriented documents in either $\text{\textcolor{teal}{P}DF}$ or $\text{\textcolor{teal}{H}TML}$ format, augmented with computer-actionable semantic information (conceptually) based on the $\text{\textcolor{teal}{O}MDoc}$ format and ontology.

At its core is the $\text{\textcolor{teal}{s}TeX}$ package for $\text{\textcolor{teal}{L}aTeX}$, that allows for semantically marking up document fragments; in particular concepts, $\text{\textcolor{teal}{f}ormulae}$ and $\text{\textcolor{teal}{m}athematical}$ statements (such as definitions, theorems and proofs). Running $\text{\textcolor{teal}{p}dflatex}$ over $\text{\textcolor{teal}{s}TeX}$ -annotated documents formats them into normal-looking $\text{\textcolor{teal}{P}DF}$.

But $\text{\textcolor{teal}{s}TeX}$ also comes with a conversion pipeline into semantically annotated $\text{\textcolor{teal}{H}TML}$ ($\text{\textcolor{teal}{F}TML}$), which can host semantic added-value services that make the documents *active* (i.e. interactive and user-adaptive) – essentially turning $\text{\textcolor{teal}{L}aTeX}$ into a document format for (mathematical) knowledge management (MKM).

The $\text{\textcolor{teal}{s}TeX}$ system consists of the following components:

- The $\text{\textcolor{teal}{s}TeX}$ package,
- the $\text{\textcolor{teal}{R}usTeX}$ system for converting $\text{\textcolor{teal}{L}aTeX}$ documents to (semantically enriched) $\text{\textcolor{teal}{F}TML}$,
- the $\text{\textcolor{teal}{F}lMf}$ system; a semantically informed back-end for managing and hosting the $\text{\textcolor{teal}{F}TML}$ with integrated added-value services,
- a collection of $\text{\textcolor{teal}{m}ath archives}$ of $\text{\textcolor{teal}{s}TeX}$ document fragments and $\text{\textcolor{teal}{s}ymbols}$ for reuse, available of $\text{\textcolor{teal}{m}athhub.info}$, and
- the $\text{\textcolor{teal}{F}lMf IDE}$: A $\text{\textcolor{teal}{V}S Code}$ extension that, besides the usual $\text{\textcolor{teal}{I}DE}$ functionalities like syntax highlighting, integrates $\text{\textcolor{teal}{F}lMf}$ and allows for accessing the public $\text{\textcolor{teal}{m}ath archives}$ on $\text{\textcolor{teal}{m}athhub.info}$ to make the entire toolchain easily accessible to authors.

If $\text{\textcolor{teal}{P}DF}$ is all you are interested in, the $\text{\textcolor{teal}{s}TeX}$ package is all you *need*. Either way, however, we recommend using the $\text{\textcolor{teal}{s}TeX IDE}$ – it very much helps with semantically annotating your $\text{\textcolor{teal}{L}aTeX}$ documents.

1.2 The $\text{\texttt{sTeX}}$ package

The $\text{\texttt{sTeX}}$ package extends $\text{\texttt{L^{A}T\textsubscript{E}X}}$ with:

- A mechanism to declare **symbols** – concepts, functions, relations, variables, etc., which can be used and referenced in text or via **semantic macros** in mathematical formulae,
- a **module system** based on logical identifiers – **modules** bundle declarations, definitions, theorems, document snippets, and **symbols** for reuse, and
- an analogous organizational structure for developing documents modularly from individual fragments and sections.

The $\text{\texttt{sTeX}}$ package has been designed to have minimal impact on other **packages** and **document classes**, or the actual document layout – formatting of semantic **environments**, **symbol** references and **semantic macros** can be fully customized.

1.3 Math archives and the MathHub Directory

To make the most of $\text{\texttt{sTeX}}$, it is strongly encouraged to follow a workflow of small document fragments and **modules** to maximize reuse.

One considerable weakness of $\text{\texttt{L^{A}T\textsubscript{E}X}}$ is the way source files are referenced: they need to be either in a **texmf** directory, or else be referenced via file paths relative to the main **.tex**-file being compiled. This is highly inconvenient if we want to collaboratively develop many highly interrelated document fragments.

$\text{\texttt{sTeX}}$ therefore adds an organizational layer on top of $\text{\texttt{L^{A}T\textsubscript{E}X}}$'s: **math archives** stored in a fixed **MathHub** directory anywhere on your hard drive. Referencing source files and **modules** is then done relative to the containing **math archive**, and is thus *independent* of user's individual setups or the current **.tex**-file.

The drawback of this approach is that $\text{\texttt{sTeX}}$ needs to know the location of your **MathHub** directory. There are multiple ways to achieve that, but the simplest and recommended approach is to set an environment variable: Simply create a new directory **<path>/MathHub** somewhere on your hard drive and set the environment variable **MATHHUB** as the path to this new directory.

Alternatively, you can let the **F\textsubscript{M}\textsubscript{J} IDE** do the work for you (see section 1.4).

For more on **math archives**, see Section 0.1 (**Math Archives** and the **MathHub** Directory) in the $\text{\texttt{sTeX}}$ Manual.

1.4 Setting Up the **F\textsubscript{M}\textsubscript{J} IDE**

$\text{\texttt{sTeX}}$ is based on $\text{\texttt{L^{A}T\textsubscript{E}X}}$, and adds additional layers of presentational and functional markup to it. As a consequence the source files of $\text{\texttt{sTeX}}$ documents look quite different from the resulting **FTML** and **PDF** documents. Thus the best way of interacting the $\text{\texttt{sTeX}}$ document collections is via an **integrated development environment (IDE)**. In this tutorial we will use the **F\textsubscript{M}\textsubscript{J}** plugin for the **VS Code**, which you should set up as a first step (this also sets up the necessary auxiliary software).

Setting up $\text{\texttt{sTeX}}$ with the dedicated **IDE** is easy:

1. Download and install **VS Code** here: <https://code.visualstudio.com/download>

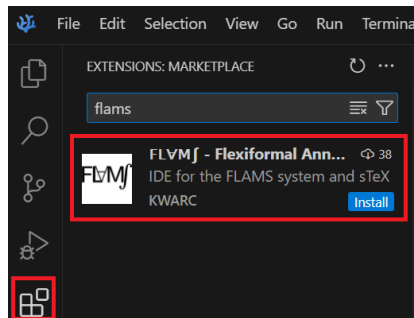


Figure 1: Installing the FLVMj IDE

2. Start **VS Code** and navigate to the *Extensions*-tab on the left. Here you can search for Extensions in the **VS Code** marketplace. Look for the FLVMj extension by *KWARC*, as in Figure 1.
3. Having done so, upon opening any folder in **VS Code** containing a `.tex`-file, you will be prompted to either download a new FLVMj or point the extension to an existing FLVMj executable. In the vast majority of cases, you'll want to download and choose a directory to install FLVMj to.

Part I

Tutorial

*The dynamic [HTML](#) version of this part can be found at
<https://mathhub.info/?a=sTeX/Documentation&d=tutorial&l=en>*

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

In this part, we will give a broad but shallow introduction to [sTeX](#), and what you can get out of it. Additionally, this serves as an introduction to the [FvMf IDE](#), and we consequently assume that you have that one set up, as described in section 1.4.

Note that in [PDFs](#), the specific highlighting of semantically annotated text is fully customizable (see chapter 9). In this document, we use [this highlighting](#) for [notation](#) components, [this highlighting](#) for [symbol](#) references, [this highlighting](#) for (local) [variables](#) and [this highlighting](#) for definienda; i.e. new concepts being introduced.

Chapter 2

The Basics

This document itself uses [sTeX](#) and serves as a direct example for the following. You can download its source files, the generated [PDF](#) files, and the generated [HTML](#) documents directly from within the [IDE](#), by navigating to the [FLMf](#) tab in the menu on the left and finding [sTeX/Documentation](#) in the list of [math archives](#) and clicking the small “Install”-button next to it, see the screenshot on the left of Figure 2.

Once downloading is finished (this may take a while since dependencies are also downloaded), you can then browse the [.tex](#)-files in [sTeX/Documentation](#) directly from the [math archives](#) panel in the [sTeX](#) tab, as you can see in the right screenshot in Figure 2.

For example, you can now navigate to the file [tutorial/intro.en](#) to see the sources of this very chapter.

As a first example, consider the following document fragment from section 1.1:

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

If you were to look at the generated [HTML](#) from this fragment, you could hover over the highlighted words ([sTeX](#), [PDF](#), [HTML](#), [OMDoc](#)) and get a little popup with their definitions (Figure 3). Neat, huh?

Here, in the [PDF](#), hovering will only show you a unique identifier ([MMT-URI](#)) for the word, and link to a definition on the web. Still useful, but not quite as neat, of course.

A plain [L^AT_EX](#)-version of the above document fragment, without any [sTeX](#) markup, could look like this:

Example 1

Input:

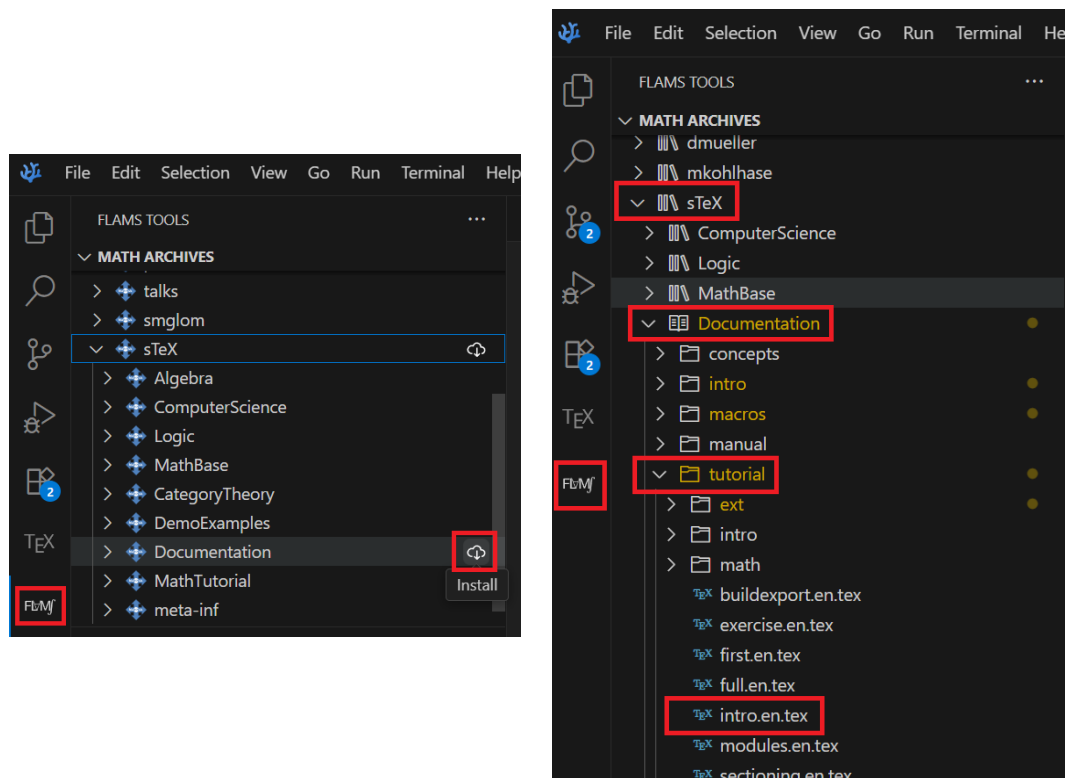


Figure 2: Installing Math Archives

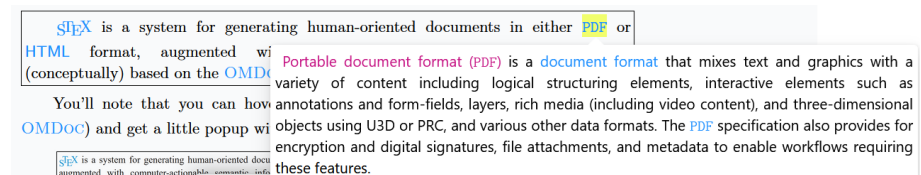


Figure 3: Definition on Hover

```

File tutorial/intro/intro1plain.en.tex
1 \documentclass{article}
2 \usepackage{stex-logo}
3 \begin{document}
4
5   \sTeX{} is a system for generating human-oriented documents
6   in either \textsf{PDF} or \textsf{HTML} format, augmented
7   with computer-actionable semantic information (conceptually)
8   based on the \textsc{OMDoc} format and ontology.
9
10 \end{document}

```

Output:

sTeX is a system for generating human-oriented documents in either PDF or HTML format, augmented with computer-actionable semantic information (conceptually) based on the OMDoc format and ontology.

(Examples like the one above always show the file the source code is in, so if you have downloaded the sTeX /Documentation [math archive](#) you can toy around with it yourself)

If you save a file in the **IDE** (regardless of whether it has unsaved changes), a preview window will pop up, showing you the **HTML** generated from the **.tex**-file; see (Figure 4).

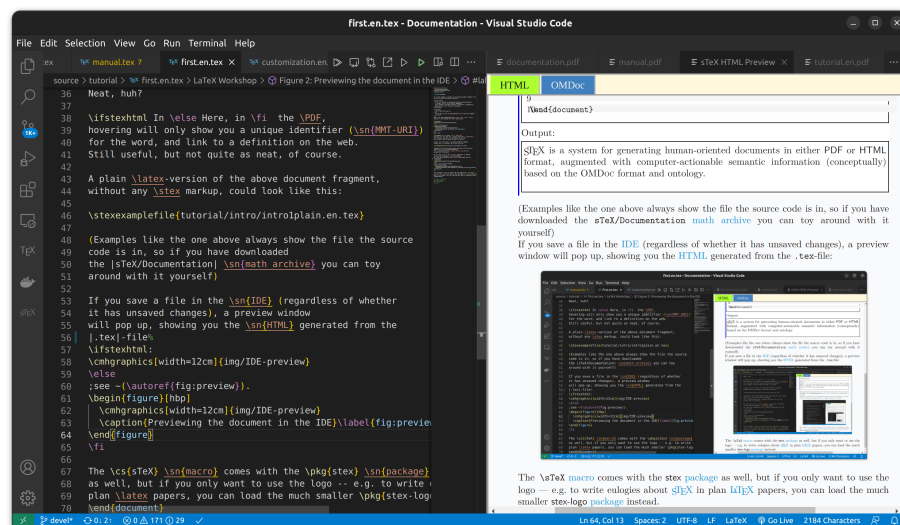


Figure 4: Previewing the document in the IDE

The sTeX macro comes with the **stex** package as well, but if you only want to use the logo – e.g. to write eulogies about sTeX in plain $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ papers, you can load the much smaller **stex-logo** package instead.

2.1 Text symbols

The most central concept behind `TeX` is that of a *symbol*:

STEX A **symbol** is a *named* concept that can be defined, documented and referenced. Examples for **symbols** are mathematical constants, functions, theorems, statements, principles – anything that has a (somewhat) precise meaning and can be referenced by name can be a **symbol**.

Before we explain how we can declare new **symbols** and associate them with definitions, **notations** and all that, let's assume an ideal world in which others have done that job already for us – after all, **STEX** is all about *reuse*, and naturally, there are **STEX symbols** for all of the above already. Let's start with the one for **STEX** itself:

2.1.1 Using Modules & Search in the IDE

In the **VS Code IDE**, navigate to the **F₅M₂**-tab on the left. In the search panel, select the “**Symbols**” radio button and search for “**sTeX**”. The third search result should be what we’re looking for (Figure 5).

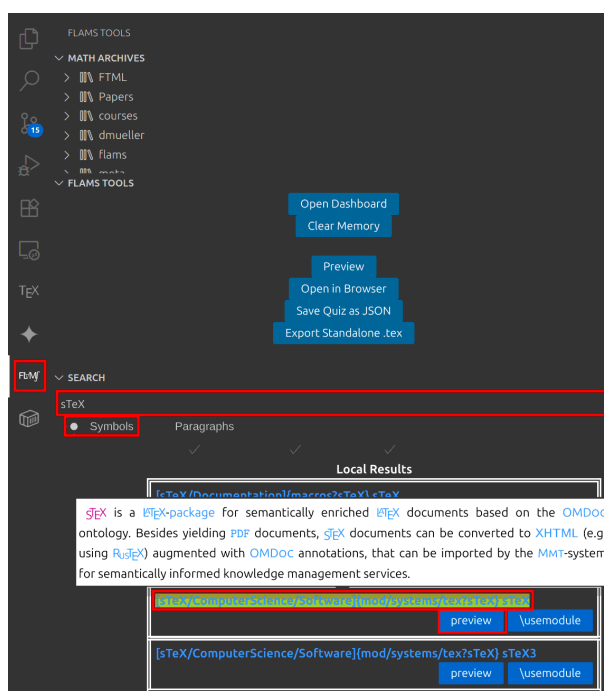


Figure 5: Search in the `STEX` IDE

Search results are grouped into *local* and *remote* results. Local ones are the ones you already have in your local MathHub directory; remote ones you can download directly from within the IDE.

You can click the preview button to see the generated FTML for the document – the resulting window that pops up also has an OMDoc button you can click, which (among other things) shows you the semantic macros provided by the respective module: In this case, it tells us that there is a *text symbol* named “sTeX” with semantic macro `\stex` in the module sTeX in the same document. It produces the presentation “STEX” as we want (Figure 6).

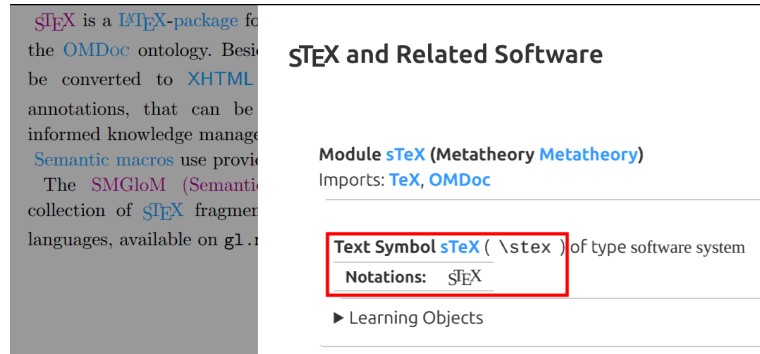


Figure 6: OMDoc Preview

STEX A *text symbol* is a *symbol* foo with an associated *semantic macro* `\foo`. The *macro* `\foo` is allowed in text or math mode and produces a predefined piece of text output annotated with foo. The variant `\fooname` produces the same output without annotation.

If we want to use the `STEX` symbol in a document – which we have open in the IDE – we simply click on the `\usemodule` button, and the IDE will automatically insert the line `\usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}`, making all *symbols* in that *module* available to use – in particular, we can now use the `\stex` *semantic macro* instead of the plain, non-semantic `\sTeX` *macro* – that is, of course, after we include the `stex` *package* first.

STEX The `\usemodule` *macro* takes as *optional* argument the name of a *math archive*, and as a regular argument the path to an `STEX` *module* (see section 7.5).

Analogously, we can also search for the `PDF`, `HTML` and `OMDoc` symbols, all of which are also *text symbols* and have the associated *semantic macros* `\PDF`, `\HTML` and `\omdoc`; the document should thus look like this:

Example 2

Input:

File tutorial/intro/intro1stex.en.tex

```
1 \documentclass{article}
2 \usepackage{stex}
3 \begin{document}
4   \usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}
5   \usemodule[sTeX/ComputerScience/Software]{mod/formats?PDF}
6   \usemodule[sTeX/ComputerScience/Software]{mod/formats?HTML}
7   \usemodule[sTeX/ComputerScience/Software]{mod/formats?OMDoc}
8
9   \stex is a system for generating human-oriented documents
10  in either \PDF or \HTML format, augmented
11  with computer-actionable semantic information (conceptually)
12  based on the \omdoc format and ontology.
13 \end{document}
```

Output:

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

Now, our generated [HTML](#) looks a lot more interesting, with highlighting, pop-ups on hover and all that. Notably however, if we compile the file with `pdflatex`, it looks pretty much exactly as before.

That's because we haven't told [sTeX](#) what to do with semantic annotations yet – and by default, it does not do anything fancy, except for wrapping them in an `\emph`. We can customize how we want [sTeX](#) to highlight various semantic text fragments (see chapter 9). A default highlighting schema is provided in the `stex-highlighting` [package](#) – including that will

- highlight semantically annotated text in [this color](#),
- show the [MMT-URI](#) of the corresponding [symbol](#) in a tooltip on hovering over the text,
- make the text link to the place the [symbol](#) is being defined in the current document (if it is), or, alternatively,
- make it link to an external resource, if one is known. In our case, they link to `stexmmt.mathhub.info/:sTeX`, where the [HTML](#) for all the [symbols](#) we use in this document are hosted.

2.2 [Symbol](#) References

Let's continue with the next paragraph of section 1.1; for now unannotated:

[Example 3](#)

Input:

File tutorial/intro/intro2plain.en.tex

```
1 \documentclass{article}
2 \usepackage{stex}
3 \begin{document}
4
5   At its core is the \sTeX{} package for \LaTeX, that allows for
6   semantically marking up document fragments; in particular
7   concepts, formulae and mathematical statements (such as
8   definitions, theorems and proofs). Running \texttt{pdflatex}
9   over \sTeX-annotated documents formats them into normal-looking
10  \textsf{PDF}.
11
12 \end{document}
```

Output:

At its core is the $\text{\sTeX}$ package for $\text{\LaTeX}$, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over $\text{\sTeX}$ -annotated documents formats them into normal-looking PDF.

We already know how to annotate “ $\text{\sTeX}$ ” and “PDF”; and if we use the search field in the IDE again, we can also find a *text symbol* for “ $\text{\LaTeX}$ ”. But if we look at the documentation, we will note that *more* is highlighted:

At its core is the $\text{\sTeX}$ package for $\text{\LaTeX}$, that allows for semantically marking up document fragments; in particular concepts, formulae and mathematical statements (such as definitions, theorems and proofs). Running `pdflatex` over $\text{\sTeX}$ -annotated documents formats them into normal-looking PDF.

The “*package*”-symbol can be found in the $\text{\LaTeX}$ module too, and searching for the keywords “formula” and “mathematics” will yield the symbols “*well-formed formula*” and “*mathematics*”, but they are not *text symbols* and “*mathematics*” and “*package*” do not even have a *semantic macro* – and the one for “*well-formed formula*” would not work outside of math mode.

Text symbols are special in that way – they are intended for *symbols* that have a specific formatting associated (such as $\text{\LaTeX}$, OMDoc, or HTML, which we prefer to typeset as sans serif). For those settings, it makes sense to associate that formatting with a *semantic macro* that does the typesetting for us.

Symbols without a text macro can be referenced with the `\symname` macro: `\symname{package}` prints the *name* of the “*package*”-symbol and annotates it accordingly, without any special formatting – in particular it is compatible with being in `\emph`, `\textbf` and similar *macros*. That takes care of *one* of the missing annotations.

More generally, the `\symref` macro can be used to annotate arbitrary text with a *symbol*: `\symref{mathematics}{mathematical}` associates the text `mathematical` with the *symbol* “*mathematics*”; thus, we get “*mathematical*” and similarly “*formulae*”.

In general, any `macro` that expects a `symbol` name can be given either

1. the *name* of the `symbol`,
2. the name of its `semantic macro`,
3. or any suffix of its `MMT-URI` containing at least the `module` name.

STEX

The second option is often short – and therefore convenient to write; for example, to achieve “`formulae`”, we can also write `\symref{wff}{formulae}`, since `\wff` is the `semantic macro` for “`well-formed formula`”.

The third option allows for distinguishing between multiple `symbols` with the same name – the `IDE` can help in the latter case, by underlining ambiguous `symbol` references in yellow, and offering the `Quick Fix` functionality to let you select and autocomplete the specific `symbol` you want to reference.

Since `\symname` and `\symref` are a lot to type for something that should ideally be used as often as possible, the `macros` `\sn` and `\sr` exist as well and behave exactly the same way. We also provide some convenience abbreviations for `\sn`; namely `\Sn` (capitalizes the first letter of the `symbol` name), `\sns` (adds an “s” at the end, for the most common pluralization of a name), and `\Sns` (both).

Using all of the above, our annotated fragment now looks like this:

Example 4

Input:

```
File tutorial/intro/intro2stex.en.tex
5 \usemodule[sTeX/ComputerScience/Software]{mod/systems/tex?sTeX}
6 \usemodule[sTeX/Logic/General]{mod/syntax?Formula}
7 \usemodule[sTeX/MathBase/General]{mod?Mathematics}
8 \usemodule[sTeX/ComputerScience/Software]{mod/formats?PDF}
9
10 At its core is the \stex \sn{package} for \latex, that allows for
11 semantically marking up document fragments; in particular
12 concepts, \sr{wff}{formulae} and \sr{mathematics}{mathematical}
13 statements (such as definitions, theorems and proofs). Running
14 \texttt{pdflatex} over \stex-annotated documents formats them
15 into normal-looking \PDF.
```

Output:

At its core is the `sTeX package` for `LATEX`, that allows for semantically marking up document fragments; in particular concepts, `formulae` and `mathematical` statements (such as definitions, theorems and proofs). Running `pdflatex` over `sTeX`-annotated documents formats them into normal-looking `PDF`.

There’s only one problem: *the document does not compile*, with an error `Undefined control sequence`. The reason being that *some* `macro` in the `module` `Formula` uses the `\text` `macro`. We can fix that by using the `amsfonts` `package` of course, but this points to a more general problem; namely that `modules` can make use of various `LATEX` `packages` for typesetting `symbols`.

Good practice suggests putting those packages into a *prelude* per `math archive`, which we can then import from anywhere, using the `\libinput` `macro`. For more on that, see

section 5.3.

For now, suffice it to say that we can import all [packages](#) required for the [module Formula](#) from the [math archive sTeX/Logic/General](#) by adding the line

```
\libinput[sTeX/Logic/General]{preamble}
```

before the `\begin{document}`.

2.3 Modules and Simple Symbol Declarations

Consider again the first two paragraphs of section 1.1:

[sTeX](#) is a system for generating human-oriented documents in either [PDF](#) or [HTML](#) format, augmented with computer-actionable semantic information (conceptually) based on the [OMDoc](#) format and ontology.

At its core is the [sTeX](#) package for [L^AT_EX](#), that allows for semantically marking up document fragments; in particular concepts, [formulae](#) and [mathematical](#) statements (such as definitions, theorems and proofs). Running `pdflatex` over [sTeX](#)-annotated documents formats them into normal-looking [PDF](#).

Firstly, note that the first paragraph would be perfectly suitable to serve as a pop-up definition on hover for the [sTeX](#) symbol. Secondly, what if all the [symbols](#) used in the above *didn't* already exist?

In this section, we will describe how to make your own [symbols](#) and collect them as reusable fragments in [modules](#) and [math archives](#) from scratch.

We start by creating a new [math archive](#). In the [IDE](#), switch to the [sTeX](#)-tab on the left and click the “New sTeX Archive” button (Figure 7). You will then be asked for

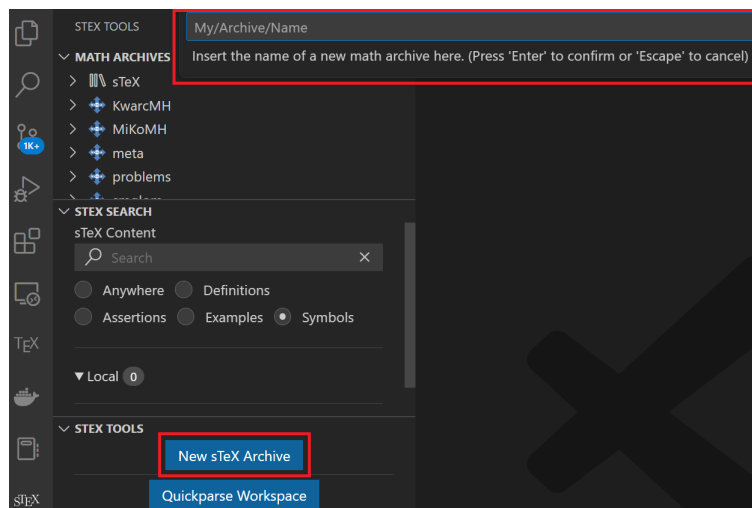


Figure 7: New [Math Archive](#) in the [IDE](#)

the name of the [archive](#), and a url-base, where the content is supposedly going to end up

online. You can safely keep the defaults for the latter. In the following, we assume that your archive is named `my/archive`.

The IDE will then create the following files and directories in your MathHub directory:

```
- my
  - archive
    - lib
      - preamble.tex
    - META-INF
    - MANIFEST.MF
    - source
    - helloworld.tex
```

...and open the file `helloworld.tex` with the content

```
1 \documentclass{stex}
2 \libinput{preamble}
3 \begin{document}
4 % A first sTeX document
5 \end{document}
```

You can now reference any newly created content in you new `archive` using for example `\usemodule[my/archive]{...}`.

Let's start with the "`LaTeX`" symbol. Rename the file `helloworld.tex` to something more meaningful, for example `latex.en.tex` – the `.en` will be picked up on by `sTeX` to signify that the fragment will be in *english* (see subsection 7.1.1).

What we want to achieve in this file is the following:

TeX is a document typesetting software developed by Donald Knuth, with a focus on mathematical formulae. It is based on a powerful and extensible **macro** expansion engine.

LaTeX is a (nowadays) default collection of **TeX macros** developed by Leslie Lamport. Among other things, **LaTeX** introduces **environments**, a distinction between preamble and document content, **packages** to bundle and distribute **macro** definitions, and **document classes**: special **packages** that govern the global layout of a document.

In particular, in the **HTML** the two paragraphs above should be shown when hovering over the **symbols** they define (as indicated by the magenta definiendum highlighting). So we need **symbols** and **semantic macros**, for: **TeX**, **macro**, **LaTeX**, **environment**, **package** and **document class**.

Symbol declarations are only allowed within **modules**:

sTeX A **module** is a *named* block that bundles **symbol** declarations for subsequent reuse.
A **module** is introduced with the **smodule**-environment.

Let's name our **module** `LaTeX`. We then wrap the contents of our document in a **smodule** environment:

```
\begin{document}
  \begin{smodule}{LaTeX}
    ...
  \end{smodule}
\end{document}
```

 Note, that the IDE immediately picks up on this and displays the full FTML URI of our new module over the `\begin{smodule}{LaTeX}` (Figure 8) –



```

my > archive > source > %X latex.tex > {} https://mathhub.info?a=my/archive&p=latex&m=L
1 \documentclass{stex}
2 \begin{document}
3   https://mathhub.info?a=my/archive&p=latex&m=LaTeX
4 \begin{smodule}{LaTeX}
5   ...
6 \end{smodule}
7 \end{document}

```

Figure 8: VS Code URI on Hover

From this, we can glimpse that the namespace of the module is `http://mathhub.info?a=my/archive&p=latex&m=LaTeX`. This implies, that to use the module somewhere else, we will have to type `\usemodule[my/archive]{latex?LaTeX}` – the `latex`-part pointing to the file and `LaTeX` referring to the actual module.

If we rename the file to `LaTeX.en.tex`, we notice that the namespace changes to `http://mathhub.info?a=my/archive&m=LaTeX`, allowing us to now use it with `\usemodule[my/archive]{LaTeX}` directly. That’s because the module name `LaTeX` and the file name `LaTeX` match now (see section 7.5, Figure 9).



```

my > archive > source > %X LaTeX.en.tex > ...
1 \documentclass{stex}
2 \begin{document}
3   https://mathhub.info?a=my/archive&m=LaTeX
4 \begin{smodule}{LaTeX}
5   ...
6 \end{smodule}
7 \end{document}

```

Figure 9: VS Code URI on Hover



Note that “`LaTeX`” and “`latex`” only differ in capitalization – if your file system is case-insensitive (as e.g. MacOS’s was until quite recently), this distinction gets murky, but remains very important especially if you want to share your [math archive](#) with others!
It is therefore *highly recommended* to treat file names as case-sensitive either way.

Within the module, we can now declare new symbols using the `\symdecl`-macro. We start with those that are not text symbols:

```

\symdecl*{macro}
\symdecl*{environment}
\symdecl*{package}
\symdecl*{document class}

```

The `*` after the `\symdecl` indicates, that we do not want a semantic macro for the symbol – otherwise, it would generate one with the same name as the symbol itself and “pollute the macro space”, so to speak.

The symbols `\TeX` and `\LaTeX`, however, have a definite way of being typeset associated with them, which can be produced using the standard `\TeX` and `\LaTeX` macros. So let's make them `text symbols`, using the `\textsymdecl` macro:

```
\textsymdecl{tex}{\TeX}
\textsymdecl{latex}{\LaTeX}
```

The first argument being the name of the generated macro (i.e. `\tex` and `\latex`) and the second one specifying the output to produce.



As we cannot rely on the underlying `\TeX` engine treating UniCode charaters natively, symbol names can only consist of printable ASCII characters.

2.4 Documenting Symbols

We can now use the two new macros, `\symname/\sn`, `\symref/\sr` etc. to mark up the above two paragraphs. However, the system does not yet know what to show a reader when hovering over the `symbol` in the `HTML`. We can fix that by using the `sdefinition` or `sparagraph` environments.

Ignoring the former for now, which is more useful for mathematical concepts, we can use the following to mark up the first paragraph:

```
\begin{sparagraph}[style=symdoc,for={tex,macro}]
\tex is a document typesetting
software developed by Donald Knuth, with a focus on
mathematical formulae. It is based on a powerful
and extensible \sn{macro} expansion engine.
\end{sparagraph}
```

In general, the `sparagraph environment` can be used to mark up arbitrary paragraphs semantically, but the `style=symdoc` option tells `\TeX` to use this paragraph as a documentation for the symbols provided in the `for=` option.

We just used the `semantic macro \stex` and the `\sn macro` to mark up the fragment – but we can do better. Both concepts are being *introduced* in the above paragraph, and we can let `\TeX` know that that is the case:

Within an `sparagraph environment` with `style=symdoc` (or an `sdefinition environment`), we can mark up *definienda*, meaning the terms *being defined*, explicitly. Analogously to `\symname` and `\symref`, we have the macros `\definame` and `\definiendum` for that purpose.

Note that the `\tex macro` induced by the `text symbol` above already marks up the “`\TeX`” it produces, so wrapping it in another `\definiendum` would be redundant. However, every `text symbol` also generates a *second macro* with the suffix `name` that generates a non-marked-up version of the same presentation. In other words, we get the macro `\texname` for free, that produces “`\TeX`” (of course, we could just as well use the `\TeX macro`, but that one you probably know already).

Furthermore, every `\definiendum` or `\definame` automatically adds the `symbol` being referenced to the internal `for=-list` of the `sparagraph environment`, obviating the need to list it explicitly.

As such, we can produce a better markup like this:

```

\begin{sparagraph}[style=symdoc]
  \definiendum{tex}{\texname} is a document typesetting
  software developed by Donald Knuth, with a focus on
  mathematical formulae. It is based on a powerful
  and extensible \definame{macro} expansion engine.
\end{sparagraph}

```

Exercise

In your archive my/archive, create additional files that produce the following outputs:

Mathematics.en.tex

To do **mathematics** is to be, at once, touched by fire and bound by reason. This is no contradiction. Logic forms a narrow channel through which intuition flows with vastly augmented force.

– Jordan Ellenberg

PDF.en.tex

Portable Document Format (PDF) is a document format that mixes text and graphics with a variety of content.

HTML.en.tex

The **HyperText Markup Language (HTML)** is a representation format for web-pages.

OMDoc.en.tex

OMDoc is a document format for representing **mathematical** documents with their flexiformal semantics.

such that the following file compiles and shows the above snippets on hover:

sTeX.en.tex

```

1 \documentclass{stex}
2 \libinput{preamble}
3 \begin{document}
4 \begin{smodule}{sTeX}
5   \usemodule{OMDoc}
6   \usemodule{PDF}
7   \usemodule{HTML}
8   \textsymdecl{stex}{\sTeX}
9   \begin{sparagraph}[style=symdoc]
10    \definiendum{stex}{\stexname} is a system for generating
11    documents in either \PDF or \HTML format, augmented with
12    computer-actionable semantic information (conceptually)
13    based on the \OMDoc format and ontology.
14  \end{sparagraph}
15 \end{smodule}
16 \end{document}

```

sTeX is a system for generating documents in either **PDF** or **HTML** format, augmented with computer-actionable semantic information (conceptually) based on the **OMDoc** format and ontology.

The preamble of every file should only be

```
\documentclass{stex}
\libinput{preamble}
```

and the macros `\OMDoc`, `\PDF`, `\HTML` should produce `\textsc{OMDoc}`, `\textsf{PDF}` and `\textsf{HTML}`, respectively (but with semantic annotations of course).

Lösung: Can be found in `[sTeX/Documentation]source/tutorial/solution`

2.5 Sectioning and Reusing Document Fragments

We know now how to import and reuse the [symbols](#) of some [module](#) (using `\usemodule`). What about the actual document *content*?

Assume we want to write a new article that includes all of the fragments in `my/archive` we made so far, in a file `all.en.tex` in the same [math archive](#):

```
1 \documentclass{article}
2 \usepackage{stex}
3 \libinput{preamble}
4 \begin{document}
5   \author{Me}
6   \title{The \texttt{my/archive} Archive}
7   \maketitle
8   \tableofcontents
9   ...
10 \end{document}
```

In there, we want sections as follows:

```
- 1 Preliminaries
  (Mathematics)
- 1.1 Document Formats
  (PDF)
  (HTML)
  (OMDoc)
- 2 \TeX and Friends
  (LaTeX)
  (sTeX)
```

We could of course do the following:

```
\section{Preliminaries}
\input{Mathematics.en}
\subsection{Document Formats}
\input{PDF.en}
\input{HTML.en}
\input{OMDoc.en}
\section{\TeX and Friends}
\input{LaTeX.en}
\input{sTeX.en}
```

...but this approach has two drawbacks:

Firstly, we need to manually keep track of the section levels, by explicitly writing `\section`, `\subsection` etc. This is fine as long as we are just interested in this particular article. But what if we want to *reuse* the article’s content in another document at some point? The section levels might be entirely different then – e.g. we might want the “Preliminaries” section to be a subsection instead.

Secondly, the `\input` macro considers the file name/path provided to be either *absolute* or relative to the *current tex file being compiled* – which means that the `\input{Mathematics.en}` only works for files in the same directory as `Mathematics.en.tex`.

In short: using `\section`, `\chapter` etc. explicitly, and `\input` to reuse fragments, breaks reusability.

Instead of using `\section` and `\subsection`, `TeX` therefore provides the `sfragment` environment.

`\begin{sfragment}{Foo}...\end{sfragment}` inserts a sectioning header depending on the current section level and availability. These are: `\part`, `\chapter`, `\section`, `\subsection`, `\subsubsection`, `\paragraph` and `\subparagraph`. This allows us to do the following instead:

```
\begin{sfragment}{Preliminaries}
  \input{Mathematics.en}
  \begin{sfragment}{Document Formats}
    \input{PDF.en}
    \input{HTML.en}
    \input{OMDoc.en}
  \end{sfragment}
\end{sfragment}
\begin{sfragment}{\TeX and Friends}
  \input{LaTeX.en}
  \input{sTeX.en}
\end{sfragment}
```

The only problem remaining now is that if we do this, `TeX` will insert a `\part` for the first `sfragment`. If we want the “top-level” sectioning level to be `\section` instead, we can insert a `\setsectionlevel{section}` in the preamble.

As a more reuse-friendly replacement of `\input`, `TeX` provides the `\inputref` macro. Using that has two advantages: Firstly, its argument is relative to some (optionally provided, or the current) `math archive` and is thus independent of the specific location of the file relative to the currently being compiled `.tex`-file. Secondly, when converting to `HTML`, it will *not* “copy” the referenced file’s content in its entirety (as `\input` would), but instead dynamically insert the already existent (if so) `HTML` of the referenced file, avoiding content duplication and having to process the file all over again.

In general `\inputref[some/archive]{file/path}` inputs the file `file/path.tex` in the `archive` `some/archive`. As the `\input`-ed files in the example above are in the same `archive` anyway, we can simply substitute the `\inputs` by `\inputrefs` and call it a day.

Finally, we can make two more minor changes:

1. The *title* of our document is only supposed to be there, if we compile the document directly – if we were to `\inputref` our file into a “driver file” `all.en.tex`, the title and the table of contents should be omitted.

We can achieve this using the `\ifinputref` conditional: by wrapping the header in an `\ifinputref \else...\fi`, it will only be processed if the file is *not* being loaded using `\inputref`. `\ifinputref` is a “classic” `TeX` conditional and is treated as such in both `PDF` and `HTML` compilation. A smarter `macro` to use is `\IfInputref`, which takes two arguments for the *true* and *false* cases, respectively. Additionally, when compiling to `HTML`, *both* arguments to `\IfInputref` will be processed, and the back-end will decide which of the two to present when serving a document.

2. The table of contents should also be omitted in `HTML` mode. To achieve that, we can use the `\ifstexhtml` conditional, which is *true* if the document is being compiled to `HTML`, and *false* if compiled to `PDF`.



Note, that since *both* arguments of `\IfInputref` are processed, they should *not* open `TeX` groups or `environments`!

In summary, we can modify our document to do the following:

```
\IfInputref{}{
  \author{Me}
  \title{The \texttt{my/archive} Archive}
  \maketitle
  \ifstexhtml \else \tableofcontents \fi
}
```

The final `all.en.tex` can be found in `[sTeX/Documentation]tutorial/solution/all.en.tex`.

2.6 Building and Exporting `HTML`

So far we know how to write `sTeX` documents, (we assume) how to build `PDF` files from them (via `pdflatex` of course), and on saving documents the `IDE` will preview the generated `HTML`. But if we do that with our new `all.en.tex`, we get presented with Figure 10 Where did all of our fragments go?



Figure 10: Missing Fragments in the `HTML` Preview

Well, they don’t exist yet as `HTML`. The `HTML` Preview window in the `IDE` is really just that: A *preview*. But when using `\inputref`, it has to find the `HTML` of the `\inputrefed` fragment *somewhere*. Meaning: we have to compile all of the fragments we used to `HTML` first. Individually, we can compile the currently open file and all its dependencies in `VS Code` using the button in Figure 11.

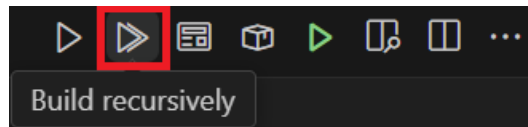


Figure 11: The Build PDF/XHTML/OMDoc Button

This will queue all dependencies of the file in a new build queue and open the [FLMf](#) dashboard for us (see Figure 12). Clicking on the run button in the dashboard will then

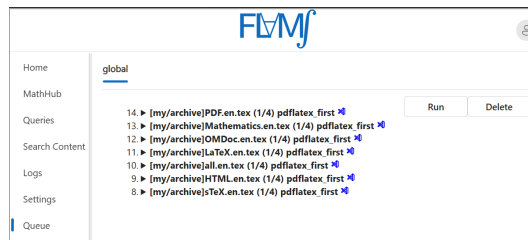


Figure 12: The FLAMS build queue

for every queued file:

1. Run `pdflatex` twice,
2. Convert the file to [HTML](#),
3. Extract all the semantics, and
4. Construct a search index.

Once that's done, saving `all.en.tex` again yields the correct [HTML](#) in the preview window.

At this point, [FLMf](#) knows about the [HTML](#) and the semantics, and we can look at the [HTML](#) in [VS Code](#) or the [FLMf](#) dashboard – of course, features like the fancy pop-up windows require a semantically informed back-end infrastructure, in the form of the [FLMf](#) system. However, [FLMf](#) can dump a standalone and self-contained [HTML](#) version for you. Let's do that now:

With our `all.en.tex` file open and everything built as above, click the **Export Standalone HTML** button in the [IDE](#) (see Figure 13).

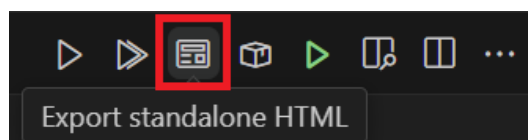


Figure 13: Exporting [HTML](#) in the [IDE](#)

In the dialog box that opens now, select an **empty** directory and [FVMf](#) will dump a standalone version of our `all.en.tex` document there. You will still not be able to open it in the browser directly without issues, because most browsers forbid javascript modules on the `file://` protocol, but opening the file via `http` will yield the desired result, and you can now upload the directory's content to wherever you might want to use it.

If you want to test this, a quick and easy way to do so is to use [VS Code](#): You can install the **Live Server** extension, open the directory and click the **Go Live** button on the lower right of the window, which will start a small web-server in the selected directory and open its `index.html` in the browser for you.

Chapter 3

Mathematical Concepts

So far, we have seen how to declare and reference [symbols](#) generate [semantic macros](#) for [text symbols](#), collect them in [modules](#) and document them properly.

But where [sTeX](#) really shines is when it comes to mathematics and related subject areas: [semantic macros](#) are significantly more useful when used for generating symbolic [notations](#) in math mode, and by associating [symbols](#) with (flexi-)formal semantics, [sTeX](#) can even *check* that your content is (to some degree) formally correct, or at least well-formed.

Alos [sTeX](#) provides specialized functionality for mathematical [statements](#): the text fragments marked as Definition, Theorem, Proof that are iconic to mathematical documents.

The example snippets in this chapter can be found in the [math archive sTeX/MathTutorial](#). If you downloaded the [sTeX/Documentation archive](#) in the [sTeX IDE](#), you already have that [archive](#). If not, you can download it from within the [IDE](#), as described in Part 1 (The Basics) in the [sTeX](#) tutorial.

3.1 Simple [Symbol](#) Declarations

We will start with [symbols](#) and [semantic macros](#) for mathematical concepts and objects and their contribution to mathematical formulae.

3.1.1 [Semantic Macros](#) and [Notations](#)

Let us start with a very fundamental concept; namely [equality](#). As you should by now know, declaring a new [symbol](#) requires a [module](#), so let's open a new one and use `\symdecl`:

```
\begin{smodule}{Equality}
  \symdecl{equal}
\end{smodule}
```

As mentioned in Section 0.1 ([Modules](#) and Simple [Symbol](#) Declarations), the starred variant `\symdecl*` does not create a [semantic macro](#), so presumably, the variant without a `*` *does*. And indeed, we now have a macro `\equal`, which however will produce errors if we try to use it. That's because we haven't told [sTeX](#) what to do with it yet.

STEX

A **semantic macro** is a $\text{\LaTeX}$ -macro that allows for referencing a **symbol** itself, or – in the case of e.g. a function – the *application* of a **symbol** to (one or multiple) *arguments*; primarily by invoking a **symbol**’s **notation** in *math mode*.

The command `\symdecl{macroname}` declares a new **symbol** with name `macroname` and a **semantic macro** `\macroname`. In the case where we want the name and the **semantic macro** to be distinct, the command `\symdecl{macroname}[name=some name]` declares the name of the **symbol** to be `some name` instead.

The starred variant `\symdecl*{name}` declares the concept with the given name, but does not generate a **semantic macro**.

So let’s provide equality with a **notation**. As a first step, we should let $\text{\TeX}$ know that “`equal`” takes two arguments. We might also want to shorten the **semantic macro** to e.g. `\eq`, without changing the name. Hence:

```
\symdecl{eq}[name=equal,args=2]
```

Next, we add an infix notation with the **notation macro**:

```
\notation{eq}{#1 = #2}
```

That seems like a lot to write, so for the very common case where we want to declare a **symbol** with a **semantic macro** and a **notation** all at once, the `\symdef` macro does all three by combining the optional and mandatory argument of `\symdecl` and `\notation`:

```
\symdef{eq}[name=equal,args=2]{#1 = #2}
```

and indeed, we can now use the `\eq` macro in *math mode* to invoke our new **notation**: `\eq{a}{b}` now yields $a = b$ – notably without any highlighting (and hover interaction in the **HTML**) though. Since our **semantic macro** takes *arguments*, which should be differently highlighted, we need to let our **notation** know which parts of the **notation** are highlightable components.

We can do so with the `\comp` and `\maincomp` macros:

STEX

The `\comp`-macro marks components to be highlighted in a **notation** for a **symbol** taking (one or more) arguments.

This is necessary because it is (nearly) impossible for $\text{\LaTeX}$ to figure out, which parts of a **notation** to highlight and which not on its own – in particular, the highlighting should stop for the *arguments* of a **semantic macro**.

Additionally, the `\maincomp` macro can be used to mark (at most) one **notation** component to represent the *primary* component of the **notation**.

Notations that do not take arguments, as well as **operator notations**, are automatically wrapped in `\maincomp`.

In our case, this applies only to the “`=`”, symbol, so:

```
\symdef{eq}[name=equal,args=2]{#1 \mathrel{\maincomp{=}} #2}
```



You may be wondering about the role of the `\mathrel` macro in the example above: $\text{\TeX}$ determines spacing/kerning in *math mode* by assigning a *class* to every

character. Both individual characters and whole subexpressions can be assigned one of these classes using dedicated macros. These are:



class	T_EX macro	examples
ordinary (default class)	<code>\mathord</code>	$\alpha \ i \ \Diamond$
large operator	<code>\mathop</code>	$\sum \prod \int$
opening	<code>\mathopen</code>	$([\langle$
closing	<code>\mathclose</code>	$)] \rangle$
binary relation	<code>\mathrel</code>	$\leq > =$
binary operator	<code>\mathbin</code>	$+ \cdot \circ$
punctuation	<code>\mathpunct</code>	$, ;$

T_EX “forgets” the class of an expression if it is wrapped in a `\comp` macro. It is therefore a good idea to wrap any occurrence of a `\comp` in the corresponding T_EX macro for the desired class (e.g. `\mathrel{\comp{\leq}}`).

Having done so, we can now type `\eq{a}{b}` to get $a = b$. Thanks to using `\maincomp`, we now also have an **operator notation**, which we can invoke using `\eq!`, yielding $=$.

What if we want to add more **notations**? Say we want to be able to invoke **equality** to get the variant notation $a \equiv b$ (without changing the intended meaning). If we want to be able to choose one of several **notations**, we should give the **notation** an *identifier*.

Let’s again modify our earlier **notation** by adding the identifier `eq` to the optional arguments of `\symdef`, like so:

```
\symdef{eq}[name=equal,args=2,eq]{#1 \mathrel{\maincomp{=}} #2}
```

We can now invoke the specific **notation** provided here by writing `\eq[eq]{a}{b}` to the same effect. But we can also add more **notations** using the `\notation` macro:

```
\notation{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
```

which we can now invoke with `\eq[equiv]{a}{b}`, yielding $a \equiv b$.

By default, the *first* **notation** provided for a given **symbol** is considered the *default notation*, which is invoked if the **semantic macro** is used without an optional argument – hence, `\eq{a}{b}` still yields $a = b$.

If we use the starred variant of the `\notation` macro, the **notation** is set as the new default. Hence, had we done

```
\notation*{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
```

then `\eq{a}{b}` would now yield $a \equiv b$.

Any already existing notation can be set as default using the `\setnotation` macro; e.g. instead of using `\notation*`, we could also do

```
\notation{eq}[equiv]{#1 \mathrel{\maincomp{\equiv}} #2}
\setnotation{eq}{equiv}
```

Exercise

Implement the **symbol** “equal” as above in a new **module** “Equality” and add a documentation such that hovering over the **symbol** in the **HTML** yields the following snippet:

Two objects a, b are considered **equal** (written $a = b$ or $a \equiv b$), if there is no property that distinguishes them.

Lösung: Can be found in [sTeX/MathTutorial]/mod/Equality1.en.tex

3.1.2 Types and Variables

sTeX Every **symbol** or **variable** can be assigned a **type**, signifying what “kind of object” the **symbol** represents, or what (primary) set it is contained in.

Types are particularly useful for *variables*:

sTeX A **variable** represents a *generic* or *unspecified* object.
Variables can be declared using the `\vardef`-macro, whose syntax is analogous to `\symdef`.
Note that **variables** are local to the current T_EX-group (e.g. environment).

Let’s leave our equality-module aside for now and turn our attention to something simpler: **natural numbers**. Consider the following module:

Example 5

Input:

```
\begin{smodule}{Nat}
\symdef{Nat}[name=natural numbers]{\mathbb N}
\begin{sparagraph}[style=symdoc]
  The \definame{Nat} $\defnotation{\Nat}$ are the numbers
  $0,1,2,\dots$
\end{sparagraph}
\symdef{plus}[name=addition,args=2]{#1 \mathbin{\maincomp{+}} #2}
\begin{sparagraph}[style=symdoc]
  \Definame{addition} $\defnotation{\plus{a}{b}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
\end{smodule}
```

Output:

The **natural numbers** $\mathbb{N}$ are the numbers 0,1,2,...
Addition $a+b$ refers to the process of adding two **natural numbers**.

(like `\definame` and `\definiendum`, the `\defnotation` macro is only allowed in documenting environments like `sparagraph[style=symdoc]` or `sdefinition`, and highlights the **notation** components marked with `\comp` or `\maincomp` the same way as `\definame` and `\definiendum` do.)

Note, that as the `\Nat` semantic macro does not take any arguments, we do not need to wrap the notation in a `\comp` or `\maincomp`.

The above fragment uses two variables a and b . In fact, `FLMj` will consider them variables even though they are not marked up as such – but since they are not marked up, we are missing out on useful functionality.

Let’s change that by adding two variable definitions¹:

Example 6

Input:

```
\begin{sparagraph}[style=symdoc]
\vardef{va}[name=a]{a}\vardef{vb}[name=b]{b}
\Definame{addition} $\defnotation{\plus{va}{vb}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
```

Output:

Addition $a+b$ refers to the process of adding two natural numbers.

Okay, so now a and b are gray, but besides that, we haven’t achieved much yet. Let’s change that by giving the variables the type $\mathbb{N}$:

Example 7

Input:

```
\begin{sparagraph}[style=symdoc]
\vardef{va}[name=a,type=\Nat]{a}\vardef{vb}[name=b,type=\Nat]{b}
\Definame{addition} $\defnotation{\plus{va}{vb}}$
  refers to the process of adding two \sn{Nat}.
\end{sparagraph}
```

Output:

Addition $a+b$ refers to the process of adding two natural numbers.

Now if we hover over the a and b (in the HTML), it will show us that their type is $\mathbb{N}$!

We can of course also assign types to symbols. In the IDE, find the symbol “function space” with semantic macro `\funspace` (in `[sTeX/MathBase/Functions]{mod?Function}`). The OMDoc preview window shows you how to use this symbol (Figure 14). This tells us that if we write `\funspace{a_1, \dots, a_n}{b}` (depending on which notation we use), we will get $a_1 \times \dots \times a_n \rightarrow b$.

We want addition to have type $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, hence we do:

```
\symdef{plus}[name=addition,args=2,
  type=\funspace{\Nat,\Nat}{\Nat}
]{#1 \mathbin{\maincomp{+}} #2}
```

¹Technically, this is called a *variable reservation*, for those in the know.

Symbol function space (`\funspace{a_1^1, \dots, a_1^n}{i_2}`)
of type `bind( (A,B),SET)`

Notations: $a_1^1 \rightarrow \dots \rightarrow a_1^{i_1} \rightarrow i_2$ $a_1^1 \times \dots \times a_1^{i_1} \rightarrow i_2$ $a_1^1 \rightarrow \dots \rightarrow a_1^{i_1} \rightarrow i_2$ $a_1^1 \times \dots \times a_1^{i_1} \rightarrow i_2$

▼ Associated Paragraphs

Figure 14: Syntax Preview



So far (and when using the `use` button in the IDE), we have been using the `\usemodule` macro to import content. `\usemodule` is allowed anywhere and imports the referenced `module` content local to the current `TeX` group. Now that we use imported `symbols` in `types` (and since we are *in* a `module`), we need to make sure that the imported `modules` are also (transitively) *exported*, since our new `symbols` now *depend* on the imported `module`. For that we use the `\importmodule` macro within the `module`; i.e. the file should now look something like this:

```
\begin{smodule}{Nat}
  \importmodule[sTeX/MathBase/Functions]{mod?Function}
  ...
```

Note that the `HTML` is aware of this now (after you save): *Clicking* on any occurrence of `addition` now yields Figure 15.

addition

Select Definition or Example

Addition $a+b$ refers to the process of adding two `natural numbers`.

Force Notation

▼ Formal Details

Symbol addition (`\plus{a_1^1, \dots, a_1^n}`)
of type `map{N to N}`

Notations: $a_1^1 + \dots + a_1^{i_1}$

Figure 15: On-Click Popup in the `HTML`

By virtue of using `[sTeX/MathBase/Functions]{mod?Function}`, we also imported `[sTeX/MathBase/Sets]{mod?Set}`, which gives us the “*collection*” `symbol`. Let’s use this as a `type` for the `natural numbers`:

```
\symdef{Nat}[name=natural numbers,type=\collection]{\mathbb N}
```

3.1.3 Flexary Macros and Argument Modes

Here is one thing you might wonder: Writing `\plus{a}{b}` is one thing, but what if we want to produce $a + b + c + d + e$? Do we really need to write `\plus{a}{\plus{b}{\plus{c}{\dots}}}`?

Of course not. We can declare the `symbol` such that the `semantic macro` `\plus` expects a (comma-separated) *sequence* of arguments instead of two “normal” arguments.

The optional `args`-argument of `\symdecl` expects a string of characters indicating the semantic macro's **argument modes**. There are four such **modes**:

- i a **simple argument**,
- a a – (left or right) associative – **sequence argument**, represented as a single $\text{\TeX}$ -argument $\{a,b,\dots\}$,
- B A **binding argument** that expects a variable that is bound by the **symbol** in its application, and
- B A **binding sequence argument** of arbitrarily many bound variables by the **symbol** $\{x,y,z,\dots\}$.

If `args` is given as a number n instead, the semantic macro takes n arguments of mode `i`.

Example 8

- For `\plus{a,b,c}` yielding $a + b + c$, we do `\symdecl{plus}[args=a]`,
- for `\inset{a,b,c}{A}` yielding $a, b, c \in A$, we do `\symdecl{inset}[args=ai]`,
- in `\add{i}{1}{n}{f(i)}` yielding $\sum_{i=1}^n f(i)$, the variable i is **bound** in the expression, we hence do `\symdecl{add}[args=biii]`,
- in `\forall{x,y,z}{P(x,y,z)}` yielding $\forall x, y, z. P(x, y, z)$, the variables x, y, z are all **bound** by the $\forall$, we hence do `\symdecl{forall}[args=Bii]`.

So when we wrote `\symdecl{plus}[args=2]`, this was actually shorthand for `\symdecl{plus}[args=ii]`.

Let's revise our previous declaration and the syntax of the `\plus` macro:

```
\symdef{plus}[name=addition,args=a,
  type=\funspace{\Nat,\Nat}{\Nat}
]{#1 \mathbin{\maincomp{+}} #2}
\begin{sparagraph}[style=symdoc]
  \vardef{va}[name=a]{a}\vardef{vb}[name=b]{b}
  \Definame{addition} $\defnotation{\plus{\va,\vb}}$
  refers to the process of adding two \sn{\Nat}.
\end{sparagraph}
```

Now we get new errors, that are easy to explain: Our **notation** `{#1 \mathbin{\maincomp{+}} #2}` refers to *two* arguments, but our semantic macro only takes *one* (albeit a **sequence argument**). We now need to let $\text{\TeX}$ know what to do with the **sequence argument** in our **notation**. Using the `\argsep` macro, we can tell $\text{\TeX}$ to insert the *separator* “+” between the individual elements of the **argument sequence** #1:

```
\symdef{plus}[name=addition,args=a,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{+}}}}
```

TODO² TODO³

Now we can finally write `$(\plus{a,b,c,d,e})$` and get $a + b + c + d + e$ – hooray!

²TODO: argmap

³TODO: argarraymap

Exercise

Analogously to the above, implement a symbol “multiplication” with semantic macro `\mult`, that takes a single sequence argument and has a default notation such that `\mult{a,b,c}` produces $a \cdot b \cdot c$.

Lösung: Can be found in [sTeX/MathTutorial]mod/Nat.en.tex

3.1.4 Precedences

If you have done the previous exercise, you now have semantic macros `\plus` and `\mult` at your disposal. We can of course nest them to produce e.g. $a + b \cdot c$ (with `$(\plus{a,\mult{b,c}})$`). If we do `$(\mult{a,\plus{b,c}})$` however, we get $a \cdot b + c$. Annoying – we now have to insert parentheses: `$(\mult{a,(\plus{b,c}))}$`... or do we?

We do *not*. Instead, we can assign *precedences* to *notations* to have sTeX insert parentheses automatically.

sTeX

`\notation` (and hence `\symdef`) take an optional argument `prec=<opprec>;<argprec1>x...<argprec n>` consisting of an **operator precedence** `<opprec>` and for each argument `k` an **argument precedence** `<argprec k>`.

All *precedences* are integers, e.g. 10 or -500. It is good practice to use *precedences* that leave enough room to smuggle values inbetween, so that we can fine-tune them later as more symbols may intervene.

The precise numbers used for *precedences* are arbitrary – what matters is which *precedence* is higher than which other *precedence* when used together.

By default, all *precedences* are 0, unless the symbol takes no arguments, in which case the **operator precedence** is `\neginfp` (negative infinity).

If we only provide a single number in `prec=`, this is taken as both the **operator precedence** and all **argument precedences**.

The *lower* a *precedence*, the *stronger* a *notation* binds its arguments. In our particular case, we want *multiplication* to bind stronger than *addition*, so we can (arbitrarily) assign them *precedences* e.g. 10 and 20:

```
\symdef{plus}[name=addition,args=a,assoc=bin,prec=20,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{+}}}}
\symdef{mult}[name=multiplication,args=a,assoc=bin,prec=10,
  type=\funspace{\Nat,\Nat}{\Nat}
]{\argsep{#1}{\mathbin{\maincomp{\cdot}}}}
```

And now if we type `$(\mult{a,\plus{b,c}})$`, sTeX will automatically insert parentheses and yield $a \cdot (b + c)$ – and conversely, if we do `$(\plus{a,\mult{b,c}})$`, sTeX will *not* insert parentheses and yield $a + b \cdot c$.

3.1.5 Finishing Equality

You might wonder if – as with `addition` – we can make “`equal`” take a `sequence` argument as well. Naturally, we can:

```
1 \symdef{eq}[name=equal,args=a,eq,
2   type=\funspace{\vA,\vA}{\prop}
3 ]{\argsep{#1}{\mathrel{\maincomp=}}}
4 \notation{eq}[equiv]{\argsep{#1}{\mathrel{\maincomp\equiv}}}
```

3.1.6 Variable Sequences

There is a special kind of `variable` in `STeX` for when we want to use *sequences* of `variables`.

We can use the `\varseq` macro to declare a new sequence `variable`; in the simplest case that looks something like the following:

```
\varseq{seqn}[name=n,type=\Nat]{1,\ellipses,k}{\maincomp{n}_{#1}}
```

We have just declared a new variable sequence of `type` `N`, that ranges over indices $1, \dots, k$, with `notation` n_i for some specific index i .

If we now do `\seqn{i}`, we get n_i , and if we do `\seqn!`, we get $n_1, \dots, n_k$.

We can also do multi-dimensional sequences, e.g.

```
\varseq{seqm}[name=m,type=\Nat,args=2]
{\{1\}{1},\ellipses,{\ell}{k}}
{\maincomp{m}_{#1}^{\#2}}
```

Now `\seqm{i}{j}` produces m_i^j , and `\seqm!` produces $m_1^1, \dots, m_\ell^k$.

Of course, we can manually change the way `\seqn!` is typeset by providing an explicit `operator notation` using `op=`; e.g. if we do

```
\varseq{seqn}[name=n,type=\Nat,op={\{n_i\}_{i=1}^k}]
{1,\ellipses,k}{\maincomp{n}_{#1}}
```

then `\seqn!` produces $(n_i)_{i=1}^k$.

So far so nice, but sequence variables get especially useful in combination with `sequence arguments`: Consider for example the `\plus semantic macro` for `addition`. This expects one `sequence argument`, or alternatively, a *sequence variable*: `\plus{\seqn}` now produces $n_1 + \dots + n_k$, and `\eq{\seqm}` now produces $m_1^1 = \dots = m_\ell^k$.

TODO⁴

3.2 Statements

Now that we have `equality`, `natural numbers`, `addition` and `multiplication` at our disposal, let’s implement some *statements*. Both `addition` and `multiplication` are, for example, *associative* and *commutative*.

We could state these properties directly for the two operations, but we can also first define *associativity* and *commutativity* in general, and then assert them specifically for `addition` and `multiplication`.

⁴TODO: `seqmap`

3.2.1 Definitions

Let's define what it means to be *associative*. This means, of course, declaring a new *symbol*. Note that we don't need a *semantic macro* for *associativity*, since there is no *notation* to attach to it. We will also for now ignore its *type*. Note however, that *associativity* is still a property of (binary) operations, so it still makes sense to have the *symbol* take an *argument*; namely the operation it applies to.

We will also finally provide an actual (more or less) formal *definition* for the *symbol*, so where we used the *sparagraph environment* with `style=symdoc` before, we will now use the *sdefinition environment*, which also gives us `\definame`, `\definiendum`, `\defnotation` and all that.

A first variant of a corresponding *module* could look like this:

Example 9

Input:

```

File [sTeX/MathTutorial]props/Associative1.en.tex
4 \begin{smodule}{Associative}
5   \importmodule{mod?Equality}
6
7   \symdecl*{associative}[args=1]
8   \begin{sdefinition}[for=associative]
9     \vardef{vA}[name=A,type=\collection]{A}
10    \vardef{vop}[name=op,type=\funspace{\vA,\vA}\vA,args=a,assoc=bin]
11      {\argsep{#1}{\mathbin{\maincomp{\circ}}}}
12    %
13    A binary operation  $\fun{\vop!}{\vA,\vA}\vA$  is called
14    \definame{associative}, if
15     $\eq{
16      \vop{(\vop{a,b}),c},
17      \vop{a,(\vop{b,c})}
18    }{}$  for all  $\inset{a,b,c}\vA$ .
19  \end{sdefinition}
20 \end{smodule}

```

Output:

Definition 3.2.1. A binary operation $\circ : A \times A \rightarrow A$ is called **associative**, if $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$.

Note, that the *semantic macros* `\fun` and `\inset` come from `[sTeX/MathBase/Functions]mod?Function` and `[sTeX/MathBase/Sets]mod?Set`, respectively. Also note, that the *variable* declaration for `\vop` makes use of all the fun features we already discussed for *addition*.



Note that the above is more than good enough, if you merely want to produce nice-looking, “wikified” *HTML* and *PDF* documents. The rest of this subsection will cover how to add more flexiformal semantics to the above. If this seems laborious and/or difficult, keep in mind that this is to some degree experimental still, and you are not forced to go overboard with semantic annota-



tions!

But if you aim to create a “library of symbols” for mathematical concepts, then all of the possibilities that we discuss here will add value for the community. Generally, the higher the ratio of readers to authors the more any investment in semantization will pay off.

Semantic Macros in Text Mode

The first thing we can do to further improve this document is marking up the “for all” in the definition – after all, there naturally is a [symbol](#) for the [universal quantifier](#), which can be found in `[sTeX/Logic/General]mod/syntax?UniversalQuantifier` and has the [semantic macro](#) `\forall` (as to not conflict with the [TeX](#) primitive `\forall`).

The naive approach would be to replace the “for all” by e.g. `\sr{forall}{for all}`. That would (correctly) associate and highlight the text fragment with the [symbol](#) “[universal quantifier](#)”, *but* we are not just referencing the [symbol](#) here – we are actually using it, by *applying* it to the [variables](#) a, b, c and the expression $(a \circ b) \circ c = a \circ (b \circ c)$.

In *math mode*, we can just use the [semantic macro](#) `\forall` – that will take two arguments (of [modes](#) B and i) and produce the corresponding [notation](#), so that

```
\forall{\inset{a,b,c}{\vA}}{
  \eq{ \vop{(\vop{a,b}),c} , \vop{a,(\vop{b,c})} }
}
```

will produce $\forall a, b, c \in A. (a \circ b) \circ c = a \circ (b \circ c)$.

In *text mode*, however, we don’t have a specific [notation](#) – instead, the specific “[notation](#)” is whatever sentence we want to mark up semantically. In text mode, [semantic macros](#) therefore behave differently:

1. They take *precisely* one argument, regardless of how many arguments the [macro](#) would take in math mode or (equivalently) the `args` property of the [symbol](#).
2. *Within* that argument, we can use `\comp` to highlight arbitrary text fragments, and
3. we can use the `\arg` [macro](#) to mark up the *actual* arguments that the [symbol](#) is supposed to be applied to.

`\arg` takes as optional argument the index of the argument that is being marked up; if not they are used consecutively. The starred variant `\arg*` produces no output.

So we could now do

```
\forall{\comp{For all} $\arg{\inset{a,b,c}{\vA}}$, we have
  $\arg{
    \eq{ \vop{(\vop{a,b}),c} , \vop{a,(\vop{b,c})} }
  }$
}
```

which produces “For all $a, b, c \in A$, we have $(a \circ b) \circ c = a \circ (b \circ c)$ ”.

In our case though, we want to “switch the arguments around” – first comes the equation, then the [variables](#) to be bound. Hence:


```

\foral{
  $\arg[2]{
    \leq{ \vop{(\vop{a,b}),c}, \vop{a,(\vop{b,c})} }
  }$
  \comp{for all}
  $\arg[1]{ \inset{a,b,c}{\vA} }$
}

```

which produces “ $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$ ”.

Definientia

Now we have a fully semantically annotated expression in the definition for “associative”. Can we let MMT know, that this expression really is *the* definition of the symbol?

Yes, we can. All we need to do is wrap the sentence in a `\definiens` macro (plural: *definientia*; like the word “*definiendum*” refers to “the term being defined”, “*definiens*” refers to “the thing the term is being defined as”).

The `\definiens` macro is only allowed within the `sdefinition` environment, and requires that the `environment` lists the `symbol` that gets the `definiens` attached explicitly in its `for=` argument. It is possible to attach `definientia` to multiple `symbols` within an `sdefinition` environment, in which case the symbol needs to be provided as an optional argument, e.g. we could do `\definiens[associative]{...}`. Since “associative” is the only `symbol` being defined in our definition, we can omit that argument.

Alternatively, as with `types` we can attach `definientia` to a `\symdecl` directly using the optional argument `def=...`.

At this point, you might justifiably wonder, why we even still need to declare `associative` with `\symdecl*` before we define it. And indeed, we don’t – the `sdefinition` environment takes the same optional arguments as the `\symdecl` macro, and if we explicitly provide a `name=` (or a `macro=`), it will generate a `symbol` for us. We can hence get rid of the `\symdecl*` and instead do:

```

1 \begin{sdefinition}[name=associative,args=1]
2   ...
3 \end{sdefinition}

```

One more problem remains: We stated that `associative` is to take one argument – but we haven’t told `TEX` what it is yet. In our case, the argument is represented by the `variable` `\vop`. In fact, chances are that arguments to symbols in `types` or `definientia` are almost always represented by some `variable`.

We can use one of two ways to a `variable` as being an argument:

1. If the `variable` (e.g. `\vop` with name `op`) was already declared prior to the `sdefinition` environment, we can use the `\varbind` macro in the `environment`; e.g. by adding `\varbind{op}`.
2. We can move (or copy) the `\vardef` for the `variable` into the `environment` and add `bind` to its optional arguments.

In total, our fully annotated definition now looks like this:

Example 10

Input:

```

File [sTeX/MathTutorial]props/Associative.en.tex
8 \begin{sdefinition}[name=associative,args=1]
9 \vardef{vA}[name=A,type=\collection]{A}
10 \vardef{vop}[name=op,type=\funspace{\vA,\vA}\vA,
11 args=a,assoc=bin,bind % <- argument for the symbol
12 ]{\argsep{#1}{\mathbin{\maincomp{\circ}}}}
13 \vardef{va}[name=a,type=\vA]{a}
14 \vardef{vb}[name=b,type=\vA]{b}
15 \vardef{vc}[name=c,type=\vA]{c}
16 %
17 A binary operation  $\fun{\vop!}{\vA,\vA}\vA$  is called
18 \defname{associative}, if
19 \definiens{\forall{\arg[2]}{\eq{
20 \vop{(\vop{va,vb}),vc},
21 \vop{va,(\vop{vb,vc})}}
22 }}{\comp{for all} \arg[1]{\inset{va,vb,vc}{\vA}}}.
23 \end{sdefinition}
24 %

```

Output:

Definition 3.2.2. A binary operation $\circ : A \times A \rightarrow A$ is called **associative**, if $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in A$.

And indeed, if we look at the [OMDOC](#) tab of the [HTML](#) preview, we can see that not only does [MMT](#) attach the `definiens` to the `symbol`, it has also inferred the `type` of “`associative`” from the `definiens` (Figure 16).

▼ Symbol `associative`


Definiens $\{A: \text{SET}\}_I (\circ: A \rightarrow A \rightarrow A) \rightarrow \forall a, b, c: A. \left(\frac{(a \circ b) \circ c = a \circ (b \circ c)}{A} \right)$

Type $\{A: \text{SET}\}_I (\circ: A \rightarrow A \rightarrow A) \rightarrow \text{Prop}$

Figure 16: Type Inferred from Definiens

Using Symbols Without Semantic Macros and Exporting Code in Modules

So now we don’t have a `semantic macro` for “`associative`”, but it *does* take an argument. How can we ever actually *use* the `symbol` now?

The answer is: with the `\symuse` macro. Like `\symref` and friends, `\symuse` takes a `symbol` name or the name of its `semantic macro` as argument, but behaves otherwise like using a `semantic macro` directly. So for, say, `addition`, `\symuse{addition}` and `\symuse{plus}` behave exactly like `\plus`.

In our case, this means we can do `\symuse{associative}`. “`associative`” does not have a `notation`, but in practice, we say something like “`+ is associative`” rather than using some specific mathematical `notation` for the same thing.

Combining this with what we just learned, we can now say that `addition` is `associative` by doing:

```
\symuse{associative}{\arg{\plus!}$ \comp{is associative}}
```

In fact, we would do the exact same thing every time we want to say that *some* operator is associative, so it makes sense to introduce a `macro` for this. In fact, such a `macro` is easy to define using standard `LATEX` methods. This is where `\STEXexport` becomes very handy:

In a `module`, we can put arbitrary `LATEX` code in an `\STEXexport`, and this code will be executed every time the `module` is imported via `\usemodule` or `\importmodule`. This is especially useful for `macro` definitions, and this way `modules` can almost act like `LATEX packages`!

So we can define a new `macro` `\isassociative` that applies “`associative`” to an arbitrary operation and produces the semantically marked-up text “`#1 is associative`”, and wrap that `macro` definition in an `\STEXexport`, and whenever we use the `Associative module`, we also get the `\isassociative macro`:

```
\STEXexport{
  \def\isassociative#1{
    \symuse{associative}{\arg{#1} ~is ~\comp{associative}}
  }
}
```

And now, we can do e.g. `\isassociative{\plus!$}` to produce “`+ is associative`”.



For technical reasons, `\STEXexport` processes its content in the `expl3` category code scheme – what this means is that all spaces are ignored entirely, and the characters `_` and `:` are valid characters in `macro` names.

In practice, this means you will have to use the `~` character for spaces, and if you want to use a subscript `_`, you should use the `macro` `\c_math_subscript_token` instead.

Exercise

Analogously to all the above, implement a `module` for *commutativity*; i.e the property of a binary operation that $a \circ b = b \circ a$ for all a, b . Make the `module` export a `macro` `\iscommutative` analogously to `\isassociative`.

Lösung: Can be found in `[sTeX/MathTutorial]props/Commutative.en.tex`

TODO⁵

3.2.2 Assertions

Having defined `associativity` and `commutativity`, we can now assert that both properties hold for `addition` and `multiplication`.

For *assertions* (i.e. theorems, lemmata, axioms, claims,...), `sTeX` provides the `sassertion environment`.

In the simplest case, that can look like the following:

⁵TODO: intent?

```

\begin{sassertion}
  \isassociative{\Sn{plus}}
\end{sassertion}

```

which yields

Addition is associative

Do we want this to be typeset as a **Theorem**? For that we just add a `[style=theorem]` to the `sassertion` environment, provided we have a customization for that – (see chapter 9). We can also load the `stexthm` package, which uses the `amsthm` package to provide common typesettings for the types: `theorem`, `observation`, `corollary`, `lemma`, `axiom` and `remark`.

So far, this is not too useful – after all, we could have just as well used e.g. the `amsthm` package and gone straight for the non- $\text{\LaTeX}$ variant

```

\begin{theorem}
  \isassociative{\Sn{plus}}
\end{theorem}

```

But as with `sdefinition`, we can immediately add a corresponding `symbol` in the `sassertion` environment, and have it be documented directly by the environment:

```

\begin{sassertion}[style=theorem,name=addition is associative]
  \isassociative{\Sn{plus}}
\end{sassertion}

```

And now, if we do `\sn{addition is associative}`, we get `addition is associative` with a corresponding hover pop-up (in the `HTML`).

Of course, the usefulness of this grows with more elaborate assertions. For very short assertions such as the above, we might not even want to typeset them in such a space hungry manner.

For that purpose, we provide the `\inlineass` macro (and analogously: `\inlinedef` for `sdefinition`), which takes the same optional arguments as the environment. So we could also do:

```

\inlineass[name=addition is associative]{\isassociative{\Sn{plus}}}

```

So far, `MMT` is blissfully unaware of the semantic contents of our assertions. We can easily remedy that by wrapping the expression representing the assertion in a `\conclusion` macro, analogously to the `definiens` macro in `sdefinitions`:

```

\inlineass[name=addition is associative]{
  \conclusion{\isassociative{\Sn{plus}}}
}

```

We can now see the statement in the `OMDoc` tab of the `HTML` preview (Figure 17).

Assertion	assertion
Term term	associative (+)

Figure 17: Assertion Statement in `OMDoc`

Not exactly pretty – the OMDoc tab uses notations to render content, and we did not provide any for associative.

3.2.3 Proofs



$\text{\LaTeX}$ provides the `sproof` environment for marking up *proofs*. The markup mechanism for `sproof` is still highly experimental and likely subject to change in the near future. As such, we omit a closer explanation of its usage until the syntax and functionality have sufficiently stabilized.

3.3 Mathematical Structures

A common concept in mathematics is that of a *mathematical structure* – a *tuple* of interdependent components. For example: A *monoid* is a *structure* $\langle M, \circ, e \rangle$ such that certain axioms hold; where M is a set, $\circ$ is a binary operation, and $e \in M$.

From a representational perspective, this is particularly interesting: M , $\circ$ and e in the above are not *symbols* in the same way that the previous *symbols* we considered were – they don’t represent definite objects. Instead, they are *components* of some other object, namely a monoid; where a *particular* monoid could either be a fixed object (such as $\langle \mathbb{Z}, +, 0 \rangle$) or an *indefinite* monoid; i.e. a *variable*. We call the components of a *mathematical structure* **fields**.

In this section, we will discuss how to declare and use *mathematical structures* in $\text{\LaTeX}$, build them up modularly, and connect them among each other to avoid duplication.

We will do so by considering *lattices* both algebraically and order-theoretically, and identify the two perspectives.

3.3.1 Declaring and Using Structures

The simplest kinds of *structures* are *magmas* and (*directed*) *graphs*, so we might as well start there:

Definition 3.3.1. A **magma** is a *structure* $\langle U, \circ \rangle$, where U is a *collection* and $\circ$ a binary operation $U \times U \rightarrow U$.

The obvious start is to create a new *module* `Magma`. Within this *module*, we import the `Functions` *module* so we can later assign a *type* to the operation. We can then use the `mathstructure` environment, that creates a new *symbol* “magma”:

```
\begin{smodule}{Magma}
\importmodule[sTeX/MathBase/Functions]{mod?Function}
\begin{mathstructure}{magma}
...
\end{mathstructure}
\end{smodule}
```

`mathstructure` behaves very similarly as `smodule` – within the `environment`, we can declare new `symbols`, `notations` and all that.

So within the `mathstructure`, we can add `symbols` for the two fields U and $\circ$:

```
\symdef{univ}[name=universe,type=\collection]{U}
\symdef{op}[name=operation,args=a,assoc=bin,
type=\funspace{\univ,\univ}\univ
]{\argsep{#1}{\mathbin{\maincomp{\circ}}}}
```

Once we close the `environment` (with `\end{mathstructure}`), the `symbols` are “gone”. However, we now have a new `symbol` “magma” with `semantic macro` `\magma`. Its usage is somewhat more complicated than “normal” `semantic macros`, but one thing we *can* do with it now is `\magma!`, which will produce $\langle U, \circ \rangle$.

Notably however, the `\magma` `macro` is already available *within* the `mathstructure environment` as well.

This allows us to provide an `sdefinition` using the `semantic macros` declared in the `structure`:

Example 11

Input:

File [sTeX/MathTutorial]algebra/Magma.en.tex

```
7 \begin{mathstructure}{magma}
8 \symdef{univ}[name=universe,type=\collection]{U}
9 \symdef{op}[name=operation,args=a,assoc=bin,
10 type=\funspace{\univ,\univ}\univ
11 {\argsep{#1}{\mathbin{\maincomp{\circ}}}}
12
13 \begin{sdefinition}[for={magma,univ,op}]
14 A \definame{magma} is a \sr{mathstruct}{structure} $\magma!$,
15 where $\univ$ is a \sn{collection} and $\op!$
16 a binary operation $\funspace{\univ,\univ}\univ$.
17 \end{sdefinition}
18 \end{mathstructure}
```

Output:

Definition 3.3.2. A **magma** is a `structure` $\langle U, \circ \rangle$, where U is a `collection` and $\circ$ a binary operation $U \times U \rightarrow U$.

Instantiating Structures

More importantly however, we can now declare a `variable magma`, using the optional `return=` argument. For example, we can now do

```
\vardef{vM}[name=M,return=\magma]{M}
```

and we get the `semantic macro` `\vM` with which we can do the following:

Syntax	Result
$\$ \backslash vM! \$$	M
$\$ \backslash vM\{\} \$$	$\langle U_M, \circ_M \rangle$
$\$ \backslash vM\{\text{univ}\} \$$	U_M
$\$ \backslash vM\{\text{op}\}! \$$	$\circ_M$
$\$ \backslash vM\{\text{op}\}\{a,b,c\} \$$	$a \circ_M b \circ_M c$

In other words: Given a [symbol](#) or [variable](#) with [semantic macro](#) `\foo` and `return=\struct`, then `\foo{<fn>}` behaves like the [semantic macro](#) `\fn` *within* the [mathstructure environment](#) for `struct` – but instantiated for the specific instance `foo`.

By default, [S_{TE}X](#) attaches the [symbol](#)’s (or [variable](#)’s) [operator notation](#) as a subscript suffix to the notation component marked with `\maincomp` – e.g., since the “`\circ`” in the [notation](#) for `op` is marked with `\maincomp`, doing `\$ \backslash vM\{\text{op}\}\{a,b\} \$` ultimately outputs `a \circ_{\backslash vM!} b`. Hence, we get $a \circ_M b$.

We can change the way the `\maincomp` notation component is modified, by using the optional argument `comp=` in the [semantic macro](#) for the [mathematical structure](#). For example, to not change it at all, we can do:

```
\vardef{vM}[name=M,return={\magma[comp=##1] }]{M}
```

...or to suffix it with a `'`, we can do

```
\vardef{vMp}[name=Mp,return={\magma[comp=##1'] }]{M'}
```

This allows us to do things like:

```
Let \$ \backslash vM! := \backslash vM\{\} \$ and \$ \backslash vMp! := \backslash vMp\{\} \$ \sns{magma}. Then...
```

yielding

Let $M := \langle U, \circ \rangle$ and $M' := \langle U', \circ' \rangle$ [magmas](#). Then...

We can also *assign* fields to (arbitrary) expressions, by doing `name=<tex>` in square brackets. For example we can do the following:

```
\vardef{vA}[type=\collection]{A}
```

```
\vardef{vM}[name=M,return={\magma[comp=##1][univ=\vA] }]{M}
```

```
\vardef{vMp}[name=Mp,return={\magma[comp={\##1}'] [univ=\vA] }]{M'}
```

```
Let \$ \backslash vM! := \backslash vM\{\} \$ and \$ \backslash vMp! := \backslash vMp\{\} \$ \sns{magma} on \$ \backslash vA$. ...
```

Let $M := \langle A, \circ \rangle$ and $M' := \langle A, \circ' \rangle$ [magmas](#).

Of course, we can also use `return=` with [variable](#) sequences – for example:

```
\varseq{vMs}[name=M,return={\magma[comp={\##1}_{\##1] },op=(M_i)_1^n]
{1,\ellipses,n}{\maincomp{M}_{\##1}}
```

```
Let \$ \backslash vMs! := \backslash vMs\{i\}_{1^n} \$ a sequence of \sns{magma}...
```

Let $(M_i)_1^n := \langle U_i, \circ_i \rangle_1^n$ a sequence of [magmas](#)...

Note that in the above, it seems that using `#1` in the `return` argument is allowed. Indeed, it is – the `return` statement takes the same arguments as the [semantic macro](#) itself does and is appropriately instantiated. Since the first (and only) argument to the

sequence `\vMs` is the index, when doing `\vMs{i}...` the `#1` in the `return`-statement will be replaced by `i`.

Also, note that if we want to produce M_i – i.e. the `magma` at index i in the sequence, we can do `\vMs{i}!`.



Think of the `!` as a “stop sign” - if the expression up to the `!` has an associated presentation, the `!` tells `gTeX` to “stop eating arguments” and present whatever it has until now.

3.3.2 Extending `Structures` and Axioms

It is extremely common to “build up” `structures` in a hierarchical manner by adding new fields or axioms: A *semigroup* is an associative magma. A *band* is an idempotent semigroup. A *monoid* is a semigroup with a unit. A *partial order* is an antisymmetric preorder.

We alluded to the fact earlier, that the `mathstructure environment` behaves like an `smodule` – that is literally true: Every `mathstructure foo` in a `module FooMod` is in fact also a `module ?FooMod/foo-module`. We can therefore easily extend `structures` using `\importmodule{...?FooMod/foo-module}` – but extending `structures` is so common, and using `\importmodule` tiring, that there is a shortcut: the `extstructure environment`. It takes as second argument a comma-separated list of `structure` names. That allows us to easily define `semigroups`:

Example 12

Input:

```
File [sTeX/MathTutorial]algebra/Semigroup.en.tex
8 \begin{extstructure}{semigroup}{magma}
9 \begin{sdefinition}
10 A \definame{semigroup} is a \sn{magma} $\semigroup!$,
11 where \inlineass[name=associative axiom]{
12 \conclusion{\isassociative{$\op!$}}.
13 }
14 \end{sdefinition}
15 \end{extstructure}
```

Output:

Definition 3.3.3. A `semigroup` is a `magma` $\langle U, \circ \rangle$, where $\circ$ is `associative`.

Note our usage of `\inlineass` to generate a new `symbol` for the `associative axiom`.

If we look at the `OMDOC` tab in the `HTML` preview window, we can see the output in Figure 18.

So `MMT` has decided that our statement is an *axiom*.

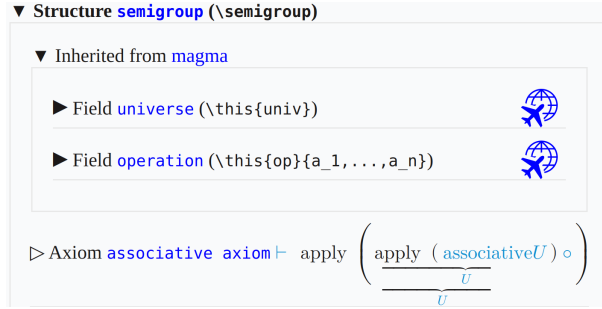


Figure 18: Axioms in OMDoc

Conservative Extensions

For **structures**, there is a *critical* distinction between *defined* and *undefined symbols*; and analogously between *theorems* and *axioms*.

Remember that **structures** are more like *templates* that are *instantiated* by particular objects. An *undefined* field in a **structure**, in that sense, is like an *obligation*: If something is supposed to be a **semigroup**, it *has to* have a **universe**, an **operation** and the **operation** needs to satisfy the **associative axiom**.

Defined fields on the other hand have a *definiens* on the basis of the remaining fields – they don’t need to be explicitly provided for something to instantiate the **structure**; if all the *undefined* fields are provided, the *defined* ones we get “for free”.

The same holds for *theorems*: If a statement is *provable* from the axioms, then we don’t need to explicitly prove it to hold for some particular instance – we have a proof already, provided the axioms hold.

The relation between axioms and theorems is not just analogous to that between undefined and defined **symbols**: It is the very same. Remember the **judgments as types** paradigm?

STEX For a **proposition** P , an assertion in **STEX** induces a **symbol** of $\text{type} \vdash P$. Without a proof, this **symbol** is *undefined* – and hence an *axiom*. A *proof* for P is a specific term of $\text{type} \vdash P$ – i.e. a potential *definiens*. To prove an assertion turns it into a *theorem*, which is to say that the **symbol** can be *defined*.

One consequence of this is: Extending a **structure** only by *defined* fields does not actually (conceptually) introduce a *new structure* – every instance of the old one *should* also be an instance of the new one. The new fields are basically just “syntactic sugar”.

There is a name for extending a **structure** only by defined fields (or theorems): A *conservative extension*.

STEX provides the **extstructure*** environment for that purpose. Unlike **extstructure**, it does *not* take a name (technically, **STEX** generates one internally). Instead, conceptually **extstructure*** modifies the extended **structure** directly, rather than generating a new **structure**. The caveat however is, that every **symbol** introduced in an **extstructure*** must be defined.

Consider the following conservative extension:

Example 13

Input:

```

File [sTeX/MathTutorial]algebra/MagmaSquare.en.tex
7 \begin{extstructure*}{magma}
8 \begin{sdefinition}[macro=sq,args=1]
9 \notation{sq}[op=\cdot^2][\{#1\}^{\comp 2}]
10 \vardef{va}[name=a,type=\univ,bind]{a}
11 Let $\inset{\va}{\univ}$. We define
12 $\defnotation{sq}{\va} := \definiens{op}{\va,\va}$.
13 \end{sdefinition}
14 \end{extstructure*}

```

Output:

Definition 3.3.4. Let $a \in U$. We define $a^2 := a \circ a$.

Via `\definiens`, the new symbol `sq` is now *defined* (note the `macro=` argument, taht generates a *semantic macro* as well). Whenever we import the containing *module*, we now have an additional field `sq` in (any extension of) `magma` – e.g., as `semigroup` extends `magma`, the following is now valid:

```

\usemodule[sTeX/MathTutorial]{algebra?MagmaSquare}
\vardef{vsg}[name=S,return=\semigroup]{S}
$\vsg{sq}{a}$

```

...producing a^2 .

3.3.3 Nesting Structures and `\this`

A perhaps not too surprising, but a notable aspect of *structures* is that fields themselves can be instances. This is important for example for implementing *vector spaces*, but can also be used to bundle things that are not normally thought of as *structures*, such as objects with certain defining properties.

Take as an example, the notion of a (magma) *homomorphism*:

Definition 3.3.5. Let $M_1 = \langle U_1, \circ_1 \rangle$ and $M_2 = \langle U_2, \circ_2 \rangle$ magmas. A **magma homomorphism** is a function $F : U_1 \rightarrow U_2$ such that $F(a \circ_1 b) = F(a) \circ_2 F(b)$ for all $a, b \in U_1$.

So a *homomorphism* is a *function* with certain properties. And *structures* can be used to “bundle” the *function* itself with both the *magmas* on whose universes the *function* operates, as well as the *axiom* that *makes* it a *homomorphism*. After all, considered as a mere *function*, $F : U_1 \rightarrow U_2$ contains no information about the operation with respect to which it is homomorphic.

The first thing to note is that we can provide `mathstructure` with an optional argument for a *name* distict from the name of its *semantic macro*. We then add two fields that *return magmas*. So far, so unexciting:

```

\begin{mathstructure}{magmahom}[magma homomorphism]
\symdef{dom}[name=domain,return={\magma[comp={##1}_1]}\{M_1\}
\symdef{cod}[name=codomain,return={\magma[comp={##1}_2]}\{M_2\}

```

For the `function` itself, we know how to give it a maningful `type`, already:

```

\symdef{f}[type=\funspace{\dom{univ}}{\cod{univ}},args=1]{???}

```

...but what should its `notation` be? Ideally we would want it to just be the `notation` of whatever particular instance it is – in informal mathematics, we rarely distinguish notationally between a `homomorphism` and its underlying `function` (to the point where it's not clear, whether *informally* the distinction is even meaningful). Similarly, we rarely distinguish e.g. between a `magma` (or semigroup, monoid, group, ring, vector space,...) and its underlying universe.

This is where `\this` comes into play (pun intended). Within an `mathstructure` or `exstructure`, or in the context of a particular instance of one, `\this` represents “the” instance.

We can set it in the context of `mathstructure` as a further optional argument; e.g.

```

\begin{mathstructure}{magmahom}[magma homomorphism,this=F]

```

and then use `\this` in the `notation` for the `function`. We can further provide the `homomorphism condition` as an axiom using `\inlineass`:

Example 14

Input:

```

File [sTeX/MathTutorial]algebra/Homomorphism.en.tex
9 \begin{mathstructure}{magmahom}[magma homomorphism,this=F]
10 \symdef{dom}[name=domain,return={\magma[comp={##1}_1]}\{M_1\}
11 \symdef{cod}[name=codomain,return={\magma[comp={##1}_2]}\{M_2\}
12 \symdef{f}[op=\this,args=1,
13 type=\funspace{\dom{univ}}{\cod{univ}}
14 ]{\this \dobrackets{##1}}
15
16 \begin{sdefinition}[for={magmahom,dom,cod,f}]
17 \vardef{va}[name=a,type=\dom{univ}]{a}
18 \vardef{vb}[name=b,type=\dom{univ}]{b}
19 Let $\dom!=\dom{}$ and $\cod!=\cod{}$ \sns{magma}.
20 A \definame{magmahom} is a function
21 $\fun{\!f!}{\dom{univ}}{\cod{univ}}$ such that
22 \inlineass[name=homomorphism condition]{\conclusion{\forall{
23 $\arg[2]{\eq{
24 \f{\dom{op}}{\va,\vb}}, \cod{op}}{\f{\va},\f{\vb}}
25 }}$ \comp{for all} $\arg[1]{\inset{\va,\vb}{\dom{univ}}}$$.
26 }}}
27 \end{sdefinition}
28 \end{mathstructure}

```

Output:

Definition 3.3.6. Let $M_1 = \langle U_1, \circ_1 \rangle$ and $M_2 = \langle U_2, \circ_2 \rangle$ magmas. A **magma homomorphism** is a function $F : U_1 \rightarrow U_2$ such that $F(a \circ_1 b) = F(a) \circ_2 F(b)$ for all $a, b \in U_1$.

Now if we instantiate our `magma homomorphism`:

```
\vardef{vh}[name=H,return={\magmahom[this=H] }]{H}
```

Here is a list of what we can do now:

Syntax	Result
$\$ \backslash \text{vh} ! \$$	H
$\$ \backslash \text{vh} \{ \} \$$	$\langle M_1, M_2, H \rangle$
$\$ \backslash \text{vh} \{ f \} ! \$$	H
$\$ \backslash \text{vh} \{ f \} \{ a \} \$$	$H(a)$
$\$ \backslash \text{vh} \{ \text{dom} \} ! \$$	M_1
$\$ \backslash \text{vh} \{ \text{cod} \} \{ \} \$$	$\langle U_2, \circ_2 \rangle$
$\$ \backslash \text{vh} \{ \text{cod} \} \{ \text{univ} \} \$$	U_2
$\$ \backslash \text{vh} \{ \text{dom} \} \{ \text{op} \} ! \$$	$\circ_1$
$\$ \backslash \text{vh} \{ \text{cod} \} \{ \text{op} \} \{ a, b, c \} \$$	$a \circ_2 b \circ_2 c$

Note how – as one would expect – we can treat $\backslash \text{vh} \{ \text{dom} \}$ and $\backslash \text{vh} \{ \text{cod} \}$ like any other instance of [magma](#).



Note that some of the outputs in the above table are probably not quite what we want. Determining the precise typesetting of an expression involving *nested paths* of fields is difficult, to say the least (e.g., what exactly should $\backslash \text{this}$ refer to in a deeply nested sequence of fields?).

Using instances within [structures](#) is still very useful; at the very least when defining [structures](#). When subsequently *using* [structures](#), however, accessing fields of fields (of fields (of ...)) of an instance should be avoided.

Luckily, there is rarely a need for doing so – in practice, those fields we might want to access in such a way, we usually also want to provide specific [notations](#) and talk about independently of the “containing” instance, such that introducing a new [variable](#) (or [symbol](#)), and assigning the corresponding field to that [variable](#), makes considerably more sense. And subsequently using the [variable](#) is easier than concatenating $\{ \dots \}$, too.

3.4 Complex Inheritance and Theory Morphisms



We are starting to approach seriously experimental territory.

While the theory behind all the following is relatively well understood, and their implementation in [MMT](#) is mature, the same can not be said out the implementation in [sTeX](#).

There are still kinks to be ironed out, but feel free to experiment.

We now have all the tools available to progress towards something more interesting. Here is a list of documents with respective [modules](#) and [symbols](#) we will build on in the following:

[sTeX/MathTutorial]props/Idempotent.en.tex

Definition 3.4.1. Let $e \in A$ and $\circ : A \times A \rightarrow A$. e is called **idempotent** with respect to $\circ$, if $e \circ e = e$.

Definition 3.4.2. The operation $\circ : A \times A \rightarrow A$ is called **idempotent**, if every element $a \in A$ is **idempotent** with respect to $\circ$.

[sTeX/MathTutorial]props/Distributive.en.tex

Definition 3.4.3. Let $\odot : B \times A \rightarrow A$ and $\oplus : A \times A \rightarrow A$. We say $\odot$ **distributes over** $\oplus$, if $b \odot (a_1 \oplus a_2) = (b \odot a_1) \oplus (b \odot a_2)$ for all $a_1, a_2 \in A$ and $b \in B$.

[sTeX/MathTutorial]props/Absorption.en.tex

Definition 3.4.4. Let $\odot : A \times B \rightarrow A$ and $\oplus : A \times B \rightarrow B$. We say $\odot$ **absorbs** $\oplus$, if $a_1 \odot (a_1 \oplus b) = a_1$ for all $a_1 \in A$ and $b \in B$.

[sTeX/MathTutorial]algebra/Band.en.tex

Definition 3.4.5. A **band** is an **idempotent semigroup**.

[sTeX/MathTutorial]algebra/Semilattice.en.tex

Definition 3.4.6. A **semilattice** is a **commutative band**.

[sTeX/MathTutorial]props/Reflexive.en.tex

Definition 3.4.7. A binary relation R on A is called **reflexive**, if $R(a, a)$ for all $a \in A$.

[sTeX/MathTutorial]props/Symmetric.en.tex

Definition 3.4.8. A binary relation R on A is called **symmetric**, if $R(a, b)$ implies $R(b, a)$ for all $a, b \in A$.

[sTeX/MathTutorial]props/Transitive.en.tex

Definition 3.4.9. A binary relation R on A is called **transitive**, if $R(a, b)$ and $R(b, c)$ implies $R(a, c)$ for all $a, b, c \in A$.

[sTeX/MathTutorial]props/Antisymmetric.en.tex

Definition 3.4.10. A binary relation R on A is called **antisymmetric**, if $R(a, b)$ and $R(b, a)$ implies $a = b$ for all $a, b \in A$.

[sTeX/MathTutorial]orders/Graph.en.tex

Definition 3.4.11. A **directed graph** is a structure $\langle U, R \rangle$, where U is a collection and R a binary relation on U .

Definition 3.4.12. An **(undirected) graph** is a directed graph $\langle U, R \rangle$, where R is symmetric.

[sTeX/MathTutorial]orders/Preorder.en.tex

Definition 3.4.13. A structure $\langle U, \leq \rangle$ is called a **preorder** (or **quasiorder**, or **preordered set**; in short **proset**), if $\leq$ is reflexive and transitive.

[sTeX/MathTutorial]orders/Poset.en.tex

Definition 3.4.14. A preorder $\langle U, R \rangle$ is called a **partial order** (or **poset**), if R is antisymmetric.

[sTeX/MathTutorial]orders/InfSup.en.tex

Definition 3.4.15. Let $\langle U, R \rangle$ a poset. An element $a \in U$ is called an **infimum** or **greatest lower bound** of x_1 and x_2 , if $a R x_1$, $a R x_2$, and for any x with $x R x_1$ and $x R x_2$, we have $x R a$.

Definition 3.4.16. Let $\langle U, R \rangle$ a poset. An element $a \in U$ is called a **supremum** or **least upper bound** of x_1 and x_2 , if $x_1 R a$, $x_2 R a$, and for any x with $x_1 R x$ and $x_2 R x$, we have $a R x$.



Note that **infima** and **suprema** are more generally defined on *sets* of elements. Doing so in sTeX is significantly more complicated *for now*, and will require some amount of research to make convenient – especially if we want to subsequently define *operators* on pairs of elements, as below. We therefore opt for the simpler version where it is defined as binary from the get go.

Definition 3.4.17. A poset $\langle U, R \rangle$ is called a **meet semilattice** if for every two elements a, b the **infimum** $a \wedge b$ exists.

Definition 3.4.18. A poset $\langle U, R \rangle$ is called a **join semilattice** if for every two elements a, b the **supremum** $a \vee b$ exists.

Definition 3.4.19. An **(order) semilattice** is a **meet** and **join semilattice**.

Exercise

Try to implement all of the above yourself!

3.4.1 Glueing Structures Together

We now want to progress towards **lattices**, i.e. the following:

Definition 3.4.20. A **lattice** is a **structure** $\langle U, \wedge, \vee \rangle$ such that $\langle U, \wedge \rangle$ and $\langle U, \vee \rangle$ are **semilattices**, and $\vee$ **absorbs** $\wedge$ and vice versa; i.e. $a \vee (a \wedge b) = a$ and $a \wedge (a \vee b) = a$. The operations $\wedge$ and $\vee$ are called **meet** and **join**, respectively.

So we make a new **module**, open an **extstructure environment** and... realize two problems:

1. We can't just extend **semilattice**: We need *two* copies of **semilattice** that share a universe, and importing **semilattice** twice is of course redundant.
2. We also want to *rename* the operations of the two **semilattices** to be subsequently called **join** and **meet**.

What we need is a way to *inherit* from **semilattice** while a) *modifying* the **symbols** therein, and b) not be **idempotent** – i.e. two imports from the same **structure** or **module** should not be identified. We can do that with the **\copymod macro**, which takes three arguments:

1. A *name* for the copy,
2. the **structure** or **module** to copy, and
3. a comma-separated list of renamings and redefinitions of the **symbol**. $\langle symbol \rangle = \langle def \rangle$ redefines $\langle symbol \rangle$, $\langle symbol \rangle @ \langle newname \rangle$ renames it, $\langle symbol \rangle = \langle def \rangle @ \langle newname \rangle$ (or $\langle symbol \rangle @ \langle newname \rangle = \langle def \rangle$) does both.

In our case, we want two copies of **semilattice**, which we will call **meets1** and **joins1**. In the first copy, we only want to rename **op** to **meet**. In the second, we want to rename **op** to **join**, and *also* redefine the universe to be the one from **meets1**:

```

\copymod{meetsl}{semilattice}{
  op @ meet
}
\copymod{joinsl}{semilattice}{
  univ = \univ,
  op @ join
}

```

You might have already noticed some problem with that – which of the two universes does `\univ` refer to now? (They are *defined* as equal, but `LTEX` does not know that!) Or which of the two commutative axioms does “commutative axiom” refer to now? Everything is ambiguous now!

Not really - if you have wondered why the `\copymod` takes a *name* as argument: The name is prefixed to every *symbol* name. Hence, the universe in `joinsl` is now called `joinsl/universe`, and the one in `meetsl` is called `meetsl/universe`. Furthermore, `\copymod` by default generates no *semantic macros* for any of the imported *symbols* – except for those renamed with `@`. In fact, what the `@` syntax actually does, is to generate a *semantic macro* by that name. If we want to change the *name* (that is shown when using `\symname` et al), we add that new name in square brackets. Hence, what we really want to do is:

```

\copymod{meetsl}{semilattice}{
  univ @ univ,
  op @ [meet]meet
}
\copymod{joinsl}{semilattice}{
  univ = \univ,
  op @ [join]join
}

```

This now gives us two copies of *semilattice*, generates *semantic macros* `\univ` for `meetsl/universe`, `\meet` for `meetsl/op` and `\join` for `joinsl/op`, and renames `meetsl/op` to `meet` and `joinsl/op` to `join`.

That allows us to then add the *absorption* axioms, an *sdefinition* for *lattice* and subsequently `\lattice!` produces $\langle U, \wedge, \vee \rangle$, with all axioms inherited (see [sTeX/MathTutorial]algebra/Lattice.en.tex).

3.4.2 Realizations

A very common situation we find in connection with *mathematical structures* is that “every *this* is a *that*” (or the concrete case “*this* is a *that*”).

With what we did so far, we are in this situation regarding the algebraic definition of *semilattices* and the order-theoretic one (exemplary *meet semilattice*).

In *MMT* parlance, this corresponds to a *total (implicit) theory morphism* from “that” to “this”.

In *sTeX* words, we want to inherit from “that” by assigning all the *symbols* in “that” to concrete terms. In our case:

[sTeX/MathTutorial]algebra/SemiLatticeOrder.en.tex

Definition 3.4.21. Let $\langle U, \circ \rangle$ a *semilattice*. We let $a \mathrel{R} b$ iff $a \circ b = a$.

Satz 3.4.22. $\langle U, R \rangle$ is a *meet semilattice*.

Proof:

We need to prove the following

1. **reflexivity** ($a R a$):

We need to show $a \circ a = a$. Follows from the **idempotent axiom**.

2. **antisymmetry** ($a R b$ and $b R a$ implies $a = b$):

Assume $a \circ b = a$ and $b \circ a = b = a \circ b$ (by the **commutative axiom**). Hence, $a = b$

3. **transitivity** (If $a R b$ and $b R c$, then $a R c$.):

Assume $a \circ b = a$ and $b \circ c = b$. Then $a \circ c = (a \circ b) \circ c = a \circ (b \circ c) = a \circ b = a$. Hence, $a R c$.

4. $a \circ b$ is the **infimum** of $\{a, b\}$:

By definition (and the **commutative axiom**), $a \circ b R a$ and $a \circ b R b$. We need to show, that if $x R a$ and $x R b$, then $x R a \circ b$. Assume $x \circ a = x$ and $x \circ b = x$. Then $x \circ (a \circ b) = (x \circ a) \circ b = x \circ b = x$. Hence $x R a \circ b$

□

So to be precise, we want to provide *definientia* for all undefined **symbols** in **meet semilattice** (i.e. the **relation** and **meet**) and *proofs* for all *axioms* (**reflexive axiom**, **antisymmetric axiom**, **transitive axiom**, and **infimum axiom**), and by so obtain the fact that every **semilattice** is a **meet semilattice**.

For that purpose, we have the **\realize macro**. It behaves like **\copymod**, but does not take a name, and additionally requires that all undefined fields get assigned. So we could do the following:

Example 15

Input:

```
File [sTeX/MathTutorial]algebra/SemiLatticeOrder1.en.tex
8 \begin{extstructure*}{semilattice}
9 \interpretmod{meetsl}{meetsl}{
10   univ = \univ,
11   meet @ [meet]meet = \op!,
12   rel @ [order]order = \map{a,b}{\eq{\op{a,b},a}},
13   reflexive axiom = trivial,
14   transitive axiom = trivial,
15   antisymmetric axiom = trivial,
16   infimum axiom = trivial
17 }
18 \end{extstructure*}
19
20 \vardef{mysl}{return=\semilattice}{S}
21 $\mysl{order}\{a,b\} \quad \mysl{}[univ,op,order]$
```

Output:

$$a \leq_S b \quad \langle U_S, \circ_S, \leq_S \rangle$$

As we can see, we can now access the field `order`, which is renamed from `relation` in `meet semilattice` and also has the desired definiens in `MMT`. But of course this approach is very “declarative”: We do all the assigning in one `macro`, rather than narratively as what they *should* be: definitions and proofs.

If we want to achieve the more narrative version at the beginning of the subsection, we can use the `realization environment` instead. It behaves like the `\realize macro`, but allows us to do the assignments and renamings individually somewhere in the body of the `environment`, interleaved with arbitrary text. Additionally, within the `environment`, all `sTeX` features that introduce *definiencia* (like the `\definiens macro`) induce assignments instead.

To declaratively rename or assign fields, we can then use the `\assign` and `\renamedecl` macros instead. That allows us to do the following instead:

```
\begin{realization}{meetsl}
  \assign{univ}{\univ}
  \assign{meet}{\op!}
  \renamedecl{rel}[order]{order}
  ...
```

...and then use text to do the remaining assignments. For example, we can use the `sdefinition environment` to assign `rel` to the desired definiens:

```
\usestructure{meetsl}
\begin{sdefinition}[for=order]
  \varbind{va,vb}
  Let  $\$ \backslash semilattice! [univ,op] \$ a \backslash sn \{ semilattice \} .$ 
  We let  $\$ \backslash rel \{ \backslash va, \backslash vb \} \$$ 
  iff  $\$ \backslash definiens \{ \backslash eq \{ \backslash op \{ \backslash va, \backslash vb \}, \backslash va \} \} \$ .$ 
\end{sdefinition}
```

And now `sTeX` will use the `\definiens` to assign $a, b \mapsto a \circ b = a$ to the `relation` of `meet semilattice`.

Analogously, we can use the `sproof` and `subproof` environments to produce “definiencia” (i.e. proofs) for the axioms (see [sTeX/MathTutorial]algebra/SemiLatticeOrder.en.tex)

Chapter 4

Extensions for Education

The last two chapters have shown generic markup and semantization facilities in `sTeX`. As said before, investments in semantic markup pay off, iff the impact of a document is high, e.g. if there are many more readers than authors or if the semantic services afforded by the semantic markup can help reduce the help readers need to understand the material.

Educational documents constitute one category of high-impact documents which are supported by the `sTeX` ecosystem, we will cover them here. In fact, educational documents have been one of the initial document categories `sTeX` has been developed for. The idea is that if we can mark up the meaning and didactic role of learning objects, we can base learning support services on that and embed them into the documents.

Another reason educational documents are particularly interesting is that in a sense all academic communication is educational, as all documents try to “teach” the reader new concepts and results.

Concretely, we cover a document class for combining slides and course notes (section 4.1) and functionality for marking up problems and exercises (section 4.2) and for marking up homework assignments and exams (section 4.3).

4.1 Slides and Course Notes

4.1.1 Introduction

The `notesslides` document class is derived from `beamer.cls` [`beamerclass:on`], it adds a “notes version” for course notes that is more suited to printing than the one supplied by `beamer.cls`.

The `notesslides` class takes the notion of a slide frame from Till Tantau’s excellent `beamer` class and adapts its notion of frames for use in `sTeX`. To support semantic course notes, it extends the notion of mixing frames and explanatory text, but rather than treating the frames as images (or integrating their contents into the flowing text), the `notesslides` package displays the slides as such in the course notes to give students a visual anchor into the slide presentation in the course (and to distinguish the different writing styles in slides and course notes).

In practice we want to generate two documents from the same source: the slides for presentation in the lecture and the course notes as a narrative document for home study.

To achieve this, the `notesslides` class has two modes: *slides mode* and *notes mode* which are determined by the package option.

4.1.2 Package Options

The `notesslides` class takes a variety of class options:

slides notes	The options <code>slides</code> and <code>notes</code> switch between slides mode and notes mode (see subsection 10.1.3).
sectocframes topsect	If the option <code>sectocframes</code> is given, then for the <code>sfragments</code> , special frames with the <code>sfragment</code> title (and number) are generated. The <code>topsect</code> allows to specify the top section level (e.g. <code>chapter</code> , <code>section</code> , ...) for documents that are formally stand-alone, but intended to be chapters, sections, or
frameimages fiboxed	If the option <code>frameimages</code> is set, then slide mode also shows the <code>\frameimage</code> -generated frames (see ???). If also the <code>fiboxed</code> option is given, the slides are surrounded by a box.

4.1.3 Notes and Slides

frame (*env.*) Slides are represented with the `frame` environment just like in the `beamer` class, see [Tantau:ugbc] for details.

note (*env.*) The `notesslides` class adds the `note` environment for encapsulating the course note fragments. Content of this environment is only shown in *notes mode*.

By interleaving the `frame` and `note` environments, we can build course notes as shown here:

```

1 \ifnotes\maketitle\else
2 \frame[noframenumbering]\maketitle\fi
3
4 \begin{note}
5   We start this course with ...
6 \end{note}
7
8 \begin{frame}
9   \frametitle{The first slide}
10  ...
11 \end{frame}
12 \begin{note}
13   ... and more explanatory text
14 \end{note}
15
16 \begin{frame}
17   \frametitle{The second slide}
18   ...
19 \end{frame}
20 ...

```

¹EDNOTE: MK: maybe this option should be used in the other `STEX` classes as well. Actually I think this already is treated by `stex.sty`; where to document?

`\ifnotes` Note the use of the `\ifnotes` conditional, which allows different treatment between `notes` and `slides` mode – manually setting `\notesttrue` or `\notesfalse` is strongly discouraged however.



We need to give the title frame the `noframenumbering` option so that the frame numbering is kept in sync between the slides and the course notes.



The `beamer` class recommends not to use the `allowframebreaks` option on frames (even though it is very convenient). This holds even more in the `notesslides` case: At least in conjunction with `\newpage`, frame numbering behaves funnily (we have tried to fix this, but who knows).

`\inputref*` If we want to transclude a the contents of a file as a note, we can use a new variant `\inputref*` of the `\inputref` macro: `\inputref*{foo}` is equivalent to `\begin{note}\inputref{foo}\end{note}`.

`nparagraph (env.)` There are some environments that tend to occur at the top-level of `note` environments.
`nparagraph (env.)` We make convenience versions of these: e.g. the `nparagraph` environment is just an
`ndefinition (env.)` `sparagraph` inside a `note` environment (but looks nicer in the source, since it avoids one
`nexample (env.)` level of source indenting). Similarly, we have the `nfragment`, `ndefinition`, `nexample`,
`nsproof (env.)` `nsproof`, and `nassertion` environments.
`nassertion (env.)`

4.1.4 Managing and Customizing Themes

As the `notesslides` package is based on the `beamer` class, it can in principle (in slides mode) take advantage of all `beamer` themes. Themes can be specified/loaded with the normal `\usetheme`, but the `notesslides` package provides the `\libusetheme` macro:

`\libusetheme` In analogy to the `\libinput` functionality of the `stex` package, this macro allows loading `beamer` themes from the `lib` directories leading up to the current archive.

Unfortunately, there is no `beamer` functionality of using a themed `frame` environment in `article` mode. Therefore `notesslides` has to hand-craft one – for a very limited set of themes. The `notesslides` package and class comes with a simple default theme named `sTeX` that provided by the `beamterthemesTeX.sty` style file. It is assumed as the default theme for `sTeX`-based notes and slides. The result in `notes` mode (which is like the `slides` version except that the slide height is variable) is

Test Slide

▷ Some text


©: MiKo
1
sTeX

The footer line can be customized. In particular the logos.

`\setslidelogo` The default logo provided by the `notesslides` package is the $\text{\LaTeX}$ logo it can be customized using `\setslidelogo{<logo name>}`.

`\setsource` The default footer line of the `notesslides` package mentions copyright and licensing. In `notesslides \source` stores the author's name as the copyright holder. By default it is the author's name as defined in the `\author` macro in the preamble. `\setsource{<name>}` can change the writer's name.

`\setlicensing` For licensing, we use the Creative Commons Attribution-ShareAlike license by default to strengthen the public domain. If package `hyperref` is loaded, then we can attach a hyperlink to the license logo. `\setlicensing[<url>]{<logo name>}` is used for customization, where `<url>` is optional.

These macros can be used set up a institution-specific theme; for instance the one for FAU simply customizes the logo:

```
1 % Beamer slide theme for FAU;
2 \typeout{Beamer FAU theme}
3 \RequirePackage{beamerthemesTeX}
4 \setslidelogo[courses/FAU/admin]{PIC/FAU_Logo_Bildmarke.png}
```

This could be placed in one of the `lib` folders available to the current archive and could then be activated with

```
1 \documentclass[slides]{notesslides}
2 \libusetheme{FAU}
```

in the preamble of the main `noteslides` file.

4.1.5 Frame Images

Sometimes, we want to integrate slides as images after all – e.g. because we already have a PowerPoint presentation, to which we want to add $\text{\LaTeX}$ notes.

`\frameimage` In this case we can use `\frameimage[<opt>]{<path>}`, where `<opt>` are the options of `\includegraphics` from the `graphicx` package [`CarRah:tpp99`] and `<path>` is the file path (extension can be left off like in `\includegraphics`). We have added the `label` key that allows to give a frame label that can be referenced like a regular `beamer` frame.

The `\mhframeimage` macro is a variant of `\frameimage` with repository support. Instead of writing

```
1 \frameimage{\MathHub{fooMH/bar/source/baz/foobar}}
```

we can simply write (assuming that `\MathHub` is defined as above)

```
1 \mhframeimage[fooMH/bar]{baz/foobar}
```

Note that the `\mhframeimage` form is more semantic, which allows more advanced document management features in `MathHub`.

If `baz/foobar` is the “current module”, i.e. if we are on the MathHub path `...MathHub/fooMH/bar...`, then stating the repository in the first optional argument is redundant, so we can just use

```
1 \mhframeimage{baz/foobar}
```

`\textwarning` The `\textwarning` macro generates a warning sign: 

4.1.6 Ending Documents Prematurely

`\prematurestop`
`\afterprematurestop` For prematurely stopping the formatting of a document, $\TeX$ provides the `\prematurestop` macro. It can be used everywhere in a document and ignores all input after that – backing out of the `sfragment` environments as needed. After that – and before the implicit `\end{document}` it calls the internal `\afterprematurestop`, which can be customized to do additional cleanup or e.g. print the bibliography.

`\prematurestop` is useful when one has a driver file, e.g. for a course taught multiple years and wants to generate course notes up to the current point in the lecture. Instead of commenting out the remaining parts, one can just move the `\prematurestop` macro. This is especially useful, if we need the rest of the file for processing, e.g. to generate a theory graph of the whole course with the already-covered parts marked up as an overview over the progress; see `import_graph.py` from the `lmhtools` utilities [[lmhtools:github:on](#)].

Text fragments and modules can be made more re-usable by the use of global variables. For instance, the admin section of a course can be made course-independent (and therefore re-usable) by using variables (actually token registers) `courseAcronym` and `courseTitle` instead of the text itself. The variables can then be set in the $\TeX$ preamble of the course notes file.

4.1.7 Global Document Variables

To make document fragments more reusable, we sometimes want to make the content depend on the context. We use **document variables** for that.

`\setSGvar` `\setSGvar{<vname>}{<text>}` to set the global variable `<vname>` to `<text>` and `\useSGvar{<vname>}`
`\useSGvar` to reference it.

`\ifSGvar` With `\ifSGvar` we can test for the contents of a global variable: the macro call `\ifSGvar{<vname>}{<val>}{<ctext>}` tests the content of the global variable `<vname>`, only if (after expansion) it is equal to `<val>`, the conditional text `<ctext>` is formatted.

4.1.8 Excursions

In course notes, we sometimes want to point to an “excursion” – material that is either presupposed or tangential to the course at the moment – e.g. in an appendix. The typical setup is the following:

```

1 \excursion{founif}{../fragments/founif.en}
2 {We will cover first-order unification in}
3 ...
4 \begin{appendix}\printexcursions\end{appendix}

```

It generates a paragraph that references the excursion whose source is in the file `../fragments/founif.en.tex` and automatically books the file for the `\printexcursions` command that is used here to put it into the appendix. We will look at the mechanics now.

`\excursion` The `\excursion{<ref>}{<path>}{<text>}` is syntactic sugar for

```

1 \begin{nparagraph}[title=Excursion]
2   \activateexcursion{founif}{../ex/founif}
3   We will cover first-order unification in \sref{founif}.
4 \end{nparagraph}

```

`\activateexcursion` Here `\activateexcursion{<path>}` augments the `\printexcursions` macro by a call
`\inputref{<path>}`. In this way, the `\printexcursions` macro (usually in the appendix)
`\printexcursion` will collect up all excursions that are specified in the main text.
`\excursionref`

Sometimes, we want to reference – in an excursion – part of another. We can use `\excursionref{<label>}` for that.

`\excursiongroup` Finally, we usually want to put the excursions into an `sfragment` environment and add an introduction, therefore we provide the a variant of the `\printexcursions` macro: `\excursiongroup[id=<id>,intro=<path>]` is equivalent to

```

1 \begin{note}
2 \begin{sfragment}[id=<id>]{Excursions}
3   \inputref{<path>}
4   \printexcursions
5 \end{sfragment}
6 \end{note}

```

Contents

4.2 Problems and Exercises

4.2.1 Background

Problems/exercises are text fragments that contain a task assigned to the learner – e.g. computing the value of a specified quantity, simplifying an expression, modeling a described situation in a mathematical structure, or judging the veracity of a given statement. Problems/exercises are used in very different contexts, in

- *homework assignments and formal exams*, where they are graded and points are awarded for course credit,

- *self-study materials* that allow learners explore applications of some concepts and/or practice,
- *automated tutoring systems* to determine learner competencies to trigger computer supported learning services.

In all of these contexts, diagnosis the correctness or suitability of an answer plays a great role, e.g. for grading, the generation of feedback, or the suggestion of remedial actions that may “heal” any diagnosed competency gaps or misconceptions. To support that we might want to specify “answer classes” that classify answers received for automating/triggering feedback and grading.

There are various types of problems/exercises in practical use. They are mainly distinguished by how they support diagnosis of student answers: there are

- open problems, where expected answers are free-form, and classification into answer classes is usually a manual process; see subsection 4.2.3
- single/multiple choice questions where the answer classes and thus feedback/grading correspond to the selection pattern of items; see subsection 4.2.7.
- fill-in-the-blanks questions where we may want to specify how the answer string is interpreted; see subsection 4.2.8.

Additionally, problems and exercises often come with text fragments that serve auxiliary functions: hints, notes, and and grading specifications. Furthermore, we can specify how long solving a given problem is estimated to take and how many points will be awarded for a perfect solution – e.g. in an exam. See subsection 4.2.10 for details.

The `problem` package gives semantic markup support for all these aspects of problems/exercises with a focus on problem libraries that collect carefully formulated and tested problems/exercises, so that they can be re-used in many contexts. It also tries to support all possible stakeholders in dealing with problems/exercises:

- *learners*, who try to solve problems and may profit from feedback
- *instructors*, who pose problems in homeworks, quiz, and exams,
- *graders*, who try to assess learner’s competencies from the answers given
- *authors* of learning objects and (automated) *course generators*, who may use problems to diagnose learner’s competencies (e.g. as prerequisites).

4.2.2 The Package, Options, and Configuration

The `problem` package provides functionality for marking up problems and exercises as semantic sources from which the presentations for the various contexts can be generated. E.g. documents without solutions for paper or online exams, and the corresponding exams with master solutions for exam reviews. Similarly with/without hints, or points. Their visibility is specified in the options of the `problem` package, which can be used in any L^AT_EX class. The following is a typical preamble for a problem file:

```
\documentclass[lang=de]{article}
\usepackage[solutions,hints,pts,min]{problem}
```

Here we have specified the options `solutions` (solutions should be shown), `hints` (hints should be given), `pts` (display the points awarded for solving the problem?), `min` (display the estimated minutes for problem solving). Leaving out the options would make the corresponding functionality invisible.

The `problem` package generates content and labels according to the language specified in the `lang` key of the main document², these can be localized by in the files `ldf/problem-<lang>.ldf`⁶.

The `problem` package also supplies the `test` option, which specifies that the content is intended for use in an assessment situation, where certain additional content (e.g. exam galse) should be shown while other content (e.g. solutions) should not be.

Note that documents (or document fragments) with problems are often formatted in different versions that can be configured by these options. Therefore the following variant of the preamble prefix above can be very useful:

```
\documentclass[lang=de]{article}
\providecommand\probopts{,solutions,min}
\usepackage[hints,pts\probopts]{problem}
```

We add a level of indirection via the `\probopts` macro (`\providecommand` sets it unless it already exists). If the current file is `foo.tex`, then we can make a L^AT_EX file `foo-test.tex` that formats in test mode as

```
\newcommand\probopts{,test}\input{foo}
```

Alternatively, we do not need a file at all: we can just (e.g. in a Makefile or a PDF generation script) issue the following command in the terminal:

```
pdflatex "\def\probopts{,test}\input{foo}"
```

4.2.3 (Open) Problems

The main environment provided by the `problem` package is (surprise surprise) the `sproblem` environment. It is used to mark up problems and exercises. The following example shows the main functionality:

Example 16

Input:

²EdNOTE: MK: document the attribute somewhere

⁶The current crop of language definition files is quite limited. Please feel free to contribute the to the localization effort

```

File tutorial/ext/simple-problem.en.tex
1 \begin{document}
2 \begin{sproblem}[id=prob.elefants,pts=10,min=2,title=Fitting Elefants]
3   How many Elefants can you fit into a Volkswagen beetle?
4   \begin{hint}
5     Think positively, this is simple!
6   \end{hint}
7   \begin{exnote}
8     Justify your answer
9   \end{exnote}
10  \begin{gnote}[title=deduct 5pts if justification is missing]
11    \anscls[feedback=you forget that elephants could be small]{none}
12    \anscls[feedback=you forget the back seats]{2}
13  \end{gnote}
14  \begin{solution}
15    Four, two in the front seats, and two in the back.

```

Output:

```

Exercise (Fitting Elefants)
How many Elefants can you fit into a Volkswagen beetle?

Lösung: Four, two in the front seats, and two in the back.

```

The `sproblem` environment takes an optional key/value argument with the keys `id` as an identifier that can be reference later, `pts` for the points to be gained from this exercise in homework or quiz situations, `min` for the estimated minutes needed to solve the problem, and finally `title` for an informative title of the problem.

The `sproblem` can contain several `solution` environments, which take the well-known `id`, `style`, and `title` attributes, and also the `testspace` and `answerclass`. The former

The additional functionality is specified in the `hint` `exnote` (notes in exercises), `solution`, and `gnote` (grading notes) environments. Here, the first three are shown whereas the grading notes are hidden, since the corresponding option was not given in the `\usepackage[...]{problem}`. All of these environments can occur any number of times in the `sproblem` environment. The `solution` environment takes an optional argument that is interpreted as the identifier.

4.2.4 Solutions and Answer Classes

Most problem libraries also contain (master) solutions, which show the intended way of solving a problem. The `solution` environment allows to specify solutions. It is only formatted if the `solutions` options is specified for the `problem` package.

We observe that many problems have more than one possible solution, so the `problem` package allows multiple `solution` environments; they can be distinguished by an `id` key in the optional first argument.

We also observe that in problem libraries we may want to keep track of classes of answers that have been observed e.g. in previous assessment situations, e.g. to provide

feedback. In this setting, the `solution` environment can be used to encode (typical representative of) **answer classes** (see also ??? below).

`solution` The `solution` environment takes an optional argument that allows the keys `id`, `title` and `style` keys that we have already seen above, further more

1. `testspace=`, which is equivalent to a `\testspace{}` (see subsection 4.2.10).
2. `answerclass=`, which gives the name of an answer class, this solution corresponds to.³

EdN:3

4.2.5 Grading Support

When problems are graded by multiple persons (e.g. for large university exams), then consistency of grading is a major issue. Both within a cohort and between cohorts e.g. in successive instances of a course.

Therefore keeping records of how to grade a problem (which problem is worth how many “points” and where to deduct how many points in which situations) and making them accessible to graders in a “master solution” – a version of an exam formatted to have the known solutions/answer classes (see above) and the grading specifications is a good practice. And it makes sense to include grading information into a problem library.

The simplest and arguably most effective measure is to specify the points that can be reached for a problem in the problem itself via the ‘pts=’ key (but see subsection 4.2.9)

The `problem` package also supplies the `gnotes` environment, which is only formatted when the `gnotes=` key is given. This environment takes an optional argument that allows the `id`, `style`, and `title` keys. The latter is used to give a short version of the grading specification. More structured grading support can be expressed in `\anscls` markup which we discuss next.

We have already encountered the notion of answer classes – classes of answers that are supposed equivalent in terms of learner’s competencies inscribed in them – above. These are the natural carriers of both grading feedback and points.

In the `gnote` environment we use the `\anscls` macro for specifying such information using meta-level descriptions rather than typical representatives. It takes an optional keyword argument for the grading/feedback information and a required one for the answer class description.

We distinguish two usages of `\anscls[...]{<desc>}`: in

- *answer classes* `<desc>` is a specification of a class of answers and the `pts=` key the points to be awarded to that.
- *answer traits* `<desc>` specifies the deviation from an assumed correct solution, and the value of the `pts=` key specifies changes to the points specified in the problem itself.

Consider for instance the following situation:

```
\begin{sproblem}[pts=3]
  Give an example for a foo and explain it in one sentence.
  \begin{gnote}[title=1 pt for the example and 2 for the explanation]
    \anscls[pts=0,feedback=I did not understand this]{complete gibberish}
    \anscls[pts=-2,feedback=You forgot the explanation]{no explanation}
  \end{gnote}
\end{sproblem}
```

³EdNOTE: This mechanism needs to be further worked out or deprecated; if we keep it we probably need to add `pts` and `feedback` attributes for parity with `\anscls`.

Here we have a problem that is worth three points, and the grading note explains how to treat deviations from a correct solutions – which is not given, since there may be hundreds of examples for foos. The `\anscls` specify this out for grading support systems⁷. The first is an answer class description for the class of answers that cannot be made sense of by the graders and specifies a default grade and feedback. The second specifies the trait of a missing explanation and specifies a deduction commensurate with the grading note title and a feedback to the learner. Note that a grading support system can distinguish answer classes from traits by the form of the value of points: absolute values vs. differences like +5 or -3.

4.2.6 Structured Problems

Problems can be structured into subproblems via the `subproblem` environment. It takes the same arguments and allows the same content model as `sproblem`.

Example 17

Input:

File tutorial/ext/structured-problem.en.tex

```

1 \begin{document}
2 \begin{sproblem}[id=prob.elephants-mod,title=Fitting Elephants Modularly]
3   Consider the problem of fitting elephants into a Volkswagen beetle.
4   \begin{subproblem}[pts=1,min=2]
5     Estimate the number of elephants on the front seats.
6     \begin{solution}
7       Two on each side one.
8     \end{solution}
9   \end{subproblem}
10  \begin{subproblem}[pts=1,min=2]
11    Estimate the number of elephants on the back seats.
12    \begin{solution}
13      Three little elephants.
14    \end{solution}
15  \end{subproblem}

```

Output:

⁷like e.g. <https://gitos.rrze.fau.de/yn06uhoc/vollkorn-prototype>

Exercise (Fitting Elephants Modularly)

Consider the problem of fitting elephants into a Volkswagen beetle.

1. Estimate the number of elephants on the front seats.

Lösung: Two on each side one.

2. Estimate the number of elephants on the back seats.

Lösung: Three little elephants.

3. What is the total number?

Lösung: A family of five; we do not want elephants in the frunk.

By default, subproblems are numbered subordinate to the “mother problem”. The `problem` package does not make any assumptions about whether subproblems can be used independently from their sister problems, or on order requirements.

Note that neither `sproblem` nor `subproblem` can be nested, if you want to have further structure, you need to resort to regular `LATEX` functionality. Note that you can add `solution` and `gnote` environments wherever you want, so that this need not force you to forego structure.

The `subproblem` environment is however very useful for modularizing problems: `subproblem` environments can appear outside of `sproblem` and in particular at the top-level of a file. This allows them to be shared between `sproblem` via `\inputref`.

4.2.7 Single/Multiple Choice Blocks

Multiple choice blocks can be formatted using the `mcb` environment, in which single choices are marked up with `\mcc` macro.

`\mcc[⟨keyvals⟩]{⟨text⟩}` takes an optional key/value argument `⟨keyvals⟩` for choice metadata and a required argument `⟨text⟩` for the proposed answer text. The following keys are supported

- `T` for correct answers, `F` (the default value) for false ones,
- `Ttext` the verdict for correct answers, `Ftext` for false ones, and
- `feedback` for a short feedback text given to the student.

What we see when this is formatted to PDF depends on the context. In solutions mode (we start the solutions in the code fragment below) we get

Example 18

Input:

```
1 \startsolutions
2 \begin{sproblem}[title=Functions,id=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def} \mcc[F,feedback=that is for C and C++] {function}
6     \mcc[F,feedback=that is for Standard ML]{fun}
7     \mcc[F,Ftext=Noooooooooooo,feedback=that is for Java]{public static void}
8   \end{mcb}
9 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☒ def
- ☐ function^a
- ☐ fun^b
- ☐ public static void^c

^a

that is for C and C++

^b

that is for Standard ML

^cNoooooooooooo

that is for Java

In “exam mode” where disable solutions (here via \stopsolutions) we get the questions without solutions (that is what the students see during the exam/quiz).

Example 19

Input:

```
1 \stopsolutions
2 \begin{sproblem}[title=Functions,id=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def} \mcc[F,feedback=that is for C and C++] {function}
6     \mcc[F,feedback=that is for Standard ML]{fun}
7     \mcc[F,Ftext=Noooooooooooo,feedback=that is for Java]{public static void}
8   \end{mcb}
9 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☐ def
- ☐ function
- ☐ fun
- ☐ public static void

What we have seen is the default rendering of the multiple choice block, which is as an itemize like environment with check boxes. There are alternatives to that which we can choose with the `style` key in the optional attribute of the `mcb` environment. Currently the only alternative is `inline` style:

Example 20

Input:

```
1 \startsolutions
2 \begin{sproblem}[title=Functions,id=functions1]
3   What is the keyword to introduce a function definition in python?
4
5   \begin{mcb}[style=inline]
6     \mcc[T]{def} \mcc[F,feedback=that is for C and C++] {function}
7     \mcc[F,feedback=that is for Standard ML] {fun}
8     \mcc[F,Ftext=Noooooooooooo,feedback=that is for Java] {public static void}
9   \end{mcb}
10 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☒ def ☐ function^a ☐ fun^b ☐ public static void^c

^a

that is for C and C++

^b

that is for Standard ML

^cNoooooooooooo

that is for Java

This is the solutions mode, i.e. with solutions, otherwise, the footnotes will fully disappear.

Analogously, single choice blocks are marked up by the `scc` environment and the individual choices by the `\scc` macro. In PDF, there is little difference to the multiple choice case. In interactive formats like HTML, single choice blocks are typically realized via radio buttons to enforce single choice.⁴

Often we only want to ask whether a statement is true or false. This is essentially a single choice block. $\text{\TeX}$ provides the macros `\yesTnoF`, `\yesFnoT`, `\trueTfalseF`, and `\trueFfalseT` for that. They are used as in the example below:

⁴EdNOTE: MK: are there any differences between `mcc` and `scc` we need to discuss here?

Example 21

Input:

```
1 \begin{sproblem}\startsolutions
2   Mark the following statements as true or false:
3   \begin{enumerate}
4     \item The moon is made of green cheese \trueFfalseT
5     \item \ldots
6   \end{enumerate}
7 \end{sproblem}
```

Output:

Exercise

Mark the following statements as true or false:

1. The moon is made of green cheese ☐ true ☒ false
2. ...

4.2.8 Filling-In Concrete Solutions

The next simplest situation, where we can implement auto-grading is the case where we have fill-in-the-blanks

The `\fillinsol` macro takes a single argument, which contains a concrete solution (i.e. a number, a string, ...), which generates a fill-in-box in test mode:

Example 22

Input:

```
1 \stopsolutions
2 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
3   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
4 \end{sproblem}
```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle?

and the actual solution in solutions mode:

Example 23

Input:

```

1 \startsolutions
2 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
3   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
4 \end{sproblem}

```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle? 4

If we do not want to leak information about the solution by the size of the blank we can also give `\fillinsol` an optional argument with a size: `\fillinsol[3cm]{12}` makes a box three cm wide.

Obviously, the required argument of `\fillinsol` can be used for auto-grading. For concrete data like numbers, this is immediate, for more complex data like strings “soft comparisons” might be in order. Indeed the default matching of answers against the string provided in the required argument of the `\fillinsol` is up to whitespace normalization.

The `problem` package allows to specify different behavior via three additional keys whose arguments are all triples of the form

```
{\langle target \rangle}{\langle TF \rangle}{\langle feedback \rangle}
```

where $\langle target \rangle$ is the target specification, $\langle TF \rangle$ is the verdict T or F on the correctness of an answer that is accepted by $\langle target \rangle$, and $\langle feedback \rangle$ a feedback string. In the following (somewhat contrived) example, we see all three at work:

Example 24

Input:

```

1 \begin{sproblem}
2   In 2019, Erlangen is a city of
3   \fillinsol[testsapce=3cm,
4   exact={111962}]{Wow, you known your numbers, according to
5   Wikipedia that is exactly correct},
6   numrange={0-1000}F{No,that would be a village},
7   numrange={1001-100000}F{No,that would be a town},
8   numrange={100000-120000}T{Yes, it is close to 109000},
9   numrange={1200001-}F{No, that is too large},
10  regex={-[0-9]+}F{What do negative Inhabitants even mean?},
11  regex={.*}F{Please give a natural number}]
12  {111962}
13  inhabitants.
14 \end{sproblem}

```

Output:

Exercise

In 2019, Erlangen is a city of 111962^a inhabitants.

^a

type	case	verdict	feedback
exact	111962	T	
exact	111962	T	Wow, you know your numbers, according to Wikipedia that is exactly correct
numrange	0-1000	F	No, that would be a village
numrange	1001-100000	F	No, that would be a town
numrange	100000-120000	T	Yes, it is close to 109000
numrange	1200001-	F	No, that is too large
regex	-[0-9]+	F	What do negative inhabitants even mean?
regex	.*	F	Please give a natural number

- `exact=` gives the exact string (no whitespace normalization).
- `numrange=` specifies a number range, i.e. a comma-separated list of pairs $\langle low \rangle$ - $\langle high \rangle$, where $\langle low \rangle$ and $\langle high \rangle$ are number representations or empty.
- `regex=` gives a POSIX regular expression that specifies all acceptable strings.

4.2.9 Including Problems

`\includeproblem` The `\includeproblem` macro can be used to include a problem from another file. It takes an optional `KeyVal` argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one problem in the include file). The keys `title`, `min`, and `pts` specify the problem title, the estimated minutes for solving the problem and the points to be gained, and their values (if given) overwrite the ones specified in the `problem` environment in the included file.

The sum of the points and estimated minutes (that we specified in the `pts` and `min` keys to the `problem` environment or the `\includeproblem` macro) to the log file and the screen after each run. This is useful in preparing exams, where we want to make sure that the students can indeed solve the problems in an allotted time period.

The `\min` and `\pts` macros allow to specify (i.e. to print to the margin) the distribution of time and reward to parts of a problem, if the `pts` and `pts` options are set. This allows to give students hints about the estimated time and the points to be awarded.

4.2.10 Testing and Spacing

The `problem` package is often used by the `hwexam` package, which is used to create homework assignments and exams. Both of these have a “test mode” (invoked by the package option `test`), where certain information –master solutions or feedback – is not shown in the presentation.

`\testspace` `\testspace` takes an argument that expands to a dimension, and leaves vertical space accordingly. Specific instances exist: `\testsmallspace`, `\testsmallspace`, `\testsmallspace` `\testsmallspace` give small (1cm), medium (2cm), and big (3cm) vertical space.

`\testsmallspace` `\testnewpage` makes a new page in `test` mode, and `\testemptypage` generates an empty page with the cautionary message that this page was intentionally left empty.

`\testemptypage`

TODO⁸

4.3 Homework Assignments and Exams

4.3.1 Introduction

The `hwexam` package and class supplies an infrastructure that allows to format nice-looking assignment sheets by simply including problems from problem files marked up with the `problem` package. It is designed to be compatible with `problems.sty`, and inherits some of the functionality.

4.3.2 Package Options

<code>solutions</code>	The <code>hwexam</code> package and class take the options <code>solutions</code> , <code>notes</code> , <code>hints</code> , <code>gnotes</code> , <code>pts</code> , <code>min</code> , and <code>boxed</code> that are just passed on to the <code>problems</code> package (cf. its documentation for a description of the intended behavior).
<code>notes</code>	
<code>hints</code>	
<code>gnotes</code>	
<code>pts</code>	
<code>min</code>	
<code>multiple</code>	Furthermore, the <code>hwexam</code> package takes the option <code>multiple</code> that allows to combine multiple assignment sheets into a compound document (the assignment sheets are treated as section, there is a table of contents, etc.).
<code>test</code>	Finally, there is the option <code>test</code> that modifies the behavior to facilitate formatting tests. Only in <code>test</code> mode, the macros <code>\testspace</code> , <code>\testnewpage</code> , and <code>\testemptypage</code> have an effect: they generate space for the students to solve the given problems. Thus they can be left in the L ^A T _E X source.

4.3.3 Assignments

<code>assignment</code> (<i>env.</i>)	This package supplies the <code>assignment</code> environment that groups problems into assignment sheets. It takes an optional KeyVal argument with the keys <code>number</code> (for the assignment number; if none is given, 1 is assumed as the default or — in multi-assignment documents — the ordinal of the <code>assignment</code> environment), <code>title</code> (for the assignment title; this is referenced in the title of the assignment sheet), <code>type</code> (for the assignment type; e.g. “quiz”, or “homework”), <code>given</code> (for the date the assignment was given), and <code>due</code> (for the date the assignment is due).
<code>number</code>	
<code>title</code>	
<code>type</code>	
<code>given</code>	
<code>due</code>	

4.3.4 Including Assignments

<code>\includeassignment</code>	The <code>\includeassignment</code> macro can be used to input an assignment from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one <code>assignment</code> environment in the included file). The keys <code>number</code> , <code>title</code> , <code>type</code> , <code>given</code> , and <code>due</code> are just as for the <code>assignment</code> environment and (if given) overwrite the ones specified in the <code>assignment</code> environment in the included file.
---	---

⁸TODO: check what is still undescribed `problem.sty` and make examples for it.

4.3.5 Typesetting Exams

`testheading` (*env.*) The `\testheading` takes an optional keyword argument where the keys `duration` specifies a string that specifies the duration of the test, `min` specifies the equivalent in number of minutes, and `reqpts` the points that are required for a perfect grade.

```
reqpts1 \title{320101 General Computer Science (Fall 2010)}
2 \begin{testheading}[duration=one hour,min=60,reqpts=27]
3   Good luck to all students!
4 \end{testheading}
```

Will result in

Name:

Matriculation Number:

320101 General Computer Science (Fall 2010)

2026-08-08

You have one hour (sharp) for the test;

Write the solutions to the sheet.

The estimated time for solving this exam is 23 minutes, leaving you 37 minutes for revising your exam.

You can reach 23 points if you solve all problems. You will only need 27 points for a perfect score, i.e. -4 points are bonus points.

Different problems test different skills and knowledge, so do not get stuck on one problem.

	To be used for grading, do not write here								
prob.	4.1	1.1	1.2	2.1	2.2	2.3	2.4	2.5	2.6
total	0	0	0	10	0	0	0	0	0
reached									

good luck

9

⁹MK: The first three “problems” come from the stex examples above, how do we get rid of this?

Part II

User Manual

*The dynamic [HTML](https://mathhub.info/?a=sTeX/Documentation&d=manual&l=en) version of this part can be found at
`https://mathhub.info/?a=sTeX/Documentation&d=manual&l=en`*

Chapter 5

Basics

5.1 Package and Class Options

- `debug=<prefixes>`: (see Developer Manual)
- `lang=<languages>`: If set, [sTeX](#) will load the `babel` package with the provided languages. Supported languages (currently) are:

<code>ar</code>	arabic
<code>bg</code>	bulgarian
<code>de</code>	german (with option <code>ngerman</code>)
<code>en</code>	english
<code>fi</code>	finnish
<code>fr</code>	french
<code>ro</code>	romanian
<code>ru</code>	russian
<code>tr</code>	turkish (with option <code>shorthands=!</code>)

- `mathhub=<path>`: Uses the provided file path as `MathHub` directory (see Section 0.1 ([Math Archives](#) and the `MathHub` Directory) in the [sTeX](#) Manual).
- `usesms/writesms`: If `writesms` is set, content loaded from external [math archives](#) (i.e. [modules](#)) is persisted in the file `\jobname.sms`.

If `usesms` is set, the content of the `.sms`-file is loaded, obviating the need to reprocess the original files.

The options are not mutually exclusive, but care should be taken if dependencies have changed between builds.

This offers two advantages:

1. If a document has many (transitive) dependencies, `usesms` should significantly speed up the build process, and
2. setting `usesms` allows for distributing the `.sms`-file to make the document *standalone*, allowing for compilation without needing imported/used modules to be present.

The options `debug`, `mathhub`, `usesms` and `writesms` can also be set by the environment variables `STEX_DEBUG`, `MATHHUB`, `STEX_USESMS` and `STEX_WRITESMS`. In fact, the `MATHHUB` environment variable is the recommended way to set the MathHub directory. This is the only option where the *package option* overrides the environment variable.

The environment variables for `USE/WRITESMS` are particularly useful, in that they allow for convenient compilation workflows. For example, the Build PDF/XHTML/OMDoc-button in the IDE does the following:

```
STEX_WRITESMS=true pdflatex <job>.tex
[bibtex|biber] <job>
STEX_USESMS=true pdflatex <job>.tex
STEX_USESMS=true pdflatex <job>.tex
```

Guaranteeing (in the first run) that all dependencies are loaded from their respective sources and persisted, and in the two subsequent runs read from the generated `.sms` file, (likely) speeding up the subsequent runs significantly.

5.2 Math Archives and the MathHub Directory

`STEX` uses [math archives](#) to organize document content modularly, without a user having to specify absolute paths, which would differ between users and machines.

All `STEX` archives need to exist in the local MathHub-directory. `STEX` knows where this folder is via one of five means:

1. If the `STEX` package is loaded with the option `mathhub=/path/to/mathhub`, then `STEX` will consider `/path/to/mathhub` as the local MathHub directory.
2. If the `mathhub` package option is *not* set, but the macro `\mathhub` exists when the `STEX`-package is loaded, then this macro is assumed to point to the local MathHub directory; i.e. `\def\mathhub{/path/to/mathhub}\usepackage{stex}` will set the MathHub directory as `path/to/mathhub`.
3. Otherwise, `STEX` will attempt to retrieve the system variable `MATHHUB`, assuming it will point to the local MathHub directory. Since this variant needs setting up only *once* and is machine-specific (rather than defined in tex code), it is compatible with collaborating and sharing tex content, and hence recommended.
4. If that too fails, `STEX` will look for a file `~/.stex/mathhub.path`. If this file exists, `STEX` will assume that it contains the path to the local MathHub-directory. This method is recommended on systems where it is difficult to set environment variables, and is used by the IDE setup.
5. Finally, if all else fails, `STEX` considers `~/MathHub` to be the MathHub directory.

The `STEX` IDE allows you to directly download [math archives](#) from `gl.mathhub.info` – currently available [archives](#) there are:

- `sTeX/*` – a group of semi-experimental documents showcasing `STEX`3.3 features,
- `smglom/*` – a vast collection of multilingual [modules](#) of concepts in mathematics and computer science. The SMGloM predates `STEX`3 and is thus largely “underannotated” with respect to (formal) semantics,
- `courses/*` – a vast collection of lecture slides and notes in computer science for courses held by Michael Kohlhase. They largely make use of SMGloM [modules](#).

5.2.1 The Structure of Math Archives

An **archive** group/name is stored in the directory <MathHub>/group/name; e.g. assuming your local MathHub-directory is set as /user/foo/MathHub, then for the sTeX/Documentation **archive** to be found by the sTeX system, it needs to be in /user/foo/MathHub/sTeX/Documentation.

Each such **archive** needs two subdirectories:

- /source – this is where all your tex files go.
- /META-INF – a directory containing a single file MANIFEST.MF, the content of which we will consider shortly.

An additional lib-directory is optional, and is discussed in section 5.3.

5.2.2 MANIFEST.MF-Files

The MANIFEST.MF in the META-INF directory consists of key-value-pairs, informing sTeX (and associated software, e.g. MMT) of various properties of an **archive**. For example, the MANIFEST.MF of the sTeX/Documentation **archive** looks like this:

```
id: sTeX/Documentation
ns: http://mathhub.info/sTeX/Documentation
narration-base: http://mathhub.info/sTeX/Documentation
format: stex
title: The sTeX Documentation
teaser: The full Documentation for the sTeX system
url-base: https://mathhub.info
dependencies:sTeX/ComputerScience/Software,sTeX/MathTutorial
ignore: */code/*|*/tikz/*|*/tutorial/solution/*
```

Many of these are in fact ignored by sTeX, but some are important:

id: The name of the **archive**, including its group (e.g. sTeX/Document). This is used by the MMT system in favor of the directory, but T_EX's limited access to the file system enforces the directory structure.

source-base or

ns: The namespace from which all **symbol** and **module** MMT-URIs in this **archive** are formed.

narration-base: The namespace from which all document MMT-URI in this repository are formed. It can safely match the **ns**-field.

url-base: A URL that is formed as a basis for *external references*. and hyperlinks. An MMT (or comparable system) instance should run there and host (sTeX-generated) HTML.

dependencies: All **archives** that this **archive** depends on. sTeX ignores this field, but MMT can pick up on them to resolve dependencies, e.g. when downloading **archives** in the IDE, which will also download all dependencies.

ignore: A regular expression of .tex files in the **source** directory that should be ignored; e.g. they will not be compiled when building a whole directory or archive in the IDE.

5.3 The lib-Directory

A **math archive** group/archive may have a **lib** directory primarily intended for preamble code, **packages**, **.bib** files, etc., the files in which can be referenced in various ways.

Additionally, a *group* of archives **group** may have an additional **archive group/meta-inf**. If this **meta-inf** archive has a **/lib**-subdirectory, they too will be considered by the following.

\libinput **\libinput** {some/file} searches for a file **some/file[.tex]** in

- the **lib**-directory of the current archive, and
- the **lib**-directory of a **meta-inf**-archive in (any of) the archive groups containing the current archive

and **\inputs** all found files in reverse order; e.g. **\libinput**{preamble} in a **.tex**-file in **sTeX/Documentation** will *first* input **.../sTeX/meta-inf/lib/preamble.tex** and then **.../sTeX/Documentation/lib/preamble.tex**.

\libinput will throw an error if *no* candidate for **some/file** is found.

\libinput[some/archive]{some/file} will do the same, but starting in the **lib** directory of the **math archive** **some/archive**.

\libusepackage **\libusepackage** [package-options]{some/file} searches for a file **some/file.sty** in the same way that **\libinput** does, but will call **\usepackage**[package-options]{path/to/some/file} instead of **\input**.
\libusepackage throws an error if not *exactly one* candidate for **some/file.sty** is found.

\addmhbibresource **\addmhbibresource** [some/archive]{some/file} searches for a file like **some/file.bib** in **some/archive**'s **lib** directory and calls **\addbibresource** to the result.

\libusetikzlibrary **\libusetikzlibrary** behaves like **\libusepackage** but looks specifically for **tikz** libraries and calls **\usetikzlibrary** on the results.

throws an error if not *exactly one* candidate for the library is found.

A good practice is to have individual **sTeX** fragments follow basically this document frame:

```
\documentclass{stex}
\libinput{preamble}
\setsectionlevel{<your preference>}
\begin{document}
  \IfInputref{}{
    ...
    \maketitle
    \ifstexhtml \else \tableofcontents \fi
  }
  ...
  \IfInputref{}{\libinput{postamble}}
\end{document}
```

Then the `preamble.tex` files can take care of loading the generally required [packages](#), setting presentation customizations etc. (per archive or archive group or both), and a `postamble.tex` can e.g. print the bibliography, index etc.

`\libusepackage` is particularly useful in such a `preamble.tex` when we want to use custom packages that are not part of a [TeX](#) distribution, or on CTAN. In this case we commit the respective packages in one of the `lib` folders and use `\libusepackage` to load them.

5.4 Basic Macros

<code>\sTeX</code> <code>\stex</code>	The <code>\sTeX</code> macro produces this $\text{\texttt{sTeX}}$ logo. It is provided by the <code>stex-logo</code> package , included with the <code>stex</code> package .
--	--

<code>\ifstexhtml</code>	The TeX conditional <code>\ifstexhtml</code> is <i>true</i> if the current compilation generates HTML , and <i>false</i> otherwise (i.e. generates PDF).
--------------------------	---

<code>\STEXinvisible</code>	<code>\STEXinvisible{<code>}</code> Processes <code><code></code> , but does not generate any output. In the HTML , <code><code></code> is exported with <code>display:none</code> . Can be used to declare formal content and preserve its semantics in HTML without generating output.
-----------------------------	--

Chapter 6

Document Features

6.1 Document Fragments

`sfragment` (*env.*) To make reusability of document fragments more feasible, `TeX` provides the `sfragment` environment. `\begin{sfragment}[id=<id>,short=<short title>]{section title}` calls `\part`, `\chapter`, `\section`, `\subsection`, `\subsubsection`, `\paragraph` or `\subparagraph` with argument `{section title}` depending on the current *section level* and availability, and increases the level accordingly.

The `<id>` can be used for cross-document references (see Section 2.3 (Cross-Document References) in the `TeX` Manual).

`blindfragment` (*env.*) In the case where we want to increase the section level *without* producing a corresponding section header, the `blindfragment` environment can be used. This allows e.g. typesetting `\sections` before the first `\chapter`.

`\skipfragment` The `\skipfragment` macro “skips an `sfragment`”, i.e. it just steps the respective sectioning counter. This macro is useful, when we want to keep two documents in sync structurally, so that section numbers match up: Any section that is left out in one becomes a `\skipfragment`.

`\setsectionlevel` The `\setsectionlevel` macro sets the current section level to that provided as argument. This is particularly useful in the preamble of a document, as to be ignored in e.g. `\inputref` and make sure that sectioning proceeds as desired; e.g. `\setsectionlevel{section}` make sure that the first `sfragment` will be typeset as a `\section` (rather than e.g. a `\part`).

`\currentsectionlevel`
`\Currentsectionlevel` The `\currentsectionlevel` macro produces the literal string corresponding to the current section level – e.g. within a chapter (but outside of a section), `\currentsectionlevel` produces “chapter”.

The `\Currentsectionlevel` macro does the same, but capitalizes the first letter; e.g. in the above situation, `\Currentsectionlevel` produces “Chapter”.

6.2 Using and Referencing Document Fragments

`\inputref` `\inputref` [`\archive`]{`\file`}

Inputs the file `\file` in `\archive`'s `source` directory. If [`\archive`] is empty, the current `archive`'s `source` directory is used. If there is no current `archive`, `\file` is resolved relative to the current file.

The file's content is processed within a `TeX` group when using `pdflatex`. When converting to `HTML` however, the file is not processed *at all*, and instead, a reference to the file is inserted, that can be replaced by the `HTML` generated by the referenced file by e.g. the `MMT` system.

This is the recommended method to assemble documents from individual `.tex` files.

`\mhinput` Like `\inputref`, but actually calls `\input` in both `PDF` and `HTML` mode. Useful for small fragments or those without `modules`, but generally `\inputref` should be preferred.

`\ifinputref` `\ifinputref` is a `TeX` conditional for whether the current file is currently processed via
`\IfInputref` `\inputref`.

`\IfInputref` {`\true code`}{`\false code`} behaves like

`\ifinputref`{`\true code`}\else{`\false code`}\fi when using `pdflatex`; in `HTML` mode however, *both* arguments are processed and marked-up accordingly, so a hosting server (like `MMT`) can dynamically decide which parts to show or omit.

`\mhgraphics` If the `graphicx` package is loaded, `\mhgraphics` takes the same arguments as `\includegraphics`,
`\cmhgraphics` with the additional optional key `archive`. It then resolves the file path in

`\mhgraphics`[`archive=some/archive`]{`some/image`} relative to the `source`-folder of the `some/archive` `archive`. If no `archive` is provided, the file path `some/image` is resolved relative to the current `archive` (if existent).

`\cmhgraphics` additionally wraps the image in a `center`-environment.

`\lstinputmhlisting` Like `\mhgraphics`, but for `\lstinputlisting` instead of `\includegraphics`. Only de-
`\clstinputmhlisting` fined if the `listings` package is loaded.

6.3 Cross-Document References

If we take features like `\inputref` and `\mhinput` (and the `sfragment` environment) seriously and build large documents modularly from individually compiling documents for sections, chapters and so on, cross-referencing becomes an interesting problem.

Say, we have a document `main.tex`, which `\inputrefs` a section `section1.tex`, which references a definition with label `some_definition` in `section2.tex` (subsequently also `\inputrefed` in `main.tex`). Then the numbering of the definition will depend on the *document context* in which the document fragment `section2.tex` occurs - in `section2.tex` itself (as a standalone document), it might be *Definition 1*, in `main.tex`

it might be *Definition 3.1*, and in `section1.tex`, the definition *does not even occur*, so it needs to be referenced by some other text.

What we would want in that instance is an equivalent of `\autoref`, that takes the document context into account to yield something like *Definition 1*, *Definition 3.1* or “*Definition 1 in the section on Foo*” respectively.

For that to work, we need to supply (up to) three pieces of information:

- The *label* of the reference target (e.g. `some_definition`),
- (optionally) the *file*/document containing the reference target (e.g. `section2`). This is not strictly necessary if the reference target occurs in the *same* document, but if not, we need to know where to find the label,
- (optionally) the document context, in which we want to refer to the reference target (e.g. `main`).

Additionally, the document in which we want to reference a label needs a title for external references.

```
\sref \sref [archive=<archive1>,file=<file1>]
      {\<label>}[archive=<archive2>,file=<file2>,title=<title>]
```

This `macro` references `<label>` (declared in `<file1>` in `math archive <archive1>`). If the object (section, figure, etc.) with that label occurs (eventually) in the same document, `\sref` will ignore the second set of optional arguments and simply defer to `\autoref` if that command exists, or `\ref` if the `hyperref` package is not included.

If the referenced object does *not* occur in the current document however, `\sref` will refer to it by the object’s name as it occurs in the file `<file2>` in `archive <archive2>`, followed by the title.

In `HTML` mode, the reference additionally links to the `HTML` of the `file1`.¹⁰

This works by storing labels during compilation in a file `<jobname>.sref`, analogous to e.g. the `.toc`. Note that this consequently requires both `file1.tex` and `file2.tex` to have been compiled previously, to generate the `.sref` file.

For example, doing

```
\sref[file=tutorial/full.en]{sec:basics}[file=tutorial.en,title=the \stex Tutorial]
```

in this very document fragment (`[sTeX/Documentation]macros/sref.en.tex`) will yield [Part I \(The Basics\) in the sTeX Tutorial](#) if compiled itself, or if compiled as part of the `sTeX` manual, and will yield the `\autoref` link [chapter 2](#) in the documentation (which includes the tutorial).

```
\srefsetin \srefsetin [<archive2>]{<file2>}{<title>}
```

Sets a default value for the optional arguments `<archive2>`, `<file2>` and `<title>` of `\sref`. If the second set of optional arguments in `\sref` are omitted, these default values are used. Particularly useful to set in a preamble.

```
\sreflabel \sreflabel {\<label>} sets a label analogous to \label{\<label>}, but for use in \sref.
```

Note that for every `sTeX macro` or `environment` that takes an optional `id=<id>` argument, the `<id>` (if non-empty) generates an `\sreflabel` automatically.

For example, `\begin{sfragment}[id=foo]{Foo}` is equivalent to `\begin{sfragment}{Foo}\sreflabel{foo}`.

`\extref` `\extref` [archive=<archive1>,file=<file1>]
`{<label>}{archive=<archive2>,file=<file2>,title=<title>}`

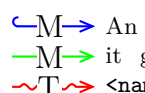
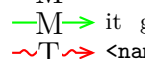
Like `\sref`, but with the third argument mandatory, `\extref` will *always* produce the output as if `<label>` would *not* occur in the current document.

Chapter 7

Modules and Symbols

7.1 Modules

A `module` is required to declare any new formal content such as `symbols` or `notations` (but not `variables`, which may be introduced anywhere).

 An `sTeX` `module` corresponds to an `MMT/OMDoc` *theory*. As such  it gets assigned an `MMT-URI` (*universal resource identifier*) of the form `<namespace>?<module-name>`.

`smodule` (*env.*) A new module is declared using the basic syntax

`\begin{smodule}[options]{ModuleName}...\end{smodule}`.

A module is required to declare any new formal content such as `symbols` or `notations` (but not `variables`, which may be introduced anywhere).

The `smodule`-environment takes several keyword arguments, all of which are optional:

`title` (*<token list>*) to display in customizations.

`style` (*<string>**) for use in customizations, see Part 1 (Customizing Typesetting) in the `sTeX` Manual

`id` (*<string>*) for cross-referencing, see `\sreflabel`.

`ns` (*<URI>*) the namespace to use. *Should not be used, unless you know precisely what you're doing.* If not explicitly set, is computed from the containing file and `archive`'s namespace.

`lang` (*<language>*) if not set, computed from the current file name (e.g. `foo.en.tex`).

`sig` (*<language>*) see below.

`\STEXexport` `\STEXexport{<code>}` executes *<code>* immediately and every time the current `module` is being used.



For technical reasons, `\STEXexport` processes its content in the `expl3` category code scheme – what this means is that all spaces are ignored entirely, and the characters `_` and `:` are valid characters in `macro` names.

In practice, this means you will have to use the `~` character for spaces, and if you want to use a subscript `_`, you should use the `macro` `\c_math_subscript_token` instead.

Also, note that no *global* `macro` definitions should happen in `\STEXexport`; this can lead to unexpected behaviour if the containing `module` has been used previously in the current document.

7.1.1 Signature `Modules`, Languages, and Multilinguality

if the current file is a translation of a file with the same base name but a different language suffix, setting `sig=<lang>` will preload the `module` from that language file. This helps ensuring that the (formal) content of both `modules` is (almost) identical across languages and avoids duplication.

For example, we can have a file `Foo.en.tex`, that declares and documents a `module` `Foo` (using `\begin{smodule}{Foo}`). If we put a file `Foo.de.tex` next to it, we can do `\begin{smodule}[sig=en]{Foo}` to have all the content in the `module` `Foo` (as declared in `Foo.en.tex`) available and translate its document content to german.

The `MMT` back-end, when serving `STEX` content as `HTML`, will always attempt to find documentation in the language corresponding to the context; e.g. a user's preference.

7.2 Symbol Declarations

`\symdecl` `\symdecl` `{<mname>}[<options>]`

The `\symdecl` `macro` is the simplest way to introduce a new `symbol`. If `<options>` contains `name=<name>`, then `<name>` is the *name* of the `symbol`; otherwise, `<mname>` is used for the *name*. Additionally, a `semantic macro` `\mname` is generated.

The starred variant `\symdecl*` does not generate a `semantic macro`, in which case the `name`-option is superfluous.

`↪M` `\symdecl` introduces a new `MMT/OMDOC constant` in the current `module` (i.e. `↪M` `MMT/OMDOC theory`). Correspondingly, they get assigned the `MMT-URI` `↪T` `<module-URI>?<constant-name>`.

`\symdecl` takes the following optional arguments:

`name` see above,

`args` the arity of the `symbol` and its `semantic macro`; may be a number 0...9 or a string consisting of the characters `i`, `a`, `b` and `B` of length ≤ 9 ,

`type` the `symbol`'s `type`,

`def` the `symbol`'s definiens,

`return` the [symbol](#)'s *return code* (see below), most commonly the [semantic macro](#) of a [mathematical structure](#),
`assoc` how to resolve arguments of [mode](#) `a` or `B`; may be `pre`, `bin`, `binl`, `binr` or `conj`,
`reorder` how to reorder the arguments in [OMDoc](#) (*advanced*),
`role` [symbols](#) with certain roles are treated in particular ways in [MMT/OMDoc](#) (*advanced*),
`argtypes` [TODO](#)¹¹.

`\textsymdecl` [\textsymdecl](#){*mname*}[*options*]{*code*}

Like [\symdecl](#), but requires that the [symbol](#) has arity 0 (hence [\textsymdecl](#) does not take the `args`-option), and generates a [semantic macro](#) that takes no arguments in either text or math mode, and produces marked-up *code* as output.

Additionally, a [macro](#) [\mname](#)name is generated that produces *code* without any semantic markup.

`\symdef` [\symdef](#){*mname*}[*options*]{*notation*}

Combines the functionalities and optional arguments of [\symdecl](#) and [\notation](#) in one.

7.2.1 Returns

Assume we have a [symbol](#) `foo` with [semantic macro](#) `\foo`, (exemplary) taking two arguments, and `return=<code>`. If we do `\foo{a}{b}!`, the return code is simply ignored. If we do `\foo{a}{b}` *without* the `!`, here is what happens:

1. [S_{TE}X](#) will replace `#1` and `#2` in *code* by `a` and `b`, yielding *retcode*.
2. [S_{TE}X](#) will insert `<retcode>{\foo{a}{b}!}` in the input token stream.

This means that *code* should contain at most *arity of foo* argument markers, and eat precisely one argument appended to *code*.

When in doubt, we recommend only using [semantic macros](#) for [mathematical structures](#) (with only optional arguments) and `\apply` (with only optional arguments) in `return`.

7.3 Referencing Symbols

`\symref` [\symref](#){*symbol*}{*text*}

`\sr` The [\symref](#) macro (and its short version [\sr](#)) is the most general variant to mark-up arbitrary [L^AT_EX](#) code *text* with the [symbol](#).

¹¹[TODO: experimental](#)



This is as good a place as any other to explain how $\text{\texttt{S\TeX}}$ resolves a string `symbol` to an actual `symbol`.

If `\symbol` is a `semantic macro`, then $\text{\texttt{S\TeX}}$ has no trouble resolving `symbolname` to the full `MMT-URI` of the `symbol` that is being invoked.

However, especially in `\symname` (or if a `symbol` was introduced using `\symdecl*` without generating a `semantic macro`), we might prefer to use the *name* of a symbol directly for readability – e.g. we would want to write `A \symname{natural number} is...` rather than `A \symname{Nat} is...`. $\text{\texttt{S\TeX}}$ attempts to handle this case thusly:

If `symbol` does *not* correspond to a `semantic macro` `\symbol` and does *not* contain a `?`, then $\text{\texttt{S\TeX}}$ checks all `symbols` currently in scope until it finds one, whose name is `symbol`. If `symbol` is of the form `pre?name`, $\text{\texttt{S\TeX}}$ first looks through all `modules` currently in scope, whose full `MMT-URI` ends with `pre`, and then looks for a symbol with name `name` in those. This allows for disambiguating more precisely, e.g. by saying `\symname{Integers?addition}` or `\symname{RealNumbers?addition}` in the case where several `additions` are in scope.

$\text{\texttt{M\TeX}}$ $\text{\texttt{M\TeX}}$ $\text{\texttt{T\TeX}}$ `\symref{<symbol>}{<text>}` in `MMT/OMDoc` generates the term `<OMS name="{<symbol URI>}" />`.

`\symname` `\symname[pre=<pre>,post=<post>]{<symbol>}`

`\sn` If the `symbol` referenced by `<symbol>` has name `name`, this is a shortcut for

`\Symname` `\symref{<symbol>}{<pre>name<post>}`.

`\Sn` For example, given a symbol `agroup` with name `abelian group`, we can do

`\sns` `\symname[pre=Non-,post=s]{agroup}` to produce `Non-abelian groups`.

`\Sns` `\sn` is a shorter variant for `\symname`; `\Symname` and `\Sn` additionally capitalize the first letter. `\sns` and `\Sns` are short for `\sn [post=s]` and `\Sn [post=s]`, respectively.

`\srefsym` `\srefsym{<symbol>}{<text>}`

`\srefsymuri` turns `<text>` into a link to

- The documentation of `<symbol>`, if it occurs in the same document, or
- the `symbol`'s documentation online, based on the containing `math archive`'s `url-base`.

`\srefsymuri` does the same, but expects a `symbol`'s full `MMT-URI` as first argument. This is particularly useful for e.g. customizing highlighting (see chapter 9).

`\symuse` `\symuse{<symbol>}` behaves exactly like a `semantic macro` for `<symbol>`.

7.4 Notations and Semantic Macros

`\notation` `\notation{<symbol>}[<options>]{<code>}`

introduces a new notation for the referenced symbol.

The starred variant `\notation*` sets this notation as the (new) default notation.

The optional arguments are:

- `prec=<opprec>;<argprec 1>x...x<argprec n>`: An operator precedence and one argument precedence for each argument of the semantic macro. If no argument precedences are given, all argument precedences are equal to the operator precedence. By default, all precedences are 0, unless the symbol takes no argument, in which case the operator precedence is `\neginfprec` (negative infinity).
`prec=nobrackets` is an abbreviation for `prec=\neginfprec;\infprec x...x\infprec`.
- `op=<code>`: An operator notation. If none is given, the notation component marked with `\maincomp` is used. If no `\maincomp` occurs in the notation, the default operator notation is `\symname{<symbol>}`.
- `variant=<id>`: An id for this notation. The key `variant=` can be omitted; i.e. `\notation[foo]` is equivalent to `\notation[variant=foo]`.

`\comp` `\comp` is used to mark notation components in a notation to be highlighted. Additionally, each notation can use `\maincomp` at most once to mark the primary notation component.



Ideally, `\comp` would not be necessary: Everything in a notation that is *not* an argument should be a notation component. Unfortunately, it is computationally expensive to determine where an argument begins and ends, and the argument markers `#n` may themselves be nested in other macro applications or `TEX` groups, making it ultimately almost impossible to determine them automatically while also remaining compatible with arbitrary highlighting customizations (such as tooltips, hyperlinks, colors) that users might employ, and that are ultimately invoked by `\comp`.



Note that it is required that

1. the argument markers `#n` never occur inside a `\comp`, and
2. no semantic macros may ever occur inside a notation.

Both criteria are not just required for technical reasons, but conceptionally meaningful:

The underlying principle is that the arguments to a semantic macro represent *arguments to the mathematical operation* represented by a symbol. For example, a semantic macro application `\plus{a}{b}` would represent *the actual addition of (mathematical objects) a and b*. It should therefore be impossible for *a* or *b* to be part of a notation component of `\plus`.

Similarly, a semantic macro can not conceptually be part of the notation of `\plus`,



since a **symbol** represents a *distinct (mathematical) concept with its own semantics*, and **notations** are syntactic representations of the very **symbol** to which the **notation** belongs.

If you want an argument to a **semantic macro** to be a purely syntactic parameter, then you are likely somewhat confused with respect to the distinction between the precise *syntax* and *semantics* of the **symbol** you are trying to declare (which happens quite often even to experienced **TeX** users, like us), and might want to give those another thought - quite likely, the concept you aim to implement does not actually represent a semantically meaningful (mathematical) concept, and you will want to use `\def` and similar native **LaTeX macro** definitions rather than **semantic macros**.

`\setnotation` The first **notation** provided will stay the default **notation** unless explicitly changed:
`\setnotation{\langle symbol \rangle}{\langle id \rangle}` sets the default **notation** of $\langle symbol \rangle$ to that with id $\langle id \rangle$.

7.4.1 Precedences and Bracketing

`\infprec` `\neginfprec` and `\neginfprec` represent *infinitely large* and *infinitely small* **precedences**, respectively.



TeX decides whether to insert parentheses by comparing **operator precedences** to a *downward precedence* p_d with initial value `\infprec`. When encountering a **semantic macro**, **TeX** takes the **operator precedence** p_{op} of the **notation** used and checks whether $p_{op} > p_d$. If so, **TeX** inserts parentheses.

When **TeX** steps into an argument of a **semantic macro**, it sets p_d to the respective **argument precedence** of the **notation** used.

Example 25

Consider **semantic macros** `\plus` and `\mult` taking two arguments, with **notations** $a+b$ and $a \cdot b$ respectively, and **precedences** 100 for `\plus` and 50 for `\mult`.

Consider $\mathcal{\$}\plus\{a,\mult\{b,\plus\{c,d\}\}\mathcal{\$}$ (i.e. $a + b \cdot (c + d)$). Then:

1. **TeX** starts out with $p_d = \text{\code{\infprec}}$.
2. **TeX** encounters `\plus` with $p_{op} = 100$. Since $100 \not> \text{\code{\infprec}}$, it inserts no parentheses.
3. Next, **TeX** encounters the two arguments for `\plus`. Both have no specifically provided **argument precedence**, so **TeX** uses $p_d = p_{op} = 100$ for both and recurses.
4. Next, **TeX** encounters `\mult\{b, \dots\}`, whose **notation** has $p_{op} = 50$.
5. We compare to the current downward **precedence** p_d set by `\plus`, arriving at $p_{op} = 50 \not> 100 = p_d$, so **TeX** again inserts no parentheses.

6. Since the notation of `\mult` has no explicitly set argument precedences, `\TeX` again uses the operator precedence for the arguments of `\mult`, hence sets $p_d = p_{op} = 50$ and recurses.
7. Next, `\TeX` encounters the inner `\plus{c, \dots}` whose notation has $p_{op} = 100$. We compare to the current downward precedence p_d set by `\mult`, arriving at $p_{op} = 100 > 50 = p_d$ – which finally prompts `\TeX` to insert parentheses, and we proceed as before.

`\dobracket` `\dobracket{\langle code \rangle}` wraps parentheses around `\langle code \rangle`.

`\withbrackets` `\withbrackets{\langle left \rangle}{\langle right \rangle}{\langle code \rangle}` uses the opening and closing parentheses `\langle left \rangle` and `\langle right \rangle` for the next pair of parentheses automatically inserted in `\langle code \rangle`.

7.4.2 Notations for Argument Sequences

The following macros can be used in notations that take mode `a` or `B` arguments:

`\argsep` `\argsep{\langle parameter token \rangle}{\langle separator \rangle}`

takes the elements of the argument sequence in position `\langle parameter token \rangle` and separates them by `\langle separator \rangle`.

Note that the first argument *must* be a parameter token of the form `\#k`, and the argument at position `k` of the notation has to have argument mode `a` or `B`.

`\argmap` `\argmap{\langle parameter token \rangle}{\langle code \rangle}{\langle separator \rangle}`

takes the elements of the argument sequence in position `\langle parameter token \rangle`, applies the code `\langle code \rangle` to each of them (which therefore should use `\##1`) and separates them by `\langle separator \rangle`.

For example, the notation `\argmap{\#1}{X^{\##1}}{++}` applied to the argument `\{a,b,c\}` produces $X^a + X^b + X^c$.

`\argarraymap` **TODO**¹²

7.4.3 Semantic Macros

Assume we have a semantic macro `\smacro` taking (exemplary) two arguments. The precise behaviour of `\smacro` depends on whether we are in text or math mode.

Math Mode `\smacro!` produces the default operator notation of its symbol. Without `!`, `\smacro` expects at least two arguments, and `\smacro{a}{b}!` produces the default notation of its symbol.

If the symbol has a return code, then `\smacro{a}{b}` continues with executing the return code. Otherwise, `\smacro{a}{b}` also simply produces the default operator notation.

The starred variants `\smacro*` and `\smacro!*` behave as in *text mode*.

Text Mode `\smacro!\{<arg>\}` marks up `<arg>` similarly to how `\symref{smacro}\{<arg>\}` would.

Without the `!`, `\smacro` still only takes a single argument, but it is expected, that within `<arg>`, the arguments for the `symbol` are explicitly marked up. The `\comp macro` is allowed in `<arg>` to determine the components of `<arg>` to be highlighted.

\arg The `\arg` macro can be used to explicitly mark the arguments of a `semantic macro` in text mode.

By default, they are numbered consecutively; e.g. `\smacro{... \arg{a}... \arg{b}}` determines `a` and `b` to be the (first and second) arguments.

The starred variant `\arg*` allows for marking up the arguments, but does not produce any output. This can be used to provide arguments that are not mentioned in the text we want to mark up, because they are implicitly obvious or mentioned elsewhere.

If we want to change the order of the arguments, we can provide the precise argument number as an optional argument; e.g. `\smacro{... \arg[2]{a}... \arg[1]{b}}` determines `b` to be the first and `a` to be the second argument.

An argument number may be used repeatedly, if the corresponding `argument mode` is `a` or `B`.

Applications of `semantic macros` with arguments are translated to $\textcolor{blue}{\text{M}} \rightarrow \textcolor{teal}{\text{MMT/OMDOC}}$ as OMA-terms with head `<OMS name="<symbol>" />`, or $\textcolor{green}{\text{M}} \rightarrow \textcolor{teal}{\text{<OMBIND name="<symbol>" />}}$, depending on the absence or presence of `argument mode b` or `B` arguments.

Semantic macros with no arguments or invoked with `!` correspond to OMS directly.

7.5 Simple Inheritance

There are three `macros` that allow for opening a `module`, making its contents available for use:

\usemodule `\usemodule{<module>}` is allowed anywhere and makes the `module`'s contents available up to the current `TeX` group. This is the right `macro` to use outside of `modules`, or when none of its contents use any of the used `module`'s `symbols` directly (e.g. in `types` or `definientia`).

\requiremodule `\requiremodule{<module>}` is only allowed in `modules` and makes the required `module`'s contents available within the current `module`. The imported `symbols` can be safely used in `types` and `definientia`, but not in the `return` code of `symbols`, and the imported content is not exported further – i.e. using the current `module` does not also open the required `module`.

`\importmodule` `\importmodule{\langle module \rangle}` is only allowed in `modules` and makes the required `module`'s contents available within the current `module`. The imported `symbols` can be safely used anywhere, and the imported content exported to any `modules` subsequently importing the current one.

$\hookrightarrow$ In `MMT`, every *document* and every `module` induces an `MMT` theory. `\usemodule`
 $\rightarrow$ induces and `MMT` `include` in the document *theory*, `\importmodule` and
 $\rightsquigarrow$ `\requiremodule` both induce an `include` in the `module`'s *theory*.

It is worth going into some detail how exactly `\usemodule`, `\importmodule` and `\requiremodule` resolve their arguments to find the desired `module` – which is closely related to the *namespace* generated for a `module`, that is used to generate its `MMT-URI`.

Ideally, `TeX` would allow for arbitrary `MMT-URIs` for `modules`, with no forced relationships between the *logical* namespace of a `module` and the *physical* location of the file declaring it – like `MMT` in fact allows for.

Unfortunately, `TeX` only provides very restricted access to the file system, so we are forced to generate namespaces systematically in such a way that they reflect the physical location of the associated files, so that `TeX` can resolve them accordingly. Largely, users need not concern themselves with namespaces at all, but for completeness sake, we describe how they are constructed:



- If `\begin{smodule}{Foo}` occurs in a file `/path/to/file/Foo[.<lang>].tex` which does not belong to an `math` *archive*, the namespace is `file://path/to/file`.
- If the same statement occurs in a file `/path/to/file/bar[.<lang>].tex`, the namespace is `file://path/to/file/bar`.

In other words: outside of `math` *archives*, the namespace corresponds to the file URI with the filename dropped iff it is equal to the `module` name, and ignoring the (optional) language suffix.

If the current file is in an *archive*, the procedure is the same except that the initial segment of the file path up to the *archive*'s *source* directory is replaced by the *archive*'s namespace URI.

Conversely, here is how namespaces/URIs and file paths are computed in import statements, exemplary `\importmodule`:



- `\importmodule{Foo}` outside of an *archive* refers to `module` `Foo` in the current namespace. Consequently, `Foo` must have been declared earlier in the same file or, if not, in a file `Foo[.<lang>].tex` in the same directory.
- The same statement *within* an *archive* refers to either the `module` `Foo` declared earlier in the same file, or otherwise to the `module` `Foo` in the *archive*'s top-level namespace. In the latter case, it has to be declared in a file `Foo[.<lang>].tex` directly in the *archive*'s *source* directory.
- Similarly, in `\importmodule{some/path?Foo}` the path `some/path` refers to either the sub-directory and relative namespace path of the current directory



and namespace outside of an `archive`, or relative to the current `archive`'s top-level namespace and `source` directory, respectively.

The `module` `Foo` must either be declared in the file `<top-directory>/some/path/Foo[.<lang>].tex`, or in `<top-directory>/some/path[.<lang>].tex` (which are checked in that order).

- Similarly, `\importmodule[Some/Archive]{some/path?Foo}` is resolved like the previous cases, but relative to the `archive` `Some/Archive` in the MathHub directory.

7.6 Variables and Sequences

`\vardef` `\vardef{<mname>}[<options>]{<notation>}`

Takes the same arguments as `\symdef`, but produces a `variable` rather than a `symbol`. `Variables` definitions are always local to the current `TeX` group and are allowed anywhere (i.e. outside of `modules`).

`<options>` may include the additional keyword `bind`, in which case the `variable` will be appropriately abstracted away in statements (see also `\varbind`).

Unlike `\symdef`, there is no starred variant `\vardef*` – `variables` always generate a `semantic macro`.

The `semantic macro` for a `variable` behaves analogously to that of a `symbol`.

$\xrightarrow{M}$ `Variables` induces the same `MMT/OMDOC` terms as `symbols` do, except for the head of the term being `<OMV name="...">` instead of `<OMS/>`.

`\varnotation` `\varnotation{<variable>}[<options>]{<notation>}`

Takes the exact same arguments as `\notation`, but attaches an additional `notation` to the `variable` `<variable>` rather than a `symbol`.

`\svar` `\svar[<name>]{<text>}`

Semantically marks up `<text>` as representing a `variable` `<name>`. The `variable` does not need to have been defined prior. If no `<name>` is given `<text>` will be used as the name.

This is useful in situations like “throwaway expressions” or remarks; e.g.

`\plus{\svar{n},\svar{m}}` means...

`\varseq` `\varseq{⟨mname⟩}[⟨options⟩]{⟨range⟩}{⟨notation⟩}`

Declares a new `variable` sequence. The `⟨options⟩` are the same as for `\vardef`. If not provided, `args=1` by default (a 0-ary sequence would just be a normal `variable`).

A `type` (given as `type=`) is interpreted to be the `type` of every element a_i of the sequence $a_1, \dots, a_n$ (not of the sequence itself). If the `type` is itself a sequence $A_1, \dots, A_n$, the assumption is that its range is the same as the one of the new sequence, and the type of every a_i in the sequence is A_i .

`⟨range⟩` needs to be a comma-separated sequence of either `args` many arguments, or `\ellipses`.

The resulting `semantic macro` is allowed anywhere `gTeX` expects an `argument mode` a or B argument.

`\ellipses` Represents ellipses in a range; produces `\ellipses` in math mode.

`\seqmap` `\seqmap{⟨code⟩}{⟨sequence⟩}`

Maps the function `⟨code⟩` (containing `#1`) over every element of the `⟨sequence⟩`.

Is allowed anywhere `gTeX` expects an `argument mode` a or B argument.

7.7 Structures

`Mathematical structure` bundle interdependent `symbols` together.

`mathstructure` (*env.*) `\begin{mathstructure}{⟨mname⟩}[⟨name⟩,this=⟨code⟩]` opens a new `mathematical structure` with name `⟨mname⟩` (if provided) or `⟨mname⟩` (otherwise), and `semantic macro` `\mname`. It subsequently behaves like the `smodule environment`.

`\this` The optional `this=⟨code⟩` option allows for setting the typesetting of the `\this` macro within a `mathstructure`. In particular, `\this` can be used in `notations` for `symbols` declared in the `structure`. `\this` can be thought of as representing “*the*” (current) instance of this `structure`.

`extstructure` (*env.*) `\begin{extstructure}{⟨mname⟩}[⟨name⟩,this=⟨code⟩]{⟨structs⟩}` opens a new `mathematical structure` extending the `structures` given in `⟨structs⟩` (a comma-separated list of names).

`extstructure*` (*env.*) `\begin{extstructure*}{⟨struct⟩}` opens a new `mathematical structure` *conservatively* extending the (single) `structure` `⟨struct⟩`. *Conservative* meaning: Every `symbol` newly introduced in this `structure` needs to have a *definiens*. The new `symbols` are attached as fields directly to `⟨struct⟩`.

`\usestructure` The `\usestructure` macro behaves like `\usemodule` for `mathematical structures`, making the `symbols` available to use directly.

$\hookrightarrow$ `mathstructure` make use of the *Theories-as-Types* paradigm (see
 $\hookrightarrow$ [MueRabKoh:tat18]):
 $\hookrightarrow$

$\hookrightarrow$ `\begin{mathstructure}{\langle name \rangle}` creates a nested *theory* with name $\langle name \rangle$ -module. The *constant* $\langle name \rangle$ is defined as a *dependent record type* with *manifest fields*, the fields of which are generated from (and correspond to) the *constants* in $\langle name \rangle$ -module.

7.7.1 Semantic Macros for Structures

Assume we have a *mathematical structure* with semantic macro `\struct`:

Example 26

```

\begin{mathstructure}{\struct}
  \symdef{fieldda}{a}
  \symdef{fielddb}[args=2]{#1 \maincomp{b} #2}
  \symdef{fielddc}[args=2,def=\sn{fielddb}]{#1 \maincomp{c} #2}
  \inlineass[name=axiom1]{\conclusion{some axiom}}
\end{mathstructure}
\notation{\struct}{\StRuCt}

```

- If `\struct` has no *notations*, then $\mathcal{\struct}!$ produces $\langle a, b, c \rangle$. Otherwise, it produces the notation, i.e. *StRuCt*. In both cases, $\mathcal{\struct}\{\}\{\}$ produces $\langle a, b, c \rangle$ (for reasons that will become clearer in a moment).
- $\mathcal{\struct}\{\}\{\}$ (or $\mathcal{\struct}!$ in the case where no *notations* are around) can be modified in the following ways:
 - $\mathcal{\struct}\{\}\{[\langle fieldname \rangle], \dots\}$ lets you pick, which precise fields to show, so e.g. $\mathcal{\struct}\{\}\{[fieldda, fielddb]\}$ produces $\langle a, b \rangle$. By default, $\mathcal{\struct}\{\}\{\}$ shows exactly the fields that have *semantic macros* (which are also used to access the fields in $\mathcal{\struct}\{\dots\}\{\langle fieldname \rangle\}$).
 - $\mathcal{\struct}\{\}\{[\langle id \rangle]\}$ lets you pick the *notation* of the “*mathematical structure*” symbol to use to typeset the *structure*; e.g. $\mathcal{\struct}\{\}\{[parens]\}$ yields $\langle a, b, c \rangle$, and (combining both) $\mathcal{\struct}\{\}\{[fieldda, fielddb] [angle]\}$ yields $\langle a, b \rangle$.
- The two arguments in $\mathcal{\struct}\{\text{first}\}\{\text{second}\}$ represent 1. the term that is to be treated as an instance of `\struct`, and 2. the precise field to invoke. If the first is empty, then there is no instance. If the second is empty, $\mathcal{\struct}$ will present all of them (that are not assertions). Hence, $\mathcal{\struct}\{S\}\{\}$ yields $\langle a_S, b_S, c_S \rangle$, $\mathcal{\struct}\{S\}\{fieldda\}$ produces a_S , $\mathcal{\struct}\{\}\{fieldda\}$ produces a , and $\mathcal{\struct}\{S\}\{fielddb\}\{x\}\{y\}$ produces xb_{Sy} .
- For the sake of completion, $\mathcal{\struct}\{\text{first}\}!$ simply produces the given argument; e.g. $\mathcal{\struct}\{S\}!$ simply produces S .

More precisely: $\mathcal{\struct}\{\langle code \rangle\}$ acts like a “type coercion” of $\langle code \rangle$ to be an instance of `\struct`.

Of course, it is (occasionally, but) rarely useful to use the *semantic macro* `\struct` by giving it two arguments *manually*; but this is what $\mathcal{\struct}$ does when using `\struct` in the *return* of a *symbol* (or *variable*).

Continuing:

- $\$ \backslash \text{struct}[\text{comp}=\langle \text{code} \rangle][\dots][\dots]\$$ applies $\langle \text{code} \rangle$ (using #1) to every occurrence of $\backslash \text{maincomp}$ in the *notations* of the fields, as a replacement for the default modification $\{ \#1 \}_{\backslash \text{this}}$. For example, $\$ \backslash \text{struct}[\text{comp}=\{ \#1 \}^{\text{Foo}}][\text{S}][\text{S}]\$$ produces $\langle a^{\text{Foo}}, b^{\text{Foo}}, c^{\text{Foo}} \rangle$, and $\$ \backslash \text{struct}[\text{comp}=\{ \#1 \}^{\backslash \text{this}}][\text{S}][\text{fieldb}]{x}{y}\$$ yields xb^Sy .
- $\$ \backslash \text{struct}[\text{this}=\langle \text{code} \rangle][\dots][\dots]\$$ modifies the way $\backslash \text{this}$ is being typeset; i.e. the presentation of the first $\{ \dots \}$ argument. For example $\$ \backslash \text{struct}[\text{this}=\text{T}][\text{S}][\text{S}]\$$ produces $\langle a, b, c \rangle$ – since $\backslash \text{this}$ is not being used in the *notations* of the fields. Note that the `this=` and `comp=` variants are (as of yet) mutually incompatible.
- Finally, $\$ \backslash \text{struct}[\langle \text{fieldname} \rangle=\langle \text{code} \rangle][\dots][\dots]\$$ assigns the field $\langle \text{fieldname} \rangle$ to the term $\langle \text{code} \rangle$. $\langle \text{code} \rangle$ is subsequently used when using $\{ \dots \}$, but not in fields directly. For example, $\$ \backslash \text{struct}[\text{fielda}=\text{A}][\text{S}][\text{S}]\$$ produces $\langle \text{A}!, b_S, c_S \rangle$, but $\$ \backslash \text{struct}[\text{fielda}=\text{A}][\text{S}][\text{fielda}]\$$ produces a_S .

Note the insertion of ! behind the A – this is to make sure that assignments to *semantic macros* that takes arguments don't accidentally eat more than they should.

Also note that multiple assignments can be done in the same pair of [], or chained – i.e. both $\$ \backslash \text{struct}[\text{fielda}=\text{A}, \text{fieldb}=\text{B}][\dots]\$$ and $\$ \backslash \text{struct}[\text{fielda}=\text{A}][\text{fieldb}=\text{B}][\dots]\$$ are valid and equivalent. $\$ \backslash \text{struct}[\text{fielda}=\text{A}, \text{fielda}=\text{B}][\dots]\$$ however is not – every field may be assigned at most once.

Chapter 8

Statements

`STEX` provides four environments to semantically annotate various kinds of statements:

`sdefinition (env.)` The `sdefinition environment` represents (primarily mathematical) *definitions*; in particular for `symbols`. The contents of the environment will be used as *documentation* for any `symbol` that either occurs as a `\definiendum` (or `\definame`) within the `sdefinition`, or that is listed in the optional `for=` argument of the `environment`.

If a `\definiens` occurs, this will be used by `MMT` as the formal *definiens* for the respective `symbol`.

`sassertion (env.)` The `sassertion environment` represents *assertions*, i.e. `propositions` such as *theorems*, *lemmata*, *axioms*, etc. If a `\conclusion` occurs within the `sassertion`, its argument will be used as the formal *statement* of the assertion.

`sexample (env.)` The `sexample environment` represents examples, the `for=` attribute specifies the concept this is an example for. For `counterexamples` use `style=counterexample`

`sparagraph (env.)` The `sparagraph environment` represents all other kinds of (logical) paragraphs, such as remarks, comments, transitions between topics, recaps, reminders, etc.

All of these take the same arguments:

- `for=<cs1>`: a comma-separated list of `symbols`.
- The same optional arguments as `\symdecl`, with `macro=` replacing the name of the `semantic macro`. All of them are only relevant, if either `name=` or `macro=` are provided.

As with `\symdecl`, if no `name` is given, but `macro` is, then the same name is used for both the `semantic macro` and the `symbol` itself.

If `name` is given but `macro` isn't, no `semantic macro` is generated. Subsequently, the newly generated `symbol` is added the `for-list`.

- `style`: see chapter 9.
- `title`: a title to use in various styles (see chapter 9).
- `id`: a label to use for `\sref`.

<code>\inlineass</code>	The macros <code>\inlineass</code> , <code>\inlinedef</code> and <code>\inlineex</code> behave like the <code>sassertion</code> , <code>sdefinition</code> and <code>sexample</code> environments respectively, but take the text to annotate as an argument, rather than as the body of an <code>environment</code> , and do not break paragraphs.
<code>\inlinedef</code>	
<code>\inlineex</code>	

The same macros available in the `environments` are also available in the argument of the `\inline*` macros.

<code>\varbind</code>	<code>\varbind{<cls>}</code>
-----------------------	------------------------------------

retroactively attaches the `bind` option to every `variable` provided (as a comma-separated list).

8.1 More on Definitions

In `sdefinition` (and `sparagraph` with `style=symdoc`), the following additional macros are available:

<code>\definiendum</code>	The <code>\definiendum</code> macro behaves largely like <code>\symref</code> , but it uses a dedicated highlighting for <i>definienda</i> and adds the referenced <code>symbol</code> to the <code>for=</code> list of the <code>environment</code> .
<code>\definame</code>	
<code>\Definame</code>	
<code>\defnotation</code>	

`\definame` is to `\definiendum` as `\symname` is to `\symref`. Analogously, `\Definame` behaves like `\Symname`.

`\defnotation` can be used in math mode to apply the `\definiendum` highlighting to *notations*.

<code>\definiens</code>	The <code>\definiens</code> macro can be used to semantically annotate the <i>definiens</i> in a <code>sdefinition</code> .
-------------------------	---

If the `sdefinition` environment has several elements in its `for` list, an optional argument `\definiens[<symbol>]{...}` can be used to tell `STEX` which `symbol`'s *definiens* this is. By default, the *first* `symbol` in the `for` list is used.

Here is how `MMT` will treat the fragment marked up with `\definiens`:

Firstly, it will attempt to translate its contents into an `MMT/OMDOC` term. This succeeds easily if `\definiens` is some `semantic macro` (applied to arguments).

Secondly, it will check for all `variables` currently in scope, that were defined with the optional argument `bind`. It then will check, whether a `symbol` is in scope, that has `role=lambda`. If so, it will use that `lambda symbol` to bind all these variables (in the order in which they were defined) in the term. If no `lambda symbol` is found, it will use the `bind symbol` that ships with `STEX`.

The final term will be attached as *definiens* to the corresponding `MMT constant`, if it was declared in the same `module` as the `\definiens` occurrence.

8.2 More on Assertions

$\backslash\text{premise}$ $\backslash\text{conclusion}$	The $\backslash\text{conclusion}$ macro can be used to mark up the actual statement within an sassertion. The $\backslash\text{premise}$ macro can be used to additionally mark up <i>premises</i>.
---	--

If the sassertion environment has several elements in its `for` list, an optional argument $\backslash\text{conclusion}[\langle\text{symbol}\rangle]\{\dots\}$ can be used to tell $\text{\LaTeX}$ which symbol 's statement this is. By default, the *first* symbol in the `for` list is used.

Here is how MMT will treat the fragments marked up with $\backslash\text{conclusion}$ and $\backslash\text{premise}$:

Firstly, it will attempt to translate the contents of $\backslash\text{conclusion}$ into an MMT/OMDOC term c . This succeeds easily if the $\backslash\text{conclusion}$ is some *semantic macro* (applied to arguments).

Secondly, it will collect all fragments marked up with $\backslash\text{premise}$ and do the same to them $(p_1, \dots, p_n)$.

It will then check, whether a symbol is in scope, that has `role=implication`. If so, it will use that *implication symbol* to attach all the premises to the conclusion, resulting in $t := \text{imply}(p_1, \dots, p_n, c)$.

Next, it will check for all *variables* currently in scope, that were defined with the optional argument `bind`. It then will check, whether a symbol is in scope, that has `role=forall`. If so, it will use that *forall symbol* to bind all these variables (in the order in which they were defined) in the term t .

Finally, it will check, whether a symbol is in scope, that has `role=judgment`. If so, it will set $t := \text{judgment}(t)$.

If no *forall symbol* is found, it will first apply the *judgment symbol* (if existent) and then use the *bind symbol* that ships with $\text{\LaTeX}$ to bind the *variables*.

The final term will be attached as *type* to the corresponding MMT *constant*, if it was declared in the same *module* as the $\backslash\text{definiens}$ occurrence.

$\hookrightarrow$ M $\rightarrow$
 $\hookrightarrow$ M $\rightarrow$
 $\hookrightarrow$ T $\rightarrow$

$\text{sproof}(\text{env.})$

TODO¹³

¹³TODO: proofs

Chapter 9

Customizing Typesetting

There are two kinds of typesetting that can be customized in $\text{\texttt{STEX}}$: *symbol* references (notation components, $\text{\texttt{\symref}}$, variables, etc.) are highlighted using a small set of *macros* that can be simply redefined by authors.

Other *macros* and *environments* usually have more complicated “typesetting rules” associated with them – often in the form of other already existing *environments* that should be used.

Lastly, in *HTML* we can provide custom CSS rules in *math archives* that determine the styling of certain *environments*, so that the actual presentation depends on the document in which the fragments are included (e.g. via $\text{\texttt{\inputref}}$), rather than the file the fragment is implemented in.

It is generally recommended to implement these customizations in a preamble in the `lib` directory (see Section 1.3 (The `lib`-Directory) in the $\text{\texttt{STEX}}$ Manual)

9.1 Highlighting Symbol References

 $\text{\texttt{\symrefemph}}$
 $\text{\texttt{\symrefemph@uri}}$

$\text{\texttt{\symrefemph}}$ governs how references via $\text{\texttt{\symref}}$ (or $\text{\texttt{\symname}}$, or their short variants) are highlighted;

Doing $\text{\texttt{\symref}}\{\langle symbol \rangle\}\{\langle text \rangle\}$ ultimately expands to $\text{\texttt{\symrefemph@uri}}\{\langle text \rangle\}\{\langle symbol URI \rangle\}$, the default implementation of which is just $\text{\texttt{\symrefemph}}\{\langle text \rangle\}$. The default implementation of $\text{\texttt{\symrefemph}}$, in turn, is just $\text{\texttt{\emph}}\{\langle text \rangle\}$.

If you only want to change e.g. the color of $\text{\texttt{\symrefs}}$, you only need to redefine $\text{\texttt{\symrefemph}}$, e.g. using

```
\renewcommand\symrefemph[1]{\textcolor{red}{#1}}
```

the `@uri` variant is useful if you want to link somewhere, or show the URI in a tooltip. The *stex-highlighting package* does both, using:

```
\usepackage{pdfcomment}
\protected\def\symrefemph@uri#1#2{%
  \pdftooltip{%
    \srefsymuri{#2}{\symrefemph{#1}}%
  }{%
    URI:~\detokenize{#2}%
  }%
}
```



```

}
\def\symrefemph#1{%
  \ifcsname textcolor\endcsname
    \textcolor{teal}{#1}%
  \else#1\fi
}

```

<code>\compemph</code> <code>\compemph@uri</code>	Like <code>\symrefemph</code> and <code>\symrefemph@uri</code> , but governs the highlighting of components (marked with <code>\comp</code> or <code>\maincomp</code>) in <i>notations</i> .
--	---

<code>\defemph</code> <code>\defemph@uri</code>	Like <code>\symrefemph</code> and <code>\symrefemph@uri</code> , but governs the highlighting of definienda marked with <code>\definiendum</code> (or <code>\definame</code>).
--	---

<code>\varemph</code> <code>\varemph@uri</code>	Like <code>\compemph</code> and <code>\compemph@uri</code> , but governs the highlighting of components (marked with <code>\comp</code> or <code>\maincomp</code>) in the <i>notations</i> of <i>variables</i> . The second argument to <code>\varemph@uri</code> is the <i>name</i> of the <i>variable</i> .
--	---

9.2 Styling Environments and Macros

A variety of *environments* and *macros* provided by $\text{\TeX}$ are *stylable* using the *macros* `\stexstyle<name>[<style>]{...}`. These stylable *environments* and *macros* bind various of their parameters to *macros* `\this<parameter>`, which can then be used in the styles.

For example, if we have a *definition environment* that we would want to use to style our *sdefinitions*, we can do (in the simplest case)

```
\stexstyledefinition{\begin{definition}}{\end{definition}}
```

This tells $\text{\TeX}$ to insert `\begin{definition}` at the beginning of every *sdefinition environment*, and `\end{definition}` at the end.

If we have a *environments theorem* and *lemma*, we probably want the *sassertion environment* to use those for theorems and lemma. We can achieve that by doing

```
\stexstyleassertion[theorem]{\begin{theorem}}{\end{theorem}}
\stexstyleassertion[lemma]{\begin{lemma}}{\end{lemma}}
```

Now if we do `\begin{sassertion}[style=theorem]`, it will wrap the *environment* with `\begin{theorem}... \end{theorem}`.

Of course, many such statements might have a title, as e.g. in

Satz 9.2.1 (Gödel's First Incompleteness Theorem). ...

In *sassertion*, we can provide that title as optional argument using `title=...`. Before calling the styling provided, *sassertion* will store that title in the macro `\thistitle`, which we can use in the styling. For example, we might prefer to pass it on to the *theorem environment*:

```
\stexstyleassertion[theorem]{\ifx\thistitle\@empty
\begin{theorem}\else\begin{theorem}[\thistitle]\fi}
{\end{theorem}}}
```

```
\stexstylemodule
\stexstylecopymodule
\stexstyleinterpretmodule
\stexstylerealization
\stexstylemathstructure
\stexstyleextstructure
\stexstyledefinition
\stexstyleassertion
\stexstyleexample
\stexstyleparagraph
\stexstyleproof
\stexstylesubproof
```

TODO¹⁴

Additionally, we can style certain [macros](#), if we want them to produce output. For example, we might (for debuggin or documentation purposes) `\symdecl` to give a short summary of the symbol.

We can achieve that by doing, for example:

```
\stexstylesymdecl[debug]{
Symbol \thisdeclname~(with arity \thisargs) of type $\thistype$.
}
```

in which case

```
\symdecl{foo}[args=2,type=\mathbb{N},style=debug]
```

will yield

Symbol foo (with arity ii) of type N.

```
\stexstyleusemodule
\stexstyleimportmodule
\stexstylerequiremodule
\stexstyleassign
\stexstylerenamedecl
\stexstyleassignMorphism
\stexstylecopymod
\stexstyleinterpretmod
\stexstylerealize
\stexstylesymdecl
\stexstyletextsymbdecl
\stexstylenotation
\stexstylevarnotation
\stexstylesymdef
\stexstylevardef
\stexstylevarseq
\stexstylespfsketch
\stexstyleMMTinclude
```

TODO¹⁵

9.3 Custom CSS for Environments

Chapter 10

Additional Packages

10.1 NotesSlides Manual

10.1.1 Introduction

The `notesslides` document class is derived from `beamer.cls` [`beamerclass:on`], it adds a “notes version” for course notes that is more suited to printing than the one supplied by `beamer.cls`.

The `notesslides` class takes the notion of a slide frame from Till Tantau’s excellent `beamer` class and adapts its notion of frames for use in the $\text{\LaTeX}$ and OMDoc. To support semantic course notes, it extends the notion of mixing frames and explanatory text, but rather than treating the frames as images (or integrating their contents into the flowing text), the `notesslides` package displays the slides as such in the course notes to give students a visual anchor into the slide presentation in the course (and to distinguish the different writing styles in slides and course notes).

In practice we want to generate two documents from the same source: the slides for presentation in the lecture and the course notes as a narrative document for home study. To achieve this, the `notesslides` class has two modes: *slides mode* and *notes mode* which are determined by the package option.

10.1.2 Package Options

The `notesslides` class takes a variety of class options:

<code>slides</code> <code>notes</code>	The options <code>slides</code> and <code>notes</code> switch between slides mode and notes mode (see subsection 4.1.3).
---	--

<code>sectocframes</code>	If the option <code>sectocframes</code> is given, then for the <code>sfragments</code> , special frames with the <code>sfragment</code> title (and number) are generated.
---------------------------	---

<code>frameimages</code> <code>fiboxed</code>	If the option <code>frameimages</code> is set, then slide mode also shows the <code>\frameimage</code> -generated frames (see ???). If also the <code>fiboxed</code> option is given, the slides are surrounded by a box.
--	---

10.1.3 Notes and Slides

frame (*env.*) Slides are represented with the **frame** environment just like in the **beamer** class, see [Tantau:ugbc] for details.

note (*env.*) The **notesslides** class adds the **note** environment for encapsulating the course note fragments.



Note that it is essential to start and end the **notes** environment at the start of the line – in particular, there may not be leading blanks – else L^AT_EX becomes confused and throws error messages that are difficult to decipher.

By interleaving the **frame** and **note** environments, we can build course notes as shown here:

```
1 \ifnotes\maketitle\else
2 \frame[noframenumbering]\maketitle\fi
3
4 \begin{note}
5   We start this course with ...
6 \end{note}
7
8 \begin{frame}
9   \frametitle{The first slide}
10  ...
11 \end{frame}
12 \begin{note}
13   ... and more explanatory text
14 \end{note}
15
16 \begin{frame}
17   \frametitle{The second slide}
18   ...
19 \end{frame}
20 ...
```

\ifnotes Note the use of the **\ifnotes** conditional, which allows different treatment between **notes** and **slides** mode – manually setting **\notestru**e or **\notesfalse** is strongly discouraged however.



We need to give the title frame the **noframenumbering** option so that the frame numbering is kept in sync between the slides and the course notes.



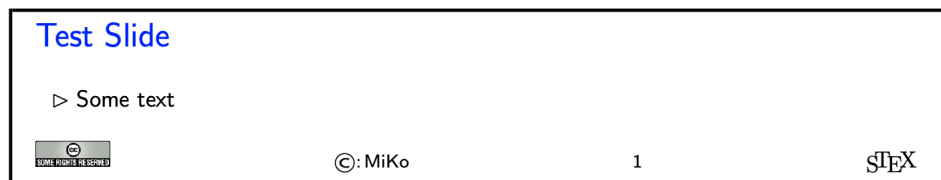
The **beamer** class recommends not to use the **allowframebreaks** option on frames (even though it is very convenient). This holds even more in the **notesslides** case: At least in conjunction with **\newpage**, frame numbering behaves funnily (we have tried to fix this, but who knows).

\inputref* If we want to transclude a the contents of a file as a note, we can use a new variant **\inputref*** of the **\inputref** macro: **\inputref*{foo}** is equivalent to **\begin{note}\inputref{foo}\end{note}**.

nparagraph (*env.*) There are some environments that tend to occur at the top-level of **note** environments. We make convenience versions of these: e.g. the **nparagraph** environment is just an **nparagraph** (*env.*) **sparagraph** inside a **note** environment (but looks nicer in the source, since it avoids one **nexample** (*env.*) level of source indenting). Similarly, we have the **nfragment**, **ndefinition**, **nexample**, **nsproof** (*env.*) **nsproof**, and **nassertion** environments. **nassertion** (*env.*)

10.1.4 Customizing Header and Footer Lines

The **notesslides** package and class comes with a simple default theme named **sTeX** that provided by the **beamterthemesTeX**. It is assumed as the default theme for **sTeX**-based notes and slides. The result in **notes** mode (which is like the **slides** version except that the slide height is variable) is



The footer line can be customized. In particular the logos.

\setslidelogo The default logo provided by the **notesslides** package is the **sTeX** logo it can be customized using **\setslidelogo{<logo name>}**.

\setsource The default footer line of the **notesslides** package mentions copyright and licensing. In **notesslides** **\source** stores the author's name as the copyright holder. By default it is the author's name as defined in the **\author** macro in the preamble. **\setsource{<name>}** can change the writer's name.

\setlicensing For licensing, we use the Creative Commons Attribution-ShareAlike license by default to strengthen the public domain. If package **hyperref** is loaded, then we can attach a hyperlink to the license logo. **\setlicensing[<url>]{<logo name>}** is used for customization, where **<url>** is optional.

10.1.5 Frame Images

Sometimes, we want to integrate slides as images after all – e.g. because we already have a PowerPoint presentation, to which we want to add **sTeX** notes.

<code>\frameimage</code> <code>\mhframeimage</code>	In this case we can use <code>\frameimage[⟨opt⟩]{⟨path⟩}</code>, where <code>⟨opt⟩</code> are the options of <code>\includegraphics</code> from the <code>graphicx</code> package [CarRah:tpp99] and <code>⟨path⟩</code> is the file path (extension can be left off like in <code>\includegraphics</code>). We have added the <code>label</code> key that allows to give a frame label that can be referenced like a regular <code>beamer</code> frame.
--	---

The `\mhframeimage` macro is a variant of `\frameimage` with repository support. Instead of writing

```
1 \frameimage{\MathHub{fooMH/bar/source/baz/foobar}}
```

we can simply write (assuming that `\MathHub` is defined as above)

```
1 \mhframeimage[fooMH/bar]{baz/foobar}
```

Note that the `\mhframeimage` form is more semantic, which allows more advanced document management features in `MathHub`.

If `baz/foobar` is the “current module”, i.e. if we are on the `MathHub` path `...MathHub/fooMH/bar...`, then stating the repository in the first optional argument is redundant, so we can just use

```
1 \mhframeimage{baz/foobar}
```

<code>\textwarning</code>	The <code>\textwarning</code> macro generates a warning sign: 
---------------------------	---

10.1.6 Ending Documents Prematurely

<code>\prematurestop</code> <code>\afterprematurestop</code>	For prematurely stopping the formatting of a document, <code>TeX</code> provides the <code>\prematurestop</code> macro. It can be used everywhere in a document and ignores all input after that – backing out of the <code>sfragment</code> environments as needed. After that – and before the implicit <code>\end{document}</code> it calls the internal <code>\afterprematurestop</code>, which can be customized to do additional cleanup or e.g. print the bibliography.
---	---

`\prematurestop` is useful when one has a driver file, e.g. for a course taught multiple years and wants to generate course notes up to the current point in the lecture. Instead of commenting out the remaining parts, one can just move the `\prematurestop` macro. This is especially useful, if we need the rest of the file for processing, e.g. to generate a theory graph of the whole course with the already-covered parts marked up as an overview over the progress; see `import_graph.py` from the `lmhtools` utilities [lmhtools:github:on].

Text fragments and modules can be made more re-usable by the use of global variables. For instance, the admin section of a course can be made course-independent (and therefore re-usable) by using variables (actually token registers) `courseAcronym` and `courseTitle` instead of the text itself. The variables can then be set in the `TeX` preamble of the course notes file.

10.1.7 Global Document Variables

To make document fragments more reusable, we sometimes want to make the content depend on the context. We use **document variables** for that.

<code>\setSGvar</code>	<code>\setSGvar{⟨vname⟩}{⟨text⟩}</code> to set the global variable <code>⟨vname⟩</code> to <code>⟨text⟩</code> and <code>\useSGvar{⟨vname⟩}</code>
<code>\useSGvar</code>	to reference it.

<code>\ifSGvar</code>	With <code>\ifSGvar</code> we can test for the contents of a global variable: the macro call <code>\ifSGvar{⟨vname⟩}{⟨val⟩}{⟨ctext⟩}</code> tests the content of the global variable <code>⟨vname⟩</code> , only if (after expansion) it is equal to <code>⟨val⟩</code> , the conditional text <code>⟨ctext⟩</code> is formatted.
-----------------------	---

10.1.8 Excursions

In course notes, we sometimes want to point to an “excursion” – material that is either presupposed or tangential to the course at the moment – e.g. in an appendix. The typical setup is the following:

```
1 \excursion{founif}{../fragments/founif.en}
2   {We will cover first-order unification in}
3   ...
4 \begin{appendix}\printexcursions\end{appendix}
```

It generates a paragraph that references the excursion whose source is in the file `../fragments/founif.en.tex` and automatically books the file for the `\printexcursions` command that is used here to put it into the appendix. We will look at the mechanics now.

<code>\excursion</code>	The <code>\excursion{⟨ref⟩}{⟨path⟩}{⟨text⟩}</code> is syntactic sugar for
-------------------------	---

```
1 \begin{nparagraph}[title=Excursion]
2   \activateexcursion{founif}{../ex/founif}
3   We will cover first-order unification in \sref{founif}.
4 \end{nparagraph}
```

<code>\activateexcursion</code>	Here <code>\activateexcursion{⟨path⟩}</code> augments the <code>\printexcursions</code> macro by a call <code>\inputref{⟨path⟩}</code> . In this way, the <code>\printexcursions</code> macro (usually in the appendix) will collect up all excursions that are specified in the main text.
<code>\printexcursion</code>	
<code>\excursionref</code>	

Sometimes, we want to reference – in an excursion – part of another. We can use `\excursionref{⟨label⟩}` for that.

<code>\excursiongroup</code>	Finally, we usually want to put the excursions into an <code>sfragment</code> environment and add an introduction, therefore we provide the a variant of the <code>\printexcursions</code> macro: <code>\excursiongroup[id=⟨id⟩,intro=⟨path⟩]</code> is equivalent to
------------------------------	---

```
1 \begin{note}
2 \begin{sfragment}[id=<id>]{Excursions}
3   \inputref{<path>}
4   \printexcursions
5 \end{sfragment}
6 \end{note}
```




When option `book` which uses `\pagestyle{headings}` is given and semantic macros are given in the `sfragment` titles, then they sometimes are not defined by the time the heading is formatted. Need to look into how the headings are made. This is a problem of the underlying `document-structure` package.

Local Variables: mode: latex TeX-master: ../stex-manual End:

10.2 Problem Manual

10.2.1 Introduction

The `problem` package supplies an infrastructure that allows specify problem. Problems are text fragments that come with auxiliary functions: hints, notes, and solutions. Furthermore, we can specify how long the solution to a given problem is estimated to take and how many points will be awarded for a perfect solution.

Finally, the `problem` package facilitates the management of problems in small files, so that problems can be re-used in multiple environment.

10.2.2 Problems and Solutions

<code>solutions</code> <code>notes</code> <code>hints</code> <code>gnotes</code> <code>pts</code> <code>min</code> <code>boxed</code> <code>test</code>	The <code>problem</code> package takes the options <code>solutions</code> (should solutions be output?), <code>notes</code> (should the problem notes be presented?), <code>hints</code> (do we give the hints?), <code>gnotes</code> (do we show grading notes?), <code>pts</code> (do we display the points awarded for solving the problem?), <code>min</code> (do we display the estimated minutes for problem solving). If theses are specified, then the corresponding auxiliary parts of the problems are output, otherwise, they remain invisible.
--	---

The `boxed` option specifies that problems should be formatted in framed boxes so that they are more visible in the text. Finally, the `test` option signifies that we are in a test situation, so this option does not show the solutions (of course), but leaves space for the students to solve them.

`problem` (*env.*) The main environment provided by the `problempackage` is (surprise surprise) the `problem` environment. It is used to mark up problems and exercises. The environment takes an optional `KeyVal` argument with the keys `id` as an identifier that can be reference later, `pts` for the points to be gained from this exercise in homework or quiz situations, `min` for the estimated minutes needed to solve the problem, and finally `title` for an informative title of the problem.

Example 27

Input:

```

1 \documentclass{article}
2 \usepackage[solutions,hints,pts,min]{problem}
3 \begin{document}
4   \begin{sproblem}[id=elephants,pts=10,min=2,title=Fitting Elephants]
5     How many Elephants can you fit into a Volkswagen beetle?
6     \begin{hint}
7       Think positively, this is simple!
8     \end{hint}
9     \begin{exnote}
10      Justify your answer
11    \end{exnote}
12  \begin{solution}
13    Four, two in the front seats, and two in the back.
14    \begin{gnote}
15      if they do not give the justification deduct 5 pts
16    \end{gnote}
17  \end{solution}
18 \end{sproblem}
19 \end{document}

```

Output:

Exercise (Fitting Elephants)
 How many Elephants can you fit into a Volkswagen beetle?

Lösung: Four, two in the front seats, and two in the back.

`solution (env.)` The `solution` environment can be to specify a solution to a problem. If the package option `solutions` is set or `\solutionstrue` is set in the text, then the solution will be presented in the output. The `solution` environment takes an optional KeyVal argument with the keys `id` for an identifier that can be reference `for` to specify which problem this is a solution for, and `height` that allows to specify the amount of space to be left in test situations (i.e. if the `test` option is set in the `\usepackage` statement).

`hint (env.)` The `hint` and `exnote` environments can be used in a `problem` environment to give hints
`exnote (env.)` and to make notes that elaborate certain aspects of the problem. The `gnote` (grading
`gnote (env.)` notes) environment can be used to document situations that may arise in grading.

`\startsolutions` Sometimes we would like to locally override the `solutions` option we have given to
`\stopsolutions` the package. To turn on solutions we use the `\startsolutions`, to turn them off,
`\stopsolutions`. These two can be used at any point in the documents.

`\ifsolutions` Also, sometimes, we want content (e.g. in an exam with master solutions) conditional
on whether solutions are shown. This can be done with the `\ifsolutions` conditional.

10.2.3 Markup for Added-Value Services

The `problem` package is all about specifying the meaning of the various moving parts of practice/exam problems. The motivation for the additional markup is that we can base added-value services from these, for instance auto-grading and immediate feedback.

The simplest example of this are multiple-choice problems, where the `problem` package allows to annotate answer options with the intended values and possibly feedback that can be delivered to the users in an interactive setting. In this section we will give some infrastructure for these, we expect that this will grow over time.

Multiple Choice Blocks

`mcb` (*env.*) Multiple choice blocks can be formatted using the `mcb` environment, in which single choices are marked up with `\mcc` macro.

`\mcc` `\mcc[⟨keyvals⟩]{⟨text⟩}` takes an optional key/value argument `⟨keyvals⟩` for choice metadata and a required argument `⟨text⟩` for the proposed answer text. The following keys are supported

- `T` for true answers, `F` for false ones,
- `Ttext` the verdict for true answers, `Ftext` for false ones, and
- `feedback` for a short feedback text given to the student.

What we see when this is formatted to PDF depends on the context. In solutions mode (we start the solutions in the code fragment below) we get

Example 28

Input:

```
1 \startsolutions
2 \begin{sproblem}[title=Functions,name=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def}
6     \mcc[F,feedback=that is for C and C++){function}
7     \mcc[F,feedback=that is for Standard ML]{fun}
8     \mcc[F,Ftext=Noooooooooo,feedback=that is for Java]{public static void}
9   \end{mcb}
10 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☒ def
- ☐ function^a
- ☐ fun^b
- ☐ public static void^c

^a

that is for C and C++

^b

that is for Standard ML

^cNoooooooooo

that is for Java

In “exam mode” where disable solutions (here via \stopsolutions)

Example 29

Input:

```
1 \stopsolutions
2 \begin{sproblem}[title=Functions,name=functions1]
3   What is the keyword to introduce a function definition in python?
4   \begin{mcb}
5     \mcc[T]{def}
6     \mcc[F,feedback=that is for C and C++){function}
7     \mcc[F,feedback=that is for Standard ML]{fun}
8     \mcc[F,Ftext=Noooooooooo,feedback=that is for Java]{public static void}
9   \end{mcb}
10 \end{sproblem}
```

Output:

Exercise (Functions)

What is the keyword to introduce a function definition in python?

- ☐ def
- ☐ function
- ☐ fun
- ☐ public static void

we get the questions without solutions (that is what the students see during the exam/quiz).

Filling-In Concrete Solutions

The next simplest situation, where we can implement auto-grading is the case where we have fill-in-the-blanks

`\fillinsol` The `\fillinsol` macro takes a single argument, which contains a concrete solution (i.e. a number, a string, ...), which generates a fill-in-box in test mode:

Example 30

Input:

```
1 \stopsolutions
2 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
3   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
4 \end{sproblem}
```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle?

Example 31

Input:

```
1 \begin{sproblem}[id=elephants.fillin,title=Fitting Elephants]
2   How many Elephants can you fit into a Volkswagen beetle? \fillinsol{4}
3 \end{sproblem}
```

Output:

Exercise (Fitting Elephants)

How many Elephants can you fit into a Volkswagen beetle?

If we do not want to leak information about the solution by the size of the blank we can also give `\fillinsol` an optional argument with a size: `\fillinsol[3cm]{12}` makes a box three cm wide.

Obviously, the required argument of `\fillinsol` can be used for auto-grading. For concrete data like numbers, this is immediate, for more complex data like strings “soft comparisons” might be in order.¹⁶

10.2.4 Including Problems

`\includeproblem` The `\includeproblem` macro can be used to include a problem from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one problem in the include file). The keys `title`, `min`, and `pts` specify the problem title, the estimated minutes for solving the problem and the points to be gained, and their values (if given) overwrite the ones specified in the `problem` environment in the included file.

¹⁶For the moment we only assume a single concrete value as correct. In the future we will almost certainly want to extend the functionality to multiple answer classes that allow different feedback like in MCQ. This still needs a bit of design. Also we want to make the formatting of the answer in solutions/test mode configurable.

The sum of the points and estimated minutes (that we specified in the `pts` and `min` keys to the `problem` environment or the `\includeproblem` macro) to the log file and the screen after each run. This is useful in preparing exams, where we want to make sure that the students can indeed solve the problems in an allotted time period.

The `\min` and `\pts` macros allow to specify (i.e. to print to the margin) the distribution of time and reward to parts of a problem, if the `pts` and `pts` options are set. This allows to give students hints about the estimated time and the points to be awarded.

10.2.5 Testing and Spacing

The `problem` package is often used by the `hwexam` package, which is used to create homework assignments and exams. Both of these have a “test mode” (invoked by the package option `test`), where certain information –master solutions or feedback – is not shown in the presentation.

<code>\testspace</code>	<code>\testspace</code> takes an argument that expands to a dimension, and leaves vertical space accordingly. Specific instances exist: <code>\testsmallspace</code> , <code>\testsmallspace</code> , <code>\testsmallspace</code> give small (1cm), medium (2cm), and big (3cm) vertical space.
<code>\testsmallspace</code>	<code>\testnewpage</code> makes a new page in <code>test</code> mode, and <code>\testemptypage</code> generates an empty page with the cautionary message that this page was intentionally left empty.
<code>\testnewpage</code>	Local Variables: mode: latex TeX-master: <code>../stex-manual</code> End:
<code>\testemptypage</code>	LocalWords: solutions,notes,hints,gnotes,pts,min,boxed,test gnotes elephants,pts gnote LocalWords: 2,title exnote hint,exnote,gnote ifsolutions mcb keyvals Ttext Ftext LocalWords: Functions,name F,feedback Noooooooooo,feedback 2,title includeproblem LocalWords: elephants.fillin,title

10.3 HWExam Manual

10.3.1 Introduction

The `hwexam` package and class supplies an infrastructure that allows to format nice-looking assignment sheets by simply including problems from problem files marked up with the `problem` package. It is designed to be compatible with `problems.sty`, and inherits some of the functionality.

10.3.2 Package Options

<code>solutions</code>	The <code>hwexam</code> package and class take the options <code>solutions</code> , <code>notes</code> , <code>hints</code> , <code>gnotes</code> , <code>pts</code> , <code>min</code> , and <code>boxed</code> that are just passed on to the <code>problems</code> package (cf. its documentation for a description of the intended behavior).
<code>notes</code>	
<code>hints</code>	
<code>gnotes</code>	
<code>pts</code>	
<code>min</code>	
<code>multiple</code>	Furthermore, the <code>hwexam</code> package takes the option <code>multiple</code> that allows to combine multiple assignment sheets into a compound document (the assignment sheets are treated as section, there is a table of contents, etc.).
<code>test</code>	Finally, there is the option <code>test</code> that modifies the behavior to facilitate formatting tests. Only in <code>test</code> mode, the macros <code>\testspace</code> , <code>\testnewpage</code> , and <code>\testemptypage</code>

have an effect: they generate space for the students to solve the given problems. Thus they can be left in the L^AT_EX source.

10.3.3 Assignments

assignment (*env.*) This package supplies the **assignment** environment that groups problems into assignment sheets. It takes an optional KeyVal argument with the keys **number** (for the assignment number; if none is given, 1 is assumed as the default or — in multi-assignment documents **title** — the ordinal of the **assignment** environment), **title** (for the assignment title; this is **type** referenced in the title of the assignment sheet), **type** (for the assignment type; e.g. “quiz”, **given** or “homework”), **given** (for the date the assignment was given), and **due** (for the date the assignment is due).

10.3.4 Including Assignments

\inputassignment The **\inputassignment** macro can be used to input an assignment from another file. It takes an optional KeyVal argument and a second argument which is a path to the file containing the problem (the macro assumes that there is only one **assignment** environment in the included file). The keys **number**, **title**, **type**, **given**, and **due** are just as for the **assignment** environment and (if given) overwrite the ones specified in the **assignment** environment in the included file.

10.3.5 Typesetting Exams

testheading (*env.*) The **\testheading** takes an optional keyword argument where the keys **duration** specifies a string that specifies the duration of the test, **min** specifies the equivalent in number of minutes, and **reqpts** the points that are required for a perfect grade.

```
reqpts 1 \title{320101 General Computer Science (Fall 2010)}
        2 \begin{testheading}[duration=one hour,min=60,reqpts=27]
        3   Good luck to all students!
        4 \end{testheading}
```

Will result in

Name: Matriculation Number:

320101 General Computer Science (Fall 2010)
2026-08-08

You have one hour (sharp) for the test;
Write the solutions to the sheet.
The estimated time for solving this exam is 23 minutes, leaving you 37 minutes for revising your exam.
You can reach 23 points if you solve all problems. You will only need 27 points for a perfect score, i.e. -4 points are bonus points.

Different problems test different skills and knowledge, so do not get stuck on one problem.

	To be used for grading, do not write here								
prob.	4.1	1.1	1.2	2.1	2.2	2.3	2.4	2.5	2.6
total	0	0	0	10	0	0	0	0	0
reached									

good luck

17
Local Variables: mode: latex TeX-master: ../stex-manual End:
LocalWords: hwexam solutions,notes,hints,gnotes,pts,min gnotes testemptypage re-
qpts LocalWords: inputassignment reqpts hour,min 60,reqpts

10.4 Tikzinput Manual

image The behavior of the `ikzinput` package is determined by whether the `image` option is given. If it is not, then the `tikz` package is loaded, all other options are passed on to it and `\tikzinput{<file>}` inputs the TIKZ file `<file>.tex`; if not, only the `graphicx` package is loaded and `\tikzinput{<file>}` loads an image file `<file>.<ext>` generated from `<file>.tex`.
The selective input functionality of the `tikzinput` package assumes that the TIKZ pictures are externalized into a standalone picture file, such as the following one

¹⁷MK: The first three “problems” come from the stex examples above, how do we get rid of this?


```

1 \documentclass{standalone}
2 \usepackage{tikz}
3 \usetikzpackage{...}
4 \begin{document}
5   \begin{tikzpicture}
6     ...
7   \end{tikzpicture}
8 \end{document}

```

The `standalone` class is a minimal $\text{\LaTeX}$ class that when loaded in a document that uses the `standalone` package: the preamble and the `document` environment are disregarded during loading, so they do not pose any problems. In effect, an `\input` of the file above only sees the `tikzpicture` environment, but the file itself is standalone in the sense that we can run $\text{\LaTeX}$ over it separately, e.g. for generating an image file from it.

$\text{\tikzinput}$ $\text{\ctikzinput}$	This is exactly where the <code>tikzinput</code> package comes in: it supplies the <code>\tikzinput</code> macro, which – depending on the <code>image</code> option – either directly inputs the TIKZ picture (source) or tries to load an image file generated from it.
---	---

Concretely, if the `image` option is not set for the `tikzinput` package, then `\tikzinput[ $\langle opt \rangle$ ]{ $\langle file \rangle$ }` disregards the optional argument $\langle opt \rangle$ and inputs $\langle file \rangle.tex$ via `\input` and resizes it to as specified in the `width` and `height` keys. If it is, `\tikzinput[ $\langle opt \rangle$ ]{ $\langle file \rangle$ }` expands to `\includegraphics[ $\langle opt \rangle$ ]{ $\langle file \rangle$ }`.

`\ctikzinput` is a version of `\tikzinput` that is centered.

$\text{\mhtikzinput}$ $\text{\cmhtikzinput}$	$\text{\mhtikzinput}$ is a variant of <code>\tikzinput</code> that treats its file path argument as a relative path in a math archive in analogy to <code>\inputref</code> . To give the archive path, we use the <code>mhrepos=</code> key. Again, <code>\cmhtikzinput</code> is a version of <code>\mhtikzinput</code> that is centered.
---	--

$\text{\libusetikzlibrary}$	Sometimes, we want to supply archive-specific TIKZ libraries in the <code>lib</code> folder of the archive or the <code>meta-inf/lib</code> of the archive group. Then we need an analogon to <code>\libinput</code> for <code>\usetikzlibrary</code> . The <code>stex-tikzinput</code> package provides the <code>libusetikzlibrary</code> for this purpose.
-----------------------------	---

Local Variables: mode: latex TeX-master: ../stex-manual End:

Part III
Documentation

Chapter 11

Utilities

Various utility methods

`\stex_ignore_spaces_and_pars:`

Ignores space characters and empty lines (which would close the current paragraph)

`\stex_undefine:N` Locally undefines a macro
`\stex_undefine:c`

`\stex_str_if_starts_with_p:nn` * `\stex_str_if_starts_with:nn` `{<str1>}` `{<str2>}`
`\stex_str_if_starts_with:nnTF` *

Tests if `<str1>` starts with `<str2>`.

`\stex_str_if_ends_with_p:nn` * `\stex_str_if_ends_with:nn` `{<str1>}` `{<str2>}`
`\stex_str_if_ends_with:nnTF` *

Tests if `<str1>` ends with `<str2>`.

`\stex_deactivate_macro:Nn` `\stex_deactivate_macro:Nn` `\macro` `{<scope>}`
`\stex_reactivate_macro:N`

redefines `\macro` to throw an error, that the `\macro` is only allowed in `<scope>`. This allows for more informative error messages than `undefined control sequence`.

`\stex_reactivate_macro:N` makes the macro valid again.

11.1 kpsewhich and Environment Variables

`\stex_kpsewhich:Nn` `\stex_kpsewhich:Nn` `\macro` `{<args>}`

Calls “`kpsewhich <args>`” and stores the result in `\macro`,

T_EXhackers note: (Usually) does not require `shell-escape`, since by default, `kpsewhich` is in the list of “allowed” commands.

<code>\stex_get_env:Nn</code>	<code>\stex_get_env:Nn \macro {<envvar>}</code>
-------------------------------	---

Stores the value of the environment variable `<envvar>` in `\macro`.

11.2 Logging

<code>\stex_debug:nn</code>	<code>\stex_debug:nn {<prefix>} {<msg>}</code>
-----------------------------	--

Logs the debug message `{<msg>}` under the prefix `{<prefix>}`. A message is shown if its prefix is in a list of prefixes given either via the package option `debug=<prefixes>` or the environment variable `STEX_DEBUG=<prefixes>`, where the latter overrides the former.

11.3 File Paths

Since `STEX` needs to handle files from various *math archives* in a user-file-system-independent manner in arbitrary *MathHub* directories, we primarily use *absolute* file paths, for operations such as `\inputref`, `\usemodule`, `\importmodule`, etc.

We therefore provide macros to explicitly handle file paths independent of the underlying operating system, resolving and canonicalizing `..` segments and relative paths, etc.

<code>\stex_file_set:Nn</code>	<code>\stex_file_set:Nn \macro {<string>}</code>
--------------------------------	--

<code>\stex_file_set:(No Ne)</code>	represents an already canonicalized file path string as a <code>L^AT_EX3</code> sequence and stores it in <code>\macro</code> .
-------------------------------------	---

<code>\stex_file_resolve:Nn</code>	<code>\stex_file_resolve:Nn \macro {<string>}</code>
------------------------------------	--

<code>\stex_file_resolve:(No Ne)</code>	resolves and canonicalizes the file path string <code><string></code> and stores the result in <code>\macro</code> .
---	--

<code>\stex_file_use:N</code>	★ expands to a string representation of the given file path (using <code>/</code> as separator, regardless of file system).
-------------------------------	---

<code>\stex_file_split_off_ext:NN</code>	<code>\stex_file_split_off_ext:NN \target \source</code>
--	--

splits off the file extension of `\source` and stores the resulting file path in `\target`.

<code>\stex_file_split_off_lang:NN</code>	<code>\stex_file_split_off_lang:NN \target \source</code>
---	---

splits off *both* the file extension *and* language component of `\source` and stores the resulting file path in `\target`; e.g. if `\source` is `foo.en.tex`, `\target` will be `foo`. Explicitly checks that the segment before the file extension is a valid language identifier; otherwise, it will not be stripped.

<code>\c_stex_pwd_file</code>	represent the <i>parent working directory</i> and absolute path to the top-level <code>.tex</code> file currently being processed (as absolute, canonicalized paths)
-------------------------------	--

`\c_stex_home_file` represents the absolute path of the user’s home directory (as absolute, canonicalized path)

`\g_stex_current_file` the current file (as absolute, canonicalized path)

`\stex_input_with_hooks:Nn` `\stex_input_with_hooks:Nn \document-URI {<file string>}`
`\stex_input_with_hooks:Ne` Inputs the given file (as a string) by passing it on to `\input`, setting `\g_stex_current_file`, `\l_stex_current_language_str` and `\l_stex_current_document_uri`, and reverting them again afterwards

`\stex_with_file_hooks:Nnn` `\stex_with_file_hooks:Nnn \document-URI {<file string>} {<code>}`
sets `\g_stex_current_file`, `\l_stex_current_language_str` and `\l_stex_current_document_uri`, executes `<code>`, and reverts them again afterwards

`\stex_if_file_absolute_p:N` `*`
`\stex_if_file_absolute:NTF` `*`

tests whether the given file path (represented as a canonicalized L^AT_EX3 sequence) is absolute.

`\stex_if_file_starts_with:NNTF` `\stex_if_file_starts_with:NN \child \parent`
tests whether the file path `\child` is a child of `\parent`.

11.4 Group-like Behaviours

Macros for mimicking the behaviour of T_EX groups without actually opening a T_EX group. This is relevant for two purposes:

- Temporarily redefining a macro, doing a bunch of stuff, and then reverting it to its previous definition; that is what a “*pseudogroup*” does.
- Defining macros at a specific group level further up. That is what a “*metagroup*” does – code wrapped in `\stex_metagroup_do_in:n` will be repeatedly executed in `\aftergroup` calls, up to the group level of the innermost metagroup. That allows for e.g. declaring new symbols in `sdefinition` paragraphs that are valid in the entire `mathstructure` or `module`.

`\stex_pseudogroup:nn` `\stex_pseudogroup:nn {<code>} {<reset code>}`
will *expand* `<reset code>` first, then process `<code>`, then process the expanded `<reset code>`.

`\stex_pseudogroup_restore:N` ★ `\stex_pseudogroup_restore:N \macro`

will expand to code that will redefine `\macro` as its current definition. Only works for *token lists* and similar macros that do not take arguments.

In combination, that allows for things like

Example 32

```
1 % \stex_pseudogroup:nn{
2 % \def \foo {..}
3 % ...
4 % }\stex_pseudogroup_restore:N \foo}
5 %
```

`\stex_pseudogroup_with:nn` `\stex_pseudogroup_with:nn {<macro-list>} {<code>}`

will store the definitions of the macros in `<macro-list>`, execute `<code>`, and then revert the macros to their previous definitions without opening a TeX group. **TODO: this might be unnecessary? Check/profile**

`\stex_metagroup_new:` opens a new *metagroup*. All code executed in `\stex_metagroup_do_in:n` will survive up to the current group level. Metagroups can be nested.

`\stex_metagroup_do_in:n` Executes the given code in the current metagroup. If there is no metagroup currently, the code will simply be executed once, so there is little danger in calling this spuriously.

TeXhackers note: This repeatedly stores the provided code in a macro and uses `\aftergroup` until the group level of the current metagroup is reached.

11.5 Key Handling

Key handling mechanisms on top of `l3keys` for inheriting key sets and initializing the associated macros

`\stex_keys_define:nnnn` `\stex_keys_define:nnnn {<id>} {<init>} {<spec>} {<exts>}`

defined the key set `<id>` with the `l3keys`-style key-parsing code `{<spec>}`; `<init>` is called every time before parsing and is intended to e.g. clear the relevant macros. `<exts>` may be a comma-separated list of key set ids from which this key set inherits.

`\stex_keys_set:nn` Like `\keys_set:nn`, but additionally calling the relevant initialization code.

11.6 Languages

`\c_stex_languages_prop`
`\c_stex_language_abbrevs_prop`

Two inverse property lists; `\c_stex_languages_prop` maps language abbreviations (e.g. `en`, `de`) to their (babel) names (e.g. `english`, `ngerman`), `\c_stex_language_abbrevs_prop` does the inverse.

`\l_stex_current_language_str`

The language of the current file; may change, if a file with a different language is imported.

11.7 Styling

`\stex_new_stylable_cmd:nnnn` `\stex_new_stylable_cmd:nnnn {<macroname>} {<argument spec>} {<definition>}`
`\stex_style_apply:` `{<default>}`

Defines a new stylable command `\macroname` with `<argument spec>` (passed on to `\NewDocumentCommand`) with `<definition>` as its macro body and default typesetting style `<default>`.

`<default>` may be empty. `<definition>` should at some point use `\stex_style_apply:` to do the typesetting according to the chosen style for this macro, which by default will be defined as `<default>`.

Also generates the macro `\stexstyle<macroname>[<name>]{<code>}` to define a new typesetting style for `\macroname`.

`\stex_new_stylable_env:nnnnnn` `\stex_new_stylable_env:nnnnnn {<environment name>} {<argument spec>} {<begin code>} {<end code>} {<default begin>} {<default end>} {<prefix>}`

Defines a new stylable environment `<prefix><environment name>` with `<argument spec>` (passed on to `\NewDocumentEnvironment`) with `<begin code>` and `<end code>` as its macro body and default typesetting style dictated by `<default begin>` and `<default end>`.

`<begin code>` and `<end code>` should at some point use `\stex_style_apply:` to do the typesetting according to the chosen style for this environment.

Also generates the macro `\stexstyle<environmentname>[<name>]{<begin>}{<end>}` to define a new typesetting style for `environment name`.

A prefix `s` is e.g. used to generate the environment `sparagraph` and the macro `\stexstyleparagraph`.

11.8 Persisting Dependencies

`\stex_persist:n`
`\stex_persist:e`
`\loadsms`

Writes the given code in the `.sms`-file (if `smsmode` is on)

Chapter 12

HTML Output

Macros for dealing with HTML output.

`\stex@backend` Which engine is running; possible values are `pdflatex`, `rustex`, `tex4ht` (**TODO**) and `latexml` (**TODO**).

`\stex_if_html_backend_p: *` L^AT_EX3 conditional testing whether the engine running compiles to HTML.
`\stex_if_html_backend:TF *`

`\ifstexhtml *` L^AT_EX2 conditional testing whether the engine running compiles to HTML.

`\stex_if_do_html_p: *` Whether to (locally) produce HTML output. May be turned off locally even if the back-
`\stex_if_do_html:TF *` end produces HTML; e.g. when parsing dependencies in `\importmodule` or `\usemodule`.

`\stex_suppress_html:n` temporarily suppresses HTML output when processing the provided code.

`\stex_annotate:nn` `\stex_annotate:nn {<keyvals>} {code}`
 attaches the provided comma-separated key-value-pairs (as `key=val`) as HTML attributes to the HTML node(s) generated from `<code>`. If `<code>` results in multiple nodes, this needs to insert a new `<div>`, `<span>` or `<mrow>` node for the attributes. Multiple nested calls to `\stex_annotate:nn` may be merged into a single node.

`stex_annotate_env (env.)` like `\stex_annotate:nn`, but as an environment; i.e. `\begin{stex\_annotate\_env}{<keyvals>}{<code>}` is equivalent to `\stex_annotate:nn{<keyvals>}{<code>}`

`\stex_html_node:nnn` `\stex_html_node:nnn {<tag>} {<keyvals>} {code}`
 like `\stex_annotate:nn`, but makes sure to wrap `<code>` in a new `<tag>` node.

`stex_env_node (env.)` like `\stex_html_node:nnn`, but as an environment; i.e. `\begin{stex\_env\_node}{<tag>}{<keyvals>}{<code>}`

is equivalent to `\stex_html_node:nnn{<tag>}{<keyvals>}{<code>}`

`\stex_annotate_invisible:nn` `\stex_annotate_invisible:nn {<keyvals>} code`
`\stex_annotate_invisible:n`

like `\stex_annotate:nn`, but additionally adds the key values `data-ftml-invisible="true"` and `style="display:none;"`. `\stex_annotate_invisible:n` does not take additional attribute-value-pairs.

`\STEXinvisible` public wrapper for `\stex_annotate_invisible:n`.

`\stex_css_link:n` `\stex_css_link:n {<url>}`

inserts a `<link rel="stylesheet" href="<url>">` node in the header of the generated HTML.

`\stex_css_literal:n` `\stex_css_literal:n {<text>}`

inserts a `<style><text></style>` node in the header of the generated HTML.

`\_stex_annotate_force_break:n`

should guarantee that the HTML nodes generated by the provided code are not merged with any outer nodes with respect to attached attributes.

`\mmlintent` `\mmlintent {<value>} {<code>}`

`\mmlarg` annotates `<code>` with the provided MathML *intent* attribute (or *intent arg* attribute, respectively).

and style patching:

```

1 <@@=stex_styles>
2 \stex_if_html_backend:TF{
3   \cs_new_protected:Nn \stex_style_apply: {}
4   \cs_new_protected:Nn \_stex_styles_patch:nnnn {}
5   \stex_keys_define:nnnn{ csspatch }{
6     \str_clear:N \l_stex_keys_cls_id
7     \str_clear:N \l_stex_keys_counter_id
8     \str_clear:N \l_stex_keys_counter_parent
9   }{
10    counter      .str_set_x:N = \l_stex_keys_counter_id,
11    parent       .str_set_x:N = \l_stex_keys_counter_parent,
12    unknown      .code:n      = {
13      \exp_args:NNo \str_set:Nn \l_stex_keys_cls_id {\l_keys_key_tl}
14    }
15  }{}
16  \cs_new_protected:Npn \_stex_styles_css_patch:nnnn #1 #2 {
17    \stex_keys_set:nn{ csspatch }{ #2 }
18    \group_begin:
19    \catcode'\ =12\relax
20    \catcode'\^J=12\relax
21    \_stex_styles_patch_i:nnn {#1}
22  }
```

```

23
24 \seq_new:N \g__stex_styles_counters_seq
25
26 \cs_new:Nn \__stex_styles_patch_html_c: {
27   \stex_html_literal:n{
28     <span~data-ftml-counter="\l_stex_keys_counter_id"~style="display:none;"~
29     data-ftml-counter-parent="\l_stex_keys_counter_parent"
30     ></span>
31   }
32 }
33
34 \cs_new:Nn \__stex_styles_patch_html: {
35   \stex_html_literal:n{
36     <span~data-ftml-style="\l_stex_keys_cls_id"~style="display:none;"~
37     data-ftml-counter="\l_stex_keys_counter_id"
38     ></span>
39   }
40 }
41
42 \cs_new_protected:Nn \__stex_styles_patch_i:nnn {
43   \str_if_empty:NTF \l_stex_keys_cls_id {
44     \str_set:Nn \l_stex_keys_cls_id { #1 }
45   }{
46     \str_set:Ne \l_stex_keys_cls_id { #1-\l_stex_keys_cls_id }
47   }
48   \exp_args:No \str_case:nnF \l_stex_keys_counter_parent {
49     {}{}
50     {part}{\str_set:Nn \l_stex_keys_counter_parent { 0 }}
51     {chapter}{\str_set:Nn \l_stex_keys_counter_parent { 1 }}
52     {section}{\str_set:Nn \l_stex_keys_counter_parent { 2 }}
53     {subsection}{\str_set:Nn \l_stex_keys_counter_parent { 3 }}
54     {subsubsection}{\str_set:Nn \l_stex_keys_counter_parent { 4 }}
55     {paragraph}{\str_set:Nn \l_stex_keys_counter_parent { 5 }}
56     {subparagraph}{\str_set:Nn \l_stex_keys_counter_parent { 6 }}
57   }{
58     \msg_error:nnx{stex}{error/notasection}\l_stex_keys_counter_parent
59   }
60   \str_set:Ne \l__stex_styles_first_str { &~.ftml-title-paragraph~ { display:inline-block}
61   \str_if_empty:NF \l_stex_keys_counter_id {
62     %\str_put_left:Ne \l__stex_styles_first_str { counter-increment:~ ftml-\l_stex_keys_c
63     \exp_args:NNo \seq_if_in:NnF \g__stex_styles_counters_seq \l_stex_keys_counter_id {
64       \seq_gpush:No \g__stex_styles_counters_seq \l_stex_keys_counter_id
65       \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
66         \__stex_styles_patch_html_c:
67         % in body
68       }{
69         \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html_c:
70         % in preamble
71       }
72     }
73     \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
74       \__stex_styles_patch_html:
75       % in body
76     }{

```

```

77     \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html:
78     % in preamble
79   }
80 }
81 \exp_args:Ne \stex_css_literal:n { .ftml-\l_stex_keys_cls_id { \l__stex_styles_first_st
82
83 % TODO export counter reset somehow
84
85 \group_end:
86 }
87 }{
88 \cs_new_protected:Nn \__stex_styles_patch:nnnn {
89   \str_if_empty:nTF {#2}{
90     \tl_set:cn{\__stex_styles_style_#1_start:}{#3}
91     \tl_set:cn{\__stex_styles_style_#1_end:}{#4}
92   }{
93     \tl_set:cn{\__stex_styles_style_#1_#2_start:}{#3}
94     \tl_set:cn{\__stex_styles_style_#1_#2_end:}{#4}
95   }
96 }
97 \cs_new_protected:Nn \__stex_styles_css_patch:nnnn {}
98 }

```

Chapter 13

Math Archives

<code>\c_stex_mathhub_files</code> <code>\mathhub</code>	represents the comma-separated, absolute path <i>strings</i> of the user's MathHub directories. If the <code>\mathhub</code> macro is set when the <code>stex</code> package is loaded, this one is used for finding archives. Otherwise it is determined from the <code>MATHHUB</code> environment variable, if set, or from the path in <code><HOME>/stex/mathhub.path</code> , if existent, or set to <code><HOME>/MathHub</code> if all else fails. In all cases, <code>\mathhub</code> will be defined.
---	--

MathHub directories are scanned in order; i.e. earlier paths are prioritized over later ones.

A *math archive* is represented as a triple `{<archive id>}{<base uri>}{<file path string>}`, where an invariant is that `<file path string>` is of the form `<mh>/<archive id>` for some directory `<mh>` in `\c_stex_mathhub_files`.

Such a triple is stored in the macro `\c_stex_mathhub_<id>_archive` when the archives metadata is first loaded.

<code>\stex_archive_id:N</code> *	expands to the id of the given archive (a macro defined as a triple as described above)
-----------------------------------	---

<code>\stex_archive_base:N</code> *	expands to the base URI of the given archive; either given as a macro defined as a triple
<code>\stex_archive_base:n</code> *	as described above, or as the archive's id. The latter requires the archive to be loaded beforehand!

<code>\stex_archive_path:N</code> *	expands to the file path <i>as a string</i> of the given archive; either given as a macro defined
<code>\stex_archive_path:n</code> *	as a triple as described above, or as the archive's id. The latter requires the archive to be loaded beforehand!

<code>\stex_in_archive:nn</code>	<code>\stex_in_archive:nn {<archive id>} {<code>}</code>
----------------------------------	--

Temporarily sets the current archive `\l_stex_current_archive` to `<archive id>`, executes `<code>` and reverts back to the previous archive. Passes the id of the *now current* archive to `<code>` as `#1`: If the first argument is empty, this will be the current archive's id without changing it. Otherwise, the archive with the desired id will be loaded; throws an error, if it does not exist.

<code>\c_stex_no_archive_str</code>	The archive ID <code>\c_stex_no_archive_str=no/archive</code> is used for documents and other content that is not contained in a <i>math archive</i> . <code>\c_stex_no_archive</code> is the corresponding archive triple.
<code>\c_stex_no_archive</code>	

<code>\c_stex_main_archive</code>	Point to the main file's archive and the <i>current</i> archive, respectively, if existent; undefined otherwise.
<code>\l_stex_current_archive</code>	

<code>\stex_source_path:n</code>	<code>\stex_source_path:n</code> $\{ \langle \textit{relative path} \rangle \}$
<code>\stex_source_path:nn</code>	<code>\stex_source_path:nn</code> expands to the absolute file path (string) of the given $\langle \textit{relative path} \rangle$ relative to the source directory of the current archive – or relative to the current file, if we are not in an archive. In the latter case, the file path will <i>not</i> be canonicalized as to remain expandable.

The variant `\stex_source_path:nn` takes the ID of an archive as first argument (instead of the current archive).

Chapter 14

URIs

There are kinds of URIs used in IMMT/OMDoc:

- *Base URIs*, i.e. namespaces,
- *Archive URIs* of the form `<base uri>?a=<archive id>`,
- *Document URIs* of the form `<archive uri>[&p=<path>]&d=<name>&l=<language>`,
- *Document element URIs* of the form `<document uri>&e=<name>`,
- *Module URIs* of the form `<archive uri>[&p=<path>]&m=<name>`, and
- *Declaration URIs* of the form `<module uri>&s=<name>`.

We do not need all of these in `STeX` itself: the *base uri* of any URI is uniquely determined by the archive ID, and we conceptually merge document URIs and document element URIs.

We therefore represent document (element) URIs as 5-tuples `{<archive id>}{<path>}{<name>}{<language>}{<element name>}`, where `<path>` and `<element name>` may be empty; module URIs as triples `{<archive id>}{<path>}{<name>}`, and declaration URIs as 4-tuples `{<archive id>}{<path>}{<module name>}{<name>}`.

`\stex_use_archive_uri:n *` expands to the string representation of the archive URI of the archive with the given ID. Requires, that the archive has been loaded first

14.1 Document URIs

`\stex_use_document_uri:N *` expands to the string representation of the document uri (represented as a macro containing a 5-tuple as above). The variant `\stex_document_uri:n *` expects a 5-tuple directly.

`\stex_document_uri_archive:N *` expands to the archive ID of the document URI

`\stex_document_uri_path:N *`

expands to the path of the document URI (possibly empty)

`\stex_document_uri_name:N *`

expands to the document name of the document URI

`\stex_document_uri_language:N *`

expands to the language of the document URI

`\stex_document_uri_element:N *`

expands to the element name of the document (element) URI (possibly empty)

`\stex_document_uri_with_language:Nn *`

expands to the document URI with the given language

`\stex_new_document_element_uri:n *`

expands to a new document element URI in the current document

`\stex_document_uri_from_archive_file:Nn \stex_uri_from_archive_file:Nn \macro {<relative path>}`

Constructs the document URI of the file *<relative path>* in the current archive and stores it in `\macro`

`\c_stex_main_document_uri`
`\l_stex_current_document_uri`

store the document URIs of the top-level file being processed and the *current* file, respectively.

14.2 Module URIs

`\stex_use_module_uri:N *` expands to the string representation of the module uri (represented as a macro containing
`\stex_use_module_uri:n *` a triple as above). The variant `\stex_module_uri:n` expects a triple directly.

`\stex_module_uri_archive:N *`

expands to the archive ID of the module URI

`\stex_module_uri_path:N *` expands to the path of the module URI (possibly empty)
`\stex_module_uri_path:n *`

`\stex_module_uri_name:N` * expands to the name of the module URI
`\stex_module_uri_name:n` *

`\stex_module_uri_split_name:NNN`
`\stex_module_uri_split_name:NNn`

sets #1 as the parent module of the module URI and #2 as the last name

`\stex_module_uri_as_qm:n` *

`\stex_new_module_uri:n` * `\stex_new_module_uri:n` {*<name>*}

expands to a new module URI triple with the given *<name>* based on the current document URI or module URI, if existent

`\stex_uri_from_pair:Nnn` `\stex_uri_from_pair:Nnn` {*<archive id>*} {*<module path>*}

resolves the pair [*<archive id>*]{*<module path>*} to a module URI and stores the result in #1

`\stex_uri_from_pair:Nnn` like `\stex_uri_from_pair:Nnn`, but takes a token list as argument that may or may not be of the form [archive]module

14.3 Symbol URIs

`\stex_use_symbol_uri:N` * expands to the string representation of the module uri (represented as a macro containing a 4-tuple as above). The variant `\stex_symbol_uri:n` expects a 4-tuple directly.
`\stex_use_symbol_uri:n` *

`\stex_new_symbol_uri:n` * `\stex_new_symbol_uri:n` {*<name>*}

expands to a new symbol URI tuple with the given *<name>* based on the current module URI, which is assumed to exist(!).

`\stex_new_symbol_uri:nn` * expands to a new symbol URI tuple with the given *<name>* based on the *given* module URI (either as macro or as tuple), which is assumed to exist(!).

`\stex_symbol_uri_archive:N` *

expands to the archive ID of the symbol URI

`\stex_symbol_uri_path:N` * expands to the path of the symbol URI (possibly empty)

`\stex_symbol_uri_module:N` ★
`\stex_symbol_uri_module:n` ★

expands to the URI of the containing module of the symbol URI

`\stex_symbol_uri_name:N` ★ expands to the name of the symbol URI
`\stex_symbol_uri_name:n` ★

Chapter 15

Documents

`\STEXftmllink`
`\STEXlinkftml`
`\STEXsetlinkftml`

`\STEXlinkftml` inserts the following link/disclaimer:

*The **FTML** version of this document can be found at*
`http://unknown.source?a=no/archive&p=/home/jazzpirate/work/`
`Software/sTeX/sTeX/doc&d=stex-doc&l=en`

The precise text can be set using `\STEXsetlinkftml{<text>}`; The link on its own can be inserted using `\STEXftmllink`.

`\stexdoctitle` `\stexdoctitle {<title>}`

Sets `<title>` to be the title of the document. Only the first call of this command does something.

15.1 Sectioning

`\l_stex_current_section_level_int`
`\l_stex_current_section_level_str`

integer keeping track of the current sectioning level:

- 0 part
- 1 chapter
- 2 section
- 3 subsection
- 4 subsubsection
- 5 paragraph
- >5 subparagraph

`\setsectionlevel` sets `\l_stex_current_section_level_int` to the corresponding integer value. `\l_stex_current_section_level_str` stores a string representation of the same value, but will initially contain `document`.

`\currentsectionlevel` will leave the current section level as text in the token input. The variant `\Currentsectionlevel` will produce the capitalized version.

If the `xspace` package is loaded, this will insert an `\xspace` afterwards.

In HTML mode, this will insert an annotation, so it can be adapted dynamically.

`sfragment` (*env.*) A section at the current section level; preferred over the primitives `\section`, `\subsection`, etc., because a) environments are better suited for such structuring, and b) it allows for adapting the inserted section header based on document context, rather than it being hardcoded – e.g. what’s a subsection in one document may be a section in another.

`titlefragment` (*env.*) like `sfragment`, but will use `\maketitle` instead of (one of) the sectioning macros, if no title has been used yet.

`blindfragment` (*env.*) skips one sectioning level in its body so that e.g. subsection 0.1 can be used before the first section

`\skipfragment` Skips one counter at the current sectioning level; e.g. increase the subsection counter to 2 if called immediately after `\section` (or, rather, after `\begin{sfragment}`).

`\setsectionlevel` `\setsectionlevel {<name>}`

Sets the current section level to the one specified (e.g. `\setsectionlevel{subsection}`). Should ideally be called at most once in the preamble of a document.

15.2 Inter-Document References

[id=foo] sets the current fragment's (e.g. Definition 3.1 (Foo)) name to foo, resulting in a DocumentElementURI ..&e=foo. It then delegates to \label{...&e=foo} (which defines \r@...&e=foo), stores ::&e=foo in \g_stex_sref_label_foo, and writes \STeXInternalSRefLabel{foo}{...&e=foo}{definition.3.1}{Foo} to the sref file.

15.3 Inputting From MathHub Resources

\inputref \inputref [*archive id*] {*filepath*}

Inputs the referenced file in the referenced archive (or current archive, if empty) in a tex group, while setting all the relevant macros for the current archive, document URI, etc.

In HTML, will instead just insert an annotation, so that the HTML of the referenced document can be dynamically inserted instead.

\mhinput \inputref [*archive id*] {*filepath*}

Like \inputref, but *always* actually inputs the referenced file (also in HTML mode!) and without opening a tex group.

\ifinputref L^AT_EX2 conditional; is true, if we currently are in an \inputref or \mhinput.

\IfInputref \IfInputref {*true*} {*false*}

Executes *true* if we currently are in an \inputref or \mhinput, otherwise executes *false*.

In HTML, *both* branches are executed(!) and inserted in the HTML with corresponding attributes, so that the relevant parts can be shown or hidden.

\libinput \libinput [*archive id*] {*file name*}

inputs all files *file name*.tex in the lib directories of the current archive. Will throw an error if we are not in an archive or no suitable file is found.

\libusepackage \libusepackage [*package options*] {*package name*}

inputs all packages *package name*.sty in the lib directories of the current archive. Will throw an error if we are not in an archive, or there is not *exactly one* file *package name*.sty somewhere in the lib directories.

Note that unlike \libinput, the optional argument does *not* refer to an archive ID, since this would conflict with package options.

Will give a spurious warning that package mathhub/directory/foo was requested, but package foo was found. This is a side-effect of using absolute file paths, which is necessary to make the package findable in the first place.

\libusetikzlibrary \libusetikzlibrary [*archive id*] {*name*}

like \libusepackage, but for \usetikzlibrary.

\addmhbibresource	\addmhbibresource [<i>archive id</i>] { <i>file name</i> }
	Calls \addbibresource on all <i>file name</i> .bib files in the lib directories of the current archive. Will throw an error if we are not in an archive or no suitable file is found.
\mhgraphics \cmhgraphics	\mhgraphics adds the archive key to the optional arguments of \includegraphics to allow for using images in math archives (relative to its source directory). \cmhgraphics additionally wraps the image in a \begin{center} . Only defined if the graphicx package is loaded (but may be loaded after the stex package).
\lstinputmhlisting \clstinputmhlisting	like \mhgraphics , but for \lstinputlisting . Only defined if the listings package is loaded (but may be loaded after the stex package).
\mhtikzinput \cmhtikzinput	like \mhgraphics , but for \tikzinput . Only defined if the tikzinput package is loaded (but may be loaded after the stex package).

Chapter 16

Modules

`\l_stex_current_module_uri` stores the URI of the current module, if existent

`\l_stex_all_modules_seq` stores the URIs of all modules *currently in scope*.

`smodule (env.)` Parses the optional arguments, (if relevant) inserts HTML annotations, and then delegates to `\stex_module_setup:n`

`\stex_module_setup:n` Sets up a new module with the given name by checking whether it is a nested module, and if not, has a signature etc. Loads signature and metatheory, if necessary **TODO: deprecate in favor of every document being a module**

`\stex_module_setup_top_nosig:n` Sets up a new top-level module with the given name and no signature. Does not load a meta theory. **TODO: deprecate in favor of every document being a module**

`\stex_close_module:` closes the current module.

`\stex_every_module:n` adds code to be executed every time a new module is opened (e.g. activating certain macros).

`\stex_do_up_to_module:n` Execute code in the current module (i.e. as if the `\begin{<smodule>}` was the current
`\stex_do_up_to_module:e` tex group)

`\stex_execute_in_module:n` Execute code in the current module (i.e. as if the `\begin{<smodule>}` was the current tex
`\stex_execute_in_module:e` group) and also adds it to the initialization code of the module; i.e. exports all macros defined.

`\stex_if_in_module_p:` * conditional whether we currently are in a module. **TODO: deprecate in favor of every document being a module**
`\stex_if_in_module:` *TF* *

`\stex_if_module_exists_p:` *N* *
`\stex_if_module_exists:` *NTF* *
`\stex_if_module_exists_p:` *n* *
`\stex_if_module_exists:` *nTF* *

conditional whether a module with the given URI exists

`\stex_activate_module:` *N*
`\stex_activate_module:` *n*

`\setmetatheory` `\setmetatheory` [*archive id*] {*module*}

sets the metatheory of all subsequent modules to the specified one

`\STEXexport` `\STEXexport` {*code*}

executes the provided code every time the current module is opened (including immediately). Processes its content in the `\ExplSyntaxOn` category code scheme.

`\stex_structural_feature_module:` *nn*
`\stex_structural_feature_module_end:`

16.1 SMS Mode

`\STEX` has to extract formal content (i.e. modules and their symbols) from `\LaTeX`-files, that may otherwise contain arbitrary code, including macros that may not be defined unless the file is fully processed by `\TeX`. Those modules and symbols also may depend on other modules that have not yet been loaded. The naive way to achieve this, which would be to just suppress output (e.g. by storing it in a box register) and then `\input` the required file, does not work thanks to `\TeX`'s limited *file stack*, which would overflow quickly for modules that have a deeply nested list of dependencies.

To solve those problems, `\STEX` reads dependencies in what we call *sms mode*, which can be summarized thusly:

- In a first pass, we parse the file token by token, ignoring everything other than a select list of macros and environments that introduce dependencies (such as `\importmodule` and `\begin{smodule}[sig=...]`). Instead of loading those, we remember them for later.
- After the file has been fully parsed thusly, the dependencies found are loaded, again in sms-mode.
- In a second pass, we parse the file *again* in the same way, but this time execute all macros that are explicitly allowed in sms mode, such as `\importmodule`, `\symdecl`, `\notation`, `\symdef`, etc.

- all this parsing happens additionally in a `\setbox\throwaway\ vbox{...}`-block to suppress any accidental output.

This means that T_EX’s input stack never grows by more than +1, but still behaves *as if* the dependencies were loaded recursively, at the detriment of being somewhat slow.

`\stex_sms_allow:N` registers the provided macro to be allowed in sms mode.

This only works, if the macro takes no arguments and/or does not touch the subsequent tokens.

`\stex_sms_allow_escape:N` registers the provided macro to be allowed in sms mode.

If the macro is subsequently encountered in sms-mode, parsing is halted and it can process arguments as desired. It then needs to continue parsing manually though, by calling `\stex_smsmode_do:` as (usually) its last token.

`\stex_sms_allow_env:n` registers the provided environment to be allowed in sms mode.

As with `\stex_sms_allow_escape:N`, the `\begin{envname}` is escaped, hence the begin-code of the environment needs to call `\stex_smsmode_do:`. Since `\end{envname}` never takes arguments, it does not need to be escaped.

`\stex_sms_allow_import:Nn` `\stex_sms_allow_import:Nn \macro {<code>}`

registers the provided macro to be allowed in the *first pass* in sms mode. The provided `<code>` is executed at the start of the first pass, to define macros accordingly.

`\stex_sms_allow_import_env:nn` `\stex_sms_allow_import_env:nn \envname {<code>}`

registers the provided environment to be allowed in the *first pass* in sms mode. The provided `<code>` is executed at the start of the first pass, to define macros accordingly.

`\g_stex_sms_import_code` inserted between the first and second pass; macros/environments allowed via `\stex_sms_allow_import_*` should store their “results” in here.

`\stex_if_smsmode_p: *` tests for whether we are currently in sms-mode.
`\stex_if_smsmode:TF *`

`\stex_file_in_smsmode:Nn` `\stex_file_in_smsmode:Nn \document-URI {<file string>}`

Loads `{<file string>}` in sms mode, setting `\l_stex_current_document_uri` etc. accordingly and reverting them afterwards.

`\stex_smsmode_do:` should be called in escaped macros or environments allowed in sms mode; switches back to parsing file contents ignoring everything not explicitly allowed in sms mode.
 Does nothing outside of sms mode, so can be called safely.

16.2 Inheritance and Morphisms

```
\stex_require_module:N
\stex_require_module_noerr:N
\stex_require_module_noerr:n
```

makes sure that the module with the given URI is in scope. If not, it will attempt to load the module from disk. `\stex_require_module:N` throws an error if the module can't be found; `\stex_require_module_noerr:N` silently fails.

The `\*:n`-variant takes the URI tuple directly rather than a macro.

```
\usemodule \usemodule [<archive ID>] {<module>}
```

loads the specified module without exporting it further.

```
\importmodule \importmodule [<archive ID>] {<module>}
```

loads the specified module and exports it further.

```
\requiremodule \requiremodule [<archive ID>] {<module>}
```

loads the specified module without exporting it further, but generates FTML equivalent to `\importmodule`.

```
\stex_add_morphism:nnnn \stex_add_morphism:nnnn {<name>} {<module URI>} {<kind>} {<contents>}
```

adds a morphism to the current module

16.3 Theory Morphisms

```
\stex_structural_feature_morphism:nnnnn
\stex_structural_feature_morphism_with_macros:nnnnn
\stex_structural_feature_morphism_end:
```

```

#1 : name
#2 : kind
archive ID
#3 : module spec
#4 : additional attributes
```

```
\stex_iterate_morphisms:nn
```

```
\stex_get_in_morphism:n
```

```
copymodule (env.) (and \copymod)
```

```
interpretmodule (env.) (and \interpretmod)
```

\assign

\renamedekl

\assignMorphism

Chapter 17

Symbols

17.1 Declarations

`\symdecl` `\symdecl[*] {⟨iname⟩} [⟨options⟩]`

takes care of the optional arguments, styling, generates the symbol, defines the relevant macros and adds them to the current module.

`\symdef` `\symdef {⟨iname⟩} [⟨options⟩] {⟨notation⟩}`

`\textsymdecl`

`\stex_symdecl_do`: does the actual work. Requires all the `\l_stex_key_*` macros are set.

`\_stex_symdecl_html`: not namespaced so it can be reused in e.g. `sdefinitions` that generate new symbols. Requires that the relevant `\l_stex_key_*` macros and `\l_stex_macroname_str` are set.

`\stex_add_symbol:nnnnnnN` #1 : {⟨Macro name⟩}
 #2 : {⟨Name⟩}
 #3 : {⟨arity⟩}
 #4 : {(⟨Arg num⟩){⟨Arg str⟩}*}
 #5 : Definiens
 #6 : type
 #7 : Return
 #8 : Command
 adds a new symbol to the current module.

`\stex_has_definiens_p:N` ★
`\stex_has_definiens:NTF` ★
`\stex_has_definiens_p:n` ★
`\stex_has_definiens:nTF` ★

17.2 Retrieval

<code>\stex_iterate_symbols:n</code>	iterates over all symbols currently in scope. The provided code will receive nine arguments: The URI of the containing module and the eight parameters as in <code>\stex_add_symbol:nnnnnnnN</code> .
<code>\stex_iterate_break:</code>	
<code>\stex_iterate_break:n</code>	iteration is stopped in the code using <code>\stex_iterate_break:</code> or <code>\stex_iterate_break:n</code> .

<code>\stex_iterate_symbols:nn</code>	<code>\stex_iterate_symbols:nn {<modules>} {<code>}</code>
---------------------------------------	--

iterates over all symbols in the comma-separated list of module URIs in `<modules>`. The provided code will receive nine arguments: The URI of the containing module and the eight parameters as in `\stex_add_symbol:nnnnnnnN`.

iteration is stopped in the code using `\stex_iterate_break:` or `\stex_iterate_break:n`.

<code>\stex_get_symbol:n</code>	attempts to find the symbol with the given name/id/URI suffix. If not successful, will execute the F code or throw an error. Otherwise, defines the following macros:
<code>\stex_get_symbol:nTF</code>	
<code>\stex_get_symbol:nF</code>	

- `\l_stex_get_symbol_uri,`
- `\l_stex_get_symbol_arity_int,`
- `\l_stex_get_symbol_args_tl,`
- `\l_stex_get_symbol_def_tl,`
- `\l_stex_get_symbol_type_tl,`
- `\l_stex_get_symbol_return_tl,`
- `\l_stex_get_symbol_invoke_cs`

17.3 Variables

<code>\l_stex_variables_prop</code>	contains all variables currently in scope.
-------------------------------------	--

`\vardef`

`\newvar`

`\newseq`

`\stex_get_var:n`

17.3.1 Variable Sequences

`\varseq`

17.4 Expressions

`\symuse` retrieves a symbol by name/id and invokes it as if using a semantic macro.

`\_stex_next_symbol:n` code to be executed the next time a symbol is invoked.

`\_stex_invoke_symbol:NnnnnnN`
`\_stex_invoke_symbol:NeooooN`
`\_stex_invoke_symbol:nnnnnnN`

invokes the symbol. Takes as arguments the following data, and defines the corresponding macros accordingly:

- `\l_stex_current_symbol_uri`,
- `\l_stex_current_symbol_arity_int`,
- `\l_stex_current_symbol_args_tl`,
- `definiens` (currently not used),
- `\l_stex_current_symbol_type_tl`,
- `\l_stex_current_symbol_return_tl`,
- the invocation macro (is called after setup).

Also defines `\l_stex_current_full_tl` and `\l_stex_current_display_tl`

For a “normal” symbol defined via `\symdecl` or `\symdef`, `\l_stex_current_symbol_invoke_cs` is defined as `\stex_invoke_symbol:.`

Opens a new $\text{\TeX}$ group that needs to be closed by `\l_stex_current_symbol_invoke_cs!`

`\stex_invoke_symbol:` Invokes `\l_stex_current_symbol_uri`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_text_symbol:` Invokes `\l_stex_current_symbol_uri` as a symbol declared via `\textsymdecl`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_sequence:` Invokes `\l_stex_current_symbol_uri` as a sequence variable declared via `\varseq`; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\stex_invoke_structure:` Invokes a mathstructure symbol; assumes all the `\l_stex_current_*`-macros have been defined prior.

`\_stex_invoke_variable:nnnnnn`

invokes the variable. Takes as arguments the following data, and defines the corresponding macros accordingly:

- variable name,
- `\l_stex_current_symbol_arity_int`,
- `\l_stex_current_symbol_args_tl`,
- definiens (currently not used),
- `\l_stex_current_symbol_type_tl`,
- `\l_stex_current_symbol_return_tl`,
- the invocation macro (is called after setup).

`\svar`

17.4.1 Term Annotations

`\_stex_eat_exclamation_point:`

removes spurious ! characters

`\_stex_term_oms:nn` `\_stex_term_oms:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMID="\l_stex_current_symbol_uri"`

`\_stex_term_omv:nn` `\_stex_term_omv:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMV="\l_stex_current_symbol_uri"`
TODO: This shouldn't use `\l_stex_current_symbol_uri`!

`\_stex_term_oma:nn` `\_stex_term_oma:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMA="\l_stex_current_symbol_uri"`

`\_stex_term_omb:nn` `\_stex_term_omb:nn {<notation id>} {<code>}`
 annotates `<code>` with `OMBIND="\l_stex_current_symbol_uri"`

17.4.2 Symbol Arguments

`\stex_term_arg:nnnnn` marks an argument of a symbol application, sets the precedence accordingly, sets `\l_stex_allow_semantic_bool` etc.

#1 : argument number
 #2 : argument mode
 #3 : precedence
 #4 : argument name (WiP)
 #5 : code / body

`\stex_term_arg:nnn` `\stex_term_arg:nnn` `{\mode}` `{\num}` `{\code}`

inserts the HTML annotations for the argument

`\stex_term_arg_aB:nnnnn` takes care of sequence arguments

`\seqmap`

17.4.3 Notation components

`\comp`
`\compemph@uri`
`\compemph`

`\varemp@uri`
`\varemp`

`\defemph@uri`
`\defemph`

`\symrefemph@uri`
`\symrefemph`

The following commands do the actual attributes:

`\do_comp:nnNn` `\do_comp:nnNn` `{\key-suffix}` `{\html-value}` `\comp-macro` `{\code}`

annotated `\code` with `data-ftml-{\key-suffix}` using the highlighting dictated by `\comp-macro`.

17.4.4 Symbol References

`\symref`
`\sr`

`\symname`
`\sn`
`\sns`
`\Symname`
`\Sn\Sns`

`\varref`
`\varname`
`\Varname`

`\definiendum`
`\definame`
`\Definame`
`\defnotation`

`\foldexpr`

17.5 Checking

`\stex_if_check_terms_p:` $\star$ conditional whether terms (types and definientia of symbols) should get syntactically
`\stex_if_check_terms:` $\underline{TF}$ $\star$ checked. In HTML mode, this is always false, since they get written to the HTML
 anyway.

`\stex_check_term:nn` typesets the given expression in a `\tiny\marginpar`, checking its syntactic validity. Does
 nothing, if term checking is not explicitly activated.

`\_stex_check_terms:` checks the type and definiens components of a symbol declaration (i.e. `\stex_key_-`
`type_tl`, `\stex_key_def_tl` and `\stex_key_return_tl!`).

Chapter 18

Notations

`\notation`

`\varnotation`

`\stex_notation_parse:n` parses the body of a notations and saves it in `\l_stex_notation_macrocode_cs`

`\_stex_notation_add:` adds the fully parsed notation to the current module

<code>\stex_add_notation:nnnnn</code>	#1 :	URI
<code>\stex_add_notation:ooeoo</code>	#2 :	variant
	#3 :	arity
	#4 :	macro body
	#5 :	op
		adds the notation to the current module

`\_stex_map_args:N`
`\_stex_map_notation_args:N`

`\_stex_var_notation_macro:`

`\setnotation`
`\_stex_notation_set_default:n`

<code>\stex_iterate_notations:nn</code>	<code>\stex_iterate_notations:nn {<modules>} {<code>}</code>
---	--

iterates over all notations in the comma-separated list of module URIs and executes `{<code>}` with #1,#2,#3,#4,#5 as the notation parameters.

<code>\stex_use_notation:nnTF</code>	<code>\stex_use_notation:nn {<uri>} {<id>} {<exists code>} {<missing code>}</code>
--------------------------------------	--

<code>\stex_use_op_notation:nnTF</code>	sets <code>\l_stex_notation_cs</code>
---	---------------------------------------

<code>_stex_notation_check:</code>

<code>_stex_notation_do_html:n</code>
--

<code>_stex_notation_make_args:</code>

<code>\stex_do_default_notation:</code>
<code>\stex_do_default_notation_op:</code>

<code>\STEXInternalNotation</code>	#1 : notation id
	#2 : operator precedence
	#3 : intent (WiP)
	#4 : arguments
	#5 : notation code
	#6 : continuation

<code>\argsep</code>

<code>\argmap</code>

<code>\argarraymap</code>

18.1 Automated Bracketing

<code>\infprec</code>	infinite and negative infinite precedences.
-----------------------	---

<code>\neginfprec</code>

<code>_stex_maybe_brackets:nn</code>	<code>_stex_maybe_brackets:nn {<prec>} {<body>}</code>
---------------------------------------	---

inserts parentheses around `<body>` iff precedences and context call for it.

`\dobrackets` actually inserts parentheses around its argument.

`\withbrackets` `\withbrackets` $\{\langle left \rangle\} \{\langle right \rangle\} \{\langle body \rangle\}$
sets $\langle left \rangle$ and $\langle right \rangle$ as the parentheses to use in $\langle body \rangle$.

`\dowithbrackets` `\dowithbrackets` $\{\langle left \rangle\} \{\langle right \rangle\} \{\langle body \rangle\}$
combines `\withbrackets` and `\dobrackets`.

Chapter 19

Structures

```
mathstructure (env.)  
extstructure (env.)
```

```
\this
```

```
\stex_get_mathstructure:n sets \l_stex_get_structure_module_uri or throws an error if no structure by the given  
name/id exists.
```

```
\usestructure
```

Chapter 20

Statements

`\stex_do_for_list:` resolves each id in the `for={...}` comma-separated list of ids `\l_stex_key_for_clist`. Stores the full resolved URIs in `\l_stex_fors_seq`.

`\stex_new_statement:nnn` `\stex_new_statement:nnn {<env name>} {<macro name>} {<code>}`
defines a new statement environment `s<env name>` and the optional macro-variant `\inline<macro name>` (may be empty). `{<code>}` does arbitrary additional setup and is called in the `\begin` part of the environment and the macro after optional arguments have been parsed. e.g. `\stex_new_statement:nnn{definition}{def}{...}` defines the `sdefinition` environment and the `\inlinedef` macro.

`sdefinition (env.)` (and `\inlinedef`)

`sassertion (env.)` (and `\inlineass`)

`sexample (env.)` (and `\inlineex`)

`sparagraph (env.)`

`definiens`

`\stex_add_definiens:nn` marks #2 as the definiens of the symbol with uri #1. Also marks the symbol as being defined, iff the symbol is declared in the current module.

`\varbind`

`\conclusion`

`\premise`

Chapter 21

Proofs

`sproof (env.)`

`subproof (env.)`

`spfsketchenv (env.)`

`\spfsketch`

`spfstepenv (env.)`

`spfblock (env.)`

`\spfstep`

`\assumption`

`\conclude`

`\eqstep`

`\yield`

`\spfjust`

`\spfby`

\spfarg

\sproofend

\stexcommentfont

Chapter 22

Metatheory

TODO

Chapter 23

Others

Chapter 24

Additional Packages

24.1 NotesSlides Documentation

TODO

24.2 Problem Documentation

TODO

24.3 HWExam Documentation

TODO

24.4 Tikzinput Documentation

TODO

Part IV

Implementation

Chapter 25

Setting up

Setup code for the document class

```
1 <*cls>
2 %%%%%%%%% stex.dtx %%%%%%%%%
3
4 \RequirePackage{expl3,l3keys2e}
5 \ProvidesExplClass{stex}{2026/06/24}{4.1.0}{sTeX document class}
6 \IfFileExists{stex-expl-compat.sty}{
7   \usepackage{stex-expl-compat}
8 }{}
9
10 \DeclareOption*{\PassOptionsToPackage{\CurrentOption}{stex}}
11 \ProcessOptions
12
13 \RequirePackage{stex}
14 \str_set:Nn \g_stex_document_kind_str{fragment}
15
16 \LoadClass{article}
17 </cls>
```

Setup code for the package

```
18 <*package>
19 \RequirePackage{expl3,l3keys2e,ltxcmds}
20 \ProvidesExplPackage{stex}{2026/06/24}{4.1.0}{sTeX package}
21 \IfFileExists{stex-expl-compat.sty}{
22   \usepackage{stex-expl-compat}
23 }{}
24 \RequirePackage{stex-logo} % externalized for backwards-compatibility reasons
25 \RequirePackage{standalone}
26
27 \bool_new:N \c_stex_use_sref_bool
28 \message{^^J*~This~is~sTeX~version~4.1.0~*^^J}
```

Package options:

```
29 \keys_define:nn { stex / package } {
30   debug      .str_set_x:N = \c_stex_debug_clist ,
31   lang       .clist_set:N = \c_stex_languages_clist ,
32   mathhub    .tl_set_x:N = \mathhub ,
33   usesms     .bool_set:N = \c_stex_persist_mode_bool ,
```

```

34  writesms .bool_set:N = \c_stex_persist_write_mode_bool ,
35  forcenousesms .bool_set:N = \c_stex_persist_force_bool ,
36  checkterms .bool_set:N = \c_stex_check_terms_bool ,
37  metadata .bool_set:N = \c_stex_metadata_bool ,
38  image .bool_set:N = \c_tikzinput_image_bool,
39  nofrontmatter .bool_set:N = \c_stex_no_frontmatter_bool,
40  unknown .code:n = {}
41 }
42 \exp_args:NNo \clist_set:Nn \c_stex_debug_clist \c_stex_debug_clist
43 \ProcessKeysOptions { stex / package }

Error messages:
44 \input{stex-en.ldf}

```

Chapter 26

Utilities

`\stex_ignore_spaces_and_pars:`

```
45 \cs_new_protected:Nn \stex_ignore_spaces_and_pars:{
46   \begingroup\catcode13=10\relax
47   \@ifnextchar\par{
48     \endgroup\expandafter\stex_ignore_spaces_and_pars:\@gobble
49   }{
50     \endgroup
51   }
52 }
```

(End of definition for \stex_ignore_spaces_and_pars:. This function is documented on page 123.)
sfunction

`\stex_undefine:N`

`\stex_undefine:c`

```
53 \cs_new_protected:Nn \stex_undefine:N {
54   \cs_set_eq:NN #1 \tex_undefined:D
55 }
56 \cs_generate_variant:Nn \stex_undefine:N {c}
```

(End of definition for \stex_undefine:N. This function is documented on page 123.)
sfunction

`\stex_str_if_starts_with_p:nn`

`\stex_str_if_starts_with:nnTF`

```
57 \prg_new_conditional:Nnn \stex_str_if_starts_with:nn {p,T,F,TF} {
58   \exp_args:Ne \str_if_eq:nnTF {
59     \str_range:nnn{#1}{1}{\str_count:n{#2}}
60   }{#2}\prg_return_true: \prg_return_false:
61 }
```

(End of definition for \stex_str_if_starts_with:nnTF. This function is documented on page 123.)
sfunction

`\stex_str_if_ends_with_p:nn`

`\stex_str_if_ends_with:nnTF`

```
62 \prg_new_conditional:Nnn \stex_str_if_ends_with:nn {p,T,F,TF} {
63   \exp_args:Ne \str_if_eq:nnTF {
64     \str_range:nnn{#1}{- \str_count:n{#2}}{-1}
65   }{#2}\prg_return_true: \prg_return_false:
66 }
```

(End of definition for `\stex_str_if_ends_with:nnTF`. This function is documented on page 123.)

```
sfunction
```

```
\stex_deactivate_macro:Nn
\stex_reactivate_macro:N
67 \cs_new_protected:Nn \stex_deactivate_macro:Nn {
68   \tl_set_eq:cN{\tl_to_str:n{#1}---orig}#1
69   \cs_set_protected:Npn#1{
70     \msg_error:nnnn{stex}{error/deactivated-macro}{#1}{#2}
71   }
72 }
73 \cs_new_protected:Nn \stex_reactivate_macro:N {
74   \cs_set_eq:Nc #1{\tl_to_str:n{#1}---orig}
75 }
```

(End of definition for `\stex_deactivate_macro:Nn` and `\stex_reactivate_macro:N`. These functions are documented on page 123.)

```
sfunction
```

26.1 kpsewhich and Environment Variables

```
76 <@@=stex_kpse>
```

```
\stex_kpsewhich:Nn
77 \cs_new_protected:Nn \stex_kpsewhich:Nn {
78   \group_begin:
79   \catcode'\ =12
80   \sys_get_shell:nnN { kpsewhich ~ #2 } { } \l_tmpa_tl
81   \tl_gset_eq:NN \l_tmpa_tl \l_tmpa_tl
82   \group_end:
83   \exp_args:NNo\str_set:Nn #1 \l_tmpa_tl
84   \tl_trim_spaces:N #1
85 }
```

(End of definition for `\stex_kpsewhich:Nn`. This function is documented on page 123.)

```
sfunction
```

```
\stex_get_env:Nn
86 \sys_if_platform_windows:TF{
87   \cs_new_protected:Nn \stex_get_env:Nn {\group_begin:
88     \escapechar=-1\catcode'\ =12
89     \exp_args:NNe \stex_kpsewhich:Nn #1 {-expand-var~\c_percent_str#2\c_percent_str}
90     \exp_args:NNx \str_if_eq:VnT #1 {\c_percent_str #2 \c_percent_str}{
91       \str_clear:N #1
92     }
93     \exp_args:NNx\use:nn\group_end:{
94       \str_set:Nn \exp_not:N #1 { #1 }
95     }
96   }
97 }{
98   \cs_new_protected:Nn \stex_get_env:Nn {
99     \stex_kpsewhich:Nn #1 {-var-value~#2}
100   }
101 }
```

(End of definition for `\stex_get_env:Nn`. This function is documented on page 124.)

sfunction

26.2 Logging

```

102 <@@=stex_debug>

\stex_debug:nn

103 \cs_new_protected:Nn \stex_debug:nn {
104   \exp_args:NNo \clist_if_in:NnTF \c_stex_debug_clist { \tl_to_str:n{all} }{
105     \__stex_debug_:nn{#1}{#2}
106   }{
107     \exp_args:NNo \clist_if_in:NnT \c_stex_debug_clist { \tl_to_str:n{#1} }{
108       \__stex_debug_:nn{#1}{#2}
109     }
110   }
111 }

112
113 \cs_new_protected:Nn \__stex_debug_:nn {
114   \msg_set:nnn{stex}{debug / #1}{
115     \Debug~#1:~#2\
116   }
117   \msg_none:nn{stex}{debug / #1}
118 }

We check an environment variable for debugging and set things up:

119 \stex_get_env:Nn\__stex_debug_env_str{STEX_DEBUG}
120 \str_if_empty:NnTF\__stex_debug_env_str {
121   \clist_set_eq:NN \l__stex_debug_cl \c_stex_debug_clist
122 }{
123   \clist_set:No \l__stex_debug_cl {\__stex_debug_env_str}
124 }
125 \clist_clear:N \c_stex_debug_clist
126 \clist_map_inline:Nn \l__stex_debug_cl {
127   \exp_args:NNo \clist_put_right:Nn \c_stex_debug_clist
128   { \tl_to_str:n{#1} }
129 }
130
131 \exp_args:NNo \clist_if_in:NnTF \c_stex_debug_clist {\tl_to_str:n{all}} {
132   \msg_redirect_module:nnn{ stex }{ none }{ warning }
133   \stex_debug:nn{all}{Logging-everything!}
134 }{
135   \clist_map_inline:Nn \c_stex_debug_clist {
136     \msg_redirect_name:nnn{ stex }{ debug / #1 }{ warning }
137     \stex_debug:nn{#1}{Logging~#1}
138   }
139 }
```

(End of definition for `\stex_debug:nn`. This function is documented on page 124.)

sfunction

26.3 File Paths

```

140 <@@=stex_path>

\stex_file_set:Nn
\stex_file_set:No
\stex_file_set:Ne
141 \cs_new_protected:Nn \stex_file_set:Nn {
142   \str_if_empty:nTF {#2} { \seq_clear:N #1 }{
143     \exp_args:NNno \seq_set_split:Nnn #1 / { \tl_to_str:n{#2} }
144   }
145 }
146 \cs_generate_variant:Nn \stex_file_set:Nn {No, Ne}

(End of definition for \stex_file_set:Nn. This function is documented on page 124.)
sfunction

\stex_file_resolve:Nn
\stex_file_resolve:No
\stex_file_resolve:Ne
147 \sys_if_platform_windows:TF{
148   \cs_new_protected:Npn \__stex_path_win_take:w #1#2#3 \__stex_path_: {
149     \uppercase{ \str_set:Nn \l__stex_path_str{#1}}
150     \str_set:Ne \l__stex_path_win_drive {\l__stex_path_str #2}
151     \str_set:Nn \l__stex_path_str{#3}
152   }
153   \cs_new_protected:Nn \stex_file_resolve:Nn {
154     \str_set:Nn \l__stex_path_str {#2}
155     \str_clear:N \l__stex_path_win_drive
156     \exp_args:NNno \str_replace_all:Nnn \l__stex_path_str \c_backslash_str /
157     \exp_args:Ne \str_if_eq:nnT {\str_item:Nn \l__stex_path_str 2} : {
158       \exp_after:wN \__stex_path_win_take:w \l__stex_path_str \__stex_path_:
159     }
160     \stex_file_set:No #1 \l__stex_path_str
161     \__stex_path_canonicalize:N #1
162     \str_if_empty:NF \l__stex_path_win_drive {
163       \seq_pop_left:NN #1 \l__stex_path_str
164       \seq_put_left:No #1 \l__stex_path_win_drive
165     }
166     %\stex_debug:nn{files}{Set-\tl_to_str:n{#1}~to~\stex_file_use:N #1}
167   }
168 }{
169   \cs_new_protected:Nn \stex_file_resolve:Nn {
170     \str_set:Nn \l__stex_path_str {#2}
171     \stex_file_set:No #1 \l__stex_path_str
172     \__stex_path_canonicalize:N #1
173     % \stex_debug:nn{files}{Set-\tl_to_str:n{#1}~to~\stex_file_use:N #1}
174   }
175 }
176 \cs_generate_variant:Nn \stex_file_resolve:Nn {No, Ne}

Auxiliary methods:
177 \cs_new_protected:Nn \__stex_path_canonicalize:N {
178   \seq_if_empty:NF #1 {
179     \seq_pop:NN #1 \l__stex_path_can_str
180     \seq_clear:N \l__stex_path_can_seq
181     \str_if_empty:NTF \l__stex_path_can_str {
182       \seq_map_function:NN #1 \__stex_path_dodots:n
183       \seq_put_left:Nn \l__stex_path_can_seq {}

```

```

184     }{
185       \seq_push:No #1 \l__stex_path_can_str
186       \seq_map_function:NN #1 \__stex_path_dodots:n
187     }
188     \seq_set_eq:NN #1 \l__stex_path_can_seq
189   }
190 }
191
192 \cs_new_protected:Nn \__stex_path_dodots:n {
193   \str_if_empty:nF{#1}{
194     \str_if_eq:nnF {#1} {.} {
195       \str_if_eq:nnTF {#1} {...} {
196         \seq_if_empty:NF \l__stex_path_can_seq {
197           \seq_pop_right:NN \l__stex_path_can_seq \l__stex_path_can_str
198         }
199       }{
200         \seq_put_right:Nn \l__stex_path_can_seq {#1}
201       }
202     }
203   }
204 }

```

(End of definition for `\stex_file_resolve:Nn`. This function is documented on page 124.)

sfunction

`\stex_file_use:N`

```

205 \cs_new:Nn \stex_file_use:N {
206   \seq_use:Nn #1 /
207 }

```

(End of definition for `\stex_file_use:N`. This function is documented on page 124.)

sfunction

`\stex_file_split_off_ext:NN`

```

208 \cs_new_protected:Nn \stex_file_split_off_ext:NN {
209   \seq_set_eq:NN #1 #2
210   \seq_pop_right:NN #1 \l__stex_path_str
211   \seq_set_split:NnV \l__stex_path_seq . \l__stex_path_str
212   \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
213   \seq_put_right:Ne #1 {\seq_use:Nn \l__stex_path_seq .}
214 }

```

(End of definition for `\stex_file_split_off_ext:NN`. This function is documented on page 124.)

sfunction

`\stex_file_split_off_lang:NN`

```

215 \cs_new_protected:Nn \stex_file_split_off_lang:NN {
216   \seq_set_eq:NN #1 #2
217   \seq_pop_right:NN #1 \l__stex_path_str
218   \seq_set_split:NnV \l__stex_path_seq . \l__stex_path_str
219   \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
220
221   \seq_pop_right:NN \l__stex_path_seq \l__stex_path_str
222   \exp_args:NNo \prop_if_in:NnF \c_stex_languages_prop \l__stex_path_str {

```

```

223   \seq_put_right:No \l__stex_path_seq \l__stex_path_str
224 }
225
226   \seq_put_right:Ne #1 {\seq_use:Nn \l__stex_path_seq .}
227 }

```

(End of definition for `\stex_file_split_off_lang:NN`. This function is documented on page 124.)

sfunction

```

\c_stex_pwd_file We determine the pwd
\c_stex_main_file
228 \sys_if_platform_windows:TF{
229   \stex_get_env:Nn \l__stex_path_str{CD}
230 }{
231   \stex_get_env:Nn \l__stex_path_str{PWD}
232 }
233 \stex_file_resolve:No \c_stex_pwd_file \l__stex_path_str
234 \seq_set_eq:NN \c_stex_main_file \c_stex_pwd_file
235 \seq_put_right:Ne \c_stex_main_file {\jobname\tl_to_str:n{.tex}}
236
237 \stex_debug:nn {files} {PWD:~\stex_file_use:N \c_stex_pwd_file}

```

(End of definition for `\c_stex_pwd_file` and `\c_stex_main_file`. These variables are documented on page 124.)

```

\c_stex_home_file
238 \sys_if_platform_windows:TF{
239   \stex_get_env:Nn \l__stex_path_str {homedrive\c_percent_str\c_percent_str homedrive}
240 }{
241   \stex_get_env:Nn \l__stex_path_str {HOME}
242 }
243 \stex_file_resolve:No \c_stex_home_file \l__stex_path_str

```

(End of definition for `\c_stex_home_file`. This variable is documented on page 125.)

```

\g_stex_current_file
244 \seq_gset_eq:NN \g_stex_current_file \c_stex_main_file

```

(End of definition for `\g_stex_current_file`. This variable is documented on page 125.)

```

\stex_input_with_hooks:Nn
\stex_input_with_hooks:Ne
245 \cs_new_protected:Nn \stex_input_with_hooks:Nn {
246   \stex_with_file_hooks:Nnn #1 {#2} {\input{#2}}
247 }
248 \cs_generate_variant:Nn \stex_input_with_hooks:Nn {Ne}

```

(End of definition for `\stex_input_with_hooks:Nn`. This function is documented on page 125.)

sfunction

```

\stex_with_file_hooks:Nnn
249 \cs_new_protected:Nn \stex_with_file_hooks:Nnn {
250   \stex_pseudogroup:nn {
251     \stex_file_resolve:Nn \l__stex_path_seq {#2}
252     \seq_gset_eq:NN \g_stex_current_file \l__stex_path_seq
253     \tl_set_eq:NN \l_stex_current_document_uri #1
254     \str_set:Ne \l_stex_current_language_str { \stex_document_uri_language:N \l_stex_current

```

```

255     #3
256   }{
257     \tl_gset:Nn \exp_not:N \g_stex_current_file { \exp_args:No \exp_not:n \g_stex_current_f
258     \stex_pseudogroup_restore:N \l_stex_current_language_str
259     \stex_pseudogroup_restore:N \l_stex_current_document_uri
260   }
261 }

```

(End of definition for \stex_with_file_hooks:Nnn. This function is documented on page 125.)

sfunction

\stex_if_file_absolute_p:N
\stex_if_file_absolute:NTF

```

262 \sys_if_platform_windows:TF {
263   \prg_new_conditional:Nnn \stex_if_file_absolute:N {p, T, F, TF} {
264     \seq_if_empty:NTF #1 \prg_return_false: {
265       \exp_args:Ne \tl_if_empty:nTF {\seq_item:Nn #1 1} \prg_return_true: {
266         \exp_args:Ne \str_if_eq:nnTF { \exp_args:Ne \str_item:nn {\seq_item:Nn #1 1} 2} :
267         \prg_return_true: \prg_return_false:
268       }
269     }
270   }
271 }{
272   \prg_new_conditional:Nnn \stex_if_file_absolute:N {p, T, F, TF} {
273     \seq_if_empty:NTF #1 \prg_return_false: {
274       \exp_args:Ne \tl_if_empty:nTF {\seq_item:Nn #1 1}
275       \prg_return_true: \prg_return_false:
276     }
277   }
278 }

```

(End of definition for \stex_if_file_absolute:NTF. This function is documented on page 125.)

sfunction

\stex_if_file_starts_with:NNTF

```

279 \prg_new_protected_conditional:Nnn \stex_if_file_starts_with:NN {T,F,TF} {
280   \seq_set_eq:NN \l__stex_path_a_seq #1
281   \seq_set_eq:NN \l__stex_path_b_seq #2
282   \tl_clear:N \l__stex_path_return_tl
283   \bool_while_do:nn{
284     \bool_not_p:n{
285       \bool_lazy_any_p:n{
286         {\seq_if_empty_p:N \l__stex_path_a_seq}
287         {\seq_if_empty_p:N \l__stex_path_b_seq}
288         {\bool_not_p:n{\tl_if_empty_p:N \l__stex_path_return_tl}}
289       }
290     }
291   }{
292     \seq_pop_left:NN \l__stex_path_a_seq \l__stex_path_a_tl
293     \seq_pop_left:NN \l__stex_path_b_seq \l__stex_path_b_tl
294     \str_if_eq:NNF \l__stex_path_a_tl \l__stex_path_b_tl {
295       \tl_set:Nn \l__stex_path_return_tl {\prg_return_false:}
296     }
297   }
298   \tl_if_empty:NTF \l__stex_path_return_tl {

```

```

299     \seq_if_empty:NTF \l__stex_path_b_seq \prg_return_true: \prg_return_false:
300   } \l__stex_path_return_tl
301 }

```

(End of definition for `\stex_if_file_starts_with:NNTF`. This function is documented on page 125.)

```
sfunction
```

26.4 Group-like Behaviours

```

302 <@@=stex_groups>

```

`\stex_pseudogroup:nn`

```

303 \cs_new_protected:Npn \stex_pseudogroup:nn {
304   \exp_args:Nne \use:nn
305 }

```

(End of definition for `\stex_pseudogroup:nn`. This function is documented on page 125.)

```
sfunction
```

`\stex_pseudogroup_restore:N`

```

306 \cs_new:Nn \stex_pseudogroup_restore:N {
307   \tl_if_exist:NTF #1 {
308     \tl_set:Nn \exp_not:N #1 { \exp_args:No \exp_not:n #1 }
309   }{
310     \stex_undefine:N \exp_not:N #1
311   }
312 }

```

(End of definition for `\stex_pseudogroup_restore:N`. This function is documented on page 126.)

```
sfunction
```

`\stex_pseudogroup_with:nn`

```

313 \cs_new_protected:Nn \stex_pseudogroup_with:nn {
314   \tl_map_inline:nn{#1}{
315     \cs_set_eq:cN{__stex_groups_\tl_to_str:n{##1}}##1
316   }
317   #2
318   \tl_map_inline:nn{#1}{
319     \cs_set_eq:Nc##1{__stex_groups_\tl_to_str:n{##1}}
320     \stex_undefine:c{__stex_groups_\tl_to_str:n{##1}}
321   }
322 }

```

(End of definition for `\stex_pseudogroup_with:nn`. This function is documented on page 126.)

```
sfunction
```

`\stex_metagroup_new:` Current metagroup group level

```

323 \int_new:N \l__stex_groups_lvl_int

```

start a new metagroup at the current group level

```

324 \cs_new_protected:Nn \stex_metagroup_new: {
325   \int_set_eq:NN \l__stex_groups_lvl_int \currentgrouplevel
326 }

```

(End of definition for `\stex_metagroup_new:`. This function is documented on page 126.)

sfunction

```

\stex_metagroup_do_in:n
\stex_metagroup_do_in:e
327 \cs_new_protected:Npn \stex_metagroup_do_in:n {
328   \int_compare:nNnTF \l__stex_groups_lvl_int = \currentgrouplevel
329     \use:n \__stex_groups_do_in:n
330 }
331 \cs_generate_variant:Nn \stex_metagroup_do_in:n {e}
332
333 \cs_new_protected:Nn \__stex_groups_do_in:n {
334   \tl_if_exist:cTF{g__stex_groups\_the\currentgrouplevel\_tl}{
335     \exp_args:Nno \tl_gput_right:cn{g__stex_groups\_the\currentgrouplevel\_tl}
336   }{
337     \exp_args:Nno \tl_gset:cn{g__stex_groups\_the\currentgrouplevel\_tl}
338   }{ #1 }
339   \bool_if_exist:cF {l__stex_groups\_the\currentgrouplevel\_bool} {
340     \group_insert_after:N \__stex_groups_do:
341     \bool_set_true:c {l__stex_groups\_the\currentgrouplevel\_bool}
342   }
343   #1
344 }
345
346 \cs_new_protected:Nn \__stex_groups_do: {
347   \tl_if_exist:cT{g__stex_groups\_int_eval:n{\currentgrouplevel+1}_tl}{
348     \exp_args:Nno \exp_args:No \stex_metagroup_do_in:n {
349       \csname g__stex_groups\_int_eval:n{\currentgrouplevel+1}_tl \endcsname
350     }
351     \cs_undefine:c{g__stex_groups\_int_eval:n{\currentgrouplevel+1}_tl}
352   }
353   \bool_if_exist:cF {l__stex_groups\_the\currentgrouplevel\_bool} {
354     \group_insert_after:N \__stex_groups_do:
355     \bool_set_true:c {l__stex_groups\_the\currentgrouplevel\_bool}
356   }
357 }

```

(End of definition for `\stex_metagroup_do_in:n`. This function is documented on page 126.)

sfunction

26.5 Key Handling

358 $\langle @@=\text{stex_keys} \rangle$

`\stex_keys_define:nnnn`

```

359 \cs_new_nopar:Nn \stex_keys_define:nnnn {
360   \tl_gset:cn {__stex_keys_keys_#1_pre_tl}{#2}
361   \tl_gset:cn {__stex_keys_keys_#1_def_tl}{#3}
362   \tl_if_empty:nF{#4}{
363     \clist_map_inline:nn{#4}{
364       \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_##1_pre_tl}
365       \tl_gput_left:co{__stex_keys_keys_#1_pre_tl} \l__stex_keys_tl
366       \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_##1_def_tl}
367       \tl_gput_left:cn{__stex_keys_keys_#1_def_tl} ,

```

```

368     \tl_gput_left:co{__stex_keys_keys_#1_def_tl} \l__stex_keys_tl
369   }
370 }
371 \tl_set_eq:Nc \l__stex_keys_tl {__stex_keys_keys_#1_def_tl}
372 \exp_args:Nno \keys_define:nn {stex / #1} {\l__stex_keys_tl}
373 }

```

(End of definition for `\stex_keys_define:nnnn`. This function is documented on page 126.)

sfunction

`\stex_keys_set:nn`

```

374 \cs_new_nopar:Nn \stex_keys_set:nn {
375   \use:c{__stex_keys_keys_#1_pre_tl}
376   \keys_set:nn {stex / #1} { #2 }
377 }

```

(End of definition for `\stex_keys_set:nn`. This function is documented on page 126.)

sfunction

Some ubiquitous key sets:

```

378 \stex_keys_define:nnnn{archive}{file}{
379   \str_clear:N \l_stex_key_archive_str
380   \str_clear:N \l_stex_key_file_str
381 }{
382   archive .str_set_x:N = \l_stex_key_archive_str ,
383   file     .str_set_x:N = \l_stex_key_file_str
384 }{}
385
386 \stex_keys_define:nnnn{id}{
387   \str_clear:N \l_stex_key_id_str
388 }{
389   id .str_set_x:N = \l_stex_key_id_str
390 }{}
391
392 \stex_keys_define:nnnn{title}{
393   \tl_clear:N \l_stex_key_title_tl
394 }{
395   title .tl_set:N = \l_stex_key_title_tl
396 }{}
397
398 \stex_keys_define:nnnn{style}{
399   \clist_clear:N \l_stex_key_style_clist
400 }{
401   style .clist_set:N = \l_stex_key_style_clist
402 }{}
403
404 \stex_keys_define:nnnn{deprecate}{
405   \str_clear:N \l_stex_key_deprecate_str
406 }{
407   deprecate .str_set_x:N = \l_stex_key_deprecate_str
408 }{}
409
410 \stex_keys_define:nnnn{uses}{}{
411   uses .code:n = {
412     \clist_map_inline:nn{#1}{

```

```

413     \stex_str_if_starts_with:nnTF{##1} [{
414         \__stex_keys_split_at_bracket:w ##1 \stex_end:
415     } {
416         \usemodule{##1}
417     }
418 }
419 }
420 } {}
421
422 \cs_new_protected:Npn \__stex_keys_split_at_bracket:w [ #1 ] #2 \stex_end: {
423     \usemodule[##1]{##2}
424 }

```

26.6 Languages

```

425 <@@=stex_lang>

```

`\c_stex_languages_prop`

`\c_stex_language_abbrevs_prop`

```

426 \exp_args:NNx \prop_const_from_keyval:Nn \c_stex_languages_prop { \tl_to_str:n {
427     en = english ,
428     de = ngerman ,
429     ar = arabic ,
430     bg = bulgarian ,
431     ru = russian ,
432     fi = finnish ,
433     ro = romanian ,
434     tr = turkish ,
435     fr = french ,
436     sl = slovenian
437 }}
438
439 \exp_args:NNx \prop_const_from_keyval:Nn \c_stex_language_abbrevs_prop { \tl_to_str:n {
440     english    = en ,
441     ngerman    = de ,
442     arabic     = ar ,
443     bulgarian = bg ,
444     russian    = ru ,
445     finnish    = fi ,
446     romanian   = ro ,
447     turkish    = tr ,
448     french     = fr ,
449     slovenian  = sl ,
450 }}
451 % todo: chinese simplified (zhs)
452 %         chinese traditional (zht)

```

(End of definition for `\c_stex_languages_prop` and `\c_stex_language_abbrevs_prop`. These variables are documented on page 127.)

`\l_stex_current_language_str`

```

453 \str_new:N \l_stex_current_language_str

```

(End of definition for `\l_stex_current_language_str`. This variable is documented on page 127.)

Loading babel (if necessary), depending on the `lang` package option:


```

454 \clist_if_empty:NF \c_stex_languages_clist {
455   \bool_set_false:N \l__stex_lang_turkish_bool
456   \seq_clear:N \l_tmpa_seq
457   \clist_map_inline:Nn \c_stex_languages_clist {
458     \str_if_empty:NF \l_stex_current_language_str {
459       \str_set:Nn \l_stex_current_language_str {#1}
460     }
461     \str_set:Ne \l_tmpa_str {#1}
462     \str_if_eq:nnT {#1}{tr}{
463       \bool_set_true:N \l__stex_lang_turkish_bool
464     }
465     \prop_get:NoNTF \c_stex_languages_prop \l_tmpa_str \l_tmpa_str {
466       \tl_set_rescan:Nno \l_tmpa_str {} \l_tmpa_str
467       \seq_put_right:No \l_tmpa_seq \l_tmpa_str
468     } {
469       \msg_error:nmx{stex}{error/unknownlanguage}{\l_tmpa_str}
470     }
471   }
472   \stex_debug:nn{lang} {Languages:~\seq_use:Nn \l_tmpa_seq {,~} }
473   \bool_if:NTF \l__stex_lang_turkish_bool {
474     \exp_args:NNe \use:nn \RequirePackage
475     {[main=\seq_use:Nn \l_tmpa_seq, ,shorthands=:!]}{babel}
476   }{
477     \exp_args:NNe \use:nn \RequirePackage
478     {[main=\seq_use:Nn \l_tmpa_seq, ]}{babel}
479   }
480 }

```

26.7 Styling

```

481 <@@=stex_styles>

```

```

\stex_new_stylable_cmd:nnnn

```

```

\stex_style_apply:

```

```

482 \cs_new_protected:Nn \stex_new_stylable_cmd:nnnn {
483   \exp_after:wN \newcommand \cs:w stexstyle#1 \cs_end:[2] [] {
484     \__stex_styles_patch:nnn{#1}{##1}{##2}
485   }
486   \exp_after:wN \NewDocumentCommand\cs:w #1\cs_end:{#2}{
487     \cs_set:Npn \stex_style_apply: {
488       \__stex_styles_apply_patch:n{#1}
489     }
490     #3
491   }
492   \tl_set:cn {\__stex_styles_style_#1:} { #4 }
493 }
494
495 \cs_new_protected:Nn \__stex_styles_patch:nnn {
496   \str_if_empty:nTF {#2}{
497     \tl_set:cn{\__stex_styles_style_#1:}{#3}
498   }{
499     \tl_set:cn{\__stex_styles_style_#1_#2:}{#3}
500   }
501 }

```

```

502
503 \cs_new_protected:Nn \__stex_styles_apply_patch:n {
504   \clist_if_empty:NTF \l_stex_key_style_clist {
505     \tl_clear:N \thisstyle
506     \use:c{__stex_styles_style_#1:}
507   }{
508     \clist_get:NN \l_stex_key_style_clist \thisstyle
509     \tl_if_exist:cTF{__stex_styles_style_#1_\thisstyle :}{
510       \use:c{__stex_styles_style_#1_\thisstyle :}
511     }{
512       \use:c{__stex_styles_style_#1:}
513     }
514   }
515 }

```

(End of definition for `\stex_new_stylable_cmd:nnnn` and `\stex_style_apply:`. These functions are documented on page 127.)

sfunction

`\stex_new_stylable_env:nnnnnnn`

```

516 \cs_new_protected:Nn \stex_new_stylable_env:nnnnnnn {
517   \exp_after:wN \newcommand \cs:w stexstyle#1 \cs_end:[3] []{
518     \__stex_styles_patch:nnnn{#1}{##1}{##2}{##3} % <- defined after \stex_if_html_backend
519   }
520   \exp_after:wN \newcommand \cs:w stexcss#1 \cs_end:[1] []{
521     \__stex_styles_css_patch:nnnn{#1}{##1}
522   }
523   \NewDocumentEnvironment{#7#1}{#2}{
524     \cs_set:Npn \stex_style_apply: {
525       \stex_apply_patch_begin:n{#1}
526     }
527     #3
528   }{
529     \stex_if_html_backend:F{
530       \cs_set:Npn \stex_style_apply: {
531         \stex_apply_patch_end:n{#1}
532       }
533     }
534     #4
535   }
536   \tl_set:cn {__stex_styles_style_#1_start:} { #5 }
537   \tl_set:cn {__stex_styles_style_#1_end:} { #6 }
538 }
539
540
541 \cs_new_protected:Nn \stex_apply_patch_begin:n {
542   \clist_if_empty:NTF \l_stex_key_style_clist {
543     \tl_clear:N \thisstyle
544     \use:c{__stex_styles_style_#1_start:}
545   }{
546     \clist_get:NN \l_stex_key_style_clist \thisstyle
547     \stex_debug:nn{styling}{dominant~style:~\thisstyle}
548     \tl_if_exist:cTF{__stex_styles_style_#1_\thisstyle _start:}{
549       \use:c{__stex_styles_style_#1_\thisstyle _start:}

```

```

550     }{
551       \use:c{__stex_styles_style_#1_start:}
552     }
553   }
554 }
555
556 \cs_new_protected:Nn \_stex_apply_patch_end:n {
557   \tl_if_empty:NTF \thisstyle {
558     \use:c{__stex_styles_style_#1_end:}
559   }{
560     \tl_if_exist:cTF{__stex_styles_style_#1\_thisstyle _end:}{
561       \use:c{__stex_styles_style_#1\_thisstyle _end:}
562     }{
563       \use:c{__stex_styles_style_#1_end:}
564     }
565   }
566 }

```

(End of definition for `\stex_new_stylable_env:nnnnnnn`. This function is documented on page 127.)

sfunction

26.8 Persisting Dependencies

```

567 <@@=stex_persist>

```

We check the environment variables:

```

568 \stex_get_env:Nn\__stex_persist_env_str{STEX_USESMS}
569 \str_if_empty:NF\__stex_persist_env_str{
570   \exp_args:No \str_if_eq:nnF \__stex_persist_env_str{false}{
571     \bool_set_true:N \c_stex_persist_mode_bool
572   }
573 }
574 \stex_get_env:Nn\__stex_persist_env_str{STEX_WRITESMS}
575 \str_if_empty:NF\__stex_persist_env_str{
576   \exp_args:No \str_if_eq:nnF \__stex_persist_env_str{false}{
577     \bool_set_true:N \c_stex_persist_write_mode_bool
578   }
579 }
580
581 \bool_if:NT \c_stex_persist_force_bool {
582   \bool_set_false:N \c_stex_persist_mode_bool
583 }
584
585 \iow_new:N \c__stex_persist_sms_iow

```

`\stex_persist:n` defined later; requires `\stex_if_html_backend:TF`

`\stex_persist:e`
`\loadsms` (End of definition for `\stex_persist:n` and `\loadsms`. These functions are documented on page 127.)

sfunction

Is called at the end of the .sty-file:

```

586
587 \cs_new_protected:Npn \loadsms #1 {
588   \stex_str_if_ends_with:nnTF{#1}{.sms}{
589     \__stex_persist_load_file:n{#1}

```

```

590   }{
591     \stex_str_if_ends_with:nnTF{#1}{.tex}{
592       \__stex_persist_load_file:n{#1}
593     }{
594       \__stex_persist_load_file:n{#1.sms}
595     }
596   }
597 }
598
599 \cs_new_protected:Nn \_stex_persist_read_now: {
600   \bool_if:NTF \c_stex_persist_mode_bool {
601     \bool_if:NTF \c_stex_persist_write_mode_bool
602     \__stex_persist_read_and_write:
603     {
604       \__stex_persist_load_file:n{\jobname.sms}
605     }
606   }{
607     \bool_if:NT \c_stex_persist_write_mode_bool \__stex_persist_write_only:
608   }
609 }
610
611 \cs_new_protected:Nn \__stex_persist_load_file:n {
612   \file_if_exist:nT{#1}{
613     \group_begin:
614     \cs_set:Npn \stex_persist:n ##1 {}
615     \cs_set:Npn \stex_persist:e ##1 {}
616     \stex_debug:nn{persist}{restoring~from~sms~file}
617     \catcode'\:=11\relax
618     \catcode'\_:=11\relax
619     \catcode'\ =10\relax
620     \cs:w @ @ input \cs_end:#1\relax
621   \group_end:
622 }
623 }
624
625 \cs_new_protected:Nn \__stex_persist_write_only: {
626   \iow_open:Nn \c__stex_persist_sms_iow {\jobname.sms}
627   \AtEndDocument{ \iow_close:N \c__stex_persist_sms_iow }
628 }
629
630 \cs_new_protected:Nn \__stex_persist_read_and_write: {
631   \file_if_exist:nTF{\jobname.sms}{
632     \ior_open:Nn \g_tmpa_ior {\jobname.sms}
633     \iow_open:Nn \g_tmpa_iow {\jobname.sms2}
634     \ior_str_map_inline:Nn \g_tmpa_ior {
635       \iow_now:Nn \g_tmpa_iow {##1}
636     }
637     \iow_close:N \g_tmpa_iow
638     \ior_close:N \g_tmpa_ior
639     \__stex_persist_write_only:
640     \ior_open:Nn \g_tmpa_ior {\jobname.sms2}
641     \ior_str_map_inline:Nn \g_tmpa_ior {
642       \iow_now:Nn \c__stex_persist_sms_iow {##1}
643     }

```

```

644 \ior_close:N \g_tmpa_ior
645 \__stex_persist_load_file:n{\jobname.sms2}
646 }\__stex_persist_write_only:
647 }

```

26.9 Auxiliary Methods

```

648 <@@=stex_aux>

```

`\_stex_do_deprecation:n`

```

649 \cs_new:Nn \_stex_do_deprecation:n {
650   \str_if_empty:NF \l_stex_key_deprecate_str {
651     \msg_warning:nxxx{stex}{warning/deprecated}{#1}{\l_stex_key_deprecate_str}
652   }
653 }

```

(End of definition for _stex_do_deprecation:n. This function is documented on page ??.)

`\_stex_do_id:`

```

654 \cs_new_protected:Nn \_stex_do_id: {
655   \stex_if_smsmode:F {
656     \str_if_empty:NF \l_stex_key_id_str {
657       \exp_args:No \stex_ref_new_doc_target:n \l_stex_key_id_str
658     }
659   }
660 }

```

(End of definition for _stex_do_id:. This function is documented on page ??.)

Chapter 27

HTML Output

```
661 <@=stex_annotate>

\stex@backend We determine and \input the backend config file:

662 \ifcstype if@rustex\endcstype\else
663   \expandafter\newif\cstype if@rustex\endcstype
664   \@rustexfalse
665 \fi

666 \stex_get_env:Nn\__stex_annotate_env_str{STEX_FORCE_PDF}
667 \exp_args:No \str_if_eq:nnTF \__stex_annotate_env_str {true} {
668   \def\stex@backend{pdflatex}
669 }{
670   \tl_if_exist:NF\stex@backend{
671     \if@rustex
672       \def\stex@backend{rustex}
673     \else
674       \cs_if_exist:NTF\HCode{
675         \def\stex@backend{tex4ht}
676       }{
677         \def\stex@backend{pdflatex}
678       }
679     \fi
680   }
681 }
682 }
683
684 \input{stex-backend-\stex@backend.cfg}

(End of definition for \stex@backend. This variable is documented on page 128.)

\stex_if_html_backend_p: is provided by the backend config file.
\stex_if_html_backend:TF (End of definition for \stex_if_html_backend:TF. This function is documented on page 128.)
sfunction

\ifstexhtml

685 \bool_new:N \l__stex_annotate_do_output_bool
686
687 \newif\ifstexhtml
688 \stex_if_html_backend:TF {
```

```

689 \stexhtmltrue
690 \bool_set_true:N \l__stex_annotate_do_output_bool
691 }{
692 \stexhtmlfalse
693 \bool_set_false:N \l__stex_annotate_do_output_bool
694 }

```

(End of definition for `\ifstexhtml`. This function is documented on page 128.)
sfunction

```

\stex_if_do_html_p:
\stex_if_do_html:TF
695 \prg_new_conditional:Nnn \stex_if_do_html: {p,T,F,TF} {
696 \bool_if:NTF \l__stex_annotate_do_output_bool
697 \prg_return_true: \prg_return_false:
698 }

```

(End of definition for `\stex_if_do_html:TF`. This function is documented on page 128.)
sfunction

```

\stex_suppress_html:n
699 \cs_new_protected:Nn \stex_suppress_html:n {
700 \stex_pseudogroup:nn{
701 \bool_set_false:N \l__stex_annotate_do_output_bool
702 #1
703 }{
704 \stex_if_do_html:T {
705 \bool_set_true:N \l__stex_annotate_do_output_bool
706 }
707 }
708 }

```

(End of definition for `\stex_suppress_html:n`. This function is documented on page 128.)
sfunction

`\stex_annotate:nn` is provided by the backend config file.

(End of definition for `\stex_annotate:nn`. This function is documented on page 128.)
sfunction

`stex_annotate_env (env.)` is provided by the backend config file.

`\stex_html_node:nnn` is provided by the backend config file.

(End of definition for `\stex_html_node:nnn`. This function is documented on page 128.)
sfunction

`stex_env_node (env.)` is provided by the backend config file.

`\stex_annotate_invisible:nn` is provided by the backend config file.

```

\stex_annotate_invisible:n

```

(End of definition for `\stex_annotate_invisible:nn` and `\stex_annotate_invisible:n`. These functions are documented on page 129.)
sfunction

`\STEXinvisible`

```
709 \cs_new_protected:Npn \STEXinvisible #1 {  
710   \stex_annotate_invisible:n { #1 }  
711 }
```

(End of definition for \STEXinvisible. This function is documented on page 129.)
sfunction

`\stex_css_link:n` is provided by the backend config file.

(End of definition for \stex_css_link:n. This function is documented on page 129.)
sfunction

`\stex_css_literal:n` is provided by the backend config file.

(End of definition for \stex_css_literal:n. This function is documented on page 129.)
sfunction

`\_stex_annotate_force_break:n` is provided by the backend config file.

(End of definition for _stex_annotate_force_break:n. This function is documented on page 129.)
sfunction

`\mmlintent`

```
\mmlarg 712 \stex_if_html_backend:TF {  
713   \cs_new_protected:Npn \mmlintent #1 #2 {  
714     \stex_annotate:nn{mml:intent={#1}}{#2}  
715   }  
716   \cs_new_protected:Npn \mmlarg #1 #2 {  
717     \stex_annotate:nn{mml:arg={#1}}{#2}  
718   }  
719 }{  
720   \cs_new_protected:Npn \mmlintent #1 #2 { #2 }  
721   \cs_new_protected:Npn \mmlarg #1 #2 { #2 }  
722 }
```

(End of definition for \mmlintent and \mmlarg. These functions are documented on page 129.)
sfunction

We can now define `\stex_persist:n`:

```
723 <@@=stex_persist>  
724 \bool_if:NTF \c_stex_persist_write_mode_bool {  
725   \stex_if_html_backend:TF{  
726     \cs_new:Npn \stex_persist:n #1 {}  
727     \cs_new:Npn \stex_persist:e #1 {}  
728   }{  
729     \cs_new_protected:Nn \stex_persist:n {  
730       \iow_now:Nn \c__stex_persist_sms_iow {#1}  
731     }  
732     \cs_generate_variant:Nn \stex_persist:n {e}  
733   }  
734 }{  
735   \cs_new:Npn \stex_persist:n #1 {}  
736   \cs_new:Npn \stex_persist:e #1 {}  
737 }
```


and style patching:

```

738 <@@=stex_styles>
739 \stex_if_html_backend:TF{
740   \cs_new_protected:Nn \stex_style_apply: {}
741   \cs_new_protected:Nn \__stex_styles_patch:nnnn {}
742   \stex_keys_define:nnnn{ csspatch }{
743     \str_clear:N \l_stex_keys_cls_id
744     \str_clear:N \l_stex_keys_counter_id
745     \str_clear:N \l_stex_keys_counter_parent
746   }{
747     counter      .str_set_x:N = \l_stex_keys_counter_id,
748     parent       .str_set_x:N = \l_stex_keys_counter_parent,
749     unknown      .code:n      = {
750       \exp_args:NNo \str_set:Nn \l_stex_keys_cls_id {\l_keys_key_tl}
751     }
752   }{}
753   \cs_new_protected:Npn \__stex_styles_css_patch:nnnn #1 #2 {
754     \stex_keys_set:nn{ csspatch }{ #2 }
755     \group_begin:
756     \catcode'\ =12\relax
757     \catcode'^=12\relax
758     \__stex_styles_patch_i:nnn {#1}
759   }
760
761   \seq_new:N \g__stex_styles_counters_seq
762
763   \cs_new:Nn \__stex_styles_patch_html_c: {
764     \stex_html_literal:n{
765       <span~data-ftml-counter="\l_stex_keys_counter_id"~style="display:none;"~
766       data-ftml-counter-parent="\l_stex_keys_counter_parent"
767     ></span>
768   }
769 }
770
771 \cs_new:Nn \__stex_styles_patch_html: {
772   \stex_html_literal:n{
773     <span~data-ftml-style="\l_stex_keys_cls_id"~style="display:none;"~
774     data-ftml-counter="\l_stex_keys_counter_id"
775   ></span>
776 }
777 }
778
779 \cs_new_protected:Nn \__stex_styles_patch_i:nnn {
780   \str_if_empty:NTF \l_stex_keys_cls_id {
781     \str_set:Nn \l_stex_keys_cls_id { #1 }
782   }{
783     \str_set:Ne \l_stex_keys_cls_id { #1-\l_stex_keys_cls_id }
784   }
785   \exp_args:No \str_case:nnF \l_stex_keys_counter_parent {
786     {}{}
787     {part}{\str_set:Nn \l_stex_keys_counter_parent { 0 }}
788     {chapter}{\str_set:Nn \l_stex_keys_counter_parent { 1 }}
789     {section}{\str_set:Nn \l_stex_keys_counter_parent { 2 }}
790     {subsection}{\str_set:Nn \l_stex_keys_counter_parent { 3 }}

```

```

791     {subsubsection}{\str_set:Nn \l_stex_keys_counter_parent { 4 }}
792     {paragraph}{\str_set:Nn \l_stex_keys_counter_parent { 5 }}
793     {subparagraph}{\str_set:Nn \l_stex_keys_counter_parent { 6 }}
794   }{
795     \msg_error:nnx{stex}{error/notasection}\l_stex_keys_counter_parent
796   }
797   \str_set:Ne \l__stex_styles_first_str { &~.ftml-title-paragraph~ { display:inline-block
798   \str_if_empty:NF \l_stex_keys_counter_id {
799     %\str_put_left:Ne \l__stex_styles_first_str { counter-increment:~ ftml-\l_stex_keys_c
800     \exp_args:NNo \seq_if_in:NnF \g__stex_styles_counters_seq \l_stex_keys_counter_id {
801       \seq_gpush:No \g__stex_styles_counters_seq \l_stex_keys_counter_id
802       \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
803         \__stex_styles_patch_html_c:
804         % in body
805       }{
806         \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html_c:
807         % in preamble
808       }
809     }
810     \cs_if_eq:ccTF{@onlypreamble}{@notprerr}{
811       \__stex_styles_patch_html:
812       % in body
813     }{
814       \exp_args:Ne \AtBeginDocument \__stex_styles_patch_html:
815       % in preamble
816     }
817   }
818   \exp_args:Ne \stex_css_literal:n { .ftml-\l_stex_keys_cls_id { \l__stex_styles_first_st
819
820   % TODO export counter reset somehow
821
822   \group_end:
823 }
824 }{
825   \cs_new_protected:Nn \__stex_styles_patch:nnnn {
826     \str_if_empty:NTF {#2}{
827       \tl_set:cn{__stex_styles_style_#1_start:}{#3}
828       \tl_set:cn{__stex_styles_style_#1_end:}{#4}
829     }{
830       \tl_set:cn{__stex_styles_style_#1_#2_start:}{#3}
831       \tl_set:cn{__stex_styles_style_#1_#2_end:}{#4}
832     }
833   }
834   \cs_new_protected:Nn \__stex_styles_css_patch:nnnn {}
835 }

```

Chapter 28

Math Archives

```
\c_stex_mathhub_files
\mathhub
836 <@@=stex_mathhub>
837 \str_if_empty:NTF\mathhub{
838   \stex_get_env:Nn \l__stex_mathhub_str {MATHHUB}
839   \str_if_empty:NTF \l__stex_mathhub_str {
840     \ior_open:NnTF \g_tmpa_ior{\stex_file_use:N \c_stex_home_file/.stex/mathhub.path}{
841       \group_begin:
842         \escapechar=-1\catcode'\=12
843         \ior_str_get:NN \g_tmpa_ior \l__stex_mathhub_str
844         \str_gset_eq:NN \l__stex_mathhub_str \l__stex_mathhub_str
845       \group_end:
846       \ior_close:N \g_tmpa_ior
847       \stex_debug:nn{mathhub}{MathHub-directory~determined~from~home~directory}
848     }{
849       \str_clear:N \l__stex_mathhub_str
850     }
851   }{
852     \stex_debug:nn{mathhub}{MathHub-directory~determined~from~environment~variable}
853   }
854 }{
855   \str_set_eq:NN \l__stex_mathhub_str \mathhub
856 }
857
858 \str_if_empty:NTF \l__stex_mathhub_str {
859   \msg_warning:nn{stex}{warning/nomathhub}
860   \stex_file_set:Ne \l_tmpa_seq {\stex_file_use:N \c_stex_home_file \tl_to_str:n{/MathHub}}
861   \seq_clear:N \c_stex_mathhub_files
862   \seq_push:Ne \c_stex_mathhub_files {\stex_file_use:N \l_tmpa_seq}
863 }{
864   \seq_clear:N \c_stex_mathhub_files
865   \seq_set_split:NnV \l_tmpa_seq , \l__stex_mathhub_str
866   \seq_map_inline:Nn \l_tmpa_seq {
867     \stex_file_resolve:Nn \l__stex_mathhub_str {#1}
868     \stex_if_file_absolute:NF \l__stex_mathhub_str {
869       \stex_file_resolve:Ne \l__stex_mathhub_str {
870         \stex_file_use:N \c_stex_pwd_file / #1
871       }

```

```

872     }
873     \seq_put_right:Ne \c_stex_mathhub_files {
874         \stex_file_use:N \l__stex_mathhub_str
875     }
876 }
877 }
878
879 \exp_args:NNe \str_set:Nn \mathhub {\seq_use:Nn \c_stex_mathhub_files ,}
880 \stex_debug:nn{mathhub}{MATHHUBS:~\mathhub}

```

(End of definition for `\c_stex_mathhub_files` and `\mathhub`. These variables are documented on page 132.)

`\stex_archive_id:N`

```

881 \cs_new:Npn \stex_archive_id:N {
882     \exp_after:wN \use_i:nnn
883 }

```

(End of definition for `\stex_archive_id:N`. This function is documented on page 132.)

sfunction

`\stex_archive_base:N`

`\stex_archive_base:n`

```

884 \cs_new:Npn \stex_archive_base:N {
885     \exp_after:wN \use_ii:nnn
886 }
887
888 \cs_new:Nn \stex_archive_base:n {
889     \exp_args:NNe \use:nn \use_ii:nnn { \use:c {c_stex_mathhub_#1_archive} }
890 }

```

(End of definition for `\stex_archive_base:N` and `\stex_archive_base:n`. These functions are documented on page 132.)

sfunction

`\stex_archive_path:N`

`\stex_archive_path:n`

```

891 \cs_new:Npn \stex_archive_path:N {
892     \exp_after:wN \use_iii:nnn
893 }
894
895 \cs_new:Nn \stex_archive_path:n {
896     \exp_args:NNe \use:nn \use_iii:nnn { \use:c {c_stex_mathhub_#1_archive} }
897 }

```

(End of definition for `\stex_archive_path:N` and `\stex_archive_path:n`. These functions are documented on page 132.)

sfunction

`\stex_in_archive:nn`

```

898 \cs_new_protected:Nn \stex_in_archive:nn {
899     \stex_pseudogroup:nn{
900         \cs_set:Npn \l__stex_mathhub_cs ##1 {#2}
901         \tl_if_empty:nTF{#1}{
902             \tl_if_exist:NTF \l_stex_current_archive {
903                 \exp_args:Ne \l__stex_mathhub_cs {\stex_archive_id:N \l_stex_current_archive }
904             }{

```

```

905     \l__stex_mathhub_cs {}
906   }
907   }{
908     \__stex_mathhub_set_current:n{#1}
909     \l__stex_mathhub_cs {#1}
910   }
911   }{
912     \stex_pseudogroup_restore:N \l_stex_current_archive
913     \cs_set:Npn \exp_not:N \l__stex_mathhub_cs ##1 {
914       \exp_args:No \exp_not:n {\l__stex_mathhub_cs {##1}}
915     }
916   }
917 }
918
919 \cs_set:Npn \l__stex_mathhub_cs #1 {}

```

Auxiliary macros for loading archives:

```

920 \cs_new_protected:Nn \__stex_mathhub_set_current:n {
921   \__stex_mathhub_require:n { #1 }
922   \stex_debug:nn{mathhub}{switching~to~archive~#1}
923   \tl_set_eq:Nc \l_stex_current_archive {
924     c_stex_mathhub_#1_archive
925   }
926 }
927
928 \cs_new_protected:Nn \_stex_set_meta_archive: {
929   \prop_if_exist:cF { c_stex_mathhub_FTML/meta_archive } {
930     \seq_if_empty:NF \c_stex_mathhub_files {
931       \stex_debug:nn{mathhub}{Opening~archive:~FTML/meta}
932       \__stex_mathhub_do_manifest:nn { FTML/meta }{
933         \tl_gset:ce {c_stex_mathhub_FTML/meta_archive}{
934           {\tl_to_str:n{FTML/meta}}
935           {\tl_to_str:n{http://mathhub.info}}
936         }
937       }
938     }
939   }
940 }
941
942 }
943
944 \cs_new_protected:Nn \__stex_mathhub_require:n {
945   \prop_if_exist:cF { c_stex_mathhub_#1_archive } {
946     \seq_if_empty:NTF \c_stex_mathhub_files {
947       \msg_fatal:nn{stex}{warning/nomathhub}
948     }{
949       \stex_debug:nn{mathhub}{Opening~archive:~#1}
950       \__stex_mathhub_do_manifest:nn { #1 }{
951         \msg_fatal:nnee{stex}{error/noarchive}
952         {#1}{\seq_use:Nn \c_stex_mathhub_files ,}
953       }
954     }
955   }
956 }

```

Attempts to find the archive's manifest file:

```

957 \cs_new_protected:Nn \__stex_mathhub_do_manifest:nn {
958   \seq_map_inline:Nn \c_stex_mathhub_files {
959     \__stex_mathhub_find_manifest:nn {##1}{#1}
960     \str_if_empty:NF \l__stex_mathhub_manifest_str \seq_map_break:
961   }
962   %\exp_args:Ne \__stex_mathhub_find_manifest:n {\stex_file_use:N \c_stex_mathhub_file / #1}
963   \str_if_empty:NTF \l__stex_mathhub_manifest_str { #2 }{
964     \__stex_mathhub_parse_manifest:n {#1}
965   }
966 }
967
968 \cs_new_protected:Nn \__stex_mathhub_find_manifest:nn {
969   \str_clear:N \l__stex_mathhub_manifest_str
970   \seq_set_split:Nnn \l_tmpa_seq / { #1 }
971   \seq_set_split:Nnn \l__stex_mathhub_seq / {#1 / #2}
972   \bool_set_true:N \l__stex_mathhub_bool
973   \bool_while_do:Nn \l__stex_mathhub_bool {
974     \tl_if_eq:NNTF \l__stex_mathhub_seq \l_tmpa_seq {
975       \bool_set_false:N \l__stex_mathhub_bool
976     }{
977       \__stex_mathhub_check_manifest:
978       \bool_if:NT \l__stex_mathhub_bool {
979         \seq_pop_right:NN \l__stex_mathhub_seq \l__stex_mathhub_tl
980       }
981     }
982   }
983 }
984
985 \cs_new_protected:Nn \__stex_mathhub_check_manifest: {
986   \__stex_mathhub_check_manifest:n {MANIFEST.MF}
987   \bool_if:NT \l__stex_mathhub_bool {
988     \__stex_mathhub_check_manifest:n {META-INF/MANIFEST.MF}
989     \bool_if:NT \l__stex_mathhub_bool {
990       \__stex_mathhub_check_manifest:n {meta-inf/MANIFEST.MF}
991     }
992   }
993 }
994
995 \cs_new_protected:Nn \__stex_mathhub_check_manifest:n {
996   \stex_debug:nn{mathhub}{Checking~\stex_file_use:N \l__stex_mathhub_seq / #1}
997   \file_if_exist:nT {\stex_file_use:N \l__stex_mathhub_seq / #1} {
998     \bool_set_false:N \l__stex_mathhub_bool
999     \str_set:Ne \l__stex_mathhub_manifest_str {\stex_file_use:N \l__stex_mathhub_seq / #1}
1000   }
1001 }

```

Parses the archive's manifest file:

```

1002 \ior_new:N \c__stex_mathhub_manifest_ior
1003
1004 \str_set:Nn \l_tmpa_str {https://}
1005
1006 \exp_after:wN \cs_new:Npn \exp_after:wN \__stex_mathhub_replace_https:w \l_tmpa_str #1 \__s
1007   http:// #1

```

```

1008 }
1009
1010 \cs_new_protected:Nn \__stex_mathhub_parse_manifest:n {
1011   \ior_open:Nn \c__stex_mathhub_manifest_ior \l__stex_mathhub_manifest_str
1012   \str_set:Ne \l__stex_mathhub_file_str {\stex_file_use:N \l__stex_mathhub_seq}
1013   \str_set:Nn \l__stex_mathhub_id_str {#1}
1014   \str_set:Nn \l__stex_mathhub_base_str {http://mathhub.info}
1015
1016   \ior_map_inline:Nn \c__stex_mathhub_manifest_ior {
1017     \exp_args:NNNo \exp_args:NNNx
1018     \seq_set_split:Nnn \l__stex_mathhub_seq \c_colon_str {\tl_to_str:n{##1}}
1019     \seq_pop_left:NNT \l__stex_mathhub_seq \l__stex_mathhub_key {
1020       \exp_args:NNo \str_set:Nn \l__stex_mathhub_key \l__stex_mathhub_key
1021       \str_set:Ne \l__stex_mathhub_val {\seq_use:Nn \l__stex_mathhub_seq :}
1022       \str_case:Nn \l__stex_mathhub_key {
1023         {id} { \str_set_eq:NN \l__stex_mathhub_id_str \l__stex_mathhub_val }
1024         {url-base} { \str_set_eq:NN \l__stex_mathhub_base_str \l__stex_mathhub_val }
1025       }
1026     }
1027   }
1028   \ior_close:N \c__stex_mathhub_manifest_ior
1029   \exp_args:No \stex_str_if_starts_with:nnT \l__stex_mathhub_base_str {https://} {
1030     \str_set:Ne \l__stex_mathhub_base_str { \exp_after:wN \__stex_mathhub_replace_https:w \
1031   }
1032   \tl_gset:ce { c_stex_mathhub_#1_archive } { {\l__stex_mathhub_id_str} {\l__stex_mathhub_b
1033   \stex_debug:nn{mathhub}{Result:~\use:c{c_stex_mathhub_#1_archive}}
1034   \str_if_eq:nnF {#1}{main}{
1035     \stex_persist:e {
1036       \__stex_mathhub_restore:nn{\l__stex_mathhub_id_str}{\l__stex_mathhub_base_str}
1037     }
1038   }
1039 }

```

The `.sms` file persisting content should not store the file path of the archive, since that depends on the directory layout of the system that generated it. We therefore, when restoring archive macros, check if we find the archive on the local system, and if not, leave the directory path empty. This will throw an error iff some functionality at some point *requires* the archive to actually exist locally.

```

1040 \cs_set_protected:Nn \__stex_mathhub_restore:nn {
1041   \__stex_mathhub_do_manifest:nn { #1 }{}
1042   \tl_if_exist:cF { c_stex_mathhub_#1_archive }{
1043     \tl_set:ce{ c_stex_mathhub_#1_archive }{
1044       {\tl_to_str:n{#1}}
1045       {\tl_to_str:n{#2}}
1046     }
1047   }
1048 }
1049 }
1050

```

(End of definition for `\stex_in_archive:nn`. This function is documented on page 132.)

sfunction

```

\c_stex_no_archive_str
\c_stex_no_archive

```

```

1051 \str_const:Nn \c_stex_no_archive_str { no/archive }
1052
1053 \tl_const:Ne \c_stex_no_archive_uri {
1054   {\c_stex_no_archive_str}
1055   {\tl_to_str:n{http://unknown.source}}
1056   {}
1057 }
1058
1059 \tl_set_eq:cN {c_stex_mathhub_ no/archive _ archive}\c_stex_no_archive_uri

```

(End of definition for `\c_stex_no_archive_str` and `\c_stex_no_archive`. These variables are documented on page 133.)

`\c_stex_main_archive`
`\l_stex_current_archive`

```

1060 \seq_map_inline:Nn \c_stex_mathhub_files {
1061   \seq_set_split:Nnn \l_tmpa_seq / {#1}
1062   \stex_if_file_starts_with:NNT \c_stex_pwd_file \l_tmpa_seq {
1063     \seq_set_eq:NN \l_tmpb_seq \c_stex_pwd_file
1064     \seq_map_inline:Nn \l_tmpa_seq { \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl }
1065     \exp_args:Nne \_stex_mathhub_find_manifest:nn {#1} { \stex_file_use:N \l_tmpb_seq }
1066     \str_if_empty:NTF \l__stex_mathhub_manifest_str {
1067       \stex_debug:nn{mathhub}{Not~currently~in~a~MathHub~archive}
1068     }{
1069       \_stex_mathhub_parse_manifest:n { main }
1070       \tl_set_eq:NN \c_stex_main_archive \c_stex_mathhub_main_archive
1071       \cs_undefine:N \c_stex_mathhub_main_archive
1072       \tl_set_eq:cN { c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _archive }
1073       \c_stex_main_archive
1074       \prop_set_eq:NN \l_stex_current_archive \c_stex_main_archive
1075       \stex_debug:nn{mathhub}{Current~archive:~
1076         \stex_archive_id:N \c_stex_main_archive
1077       }
1078       \stex_persist:e {
1079         \_stex_mathhub_restore:nn{\stex_archive_id:N \c_stex_main_archive}{\stex_archive_b
1080         \tl_gset_eq:Nc \exp_not:N \c_stex_main_archive {c_stex_mathhub_ \stex_archive_id:N
1081         \tl_gset_eq:NN \exp_not:N \l_stex_current_archive \exp_not:N \c_stex_main_archive
1082       }
1083       %\bool_if:NT \c_stex_persist_write_mode_bool {
1084       %   \tl_put_right:Ne \_stex_persist_read_now: {
1085       %     \stex_persist:n {c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _archi
1086       %       \tl_gset_eq:cN{c_stex_mathhub_ \stex_archive_id:N \c_stex_main_archive _manife
1087       %       \tl_gset_eq:NN \exp_not:N \l_stex_current_archive \exp_not:N \c_stex_main_archi
1088       %     }
1089       %   }
1090       %}
1091   }
1092   \seq_map_break:
1093 }
1094 }
1095 \tl_if_exist:NF \c_stex_main_archive {
1096   \stex_debug:nn{mathhub}{Not~currently~in~a~MathHub~directory}
1097 }

```

(End of definition for `\c_stex_main_archive` and `\l_stex_current_archive`. These variables are documented on page 133.)


```

\stex_source_path:n
\stex_source_path:nn
1098 \cs_new:Nn \stex_source_path:n {
1099   \tl_if_exist:NTF \l_stex_current_archive {
1100     \stex_archive_path:N \l_stex_current_archive / source \tl_if_empty:nF{#1}{/ #1}
1101   }{
1102     \stex_file_use:N \g_stex_current_file / .. \tl_if_empty:nF{#1}{/ #1}
1103   }
1104 }
1105 \cs_new:Nn \stex_source_path:nn {
1106   \tl_if_empty:nTF{#1}{
1107     \stex_source_path:n {#2}
1108   }{
1109     \tl_if_exist:cTF {c_stex_mathhub_ #1 _archive } {
1110       \stex_archive_path:n {#1} / source \tl_if_empty:nF{#2}{/ #2}
1111     }{
1112       \stex_file_use:N \g_stex_current_file / .. \tl_if_empty:nF{#2}{/ #2}
1113     }
1114   }
1115 }

```

(End of definition for \stex_source_path:n and \stex_source_path:nn. These functions are documented on page 133.)

sfunction

Chapter 29

URIs

```
1116 <@@=stex_uris>
```

```
\stex_use_archive_uri:n
```

```
1117 \cs_new:Nc \stex_use_archive_uri:n {  
1118   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1  
1119 }
```

(End of definition for \stex_use_archive_uri:n. This function is documented on page 134.)
sfunction

29.1 Document URIs

```
\stex_use_document_uri:N
```

```
\stex_use_document_uri:n
```

```
1120 \cs_new:Nc \stex_use_document_uri:N {  
1121   \exp_after:wN \__stex_uris_doc:nnnnn #1  
1122 }  
1123 \cs_new:Nc \stex_use_document_uri:n {  
1124   \__stex_uris_doc:nnnnn #1  
1125 }  
1126 \cs_new:Nc \__stex_uris_doc:nnnnn {  
1127   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1  
1128   \exp_not:N \tl_if_empty:nF{#2}{  
1129     \c_ampersand_str\tl_to_str:n{p=}#2  
1130   }  
1131   \c_ampersand_str\tl_to_str:n{d=}#3  
1132   \c_ampersand_str\tl_to_str:n{l=}#4  
1133   \exp_not:N \tl_if_empty:nF{#5}{  
1134     \c_ampersand_str\tl_to_str:n{e=}#5  
1135   }  
1136 }
```

(End of definition for \stex_use_document_uri:N and \stex_use_document_uri:n. These functions are documented on page 134.)
sfunction

```
\stex_document_uri_archive:N
```

```
1137 \cs_new:Nc \stex_document_uri_archive:N {  
1138   \exp_after:wN \use_i:nnnnn #1
```

```
1139 }
(End of definition for \stex_document_uri_archive:N. This function is documented on page 134.)
sfunction
```

`\stex_document_uri_path:N`

```
1140 \cs_new:Nn \stex_document_uri_path:N {
1141   \exp_after:wN \use_ii:nnnnn #1
1142 }
(End of definition for \stex_document_uri_path:N. This function is documented on page 135.)
sfunction
```

`\stex_document_uri_name:N`

```
1143 \cs_new:Nn \stex_document_uri_name:N {
1144   \exp_after:wN \use_iii:nnnnn #1
1145 }
(End of definition for \stex_document_uri_name:N. This function is documented on page 135.)
sfunction
```

`\stex_document_uri_language:N`

```
1146 \cs_new:Nn \stex_document_uri_language:N {
1147   \exp_after:wN \use_iv:nnnnn #1
1148 }
(End of definition for \stex_document_uri_language:N. This function is documented on page 135.)
sfunction
```

`\stex_document_uri_element:N`

```
1149 \cs_new:Nn \stex_document_uri_element:N {
1150   \exp_after:wN \use_v:nnnnn #1
1151 }
(End of definition for \stex_document_uri_element:N. This function is documented on page 135.)
sfunction
```

`\stex_document_uri_with_language:Nn`

```
1152 \cs_new:Npn \stex_document_uri_with_language:Nn {
1153   \exp_after:wN \__stex_uris_with_language:nnnnnn
1154 }
1155 \cs_new:Nn \__stex_uris_with_language:nnnnnn {
1156   {#1}{#2}{#3}{#6}{#4}
1157 }
(End of definition for \stex_document_uri_with_language:Nn. This function is documented on page 135.)
sfunction
```

`\stex_new_document_element_uri:n`

```
1158 \cs_new:Npn \stex_new_document_element_uri:n {
1159   \exp_after:wN \__stex_uris_with_elem:nnnnnn \l_stex_current_document_uri
1160 }
1161 \cs_new:Nn \__stex_uris_with_elem:nnnnnn {
1162   {#1}{#2}{#3}{#4}{ \tl_if_empty:nTF{#5}{#6}{#5/#6}}
1163 }
```

(End of definition for \stex_new_document_element_uri:n. This function is documented on page 135.)

sfunction

\stex_document_uri_from_archive_file:Nn

```

1164 \cs_new_protected:Nn \stex_document_uri_from_archive_file:Nn {
1165   \stex_file_set:Nn \l__stex_uris_file {#2}
1166   \stex_debug:nn{URI}{relative~path:~\stex_file_use:N \l__stex_uris_file}
1167   \__stex_uris_doc_from_archive_file:NN #1 \l__stex_uris_file
1168 }
1169
1170 \cs_new_protected:Nn \__stex_uris_doc_from_archive_file:NN {
1171   \__stex_uris_split_file:N #2
1172   \prop_if_exist:NTF \l_stex_current_archive {
1173     \str_set:Ne \l__stex_uris_archive {
1174       \stex_archive_id:N \l_stex_current_archive
1175     }
1176   }{
1177     \str_set_eq:NN \l__stex_uris_archive \c_stex_no_archive_str
1178   }
1179   \tl_set:Ne #1 {
1180     { \l__stex_uris_archive }
1181     { \stex_file_use:N \l__stex_uris_path_seq }
1182     { \l__stex_uris_name_str }
1183     { \l__stex_uris_lang_str }
1184   }{}
1185 }
1186 }
1187
1188 \cs_new_protected:Nn \__stex_uris_split_file:N {
1189   \seq_set_eq:NN \l__stex_uris_path_seq #1
1190   \seq_pop_right:NN \l__stex_uris_path_seq \l__stex_uris_name_str
1191   \seq_set_split:NnV \l_tmpa_seq . \l__stex_uris_name_str
1192   \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_lang_str % .tex?
1193   \seq_if_empty:NTF \l_tmpa_seq {
1194     \str_set_eq:NN \l__stex_uris_name_str \l__stex_uris_lang_str
1195     \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
1196   }\__stex_uris_split_file_i:
1197 }
1198
1199 \cs_new_protected:Nn \__stex_uris_split_file_i: {
1200   \cs_if_eq:NNTF \l__stex_uris_lang_str \q_no_value {
1201     \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
1202   }{
1203     \prop_if_in:NoF \c_stex_languages_prop \l__stex_uris_lang_str {
1204       \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_lang_str % actual language maybe?
1205       \cs_if_eq:NNTF \l__stex_uris_lang_str \q_no_value {
1206         \str_set_eq:NN \l__stex_uris_lang_str \l_stex_current_language_str
1207       }{
1208         \prop_if_in:NoF \c_stex_languages_prop \l__stex_uris_lang_str {
1209           \seq_put_right:No \l_tmpa_seq \l__stex_uris_lang_str
1210           \str_set:Nn \l__stex_uris_lang_str {en}
1211         }
1212       }
1213     }

```

```

1214 }
1215 \str_set:Ne \l__stex_uris_name_str {\seq_use:Nn \l_tmpa_seq .}
1216 }
1217
1218 \cs_new_protected:Nn \__stex_uris_split_file_ii: {
1219
1220 }

```

(End of definition for `\stex_document_uri_from_archive_file:Nn`. This function is documented on page 135.)

sfunction

```

\c_stex_main_document_uri
\l_stex_current_document_uri
1221 \tl_if_exist:NTF \l_stex_current_archive {
1222   \str_set:Ne \l_tmpa_str { \stex_archive_path:N \l_stex_current_archive }
1223   \seq_set_split:NnV \l_tmpa_seq / \l_tmpa_str
1224   \seq_set_eq:NN \l_tmpb_seq \c_stex_main_file
1225   \seq_map_inline:Nn \l_tmpa_seq {
1226     \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl
1227   }
1228   \seq_pop_left:NN \l_tmpb_seq \l_tmpa_tl
1229   \__stex_uris_doc_from_archive_file:NN \l_stex_current_document_uri \l_tmpb_seq
1230 }{
1231   \__stex_uris_doc_from_archive_file:NN \l_stex_current_document_uri \c_stex_main_file
1232 }
1233
1234 \tl_set_eq:NN \c_stex_main_document_uri \l_stex_current_document_uri
1235 \str_set:Ne \l_stex_current_language_str { \stex_document_uri_language:N \l_stex_current_do
1236
1237 \stex_debug:nn{URIs}{Current~document::~ \stex_use_document_uri:N \c_stex_main_document_uri}

```

(End of definition for `\c_stex_main_document_uri` and `\l_stex_current_document_uri`. These variables are documented on page 135.)

29.2 Module URIs

```

\stex_use_module_uri:N
\stex_use_module_uri:n
1238 \cs_new:Nn \stex_use_module_uri:N {
1239   \exp_after:wN \__stex_uris_module:nnn #1
1240 }
1241 \cs_new:Nn \stex_use_module_uri:n {
1242   \__stex_uris_module:nnn #1
1243 }
1244 \cs_new:Ne \__stex_uris_module:nnn {
1245   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
1246   \exp_not:N \tl_if_empty:nF{#2}{
1247     \c_ampersand_str\tl_to_str:n{p=}#2
1248   }
1249   \c_ampersand_str\tl_to_str:n{m=}#3
1250 }

```

(End of definition for `\stex_use_module_uri:N` and `\stex_use_module_uri:n`. These functions are documented on page 135.)

sfunction

`\stex_module_uri_archive:N`

```
1251 \cs_new:Nn \stex_module_uri_archive:N {  
1252   \exp_after:wN \use_i:nnn #1  
1253 }
```

(End of definition for \stex_module_uri_archive:N. This function is documented on page 135.)
sfunction

`\stex_module_uri_path:N`

`\stex_module_uri_path:n`

```
1254 \cs_new:Nn \stex_module_uri_path:N {  
1255   \exp_after:wN \use_ii:nnn #1  
1256 }  
1257 \cs_new:Nn \stex_module_uri_path:n {  
1258   \use_ii:nnn #1  
1259 }
```

(End of definition for \stex_module_uri_path:N and \stex_module_uri_path:n. These functions are documented on page 135.)
sfunction

`\stex_module_uri_name:N`

`\stex_module_uri_name:n`

```
1260 \cs_new:Nn \stex_module_uri_name:N {  
1261   \exp_after:wN \use_iii:nnn #1  
1262 }  
1263  
1264 \cs_new:Nn \stex_module_uri_name:n {  
1265   \use_iii:nnn #1  
1266 }
```

(End of definition for \stex_module_uri_name:N and \stex_module_uri_name:n. These functions are documented on page 136.)
sfunction

`\stex_module_uri_split_name:NNN`

`\stex_module_uri_split_name:NNn`

```
1267 \cs_new_protected:Npn \stex_module_uri_split_name:NNN #1 {  
1268   \exp_after:wN \__stex_uris_parent:NNnnn \exp_after:wN #1 \exp_after:wN  
1269 }  
1270  
1271 \cs_new_protected:Nn \stex_module_uri_split_name:NNn {  
1272   \__stex_uris_parent:NNnnn #1 #2 #3  
1273 }  
1274  
1275 \cs_new_protected:Nn \__stex_uris_parent:NNnnn {  
1276   \seq_set_split:Nnn #1 / {#5}  
1277   \seq_pop_right:NN #1 #2  
1278   \seq_if_empty:NTF #1 {  
1279     \tl_set:Ne #1 { {#3} {#4} {#3}}  
1280     \tl_clear:N #2  
1281   }{  
1282     \tl_set:Ne #1 { {#3} {#4} {\seq_use:Nn #1 /} }  
1283   }  
1284 }
```

(End of definition for `\stex_module_uri_split_name:NNN` and `\stex_module_uri_split_name:NNn`. These functions are documented on page 136.)

sfunction

`\stex_module_uri_as_qm:n`

```
1285 \cs_new:Nn \stex_module_uri_as_qm:n {
1286   \__stex_uris_as_qm:nnn #1
1287 }
1288 \cs_new:Nn \__stex_uris_as_qm:nnn {
1289   #1 / #2 ? #3
1290 }
```

(End of definition for `\stex_module_uri_as_qm:n`. This function is documented on page 136.)

sfunction

`\stex_new_module_uri:n`

```
1291 \cs_new:Npn \stex_new_module_uri:n {
1292   \stex_if_in_module:TF {
1293     \exp_after:wN \__stex_uris_new_nested_mod:nnnn \l_stex_current_module_uri
1294   }{
1295     \exp_after:wN \__stex_uris_new_mod:nnnnnn \l_stex_current_document_uri
1296   }
1297 }
1298
1299 \cs_new:Nn \__stex_uris_new_mod:nnnnnn {
1300   {#1}
1301   \str_if_eq:nnTF{#3}{#6}{
1302     {#2}{#3}
1303   }{
1304     {#2 \tl_if_empty:nF{#2}/ #3}{ \tl_to_str:n{ #6 } }
1305   }
1306 }
1307 \cs_new:Nn \__stex_uris_new_nested_mod:nnnn {
1308   {#1}
1309   {#2}
1310   {#3 / \tl_to_str:n{#4}}
1311 }
```

(End of definition for `\stex_new_module_uri:n`. This function is documented on page 136.)

sfunction

`\stex_uri_from_pair:Nnn`

```
1312 \bool_new:N \l_stex_relative_import_bool
1313 \cs_new_protected:Nn \stex_uri_from_pair:Nnn {
1314   \bool_set_false:N \l_stex_relative_import_bool
1315   \stex_debug:nn{uri}{resolving~<#2,#3>~in~\stex_use_document_uri:N \l_stex_current_document_uri}
1316   \seq_set_split:Nne \l_tmpa_seq ? { \tl_to_str:n {#3} }
1317   \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_name_str
1318   \str_clear:N \l__stex_uris_path_str
1319   \seq_if_empty:NF \l_tmpa_seq {
1320     \seq_pop_right:NN \l_tmpa_seq \l__stex_uris_path_str
1321   }
1322   \tl_if_empty:nTF { #2 }{
1323     \str_if_empty:NTF \l__stex_uris_path_str
```

```

1324     \__stex_uris_maybe_relative:N \__stex_uris_absolute:N
1325     #1
1326   }{
1327     \__stex_uris_pair_in_archive:Nn #1 { #2 }
1328   }
1329 }
1330 %\cs_generate_variant:Nn \stex_uri_from_pair:Nnn {Nee}
1331
1332 \cs_new_protected:Nn \__stex_uris_maybe_relative:N {
1333   \tl_set:Ne \l_tmpb_tl {
1334     \stex_document_uri_path:N \l_stex_current_document_uri
1335   }
1336   \tl_set:Ne \l_tmpa_tl {
1337     {
1338       \stex_document_uri_archive:N \l_stex_current_document_uri
1339     }
1340     {
1341       \l_tmpb_tl
1342       \exp_args:NNo \exp_args:Nne \str_if_eq:nnF \l__stex_uris_name_str {
1343         \stex_document_uri_name:N \l_stex_current_document_uri
1344       }{
1345         \str_if_empty:NF \l_tmpb_tl / \stex_document_uri_name:N \l_stex_current_document_uri
1346       }
1347     }
1348     { \l__stex_uris_name_str }
1349   }
1350   \stex_if_module_exists:NTF \l_tmpa_tl {
1351     \tl_set_eq:NN #1 \l_tmpa_tl
1352     \bool_set_true:N \l_stex_relative_import_bool
1353     \stex_debug:nn{uri}{returning~relative~\stex_use_module_uri:N #1}
1354   }{
1355     \__stex_uris_absolute:N #1
1356   }
1357 }
1358
1359 \cs_new_protected:Nn \__stex_uris_absolute:N {
1360   \tl_if_exist:NTF \l_stex_current_archive {
1361     \tl_set:Ne #1 {
1362       { \stex_archive_id:N \l_stex_current_archive }
1363       { \l__stex_uris_path_str }
1364       { \l__stex_uris_name_str }
1365     }
1366     \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
1367   }{
1368     \__stex_uris_pair_no_archive:N #1
1369   }
1370 }
1371
1372 \cs_new_protected:Nn \__stex_uris_pair_no_archive:N {
1373   \stex_file_resolve:Ne \l_tmpa_seq {
1374     \stex_file_use:N \g_stex_current_file / .. / \l__stex_uris_path_str
1375   }
1376   \tl_set:Ne #1 {
1377     { \c_stex_no_archive_str }

```



```

1378     { \stex_file_use:N \l_tmpa_seq }
1379     { \l__stex_uris_name_str }
1380   }
1381   \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
1382 }
1383
1384 \cs_new_protected:Nn \__stex_uris_pair_in_archive:Nn {
1385   \tl_set:Nc #1 {
1386     { \tl_to_str:n{ #2 } }
1387     { \l__stex_uris_path_str }
1388     { \l__stex_uris_name_str }
1389   }
1390   \stex_debug:nn{uri}{returning~\stex_use_module_uri:N #1}
1391 }

```

(End of definition for \stex_uri_from_pair:Nnn. This function is documented on page 136.)

sfunction

\stex_uri_from_pair:Nn

```

1392 \cs_new_protected:Nn \stex_uri_from_pair:Nn {
1393   \peek_charcode:NTF [ {
1394     \__stex_uris_parse_i:Nw #1
1395   }{
1396     \__stex_uris_parse_ii:Nw #1
1397   } #2 \__stex_uris_end:
1398 }
1399
1400 \cs_new_protected:Npn \__stex_uris_parse_i:Nw #1 [#2] #3 \__stex_uris_end: {
1401   \stex_uri_from_pair:Nnn #1 {#2} {#3}
1402 }
1403
1404 \cs_new_protected:Npn \__stex_uris_parse_ii:Nw #1 #2 \__stex_uris_end: {
1405   \stex_uri_from_pair:Nnn #1 {} {#2}
1406 }

```

(End of definition for \stex_uri_from_pair:Nn. This function is documented on page 136.)

sfunction

29.3 Symbol URIs

\stex_use_symbol_uri:N

\stex_use_symbol_uri:n

```

1407 \cs_new:Nn \stex_use_symbol_uri:N {
1408   \exp_after:wN \__stex_uris_symbol:nnnn #1
1409 }
1410 \cs_new:Nn \stex_use_symbol_uri:n {
1411   \__stex_uris_symbol:nnnn #1
1412 }
1413 \cs_new:Nc \__stex_uris_symbol:nnnn {
1414   \exp_not:N \stex_archive_base:n{#1} \tl_to_str:n{?a=} #1
1415   \exp_not:N \tl_if_empty:nF{#2}{
1416     \c_ampersand_str\tl_to_str:n{p=}#2
1417   }
1418   \c_ampersand_str\tl_to_str:n{m=}#3

```

```

1419 \c_ampersand_str\tl_to_str:n{s=}#4
1420 }

```

(End of definition for \stex_use_symbol_uri:N and \stex_use_symbol_uri:n. These functions are documented on page 136.)

```
sfunction
```

```
\stex_new_symbol_uri:n
```

```

1421 \cs_new:Nn \stex_new_symbol_uri:n {
1422   \l_stex_current_module_uri { \tl_to_str:n{ #1 } }
1423 }

```

(End of definition for \stex_new_symbol_uri:n. This function is documented on page 136.)

```
sfunction
```

```
\stex_new_symbol_uri:nn
```

```

1424 \cs_new:Nn \stex_new_symbol_uri:nn {
1425   #1 { #2 }
1426 }

```

(End of definition for \stex_new_symbol_uri:nn. This function is documented on page 136.)

```
sfunction
```

```
\stex_symbol_uri_archive:N
```

```

1427 \cs_new:Nn \stex_symbol_uri_archive:N {
1428   \exp_after:wN \use_i:nnnn #1
1429 }

```

(End of definition for \stex_symbol_uri_archive:N. This function is documented on page 136.)

```
sfunction
```

```
\stex_symbol_uri_path:N
```

```

1430 \cs_new:Nn \stex_symbol_uri_path:N {
1431   \exp_after:wN \use_ii:nnnn #1
1432 }

```

(End of definition for \stex_symbol_uri_path:N. This function is documented on page 136.)

```
sfunction
```

```
\stex_symbol_uri_module:N
```

```
\stex_symbol_uri_module:n
```

```

1433 \cs_new:Nn \stex_symbol_uri_module:N {
1434   \exp_after:wN \__stex_uris_first_three:nnnn #1
1435 }
1436 \cs_new:Nn \stex_symbol_uri_module:n {
1437   \__stex_uris_first_three:nnnn #1
1438 }
1439
1440 \cs_new:Nn \__stex_uris_first_three:nnnn {
1441   {#1}{#2}{#3}
1442 }

```

(End of definition for \stex_symbol_uri_module:N and \stex_symbol_uri_module:n. These functions are documented on page 137.)

```
sfunction
```

`\stex_symbol_uri_name:N`

`\stex_symbol_uri_name:n`

```
1443 \cs_new:Nn \stex_symbol_uri_name:N {  
1444   \exp_after:wN \use_iv:nnnn #1  
1445 }  
1446 \cs_new:Nn \stex_symbol_uri_name:n {  
1447   \use_iv:nnnn #1  
1448 }
```

(End of definition for `\stex_symbol_uri_name:N` and `\stex_symbol_uri_name:n`. These functions are documented on page 137.)

sfunction

Chapter 30

Documents

```
1449 (@@=stex_doc)

\STEXftmllink
\STEXlinkftml
\STEXsetlinkftml

1450 \cs_new_protected:Npn \STEXftmllink {
1451   \exp_args:Ne\url{
1452     \stex_use_document_uri:N \l_stex_current_document_uri
1453   }
1454 }
1455
1456 \cs_new_protected:Npn \STEXsetlinkftml #1 {
1457   \tl_gset:Nn \g__stex_doc_ftml_link_text_tl { #1 }
1458 }
1459
1460 \tl_gset:Nn \g__stex_doc_ftml_link_text_tl{
1461   The~
1462   \stex_get_symbol:nF{FTML}{~}
1463   \tl_if_empty:NTF \l_stex_get_symbol_uri {FTML}{\sn{FTML}}
1464   {~ version ~ of ~ this ~ document ~ can ~ be ~ found ~ at\\
1465   \STEXftmllink
1466 }
1467
1468 \NewDocumentCommand \STEXlinkftml { 0{ } } {
1469   \ifstexhtml\else
1470     \begin{center}
1471       \emph{
1472         \tl_if_empty:nTF{#1}\g__stex_doc_ftml_link_text_tl{#1}
1473       }
1474     \end{center}
1475   \fi
1476 }
```

(End of definition for \STEXftmllink, \STEXlinkftml, and \STEXsetlinkftml. These functions are documented on page 138.)

sffunction

\stexdoctitle Initial definition (before \begin{document}):

```
1477 \tl_new:N \g__stex_doc_title_tl
1478
1479 \str_new:N \g_stex_document_kind_str
```

```

1480 \tl_new:N \g_stex_document_kind_parameters_tl
1481 \str_set:Nn \g_stex_document_kind_str{article}
1482 \stex_if_html_backend:T{
1483   \AtBeginDocument{
1484     \exp_args:Nne\stex_html_node:nnn{span}{
1485       data-ftml-document-kind=\g_stex_document_kind_str
1486       \g_stex_document_kind_parameters_tl
1487     }{}
1488   }
1489 }

```

```

1490
1491 \cs_new_protected:Npn \stexdoctitle #1 {
1492   \tl_gset:Nn \g__stex_doc_title_tl { #1 }
1493   \global\def\stexdoctitle##1{}
1494 }

```

At begin document, we switch to:

```

1495 \cs_new_protected:Nn \__stex_doc_set_title:n {
1496   \stex_if_smsmode:F{
1497     \gdef\stexdoctitle##1{}
1498     \stex_debug:nn{title}{Setting~title~to:~\tl_to_str:n{#1}}
1499     \tl_gset:Nn \g__stex_doc_title_tl { #1 }
1500     \__stex_doc_title_html:
1501   }
1502 }

```

```

1503
1504 \cs_new_protected:Nn \__stex_doc_title_html: {
1505   \stex_if_do_html:T{
1506     \stex_annotate_invisible:nn{data-ftml-doctitle={}}{ \hbox{\g__stex_doc_title_tl} }
1507   }
1508 }

```

```

1509
1510 \AtBeginDocument {
1511   \tl_if_empty:NTF \g__stex_doc_title_tl {
1512     \cs_set_eq:NN \stexdoctitle \__stex_doc_set_title:n
1513   }{
1514     \gdef\stexdoctitle#1{}
1515     \__stex_doc_title_html:
1516   }
1517
1518   \cs_set_eq:NN \__stex_doc_maketitle: \maketitle
1519   \protected\gdef\maketitle{
1520     \tl_if_empty:NF \@title {
1521       \exp_args:No \stexdoctitle \@title
1522     }
1523     \__stex_doc_maketitle:
1524   }
1525 }

```

(End of definition for \stexdoctitle. This function is documented on page 138.)

sfunction

30.1 Sectioning

`\l_stex_current_section_level_int`
`\l_stex_current_section_level_str`

```
1526 \int_new:N \l_stex_current_section_level_int
1527 \str_set:Nn \l_stex_current_section_level_str {document}
```

(End of definition for `\l_stex_current_section_level_int` and `\l_stex_current_section_level_str`. These variables are documented on page 139.)

`\currentsectionlevel`
`\Currentsectionlevel`

```
1528 \cs_set_protected:Npn \currentsectionlevel {
1529   \stex_if_do_html:TF{
1530     \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
1531     \stex_annotate:nn{data-ftml-currentsectionlevel={},data-ftml-capitalize=false}{}
1532   }{
1533     \l_stex_current_section_level_str
1534   }
1535   \tl_if_exist:NT\xspace\xspace
1536 }
1537
1538 \cs_set_protected:Npn \Currentsectionlevel {
1539   \stex_if_do_html:TF{
1540     \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
1541     \stex_annotate:nn{data-ftml-currentsectionlevel={},data-ftml-capitalize=true}{}
1542   }{
1543     \exp_after:wN \str_uppercase:n \l_stex_current_section_level_str
1544   }
1545   \tl_if_exist:NT\xspace\xspace
1546 }
```

(End of definition for `\currentsectionlevel` and `\Currentsectionlevel`. These functions are documented on page 139.)

sfunction

`sfragment (env.)`

```
1547 \stex_keys_define:nnnn{ sfragment }{
1548   \tl_clear:N \l_stex_key_short_tl
1549 }{
1550   short .tl_set:N = \l_stex_key_short_tl
1551 }{id}
1552
1553 \NewDocumentEnvironment{sfragment}{0{} m}{
1554   \stex_keys_set:nn{sfragment}{#1}
1555   \__stex_doc_do_section:n{#2}
1556   \stex_do_id: % TODO
1557 }{
1558   \_sfragment_end:
1559 }
1560
1561 \cs_new_protected:Npn \__stex_doc_do_section:n {
1562   \int_case:nnF \l_stex_current_section_level_int {
1563     {0}{\cs_if_exist:NTF \thepart {\_sfragment_do_level:nn{part}}{
1564       \int_incr:N \l_stex_current_section_level_int
1565       \__stex_doc_do_section:n
1566     }}{
1567     }
1568   }
```

```

1566     }}
1567     {1}{\cs_if_exist:NTF \thechapter {\_sfragment_do_level:nn{chapter}}{
1568         \int_incr:N \l_stex_current_section_level_int
1569         \__stex_doc_do_section:n
1570     }}
1571     {2}{\_sfragment_do_level:nn{section}}
1572     {3}{\_sfragment_do_level:nn{subsection}}
1573     {4}{\_sfragment_do_level:nn{subsubsection}}
1574     {5}{\_sfragment_do_level:nn{paragraph}}
1575 }{\_sfragment_do_level:nn{subparagraph}}
1576 }
1577
1578 \stex_if_html_backend:TF {
1579     \cs_new_protected:Nn \_sfragment_do_level:nn {
1580         \stexdoctitle{#2}
1581         \par
1582         \begin{stex_env_node}{section}{data-ftml-section={\int_use:N \l_stex_current_section_level_int}
1583             \noindent\stex_html_node:nnn{h1}{data-ftml-title={}}{
1584                 \stex_annotate_force_break:n{#2}
1585             }\par
1586         }
1587         \cs_new_protected:Nn \_sfragment_end: {
1588             \end{stex_env_node}
1589         }
1590     }{
1591         \cs_new_protected:Nn \_sfragment_do_level:nn {
1592             \stexdoctitle{#2}
1593             \tl_if_empty:NTF \l_stex_key_short_tl {
1594                 \use:c{#1}
1595             }{
1596                 \exp_args:Nne \use:nn{\use:c{#1}}{[\exp_args:No \exp_not:n \l_stex_key_short_tl]}
1597             }{#2}
1598
1599             \int_compare:nNnF \l_stex_current_section_level_int = 6{
1600                 \int_incr:N \l_stex_current_section_level_int
1601             }
1602             \tl_set:Nn\l_stex_current_section_level_str{#1}
1603         }
1604         \cs_new_protected:Nn \_sfragment_end: {}
1605     }

```

`titlefragment (env.)`

```

1606 \bool_new:N \l__stex_doc_titlefragment_bool
1607
1608 \NewDocumentEnvironment{titlefragment}{0}{m}{
1609     \stex_keys_set:nn{sfragment}{#1}
1610     \cs_if_exist:NTF \maketitle {
1611         \bool_set_true:N \l__stex_doc_titlefragment_bool
1612         \title{#2}
1613     }{\tl_if_empty:NF \l_stex_key_short_tl {
1614         \cs_if_exist:NT \shorttitle {
1615             \exp_args:No \shorttitle \l_stex_key_short_tl
1616         }
1617     }}

```

```

1618     \stexdoctitle{#2}
1619     \maketitle
1620   }{
1621     \bool_set_false:N \l__stex_doc_titlefragment_bool
1622     \__stex_doc_do_section:n{#2}
1623     \_stex_do_id:
1624   }
1625 }{
1626   \bool_if:NF \l__stex_doc_titlefragment_bool \_sfragment_end:
1627 }

```

blindfragment (*env.*)

```

1628 \cs_new_protected:Nn \__stex_doc_skip_section_i: {
1629   \int_case:nn \l_stex_current_section_level_int {
1630     {0}{\cs_if_exist:NF \thepart {
1631       \int_incr:N \l_stex_current_section_level_int \__stex_doc_skip_section_i:
1632     }}
1633     {1}{\cs_if_exist:NF \thechapter {
1634       \int_incr:N \l_stex_current_section_level_int \__stex_doc_skip_section_i:
1635     }}
1636   }
1637   \int_compare:nNnF \l_stex_current_section_level_int = 6{
1638     \int_incr:N \l_stex_current_section_level_int
1639   }
1640 }
1641
1642 \stex_if_html_backend:TF {
1643   \cs_new_protected:Nn \__stex_doc_skip_section: {
1644     \__stex_doc_skip_section_i:
1645     \begin{stex_annotate_env}{data-ftml-skipsection={\int_use:N \l_stex_current_section_level_int}}
1646     \_stex_annotate_force_break:n{ }
1647   }
1648 }{
1649   \cs_set_eq:NN \__stex_doc_skip_section: \__stex_doc_skip_section_i:
1650 }
1651
1652 \NewDocumentEnvironment{blindfragment}{}{
1653   \__stex_doc_skip_section:
1654 }{
1655   \stex_if_html_backend:T{
1656     \stex_annotate_invisible:n{~}
1657     \end{stex_annotate_env}
1658   }
1659 }

```

\skipfragment

```

1660 \cs_new_protected:Npn \skipfragment {
1661   \int_case:nnF \l_stex_current_section_level_int {
1662     {0}{\cs_if_exist:NTF \thepart {\stepcounter{part}}{
1663       \int_incr:N \l_stex_current_section_level_int
1664       \skipfragment
1665     }}
1666     {1}{\cs_if_exist:NTF \thechapter {\stepcounter{chapter}}{
1667       \int_incr:N \l_stex_current_section_level_int

```



```

1668     \skipfragment
1669   }}
1670   {2}{\stepcounter{section}}
1671   {3}{\stepcounter{subsection}}
1672   {4}{\stepcounter{subsubsection}}
1673   {5}{\stepcounter{paragraph}}
1674   }{\stepcounter{subparagraph}}
1675 }

```

(End of definition for \skipfragment. This function is documented on page 139.)

sfunction

\setsectionlevel

```

1676 \cs_new_protected:Npn \setsectionlevel #1 {
1677   \str_case:nnF{#1}{
1678     {part}{\int_set:Nn \l_stex_current_section_level_int 0}
1679     {chapter}{\int_set:Nn \l_stex_current_section_level_int 1}
1680     {section}{\int_set:Nn \l_stex_current_section_level_int 2}
1681     {subsection}{\int_set:Nn \l_stex_current_section_level_int 3}
1682     {subsubsection}{\int_set:Nn \l_stex_current_section_level_int 4}
1683     {paragraph}{\int_set:Nn \l_stex_current_section_level_int 5}
1684   }{
1685     \int_set:Nn \l_stex_current_section_level_int 6
1686   }
1687   \stex_if_do_html:T{
1688     \cs_if_eq:NNTF\@onlypreamble\@notprerr{
1689       \stex_html_literal:n{<span~ data-ftml-sectionlevel="\int_use:N\l_stex_current_section_level_int}
1690     }{
1691       \AtBeginDocument{
1692         \stex_html_literal:n{<span~ data-ftml-sectionlevel="\int_use:N\l_stex_current_section_level_int}
1693       }
1694     }
1695   }
1696 }

```

In HTML mode, we make sure to record the top section level:

```

1697 \stex_if_html_backend:T{
1698   \cs_new_protected:Nn \__stex_doc_check_topsect: {
1699     \int_case:nnF \l_stex_current_section_level_int {
1700       {0}{\cs_if_exist:NTF \thepart {
1701         \stex_annotate_invisible:nn{data-ftml-sectionlevel=0}{}}
1702       }{
1703         \int_incr:N \l_stex_current_section_level_int
1704         \__stex_doc_check_topsect:
1705       }}
1706     {1}{\cs_if_exist:NTF \thechapter {
1707       \stex_annotate_invisible:nn{data-ftml-sectionlevel=1}{}}
1708     }{
1709       \int_incr:N \l_stex_current_section_level_int
1710       \__stex_doc_check_topsect:
1711     }}
1712   }{
1713     \stex_annotate_invisible:nn{data-ftml-sectionlevel={\int_use:N\l_stex_current_section_level_int}}
1714   }
1715 }

```

```

1716 \AtBeginDocument{\__stex_doc_check_topsect:
1717 \cs_if_exist:NT \thechapter {
1718 \int_case:nnT \l_stex_current_section_level_int{
1719 {0}{}}
1720 {1}{}}
1721 }{
1722 \stex_css_literal:n {
1723 .ftml-section {
1724 &~.ftml-title-section { &::before {
1725 content:~counter(ftml-chapter)~"."~counter(ftml-section)~"";
1726 } }
1727 }
1728 .ftml-subsection {
1729 &~.ftml-title-subsection { &::before {
1730 content:~counter(ftml-chapter)~"."~counter(ftml-section)~"."~counter(ftml-sub
1731 } }
1732 }
1733 .ftml-subsubsection {
1734 &~.ftml-title-subsubsection { &::before {
1735 content:~counter(ftml-chapter)~"."~counter(ftml-section)~"."~counter(ftml-sub
1736 } }
1737 }
1738 }
1739 }
1740 }
1741 }
1742 }

```

(End of definition for `\setsectionlevel`. This function is documented on page 139.)

sfunction

We make sure `\frontmatter` and `\backmatter` are always defined. **TODO: this seems like a hack and probably shouldn't be here**

```

1743 \bool_if:NF \c_stex_no_frontmatter_bool {
1744 \cs_new_protected:Nn \__stex_doc_x_matter: {
1745 \cs_if_exist:NTF\frontmatter{
1746 \let\__stex_doc_orig_frontmatter\frontmatter
1747 \let\__stex_doc_orig_endfrontmatter\endfrontmatter
1748 \let\endfrontmatter\relax
1749 \let\frontmatter\relax
1750 }{
1751 \tl_set:Nn\__stex_doc_orig_frontmatter{
1752 \clearpage
1753 %\@mainmatterfalse
1754 \pagenumbering{roman}
1755 }
1756 \tl_set:Nn\__stex_doc_orig_endfrontmatter{}
1757 }
1758 \cs_if_exist:NTF\backmatter{
1759 \let\__stex_doc_orig_backmatter\backmatter
1760 \let\__stex_doc_orig_endbackmatter\endbackmatter
1761 \let\backmatter\relax
1762 \let\endbackmatter\relax
1763 }{
1764 \tl_set:Nn\__stex_doc_orig_backmatter{

```

```

1765     \clearpage
1766     %\@mainmatterfalse
1767     \pagenumbering{roman}
1768   }
1769 }
1770 \newenvironment{frontmatter}{
1771   \__stex_doc_orig_frontmatter
1772 }{
1773   \cs_if_exist:NTF\mainmatter{
1774     \mainmatter
1775   }{
1776     \clearpage
1777     %\@mainmattertrue
1778     \pagenumbering{arabic}
1779   }
1780 }
1781 \newenvironment{backmatter}{
1782   \__stex_doc_orig_backmatter
1783 }{
1784   \cs_if_exist:NTF\mainmatter{
1785     \mainmatter
1786   }{
1787     \clearpage
1788     %\@mainmattertrue
1789     \pagenumbering{arabic}
1790   }
1791 }
1792 }
1793 \AddToHook{begindocument/end}\__stex_doc_x_matter:
1794 }

```

30.2 Inter-Document References

1795 <@@=stex_refs>

The .sref-file:

```

1796 \iow_new:N \c__stex_refs_iow
1797 \AtBeginDocument{ \iow_open:Nn \c__stex_refs_iow {\jobname.sref} }
1798 \AtEndDocument{\iow_close:N \c__stex_refs_iow}
1799 \str_set:Ne \c__stex_refs_iow_str { \stex_file_use:N \c_stex_pwd_file / \jobname.sref }

```

Keys:

```

1800 \stex_keys_define:nnnn{sref / 1}{
1801   \tl_clear:N \l_stex_key_pre_tl
1802   \tl_clear:N \l_stex_key_post_tl
1803   \tl_clear:N \l_stex_key_fallback_tl
1804 }{
1805   % TODO get rid of this
1806   fallback .tl_set:N = \l_stex_key_fallback_tl,
1807   pre      .code:n = {},
1808   post     .code:n = {}
1809 }{archive file}
1810 \stex_keys_define:nnnn{sref / 2}{}{archive file, title}

```

Target declarations:

```

\reflabel
\stex_ref_new_doc_target:n
1811 \cs_new_protected:Nn \stex_ref_new_doc_target:n {
1812   \stex_if_smsmode:F{
1813     \tl_set:Nx \l__stex_refs_new_tl { \stex_new_document_element_uri:n { #1 } }
1814     \exp_args:Nx \label { \stex_use_document_uri:N \l__stex_refs_new_tl }
1815     \exp_args:Nne \STeXInternalNewSRefLabel{#1}{ \stex_use_document_uri:N \l__stex_refs_new_tl }
1816     \iow_now:Nx \auxout {
1817       \STeXInternalNewSRefLabel{#1}{ \stex_use_document_uri:N \l__stex_refs_new_tl }
1818     }
1819     \iow_now:Nx \c__stex_refs_iow {
1820       \exp_not:N \STeXInternalSRefLabel
1821       { #1 }
1822       { \stex_use_document_uri:N \l__stex_refs_new_tl }
1823       { \exp_not:o \@currentHref }
1824       { \exp_not:o \@currentlabelname }
1825     }
1826   }
1827 }
1828
1829 \NewDocumentCommand \sreflabel {m} { \stex_ref_new_doc_target:n{#1} }

```

(End of definition for \sreflabel and \stex_ref_new_doc_target:n. These functions are documented on page 86.)

sfunction

```

\stex_ref_new_sym_target:n
1830 \cs_new_protected:Nn \stex_ref_new_sym_target:n {
1831   \stex_if_smsmode:F{
1832     \cs_if_exist:cTF{r@\tl_to_str:n{#1}}{
1833       \cs_if_exist:cTF{__stex_sref_aux_sym_ \tl_to_str:n{#1}}{
1834         \cs_undefine:c{__stex_sref_aux_sym_ \tl_to_str:n{#1}}
1835         \exp_args:Nx \label { \tl_to_str:n{#1} }
1836       }
1837     }{ \exp_args:Nx \label { \tl_to_str:n{#1} } }
1838   }
1839 }

```

(End of definition for \stex_ref_new_sym_target:n. This function is documented on page ??.)

sfunction

Inserting References:

```

\sref
1840 \NewDocumentCommand \sref { 0{} m 0{} }{
1841   \stex_keys_set:nn { sref / 1 }{ #1 }
1842   \tl_clear:N \l__stex_refs_tmp
1843   \str_if_empty:NTF \l_stex_key_file_str {
1844     \__stex_refs_local:n
1845   }{
1846     \__stex_refs_remote:nn{#3}
1847   }{#2}
1848 }
1849
1850 \cs_new_protected:Nn \__stex_refs_local:n {
1851   \stex_debug:nn{sref}{Checking~#1}

```

```

1852 \tl_if_exist:cTF{ g_stex_sref_label_ #1 }{
1853   \%exp_args:Ne \stex_annotate:nn{data-ftml-sref={\use:c{ g_stex_sref_label_ #1 }}}{
1854     \exp_args:Ne \__stex_refs_local_ref:n {\use:c{ g_stex_sref_label_ #1 }}
1855   }%}
1856 }{
1857   \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
1858 }
1859 }
1860
1861 \cs_new_protected:Nn \__stex_refs_local_ref:n {
1862   \cs_if_exist:cTF{autoref}{
1863     \l_stex_key_pre_tl \autoref{#1} \l_stex_key_post_tl
1864   }{
1865     \message{^^J^^J
1866       sTeX~Warning: \c_backslash_str sref ~ with ~ local ~ target ~ used ~ without ~ hyperr
1867       ^^J^^J}
1868     \l_stex_key_pre_tl \ref{#1} \l_stex_key_post_tl
1869   }
1870 }
1871
1872 \cs_new_protected:Nn \__stex_refs_remote:nn {
1873   \str_set:Nn \l__stex_refs_key { #2 }
1874   \tl_set:Nn \l__stex_refs_in { #1 }
1875   \exp_args:No \stex_in_archive:nn \l_stex_key_archive_str {
1876     \exp_args:NNo \stex_document_uri_from_archive_file:Nn \l__stex_refs_uri {\l_stex_key_fi
1877     \stex_debug:nn{sref}{Checking~#2~from~\stex_use_document_uri:N \l__stex_refs_uri}
1878     \tl_if_eq:NNTF \l__stex_refs_uri \c_stex_main_document_uri {
1879       \stex_debug:nn{sref}{From~current~document;~delegating~to~\string\autoref}
1880     }
1881     \__stex_refs_local:n{ #2 }
1882   }{
1883     \str_set:Ne \l__stex_refs_uri { \stex_use_document_uri:N \l__stex_refs_uri }
1884     \str_set:Ne \l__stex_refs_file { \exp_args:No \stex_source_path:n {\l_stex_key_file_s
1885     \exp_args:No \IfFileExists \l__stex_refs_file {
1886       \__stex_refs_find:
1887     }{}
1888     \tl_if_empty:NTF \l__stex_refs_tmp {
1889       \stex_debug:nn{sref}{Not~found}
1890       \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
1891     }{
1892       \str_set_eq:NN \l__stex_refs_uri \l__stex_refs_tmp
1893       \stex_debug:nn{sref}{Found~ \l__stex_refs_uri}
1894       \__stex_refs_maybe_in:
1895     }
1896   }
1897 }
1898
1899 \cs_new_protected:Nn \__stex_refs_find: {
1900   \setbox0\vbox\bgroup
1901   \cs_set_eq:NN \STeXInternalSRefLabel \__stex_refs_check_i:nnnn
1902   \use:c{@ @ input}{\l__stex_refs_file}
1903   \exp_args:NNe \use:nn
1904   \egroup {
1905     \tl_set:Nn \exp_not:N \l__stex_refs_tmp { \l__stex_refs_tmp }

```

```

1906     }
1907 }
1908
1909 \cs_new_protected:Nn \__stex_refs_check_i:nnnn {
1910   \exp_args:No \str_if_eq:nnT{ \l__stex_refs_key }{#1}{
1911     \exp_args:Nno \stex_str_if_starts_with:nnT{#2}{\l__stex_refs_uri}{
1912       \str_set:Nn \l__stex_refs_tmp {#2}
1913     }
1914   }
1915 }
1916 }
1917
1918 \cs_new_protected:Nn \__stex_refs_maybe_in: {
1919   \tl_if_empty:NTF\l__stex_refs_in {
1920     \cs_if_exist:cTF{r@\l__stex_refs_uri}{
1921       \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
1922     }{
1923       \tl_if_empty:NTF \l__stex_refs_default_file {
1924         \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
1925       }{
1926         \tl_set_eq:NN \l_stex_key_title_tl \l__stex_refs_default_title
1927         \str_set_eq:NN \l__stex_refs_file \l__stex_refs_default_file
1928         \str_set_eq:NN \l_stex_key_archive_str \l__stex_refs_default_archive
1929         \str_set_eq:NN \l_stex_key_file_str \l__stex_refs_default_relpath
1930         \stex_debug:nn{sref}{
1931           in~file:~\l__stex_refs_default_file;~out~file:~\c__stex_refs_iow_str
1932         }
1933         \str_if_eq:NNTF \l__stex_refs_default_file \c__stex_refs_iow_str {
1934           \exp_args:No \__stex_refs_local_ref:n \l__stex_refs_uri
1935         }{
1936           \stex_debug:nn{sref}{in~title=\exp_args:No\exp_not:n\l_stex_key_title_tl,file=\l__
1937             \__stex_refs_in:
1938         }
1939       }
1940     }
1941   }{
1942     \stex_debug:nn{sref}{in~\exp_args:No\exp_not:n\l__stex_refs_in,archive=\l_stex_key_arch
1943     \exp_args:No \stex_in_archive:nn \l_stex_key_archive_str {
1944       \str_set:Ne \l__stex_refs_file { \exp_args:No \stex_source_path:n {\l_stex_key_file_s
1945     }
1946     \exp_args:Nno \stex_keys_set:nn{ sref / 2 }{ \l__stex_refs_in }
1947     \__stex_refs_in:
1948   }
1949 }
1950
1951 \cs_new_protected:Nn \__stex_refs_in: {
1952   \tl_clear:N \l__stex_refs_tmp
1953   \exp_args:No \IfFileExists \l__stex_refs_file {
1954     \__stex_refs_find_in:
1955   }{
1956     \stex_debug:nn{sref}{not~found:~\l__stex_refs_file}
1957   }
1958   \tl_if_empty:NTF\l__stex_refs_tmp{
1959     \stex_debug:nn{sref}{Not~found}

```

```

1960 \tl_if_empty:NTF \l_stex_key_fallback_tl { ??? }{ \l_stex_key_fallback_tl }
1961 }{
1962 \stex_debug:nn{sref}{found:~\meaning\l__stex_refs_tmp}
1963 \tl_set:Ne \l__stex_refs_tmp {
1964 \exp_after:wN \__stex_refs_in_text:nn \l__stex_refs_tmp
1965 }
1966 \stex_if_html_backend:TF{
1967 \exp_args:No \stex_in_archive:nn \l_stex_key_archive_str {
1968 \exp_args:NNo \stex_document_uri_from_archive_file:Nn \l__stex_refs_in \l_stex_key_
1969 \exp_args:Ne \stex_annotate:nn{data-ftml-sref={\l__stex_refs_uri},data-ftml-sref-in
1970 \l__stex_refs_tmp
1971 }
1972 }
1973 }{
1974 \cs_if_exist:NT \href {\exp_args:No \href \l__stex_refs_uri }\l__stex_refs_tmp
1975 }
1976 }
1977 }
1978
1979 \cs_new:Nn \__stex_refs_in_text:nn {
1980 \__stex_refs_in_text:w #1 \__stex_refs_stop:
1981 \tl_if_empty:nF{#2}{~(\exp_not:n{#2})}
1982 \tl_if_empty:NF \l_stex_key_title_tl {
1983 {}~in~\exp_args:No \exp_not:n \l_stex_key_title_tl
1984 }
1985 }
1986
1987 \cs_new:Npn \__stex_refs_in_text:w #1.#2 \__stex_refs_stop: {
1988 \__stex_refs_uppercase:n #1~#2~
1989 }
1990
1991 \cs_new:Nn \__stex_refs_uppercase:n { \uppercase{#1}}
1992
1993 \cs_new_protected:Nn \__stex_refs_find_in: {
1994 \str_if_eq:NNTF \l__stex_refs_file \c__stex_refs_iow_str {
1995 \stex_debug:nn{sref}{Same~file!}
1996 }{
1997 \stex_debug:nn{sref}{finding~\l__stex_refs_uri;~in~\l__stex_refs_file...?}
1998 \setbox0\vbox\bgroup
1999 \catcode'\J=9\relax
2000 \cs_set_eq:NN \STeXInternalSRefLabel \__stex_refs_check_in:nnnn
2001 \use:c{@ @ input}{\l__stex_refs_file}
2002 \exp_args:NNe \use:nn
2003 \egroup {
2004 \tl_set:Nn \exp_not:N \l__stex_refs_tmp { \exp_args:No \exp_not:n \l__stex_refs_tmp }
2005 }
2006 }
2007 }
2008
2009 \cs_new_protected:Nn \__stex_refs_check_in:nnnn {
2010 \exp_args:No \str_if_eq:nnT{ \l__stex_refs_uri }{#2}{
2011 \tl_set:Nn \l__stex_refs_tmp { {#3}{#4} }
2012 \endinput
2013 }

```

```

2014 }
2015
2016 (End of definition for \sref. This function is documented on page 86.)
2017
2018 sfunction
2019 TODO
2020
2021 %\int_new:N \l__stex_refs_unnamed_counter_int
2022
2023
2024
2025
2026 \cs_new:Npn \STeXInternalSRefLabel #1 #2 #3 #4 {}
2027
2028 \cs_new_protected:Npn \STeXInternalNewSRefLabel #1 #2 {
2029   \tl_gset:ce{ g_stex_sref_label_ #1 }{ \tl_to_str:n{ #2 } }
2030 }
2031
2032
2033 \tl_new:N \l__stex_refs_default_title
2034 \str_new:N \l__stex_refs_default_file
2035 \str_new:N \l__stex_refs_default_archive
2036 \str_new:N \l__stex_refs_default_relpath
2037
2038 \newcommand\srefsetin[3][]{
2039   \str_set:Ne \l__stex_refs_default_relpath { #2 }
2040   \tl_if_empty:nTF{#1}{
2041     \str_set:Ne \l__stex_refs_default_archive { \stex_archive_id:N \c_stex_main_archive }
2042   }{
2043     \str_set:Nn \l__stex_refs_default_archive { #1 }
2044   }
2045   \exp_args:No \stex_in_archive:nn \l__stex_refs_default_archive {
2046     \str_set:Ne \l__stex_refs_default_file { \exp_args:No \stex_source_path:n {\l__stex_ref
2047   }
2048   \str_set:Nn \l__stex_refs_default_title { #3 }
2049 }
2050 \NewDocumentCommand \extref { 0{} m m }{???}
2051
2052 \cs_new_protected:Nn \stex_ref_new_symbol:n {}
2053
2054
2055 \NewDocumentCommand \srefsym { m m }{
2056   \stex_get_symbol:n { #1 }
2057   \exp_args:Ne \srefsymuri { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }{ #2 }
2058 }
2059
2060 \cs_new_protected:Npn \srefsymuri #1 {
2061   \cs_if_exist:cTF{r@\tl_to_str:n{#1}}{
2062     \cs_if_exist:NT \hyperlink { \hyperlink{#1} }
2063   }{
2064     \cs_if_exist:NT \href { \href{#1} }
2065   }
2066 }

```

30.3 Inputting From MathHub Resources


```

2060 <@@=stex_inputs>

\inputref

2061 \NewDocumentCommand \inputref { 0{} m}{
2062   \stex_in_archive:nn{#1}{
2063     \stex_document_uri_from_archive_file:Nn \l__stex_inputs_uri {#2}
2064     \__stex_inputs_ref:n{#2}
2065   }
2066 }

2067
2068 \stex_if_html_backend:TF{
2069   \str_set:Nn \c__stex_inputs_ref_pre_str {<span~data-ftml-inputref=}
2070   \str_set:Nn \c__stex_inputs_ref_post_str {"~><!--></span>}
2071   \cs_new_protected:Nn \__stex_inputs_ref_i:n {
2072     \IfFileExists{\stex_source_path:n {#1}}{
2073       %\relax
2074       %\noindent\vbox{
2075         \exp_args:N\stex_html_literal:n{
2076           \c__stex_inputs_ref_pre_str
2077           \stex_use_document_uri:N\l__stex_inputs_uri
2078           \c__stex_inputs_ref_post_str
2079         }
2080       %}
2081     }{
2082       \stex_input_with_hooks:Ne\l__stex_inputs_uri {\stex_source_path:n {#1}}
2083     }
2084   }
2085   \cs_new_protected:Nn \__stex_inputs_ref:n {
2086     \ifnum\@listdepth>0
2087       \noindent\vbox{\vskip-26pt\relax\__stex_inputs_ref_i:n{#1}}\par
2088       %\begin{stex_env_node}{div}{style:foo=bar}\vskip-700pt\end{stex_env_node}
2089     \else
2090       \__stex_inputs_ref_i:n{#1}
2091     \fi
2092   }
2093 }
2094 }{
2095   \cs_new_protected:Nn \__stex_inputs_ref:n {
2096     \begingroup
2097     \inputreftrue
2098     \let \l_stex_metatheory_uri \c_stex_default_metatheory
2099     %\seq_clear:N \l_stex_all_modules_seq
2100     \stex_undefine:N \l_stex_current_module_uri
2101     \stex_debug:nn{inputref}{Inputrefing~document~\stex_use_document_uri:N \l__stex_input
2102     \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#1}}
2103     \endgroup\medskip
2104   }
2105 }

```

(End of definition for \inputref. This function is documented on page 140.)

sfunction

\mhinput

```

2106 \NewDocumentCommand \mhinput { 0{} m}{

```

```

2107 \stex_in_archive:nn {#1} {
2108   \stex_document_uri_from_archive_file:Nn \l__stex_inputs_uri {#2}
2109   \ifinputref
2110     \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#2}}
2111   \else
2112     \inputreftrue
2113     \stex_input_with_hooks:Ne\l__stex_inputs_uri{\stex_source_path:n {#2}}
2114     \inputreffalse
2115   \fi
2116 }
2117 }

```

(End of definition for \mhinput. This function is documented on page 140.)

sfunction

\ifinputref

```

2118 \newif \ifinputref \inputreffalse

```

(End of definition for \ifinputref. This function is documented on page 140.)

sfunction

\IfInputref

```

2119 \stex_if_html_backend:TF{
2120   \newcommand \IfInputref[2]{
2121     \stex_annotate:nn{data-ftml-ifinputref=true}{#1}
2122     \stex_annotate:nn{data-ftml-ifinputref=false}{#2}
2123   }
2124 }{
2125   \newcommand \IfInputref[2]{
2126     \ifinputref #1 \else #2 \fi
2127   }
2128 }

```

(End of definition for \IfInputref. This function is documented on page 140.)

sfunction

\libinput

```

2129 \newcommand \libinput [2] [] {
2130   \stex_in_archive:nn{#1}{
2131     \__stex_inputs_up_archive:nn{#2}{tex}
2132     \stex_debug:nn{lib}{Result:-\seq_use:Nn \l__stex_inputs_libinput_files_seq ,}
2133     \seq_if_empty:NTF \l__stex_inputs_libinput_files_seq {
2134       \msg_error:nnnn{stex}{error/nofile}{\libinput}{#2.tex}
2135     }{
2136       \seq_map_function:NN \l__stex_inputs_libinput_files_seq \input
2137     }
2138   }
2139 }

```

(End of definition for \libinput. This function is documented on page 140.)

sfunction

\libusepackage

```

2140 \newcommand\libusepackage[2][]{
2141   \__stex_inputs_up_archive:nn{#2}{sty}
2142   \int_compare:nNnTF {\seq_count:N \l__stex_inputs_libinput_files_seq} = 1 {
2143     \str_set:Nx \l__stex_inputs_tmp_str {\seq_item:Nn \l__stex_inputs_libinput_files_seq 1}
2144     \exp_args:Nne \use:n {\usepackage[#1]} {
2145       \str_range:Nnn\l__stex_inputs_tmp_str 1 {-5}
2146     }
2147   }{
2148     \msg_fatal:nnnn{stex}{error/nofile}{\libusepackage}{#2.sty}
2149   }
2150 }

```

(End of definition for \libusepackage. This function is documented on page 140.)

sfunction

\libusetikzlibrary

```

2151 \newcommand \libusetikzlibrary [2]{} {
2152   \cs_if_exist:NF \usetikzlibrary {
2153     \msg_error:nnx{stex}{error/notikz}{\tl_to_str:n{\libusetikzlibrary}}
2154   }
2155   \tl_if_empty:nTF{#1}{
2156     \__stex_inputs_usetikzlibrary:n{#2}
2157   }{
2158     \stex_in_archive:nn{#1}{\__stex_inputs_usetikzlibrary:n{#2}}
2159   }
2160 }
2161
2162 \cs_new_protected:Nn \__stex_inputs_usetikzlibrary:n{
2163   \__stex_inputs_up_archive:nn{tikzlibrary#1}{code.tex}
2164   \int_compare:nNnTF {\seq_count:N \l__stex_inputs_libinput_files_seq} = 1 {
2165     \exp_args:Nne \__stex_inputs_usetikzlibrary_i:nn{#1}{ \seq_item:Nn \l__stex_inputs_libinput_files_seq 1}
2166   }{
2167     \msg_fatal:nnnn{stex}{error/nofile}{\libusetikzlibrary}{tikzlibrary#1.code.tex}
2168   }
2169 }
2170
2171 \cs_new_protected:Nn \__stex_inputs_usetikzlibrary_i:nn {
2172   \pgfkeys@spdef\pgf@temp{#1}
2173   \expandafter\ifx\csname tikz@library@\pgf@temp @loaded\endcsname\relax%
2174   \expandafter\global\expandafter\let\csname tikz@library@\pgf@temp @loaded\endcsname=\pgf@temp
2175   \expandafter\edef\csname tikz@library@#1@atcode\endcsname{\the\catcode'\@}
2176   \expandafter\edef\csname tikz@library@#1@barcode\endcsname{\the\catcode'\|}
2177   \expandafter\edef\csname tikz@library@#1@dollarcode\endcsname{\the\catcode'\$}
2178   \catcode'\@=11
2179   \catcode'\|=12
2180   \catcode'\$=3
2181   \pgfutil@InputIfFileExists{#2}{\pgf@temp}{\pgf@temp}
2182   \catcode'\@=\csname tikz@library@#1@atcode\endcsname
2183   \catcode'\|=\csname tikz@library@#1@barcode\endcsname
2184   \catcode'\$=\csname tikz@library@#1@dollarcode\endcsname
2185 }

```

(End of definition for \libusetikzlibrary. This function is documented on page 140.)

sfunction

\addmhbibresource

```

2186 \newcommand \addmhbibresource [2] [] {
2187   \stex_in_archive:nn{#1}{
2188     \__stex_inputs_up_archive:nn{#2}{bib}
2189     \stex_debug:nn{lib}{Result:~\seq_use:Nn \l__stex_inputs_libinput_files_seq ,}
2190     \seq_if_empty:NTF \l__stex_inputs_libinput_files_seq {
2191       \msg_error:nnnn{stex}{error/nofile}{\addmhbibresource}{#2.bib}
2192     }{
2193       \seq_map_function:NN \l__stex_inputs_libinput_files_seq \addbibresource
2194     }
2195   }
2196 }

```

(End of definition for \addmhbibresource. This function is documented on page 141.)

sfunction

__stex_inputs_up_archive:nn Iterates up the current archive's directory in search for lib files with name #1.#2

```

2197 \cs_new_protected:Nn \__stex_inputs_up_archive:nn {
2198   \tl_if_exist:NF \l_stex_current_archive {
2199     \msg_error:nnn{stex}{error/notinarchive}\libinput
2200   }
2201   \seq_clear:N \l__stex_inputs_libinput_files_seq
2202   \seq_set_split:Nne \l__stex_inputs_path_seq / {\stex_archive_path:N \l_stex_current_archive}
2203   \seq_set_split:Nne \l__stex_inputs_id_seq / {\stex_archive_id:N \l_stex_current_archive}
2204   \seq_map_inline:Nn \l__stex_inputs_id_seq {
2205     \seq_pop_right:NN \l__stex_inputs_path_seq \l_tmpa_tl
2206   }
2207   \stex_debug:nn{lib}{Checking~#1~in~\seq_use:Nn \l__stex_inputs_path_seq /}
2208
2209   \bool_while_do:nn { ! \seq_if_empty_p:N \l__stex_inputs_id_seq }{
2210     \str_set:Ne \l__stex_inputs_path_str {\stex_file_use:N \l__stex_inputs_path_seq / meta-
2211     \stex_debug:nn{lib}{Checking~\l__stex_inputs_path_str}
2212     \IfFileExists{ \l__stex_inputs_path_str }{
2213       \exp_args:NNo \seq_if_in:NnF \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
2214       \seq_put_right:No \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
2215     }
2216     }{}
2217     \seq_pop_left:NN \l__stex_inputs_id_seq \l__stex_inputs_path_str
2218     \seq_put_right:No \l__stex_inputs_path_seq \l__stex_inputs_path_str
2219   }
2220
2221   \str_set:Ne \l__stex_inputs_path_str {\stex_file_use:N \l__stex_inputs_path_seq / lib / #
2222   \stex_debug:nn{lib}{Checking~\l__stex_inputs_path_str}
2223   \IfFileExists{ \l__stex_inputs_path_str }{
2224     \exp_args:NNo \seq_if_in:NnF \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
2225     \seq_put_right:No \l__stex_inputs_libinput_files_seq \l__stex_inputs_path_str
2226   }
2227   }{}
2228 }

```

(End of definition for __stex_inputs_up_archive:nn.)

\mhgraphics

\cmhgraphics

```

2229 \ltx@ifpackageloaded{graphicx}{\use:n}{\AtEndOfPackageFile{graphicx}}{

```

```

2230 \define@key{Gin}{archive}{
2231     \stex_in_archive:nn{#1}{}
2232     \str_set:Ne \Gin@mhrepos {
2233         \stex_source_path:nn{#1}{}
2234     }
2235 }
2236 \providecommand\mhgraphics[2] [] {
2237     \str_set:Ne \Gin@mhrepos {
2238         \stex_source_path:n{ }
2239     }
2240     \setkeys{Gin}{#1}
2241     \includegraphics[#1]{ \Gin@mhrepos / #2 }
2242 }
2243 \providecommand\cmhgraphics[2] [] {\begin{center}\mhgraphics[#1]{#2}\end{center}}
2244 }

```

(End of definition for `\mhgraphics` and `\cmhgraphics`. These functions are documented on page 141.)

sfunction

`\lstinputmhlisting`
`\clstinputmhlisting`

```

2245 \ltx@ifpackageloaded{listings}{\use:n}{\AtEndOfPackageFile{listings}}{
2246     \define@key{lst}{archive}{
2247         \stex_in_archive:nn{#1}{}
2248         \str_set:Ne \lst@mhrepos{
2249             \stex_source_path:nn{#1}{}
2250         }
2251     }
2252     \newcommand\lstinputmhlisting[2] [] {%
2253         \str_set:Ne \lst@mhrepos{
2254             \stex_source_path:n{ }
2255         }
2256         \setkeys{lst}{#1}%
2257         \lstinputlisting[#1]{\lst@mhrepos / #2}}
2258     \newcommand\clstinputmhlisting[2] [] {\begin{center}\lstinputmhlisting[#1]{#2}\end{center}}
2259 }

```

(End of definition for `\lstinputmhlisting` and `\clstinputmhlisting`. These functions are documented on page 141.)

sfunction

`\mhtikzinput`
`\cmhtikzinput`

```

2260 \ltx@ifpackageloaded{tikzinput}{\use:n}{\AtEndOfPackageFile{tikzinput}}{
2261     \define@key{Gin}{archive}{
2262         \stex_in_archive:nn{#1}{}
2263         \str_set:Ne \Gin@mhrepos{
2264             \stex_source_path:nn{#1}{}
2265         }
2266         \str_set:Ne \l__stex_inputs_gin_repo_str {#1}
2267     }
2268     \newcommand\mhtikzinput[2] [] {%
2269         \str_clear:N \l__stex_inputs_gin_repo_str
2270         \str_set:Ne \Gin@mhrepos{
2271             \stex_source_path:n{ }
2272         }

```

```

2273     \setkeys{Gin}{#1}%
2274     \exp_args:No \stex_in_archive:nn \l__stex_inputs_gin_repo_str {
2275         \tikzinput[#1]{\Gin@mhrepos / #2}
2276     }
2277 }
2278 \newcommand\cmhtikzinput[2][\begin{center}\mhtikzinput[#1]{#2}\end{center}]
2279 }

```

(End of definition for \mhtikzinput and \cmhtikzinput. These functions are documented on page 141.)

sfunction

Chapter 31

Modules

```
2280 (@@=stex_modules)
```

A module consists of four separate macros, storing its *symbols*, *incoming morphisms* (e.g. `\importmodules`), *notations*, and “initialization code”, respectively:

```
2281 \cs_new:Nn \__stex_modules_macro_short:nnn {
2282   #1 ? #2 ? #3
2283 }
2284 \cs_new:Nn \_stex_symbol_macro:N {
2285   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _symbols_prop
2286 }
2287 \cs_new:Nn \_stex_symbol_macro:n {
2288   c_stex_module_ \__stex_modules_macro_short:nnn #1 _symbols_prop
2289 }
2290 \cs_new:Nn \_stex_morphisms_macro:N {
2291   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _morphisms_prop
2292 }
2293 \cs_new:Nn \_stex_morphisms_macro:n {
2294   c_stex_module_ \__stex_modules_macro_short:nnn #1 _morphisms_prop
2295 }
2296 \cs_new:Nn \_stex_notations_macro:N {
2297   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _notations_prop
2298 }
2299 \cs_new:Nn \_stex_notations_macro:n {
2300   c_stex_module_ \__stex_modules_macro_short:nnn #1 _notations_prop
2301 }
2302 \cs_new:Nn \_stex_module_code_macro:N {
2303   c_stex_module_ \exp_after:wN \__stex_modules_macro_short:nnn #1 _code
2304 }
2305 \cs_new:Nn \_stex_module_code_macro:n {
2306   c_stex_module_ \__stex_modules_macro_short:nnn #1 _code
2307 }
```

`\l_stex_current_module_uri` initially undefined.

(End of definition for `\l_stex_current_module_uri`. This variable is documented on page 142.)

`\l_stex_all_modules_seq`

```
2308 \seq_new:N \l_stex_all_modules_seq
```

(End of definition for `\l_stex_all_modules_seq`. This variable is documented on page 142.)

smodule (*env*.)

```

2309 \tl_new:N \l_stex_metatheory_uri
2310
2311 \stex_keys_define:nnnn{smodule}{
2312   \str_clear:N \l_stex_key_sig_str
2313 }{
2314   meta .code:n = {
2315     \tl_if_empty:nTF {#1}{
2316       \tl_clear:N \l_stex_metatheory_uri
2317     }{
2318       \stex_uri_from_pair:Nn \l_stex_metatheory_uri { #1 }
2319     }
2320   },
2321   %ns .code:n = {
2322     % \stex_uri_resolve:Ne \l_stex_current_ns_uri { #1 }
2323     %} ,
2324   %lang .code:n = {
2325     % \stex_set_language:n { #1 }
2326     %} ,
2327   sig .str_set_x:N = \l_stex_key_sig_str ,
2328   %creators .code:n = {} , % todo ?
2329   %contributors .code:n = {} , % todo ?
2330 }{id, title, style, deprecate}
2331
2332 \stex_if_html_backend:T {
2333   \cs_new_protected:Nn \__stex_modules_html_annots: {
2334     \exp_args:Ne \stex_annotate_env {
2335       data-ftml-module={\stex_module_uri_name:N \l_stex_current_module_uri}
2336       %data-ftml-language={ \l_stex_current_language_str},
2337       \str_if_empty:NF\l_stex_key_sig_str {, data-ftml-signature={\l_stex_key_sig_str}}
2338       \tl_if_empty:NF \l_stex_metatheory_uri {,
2339         data-ftml-metatheory={\stex_use_module_uri:N \l_stex_metatheory_uri}
2340       }
2341     }
2342     \stex_annotate_invisible:n{ }
2343   }
2344 }
2345
2346 \stex_new_stylable_env:nnnnnn {module} {0{ } m}{
2347   \stex_keys_set:nn { smodule }{ #1 }
2348   \tl_set_eq:NN \thistitle \l_stex_key_title_tl
2349   \tl_if_empty:NF \thistitle {
2350     \exp_args:No \stexdoctitle \thistitle
2351   }
2352   \stex_module_setup:n { #2 }
2353
2354   \stex_if_do_html:T \__stex_modules_html_annots:
2355   \stex_if_smsmode:F {
2356     \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_current_module_uri }
2357     \str_set:Nn \thismodulename {#2}
2358     \stex_style_apply:
2359   }
2360   \stex_smsmode_do:
2361 }{

```



```

2362 \stex_close_module:
2363 \stex_if_smsmode:F \stex_style_apply:
2364 \stex_if_do_html:T{ \endstex_annotate_env }
2365 }{}{}{s}

```

\stex_module_setup:n

```

2366 \cs_new_protected:Npn \stex_module_setup:n {
2367   \stex_if_in_module:TF \__stex_modules_nested:n \__stex_modules_top:n
2368 }
2369
2370 \cs_new_protected:Nn \__stex_modules_top:n {
2371   \str_if_empty:NTF \l_stex_key_sig_str
2372     \stex_module_setup_top_nosig:n \__stex_modules_top_sig:n {#1}
2373   \g_stex_every_module_tl
2374   \tl_if_empty:NF \l_stex_metatheory_uri {
2375     \stex_debug:nn{modules}{loading~metatheory~\stex_use_module_uri:N \l_stex_metatheory_uri}
2376     \__stex_modules_load_meta:
2377   }
2378 }

```

(End of definition for \stex_module_setup:n. This function is documented on page 142.)

sfunction

\stex_module_setup_top_nosig:n

```

2379 \cs_new_protected:Nn \stex_module_setup_top_nosig:n {
2380   \stex_metagroup_new:
2381   \tl_set:Ne \l__stex_modules_uri { \stex_new_module_uri:n {#1} }
2382   \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
2383   \exp_args:No \stex_if_module_exists:NTF \l__stex_modules_uri {
2384     \stex_debug:nn{modules}{(already exists)}
2385   }{
2386     \tl_gclear:c{ \stex_module_code_macro:N \l__stex_modules_uri }
2387     \prop_gclear:c{ \stex_morphisms_macro:N \l__stex_modules_uri }
2388     \prop_gclear:c{ \stex_symbol_macro:N \l__stex_modules_uri }
2389     \prop_gclear:c{ \stex_notations_macro:N \l__stex_modules_uri }
2390   }
2391   \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
2392   \seq_put_right:No \l_stex_all_modules_seq \l__stex_modules_uri
2393 }

```

(End of definition for \stex_module_setup_top_nosig:n. This function is documented on page 142.)

sfunction

__stex_modules_load_meta: loads the metatheory:

```

2394 \bool_new:N \l_stex_in_meta_bool
2395
2396 \cs_new_protected:Nn \__stex_modules_load_meta: {
2397   \exp_after:wN \stex_execute_in_module:n \exp_after:wN {
2398     \exp_after:wN \__stex_modules_load_meta_i:n \exp_after:wN { \l_stex_metatheory_uri }
2399   }
2400 }
2401
2402 \cs_new_protected:Nn \__stex_modules_load_meta_i:n {
2403   \bool_if:NTF \l_stex_in_meta_bool {

```

```

2404 \stex_activate_module:n {#1}
2405 }{
2406 \bool_set_true:N \l_stex_in_meta_bool
2407 \stex_activate_module:n {#1}
2408 \bool_set_false:N \l_stex_in_meta_bool
2409 }
2410 }

```

(End of definition for _stex_modules_load_meta:.)

_stex_modules_top_sig:n sets up a new module with a signature:

```

2411 \cs_new_protected:Nn \_stex_modules_top_sig:n {
2412 \stex_debug:nn{modules}{Signature:\l_stex_key_sig_str}
2413 \stex_metagroup_new:
2414 \tl_set:Nc \l__stex_modules_uri { \stex_new_module_uri:n {#1} }
2415 \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
2416 \stex_if_module_exists:NTF \l__stex_modules_uri {
2417 \stex_debug:nn{modules}{(already exists)}
2418 \stex_activate_module:N \l__stex_modules_uri
2419 }{
2420 \stex_debug:nn{modules}{(needs loading)}
2421 \_stex_modules_load_sig:
2422 }
2423 \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
2424 }
2425
2426 \cs_new_protected:Nn \_stex_modules_load_sig: {
2427 \stex_file_split_off_lang:NN \l__stex_modules_sigfile \g_stex_current_file
2428 \tl_set:Nc \l__stex_modules_sig {
2429 \exp_args:NNo \stex_document_uri_with_language:Nn
2430 \l_stex_current_document_uri \l_stex_key_sig_str
2431 }
2432 \stex_debug:nn{modules}{Loading~signature~\stex_use_document_uri:N \l__stex_modules_sig;~
2433 \exp_args:NNe \stex_file_in_smsmode:Nn \l__stex_modules_sig {
2434 \stex_file_use:N \l__stex_modules_sigfile . \l_stex_key_sig_str . tex
2435 }
2436 \stex_activate_module:N \l__stex_modules_uri
2437 }

```

(End of definition for _stex_modules_top_sig:n.)

_stex_modules_nested:n sets up a new nested module:

```

2438 \cs_new_protected:Nn \_stex_modules_nested:n {
2439 \stex_metagroup_new:
2440 \stex_debug:nn{module}{Nested~module~#1~in~\stex_use_module_uri:N \l_stex_current_module_uri}
2441 \tl_set:Nc \l__stex_modules_uri { \stex_new_module_uri:n{#1} }
2442 \stex_debug:nn{module}{URI:\stex_use_module_uri:N \l__stex_modules_uri}
2443 \exp_args:No \stex_if_module_exists:NTF \l__stex_modules_uri {
2444 \stex_debug:nn{modules}{(already exists)}
2445 }{
2446 \tl_gclear:c{ \_stex_module_code_macro:N \l__stex_modules_uri }
2447 \prop_gclear:c{ \_stex_morphisms_macro:N \l__stex_modules_uri }
2448 \prop_gclear:c{ \_stex_symbol_macro:N \l__stex_modules_uri }
2449 \prop_gclear:c{ \_stex_notations_macro:N \l__stex_modules_uri }
2450 }

```

```

2451 \tl_set_eq:NN \l_stex_current_module_uri \l__stex_modules_uri
2452 \seq_put_left:No \l_stex_all_modules_seq \l__stex_modules_uri
2453 }

```

(End of definition for __stex_modules_nested:n.)

\stex_close_module:

```

2454 \cs_new_protected:Nn \stex_close_module: {
2455   \bool_if:NT \c_stex_persist_write_mode_bool \__stex_modules_persist_module:
2456   \stex_debug:nn{module}{
2457     Closing~module~\stex_use_module_uri:N \l_stex_current_module_uri^^J
2458     Code:~\cs_meaning:c{ \stex_module_code_macro:N \l_stex_current_module_uri }^^J
2459     Imports:~ \exp_after:wN \prop_to_keyval:N \cs:w \stex_morphisms_macro:N \l_stex_current_module_uri
2460     Declarations:~ \exp_after:wN \prop_to_keyval:N \cs:w \stex_symbol_macro:N \l_stex_current_module_uri
2461     Notations:~ \exp_after:wN \prop_to_keyval:N \cs:w \stex_notations_macro:N \l_stex_current_module_uri
2462   }
2463 }

```

Persist and restore modules in the .sms-file:

```

2464 \cs_new_protected:Nn \__stex_modules_persist_module: {
2465   \stex_persist:e {
2466     \__stex_modules_restore_module:nnnn {\l_stex_current_module_uri}{
2467       \exp_after:wN \prop_to_keyval:N \cs:w
2468         \stex_morphisms_macro:N \l_stex_current_module_uri
2469       \cs_end:
2470     }{
2471       \exp_after:wN \prop_to_keyval:N \cs:w
2472         \stex_symbol_macro:N \l_stex_current_module_uri
2473       \cs_end:
2474     }{
2475       \exp_after:wN \exp_after:wN \exp_after:wN \exp_not:n
2476       \exp_after:wN \exp_after:wN \exp_after:wN
2477       { \cs:w \stex_module_code_macro:N \l_stex_current_module_uri \cs_end: }
2478     }{}
2479     \prop_map_function:cN{\stex_notations_macro:N \l_stex_current_module_uri}
2480     \__stex_modules_persist_not_i:nn
2481     \exp_not:N \STEXRestoreNotEnd {}
2482   }
2483 }
2484
2485 \cs_new:Nn \__stex_modules_persist_not_i:nn {
2486   \exp_not:n{#2}
2487 }
2488
2489
2490 \cs_new:Nn \__stex_modules_stringify_uri:nnn {
2491   {\tl_to_str:n{#1}}
2492   {\tl_to_str:n{#2}}
2493   {\tl_to_str:n{#3}}
2494 }
2495 \cs_new:Nn \__stex_modules_stringify_uri:nnnn {
2496   {\tl_to_str:n{#1}}
2497   {\tl_to_str:n{#2}}
2498   {\tl_to_str:n{#3}}
2499   {\tl_to_str:n{#4}}

```

```

2500 }
2501
2502 \cs_new_protected:Nn \__stex_modules_restore_module:nnnn {
2503   \tl_set:Nc \l__stex_modules_uri { \__stex_modules_stringify_uri:nnn #1 }
2504   \stex_debug:nn{persist}{restoring~\stex_use_module_uri:N \l__stex_modules_uri}
2505   \prop_gset_from_keyval:cn{ \stex_morphisms_macro:N \l__stex_modules_uri }{#2}
2506   \prop_gset_from_keyval:cn{ \stex_symbol_macro:N \l__stex_modules_uri }{#3}
2507   \prop_map_inline:cn{ \stex_symbol_macro:N \l__stex_modules_uri }{
2508     \stex_ref_new_symbol:n{#1?##1}
2509   }
2510   \def \l_tmpa_tl { #4 }
2511   \exp_args:Nno \tl_gset:cn{ \stex_module_code_macro:N \l__stex_modules_uri } \l_tmpa_tl
2512   \prop_gclear:c{ \stex_notations_macro:N \l__stex_modules_uri }
2513   \group_begin:
2514     \catcode'_{=8\relax
2515     \catcode'_{=12\relax
2516     \__stex_modules_restore_notcs:n
2517   }
2518
2519 \quark_new:N \STEXRestoreNotcsEnd
2520
2521 \cs_new_protected:Nn \__stex_modules_restore_notcs:n {
2522   \__stex_modules_restore_notcs_i:n
2523 }
2524
2525 \cs_new_protected:Nn \__stex_modules_restore_notcs_i:n {
2526   \tl_if_eq:nnTF{#1}{\STEXRestoreNotcsEnd}{
2527     \group_end:
2528   }{
2529     \__stex_modules_restore_notcs_ii:nnnnn {#1}
2530   }
2531 }
2532
2533 \cs_new_protected:Nn \__stex_modules_restore_notcs_ii:nnnnn {
2534   \tl_set:Nc \l__stex_modules_tl {{{#4}{#5}}}
2535   % eliminate ## => #
2536   \exp_after:wN \def \exp_after:wN \l__stex_modules_tl \exp_after:wN {\l__stex_modules_tl}
2537   \exp_args:NNe\use:nn\prop_gput:cnn{
2538     { \stex_notations_macro:N \l__stex_modules_uri }
2539     { \stex_use_symbol_uri:n{#1}\tl_to_str:n{?#2} }{
2540       { \__stex_modules_stringify_uri:nnnn #1 }{ \tl_to_str:n{#2} }{#3}
2541       \exp_args:No \exp_not:n \l__stex_modules_tl
2542     }
2543   }
2544   \__stex_modules_restore_notcs_i:n
2545 }

```

(End of definition for \stex_close_module:. This function is documented on page 142.)

sfunction

\stex_every_module:n

```

2546 \tl_new:N \g_stex_every_module_tl
2547 \cs_new_protected:Nn \stex_every_module:n {
2548   \tl_gput_right:Nn \g_stex_every_module_tl { #1 }
2549 }

```

(End of definition for `\stex_every_module:n`. This function is documented on page 142.)

sfunction

`\stex_do_up_to_module:n`

`\stex_do_up_to_module:e`

```
2550 \cs_new_protected:Nn \stex_do_up_to_module:n {
2551   \stex_metagroup_do_in:n {#1}
2552 }
2553 \cs_generate_variant:Nn \stex_do_up_to_module:n {e}
```

(End of definition for `\stex_do_up_to_module:n`. This function is documented on page 142.)

sfunction

`\stex_execute_in_module:n`

`\stex_execute_in_module:e`

```
2554 \cs_new_protected:Nn \stex_execute_in_module:n { \stex_if_in_module:TF {
2555   \tl_gput_right:cn { \_stex_module_code_macro:N \l_stex_current_module_uri } { #1 }
2556   \stex_debug:nn{modules}{extending~activation~for~\stex_use_module_uri:N \l_stex_current_m
2557   \stex_do_up_to_module:n { #1 }
2558 }{ #1 }}
2559 \cs_generate_variant:Nn \stex_execute_in_module:n {e}
```

(End of definition for `\stex_execute_in_module:n`. This function is documented on page 142.)

sfunction

`\stex_if_in_module_p:`

`\stex_if_in_module:TF`

```
2560 \prg_new_conditional:Nnn \stex_if_in_module: {p, T, F, TF} {
2561   \tl_if_exist:NTF \l_stex_current_module_uri
2562   \prg_return_true: \prg_return_false:
2563 }
```

(End of definition for `\stex_if_in_module:TF`. This function is documented on page 143.)

sfunction

`\stex_if_module_exists_p:N`

`\stex_if_module_exists:NTF`

`\stex_if_module_exists_p:n`

`\stex_if_module_exists:nTF`

```
2564 \prg_new_conditional:Nnn \stex_if_module_exists:N {p, T, F, TF} {
2565   \tl_if_exist:cTF { \_stex_module_code_macro:N #1 }
2566   \prg_return_true: \prg_return_false:
2567 }
2568 \prg_new_conditional:Nnn \stex_if_module_exists:n {p, T, F, TF} {
2569   \tl_if_exist:cTF { \_stex_module_code_macro:n {#1} }
2570   \prg_return_true: \prg_return_false:
2571 }
```

(End of definition for `\stex_if_module_exists:N` and `\stex_if_module_exists:n`. These functions are documented on page 143.)

sfunction

`\stex_activate_module:N`

`\stex_activate_module:n`

```
2572 \cs_new_protected:Nn \stex_activate_module:N {
2573   \exp_args:No \stex_activate_module:n #1
2574 }
2575
2576 \cs_new_protected:Nn \stex_activate_module:n {
2577   \seq_if_in:NnF \l_stex_all_modules_seq {#1} {
2578     \stex_debug:nn{modules}{Activating~module~\stex_use_module_uri:n{#1}}
```

```

2579 \seq_put_right:Nn \l_stex_all_modules_seq {#1}
2580 \stex_pseudogroup:nn{
2581   \tl_set:Nn \l_stex_current_module_uri {#1}
2582   \tl_if_exist:cF { \_stex_module_code_macro:N \l_stex_current_module_uri }{
2583     \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:n {#1} }
2584   }
2585   \_stex_activate_symbols:
2586   \_stex_activate_notations:
2587   \stex_debug:nn{modules}{Executing:~\expandafter\meaning\csname\_stex_module_code_macro:N \l_stex_current_module_uri }
2588   \use:c{ \_stex_module_code_macro:N \l_stex_current_module_uri }
2589 }{
2590   \stex_pseudogroup_restore:N \l_stex_current_module_uri
2591 }
2592 \stex_debug:nn{modules}{Activated~module~\stex_use_module_uri:n {#1} }
2593 }
2594 }

```

(End of definition for `\stex_activate_module:N` and `\stex_activate_module:n`. These functions are documented on page 143.)

sfunction

`\setmetatheory`

```

2595 \NewDocumentCommand \setmetatheory {0{} m}{
2596   \stex_debug:nn{metatheory}{Setting~metatheory~[#1]#2}
2597   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2598   \stex_debug:nn{metatheory}{Here:\stex_use_module_uri:N \l_stex_import_uri
2599 }
2600   \tl_set_eq:NN \l_stex_metatheory_uri \l_stex_import_uri
2601   \begingroup
2602     \stex_require_module:N \l_stex_metatheory_uri
2603   \endgroup
2604   \stex_smsmode_do:
2605 }

```

(End of definition for `\setmetatheory`. This function is documented on page 143.)

sfunction

`\STEXexport`

```

2606 \cs_new_protected:Npn \STEXexport {
2607   \ExplSyntaxOn
2608   \__stex_modules_export:n
2609 }
2610 \cs_new_protected:Nn \__stex_modules_export:n {
2611   \stex_ignore_spaces_and_pars:#1\ExplSyntaxOff
2612   \tl_gput_right:cn { \_stex_module_code_macro:N \l_stex_current_module_uri } {
2613     \stex_ignore_spaces_and_pars: #1
2614   }
2615   \stex_smsmode_do:
2616 }
2617
2618 \stex_deactivate_macro:Nn \STEXexport {module~environments}
2619 \stex_every_module:n {\stex_reactivate_macro:N \STEXexport}

```

(End of definition for `\STEXexport`. This function is documented on page 143.)

sfunction

```

\stex_structural_feature_module:nn
\stex_structural_feature_module_end:
2620 \cs_new_protected:Nn \stex_structural_feature_module:nn {
2621   \stex_if_do_html:TF {
2622     \exp_args:Nne \begin{stex_annotate_env} {
2623       data-ftml-feature-#2={#1}
2624       \str_if_empty:NF \l_stex_macroname_str {,
2625         data-ftml-macroname={\l_stex_macroname_str}
2626       }
2627     }
2628     \stex_annotate_invisible:n{ }
2629   }\group_begin:
2630   \stex_module_setup:n {#1}
2631 }
2632
2633 \cs_new_protected:Nn \stex_structural_feature_module_end: {
2634   \tl_gset_eq:NN \g_stex_last_feature_str \l_stex_current_module_str
2635   \stex_close_module:
2636   \stex_if_do_html:TF{
2637     \end{stex_annotate_env}
2638   }\group_end:
2639 }

(End of definition for \stex_structural_feature_module:nn and \stex_structural_feature_module_end:.)
These functions are documented on page 143.)
sfunction

```

31.1 SMS Mode

```

2640 <@@=stex_smsmode>

\stex_sms_allow:N

2641 \tl_new:N \g__stex_smsmode_allowed_tl
2642
2643 \cs_new_protected:Nn \stex_sms_allow:N {
2644   \tl_gput_right:Nn \g__stex_smsmode_allowed_tl {#1}
2645 }

(End of definition for \stex_sms_allow:N. This function is documented on page 144.)
sfunction

\stex_sms_allow_escape:N

2646 \tl_new:N \g__stex_smsmode_allowed_escape_tl
2647
2648 \cs_new_protected:Nn \stex_sms_allow_escape:N {
2649   \tl_gput_right:Nn \g__stex_smsmode_allowed_escape_tl {#1}
2650 }

(End of definition for \stex_sms_allow_escape:N. This function is documented on page 144.)
sfunction

\stex_sms_allow_env:n

2651 \seq_new:N \g__stex_smsmode_allowedenvs_seq
2652
2653 \cs_new_protected:Nn \stex_sms_allow_env:n {

```

```

2654 \seq_gput_right:Ne \g__stex_smsmode_allowedenvs_seq {\tl_to_str:n{#1}}
2655 }

```

(End of definition for \stex_sms_allow_env:n. This function is documented on page 144.)

```

sfunction
Some initial allowed macros:
2656 \stex_sms_allow:N \makeatletter
2657 \stex_sms_allow:N \makeatother
2658 \stex_sms_allow:N \ExplSyntaxOn
2659 \stex_sms_allow:N \ExplSyntaxOff
2660 \stex_sms_allow:N \rustexBREAK
2661 \stex_sms_allow_env:n{smodule}
2662 \stex_sms_allow_escape:N \setmetatheory
2663 \stex_sms_allow_escape:N \STEXexport

```

\stex_sms_allow_import:Nn

```

2664 \tl_new:N \g__stex_smsmode_allowed_import_tl
2665 \tl_new:N \g__stex_smsmode_import_setup_tl
2666
2667 \cs_new_protected:Nn \stex_sms_allow_import:Nn {
2668   \tl_gput_right:Nn \g__stex_smsmode_allowed_import_tl {#1}
2669   \tl_gput_right:Nn \g__stex_smsmode_import_setup_tl {#2}
2670 }

```

(End of definition for \stex_sms_allow_import:Nn. This function is documented on page 144.)

sfunction

\stex_sms_allow_import_env:nn

```

2671 \seq_new:N \g__stex_smsmode_allowed_import_env_seq
2672
2673 \cs_new_protected:Nn \stex_sms_allow_import_env:nn {
2674   \seq_gput_right:Ne \g__stex_smsmode_allowed_import_env_seq {\tl_to_str:n{#1}}
2675   \tl_gput_right:Nn \g__stex_smsmode_import_setup_tl {#2}
2676 }
2677
2678 %\stex_sms_allow_import_env:nn{smodule}{}

```

(End of definition for \stex_sms_allow_import_env:nn. This function is documented on page 144.)

sfunction

\g_stex_sms_import_code

```

2679 \tl_new:N \g_stex_sms_import_code

```

(End of definition for \g_stex_sms_import_code. This variable is documented on page 144.)

\stex_if_smsmode_p:

\stex_if_smsmode:TF

```

2680 \bool_new:N \g__stex_smsmode_bool
2681
2682 \prg_new_conditional:Nnn \stex_if_smsmode: { p, T, F, TF } {
2683   \bool_if:NTF \g__stex_smsmode_bool \prg_return_true: \prg_return_false:
2684 }

```

(End of definition for \stex_if_smsmode:TF. This function is documented on page 144.)

sfunction

\stex_file_in_smsmode:Nn

```

2685
2686 \seq_new:N \l__stex_smsmode_cycles_seq
2687
2688 \cs_new_protected:Nn \stex_file_in_smsmode:Nn {
2689   \seq_gclear:N \l__stex_smsmode_importmodules_seq
2690   \seq_gclear:N \l__stex_smsmode_sigmodules_seq
2691   \tl_clear:N \g_stex_sms_import_code
2692   \seq_if_in:NnF \l__stex_smsmode_cycles_seq {#2} {
2693     \group_begin:
2694       \seq_push:Nn \l__stex_smsmode_cycles_seq {#2}
2695       \let \l_stex_metatheory_uri \c_stex_default_metatheory
2696       \seq_clear:N \l_stex_all_modules_seq
2697       \stex_undefine:N \l_stex_current_module_uri
2698       \cs_set:Npn \stex_check_term:nn ##1 ##2 {}
2699       \stex_debug:nn{sms}{Loading~#2~in~\stex_use_document_uri:N #1}
2700       \stex_with_file_hooks:Nnn #1 {#2}{
2701         \__stex_smsmode_in_smsmode:n {
2702           \group_begin:
2703             \let \__stex_smsmode_do_aux_curr:N \__stex_smsmode_do_aux_imports:N
2704             \g__stex_smsmode_import_setup_tl
2705             \__stex_smsmode_start_smsmode:n{#2}
2706           \group_end:
2707           %{
2708             \g_stex_sms_import_code
2709           %}
2710           %\group_begin:
2711             \let \__stex_smsmode_do_aux_curr:N \__stex_smsmode_do_aux_normal:N
2712             \__stex_smsmode_start_smsmode:n{#2}
2713           %\group_end:
2714         }
2715       }
2716     \group_end:
2717   }
2718 }
2719
2720 \cs_new_protected:Nn \__stex_smsmode_in_smsmode:n {
2721   \stex_suppress_html:n {
2722     \vbox_set:Nn \l_tmpa_box {
2723       \bool_set_true:N \g__stex_smsmode_bool
2724       #1
2725     }
2726   }
2727 }
2728
2729 \quark_new:N \q__stex_smsmode_break
2730
2731 \cs_new_protected:Nn \__stex_smsmode_start_smsmode:n {
2732   \everyeof{\q__stex_smsmode_break\exp_not:N}
2733   \let\stex_smsmode_do:\__stex_smsmode_smsmode_do:
2734   \exp_after:wN \exp_after:wN \exp_after:wN
2735   \stex_smsmode_do:
2736   \cs:w @ @ input\cs_end: "#1" \relax
2737 }

```

(End of definition for \stex_file_in_smsmode:Nn. This function is documented on page 144.)

sfunction

\stex_smsmode_do:

```

2738 \let\stex_smsmode_do:\relax
2739
2740 \cs_new_protected:Nn \__stex_smsmode_smsmode_do: { \__stex_smsmode_do:w }
2741
2742 \cs_new_protected:Npn \__stex_smsmode_do:w #1 {
2743   \exp_args:Ne \tl_if_empty:nTF { \tl_tail:n{ #1 } }{
2744     %\__stex_smsmode_check_cs:NNn \__stex_smsmode_do_aux:N \__stex_smsmode_do:w { #1 }
2745     \__stex_smsmode_check_cs:N {#1}
2746   }{
2747     \__stex_smsmode_do:w #1
2748   }
2749 }
2750
2751 \cs_new_protected:Nn \__stex_smsmode_check_cs:N {
2752   %\stex_debug:nn{temp}{Checking \exp_not:N#1 \relax}
2753   \exp_after:wN\if\exp_after:wN\relax\noexpand #1 ~
2754   \exp_after:wN \__stex_smsmode_do_aux:N \exp_after:wN#1\else
2755   \exp_after:wN \__stex_smsmode_do:w \fi
2756 }
2757
2758 %\cs_new_protected:Nn \__stex_smsmode_check_cs:NNn {
2759 %  \message{^^JHERE: \tl_to_str:n{#1~and~#2~and~#3}}
2760 %  \exp_after:wN\if\exp_after:wN\relax\exp_not:N#3
2761 %  \exp_after:wN#1\exp_after:wN#3\else
2762 %  \exp_after:wN#2\fi
2763 %}
2764
2765 \cs_new_protected:Nn \__stex_smsmode_do_aux:N {
2766   \cs_if_eq:NNF #1 \q__stex_smsmode_break {
2767     \__stex_smsmode_do_aux_curr:N #1
2768   }
2769 }
2770
2771 \cs_new_protected:Nn \__stex_smsmode_do_aux_normal:N {
2772   \tl_if_in:NnTF \g__stex_smsmode_allowed_tl {#1} {
2773     \stex_debug:nn{sms}{Executing~\tl_to_str:n{#1}}
2774     #1\__stex_smsmode_do:w
2775   }{
2776     \tl_if_in:NnTF \g__stex_smsmode_allowed_escape_tl {#1} {
2777       \stex_debug:nn{sms}{Executing~escaped~\tl_to_str:n{#1}}
2778       #1
2779     }{
2780       \cs_if_eq:NNTF \begin #1 {
2781         \__stex_smsmode_check_begin:Nn \g__stex_smsmode_allowedenvs_seq
2782       }{
2783         \cs_if_eq:NNTF \end #1 {
2784           \__stex_smsmode_check_end:Nn \g__stex_smsmode_allowedenvs_seq
2785         }{
2786           \__stex_smsmode_do:w
2787         }

```

```

2788     }
2789   }
2790 }
2791 }
2792
2793 \cs_new_protected:Nn \__stex_smsmode_do_aux_imports:N {
2794   \tl_if_in:NnTF \g__stex_smsmode_allowed_import_tl {#1} {
2795     \stex_debug:nn{sms}{Executing~\tl_to_str:n{#1}~in~import}
2796     #1
2797   }{
2798     \cs_if_eq:NNTF \begin #1 {
2799       \__stex_smsmode_check_begin:Nn \g__stex_smsmode_allowed_import_env_seq
2800     }{
2801       \cs_if_eq:NNTF \end #1 {
2802         \__stex_smsmode_check_end:Nn \g__stex_smsmode_allowed_import_env_seq
2803       }{
2804         \__stex_smsmode_do:w
2805       }
2806     }
2807   }
2808 }
2809
2810 \cs_new_protected:Nn \__stex_smsmode_check_begin:Nn {
2811   \seq_if_in:NeTF #1 { \tl_to_str:n{#2} }{
2812     \stex_debug:nn{sms}{Environment~#2}
2813     \begin{#2}
2814   }{
2815     \__stex_smsmode_do:w
2816   }
2817 }
2818 \cs_new_protected:Nn \__stex_smsmode_check_end:Nn {
2819   \seq_if_in:NeTF #1 { \tl_to_str:n{#2} }{
2820     \stex_debug:nn{sms}{End~Environment~#2}
2821     \end{#2}\__stex_smsmode_do:w
2822   }{
2823     \__stex_smsmode_do:w
2824   }
2825 }

```

(End of definition for `\stex_smsmode_do:.` This function is documented on page 144.)

sfunction

31.2 Inheritance and Morphisms

```

2826 <@@=stex_import>

\stex_require_module:N
\stex_require_module_noerr:N
\stex_require_module_noerr:n
2827 \cs_new_protected:Nn \stex_require_module:N {
2828   \stex_if_module_exists:NF #1 {
2829     \__stex_import_load_module:NF #1 {
2830       \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:N #1}
2831     }
2832   }
2833   \stex_activate_module:N #1

```

```

2834 }
2835
2836 \cs_new_protected:Nn \stex_require_module_noerr:N {
2837   \stex_if_module_exists:NF #1 {
2838     \__stex_import_load_module:NF #1 {}
2839     \str_if_empty:NF \l__stex_import_file_str {
2840       \stex_activate_module:N #1
2841     }
2842   }
2843 }
2844
2845 \cs_new_protected:Nn \stex_require_module_noerr:n {
2846   \tl_set:Nn \l__stex_import_uri { #1 }
2847   \stex_require_module_noerr:N \l__stex_import_uri
2848 }
2849
2850 \cs_new_protected:Npn \__stex_import_load_module:NF #1 #2 {
2851   \tl_set_eq:NN \l__stex_import_uri #1
2852   \tl_set:Ne \l__stex_import_archive_str { \stex_module_uri_archive:N #1 }
2853   \tl_set:Ne \l__stex_import_path_str { \stex_module_uri_path:N #1 }
2854   \tl_set:Ne \l__stex_import_name_str { \stex_module_uri_name:N #1 }
2855   \str_if_eq:NNTF \l__stex_import_archive_str \c_stex_no_archive_str {
2856     \str_set_eq:NN \l__stex_import_pre_str \l__stex_import_path_str
2857     \__stex_import_load_check:n {#2}
2858   }{
2859     \exp_args:No \stex_in_archive:nn \l__stex_import_archive_str {
2860       \str_set:Ne \l__stex_import_pre_str {
2861         \exp_args:Ne \stex_source_path:n{ \l__stex_import_path_str }
2862       }
2863       \exp_args:No \__stex_import_load_check:n {#2}
2864     }
2865   }
2866 }
2867
2868 \cs_new_protected:Nn \__stex_import_load_check:n {
2869   \str_clear:N \l__stex_import_file_str
2870   \str_set_eq:NN \l__stex_import_language_str \l_stex_current_language_str
2871   \__stex_import_check_file:nnn{ / \l__stex_import_name_str .tex }{}{
2872     \__stex_import_check_file:nnn{/ \l__stex_import_name_str . \l_stex_current_language_str
2873     \__stex_import_check_file:nnn{/ \l__stex_import_name_str .en.tex}{
2874       \str_set:Nn \l__stex_import_language_str {en}
2875     }{
2876       \__stex_import_load_check_i:n{#1}
2877     }
2878   }
2879 }
2880 \__stex_import_load_check_ii:
2881 }
2882
2883 \cs_new_protected:Nn \__stex_import_load_check_i:n {
2884   \exp_args:NNNo \seq_set_split:Nnn \l_tmpa_seq / \l__stex_import_path_str
2885   \seq_pop_right:NN \l_tmpa_seq \l__stex_import_name_str
2886   \str_set:Ne \l__stex_import_path_str { \seq_use:Nn \l_tmpa_seq /}
2887   \__stex_import_check_file:nnn{.tex}{}{

```

```

2888 \__stex_import_check_file:nnn{. \l_stex_current_language_str .tex}{ }{
2889 \__stex_import_check_file:nnn{.en.tex}{
2890 \str_set:Nn \l__stex_import_language_str {en}
2891 }{
2892 #1
2893 }
2894 }
2895 }
2896 }
2897
2898 \cs_new_protected:Nn \__stex_import_load_check_ii: {
2899 \str_if_empty:NF \l__stex_import_file_str {
2900 \stex_if_smsmode:TF{
2901 \exp_args:NNo \exp_args:Nne \str_if_eq:nnTF{\l__stex_import_file_str}{\stex_file_use:
2902 \stex_debug:nn{imports}{Skipping~current~file}
2903 }{
2904 \IfFileExists{ \l__stex_import_file_str }{
2905 \__stex_import_load_file:
2906 }{}
2907 }
2908 }{
2909 \IfFileExists{ \l__stex_import_file_str }{
2910 \__stex_import_load_file:
2911 }{}
2912 }
2913 }
2914 }
2915
2916 \cs_new_protected:Npn \__stex_import_check_file:nnn #1 #2 {
2917 \stex_debug:nn{imports}{Checking~ \l__stex_import_pre_str #1}
2918 \IfFileExists{ \l__stex_import_pre_str #1 }{
2919 \stex_debug:nn{imports}{Success}
2920 \str_set:Ne \l__stex_import_file_str { \l__stex_import_pre_str #1 }
2921 #2
2922 }
2923 }
2924
2925 \cs_new_protected:Nn \__stex_import_load_file: {
2926 \tl_set:Ne \l__stex_import_doc_uri {
2927 { \l__stex_import_archive_str }
2928 { \l__stex_import_path_str }
2929 { \l__stex_import_name_str }
2930 { \l__stex_import_language_str }
2931 {}
2932 }
2933 \exp_args:NNo \stex_file_in_smsmode:Nn \l__stex_import_doc_uri \l__stex_import_file_str
2934 \stex_if_module_exists:NF \l__stex_import_uri {
2935 \msg_error:nnx{stex}{error/unknownmodule}{\stex_use_module_uri:N \l__stex_import_uri}
2936 }
2937 }

```

(End of definition for `\stex_require_module:N`, `\stex_require_module_noerr:N`, and `\stex_require_module_noerr:n`. These functions are documented on page 145.)

sfunction

\usemodule

```

2938 \stex_new_stylable_cmd:nnnn {usemodule} { 0{ } m } {
2939   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2940   \stex_require_module:N \l_stex_import_uri
2941   \__stex_import_usemodule:
2942 }{ }%{\aftergroup\ignorespaces}
2943
2944 \cs_new_protected:Nn \__stex_import_usemodule: {
2945   \stex_debug:nn{usemodule}{Done.}
2946   \stex_if_do_html:T {
2947     \hbox{\stex_annotate_invisible:nn
2948       {data-ftml-usemodule=\stex_use_module_uri:N \l_stex_import_uri } {}}
2949   }
2950   \stex_if_smsmode:F{
2951     \group_begin:
2952     \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
2953     \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
2954     \tl_clear:N \thisstyle
2955     \stex_style_apply:
2956     \group_end:
2957   }
2958 }
2959 \stex_deactivate_macro:Nn \usemodule {document~environment}
2960 \AtBeginDocument{\stex_reactivate_macro:N \usemodule}

```

(End of definition for \usemodule. This function is documented on page 145.)

sfunction

\importmodule

```

2961 \stex_new_stylable_cmd:nnnn{importmodule} { 0{ } m } {
2962   \__stex_import_import_module:nn {#1}{#2}
2963   \stex_smsmode_do:
2964 }{ }\aftergroup\ignorespaces}
2965
2966 \stex_deactivate_macro:Nn \importmodule {module~environments}
2967 \stex_every_module:n {\stex_reactivate_macro:N \importmodule}
2968 \stex_sms_allow_escape:N \importmodule
2969
2970 \cs_new_protected:Nn \__stex_import_import_module:nn {
2971   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2972   \stex_require_module:N \l_stex_import_uri
2973   \__stex_import_import_module_i:n {#1}
2974 }
2975
2976 \cs_new_protected:Nn \__stex_import_import_module_i:n {
2977   \stex_execute_in_module:e{
2978     \stex_activate_module:n { \l_stex_import_uri }
2979   }
2980   \stex_add_morphism:nonn
2981     {}{\l_stex_import_uri}{import}{ }{ }
2982   \stex_if_do_html:T {
2983     \stex_annotate_invisible:nn
2984       {data-ftml-import=\stex_use_module_uri:N \l_stex_import_uri } { }
2985   }

```

```

2986 \stex_if_smsmode:F{
2987   \group_begin:
2988   \tl_set:Nn \thisarchive {#1}
2989   \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
2990   \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
2991   \tl_clear:N \thisstyle
2992   \stex_style_apply:
2993   \group_end:
2994 }
2995 }

```

In sms mode, we change the definition:

```

2996 \cs_new_protected:Nn \__stex_import_import_module_presms:nn {
2997   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
2998   \bool_if:NF \l_stex_relative_import_bool {
2999     \tl_gput_right:Ne \g_stex_sms_import_code {
3000       \stex_require_module_noerr:n { \l_stex_import_uri }
3001     }
3002   }
3003 }
3004
3005 \stex_sms_allow_import:Nn \importmodule {
3006   \stex_reactivate_macro:N \importmodule
3007   \let \__stex_import_import_module:nn \__stex_import_import_module_presms:nn
3008 }

```

(End of definition for `\importmodule`. This function is documented on page 145.)

sfunction

`\requiremodule`

```

3009 \stex_new_stylable_cmd:nnnn {requiremodule} { 0{ } m } {
3010   \stex_uri_from_pair:Nnn \l_stex_import_uri { #1 }{ #2 }
3011   \stex_require_module:N \l_stex_import_uri
3012   \__stex_import_requiremodule:
3013 }{ }{\aftergroup\ignorespaces}
3014 \stex_deactivate_macro:Nn \requiremodule {module~environments}
3015 \stex_every_module:n {\stex_reactivate_macro:N \requiremodule}
3016
3017 \cs_new_protected:Nn \__stex_import_requiremodule: {
3018   \stex_debug:nn{requiremodule}{Done.}
3019   \stex_if_do_html:T {
3020     \hbox{\stex_annotate_invisible:nn
3021       {data-ftml-import=\stex_use_module_uri:N \l_stex_import_uri } {}}
3022   }
3023   \stex_if_smsmode:F{
3024     \group_begin:
3025     \str_set:Ne \thismoduleuri {\stex_use_module_uri:N \l_stex_import_uri }
3026     \str_set:Ne \thismodulename {\stex_module_uri_name:N \l_stex_import_uri}
3027     \tl_clear:N \thisstyle
3028     \stex_style_apply:
3029     \group_end:
3030   }
3031 }

```

(End of definition for `\requiremodule`. This function is documented on page 145.)

sfunction

`\stex_add_morphism:nnnn`

```

3032 \cs_new_protected:Nn \stex_add_morphism:nnnn {
3033   \exp_args:Nne \prop_gput:cnn{ \stex_morphisms_macro:N \l_stex_current_module_uri }{
3034     \tl_if_empty:nTF{#1}{[\stex_use_module_uri:n {#2}]}{#1}
3035   }{{#1}{#2}{#3}{#4}}
3036 }
3037 \cs_generate_variant:Nn \stex_add_morphism:nnnn {nonn,oooe}

```

(End of definition for `\stex_add_morphism:nnnn`. This function is documented on page 145.)

sfunction

31.3 Theory Morphisms

3038 `<@@=stex_morphisms>`

`\stex_structural_feature_morphism:nnnnn`

`\stex_structural_feature_morphism_with_macros:nnnnn`

`\stex_structural_feature_morphism_end:`

```

3039 \tl_new:N \l_stex_current_domain_uri
3040 \bool_new:N \l__stex_morphisms_with_macros_bool
3041
3042 \cs_new_protected:Npn \stex_structural_feature_morphism_with_macros:nnnnn {
3043   \bool_set_true:N \l__stex_morphisms_with_macros_bool
3044   \__stex_morphisms_morphism:nnnnn
3045 }
3046 \cs_new_protected:Npn \stex_structural_feature_morphism:nnnnn {
3047   \bool_set_false:N \l__stex_morphisms_with_macros_bool
3048   \__stex_morphisms_morphism:nnnnn
3049 }
3050
3051 \cs_new_protected:Nn \__stex_morphisms_morphism:nnnnn {
3052   \tl_clear:N \l_stex_current_domain_uri
3053   \stex_debug:nn{morphisms}{new~morphism:{#1}{#2}{#3}{#4}{#5}}
3054   \tl_if_empty:nTF{#3}{
3055     \stex_get_mathstructure:n{#4}
3056     \tl_set_eq:NN \l_stex_current_domain_uri \l_stex_get_structure_module_uri
3057   }
3058   \tl_if_empty:NT \l_stex_current_domain_uri {
3059     \stex_uri_from_pair:Nnn \l_stex_import_uri { #3 }{ #4 }
3060     \group_begin:
3061     \stex_require_module:N \l_stex_import_uri
3062     \exp_args:Nne \use:nn \group_end: {
3063       \tl_set:Nn \exp_not:N\l_stex_current_domain_uri {\l_stex_import_uri}
3064     }
3065   }
3066   \tl_if_empty:nTF{#1}{
3067     \msg_error:nn{stex}{error/morphism-needs-name}
3068   }
3069   \str_set:Nn \l_tmpa_str {#1}
3070
3071   \stex_debug:nn{morphisms}{#2:~\l_tmpa_str;~for~\stex_use_module_uri:N \l_stex_current_dom
3072

```



```

3073 \stex_suppress_html:n{
3074   \stex_add_symbol:nnnnnnnN
3075   {}
3076   {#1}
3077   {0}
3078   {}
3079   {DEFED}
3080   {}
3081   {}
3082   \stex_invoke_symbol:
3083 }
3084
3085 \str_set:Nn \l__stex_morphisms_feature_str {#2}
3086 \str_set_eq:NN \l_stex_feature_name_str \l_tmpa_str
3087
3088 \stex_if_do_html:TF {
3089   \begin{stex_annotate_env} {
3090     data-ftml-feature-#2={\l_tmpa_str},
3091     data-ftml-domain={\stex_use_module_uri:N \l_stex_current_domain_uri}
3092     #5
3093   }
3094   \stex_annotate_invisible:n{ }
3095 } \group_begin:
3096 \__stex_morphisms_setup:
3097 \__stex_morphisms_reactivate:
3098 \stex_metagroup_new:
3099 %\message{^^JHERE:\l_stex_current_module_str}
3100 }
3101
3102 \cs_new_protected:Nn \__stex_morphisms_setup: {
3103   \seq_clear:N \l_stex_morphism_symbols_seq
3104   \seq_clear:N \l_stex_morphism_renames_seq
3105   \seq_clear:N \l_stex_morphism_morphisms_seq
3106   \seq_clear:N \l_stex_morphism_assigns_seq
3107   \__stex_morphisms_do_decls:
3108   \seq_clear:N \l__stex_morphisms_do_dones_seq
3109   \exp_args:No \__stex_morphisms_do:n \l_stex_current_domain_uri
3110 }
3111
3112 \cs_new_protected:Nn \__stex_morphisms_do_decls: {
3113   \exp_args:No \stex_iterate_symbols:nn \l_stex_current_domain_uri {
3114     \exp_args:NNe \seq_put_right:Nn \l_stex_morphism_symbols_seq {
3115       { \stex_new_symbol_uri:nn{##1}{##3} }
3116       { {##2}{##4}{##5}{##6}{##7}{##8}{##9} }
3117     }
3118   }
3119 }
3120
3121 \cs_new_protected:Nn \__stex_morphisms_do:n {
3122   \prop_map_inline:cn {\_stex_morphisms_macro:n{#1}}{
3123     \seq_if_in:NnF \l__stex_morphisms_do_dones_seq {##2}{
3124       \seq_push:Nn \l__stex_morphisms_do_dones_seq {##2}
3125       \stex_debug:nn{morphisms}{Doing ~ \tl_to_str:n{##2}}
3126       \__stex_morphisms_do_morph:nnnn ##2

```

```

3127     }
3128   }
3129 }
3130
3131 \cs_new_protected:Nn \__stex_morphisms_do_morph:nnnn {
3132   \tl_if_empty:nF{#3}{
3133     \seq_put_right:Nn \l_stex_morphism_morphisms_seq {{#1}{#2}{#3}}
3134   }
3135   \__stex_morphisms_do:n{#2}
3136 }
3137
3138
3139 \cs_new_protected:Nn \stex_structural_feature_morphism_end: {
3140   \tl_gset_eq:NN \l_stex_current_domain_uri \l_stex_current_domain_uri
3141   \seq_gset_eq:NN \l_stex_morphism_symbols_seq \l_stex_morphism_symbols_seq
3142   \seq_gset_eq:NN \l_stex_morphism_renames_seq \l_stex_morphism_renames_seq
3143   \seq_gset_eq:NN \l_stex_morphism_assigns_seq \l_stex_morphism_assigns_seq
3144   \seq_gset_eq:NN \l_stex_morphism_morphisms_seq \l_stex_morphism_morphisms_seq
3145   \bool_gset_eq:NN \l__stex_morphisms_with_macros_bool \l_stex_morphisms_with_macros_bool
3146   \stex_if_do_html:TF{
3147     \end{stex_annotate_env}
3148   }\group_end:
3149   \__stex_morphisms_do_elaboration:
3150 }
3151
3152 \cs_new_protected:Nn \__stex_morphisms_do_elaboration: {
3153   \stex_debug:nn{morphisms}{
3154     Elaborating:^^J\meaning \l_stex_morphism_symbols_seq
3155     ^^J^^J
3156     Renamings:^^J
3157     \meaning \l_stex_morphism_renames_seq
3158     ^^J^^J
3159     Assignments:^^J
3160     \meaning \l_stex_morphism_assigns_seq
3161   }
3162
3163   \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3164     \__stex_morphisms_elab:nn ##1
3165   }
3166
3167   \stex_add_morphism:oooo
3168     \l_stex_feature_name_str
3169     \l_stex_current_domain_uri
3170     \l_stex_morphisms_feature_str
3171     {\seq_map_function:NN \l_stex_morphism_renames_seq \__stex_morphisms_rename:n}
3172 }
3173
3174 \cs_new:Nn \__stex_morphisms_rename:n{
3175   \__stex_morphisms_rename:nn #1
3176 }
3177
3178 \cs_new:Nn \__stex_morphisms_rename:nn{
3179   {#1}{\use_ii:nn#2}
3180 }

```

```

3181
3182 \cs_new_protected:Nn \__stex_morphisms_elab:nn {
3183   \tl_clear:N \l__stex_morphisms_tmp
3184   \seq_map_inline:Nn \l_stex_morphism_renames_seq {
3185     \__stex_morphisms_elab_check_rename:nnn{#1}##1
3186   }
3187   \tl_if_empty:NTF \l__stex_morphisms_tmp {
3188     \exp_args:Ne \__stex_morphisms_elab_i:nnn {\stex_symbol_uri_name:n {#1}}{#1}
3189   }{
3190     \stex_debug:nn{morphisms}{Generating~1~\l__stex_morphisms_tmp}
3191     \use_i:nn{ \exp_args:Nno \use:nn {\__stex_morphisms_add_symbol:nnnnnnnN {#1}} \l__stex
3192   }#2
3193
3194   \exp_args:No\stex_iterate_notations:nn\l_stex_current_domain_uri {
3195     \str_if_eq:nnT{#1}{##1}{
3196       \exp_args:Ne \stex_add_notation:nnnnn {
3197         \exp_args:Ne \stex_new_symbol_uri:n {\exp_after:wN \use_ii:nn \l__stex_morphisms_tm
3198       }{##2}{##3}{##4}{##5}
3199     }
3200   }
3201 }
3202
3203 \cs_new_protected:Nn \__stex_morphisms_elab_i:nnn {
3204   \tl_set:Ne \l__stex_morphisms_tmp {{{\l_stex_feature_name_str / #1}}}
3205   \stex_debug:nn{morphisms}{Generating~2~\l_stex_feature_name_str / #1}
3206   \bool_if:NTF \l__stex_morphisms_with_macros_bool {
3207     \exp_args:Nnno \__stex_morphisms_add_symbol:nnnnnnnnN {#2} {#3}
3208   }{
3209     \exp_args:Nnno \__stex_morphisms_add_symbol:nnnnnnnnN {#2} {}
3210   }
3211   {\l_stex_feature_name_str / #1}
3212 }
3213
3214 \cs_new_protected:Npn \__stex_morphisms_add_symbol:nnnnnnnnN #1 {
3215   \tl_clear:N \l__stex_morphisms_tmp_b
3216   \seq_map_inline:Nn \l_stex_morphism_assigns_seq {
3217     \__stex_morphisms_elab_check_assign:nnnnn{#1}##1
3218   }
3219   \tl_if_empty:NTF \l__stex_morphisms_tmp_b
3220   \stex_add_symbol:nnnnnnnN
3221   \__stex_morphisms_add_symbol_i:nnnnnnnnN
3222 }
3223
3224 \cs_new_protected:Npn \__stex_morphisms_add_symbol_i:nnnnnnnnN #1 #2 #3 #4 #5 {
3225   \stex_add_symbol:nnnnnnnnN {#1}{#2}{#3}{#4}{assigned}
3226 }
3227
3228 \cs_new_protected:Nn \__stex_morphisms_elab_check_assign:nnnnn {
3229   \tl_if_eq:nnT{#1}{{#2}{#3}{#4}{#5}}{
3230     \seq_map_break:n{
3231       \tl_set:Nn \l__stex_morphisms_tmp_b {assigned}
3232     }
3233   }
3234 }

```

```

3235
3236 \cs_new_protected:Nn \__stex_morphisms_elab_check_rename:nnn {
3237   \tl_if_eq:nnT{#1}{#2}{
3238     \seq_map_break:n{
3239       \tl_set:Nn \l__stex_morphisms_tmp {#3}
3240     }
3241   }
3242 }
3243
3244 \cs_new_protected:Nn \__stex_morphisms_do_for_list: {
3245   \seq_clear:N \l_stex_fors_seq
3246   \clist_map_inline:Nn \l_stex_key_for_clist {
3247     \exp_args:Ne\stex_get_in_morphism:n{\tl_to_str:n{##1}}
3248     \seq_put_right:No \l_stex_fors_seq
3249       {\l_stex_get_symbol_uri}
3250   }
3251 }
3252
3253
3254 \cs_new_protected:Nn \__stex_morphisms_definiens_impl:nn {
3255   \tl_if_empty:nF {#1} {
3256     \stex_get_in_morphism:n { #1 }
3257     \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
3258   }
3259   \tl_if_empty:NT \l_stex_current_def_uri {
3260     \msg_error:nn{stex}{error/definiensfor}
3261   }
3262   \stex_debug:nn{definiens}{Checking-\stex_use_symbol_uri:N \l_stex_current_def_uri }
3263   \stex_if_do_html:TF{
3264     \exp_after:wN \stex_metagroup_do_in:n \exp_after:wN {
3265       \exp_after:wN \seq_put_right:Nn \exp_after:wN \l_stex_morphism_assigns_seq
3266       \exp_after:wN { \l_stex_current_def_uri }
3267     }
3268     \mode_leave_vertical: \stex_annotate:nn{data-ftml-assign={\stex_use_symbol_uri:N \l_st
3269       \stex_annotate_force_break:n{
3270         \stex_annotate:nn{data-ftml-definiens={}}{#2}
3271       }
3272     }
3273   }{
3274     \exp_args:No \__stex_morphisms_add_definiens:nn \l_stex_current_def_uri {#2}
3275     \stex_if_smsmode:F{#2}
3276   }
3277 }
3278
3279 \cs_new_protected:Nn \__stex_morphisms_add_definiens:nn {
3280   \tl_set:Nn \l_stex_get_symbol_uri {#1}
3281   \stex_debug:nn{morphisms}{Adding~definiens~\tl_to_str:n{#1~#2}}
3282   %\__stex_morphisms_set_definiens_macros: #1\__stex_morphisms_break:
3283   \stex_assign_do:n{#2}
3284 }
3285
3286 %\cs_new_protected:Npn \__stex_morphisms_set_definiens_macros: #1?#2?#3\__stex_morphisms_br
3287 % \str_set:Nn \l_stex_get_symbol_uri { #1?#2} % TODO
3288 % \str_set:Nn \l_stex_get_symbol_name_str {#3}

```

```

3289 % \exp_args:Nne\use:nn{\_stex_morphisms_set_definiens_macros_i:nnnnnnn}{
3290 % \prop_item:Nn \l_stex_morphism_symbols_prop {[#1?#2]/[#3]}
3291 % }
3292 %}
3293
3294 %\cs_new_protected:Nn \_stex_morphisms_set_definiens_macros_i:nnnnnnn {
3295 % \tl_set:Nn \l_stex_get_symbol_def_tl{#4}
3296 %}
3297
3298
3299 \cs_new_protected:Nn \_stex_morphisms_rename_all: {
3300 % TODO
3301 }
3302
3303
3304 \cs_new_protected:Nn \stex_structural_feature_morphism_check_total: {
3305 \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3306 \_stex_morphisms_total_check:nn ##1
3307 }
3308 }
3309
3310 \cs_new_protected:Nn \_stex_morphisms_total_check:nn {
3311 \_stex_morphisms_total_check:nnnnnnN {#1} #2
3312 }
3313
3314 \cs_new_protected:Nn \_stex_morphisms_total_check:nnnnnnN {
3315 \tl_if_empty:nT{#5}{
3316 \seq_if_in:NnF \l_stex_morphism_assigns_seq {#1} {
3317 \msg_error:nnxx{stex}{error/needsdefiniens}{\stex_use_symbol_uri:n {#1}}{total-morphi
3318 }
3319 }
3320 }
3321
3322 \cs_new:Npn \_stex_morphisms_split_qm:w #1 ? #2 ? #3 { #3 }
3323
3324
3325
3326 \cs_new_protected:Npn \_stex_morphisms_reactivate: {
3327 \stex_deactivate_macro:Nn \symdecl {module~environments}
3328 \stex_deactivate_macro:Nn \textsymdecl {module~environments}
3329 \stex_deactivate_macro:Nn \symdef {module~environments}
3330 \stex_deactivate_macro:Nn \notation {module~environments}
3331 \stex_deactivate_macro:Nn \importmodule {module~environments}
3332 \stex_deactivate_macro:Nn \requiremodule {module~environments}
3333 \stex_deactivate_macro:Nn \smodule {outside-of~morphisms}
3334 \stex_reactivate_macro:N \assign
3335 \stex_reactivate_macro:N \assignMorphism
3336 \stex_reactivate_macro:N \renamedec1
3337 \cs_set_eq:NN \_stex_do_for_list: \_stex_morphisms_do_for_list:
3338 \cs_set_eq:NN \_stex_add_definiens:nn \_stex_morphisms_add_definiens:nn
3339 \cs_set_eq:NN \_stex_definiens_impl:nn \_stex_morphisms_definiens_impl:nn
3340 }

```

(End of definition for \stex_structural_feature_morphism:nnnnn, \stex_structural_feature_morphism_with_macros:nnnnn, and \stex_structural_feature_morphism_end:. These functions are documented

on page 145.)

sfunction

\stex_iterate_morphisms:nn

```

3341 \cs_new_protected:Nn \stex_iterate_morphisms:nn {
3342   \seq_clear:N \l__stex_morphisms_mods_seq
3343   \bool_set_true:N \l__stex_morphisms_continue_bool
3344   \cs_set:Npn \__stex_morphisms_cs:nnnn ##1 ##2 ##3 ##4 ##5 {
3345     #2
3346     \bool_if:NT \l__stex_morphisms_continue_bool {
3347       \str_if_eq:nnTF{##1}{[#2]}{
3348         \tl_put_right:Nn \l__stex_morphisms_todo_tl {
3349           \__stex_morphisms_iterate:nn{##5}{##2}
3350         }
3351       }{
3352         \tl_put_right:Nn \l__stex_morphisms_todo_tl {
3353           \__stex_morphisms_iterate:nn{##5 / ##1}{##2}
3354         }
3355       }
3356     }
3357   }
3358   \cs_set:Npn \stex_iterate_break:n ##1 {
3359     \bool_set_false:N \l__stex_morphisms_continue_bool
3360     \prop_map_break:n{##1}
3361   }
3362   \__stex_morphisms_iterate:nn{}{#1}
3363 }
3364 \cs_new_protected:Nn \__stex_morphisms_iterate:nn {
3365   \tl_clear:N \l__stex_morphisms_todo_tl
3366   \seq_if_in:NnF \l__stex_morphisms_mods_seq {#1 #2}{
3367     \seq_put_right:Nn \l__stex_morphisms_mods_seq {#1 #2}
3368     \prop_map_inline:cn{\__stex_morphisms_macro:n{#2}}{
3369       \__stex_morphisms_cs:nnnn ##2 {#1}
3370       % TODO
3371       % ##1: name or [mpath]
3372       % ##2 = {####1}{####2}{####3}{####4}
3373       % ####1 = name
3374       % ####2 = mpath
3375       % ####3 = type
3376       % ####4 = {origname}{newname}*
3377     }
3378     \bool_if:NT \l__stex_morphisms_continue_bool \l__stex_morphisms_todo_tl
3379   }
3380 }
```

(End of definition for \stex_iterate_morphisms:nn. This function is documented on page 145.)

sfunction

\stex_get_in_morphism:n

```

3381 \cs_new_protected:Nn \stex_get_in_morphism:n {
3382   \stex_debug:nn{morphisms}{Getting-in-morphism:~#1}
3383   \tl_clear:N \l_stex_get_symbol_uri
3384   \str_set:Nn \l__stex_morphisms_get_str {#1}
3385   \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
```

```

3386     \__stex_morphisms_get_check:nn ##1
3387 }
3388 \tl_if_empty:NT \l_stex_get_symbol_uri {
3389   \seq_map_inline:Nn \l_stex_morphism_renames_seq {
3390     \__stex_morphisms_renamed_check:nn##1
3391   }
3392   \tl_if_empty:NT \l_stex_get_symbol_uri {
3393     \msg_error:nnxx{stex}{error/unknownsymbolin}{#1}{
3394       morphism~\l_stex_feature_name_str
3395     }
3396   }
3397 }
3398 }
3399
3400
3401 \cs_new_protected:Nn \__stex_morphisms_get_check:nn {
3402   \__stex_morphisms_check_name:nnnn #1 #2
3403 }
3404
3405 \cs_new_protected:Nn \__stex_morphisms_check_name:nnnn {
3406   % TODO module name, path, archive, macroname ?
3407   \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str {#4} {
3408     \tl_set:Nn \l_stex_get_symbol_uri {{{#1}{#2}{#3}{#4}}}
3409     \__stex_morphisms_set:nnnnnnN
3410   }{
3411     \__stex_morphisms_macro:nnnnnnN{{{#1}{#2}{#3}{#4}}}
3412   }
3413 }
3414
3415 \cs_new_protected:Nn \__stex_morphisms_set:nnnnnnN {
3416   \seq_map_break:n{
3417     \str_set:Nn \l_stex_get_symbol_macro_str{#1}
3418     \int_set:Nn \l_stex_get_symbol_arity_int {#2}
3419     \tl_set:Nn \l_stex_get_symbol_args_tl {#3}
3420     \tl_set:Nn \l_stex_get_symbol_def_tl {#4}
3421     \tl_set:Nn \l_stex_get_symbol_type_tl {#5}
3422     \tl_set:Nn \l_stex_get_symbol_return_tl {#6}
3423     \tl_set:Nn \l_stex_get_symbol_invoke_cs {#7}
3424   }
3425 }
3426
3427 \cs_new_protected:Npn \__stex_morphisms_macro:nnnnnnN #1 #2 {
3428   \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str {#2}{
3429     \tl_set:Nn \l_stex_get_symbol_uri{#1}
3430     \__stex_morphisms_set:nnnnnnN
3431   }\__stex_morphisms_gobble:nnnnnnN
3432   {#2}
3433 }
3434
3435 \cs_new_protected:Nn \__stex_morphisms_gobble:nnnnnnN {}
3436
3437 \cs_new_protected:Nn \__stex_morphisms_renamed_check:nn {
3438   \stex_debug:nn{here}{\tl_to_str:n{{{#1}{#2}}}:~\l__stex_morphisms_get_str}
3439   \__stex_morphisms_renamed_check:nnnnnn #1 #2

```

```

3440 }
3441
3442 \cs_new_protected:Nn \__stex_morphisms_renamed_check:nnnnnn {
3443   \exp_args:No \str_if_eq:nnTF \l__stex_morphisms_get_str{#5}{
3444     \seq_map_break:n{
3445       \str_set:Nn \l__stex_morphisms_get_str {#4}
3446       \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3447         \__stex_morphisms_get_check:nn ##1
3448       }
3449     }
3450   }{
3451     \exp_args:No \str_if_eq:nnT \l__stex_morphisms_get_str{#6}{
3452       \seq_map_break:n{
3453         \str_set:Nn \l__stex_morphisms_get_str {#4}
3454         \seq_map_inline:Nn \l_stex_morphism_symbols_seq {
3455           \__stex_morphisms_get_check:nn ##1
3456         }
3457       }
3458     }
3459   }
3460 }

```

(End of definition for `\stex_get_in_morphism:n`. This function is documented on page 145.)

sfunction

copymodule (env.)

```

3461 \stex_new_stylable_env:nnnnnnn {copymodule}{m 0{} m}{
3462   \__stex_morphisms_begin_copy:Nnnn\stex_structural_feature_morphism:nnnnn {#1}{#2}{#3}
3463 }{
3464   \stex_if_smsmode:F {
3465     \stex_style_apply:
3466   }
3467   \stex_structural_feature_morphism_end:
3468 }{}{}{}
3469
3470 \stex_new_stylable_env:nnnnnnn {copymodule*}{m 0{} m}{
3471   \cs_set:Npn \stex_style_apply: {
3472     \stex_apply_patch_begin:n{copymodule}
3473   }
3474   \__stex_morphisms_begin_copy:Nnnn\stex_structural_feature_morphism_with_macros:nnnnn {#1}
3475 }{
3476   \cs_set:Npn \stex_style_apply: {
3477     \stex_apply_patch_end:n{copymodule}
3478   }
3479   \stex_if_smsmode:F {
3480     \stex_style_apply:
3481   }
3482   \stex_structural_feature_morphism_end:
3483 }{}{}{}
3484
3485 \cs_new_protected:Nn \__stex_morphisms_begin_copy:Nnnn {
3486   #1{#2}{morphism}{#3}{#4}{,data-ftml-total=false}
3487   \stex_if_smsmode:F {
3488     \tl_set:Nn \thiscopyname { #2 }

```



```

3489     \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3490     \stex_style_apply:
3491   }
3492   \stex_smsmode_do:
3493 }
3494
3495
3496 \stex_deactivate_macro:Nn \copymodule {module~environments}
3497 \stex_every_module:n {
3498   \stex_reactivate_macro:N \copymodule
3499 }
3500 \stex_sms_allow_env:n{copymodule}
3501
3502 \exp_after:wN \stex_deactivate_macro:Nn \csname copymodule*\endcsname {module~environments}
3503 \stex_every_module:n {
3504   \exp_after:wN\stex_reactivate_macro:N \csname copymodule*\endcsname
3505 }
3506 \stex_sms_allow_env:n{copymodule*}
3507
3508 \stex_new_stylable_cmd:nnnn{copymod}{s m O{} m m}{
3509   \IfBooleanTF#1\stex_structural_feature_morphism_with_macros:nnnnn
3510   \stex_structural_feature_morphism:nnnnn{#2}{morphism}{#3}{#4}{,data-ftml-total={false}}
3511   \clist_map_function:nN{#5}\__stex_morphisms_parse_assign:n
3512   \stex_if_smsmode:F {
3513     \tl_set:Nn \thiscopyname { #2 }
3514     \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3515     \stex_style_apply:
3516   }
3517   \stex_structural_feature_morphism_end:
3518   \stex_smsmode_do:
3519 }{}
3520
3521 \stex_deactivate_macro:Nn \copymod {module~environments}
3522 \stex_every_module:n {
3523   \stex_reactivate_macro:N \copymod
3524 }
3525 \stex_sms_allow_escape:N\copymod

```

interpretmodule (env.)

```

3526 \stex_new_stylable_env:nnnnnn {interpretmodule}{m O{} m}{
3527   \__stex_morphisms_begin_interpret:Nnn\stex_structural_feature_morphism:nnnnn {#1}{#2}{#3}
3528 }{}
3529   \stex_structural_feature_morphism_check_total:
3530   \stex_if_smsmode:F {
3531     \stex_style_apply:
3532   }
3533   \stex_structural_feature_morphism_end:
3534 }{}{}{}
3535
3536 \stex_new_stylable_env:nnnnnn {interpretmodule*}{m O{} m}{
3537   \cs_set:Npn \stex_style_apply: {
3538     \_stex_apply_patch_begin:n{interpretmodule}
3539   }
3540   \__stex_morphisms_begin_interpret:Nnn\stex_structural_feature_morphism_with_macros:nnnnn

```

```

3541 }{
3542   \cs_set:Npn \stex_style_apply: {
3543     \stex_apply_patch_end:n{interpretmodule}
3544   }
3545   \stex_structural_feature_morphism_check_total:
3546   \stex_if_smsmode:F {
3547     \stex_style_apply:
3548   }
3549   \stex_structural_feature_morphism_end:
3550 }{}{}{}
3551
3552 \cs_new_protected:Nn \__stex_morphisms_begin_interpret:Nnnn {
3553   #1{#2}{morphism}{#3}{#4}{,data-ftml-total=true}
3554   \stex_if_smsmode:F {
3555     \tl_set:Nn \thiscopyname { #2 }
3556     \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3557     \stex_style_apply:
3558   }
3559   \stex_smsmode_do:
3560 }
3561
3562 \stex_deactivate_macro:Nn \interpretmodule {module-environments}
3563 \stex_every_module:n {
3564   \stex_reactivate_macro:N \interpretmodule
3565 }
3566 \stex_sms_allow_env:n{interpretmodule}
3567
3568 \exp_after:wN \stex_deactivate_macro:Nn \csname interpretmodule*\endcsname {module-environm
3569 \stex_every_module:n {
3570   \exp_after:wN \stex_reactivate_macro:N \csname interpretmodule*\endcsname
3571 }
3572 \stex_sms_allow_env:n{interpretmodule*}
3573
3574 \stex_new_stylable_cmd:nnnn{interpretmod}{s m O{} m m}{
3575   \IfBooleanTF#1\stex_structural_feature_morphism_with_macros:nnnnn
3576   \stex_structural_feature_morphism:nnnnn{#2}{morphism}{#3}{#4}{,data-ftml-total=true}
3577   \clist_map_function:nN{#5}\__stex_morphisms_parse_assign:n
3578   \stex_if_smsmode:F {
3579     \tl_set:Nn \thiscopyname { #2 }
3580     \tl_set:Nn \thismoduleuri {\stex_use_module_uri:N \l_stex_current_domain_uri}
3581     \stex_style_apply:
3582   }
3583   \stex_structural_feature_morphism_check_total:
3584   \stex_structural_feature_morphism_end:
3585   \stex_smsmode_do:
3586 }{}
3587
3588 \stex_deactivate_macro:Nn \interpretmod {module-environments}
3589 \stex_every_module:n {
3590   \stex_reactivate_macro:N \interpretmod
3591 }
3592 \stex_sms_allow_escape:N\interpretmod

```

\assign

```

3593 \stex_new_stylable_cmd:nnnn {assign} { m m }{
3594   \stex_get_in_morphism:n{#1}
3595   \stex_if_do_html:TF{
3596     \stex_annotate_invisible:n{\hbox{$\_stex_assign_do:n{#2}$}}
3597   }{
3598     \_stex_assign_do:n{#2}
3599   }
3600   \stex_smsmode_do:
3601 }{}
3602 \stex_sms_allow_escape:N\assign
3603 \stex_deactivate_macro:Nn \assign {morphism~environments}
3604
3605 \cs_new_protected:Nn \_stex_assign_do:n{
3606   \stex_debug:nn{assign}{Assigning~\stex_use_symbol_uri:N \l_stex_get_symbol_uri~to~\tl_to_
3607   \stex_check_term:nn{assignment}{#1}
3608   \stex_if_do_html:T{
3609     \stex_annotate_invisible:nn{data-ftml-assign={\stex_use_symbol_uri:N \l_stex_get_symbol
3610     \_stex_annotate_force_break:n{
3611       \mode_if_math:TF{\stex_annotate:nn{data-ftml-definiens={}}{#1}}{\hbox{$\_stex_annota
3612     }
3613   }
3614 }
3615 \exp_after:wN \stex_metagroup_do_in:n \exp_after:wN {
3616   \exp_after:wN \seq_put_right:Nn \exp_after:wN \l_stex_morphism_assigns_seq
3617   \exp_after:wN { \l_stex_get_symbol_uri }
3618 }
3619 }
3620
3621 \cs_new_protected:Nn \__stex_morphisms_parse_assign:n {
3622   \str_clear:N \l__stex_morphisms_name_str
3623   \str_clear:N \l__stex_morphisms_newname_str
3624   \tl_clear:N \l__stex_morphisms_ass_tl
3625   \stex_debug:nn{morphisms}{Parsing~#1}
3626   \exp_args:NNe \seq_set_split:Nnn \l__stex_morphisms_seq {\tl_to_str:n{0}} {#1}
3627   \int_compare:nNnTF {\seq_count:N \l__stex_morphisms_seq} = 1 {
3628     \stex_debug:nn{morphisms}{No~@}
3629     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_next_tl
3630   }{
3631     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_name_str
3632     \stex_debug:nn{morphisms}{Name:~\l__stex_morphisms_name_str}
3633     \exp_args:NNo \str_set:Nn \l__stex_morphisms_name_str \l__stex_morphisms_name_str
3634     \tl_set:Ne \l__stex_morphisms_next_tl {\seq_use:Nn \l__stex_morphisms_seq @}
3635   }
3636   \exp_args:NNo \seq_set_split:Nnn \l__stex_morphisms_seq = \l__stex_morphisms_next_tl
3637   \str_if_empty:NTF \l__stex_morphisms_name_str {
3638     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_name_str
3639     \exp_args:NNo \str_set:Nn \l__stex_morphisms_name_str \l__stex_morphisms_name_str
3640     \tl_set:Ne \l__stex_morphisms_ass_tl {\seq_use:Nn \l__stex_morphisms_seq =}
3641   }{
3642     \seq_pop_left:NN \l__stex_morphisms_seq \l__stex_morphisms_newname_str
3643     \exp_args:NNo \str_set:Nn \l__stex_morphisms_newname_str \l__stex_morphisms_newname_str
3644     \tl_set:Ne \l__stex_morphisms_ass_tl {\seq_use:Nn \l__stex_morphisms_seq =}
3645   }
3646   \__stex_morphisms_do_parsed_assign:

```

```

3647 }
3648
3649 \cs_new_protected:Nn \__stex_morphisms_do_parsed_assign: {
3650   \exp_args:No \stex_get_in_morphism:n \l__stex_morphisms_name_str
3651   \str_if_empty:NF \l__stex_morphisms_newname_str {
3652     \stex_debug:nn{morphisms}{renaming~\l__stex_morphisms_name_str;~to~\l__stex_morphisms_n
3653     \exp_after:wN \__stex_morphisms_do_parsed_newname: \l__stex_morphisms_newname_str \__st
3654   }
3655   \tl_if_empty:NF \l__stex_morphisms_ass_tl {
3656     \stex_debug:nn{morphisms}{assigning~\l__stex_morphisms_name_str;~to~\exp_args:No \tl_to
3657     \exp_args:No \stex_assign_do:n \l__stex_morphisms_ass_tl
3658   }
3659 }
3660
3661 \cs_new_protected:Nn \__stex_morphisms_do_parsed_newname: {
3662   \peek_charcode:NTF [ {
3663     \__stex_morphisms_do_parsed_newname:w
3664   }{
3665     \__stex_morphisms_do_parsed_newname:w []
3666   }
3667 }
3668
3669 \cs_new_protected:Npn \__stex_morphisms_do_parsed_newname:w [#1] #2 \__stex_morphisms_end:
3670   \stex_renamedec1_do:nn{#1}{#2}
3671 }

```

(End of definition for \assign. This function is documented on page 146.)

sfunction

\renamedec1

```

3672 \stex_new_stylable_cmd:nnnn {renamedec1} { m 0{} m }{
3673   \stex_get_in_morphism:n{#1}
3674   \stex_renamedec1_do:nn{#2}{#3}
3675   \stex_smsmode_do:
3676 }{}
3677 \stex_sms_allow_escape:N\renamedec1
3678 \stex_deactivate_macro:Nn \renamedec1 {morphism~environments}
3679
3680 \cs_new_protected:Nn \stex_renamedec1_do:nn {
3681   \stex_debug:nn{renamedec1}{Renaming~\stex_use_symbol_uri:N \l_stex_get_symbol_uri~to~[#1]
3682   \stex_if_do_html:T{
3683     \exp_args:Ne \stex_annotate_invisible:nn{
3684       data-ftml-rename={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
3685       data-ftml-macroname={#2}
3686       \str_if_empty:nF{#1}{ ,data-ftml-to={#1} }
3687     }{}
3688   }
3689   \stex_metagroup_do_in:e {
3690     \seq_push:Nn \exp_not:N \l_stex_morphism_renames_seq
3691     {{\l_stex_get_symbol_uri}{#2}{
3692       \tl_if_empty:nTF{#1}{
3693         \l_stex_feature_name_str/\stex_symbol_uri_name:N \l_stex_get_symbol_uri
3694       }{#1}
3695     }}}

```

```

3696 }
3697 }

```

(End of definition for \renamedec1. This function is documented on page 146.)

```

sfunction

```

```

\assignMorphism

```

```

3698 \stex_new_stylable_cmd:nnnn {assignMorphism} { m m }{
3699   \str_clear:N \l__stex_morphisms_morphism_dom_str
3700   \exp_args:No \stex_iterate_morphisms:nn\l_stex_current_domain_uri{
3701     \stex_debug:nn{assignMorphism}{
3702       Checking:~#1~vs:^^J##1^^J##2^^J##3^^J##4
3703     }
3704     \str_if_eq:nnTF{#1}{##1}{
3705       \__stex_morphisms_do_morph_assign:nnn{##1}{##2}{##4}
3706     }{
3707       \stex_str_if_ends_with:nnT{##2}{#1}{
3708         \__stex_morphisms_do_morph_assign:nnn{##1}{##2}{##4}
3709       }
3710     }
3711   }
3712   \str_if_empty:NT \l__stex_morphisms_morphism_dom_str {
3713     \msg_error:nnn{stex}{error/nomorphism}{#1}
3714   }
3715   \bool_set_false:N \l_tmpa_bool
3716   \stex_iterate_morphisms:nn \l_stex_current_module_str {
3717     \stex_debug:nn{assignMorphism}{
3718       Checking:~#2~vs:^^J##1^^J##2^^J##3^^J##4
3719     }
3720     \str_if_eq:nnTF{#2}{##1}{
3721       \stex_debug:nn{assignMorphism}{match!}
3722       \stex_iterate_break:n{
3723         \stex_annotate_invisible:nn{
3724           data-ftml-assignmorphismfrom={\l__stex_morphisms_morphism_dom_str}
3725           data-ftml-assignmorphismto={\l_stex_current_module_str?##1}
3726         }{}
3727         \bool_set_true:N \l_tmpa_bool
3728       }
3729     }{
3730       \stex_str_if_ends_with:nnT{##2}{#2}{
3731         \stex_debug:nn{assignMorphism}{match!}
3732         \stex_iterate_break:n{
3733           \stex_annotate_invisible:nn{
3734             data-ftml-assignmorphismfrom={\l__stex_morphisms_morphism_dom_str},
3735             data-ftml-assignmorphismto={\l_stex_current_module_str?##1}
3736           }{}
3737           \bool_set_true:N \l_tmpa_bool
3738         }
3739       }
3740     }
3741   }
3742   \bool_if:NF \l_tmpa_bool {
3743     \msg_error:nnn{stex}{error/nomorphism}{#2}
3744   }

```

```

3745 }{}
3746 \stex_sms_allow_escape:N \assignMorphism
3747 \stex_deactivate_macro:Nn \assignMorphism {morphism-environments}
3748
3749 \cs_new_protected:Nn \__stex_morphisms_do_morph_assign:nnn {
3750   \stex_iterate_break:n{
3751     \str_set:Ne \l__stex_morphisms_morphism_dom_str { \l_stex_current_domain_str ? #1 }
3752     \stex_debug:nn{assignMorphism}{match!}
3753     \stex_iterate_symbols:nn{#2}{
3754       \stex_debug:nn{assignMorphism}{removing-##1?##3}
3755       % TODO: non-trivial assignments
3756       \prop_remove:Nn \l_stex_morphism_symbols_prop {
3757         [##1]/[##3]
3758       }
3759     }
3760   }
3761 }

```

(End of definition for \assignMorphism. This function is documented on page 146.)

sfunction

Chapter 32

Symbols

3762 <@@=stex_syms>

32.1 Declarations

Keys used in various symbol declarations:

```
3763 \stex_keys_define:nnnn{symargs}{
3764   \str_clear:N \l_stex_key_args_str
3765   \str_clear:N \l_stex_key_role_str
3766   \str_clear:N \l_stex_key_reorder_str
3767   \str_clear:N \l_stex_key_assoc_str
3768 }{
3769   args      .str_set:N = \l_stex_key_args_str ,
3770   reorder   .str_set:N = \l_stex_key_reorder_str ,
3771   assoc     .choices:nn = {bin,binl,binr,pre,conj,pwconj}
3772             {\str_set:Ne \l_stex_key_assoc_str \l_keys_choice_tl},
3773   role      .str_set:N = \l_stex_key_role_str
3774 }{}
3775
3776 \stex_keys_define:nnnn{decl}{
3777   \str_clear:N \l_stex_key_name_str
3778   \str_clear:N \l_stex_key_args_str
3779   \tl_clear:N \l_stex_key_type_tl
3780   \tl_clear:N \l_stex_key_def_tl
3781   \tl_clear:N \l_stex_key_return_tl
3782   \str_clear:N \l_stex_key_wikidata_str
3783   \clist_clear:N \l_stex_key_argtypes_clist
3784 }{
3785   name      .str_set:N = \l_stex_key_name_str ,
3786   return    .tl_set:N = \l_stex_key_return_tl ,
3787   argtypes  .clist_set:N = \l_stex_key_argtypes_clist ,
3788   wikidata  .str_set:N = \l_stex_key_wikidata_str ,
3789   type      .tl_set:N = \l_stex_key_type_tl ,
3790   def       .tl_set:N = \l_stex_key_def_tl ,
3791   align     .code:n = {},
3792   gf        .code:n = {}
3793 }{style,deprecate,symargs}
```

\symdecl

```

3794 \str_new:N \l_stex_macroname_str
3795
3796 \stex_new_stylable_cmd:nnnn {symdecl} { s m 0{}} {
3797   \stex_keys_set:nn{decl}{#3}
3798   \IfBooleanTF #1 {
3799     \str_clear:N \l_stex_macroname_str
3800   }{
3801     \str_set:Ne \l_stex_macroname_str { #2 }
3802   }
3803   \__stex_syms_top:n{#2}
3804   \stex_if_smsmode:F \__stex_syms_style:
3805   \stex_smsmode_do:
3806 }{}
3807
3808 \stex_deactivate_macro:Nn \symdecl {module~environments}
3809 \stex_every_module:n {\stex_reactivate_macro:N \symdecl}
3810 \stex_sms_allow_escape:N \symdecl
3811
3812 \cs_new_protected:Nn \__stex_syms_style: {
3813   \group_begin:
3814     \tl_set:Ne \thisdecluri {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3815     \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
3816     \tl_set_eq:NN \thistype \l_stex_key_type_tl
3817     \tl_set_eq:NN \thisdefiniens \l_stex_key_def_tl
3818     \tl_set_eq:NN \thisargs \l_stex_key_args_str
3819     \tl_clear:N \thisstyle
3820     \stex_style_apply:
3821   \group_end:
3822 }

```

(End of definition for \symdecl. This function is documented on page 147.)

sfunction

\symdef

```

3823 \stex_new_stylable_cmd:nnnn {symdef} { m 0{ } m } {
3824   \stex_keys_set:nn{symdef}{#2}
3825   \str_set:Ne \l_stex_macroname_str { #1 }
3826   \__stex_syms_top:n{#1}
3827   \stex_debug:nn{symdef}{Doing~\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3828   \str_set_eq:NN \l_stex_get_symbol_uri \l_stex_current_symbol_uri
3829   \stex_notation_parse:n{#3}
3830   \stex_if_check_terms:T{ \_stex_notation_check: }
3831   \_stex_notation_add:
3832   \stex_if_do_html:T{
3833     \_stex_notation_do_html:n{\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3834   }
3835   \stex_if_smsmode:F \__stex_syms_def_style:
3836   \stex_smsmode_do:
3837 }{}
3838
3839 \stex_deactivate_macro:Nn \symdef {module~environments}
3840 \stex_every_module:n {\stex_reactivate_macro:N \symdef}
3841 \stex_sms_allow_escape:N \symdef
3842

```



```

3843 \cs_new_protected:Nn \__stex_syms_def_style: {
3844   \group_begin:
3845   \tl_set:Nx \thisdecluri {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3846   \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
3847   \tl_set_eq:NN \thistype \l_stex_key_type_tl
3848   \tl_set_eq:NN \thisdefiniens \l_stex_key_def_tl
3849   \tl_set_eq:NN \thisargs \l_stex_key_args_str
3850   \tl_clear:N \thisstyle
3851   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
3852   \def\thisnotation{
3853     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{}}{
3854     \stex_notation_make_args:
3855   }
3856 }
3857 \stex_style_apply:
3858 \group_end:
3859 }

```

(the symdef keyset is defined after notations as

`\stex_keys_define:nnnn{symdef}{-}{-}{decl,notation}`)

(End of definition for `\symdef`. This function is documented on page 147.)

sffunction

`\textsymdecl`

```

3860 \stex_keys_define:nnnn{textsymdecl}{
3861   \str_clear:N \l_stex_key_name_str
3862   \tl_clear:N \l_stex_key_type_tl
3863   \tl_clear:N \l_stex_key_def_tl
3864 }{
3865   name      .str_set:N = \l_stex_key_name_str ,
3866   type      .tl_set:N  = \l_stex_key_type_tl ,
3867   def       .tl_set:N  = \l_stex_key_def_tl,
3868   gf        .code:n    = {}
3869 }{style,deprecate}
3870
3871 \stex_new_stylable_cmd:nnnn {textsymdecl} {m 0{ } m} {
3872   \stex_keys_set:nn{symdef}{-}
3873   \stex_keys_set:nn{textsymdecl}{#2}
3874   \str_set:Ne \l_stex_macroname_str { #1 }
3875   \str_if_empty:NT \l_stex_key_name_str {
3876     \str_set:Nn \l_stex_key_name_str {#1}
3877   }%{
3878   % \str_set:Ne \l_stex_key_name_str {\l_stex_key_name_str-sym}
3879   %}
3880   \str_set:Nn \l_stex_key_role_str {textsymdecl}
3881   \tl_set:Ne \l_stex_current_symbol_uri {
3882     \exp_args:No \stex_new_symbol_uri:n \l_stex_key_name_str
3883   }
3884   \stex_symdecl_do:
3885   \stex_check_terms:
3886   \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnN}{
3887     {\l_stex_macroname_str}
3888     {\l_stex_key_name_str}
3889     {0}{-}

```

```

3890     {\tl_if_empty:NF \l_stex_key_def_tl{DEFED} }
3891     {}% type
3892     {\use:c{#1name_nospace}}}% return
3893     \stex_invoke_text_symbol:
3894 }
3895 \exp_args:Ne \stex_ref_new_symbol:n
3896 { \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3897 \stex_if_do_html:T {
3898     \stex_symdecl_html:
3899 }
3900
3901 \int_set:Nn \l_stex_get_symbol_arity_int 0
3902 \tl_clear:N \l_stex_key_op_tl
3903 \str_clear:N \l_stex_key_intent_str
3904 \str_clear:N \l_stex_key_prec_str
3905 \tl_set_eq:NN \l_stex_get_symbol_uri \l_stex_current_symbol_uri
3906 \stex_notation_parse:n{\hbox{#3}}
3907 \stex_notation_add:
3908 \stex_if_do_html:T {
3909     \def\comp{\_comp}
3910     \stex_notation_do_html:n{ \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3911 }
3912 \stex_execute_in_module:e{
3913     \__stex_syms_set_textsymdecl_macro:nnn{#1}{ \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3914     \exp_not:n{#3}}
3915 }
3916
3917 \stex_if_smsmode:F{
3918     \group_begin:
3919     \tl_set:Ne \thisdecluri { \stex_use_symbol_uri:N \l_stex_current_symbol_uri }
3920     \tl_set_eq:NN \thisdeclname \l_stex_key_name_str
3921     \tl_clear:N \thisstyle
3922     \stex_style_apply:
3923     \group_end:
3924 }
3925 \stex_smsmode_do:
3926 }{}
3927
3928 \stex_deactivate_macro:Nn \textsymdecl {module~environments}
3929 \stex_every_module:n { \stex_reactivate_macro:N \textsymdecl }
3930 \stex_sms_allow_escape:N \textsymdecl
3931
3932 \cs_new_protected:Nn \__stex_syms_set_textsymdecl_macro:nnn {
3933     \cs_set_protected:cpn{#1name_nospace}{#3}
3934     \cs_set_protected:cpn{#1name}{
3935         \mode_if_vertical:T{\hbox_unpack:N\c_empty_box}
3936         \mode_if_math:T{\hbox{\let\xspace\relax #3}
3937         \mode_if_math:F{\cs_if_exist:NT\xspace\xspace}
3938     }
3939 }

```

(End of definition for \textsymdecl. This function is documented on page 147.)

sfunction

__stex_syms_top:n shared behaviour for \symdecl and \symdef; calls \stex_symdecl_do:, optionally checks

the term components, adds the symbol to the current module and produces the FTML for the declaration.

```

3940 \cs_new_protected:Nn \__stex_syms_top:n {
3941   \str_if_empty:NT \l_stex_key_name_str {
3942     \str_set:Ne \l_stex_key_name_str { #1 }
3943   }
3944   \tl_set:Ne \l_stex_current_symbol_uri {
3945     \exp_args:No \stex_new_symbol_uri:n \l_stex_key_name_str
3946   }
3947
3948   \stex_symdecl_do:
3949   \_stex_check_terms:
3950   \__stex_syms_add_decl:
3951   \stex_if_do_html:T \_stex_symdecl_html:
3952 }

```

(End of definition for __stex_syms_top:n.)

Adds the symbol to the current module:

```

3953 \cs_new_protected:Nn \__stex_syms_add_decl: {
3954   \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnN}{
3955     {\l_stex_macroname_str}
3956     {\l_stex_key_name_str}
3957     {\int_use:N \l_stex_get_symbol_arity_int}
3958     {\l_stex_get_symbol_args_tl}
3959     {\tl_if_empty:NF \l_stex_key_def_tl{DEFED} }
3960     {}
3961     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
3962     \stex_invoke_symbol:
3963   }
3964   \exp_args:Ne \stex_ref_new_symbol:n
3965     {\stex_use_symbol_uri:N \l_stex_current_symbol_uri}
3966 }

```

\stex_symdecl_do:

```

3967 \cs_new_protected:Nn \stex_symdecl_do: {
3968   \_stex_do_deprecation:n \l_stex_key_name_str
3969   \__stex_syms_parse_arity:
3970   \__stex_syms_do_args:
3971 }
3972
3973 \cs_new:Nn \_stex_return_args:nn {
3974   {\svar{ARGUMENT_#1}\_stex_eat_exclamation_point:}
3975 }
3976
3977 \cs_new_protected:Nn \__stex_syms_do_args: {
3978   \tl_clear:N \l_stex_get_symbol_args_tl
3979   \int_step_inline:nn \l_stex_get_symbol_arity_int {
3980     \tl_put_right:Nn \l_stex_get_symbol_args_tl {##1}
3981     \tl_put_right:Ne \l_stex_get_symbol_args_tl {
3982       \str_item:Nn \l_stex_key_args_str {##1}
3983     }
3984   }
3985 }

```

Constructs the arity string of the symbol:

```

3986 \int_new:N \l_stex_assoc_args_count
3987
3988 \cs_new_protected:Nn \__stex_syms_parse_arity: {
3989   \int_zero:N \l_stex_get_symbol_arity_int
3990   \int_zero:N \l_stex_assoc_args_count
3991   \str_map_inline:Nn \l_stex_key_args_str {
3992     \str_case:nnF ##1 {
3993       0 { \str_map_break: }
3994       1 { \str_map_break:n{
3995         \int_set:Nn \l_stex_get_symbol_arity_int {1}
3996         \str_set:Nn \l_stex_key_args_str {i}
3997       } }
3998       2 { \str_map_break:n{
3999         \int_set:Nn \l_stex_get_symbol_arity_int {2}
4000         \str_set:Nn \l_stex_key_args_str {ii}
4001       } }
4002       3 { \str_map_break:n{
4003         \int_set:Nn \l_stex_get_symbol_arity_int {3}
4004         \str_set:Nn \l_stex_key_args_str {iii}
4005       } }
4006       4 { \str_map_break:n{
4007         \int_set:Nn \l_stex_get_symbol_arity_int {4}
4008         \str_set:Nn \l_stex_key_args_str {iiii}
4009       } }
4010       5 { \str_map_break:n{
4011         \int_set:Nn \l_stex_get_symbol_arity_int {5}
4012         \str_set:Nn \l_stex_key_args_str {iiiii}
4013       } }
4014       6 { \str_map_break:n{
4015         \int_set:Nn \l_stex_get_symbol_arity_int {6}
4016         \str_set:Nn \l_stex_key_args_str {iiiiii}
4017       } }
4018       7 { \str_map_break:n{
4019         \int_set:Nn \l_stex_get_symbol_arity_int {7}
4020         \str_set:Nn \l_stex_key_args_str {iiiiiii}
4021       } }
4022       8 { \str_map_break:n{
4023         \int_set:Nn \l_stex_get_symbol_arity_int {8}
4024         \str_set:Nn \l_stex_key_args_str {iiiiiiii}
4025       } }
4026       9 { \str_map_break:n{
4027         \int_set:Nn \l_stex_get_symbol_arity_int {9}
4028         \str_set:Nn \l_stex_key_args_str {iiiiiii}
4029       } }
4030       i {\int_incr:N \l_stex_get_symbol_arity_int}
4031       b {\int_incr:N \l_stex_get_symbol_arity_int}
4032       a {\int_incr:N \l_stex_get_symbol_arity_int \int_incr:N \l_stex_assoc_args_count}
4033       B {\int_incr:N \l_stex_get_symbol_arity_int \int_incr:N \l_stex_assoc_args_count}
4034     }{
4035       \msg_error:nnxx{stex}{error/wrongargs}{-}{##1}
4036     }
4037   }
4038 }
```

(End of definition for `\stex_symdecl_do`:. This function is documented on page 147.)

sfunction

`\_stex_symdecl_html`:

```

4039 \cs_new_protected:Nn \_stex_symdecl_html: {
4040   \stex_annotate_invisible:n{
4041     \hbox{\exp_args:Ne \stex_annotate:nn {
4042       data-ftml-symdecl = { \l_stex_key_name_str},
4043       data-ftml-args = {\l_stex_key_args_str}
4044       \str_if_empty:NF \l_stex_macroname_str {,
4045         data-ftml-macroname={\l_stex_macroname_str}
4046       }
4047       \str_if_empty:NF \l_stex_key_wikidata_str {,
4048         data-ftml-wikidata={\l_stex_key_wikidata_str}
4049       }
4050       \str_if_empty:NF \l_stex_key_assoc_str {,
4051         data-ftml-assoctype={\l_stex_key_assoc_str}
4052       }
4053       \str_if_empty:NF \l_stex_key_reorder_str {,
4054         data-ftml-reorderargs={\l_stex_key_reorder_str}
4055       }
4056       \str_if_empty:NF \l_stex_key_role_str {,
4057         data-ftml-role={\l_stex_key_role_str}
4058       }
4059     }\stex_annotate_force_break:n{
4060       \bool_set_true:N \stex_in_invisible_html_bool
4061       \tl_if_empty:NF \l_stex_key_type_tl {
4062         $\stex_annotate:nn{data-ftml-type={}}{\l_stex_key_type_tl}$
4063       }
4064       \tl_if_empty:NF \l_stex_key_def_tl {
4065         $\stex_annotate:nn{data-ftml-definiens={}}{\l_stex_key_def_tl}$
4066       }
4067       \tl_if_empty:NF \l_stex_key_return_tl{
4068         \exp_args:Nno \use:n{
4069           \cs_generate_from_arg_count:NNnn \l__stex_syms_cs
4070           \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
4071           \tl_set:Ne \l__stex_syms_args_tl {\_stex_map_args:N \_stex_return_args:nn}
4072           $\stex_annotate:nn{data-ftml-returntype={}}{
4073             \exp_after:wN \l__stex_syms_cs \l__stex_syms_args_tl!
4074           }$
4075         }
4076       \clist_if_empty:NF \l_stex_key_argtypes_clist {
4077         \stex_annotate:nn{data-ftml-argtypes={}}{\_stex_annotate_force_break:n{
4078           \clist_map_inline:Nn \l_stex_key_argtypes_clist {
4079             $\stex_annotate:nn{data-ftml-type={}}{##1}$
4080           }
4081         }}
4082       }
4083     }}
4084   }}
4085 }
```

(End of definition for `\_stex_symdecl_html`:. This function is documented on page 147.)

sfunction

\stex_add_symbol:nnnnnnN

```

4086 \cs_new_protected:Nn \stex_add_symbol:nnnnnnN {
4087   \stex_debug:nn{declaration}{New~declaration:~\stex_use_module_uri:N \l_stex_current_modul
4088     Macro:#1^^JArity:#3~(#4)^^J
4089     Def:~\tl_to_str:n{#5}^^J
4090     Type:~\tl_to_str:n{#6}^^J
4091     Returns:~\tl_to_str:n{#7}^^J
4092     Invokes:~\tl_to_str:n{#8}
4093   }
4094   \prop_gput:cn{ \_stex_symbol_macro:N \l_stex_current_module_uri }
4095   {#2}{#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}}
4096   \tl_if_empty:nF{#1}{
4097     \stex_do_up_to_module:n {
4098       \__stex_syms_activate_i:nnnnnnnn{#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}
4099     }
4100   }
4101 }

```

Generate semantic macros etc.:

```

4102 \cs_new_protected:Nn \_stex_activate_symbols: {
4103   \stex_debug:nn{activating}{All~symbols:~\cs_meaning:c{ \_stex_symbol_macro:N \l_stex_curr
4104   \prop_map_inline:cn{ \_stex_symbol_macro:N \l_stex_current_module_uri }{
4105     \__stex_syms_maybe_activate:nnnnnnnn ##2
4106   }
4107 }
4108
4109 \cs_new_protected:Nn \__stex_syms_maybe_activate:nnnnnnnn {
4110   \str_if_empty:nF{#1}{
4111     \__stex_syms_activate_i:nnnnnnnn {#1}{#2}{#3}{#4}{#5}{#6}{#7}{#8}
4112   }
4113 }
4114
4115 \cs_new_protected:Nn \__stex_syms_activate_i:nnnnnnnn {
4116   \stex_debug:nn{activating}{macro~#1:\stex_use_module_uri:N \l_stex_current_module_uri ? #
4117   \tl_to_str:n{#3}{#4}{#5}{#6}{#7}{#8}
4118 }
4119   \cs_set:cpe{#1} {
4120     \_stex_invoke_symbol:nnnnnnN
4121     {\exp_args:No \stex_new_symbol_uri:nn {\l_stex_current_module_uri} {#2} }
4122     \exp_not:n{#3}{#4}{#5}{#6}{#7}{#8}}
4123 }
4124 }

```

(End of definition for \stex_add_symbol:nnnnnnN. This function is documented on page 147.)

sfunction

\stex_has_definiens_p:N

\stex_has_definiens:N $\overline{TF}$

\stex_has_definiens_p:n

\stex_has_definiens:n $\overline{TF}$

```

4125 \prg_new_conditional:Nnn \stex_has_definiens:N { p, T, F, TF } {
4126   \exp_args:No \stex_has_definiens:nTF #1 \prg_return_true: \prg_return_false:
4127 }
4128 \prg_new_conditional:Nnn \stex_has_definiens:n { p, T, F, TF } {
4129   \exp_args:Nne \use:nn{\__stex_syms_has_definiens:nnnnnnnn}{\exp_args:Nne \prop_item:cn{
4130     \exp_args:Ne \_stex_symbol_macro:n {\stex_symbol_uri_module:n{#1}}
4131   }{

```

```

4132     \stex_symbol_uri_name:n {#1}
4133   }}
4134 }
4135
4136 \cs_new:Nn \__stex_syms_has_definiens:nnnnnnnN {
4137   \tl_if_empty:nTF{#5}\prg_return_false:\prg_return_true:
4138 }

```

(End of definition for `\stex_has_definiens:NTF` and `\stex_has_definiens:nTF`. These functions are documented on page 147.)

sfunction

32.2 Retrieval

```

\stex_iterate_symbols:n
  \stex_iterate_break:
  \stex_iterate_break:n
4139 \cs_new_protected:Nn \stex_iterate_symbols:n {
4140   \stex_pseudogroup_with:nn{\__stex_syms_sym_cs:nnnnnnnnN\stex_iterate_break:\stex_iterate_
4141     \cs_set:Npn \__stex_syms_sym_cs:nnnnnnnnN
4142     ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #1 }
4143     \cs_set:Npn \stex_iterate_break: {
4144       \prop_map_break:n{\seq_map_break:}
4145     }
4146     \cs_set:Npn \stex_iterate_break:n ##1 {
4147       \prop_map_break:n{\seq_map_break:n{##1}}
4148     }
4149     \seq_map_inline:Nn \l_stex_all_modules_seq {
4150       \prop_map_inline:cn{\_stex_symbol_macro:n {##1}}{
4151         \__stex_syms_sym_cs:nnnnnnnnN {##1} #####2
4152       }
4153     }
4154   }
4155 }

```

(End of definition for `\stex_iterate_symbols:n`, `\stex_iterate_break:`, and `\stex_iterate_break:n`. These functions are documented on page 148.)

sfunction

```

\stex_iterate_symbols:nn
4156 \cs_new_protected:Nn \stex_iterate_symbols:nn {
4157   \seq_clear:N \l__stex_syms_mods_seq
4158   \stex_pseudogroup_with:nn{\__stex_syms_sym_cs:nnnnnnnnN}{
4159     \cs_set:Npn \__stex_syms_sym_cs:nnnnnnnnN
4160     ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #2 }
4161     \clist_map_function:nN {#1} \__stex_syms_it_decl_i:n
4162   }
4163 }
4164
4165 \cs_new_protected:Nn \__stex_syms_it_decl_i:n {
4166   \seq_if_in:NnF \l__stex_syms_mods_seq {#1} {
4167     \seq_put_left:Nn \l__stex_syms_mods_seq {#1}
4168     \prop_map_inline:cn{\_stex_morphisms_macro:n {#1}}{
4169       \__stex_syms_it_decl_check:nnnn ##2
4170     }

```

```

4171 \prop_map_inline:cn{\_stex_symbol_macro:n {#1} }{
4172 \__stex_syms_sym_cs:nnnnnnnnN {#1} ##2
4173 }
4174 }
4175 }
4176
4177 \cs_new_protected:Nn \__stex_syms_it_decl_check:nnnn {
4178 \tl_if_empty:nT{#1}{
4179 \__stex_syms_it_decl_i:n {#2}
4180 }
4181 }

```

(End of definition for \stex_iterate_symbols:nn. This function is documented on page 148.)

sfunction

```

\stex_get_symbol:n
\stex_get_symbol:nTF
\stex_get_symbol:nF
4182 \int_new:N \l_stex_get_symbol_arity_int
4183
4184 \cs_new_protected:Nn \stex_get_symbol:n {
4185 \stex_get_symbol:nF{ #1 }{
4186 \msg_error:nnn{stex}{error/unknownsymbol}{#1}
4187 }
4188 }
4189
4190 \cs_new_protected:Npn \stex_get_symbol:nTF #1 #2 #3 {
4191 \tl_clear:N \l_stex_get_symbol_uri
4192 \cs_if_exist:cTF { #1 }{
4193 \cs_set_eq:Nc \l__stex_syms_cs { #1 }
4194 % command name
4195 \exp_args:Ne \tl_if_empty:nTF { \cs_argument_spec:N \l__stex_syms_cs }{
4196 % ...that takes no arguments
4197 \exp_args:Ne \cs_if_eq:NNTF {\tl_head:N \l__stex_syms_cs}
4198 \stex_invoke_symbol:nnnnnnN
4199 \__stex_syms_get_symbol_from_cs:
4200 {\__stex_syms_get_symbol_from_string:n { #1 }}
4201 }{
4202 \__stex_syms_get_symbol_from_string:n { #1 }
4203 }
4204 }{
4205 \__stex_syms_get_symbol_from_string:n { #1 }
4206 }
4207 \tl_if_empty:NTF \l_stex_get_symbol_uri { #3 } { #2 }
4208 }
4209
4210 \cs_new_protected:Npn \stex_get_symbol:nF #1 #2 {
4211 \stex_get_symbol:nTF{ #1 }{}{ #2 }
4212 }
4213
4214 \cs_new_protected:Nn \__stex_syms_get_symbol_from_cs: {
4215 \stex_pseudogroup_with:nn{\stex_invoke_symbol:nnnnnnN}{
4216 \cs_set:Npn \stex_invoke_symbol:nnnnnnN ##1 ##2 ##3 ##4 ##5 ##6 ##7 {
4217 \tl_set:Nn \l_stex_get_symbol_uri { ##1 }
4218 \int_set:Nn \l_stex_get_symbol_arity_int {##2}
4219 \tl_set:Nn \l_stex_get_symbol_args_tl {##3}

```



```

4220     \tl_set:Nn \l_stex_get_symbol_def_tl {##4}
4221     \tl_set:Nn \l_stex_get_symbol_type_tl {##5}
4222     \tl_set:Nn \l_stex_get_symbol_return_tl {##6}
4223     \tl_set:Nn \l_stex_get_symbol_invoke_cs {##7}
4224   }
4225   \l__stex_syms_cs
4226 }
4227 }
4228
4229 \cs_new_protected:Nn \__stex_syms_get_symbol_from_string:n {
4230   \stex_debug:nn{symbols}{Getting~from~string~#1...}
4231   \seq_set_split:Nnn \l__stex_syms_seq ? {#1}
4232   \seq_pop_right:NN \l__stex_syms_seq \l__stex_syms_name
4233   \seq_if_empty:NTF \l__stex_syms_seq {
4234     \exp_args:No \__stex_syms_get_from_one_string:n {#1}
4235   }{
4236     \exp_args:NNe \exp_args:Nno \__stex_syms_get_symbol_from_modules:nn {
4237       \seq_use:Nn \l__stex_syms_seq ?
4238     } \l__stex_syms_name
4239   }
4240 }
4241
4242 \cs_new_protected:Nn \__stex_syms_sym_from_str_i:nnnn {
4243   \bool_lazy_any:nTF{
4244     {\str_if_eq_p:nn{#2}{#3}}
4245     {\str_if_eq_p:nn{#2}{#4}}
4246     {\stex_str_if_ends_with_p:nn{#4}{/#2}}
4247   }{
4248     \__stex_syms_sym_i_finish:nnnnnnN{#1}{#4}
4249   }{
4250     \__stex_syms_sym_i_gobble:nnnnnn
4251   }
4252 }
4253 \cs_new_protected:Nn \__stex_syms_sym_i_gobble:nnnnnn {}
4254
4255 \cs_new_protected:Nn \__stex_syms_sym_i_finish:nnnnnnN {
4256   \prop_map_break:n{\seq_map_break:n{
4257     \tl_set:Ne \l_stex_get_symbol_uri { \stex_new_symbol_uri:nn {#1} {#2}}
4258     \int_set:Nn \l_stex_get_symbol_arity_int {#3}
4259     \tl_set:Nn \l_stex_get_symbol_args_tl {#4}
4260     \tl_set:Nn \l_stex_get_symbol_def_tl {#5}
4261     \tl_set:Nn \l_stex_get_symbol_type_tl {#6}
4262     \tl_set:Nn \l_stex_get_symbol_return_tl {#7}
4263     \tl_set:Nn \l_stex_get_symbol_invoke_cs {#8}
4264   }}
4265 }
4266
4267 \cs_new_protected:Nn \__stex_syms_get_symbol_from_modules:nn {
4268   %\seq_set_split:Nnn \l_tmpa_seq ? {#1}
4269   %\seq_pop_right:NN \l_tmpa_seq \l__stex_syms_name_str
4270   %\str_set:Ne \l__stex_syms_path_str {\seq_use:Nn \l_tmpa_seq ?}
4271   \stex_debug:nn{symbols}{Getting~#2~in~#1...}
4272   \seq_map_inline:Nn \l_stex_all_modules_seq {
4273     %\stex_debug:nn{symbols}{comparing~\stex_module_uri_as_qm:n{##1}~and~#1}

```

```

4274 \exp_args:Ne \stex_str_if_ends_with:nnT{\stex_module_uri_as_qm:n{##1}}{#1}{
4275 \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
4276 \_stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
4277 }
4278 }
4279
4280 %\str_if_empty:NTF \l__stex_syms_path_str {
4281 % \exp_args:NNe \exp_args:Nno \stex_str_if_ends_with:nnT {\stex_module_uri_name:n{##1}}
4282 %}{
4283 % \exp_args:NNNe \exp_args:Nno \str_if_eq:nnT\l__stex_syms_name_str{
4284 % \stex_module_uri_name:n {##1}
4285 % }
4286 %}{
4287 % \str_if_empty:NTF \l__stex_syms_path_str {
4288 % \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
4289 % \_stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
4290 % }
4291 % }{
4292 % \stex_debug:nn{symbols}{Comparing~\l__stex_syms_path_str;~with~\tl_to_str:n{##1}}
4293 % \exp_args:NNe \exp_args:Nno \stex_str_if_ends_with:nnT{\stex_module_uri_path:n {##1}}
4294 % \prop_map_inline:cn{\_stex_symbol_macro:n {##1} }{
4295 % \_stex_syms_sym_from_str_i:nnnn{##1}{#2} ####2
4296 % }
4297 % }
4298 % }
4299 %}
4300 }
4301 }
4302
4303 \prg_new_conditional:Nnn \_stex_syms_uri_match:n {T,F,TF} {
4304 \str_if_eq:nnT
4305 }
4306
4307
4308 \cs_new_protected:Nn \_stex_syms_get_from_one_string:n {
4309 \stex_debug:nn{symbols}{Getting~#1~anywhere...}
4310 \stex_iterate_symbols:n{
4311 %\stex_debug:nn{symbols}{>#1==#2~|~#1==#3<...}
4312 \bool_lazy_any:nT{
4313 {\str_if_eq_p:nn{#1}{##2}}
4314 {\str_if_eq_p:nn{#1}{##3}}
4315 {\stex_str_if_ends_with_p:nn{##3}{/#1}}
4316 }{
4317 \stex_iterate_break:n{
4318 \tl_set:Ne \l_stex_get_symbol_uri { \stex_new_symbol_uri:nn {##1} {##3}}
4319 \int_set:Nn \l_stex_get_symbol_arity_int {##4}
4320 \tl_set:Nn \l_stex_get_symbol_args_tl {##5}
4321 \tl_set:Nn \l_stex_get_symbol_def_tl {##6}
4322 \tl_set:Nn \l_stex_get_symbol_type_tl {##7}
4323 \tl_set:Nn \l_stex_get_symbol_return_tl {##8}
4324 \tl_set:Nn \l_stex_get_symbol_invoke_cs {##9}
4325 }
4326 }
4327 }

```

4328 }

(End of definition for `\stex_get_symbol:n`, `\stex_get_symbol:nTF`, and `\stex_get_symbol:nF`. These functions are documented on page 148.)

sfunction

32.3 Variables

4329 `<@@=stex_vars>`

`\l_stex_variables_prop`

4330 `\prop_new:N \l_stex_variables_prop`

(End of definition for `\l_stex_variables_prop`. This variable is documented on page 148.)

`\vardef`

```

4331 \bool_new:N \l__stex_vars_bind_bool
4332 \cs_new_protected:Nn \_stex_variable:nnnnnnnN {}
4333
4334 \stex_new_stylable_cmd:nnnn {vardef} { m O{} m} {
4335   \stex_keys_set:nn{vardef}{#2}
4336   \str_set:Ne \l_stex_macroname_str { #1 }
4337   \str_if_empty:NT \l_stex_key_name_str {
4338     \str_set:Ne \l_stex_key_name_str { #1 }
4339   }
4340
4341   \stex_symdecl_do:
4342   \stex_if_check_terms:T \_stex_check_terms:
4343   \__stex_vars_add:
4344   \__stex_vars_macro:
4345   \stex_if_do_html:T \__stex_vars_html:
4346
4347   \int_set:Nn \l_stex_get_symbol_arity_int {\l_stex_get_symbol_arity_int}
4348   \stex_debug:nn{vardef}{Doing~\l_stex_key_name_str}
4349   \tl_set_eq:NN \l_stex_get_symbol_return_tl \l_stex_key_return_tl
4350   \stex_notation_parse:n{#3}
4351   \stex_if_check_terms:T{ \_stex_notation_check: }
4352   \_stex_var_notation_macro:
4353   \stex_if_do_html:T {
4354     \def\comp{\_varcomp}
4355     \_stex_notation_do_html:n \l_stex_key_name_str
4356   }
4357   \group_begin:
4358   \tl_set_eq:NN \thisvarname \l_stex_key_name_str
4359   \tl_clear:N \thisstyle
4360   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
4361   \def\thisnotation{
4362     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{}}{
4363     \_stex_notation_make_args:
4364   }
4365   }
4366   \stex_style_apply:
4367   \group_end:\ignorespaces
4368 }{}

```

(End of definition for `\vardef`. This function is documented on page 148.)

sfunction

Adds a new variable to the current scope:

```

4369 \cs_new_protected:Nn \__stex_vars_add: {
4370   \exp_args:NNNo \exp_args:NNne
4371   \prop_put:Nnn \l_stex_variables_prop \l_stex_key_name_str {
4372     {\l_stex_macroname_str}
4373     {\l_stex_key_name_str}
4374     {\int_use:N \l_stex_get_symbol_arity_int}
4375     {\l_stex_get_symbol_args_tl}
4376     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4377     {\exp_args:No \exp_not:n \l_stex_key_type_tl}
4378     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4379     \stex_invoke_symbol:
4380   }
4381 }
```

Defines the macro for a variable:

```

4382 \cs_new_protected:Nn \__stex_vars_macro: {
4383   \tl_set:ce{\l_stex_macroname_str}{
4384     \stex_invoke_variable:nnnnnnN
4385     {\l_stex_key_name_str}
4386     {\int_use:N \l_stex_get_symbol_arity_int}
4387     {\l_stex_get_symbol_args_tl}
4388     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4389     {\exp_args:No \exp_not:n \l_stex_key_type_tl}
4390     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4391     \stex_invoke_symbol:
4392   }
4393 }
```

Insert the HTML for a variable declaration:

```

4394
4395 \cs_new_protected:Nn \__stex_vars_html: {
4396   \stex_if_do_html:T {
4397     \hbox\bgroup\exp_args:Ne \stex_annotate_invisible:nn {
4398       data-ftml-vardef = {\l_stex_key_name_str},
4399       data-ftml-args = {\l_stex_key_args_str}
4400       \str_if_empty:NF \l_stex_macroname_str {,
4401         data-ftml-macroname={\l_stex_macroname_str}
4402       }
4403       \str_if_empty:NF \l_stex_key_assoc_str {,
4404         data-ftml-assoctype={\l_stex_key_assoc_str}
4405       }
4406       \str_if_empty:NF \l_stex_key_role_str {,
4407         data-ftml-role={\l_stex_key_role_str}
4408       }
4409       \str_if_empty:NF \l_stex_key_reorder_str {,
4410         data-ftml-reorderargs={\l_stex_key_reorder_str}
4411       }
4412       \bool_if:NT \l__stex_vars_bind_bool {,
4413         data-ftml-bind={true}
4414       }
4415     }\}
```

```

4416 \stex_annotate_force_break:n{
4417 \bool_set_true:N \stex_in_invisible_html_bool
4418 \tl_if_empty:NF \l_stex_key_type_tl {
4419 \stex_annotate:nn{data-ftml-type={}}{${\l_stex_key_type_tl$}
4420 }
4421 \tl_if_empty:NF \l_stex_key_def_tl {
4422 \stex_annotate:nn{data-ftml-definiens={}}{${\l_stex_key_def_tl$}
4423 }
4424 \tl_if_empty:NF \l_stex_key_return_tl{
4425 \exp_args:Nno \use:n{
4426 \cs_generate_from_arg_count:NNnn \l__stex_vars_cs
4427 \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
4428 \tl_set:Ne \l__stex_vars_args_tl {\stex_map_args:N \stex_return_args:nn}
4429 ${\stex_annotate:nn{data-ftml-returntype={}}}{
4430 \exp_after:wN \l__stex_vars_cs \l__stex_vars_args_tl!}$
4431 }
4432 \tl_if_empty:NF \l_stex_key_argtypes_clist {
4433 \stex_annotate:nn{data-ftml-argtypes={}}{
4434 \stex_annotate_force_break:n{
4435 \clist_map_inline:Nn \l_stex_key_argtypes_clist {
4436 ${\stex_annotate:nn{data-ftml-type={}}}{##1}$
4437 }
4438 }
4439 }
4440 }
4441 }
4442 }\egroup
4443 }
4444 }

```

`\newvar`

```

4445 \NewDocumentCommand\newvar{ m O{} }{
4446 \vardef{v#1}[name=#1,#2]{#1}
4447 }

```

(End of definition for `\newvar`. This function is documented on page 148.)

sfunction

`\newseq`

```

4448 \NewDocumentCommand\newseq{ m O{} m }{
4449 \varseq{v#1}[name=#1,#2]{#3}{\maincomp{#1}}\c_math_subscript_token{##1}}
4450 }

```

(End of definition for `\newseq`. This function is documented on page 148.)

sfunction

`\stex_get_var:n`

```

4451 \cs_new_protected:Nn \stex_get_var:n {
4452 \str_clear:N \l_stex_get_variable_str
4453 \__stex_vars_get_var:n{#1}
4454 \str_if_empty:NT \l_stex_get_variable_str {
4455 \msg_error:nnn{stex}{error/unknownsymbol}{#1}
4456 }
4457 }

```

```

4458
4459 \cs_new_protected:Nn \__stex_vars_get_var:n {
4460   \prop_map_inline:Nn \l_stex_variables_prop {
4461     \__stex_vars_check_var:nnnnnnnnN {#1} ##2
4462   }
4463 }
4464
4465 \cs_new_protected:Nn \__stex_vars_check_var:nnnnnnnnN {
4466   \str_if_eq:nnTF{#1}{#2}{
4467     \prop_map_break:n{\__stex_vars_set_vars:nnnnnnN {#3}{#4}{#5}{#6}{#7}{#8}{#9}}
4468   }{
4469     \str_if_eq:nnT{#1}{#3}{
4470       \prop_map_break:n{\__stex_vars_set_vars:nnnnnnN {#3}{#4}{#5}{#6}{#7}{#8}{#9}}
4471     }
4472   }
4473 }
4474
4475 \cs_new_protected:Nn \__stex_vars_set_vars:nnnnnnN {
4476   \stex_debug:nn{symbols}{Variable~#1~found}
4477   \cs_set:Npn \_stex_variable:nnnnnnnnN ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 {}
4478   \str_set:Nn \l_stex_get_variable_str {#1}
4479   \int_set:Nn \l_stex_get_symbol_arity_int {#2}
4480   \tl_set:Nn \l_stex_get_symbol_args_tl {#3}
4481   \tl_set:Nn \l_stex_get_symbol_def_tl {#4}
4482   \tl_set:Nn \l_stex_get_symbol_type_tl {#5}
4483   \tl_set:Nn \l_stex_get_symbol_return_tl {#6}
4484   \tl_set:Nn \l_stex_get_symbol_invoke_cs {#7}
4485 }

```

(End of definition for `\stex_get_var:n`. This function is documented on page 149.)

sfunction

32.3.1 Variable Sequences

```

4486 <@@=stex_seqs>

\varseq

4487 \stex_new_stylable_cmd:nnnn {varseq}{m 0{} m m} {
4488   \stex_keys_set:nn{symdef}{#2}
4489   \str_set:Ne \l_stex_macroname_str { #1 }
4490   \str_if_empty:NT \l_stex_key_name_str {
4491     \str_set:Ne \l_stex_key_name_str { #1 }
4492   }
4493   \str_if_empty:NT \l_stex_key_args_str {
4494     \str_set:Nn \l_stex_key_args_str {1}
4495   }
4496   \stex_symdecl_do:
4497
4498   \tl_set_eq:NN \l_stex_get_symbol_return_tl \l_stex_key_return_tl
4499   \clist_set:Nn \l__stex_seqs_range_clist {#3}
4500   \tl_if_empty:NTF \l_stex_key_op_tl {
4501     \stex_notation_parse:n{#4}
4502     \tl_clear:N \l_stex_key_op_tl
4503   }{

```

```

4504     \stex_notation_parse:n{#4}
4505   }
4506   \stex_if_do_html:T \__stex_seqs_html:
4507   \stex_if_check_terms:T \_stex_check_terms:
4508   \__stex_seqs_add:
4509   \__stex_seqs_macro:
4510   \stex_if_check_terms:T \_stex_notation_check:
4511   \_stex_var_notation_macro:
4512   \stex_if_do_html:T {
4513     \def\comp{\_varcomp}
4514     \_stex_notation_do_html:n \l_stex_key_name_str
4515   }
4516   \group_begin:
4517   \tl_set_eq:NN \thisvarname \l_stex_key_name_str
4518   \tl_clear:N \thisstyle
4519   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
4520   \def\thisnotation{
4521     \l_stex_notation_macrocode_cs{}!
4522   }
4523   \stex_style_apply:
4524   \group_end:\ignorespaces
4525 }{}
4526
4527 \cs_new_protected:Nn \__stex_seqs_add: {
4528   \exp_args:NNNo \exp_args:NNne
4529   \prop_put:Nnn \l_stex_variables_prop \l_stex_key_name_str {
4530     {\l_stex_macroname_str}
4531     {\l_stex_key_name_str}
4532     {\int_use:N \l_stex_get_symbol_arity_int}
4533     {\l_stex_get_symbol_args_tl}
4534     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4535     {\exp_args:No \exp_not:n \l__stex_seqs_range_clist}
4536     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4537     \stex_invoke_sequence:
4538   }
4539 }
4540
4541 \cs_new_protected:Nn \__stex_seqs_macro: {
4542   \tl_set:ce{\l_stex_macroname_str}{
4543     \_stex_invoke_variable:nnnnnnN
4544     {\l_stex_key_name_str}
4545     {\int_use:N \l_stex_get_symbol_arity_int}
4546     {\l_stex_get_symbol_args_tl}
4547     {\exp_args:No \exp_not:n \l_stex_key_def_tl}
4548     {\exp_args:No \exp_not:n \l__stex_seqs_range_clist}
4549     {\exp_args:No \exp_not:n \l_stex_key_return_tl}
4550     \stex_invoke_sequence:
4551   }
4552 }
4553
4554 \cs_new_protected:Nn \__stex_seqs_html: {
4555   \exp_args:Ne \stex_annotate_invisible:nn {
4556     data-ftml-varseq = {\l_stex_key_name_str},
4557     data-ftml-args = {\l_stex_key_args_str}

```

```

4558 \str_if_empty:NF \l_stex_macroname_str {,
4559   data-ftml-macroname={\l_stex_macroname_str}
4560 }
4561 \str_if_empty:NF \l_stex_key_assoc_str {,
4562   data-ftml-assoctype={\l_stex_key_assoc_str}
4563 }
4564 \str_if_empty:NF \l_stex_key_role_str {,
4565   data-ftml-role={\l_stex_key_role_str}
4566 }
4567 \str_if_empty:NF \l_stex_key_reorder_str {,
4568   data-ftml-reorderargs={\l_stex_key_reorder_str}
4569 }
4570 }{\hbox\bgroup
4571   \stex_annotate_force_break:n{
4572
4573     $\stex_annotate:nn{data-ftml-seqrange={}}{
4574       \exp_args:Nno\symuse{Metatheory?sequence-range}\l__stex_seqs_range_clist
4575     }$
4576
4577     \tl_if_empty:NF \l_stex_key_type_tl {
4578       \stex_annotate:nn{data-ftml-type={}}{${\l_stex_key_type_tl$}
4579     }
4580     \tl_if_empty:NF \l_stex_key_def_tl {
4581       \stex_annotate:nn{data-ftml-definiens={}}{${\l_stex_key_def_tl$}
4582     }
4583     \tl_if_empty:NF \l_stex_key_return_tl{
4584       \exp_args:Nno \use:n{
4585         \cs_generate_from_arg_count:NNnn \l__stex_seqs_cs
4586         \cs_set:Npn \l_stex_get_symbol_arity_int} \l_stex_key_return_tl
4587         \tl_set:Nx \l__stex_seqs_args_tl {\stex_map_args:N \stex_return_args:nn}
4588         \stex_annotate:nn{data-ftml-returntype={}}{
4589           $\exp_after:wN \l__stex_seqs_cs \l__stex_seqs_args_tl!$}
4590       }
4591     \tl_if_empty:NF \l_stex_key_argtypes_clist {
4592       \stex_annotate:nn{data-ftml-argtypes={}}{
4593         \stex_annotate_force_break:n{
4594           \clist_map_inline:Nn \l_stex_key_argtypes_clist {
4595             \stex_annotate:nn{data-ftml-type={}}{###1$}
4596           }
4597         }
4598       }
4599     }
4600   }
4601 \egroup}
4602 }

```

(End of definition for \varseq. This function is documented on page 149.)

sfunction

32.4 Expressions

```

4603 <@@=stex_expr>
\symuse

```



```

4604 \cs_new_protected:Npn \symuse #1 {
4605   \stex_get_symbol:n{#1}
4606   \exp_args:Nno \use:n {\_stex_invoke_symbol:NeooooN
4607   \l_stex_get_symbol_uri
4608   {\int_use:N \l_stex_get_symbol_arity_int}
4609   \l_stex_get_symbol_args_tl
4610   \l_stex_get_symbol_def_tl
4611   \l_stex_get_symbol_type_tl
4612   \l_stex_get_symbol_return_tl}
4613   \l_stex_get_symbol_invoke_cs
4614 }

```

(End of definition for \symuse. This function is documented on page 149.)
sfunction

_stex_next_symbol:n

```

4615 \tl_new:N \l__stex_expr_reset_tl
4616
4617 \cs_new_protected:Nn \_stex_next_symbol:n {
4618   \tl_set:Ne \l_stex_every_symbol_tl {
4619     \tl_gset:Nn \exp_not:N \l__stex_expr_reset_tl {
4620       \tl_set:Nn \exp_not:N \l_stex_every_symbol_tl {
4621         \exp_args:No \exp_not:n \l_stex_every_symbol_tl
4622       }
4623       \tl_gset:Nn \exp_not:N \l__stex_expr_reset_tl {
4624         \exp_args:No \exp_not:n \l__stex_expr_reset_tl
4625       }
4626     }
4627     \tl_set:Nn \exp_not:N \l_stex_every_symbol_tl {
4628       \exp_args:No \exp_not:n \l_stex_every_symbol_tl
4629     }
4630     \exp_not:n{ \aftergroup \l__stex_expr_reset_tl }
4631     \exp_not:N \l_stex_every_symbol_tl
4632     \exp_not:n{ #1 }
4633   }
4634 }
4635 \cs_generate_variant:Nn \_stex_next_symbol:n {e}

```

(End of definition for _stex_next_symbol:n. This function is documented on page 149.)
sfunction

_stex_invoke_symbol:NnnnnnN
_stex_invoke_symbol:NeooooN
_stex_invoke_symbol:nnnnnnN

- \l_stex_current_symbol_uri,
- \l_stex_current_symbol_arity_int,
- \l_stex_current_symbol_args_tl,
- definiens (currently not used),
- \l_stex_current_symbol_type_tl,
- \l_stex_current_symbol_return_tl,

- the invocation macro (is called after setup).

```

4636 \cs_new_protected:Nn \_stex_invoke_symbol:nnnnnnN {
4637   \bool_if:NTF \l_stex_allow_semantic_bool{
4638     \group_begin:
4639     \tl_set:Nn \l_stex_current_symbol_uri {#1}
4640     \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_current_symbol_uri
4641     \tl_set:Nn \l_stex_current_display_tl {
4642       \stex_symbol_uri_name:N \l_stex_current_symbol_uri
4643     }
4644     \__stex_expr_invoke:nnnnN{#2}{#3}{#5}{#6}{#7}
4645   }{
4646     \msg_error:nnxx{stex}{error/notallowed}{
4647       \stex_use_symbol_uri:n{#1}
4648     }{
4649       \l_stex_current_full_tl
4650     }
4651   }
4652 }
4653
4654 \cs_new_protected:Npn \_stex_invoke_symbol:NnnnnnnN {
4655   \exp_args:No \_stex_invoke_symbol:nnnnnnN
4656 }
4657 \cs_generate_variant:Nn \_stex_invoke_symbol:NnnnnnnN {NeooooN}

```

Whether semantic macros are allowed currently. This is false e.g. in the body of a semantic macro outside of one of its arguments:

```

4658 \bool_new:N \l_stex_allow_semantic_bool
4659 \bool_set_true:N \l_stex_allow_semantic_bool

```

Code to execute every time a semantic macro is invoked:

```

4660 \tl_set:Nn \l_stex_every_symbol_tl {
4661   \bool_set_false:N \l_stex_allow_semantic_bool
4662 }

```

The current top-level term; may be undefined:

```

4663 \tl_new:N \l_stex_current_term_tl

```

Keeps track of all set up so it can be reexecuted; this may be necessary to e.g. typeset `\this` in a struct field:

```

4664 \tl_new:N \l_stex_current_redo_tl

```

Setup for symbols; takes: arity, args, type, return, invoke. Calls invoke at the end.

In HTML mode, we make sure we enter horizontal mode *before* any annotations are inserted

```

4665 \cs_new_protected:Nn \__stex_expr_invoke:nnnnN {
4666   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
4667   \__stex_expr_setup:nnnnn{\_comp}{#1}{#2}{#4}{#3}
4668   \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_oms:nn
4669   \tl_put_right:Nn \l_stex_current_redo_tl{
4670     \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_oms:nn
4671   }
4672   #5
4673 }

```

general setup; takes: comp, arity, args, return, type

```

4674 \cs_new_protected:Nn \__stex_expr_setup:nnnnn {
4675   \tl_clear:N \l_stex_return_notation_tl
4676   \tl_set:Nn \l_stex_current_redo_tl {
4677     \let \this \stex_current_this:
4678     \def\comp{#1}
4679     \def\maincomp{\comp}
4680     \str_set:Nn \l_stex_current_arity_str{ #2 }
4681     \tl_set:Nn \l_stex_current_args_tl{ #3 }
4682     \tl_set:Nn \l_stex_current_return_tl{ #4 }
4683     \tl_set:Nn \l_stex_current_type_tl{ #5 }
4684     \tl_clear:N \l_stex_current_term_tl
4685   }
4686   \tl_put_right:N \l_stex_current_redo_tl \l_stex_every_symbol_tl
4687   \tl_put_left:Ne \l_stex_current_redo_tl {
4688     \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
4689   }
4690   \l_stex_current_redo_tl
4691 }

```

(End of definition for `\stex_invoke_symbol:NnnnnnN` and `\stex_invoke_symbol:nnnnnnN`. These functions are documented on page 149.)

sfunction

`\stex_invoke_symbol:` Math or text mode?

```

4692 \cs_new_protected:Nn \stex_invoke_symbol: {
4693   \stex_debug:nn{expressions}{Invoking~ \l_stex_current_full_tl}
4694   \mode_if_math:TF \__stex_expr_math: \__stex_expr_text:
4695 }

```

(End of definition for `\stex_invoke_symbol:.` This function is documented on page 149.)

sfunction

`\stex_invoke_text_symbol:`

```

4696 \cs_new_protected:Nn \stex_invoke_text_symbol: {
4697   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
4698   \stex_term_oms_or_omv:nn{}{\maincomp{\let\xspace\relax\l_stex_current_return_tl}}
4699   \group_end:\mode_if_math:F{\cs_if_exist:NT\xspace\xspace}
4700 }

```

(End of definition for `\stex_invoke_text_symbol:.` This function is documented on page 149.)

sfunction

`\stex_invoke_sequence:`

```

4701 \cs_new_protected:Nn \stex_invoke_sequence: {
4702   \peek_charcode_remove:NTF ! {
4703     \peek_charcode:NTF [ \__stex_expr_seq_op:w { \__stex_expr_seq_op:w [] }
4704     } \__stex_expr_seq_first:
4705   }
4706
4707   \cs_new_protected:Npn \__stex_expr_seq_op:w [#1] {
4708     \stex_use_op_notation:nnTF \l_stex_current_full_tl{#1}{
4709       %\stex_debug:nn{vars}{Here: \meaning\l_stex_notation_cs}
4710       \stex_maybe_brackets:nn{\neginfprec}{

```

```

4711     \_stex_term_oms_or_omv:nn{#1}
4712     {\l_stex_notation_cs}\group_end:
4713   }
4714   ){
4715     \__stex_expr_get_index_notation:n{#1}
4716     \peek_charcode:NTF [ \__stex_expr_range:w { \__stex_expr_range:w[] }
4717   }
4718 }
4719
4720 \cs_new_protected:Nn \__stex_expr_get_index_notation:n {
4721   \stex_use_notation:nnTF \l_stex_current_full_tl{#1}{}{
4722     \stex_do_default_notation:
4723     \cs_set_eq:NN \l_stex_notation_cs \l_stex_default_notation
4724   }
4725 }
4726
4727 \cs_new_protected:Npn \__stex_expr_range:w [#1] {
4728   \bool_set_true:N \l_stex_allow_semantic_bool
4729   \clist_clear:N \l__stex_expr_clist
4730   \clist_map_function:NN \l_stex_current_type_tl \__stex_expr_seq_arg:n
4731   \stex_annotate:nn{
4732     data-ftml-term=OMV,
4733     data-ftml-head={\l_stex_current_full_tl},
4734     data-ftml-notationid={}
4735   }{
4736     \l__stex_expr_clist
4737   }
4738   \group_end:
4739 }
4740
4741 \cs_new_protected:Nn \__stex_expr_seq_arg:n {
4742   \tl_if_eq:nnTF{#1}{\ellipses}{
4743     \clist_put_right:Nn \l__stex_expr_clist {
4744       \ellipses
4745     }
4746   ){
4747     \clist_put_right:Nn \l__stex_expr_clist {
4748       \exp_args:No \str_if_eq:nnTF \l_stex_current_arity_str {1}{
4749         \group_begin:
4750         \l_stex_notation_cs \group_end: {#1}
4751       }{
4752         \group_begin:
4753         \l_stex_notation_cs \group_end: #1
4754       }
4755     }
4756   }
4757 }
4758 }
4759
4760 \cs_new_protected:Nn \__stex_expr_seq_first: {
4761   \exp_args:Nne \use:nn{
4762     \cs_generate_from_arg_count:NNnn \l_stex_notation_cs \cs_set:Npn
4763     \l_stex_current_arity_str}{
4764     \tl_set:Nn \exp_not:N \l__stex_expr_first_args_tl {

```

```

4765     \int_step_function:nN \l_stex_current_arity_str \__stex_expr_do_first_arg:n
4766   }
4767   \exp_not:N \__stex_expr_do_first_next:
4768 }
4769 \l_stex_notation_cs
4770 }
4771
4772 \cs_new:Nn \__stex_expr_do_first_arg:n {{\exp_not:n{## #1}}}
4773
4774 \cs_new_protected:Nn \__stex_expr_do_first_next: {
4775   \peek_charcode_remove:NTF ! {
4776     \peek_charcode:NTF [ \__stex_expr_do_one:w {\__stex_expr_do_one:w []}
4777   }{
4778     \peek_charcode:NTF [ \__stex_expr_do_all:w {\__stex_expr_do_all:w []}
4779   }
4780 }
4781
4782 \cs_new_protected:Npn \__stex_expr_do_one:w [#1] {
4783   \__stex_expr_get_index_notation:n{#1}
4784   \exp_args:Nno\use:nn{\l_stex_notation_cs\group_end:}\l__stex_expr_first_args_tl
4785 }
4786
4787 \cs_new_protected:Npn \__stex_expr_do_all:w [#1] {
4788   \exp_args:Nno\use:nn{\__stex_expr_notation:w [#1]}\l__stex_expr_first_args_tl
4789 }

```

(End of definition for `\stex_invoke_sequence:..` This function is documented on page 150.)

sfunction

`\stex_invoke_structure:`

```

4790 \cs_new_protected:Nn \stex_invoke_structure: {
4791   \tl_set:Nn \l__stex_expr_set_comp_tl {\__stex_expr_set_thiscomp:}
4792   \__stex_expr_struct_top:n {}
4793 }
4794
4795 \cs_new_protected:Nn \__stex_expr_set_thiscomp: {
4796   \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_thiscomp {
4797     \edef\maincomp {\_thiscomp{\comp}}
4798   }
4799 }
4800
4801 \cs_new_protected:Nn \__stex_expr_struct_top:n {
4802   \stex_debug:nn{structure}{
4803     invoking~structure~{\l_stex_current_type_tl}<\tl_to_str:n{#1}>
4804   }
4805   \peek_charcode:NTF [ {
4806     \__stex_expr_merge:nw{#1}
4807   }{
4808     \__stex_expr_struct_type:n {#1}
4809     \tl_set:Nn \l_stex_struct_this_tl {}
4810     \peek_charcode_remove:NTF ! {
4811       \peek_charcode:NTF [ {
4812         \__stex_expr_maybe_notation:w
4813       }{

```

```

4814     \_stex_expr_maybe_notation:w []
4815   }
4816   }{
4817     \_stex_expr_invoke_this:n
4818   }
4819 }
4820 }
4821
4822 \cs_new_protected:Npn \_stex_expr_merge:nw #1 [ #2 ] {
4823   \exp_args:Ne \stex_str_if_starts_with:nnTF {\tl_to_str:n{#2}}{comp=}{
4824     \_stex_expr_set_customcomp: #2 \_stex_expr_end:
4825     \_stex_expr_struct_top:n{#1}
4826   }{
4827     \exp_args:Ne \stex_str_if_starts_with:nnTF {\tl_to_str:n{#2}}{this=}{
4828       \_stex_expr_set_thisnotation: #2 \_stex_expr_end:
4829       \_stex_expr_struct_top:n{#1}
4830     }{
4831       \tl_if_empty:nTF{#1}{
4832         \_stex_expr_struct_top:n{#2}
4833       }{
4834         \tl_if_empty:nTF{#2}{
4835           \_stex_expr_struct_top:n{#1}
4836         }{
4837           \_stex_expr_struct_top:n{#1,#2}
4838         }
4839       }
4840     }
4841   }
4842 }
4843
4844 \cs_new_protected:Npn \_stex_expr_set_thisnotation: this= #1 \_stex_expr_end: {
4845   \tl_set:Nn \l_stex_return_notation_tl { \comp{#1} }
4846   \tl_set:Nn \l__stex_expr_set_comp_tl {}
4847 }
4848
4849 \cs_new_protected:Npn \_stex_expr_set_customcomp: comp= #1 \_stex_expr_end: {
4850   \tl_set:Nn \l__stex_expr_set_comp_tl {
4851     \_stex_expr_set_custom_comp:n{#1}
4852   }
4853   \tl_set:Nn \l_stex_return_notation_tl { \comp{} }
4854 }

```

The structure type:

```

4855 \cs_new_protected:Nn \_stex_expr_struct_type:n {
4856   \_stex_expr_do_assign_list:n{#1}
4857   \clist_if_empty:NTF \l__stex_expr_fields_clist {
4858     \int_compare:nNnTF {\clist_count:N \l_stex_current_type_tl}
4859       = 1 {
4860       \tl_set:Ne \l__stex_expr_current_type_tl {
4861         \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl }
4862         \exp_args:No \exp_not:n \l_stex_current_redo_tl
4863         \_stex_term oms_or_omv:nn{}{}
4864       }
4865     }{
4866       \exp_args:No \_stex_expr_make_type:n \l_stex_current_type_tl

```

```

4867   }
4868   }{
4869     \int_compare:nNnTF {\clist_count:N \l_stex_current_type_tl}
4870       = 1 {
4871         \__stex_expr_make_type:n {}
4872       }{
4873         \exp_args:No \__stex_expr_make_type:n \l_stex_current_type_tl
4874       }
4875   }
4876 }
4877
4878 \cs_new_protected:Nn \__stex_expr_do_assign_list:n {
4879   \clist_clear:N \l__stex_expr_fields_clist
4880   \tl_if_empty:nF {#1} {
4881     \keyval_parse:NNn\TODO\__stex_expr_do_assign:nn{#1}
4882   }
4883 }
4884
4885 \cs_new_protected:Nn \__stex_expr_do_assign:nn {
4886   \clist_put_right:Nn \l__stex_expr_fields_clist {{#1}{#2}}
4887 }
4888
4889 \cs_new_protected:Nn \__stex_expr_make_type:n {
4890   \tl_if_empty:nTF{#1}{
4891     \seq_clear:N \l_tmpa_seq
4892   }{
4893     \seq_set_split:Nnn \l_tmpa_seq ,{#1}
4894     \seq_pop_right:NN \l_tmpa_seq \l_tmpa_tl
4895     \seq_reverse:N \l_tmpa_seq
4896   }
4897   \tl_set:Ne \l__stex_expr_current_type_tl {
4898     \symuse{Metatheory?module~type~merge}{
4899       {
4900         \exp_args:No \exp_not:n \l_stex_current_redo_tl
4901         \stex_term_oms_or_omv:nn{}{}}
4902     }
4903     \seq_map_function:NN \l_tmpa_seq \__stex_expr_make_mod:n
4904     \clist_if_empty:NF \l__stex_expr_fields_clist {
4905       ,\symuse{Metatheory?anonymous~record}{
4906         \exp_args:Ne \tl_tail:n{
4907           \clist_map_function:NN \l__stex_expr_fields_clist \__stex_expr_make_oml:n
4908         }
4909       }
4910     }
4911   }
4912 }
4913 }
4914
4915 \cs_new:Nn \__stex_expr_make_mod:n {
4916   ,\symuse{Metatheory?module~type}{
4917     \exp_args:Ne \stex_annotate:nn{data-ftml-term=OMMOD,data-ftml-head={\stex_use_module_ur
4918   }
4919 }
4920

```

```

4921
4922 \cs_new:Nn \__stex_expr_make_oml:n {
4923   \__stex_expr_make_oml:nn #1
4924 }
4925 \cs_new:Nn \__stex_expr_make_oml:nn {
4926   ,\stex_annotate:nn{
4927     data-ftml-term=OML,
4928     data-ftml-head={#1}
4929   }{
4930     \stex_annotate_force_break:n{
4931       \stex_annotate:nn{data-ftml-definiens={}}{\exp_not:n{#2!}}
4932     }
4933   }
4934 }

```

Insert the structure type as a term:

```

4935 \cs_new:Nn \__stex_expr_current_type: {
4936   %\exp_args:No \exp_not:n \l_stex_current_redo_tl
4937   \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
4938     \exp_args:No\exp_not:n\l__stex_expr_current_type_tl
4939   }
4940   \stex_term_oms_or_omv:nn{}{}
4941 }

```

The structure type itself:

```

4942 \cs_new_protected:Npn \__stex_expr_maybe_notation:w [ #1 ] {
4943   \tl_set_eq:NN \l_stex_current_term_tl \l__stex_expr_current_type_tl
4944   \stex_use_notation:nnTF \l_stex_current_full_tl {#1}{
4945     \l_stex_notation_cs\group_end:
4946   }{
4947     \__stex_expr_make_prop:
4948     \__stex_expr_make_prop_assign:
4949     \__stex_expr_present_i:w [ #1 ]
4950   }
4951 }
4952
4953 \cs_new_protected:Nn \__stex_expr_present: {
4954   \peek_charcode:NTF [ {
4955     \__stex_expr_present_i:w
4956   }{
4957     \__stex_expr_present:nn{}{}
4958   }
4959 }
4960
4961 \cs_new_protected:Npn \__stex_expr_present_i:w [ #1 ] {
4962   \int_compare:nNnTF{\clist_count:n{#1}} = 1 {
4963     \__stex_expr_present:nn{}{#1}
4964   }{
4965     \peek_charcode:NTF [ {
4966       \__stex_expr_present_ii:nw{#1}
4967     }{
4968       \__stex_expr_present:nn{#1}{}
4969     }
4970   }
4971 }

```



```

4972
4973 %First: clist, second:notation-id
4974 \cs_new_protected:Npn \__stex_expr_present_ii:nw #1 [#2] {
4975   \__stex_expr_present:nn{#1}{#2}
4976 }
4977
4978 \cs_new_protected:Nn \__stex_expr_present:nn {
4979   \clist_clear:N \l__stex_expr_clist
4980   \tl_if_empty:nTF{#1}{
4981     \cs_set:Npn \l__stex_expr_cs ##1 ##2 ##3 {
4982       \tl_if_empty:nF{##2}{
4983         \__stex_expr_present_entry:nn {##1}{##3}
4984       }
4985     }
4986   }{
4987     \cs_set:Npn \l__stex_expr_cs ##1 ##2 ##3 {
4988       \exp_args:Ne \clist_if_in:nnT{\tl_to_str:n{#1}}{##1}{
4989         \__stex_expr_present_entry:nn {##1}{##3}
4990       }
4991     }
4992   }
4993   \prop_map_inline:Nn \l__stex_expr_prop {
4994     \l__stex_expr_cs {##1} ##2
4995   }
4996   \stex_term_oms_or_omv:nn{}{
4997     \exp_args:Nno \use:n{
4998       \bool_set_true:N \l_stex_allow_semantic_bool
4999       \symuse{Metatheory?mathematical~structure}[#2]
5000     }{\l__stex_expr_clist}
5001   }\group_end:
5002 }
5003
5004 \cs_new_protected:Nn \__stex_expr_present_entry:nn {
5005   \seq_if_in:NnTF \l__stex_expr_assigned_seq {#1}{
5006     \clist_put_right:Nn \l__stex_expr_clist {#2!}
5007   }{
5008     \exp_args:Nne \clist_put_right:Nn \l__stex_expr_clist {
5009       \stex_next_symbol:n {
5010         \exp_args:No \exp_not:n \l__stex_expr_set_comp_tl
5011         \tl_set:Nn \exp_not:N \l_stex_struct_this_tl {
5012           \exp_args:No \exp_not:n \l_stex_struct_this_tl
5013         }
5014         \exp_not:n {
5015           \tl_set_eq:NN \this \l_stex_struct_this_tl
5016         }
5017         \tl_set:Nn \exp_not:N \l_stex_return_notation_tl {
5018           \exp_args:No \exp_not:n \l_stex_return_notation_tl
5019         }
5020       }
5021       \exp_not:n{#2!}
5022     }
5023   }
5024 }
5025

```

```

5026 %\cs_new_protected:Nn \__stex_expr_set_custom_comp:n {
5027 % \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_customthiscomp {
5028 % \cs_set_protected:Npx \_customthiscomp ##1 {
5029 % \group_begin:
5030 % \bool_set_true:N \l_stex_allow_semantic_bool
5031 % \exp_not:n{
5032 % \cs_set:Npn \l__stex_expr_comp_cs ##1 {
5033 % #1
5034 % }
5035 % \def\maincomp
5036 % }{\comp}
5037 % \exp_not:N\l__stex_expr_comp_cs{\comp{##1}}
5038 % \group_end:
5039 % }
5040 % \def\maincomp {\_customthiscomp}
5041 % }
5042 %}

5043
5044 \cs_new_protected:Nn \__stex_expr_set_custom_comp:n {
5045 \exp_args:Ne \tl_if_eq:NNF {\tl_head:N \maincomp} \_customthiscomp {
5046 \stex_debug:nn{structs}{Setting~custom~comp:~\tl_to_str:n{##1}}
5047 \exp_args:Nne \__stex_expr_set_custom_i:nn{##1}{\comp}
5048 \def\maincomp {\_customthiscomp}
5049 }
5050 }
5051
5052 \cs_new_protected:Nn \__stex_expr_set_custom_i:nn {
5053 \cs_set_protected:Npn \_customthiscomp ##1 {
5054 \group_begin:
5055 \bool_set_true:N \l_stex_allow_semantic_bool
5056 \cs_set:Npn \l__stex_expr_comp_cs ####1 { #1 }
5057 \def \maincomp {#2}
5058 \l__stex_expr_comp_cs{#2{##1}}
5059 \group_end:
5060 }
5061 }

this (of type structure):
5062 \cs_new_protected:Nn \__stex_expr_invoke_this:n {
5063 \peek_charcode_remove:NTF ! {
5064 \exp_args:Nne\use:nn{
5065 \group_end:\symuse{Metatheory?of~type}[invisible]{
5066 \tl_if_empty:nTF{##1}{\_stex_annotate_force_break:n{}}{##1}
5067 }
5068 }{
5069 {
5070 \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
5071 \__stex_expr_current_type:
5072 }
5073 }
5074 }{
5075 \stex_debug:nn{structure}{This:~\tl_to_str:n{##1}}
5076 \__stex_expr_invoke_maybe_field:nn{##1}
5077 }
5078 }

```

```

5079
5080 \cs_new_protected:Nn \__stex_expr_invoke_maybe_field:nn {
5081   \__stex_expr_make_prop:
5082   \__stex_expr_set_this:n{#1}
5083   \tl_if_empty:nTF{#2}{
5084     \__stex_expr_make_prop_assign:
5085     \__stex_expr_present:
5086   }{
5087     \__stex_expr_invoke_field:n{#2}
5088   }
5089 }
5090
5091 \tl_new:N \l_stex_current_rec_tl
5092 \cs_new_protected:Nn \__stex_expr_set_this:n {
5093   \stex_debug:nn{structure}{setting~this~to:\tl_to_str:n{#1}}
5094   \tl_if_empty:nTF{#1}{
5095     %\tl_put_right:Nn \l_stex_current_redo_tl {
5096     % \tl_clear:N \l_stex_struct_this_tl
5097     %}
5098   }{
5099     \tl_set:Ne \l_stex_struct_this_tl {{
5100       \bool_set_true:N \l_stex_allow_semantic_bool
5101       \bool_set_true:N \l_stex_in_this_bool
5102       \tl_set:Nn \exp_not:N \this {
5103         \exp_args:No \exp_not:n \this
5104       }
5105       \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl }
5106       \exp_not:n{#1}
5107     }}
5108     \tl_set:Ne \l_stex_current_rec_tl {{
5109       \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl }
5110       \exp_not:n{#1}
5111     }}
5112     \tl_set_eq:NN \this \l_stex_struct_this_tl
5113     % \l_stex_return_notation_tl
5114   }
5115 }
5116
5117 \cs_new_protected:Nn \__stex_expr_get_field_name:n {
5118   \str_set:Ne \l__stex_expr_field_name_str {
5119     \exp_args:Nne \use:n {\exp_after:wN \use_i:nn \use:n}
5120     {\prop_item:Nn \l__stex_expr_prop {#1}}
5121   }
5122   \str_if_empty:NT \l__stex_expr_field_name_str {
5123     \str_set:Nn \l__stex_expr_field_name_str {#1}
5124   }
5125 }
5126
5127 \cs_new_protected:Nn \__stex_expr_invoke_field:n {
5128   \stex_debug:nn{structure}{Field~for~\tl_to_str:o \l_stex_struct_this_tl}
5129   \prop_if_in:NnTF \l__stex_expr_prop {#1}{
5130     \__stex_expr_get_field_name:n{#1}
5131     \tl_clear:N \l__stex_expr_more_nextsymbol_tl
5132     %\exp_args:NNe \seq_if_in:NnF \l__stex_expr_assigned_seq {\tl_to_str:n{#1}}{

```

```

5133 \tl_set:Nn \l__stex_expr_more_nextsymbol_tl {
5134 \tl_set:Nn \exp_not:N \l_stex_struct_this_tl {
5135 \exp_args:No \exp_not:n \l_stex_struct_this_tl
5136 }
5137 \tl_set:Nn \exp_not:N \l_stex_current_rec_tl {
5138 \exp_args:No \exp_not:n \l_stex_current_rec_tl
5139 }
5140 \exp_not:n {
5141 \tl_set_eq:NN \this \l_stex_struct_this_tl
5142 }
5143 \tl_set:Nn \exp_not:N \l_stex_return_notation_tl {
5144 \exp_args:No \exp_not:n \l_stex_return_notation_tl
5145 }
5146 \exp_args:No \exp_not:n \l__stex_expr_set_comp_tl
5147 }
5148 %}
5149 \exp_args:NNe \use:nn \group_end: {
5150 \_stex_next_symbol:n {
5151 \exp_args:No \exp_not:n \l__stex_expr_redo_tl
5152 \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
5153 \symuse{Metatheory?record-field}{
5154 \symuse{Metatheory?of~type}{
5155 \exp_args:No\tl_if_empty:nTF \l_stex_current_rec_tl {\_stex_annotate_force_br
5156 }{
5157 \tl_set:Nn \exp_not:N \l_stex_current_full_tl { \l_stex_current_full_tl}
5158 \__stex_expr_current_type:
5159 }
5160 }{
5161 \stex_annotate:nn{data-ftml-term=OML,data-ftml-head={\l__stex_expr_field_name_s
5162 }
5163 }
5164 \exp_args:No \exp_not:n \l__stex_expr_more_nextsymbol_tl
5165 }
5166 \exp_not:N \use_ii:nn
5167 \prop_item:Nn \l__stex_expr_prop {#1}
5168 }
5169 }{
5170 \msg_error:nnn{stex}{error/unknownfield}{#1}
5171 }
5172 }
5173
5174 \cs_new_protected:Nn \__stex_expr_make_prop: {
5175 \prop_clear:N \l__stex_expr_prop
5176 \seq_clear:N \l__stex_expr_seq
5177 \seq_clear:N \l__stex_expr_assigned_seq
5178 \tl_clear:N \l__stex_expr_redo_tl
5179 \__stex_expr_prop_do_decls:
5180 }
5181
5182 \cs_new_protected:Nn \__stex_expr_prop_do_decls: {
5183 \group_begin:
5184 \stex_debug:nn{expressions}{constructing-record}
5185 \seq_clear:N \l_stex_all_module_seq
5186 \exp_args:Ne \stex_activate_module:n {\clist_item:Nn \l_stex_current_type_tl 1}

```

```

5187 \exp_args:No \stex_iterate_symbols:nn \l_stex_current_type_tl {
5188   % uri, macro name, name, arity, args, type, def, return, macro
5189   \tl_if_empty:nTF{##2}{
5190     \exp_args:Nne\__stex_expr_do_decl_nomacro:nnnnnnnn{##3}
5191   }{
5192     \exp_args:Nne\__stex_expr_do_decl:nnnnnnnn{##2}
5193   }
5194   {\stex_new_symbol_uri:nn{##1}{##3}}{##3}{##4}{##5}{##6}{##7}{##8}##9
5195 }
5196 \exp_after:wN \tl_set:Nn \exp_after:wN \l__stex_expr_after_tl \exp_after:wN {
5197   \exp_after:wN \tl_set:Nn \exp_after:wN \l__stex_expr_prop \exp_after:wN { \l__stex_ex
5198 }
5199 \__stex_expr_prop_do_notations:
5200 \stex_debug:nn{expressions}{record:~\meaning\l__stex_expr_after_tl}
5201 \tl_put_right:Ne \l__stex_expr_after_tl {\tl_set:Nn \exp_not:N \l__stex_expr_redo_tl {\
5202 \exp_after:wN \group_end: \l__stex_expr_after_tl
5203 }
5204
5205 % id, uri, name, arity, args, type, def, return, macro
5206 \cs_new_protected:Nn \__stex_expr_do_decl_nomacro:nnnnnnnn {
5207   \prop_if_in:NnF \l__stex_expr_prop {#1} {
5208     \seq_put_left:Ne \l__stex_expr_seq {\tl_to_str:n{#2}}
5209     \prop_put:Nnn \l__stex_expr_prop {#1}{
5210       {}{
5211         \stex_invoke_symbol:nnnnnnN
5212         {#2}
5213         {#4}
5214         {#5}{#6}{#7}{#8}#9
5215       }
5216     }
5217   }
5218 }
5219
5220 \cs_new_protected:Nn \__stex_expr_do_decl:nnnnnnnn {
5221   \prop_if_in:NnF \l__stex_expr_prop {#1} {
5222     \seq_put_left:Ne \l__stex_expr_seq {\tl_to_str:n{#2}}
5223     \prop_put:Nnn \l__stex_expr_prop {#1}{
5224       {#3}{
5225         \stex_invoke_symbol:nnnnnnN
5226         {#2}
5227         {#4}
5228         {#5}{#6}{#7}{#8}#9
5229       }
5230     }
5231   }
5232 }
5233
5234 \cs_new_protected:Nn \__stex_expr_make_prop_assign: {
5235   \clist_if_empty:NF \l__stex_expr_fields_clist {
5236     \clist_map_inline:Nn \l__stex_expr_fields_clist {
5237       \__stex_expr_make_prop_assign:nn ##1
5238     }
5239   }
5240 }

```

```

5241
5242 \cs_new_protected:Nn \__stex_expr_make_prop_assign:nn {
5243   \prop_if_in:NnTF \l__stex_expr_prop {#1}{
5244     \exp_args:NNe \seq_put_right:Nn \l__stex_expr_assigned_seq {\tl_to_str:n{#1}}
5245     \exp_args:Nne \use:nn {\__stex_expr_make_prop_assign_replace:nnnn {#1}{#2}}
5246     {\prop_item:Nn \l__stex_expr_prop {#1}}
5247   }{
5248     \msg_error:nnn{stex}{error/unknownfieldass}{#1}
5249   }
5250 }
5251 \cs_new_protected:Nn \__stex_expr_make_prop_assign_replace:nnnn {
5252   \prop_put:Nnn \l__stex_expr_prop {#1}{#{3}{#2}}
5253   \tl_if_empty:nF{#3}{
5254     \tl_set:cn{#1}{ #2 }
5255     \tl_put_right:Nn \l__stex_expr_redo_tl {
5256       \tl_set:cn{#1}{ #2 }
5257     }
5258   }
5259 }
5260
5261 \cs_new_protected:Nn \__stex_expr_prop_do_notations: {
5262   \exp_args:No \stex_iterate_notations:nn \l_stex_current_type_tl {
5263     \exp_args:NNe \seq_if_in:NnTF \l__stex_expr_seq {\tl_to_str:n{##1}}{
5264       \tl_set:Nn \l_tmpa_tl {
5265         \cs_if_exist:cF{l_stex_notation_ \stex_use_symbol_uri:n{##1}??#2_cs}{
5266           \tl_set:cn{l_stex_notation_ \stex_use_symbol_uri:n{##1}??#2_cs}{##4}
5267         }
5268         \cs_if_exist:cF{l_stex_notation_ \stex_use_symbol_uri:n{##1}?_cs}{
5269           \tl_set:cn{l_stex_notation_ \stex_use_symbol_uri:n{##1}?_cs}{##4}
5270         }
5271       }
5272       \exp_args:NNo \tl_put_right:Nn \l__stex_expr_redo_tl \l_tmpa_tl
5273       \exp_args:NNo \tl_put_right:Nn \l__stex_expr_after_tl \l_tmpa_tl
5274       \tl_if_empty:nF{##5}{
5275         \tl_set:Nn \l_tmpa_tl {
5276           \cs_if_exist:cF{l_stex_notation_ \stex_use_symbol_uri:n{##1}??#2_op_cs}{
5277             \tl_set:cn{l_stex_notation_ \stex_use_symbol_uri:n{##1}??#2_op_cs}{##5}
5278           }
5279           \cs_if_exist:cF{l_stex_notation_ \stex_use_symbol_uri:n{##1}?_op_cs}{
5280             \tl_set:cn{l_stex_notation_ \stex_use_symbol_uri:n{##1}?_op_cs}{##5}
5281           }
5282         }
5283         \exp_args:NNo \tl_put_right:Nn \l__stex_expr_redo_tl \l_tmpa_tl
5284         \exp_args:NNo \tl_put_right:Nn \l__stex_expr_after_tl \l_tmpa_tl
5285       }
5286     }
5287   }
5288 }

```

(End of definition for `\stex_invoke_structure:`. This function is documented on page 150.)

sfunction

`\_stex_invoke_variable:nnnnnn`

```

5289 \cs_new_protected:Nn \_stex_invoke_variable:nnnnnnN {

```

```

5290 \bool_if:NTF \l_stex_allow_semantic_bool{
5291   \group_begin:
5292   \tl_set:Nn \l_stex_current_full_tl { #1 }
5293   \tl_set:Nn \l_stex_current_display_tl { #1 }
5294   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
5295   \__stex_expr_setup:nnnnn{\_varcomp}{#2}{#3}{#6}{#5}
5296   \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_omv:nn
5297   \tl_put_right:Nn \l_stex_current_redo_tl {
5298     \cs_set_eq:NN \_stex_term_oms_or_omv:nn \_stex_term_omv:nn
5299   }
5300   #7
5301 }{
5302   \msg_error:nnxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
5303 }
5304 }

```

(End of definition for `\_stex_invoke_variable:nnnnnn`. This function is documented on page 150.)

sfunction

`\svar`

```

5305 \NewDocumentCommand \svar {0{} m}{
5306   \group_begin:
5307   \tl_if_empty:nTF{#1}{
5308     \tl_set:Nn \l_stex_current_full_tl { #2 }
5309     \tl_set:Nn \l_stex_current_display_tl { #2 }
5310   }{
5311     \tl_set:Nn \l_stex_current_full_tl { #1 }
5312     \tl_set:Nn \l_stex_current_display_tl { #1 }
5313   }
5314   \bool_if:NTF \l_stex_allow_semantic_bool{
5315     \tl_clear:N \l_stex_current_term_tl
5316     \_stex_term_omv:nn{\\_varcomp{#2}}
5317   }{
5318     \msg_error:nnxx{stex}{error/notallowed}{Variable}{\l_stex_current_full_tl}
5319   }
5320   \group_end:
5321 }

```

(End of definition for `\svar`. This function is documented on page 150.)

sfunction

Now for the brunt of the work:

Math

Modifiers (! or *)?

```

5322 \cs_new_protected:Nn \__stex_expr_math: {
5323   \stex_debug:nn{expressions}{math-mode}
5324   \peek_charcode_remove:NTF ! {
5325     % operator
5326     \peek_charcode_remove:NTF * \__stex_expr_op_custom:n {
5327       % op notation
5328       \peek_charcode:NTF [ \__stex_expr_op_notation:w {
5329         \__stex_expr_op_notation:w []
5330       }
5331     }

```

```

5332 }{
5333   \peek_charcode_remove:NTF * \__stex_expr_custom:n {
5334     % normal
5335     \peek_charcode:NTF [ \__stex_expr_notation:w {
5336       \__stex_expr_notation:w []
5337     }
5338   }
5339 }
5340 }

```

Text

Operator?

```

5341 \cs_new_protected:Nn \__stex_expr_text: {
5342   \stex_debug:nn{expressions}{text~mode}
5343   \peek_charcode_remove:NTF ! \__stex_expr_op_custom:n \__stex_expr_custom:n
5344 }

```

Notation

Operator?

```

5345 \cs_new_protected:Npn \__stex_expr_notation:w [ #1 ] {
5346   \peek_charcode_remove:NTF ! {
5347     \__stex_expr_op_notation:w [#1]
5348   }{
5349     \__stex_expr_full_notation:n {#1}
5350   }
5351 }

```

(Possibly) complex notation taking arguments with optional particular identifier:

```

5352 \cs_new_protected:Nn \__stex_expr_full_notation:n {
5353   \stex_use_notation:nnTF \l_stex_current_full_tl {#1}{
5354     \stex_debug:nn{expressions}{using~notation~"#1"~for~\l_stex_current_full_tl}
5355     \tl_if_empty:NTF \l_stex_current_return_tl {
5356       \stex_debug:nn{expressions}{return~empty}
5357       \l_stex_notation_cs{\group_end:\stex_eat_exclamation_point:}
5358     }{
5359       \stex_debug:nn{expressions}{return?}
5360       \exp_after:wN \__stex_expr_maybe_return:n \exp_after:wN {
5361         \l_stex_notation_cs{
5362       }
5363     }
5364   }{
5365     \stex_debug:nn{expressions}{notation~"#1"~for~\l_stex_current_full_tl}~does~not~exist;
5366     \stex_do_default_notation:
5367     \tl_if_empty:NTF \l_stex_current_return_tl {
5368       \l_stex_default_notation{\group_end:\stex_eat_exclamation_point:}
5369     }{
5370       \exp_after:wN
5371       \__stex_expr_maybe_return:n
5372       \exp_after:wN
5373       {\l_stex_default_notation {}}
5374     }
5375   }
5376 }

```


Operator notation:

```

5377 \cs_new_protected:Npn \__stex_expr_op_notation:w [#1] {
5378   \stex_debug:nn{expressions}{op~notation~for~\l_stex_current_full_tl}
5379   \stex_use_op_notation:nnTF \l_stex_current_full_tl {#1}{
5380     \stex_maybe_brackets:nn{\neginfprec}{
5381       \stex_term_oms_or_omv:nn{#1}
5382       {\l_stex_notation_cs}
5383     }
5384     \group_end:
5385   }{
5386     \int_compare:nNnTF \l_stex_current_arity_str = 0 {
5387       \tl_clear:N \l_stex_current_return_tl
5388       \__stex_expr_notation:w [#1]
5389     }{
5390       \stex_do_default_notation_op:
5391       \stex_maybe_brackets:nn{\neginfprec}{
5392         \stex_term_oms_or_omv:nn{#1}
5393         {\l_stex_default_notation}
5394       }
5395       \group_end:
5396     }
5397   }
5398 }

```

Custom Notations

Operator:

```

5399 \cs_new_protected:Nn \__stex_expr_op_custom:n {
5400   \stex_debug:nn{expressions}{custom~op}
5401   \bool_set_true:N \l_stex_allow_semantic_bool
5402   \stex_term_oms_or_omv:nn{}{\maincomp{#1}}
5403   \group_end:
5404 }

```

Complex:

```

5405 \int_new:N \l__stex_expr_arg_counter_int
5406
5407 \cs_new_protected:Nn \__stex_expr_custom:n {
5408   \stex_debug:nn{custom}{custom~notation~for~\l_stex_current_full_tl}
5409   \stex_pseudogroup:nn{
5410     \bool_set_true:N \l_stex_allow_semantic_bool
5411     \prop_gclear:N \l__stex_expr_customs_prop
5412     \seq_gclear:N \l__stex_expr_customs_seq
5413     \int_gzero:N \l__stex_expr_arg_counter_int
5414     \tl_if_empty:NF \l_stex_current_args_tl {
5415       \exp_after:wN \__stex_expr_add_prop_arg:nnw \l_stex_current_args_tl \stex_args_end:
5416       \cs_set_eq:NN \arg \__stex_expr_arg:n
5417     }
5418     \tl_set_eq:NN \l_stex_get_symbol_args_tl \l_stex_current_args_tl
5419     \cs_set_eq:NN \__stex_expr_do_ab_next:nn \stex_term_oma:nn
5420     \stex_map_args:N \__stex_expr_check_b:nn
5421     \__stex_expr_do_ab_next:nn{}{#1}
5422   }{
5423     \prop_if_exist:NT \l__stex_expr_customs_prop {

```

```

5424     \prop_gset_from_keyval:Nn \exp_not:N \l__stex_expr_customs_prop {
5425     \prop_to_keyval:N \l__stex_expr_customs_prop
5426   }
5427 }
5428 \int_gset:Nn \l__stex_expr_arg_counter_int { \int_use:N \l__stex_expr_arg_counter_int}
5429 \seq_if_exist:NT \l__stex_expr_customs_seq {
5430   \seq_gset_split:Nnn \exp_not:N \l__stex_expr_customs_seq , {
5431     \seq_use:Nn \l__stex_expr_customs_seq ,
5432   }
5433 }
5434 }
5435 % TODO check that all arguments are present
5436 \group_end:
5437 }
5438
5439 \cs_new_protected:Npn \__stex_expr_add_prop_arg:nnw #1 #2 #3\stex_args_end: {
5440   \prop_gput:Nnn \l__stex_expr_customs_prop {#1} {}
5441   \seq_gput_right:Nn \l__stex_expr_customs_seq {#2}
5442   \tl_if_empty:nF{#3}{\__stex_expr_add_prop_arg:nnw #3 \stex_args_end:}
5443 }
5444
5445 \cs_new:Nn \__stex_expr_check_b:nn {
5446   \str_case:nn #2 {
5447     b {\cs_set_eq:NN \__stex_expr_do_ab_next:nn \stex_term_omb:nn}
5448     B {\cs_set_eq:NN \__stex_expr_do_ab_next:nn \stex_term_omb:nn}
5449   }
5450 }
5451
5452 Arguments:
5453 \NewDocumentCommand \__stex_expr_arg:n {s O{} m} {
5454   \IfBooleanTF #1 {
5455     \stex_annotate_invisible:n{
5456       \__stex_expr_arg_inner:nn{#2}{#3}
5457     }
5458   }{
5459     \__stex_expr_arg_inner:nn{#2}{#3}
5460   }
5461 }
5462
5463 \cs_new_protected:Nn \__stex_expr_arg_inner:nn {
5464   \tl_if_empty:nTF{#1}{
5465     \int_gincr:N \l__stex_expr_arg_counter_int
5466     \exp_args:Ne \__stex_expr_check:nTF{ \int_use:N \l__stex_expr_arg_counter_int }{
5467       \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl
5468     }{
5469       \__stex_expr_arg_inner:nn{}
5470     }{ #2 }
5471   }{
5472     \__stex_expr_check:nTF {#1}{
5473       \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl { #2 }
5474     }{
5475       \exp_args:No \str_case:nnTF \l_tmpb_tl {
5476         {a}{
5477           \exp_args:NNne \prop_gput:Nnn \l__stex_expr_customs_prop {#1}{
5478             \l_tmpa_tl X

```

```

5477     }
5478     \tl_set:Nx \l_tmpa_tl { #1 \int_eval:n {\tl_count:N \l_tmpa_tl + 1} }
5479   }
5480   {B}{
5481     \exp_args:NNne \prop_gput:Nnn \l__stex_expr_customs_prop {#1}{
5482       \l_tmpa_tl X
5483     }
5484     \tl_set:Ne \l_tmpa_tl { #1 \int_eval:n {\tl_count:N \l_tmpa_tl + 1} }
5485   }
5486 }{
5487   \__stex_expr_arg_do:oon \l_tmpa_tl \l_tmpb_tl { #2 }
5488 }{
5489   \msg_error:nxxx{stex}{error/invalidarg}{#1}{\l_stex_current_full_tl}
5490 }
5491 }
5492 }
5493 }
5494
5495 \prg_new_conditional:Nnn \__stex_expr_check:n {TF} {
5496   \exp_args:NNe \prop_get:NnNTF \l__stex_expr_customs_prop {#1} \l_tmpa_tl {
5497     \tl_set:Ne \l_tmpb_tl {\seq_item:Nn \l__stex_expr_customs_seq {#1} }
5498     \tl_if_empty:NNTF \l_tmpa_tl {
5499       \exp_args:NNe \prop_gput:Nnn \l__stex_expr_customs_prop
5500         { #1 }{X}
5501       \exp_args:No \str_case:nnF \l_tmpb_tl {
5502         {a}{
5503           \tl_set:Ne \l_tmpa_tl{ #1 1 }
5504         }
5505         {B}{
5506           \tl_set:Ne \l_tmpa_tl{ #1 1 }
5507         }
5508       }{
5509         \tl_set:Ne \l_tmpa_tl{ #1 }
5510       }
5511       \prg_return_true:
5512     }{
5513       \prg_return_false:
5514     }
5515   }{
5516     \msg_error:nxxx{stex}{error/invalidarg}{#1}{\l_stex_current_full_tl}
5517     \prg_return_false:
5518   }
5519 }
5520
5521 % #1 argnum #2 argmode #3 code
5522 \cs_new_protected:Nn \__stex_expr_arg_do:nnn {
5523   \stex_debug:nn{custom}{Doing~argument~#1~of~mode~#2:~\tl_to_str:n{#3}}
5524   \group_begin:
5525     \bool_set_true:N \l_stex_allow_semantic_bool
5526     \stex_term_arg:nnn {#2}{#1}{#3}
5527   \group_end:
5528 }
5529 \cs_generate_variant:Nn \__stex_expr_arg_do:nnn {oon}

```

Return code

```

5530 \cs_new_protected:Nn \__stex_expr_maybe_return:n {
5531   \tl_set:Nn \l__stex_expr_return_this_tl {#1}
5532   \tl_clear:N \l__stex_expr_return_args_tl
5533   \exp_args:Nne \use:n {
5534     \cs_generate_from_arg_count:NNnn \__stex_expr_ret_cs:
5535     \cs_set:Npn \l_stex_current_arity_str } {
5536       \int_step_function:nN \l_stex_current_arity_str \__stex_expr_return_arg:n
5537       \__stex_expr_return_next:
5538     }
5539   \__stex_expr_ret_cs:
5540 }
5541
5542 \cs_new:Nn \__stex_expr_return_arg:n {
5543   \tl_put_right:Nn \exp_not:N \l__stex_expr_return_args_tl {{## #1}}
5544 }
5545
5546 \cs_new_protected:Nn \__stex_expr_return_next: {
5547   \peek_charcode_remove:NTF ! {
5548     \exp_after:wN \l__stex_expr_return_this_tl \l__stex_expr_return_args_tl \group_end:
5549   }\__stex_expr_return:
5550 }
5551
5552 \cs_new_protected:Nn \__stex_expr_return: {
5553   \tl_set:Ne \l__stex_expr_return_this_tl {
5554     \tl_if_empty:NTF \l_stex_return_notation_tl {
5555       \exp_after:wN \exp_after:wN \exp_after:wN
5556       \exp_not:n \exp_after:wN \exp_after:wN \exp_after:wN {
5557         \exp_after:wN \l__stex_expr_return_this_tl \l__stex_expr_return_args_tl
5558       }
5559     }{
5560       \l_stex_return_notation_tl
5561     }
5562   }
5563   \stex_debug:nn{return}{Notation:~\meaning\l__stex_expr_return_this_tl}
5564
5565   \exp_after:wN
5566   \tl_put_left:Nn \exp_after:wN \l__stex_expr_return_this_tl \exp_after:wN {
5567     \exp_after:wN \group_begin: \l_stex_current_redo_tl
5568   }
5569   \exp_args:Nne \use:n {
5570     \cs_generate_from_arg_count:NNnn \__stex_expr_ret_cs:
5571     \cs_set:Npn \l_stex_current_arity_str } {
5572       \exp_args:No \exp_not:n \l_stex_current_return_tl
5573     }
5574   \stex_debug:nn{return}{
5575     \meaning\__stex_expr_ret_cs:^^J
5576     \meaning\l__stex_expr_return_this_tl^^J
5577     \exp_args:No \exp_not:n \l__stex_expr_return_args_tl^^J
5578   }
5579   \exp_args:Nne \use:nn {
5580     \exp_after:wN \group_end: \__stex_expr_ret_cs:
5581   }{
5582     \exp_args:No \exp_not:n \l__stex_expr_return_args_tl

```

```

5583     {
5584       \exp_args:No \exp_not:n \l__stex_expr_return_this_tl
5585       \group_end:
5586     }
5587   }
5588 }

```

32.4.1 Term Annotations

`\stex_eat_exclamation_point:`

```

5589 \cs_new_protected:Nn \stex_eat_exclamation_point: {
5590   \peek_charcode_remove:NT ! \stex_eat_exclamation_point:
5591 }

```

(End of definition for `\stex_eat_exclamation_point:`. This function is documented on page 150.)

sfunction

We occasionally allow for semantic macros where they otherwise shouldn't be, if we are in “invisible” parts:

```

5592 \bool_new:N \stex_in_invisible_html_bool

```

`\stex_term_oms:nn`

```

5593 \stex_if_html_backend:TF {
5594   \cs_new_protected:Nn \stex_term_oms:nn {
5595     \bool_if:NTF\l_stex_in_this_bool{#2}{
5596       \__stex_expr_check_comp:
5597       \tl_if_empty:NTF \l_stex_current_term_tl {
5598         \__stex_expr_annotate:nnn{OMID}{#1}{#2}
5599       }{
5600         \__stex_expr_do_headterm:nn{#1}{#2}
5601       }
5602     }
5603   }
5604 }{
5605   \cs_new_protected:Nn \stex_term_oms:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_chec
5606 }
5607
5608 \cs_set_eq:NN \stex_term_oms_or_omv:nn \stex_term_oms:nn

```

(End of definition for `\stex_term_oms:nn`. This function is documented on page 150.)

sfunction

`\stex_term_omv:nn`

```

5609 \stex_if_html_backend:TF {
5610   \cs_new_protected:Nn \stex_term_omv:nn {
5611     \bool_if:NTF\l_stex_in_this_bool{#2}{
5612       \__stex_expr_check_comp:
5613       \tl_if_empty:NTF \l_stex_current_term_tl {
5614         \__stex_expr_annotate:nnn{OMV}{#1}{#2}
5615       }{
5616         \__stex_expr_do_headterm:nn{#1}{#2}
5617       }
5618     }
5619   }

```

```

5620 }{
5621   \cs_new_protected:Nn \_stex_term_omv:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_check_comp:}
5622 }

```

(End of definition for `\_stex_term_omv:nn`. This function is documented on page 150.)

sfunction

In the case where the “symbol” (OMS or OMV) is the field of some structure or otherwise a complex expression, we have to insert the full term representing the operator:

```

5623 \stex_if_html_backend:TF {
5624   \cs_new_protected:Nn \__stex_expr_do_headterm:nn {
5625     \bool_if:NTF \stex_in_invisible_html_bool {
5626       {\bool_set_true:N \l_stex_allow_semantic_bool
5627        \ensuremath{\l_stex_current_term_tl}
5628       }
5629     }{
5630       \__stex_expr_annotate:nnn{complex}{#1}{
5631         \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5632           {\bool_set_true:N \l_stex_allow_semantic_bool
5633            \ensuremath{\l_stex_current_term_tl}
5634           }
5635         }
5636         #2
5637       }
5638     }
5639   }
5640 }{
5641   \cs_new_protected:Nn \__stex_expr_do_headterm:nn { #2 }
5642 }

```

`\_stex_term_oma:nn`

```

5643 \stex_if_html_backend:TF {
5644   \cs_new_protected:Nn \_stex_term_oma:nn {
5645     \bool_if:NTF\l_stex_in_this_bool{#2}{
5646       \__stex_expr_check_comp:
5647       \__stex_expr_annotate:nnn{OMA}{#1}{
5648         \tl_if_empty:NF \l_stex_current_term_tl {
5649           \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5650             \bool_set_true:N \l_stex_allow_semantic_bool
5651             \l_stex_current_term_tl
5652           }}
5653         }
5654         #2
5655       }
5656     }
5657   }
5658 }{
5659   \cs_new_protected:Nn \_stex_term_oma:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_check_comp:}
5660 }

```

(End of definition for `\_stex_term_oma:nn`. This function is documented on page 150.)

sfunction

`\_stex_term_omb:nn`

```

5661 \stex_if_html_backend:TF {

```

```

5662 \cs_new_protected:Nn \_stex_term_omb:nn {
5663   \bool_if:NTF\l_stex_in_this_bool{#2}{
5664     \__stex_expr_check_comp:
5665     \__stex_expr_annotate:nnn{OMBIND}{#1}{
5666       \tl_if_empty:NF \l_stex_current_term_tl {
5667         \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5668           \bool_set_true:N \l_stex_allow_semantic_bool
5669           \l_stex_current_term_tl
5670         }}
5671       }
5672     #2
5673   }
5674 }
5675 }
5676 }{
5677 \cs_new_protected:Nn \_stex_term_omb:nn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_che
5678 }

```

(End of definition for `\_stex_term_omb:nn`. This function is documented on page 150.)

sfunction

Does the actual annotating: **TODO: This shouldn't use `\l_stex_current_symbol_`**
`uri`!

```

5679 % kind, notation id, body
5680 \stex_if_html_backend:TF {
5681   \cs_new_protected:Nn \__stex_expr_annotate:nnn {
5682     \exp_args:Ne \stex_annotate:nn{
5683       data-ftml-term=#1,
5684       data-ftml-head={\l_stex_current_full_tl},
5685       data-ftml-notationid={#2},
5686     }{
5687       \_stex_annotate_force_break:n{#3}
5688     }
5689   }
5690 }{
5691   \cs_new_protected:Nn \__stex_expr_annotate:nnn { #3 }
5692 }
5693

```

32.4.2 Symbol Arguments

`\_stex_term_arg:nnnnn` #1 : argument number
 #2 : argument mode
 #3 : precedence
 #4 : argument name (WiP)
 #5 : code / body

```

5694 \cs_new_protected:Nn \_stex_term_arg:nnnnn {
5695   \group_begin:
5696     \tl_clear:N \l_stex_current_symbol_uri
5697     \tl_clear:N \l_stex_current_full_tl
5698     \tl_clear:N \l_stex_current_display_tl
5699     \tl_clear:N \l_stex_current_term_tl
5700     \int_set:Nn \l_stex_notation_downprec { #3 }
5701     \bool_set_true:N \l_stex_allow_semantic_bool

```

```

5702     \stex_term_arg:nnn {#2}{#1}{#5}
5703     \group_end:
5704 }

```

(End of definition for `\stex_term_arg:nnnnn`. This function is documented on page 151.)

sfunction

`\stex_term_arg:nnn`

```

5705 \cs_new_protected:Nn \__stex_expr_check_comp: {
5706   \bool_if:NT \g_stex_in_comp_bool {
5707     \msg_error:nn{stex}{error/argincomp}
5708   }
5709 }
5710 \stex_if_html_backend:TF {
5711   \cs_new_protected:Nn \stex_term_arg:nnn {
5712     \stex_if_do_html:TF{
5713       \bool_if:NTF\l_stex_in_this_bool{#3}{
5714         \stex_annotate_force_break:n{
5715           \stex_annotate:nn{
5716             data-ftml-arg={#2}, data-ftml-argmode={#1}
5717           }{
5718             \stex_annotate_force_break:n{
5719               \__stex_expr_check_comp:
5720               #3
5721             }
5722           }
5723         }{ #3 }
5724       }{ #3 }
5725     }{
5726     }{
5727       \cs_new_protected:Nn \stex_term_arg:nnn {\bool_if:NF\l_stex_in_this_bool\__stex_expr_che
5728     }

```

(End of definition for `\stex_term_arg:nnn`. This function is documented on page 151.)

sfunction

`\stex_term_arg_aB:nnnnn` The following macros fill up `\l_stex_aB_args_seq` and ultimately call `\stex_term_do_aB_clist:`. By default, this does:

```

5729 \cs_new:Nn \__stex_expr_do_aB_clist: {
5730   \stex_annotate_force_break:n{
5731     \seq_use:Nn \l_stex_aB_args_seq {
5732       \mathpunct{\comp{,}}
5733     }
5734   }
5735 }
5736
5737 \cs_set_eq:NN \stex_term_do_aB_clist: \__stex_expr_do_aB_clist:
5738
5739 \cs_new_protected:Nn \stex_is_sequentialized:n {
5740   \group_begin: #1 \group_end:
5741 }

```

Setup:

```

5742 \int_new:N \l__stex_expr_count_int
5743 \cs_new_protected:Nn \stex_term_arg_aB:nnnnn {

```



```

5744 \stex_debug:nn{sequences}{Processing~argument:~ \tl_to_str:n{#5}}
5745 \tl_if_empty:nTF{#5}{
5746   \_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{
5747   }{
5748     \seq_clear:N \l_stex_aB_args_seq
5749     \int_zero:N \l__stex_expr_count_int
5750     \clist_map_inline:nn{#5}{
5751       \__stex_expr_aB_arg:nnnnn{##1}{#1}{#2}{#3}{#4}
5752     }
5753     \_stex_term_do_aB_clist:
5754   }
5755 }

```

We “pattern match” the sequence argument - is it a comma-separated list? A sequence variable macro? `\argmap?` `\seqmap?`

```

5756 % 1: code 2: argnum 3: argmode 4: precedence 5: argname
5757 \cs_new_protected:Npn \__stex_expr_aB_arg:nnnnn #1 #2 #3 {
5758   \int_incr:N \l__stex_expr_count_int
5759   \stex_debug:nn{expressions}{matching~argument~ #2 #3:~ \tl_to_str:n{#1}}
5760   \__stex_expr_is_varseq:nTF{#1}{
5761     \stex_debug:nn{expressions}{=sequence~variable}
5762     \exp_after:wN \exp_after:wN \exp_after:wN
5763     \__stex_expr_assoc_seq:nnnnnnn
5764     \exp_after:wN
5765     \__stex_expr_gobble:nnnnnnnn #1 \__stex_expr_end:
5766   }{
5767     \__stex_expr_is_seqmap:nTF{#1}{
5768       \stex_debug:nn{expressions}{=seqmap}
5769       \exp_args:NNe \use:nn \__stex_expr_do_seqmap:nnnnnn {\tl_tail:n{#1}}
5770     }{
5771       \stex_debug:nn{expressions}{=simple}
5772       \__stex_expr_aB_simple_arg:nnnnn{#1}
5773     }
5774   }
5775   {#2}{#3}
5776 }
5777
5778 \cs_new:Npn \__stex_expr_gobble:nnnnnnnn #1 #2 #3 #4 #5 #6 #7 #8 #9 \__stex_expr_end: {
5779   {#2} #3 {#6}
5780 }

```

Is it a varseq?

```

5781 \prg_new_conditional:Nnn \__stex_expr_is_varseq:n {TF} {
5782   \int_compare:nNnTF {\tl_count:n{#1}} = 1 {
5783     \exp_args:Ne \cs_if_eq:NNTF {\exp_args:No \tl_head:n{#1}}
5784     \_stex_invoke_variable:nnnnnnN {
5785       \exp_args:Ne \cs_if_eq:NNTF {\exp_args:No\tl_item:nn{#1}{8}}
5786       \stex_invoke_sequence:
5787       \prg_return_true:\prg_return_false:
5788     }\prg_return_false:
5789   }\prg_return_false:
5790 }

```

Is it a seqmap?

```

5791 \prg_new_conditional:Nnn \__stex_expr_is_seqmap:n {TF} {

```

```

5792 \int_compare:nNnTF {\tl_count:n{#1}} = 3 {
5793   \exp_args:Ne \tl_if_eq:nnTF {\tl_head:n{#1}} {\seqmap}
5794   \prg_return_true:\prg_return_false:
5795 } \prg_return_false:
5796 }

```

Is it an argsep?

```

5797 \prg_new_conditional:Nnn \__stex_expr_is_argsep:n {TF} {
5798   \int_compare:nNnTF {\tl_count:n{#1}} = 3 {
5799     \exp_args:Ne \tl_if_eq:nnTF {\tl_head:n{#1}} {\argsep}
5800     \prg_return_true:\prg_return_false:
5801   } \prg_return_false:
5802 }

```

Simple argument:

```

5803 \cs_new_protected:Nn \__stex_expr_aB_simple_arg:nnnnn{
5804   \seq_put_right:Ne \l_stex_aB_args_seq {
5805     \stex_term_arg:nnnnn{#2\int_use:N\l__stex_expr_count_int}{#3}{#4}{#5}{
5806       \exp_not:n{
5807         \tl_set_eq:NN \stex_term_do_aB_clist: \__stex_expr_do_aB_clist:
5808         #1
5809       }
5810     }
5811   }
5812 }

```

Sequence variable:

```

5813 % 1: name 2: arity 3: clist 4: argnum 5: argmode 6: precedence 7: argname
5814 \cs_new_protected:Nn \__stex_expr_assoc_seq:nnnnnn {
5815   \group_begin:
5816     \seq_clear:N \l_stex_aB_args_seq
5817     \tl_set:Nn \l_stex_current_full_tl{#1}
5818     \tl_set:Nn \l_stex_current_display_tl{#1}
5819     \__stex_expr_assoc_make_seq:nnn{#1}{#3}{#2}
5820   \exp_args:NNe \use:nn \group_end: {
5821     \seq_put_right:Nn \exp_not:N \l_stex_aB_args_seq {
5822       \stex_is_sequentialized:n{
5823         \stex_term_arg:nnnnn{#4\int_use:N\l__stex_expr_count_int}{#5}{#6}{#7}{
5824           \bool_set_true:N \l_stex_allow_semantic_bool
5825           \tl_if_empty:NF \l_stex_current_term_tl {
5826             \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
5827               \exp_args:No \exp_not:n \l_stex_current_term_tl
5828             }
5829           }
5830         \stex_annotate:nn{
5831           data-ftml-term=OMV,
5832           data-ftml-head={#1},
5833           data-ftml-notationid={}
5834         }{
5835           \stex_annotate_force_break:n{
5836             \stex_term_do_aB_clist:
5837           }
5838         }
5839       }
5840     }

```

```

5841     }
5842   }
5843 }
5844
5845 % #1: name, #2: clist, #3:arity
5846 \cs_new_protected:Nn \__stex_expr_assoc_make_seq:nnn {
5847   \stex_use_notation:nnTF {#1}{}{
5848     \cs_set_eq:NN \l__stex_expr_cs \l_stex_notation_cs
5849   }{
5850     \stex_do_default_notation:
5851     \cs_set_eq:NN \l__stex_expr_cs \l_stex_default_notation
5852   }
5853   \clist_map_inline:nn{#2}{
5854     \tl_if_eq:nnTF{##1}{\ellipses}{
5855       \seq_put_right:Nn \l_stex_aB_args_seq { ##1 }
5856     }{
5857       \int_compare:nNnTF {#3} = 1 {
5858         \tl_set:Nn \l__stex_expr_iarg_tl { {##1} }
5859       }{
5860         \tl_set:Nn \l__stex_expr_iarg_tl { ##1 }
5861       }
5862       \seq_put_right:Ne \l_stex_aB_args_seq {
5863         \group_begin:
5864         \exp_not:n {
5865           \tl_set_eq:NN \stex_term_do_aB_clist: \__stex_expr_do_aB_clist:
5866           \def\comp{\varcomp}
5867           \tl_set:Nn \l_stex_current_full_tl{#1}
5868         }
5869         \exp_after:wN \exp_after:wN \exp_after:wN \exp_not:n
5870         \exp_after:wN \exp_after:wN \exp_after:wN {
5871           \exp_after:wN \l__stex_expr_cs \exp_after:wN \group_end: \l__stex_expr_iarg_tl
5872         }
5873       }
5874     }
5875   }
5876 }

```

seqmap:

```

5877 % 1: fun 2: clist 3: argnum 4: argmode 5: precedence 6: argname
5878 \cs_new_protected:Nn \__stex_expr_do_seqmap:nnnnnn {
5879   \group_begin:
5880   \cs_set:Npn \l_tmpa_cs ##1 {#1}
5881   \seq_clear:N \l_stex_aB_args_seq
5882   \clist_map_inline:nn{#2}{
5883     \__stex_expr_is_varseq:nnTF{##1}{
5884       \exp_after:wN
5885       \__stex_expr_varseq_in_map:nnnnnnnn ##1
5886     }{
5887       \seq_put_right:Nn \l_stex_aB_args_seq {##1}
5888     }
5889   }
5890   \seq_clear:N \l__stex_expr_old_seq
5891   \seq_map_inline:Nn \l_stex_aB_args_seq {
5892     \tl_if_eq:nnTF{##1}{\ellipses}{
5893       \seq_put_right:Nn \l__stex_expr_old_seq {##1}

```

```

5894     }
5895     % TODO \_stex_is_sequentialized:n
5896     {
5897       \exp_args:NNo \seq_put_right:Nn \l__stex_expr_old_seq {
5898         \l_tmpa_cs {##1}
5899       }
5900     }
5901   }
5902   \seq_set_eq:NN \l_stex_aB_args_seq \l__stex_expr_old_seq
5903   \tl_set:Ne \l_tmpa_tl {
5904     \symuse{Metatheory?bind}{
5905       \svar[MAP_DUMMY]{\exp_not:N\mathcal{m}}
5906     }{
5907       \exp_args:No\exp_not:n{\l_tmpa_cs{\svar[MAP_DUMMY]{\mathcal{m}}}}
5908     }
5909   }
5910   \exp_args:NNe \use:nn \group_end: {
5911     \seq_put_right:Nn \exp_not:N \l_stex_aB_args_seq {
5912       \_stex_is_sequentialized:n{
5913         \_stex_term_arg:nnnnn{#3\int_use:N\l__stex_expr_count_int}{#4}{#5}{#6}{
5914           \bool_set_true:N \l_stex_allow_semantic_bool
5915           \tl_set:Nn \exp_not:N \l_stex_current_full_tl
5916             {\l_stex_current_full_tl}
5917           \tl_set:Nn \exp_not:N \l_stex_current_term_tl {
5918             \exp_not:N \symuse{Metatheory?sequence-map}
5919             {\exp_not:n{#2}}
5920             {\exp_args:No \exp_not:n \l_tmpa_tl}
5921           }
5922           \__stex_expr_do_headterm:nn{}{
5923             \_stex_term_do_aB_clist:
5924           }
5925         }
5926       }
5927     }
5928   }
5929 }
5930
5931 \cs_new_protected:Nn \__stex_expr_varseq_in_map:nnnnnnnn {
5932   \__stex_expr_assoc_make_seq:nnn{#2}{#6}{#3}
5933 }
5934

```

(End of definition for _stex_term_arg_aB:nnnnn. This function is documented on page 151.)

sfunction

\seqmap

```

5935 \cs_new_protected:Npn \seqmap #1 #2 {
5936   \symuse{Metatheory?sequence-expression}{\seqmap{#1}{#2}}
5937   % \cs_set:Npn \l_tmpa_cs ##1 {#1}
5938   % \tl_set:Ne \l_tmpa_tl {
5939     % \symuse{Metatheory?bind}{
5940       % \svar[MAP_DUMMY]{\exp_not:N\mathcal{m}}
5941     % }{
5942       % \exp_args:No\exp_not:n{\l_tmpa_cs{\svar[MAP_DUMMY]{\mathcal{m}}}}

```

```

5943 %   }
5944 % }
5945 % \_stex_expr_annotate:nnn{complex}{OMS}{
5946 %   \stex_annotate_invisible:nn{data-ftml-headterm={}}{
5947 %     {\bool_set_true:N \l_stex_allow_semantic_bool
5948 %       \ensuremath{
5949 %         \exp_args:Nnno \symuse{Metatheory?sequence~map}{#2}\l_tmpa_tl
5950 %       }
5951 %     }
5952 %   }
5953 %   \symuse{Metatheory?sequence~expression}{\seqmap{#1}{#2}}
5954 % }
5955 }

```

(End of definition for `\seqmap`. This function is documented on page 151.)

sfunction

32.4.3 Notation components

```

5956 <@@=stex_comps>

```

`\comp`
`\compemph@uri`
`\compemph`

```

5957 \cs_set_protected:Npn \comp {}
5958
5959 \cs_new_protected:Npn \compemph@uri #1 #2 {
5960   \compemph{ #1 }
5961 }
5962
5963 \cs_new_protected:Npn \compemph #1 { #1 }
5964
5965 \cs_new_protected:Npn \_comp {
5966   \_do_comp:nnNn {comp}{ }\compemph@uri
5967 }
5968
5969 \cs_new_protected:Npn \_thiscomp #1 #2 {
5970   {\tl_set:cn{this}{ }}#1{#2}\c_math_subscript_token{
5971     \group_begin:
5972     \bool_set_true:N \l_stex_allow_semantic_bool
5973     \bool_set_true:N \l_stex_in_this_bool
5974     \l_stex_struct_this_tl
5975     \group_end:
5976   }
5977 }
5978 }
5979
5980 \def\maincomp{\comp}

```

(End of definition for `\comp`, `\compemph@uri`, and `\compemph`. These functions are documented on page 151.)

sfunction

`\varemp@uri`
`\varemp`

```

5981 \cs_new_protected:Npn \varemp@uri #1 #2 {
5982   \varemp{#1}
5983 }

```

```

5984
5985 \cs_new_protected:Npn \varemp #1 { #1 }
5986
5987 \cs_new_protected:Npn \_varcomp {
5988   \_do_comp:nnNn {comp}{ }\varemp@uri
5989 }

```

(End of definition for \varemp@uri and \varemp. These functions are documented on page 151.)

sfunction

```

\defemph@uri
\defemph
5990 \cs_new_protected:Npn \defemph@uri #1 #2 {
5991   \defemph{#1}
5992 }
5993
5994 \cs_new_protected:Npn \defemph #1 {
5995   \ifmmode\else\expandafter\textbf\fi{#1}
5996 }
5997
5998 \cs_new_protected:Npn \_defcomp {
5999   \_do_comp:nnNn {defcomp}{ }\defemph@uri
6000 }

```

(End of definition for \defemph@uri and \defemph. These functions are documented on page 151.)

sfunction

```

\symrefemph@uri
\symrefemph
6001 \cs_new_protected:Npn \symrefemph@uri #1 #2 {
6002   \symrefemph{#1}
6003 }
6004
6005 \cs_new_protected:Npn \symrefemph #1 { \emph{#1} }

```

(End of definition for \symrefemph@uri and \symrefemph. These functions are documented on page 151.)

sfunction

The following commands do the actual attributes:

```

\_do_comp:nnNn
6006 \bool_new:N \g_stex_in_comp_bool
6007
6008 \cs_new_protected:Nn \_do_comp:nnNn {
6009   \stex_pseudogroup_with:nn{\comp}{
6010     \def\comp##1{##1}
6011     \bool_set_true:N \g_stex_in_comp_bool
6012     \stex_if_html_backend:TF {
6013       \stex_annotate:nn { data-ftml-#1 = {#2}}{ #4 }
6014     }{
6015       \tl_if_empty:NTF \l_stex_current_full_tl {
6016         #4
6017       }{
6018         #3 { #4 } \l_stex_current_full_tl
6019       }
6020     }

```

```

6021     }
6022     \bool_set_false:N \g_stex_in_comp_bool
6023   }
6024 }

```

(End of definition for `\_do_comp:nnNn`. This function is documented on page 151.)

sfunction

32.4.4 Symbol References

Keys:

```

6025 \stex_keys_define:nnnn{symname}{
6026   \tl_clear:N \l_stex_key_pre_tl
6027   \tl_clear:N \l_stex_key_post_tl
6028   %\tl_clear:N \l_stex_key_root_tl
6029 }{
6030   pre      .tl_set:N   = \l_stex_key_pre_tl ,
6031   post     .tl_set:N   = \l_stex_key_post_tl ,
6032   %root     .code:n     = {}%tl_set:N   = \l_stex_key_root_tl
6033 }{}
6034
6035 \stex_keys_define:nnnn{symref}{
6036   %\tl_clear:N \l_stex_key_root_tl
6037 }{
6038   root     .code:n     = {}%tl_set:N   = \l_stex_key_root_tl
6039 }{}

```

`\symref`

`\sr`

```

6040 \NewDocumentCommand \symref { 0{ } m m } {
6041   \group_begin:
6042   \stex_keys_set:nn{symname}{#1}
6043   \stex_get_symbol:n{#2}
6044   \__stex_comps_do_ref:nNn{#3}\symrefemph@uri\stex_term_oms:nn
6045 }
6046 \let\sr\symref

```

(End of definition for `\symref` and `\sr`. These functions are documented on page 152.)

sfunction

`\symname`

`\sn`

`\sns`

`\Symname`

`\Sn\Sns`

```

6047 \cs_new:Nn \__stex_comps_split_slash:n {
6048   \__stex_comps_split_slash_i:nw #1//
6049 }
6050
6051 \cs_new:Npn \__stex_comps_split_slash_i:nw #1/#2/ {
6052   \tl_if_empty:nTF{#2}{#1}{
6053     \__stex_comps_split_slash_i:nw #2/
6054   }
6055 }
6056
6057 \NewDocumentCommand \symname { 0{ } m } {
6058   \group_begin:
6059   \stex_keys_set:nn{symname}{#1}
6060   \stex_get_symbol:n{#2}

```

```

6061 \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
6062 \tl_set:Ne \l_stex_current_display_tl {
6063   \exp_args:Ne \__stex_comps_split_slash:n {\stex_symbol_uri_name:N \l_stex_get_symbol_uri}
6064 }
6065 \__stex_comps_do_ref:nNn{
6066   \l_stex_key_pre_tl \l_stex_current_display_tl \l_stex_key_post_tl
6067 }\symrefemph@uri\stex_term_oms:nn
6068 }
6069 \let\sn\symname
6070 \protected\def\sns{\symname[post=s]}
6071
6072 \cs_new:Nn \__stex_comps_uppercase:n { \uppercase{#1}}
6073
6074 \NewDocumentCommand \Symname { 0{} m} {
6075   \group_begin:
6076   \stex_keys_set:nn{symname}{#1}
6077   \stex_get_symbol:n{#2}
6078   \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
6079   \tl_set:Ne \l_stex_current_display_tl {
6080     \exp_args:Ne \__stex_comps_split_slash:n {\stex_symbol_uri_name:N \l_stex_get_symbol_uri}
6081   }
6082   \__stex_comps_do_ref:nNn{
6083     \l_stex_key_pre_tl \exp_args:NNe \use:nn \__stex_comps_uppercase:n \l_stex_current_display_tl
6084   }\symrefemph@uri\stex_term_oms:nn
6085 }
6086 \let\Sn\Symname
6087 \protected\def\Sns{\Symname[post=s]}

```

(End of definition for \symname and others. These functions are documented on page 152.)

sfunction

\varref

\varname

\Varname

```

6088 \NewDocumentCommand \varref { 0{} m m} {
6089   \group_begin:
6090   \stex_keys_set:nn{symname}{#1}
6091   \stex_get_var:n{#2}
6092   \__stex_comps_do_ref:nNn{#3}\varemp@uri{
6093     \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
6094     \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
6095     \def\comp{\_varcomp}
6096     \_stex_term_omv:nn
6097   }
6098 }
6099
6100 \NewDocumentCommand \varname { 0{} m} {
6101   \group_begin:
6102   \stex_keys_set:nn{symname}{#1}
6103   \stex_get_var:n{#2}
6104   \__stex_comps_do_ref:nNn{
6105     \l_stex_key_pre_tl\l_stex_get_variable_str\l_stex_key_post_tl
6106   }\varemp@uri{
6107     \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
6108     \tl_set:Nn \l_stex_current_display_tl {\l_stex_get_variable_str}
6109     \def\comp{\_varcomp}

```



```

6110     \stex_term_omv:nn
6111   }
6112 }
6113
6114 \NewDocumentCommand \Varname { 0{} m } {
6115   \group_begin:
6116   \stex_keys_set:nn{symname}{#1}
6117   \stex_get_var:n{#2}
6118   \__stex_comps_do_ref:nNn{
6119     \l_stex_key_pre_tl\exp_after:wN\__stex_comps_uppercase:n\l_stex_get_variable_str\l_stex
6120   }\vareph@uri{
6121     \tl_set:Nn \l_stex_current_full_tl { \l_stex_get_variable_str }
6122     \tl_set:Nn \l_stex_current_display_tl { \l_stex_get_variable_str }
6123     \def\comp{\_varcomp}
6124     \stex_term_omv:nn
6125   }
6126 }

```

(End of definition for `\varref`, `\varname`, and `\Varname`. These functions are documented on page 152.)

sfunction

```

\definiendum
\definame
\Definame
\defnotation
6127 \stex_keys_define:nnnn{defname}{}{
6128   gf      .code:n      = {},
6129   root    .code:n      = {}
6130 }{symname}
6131
6132 \stex_keys_define:nnnn{definiendum}{}{
6133   gf      .code:n      = {},
6134   root    .code:n      = {}
6135 }{}
6136
6137 \NewDocumentCommand \definiendum { 0{} m m } {
6138   \stex_keys_set:nn{definiendum}{ #1 }
6139   \__stex_comps_do_defref:nn{#2}{#3}
6140 }
6141 \stex_deactivate_macro:Nn \definiendum {definition-environments}
6142
6143 \NewDocumentCommand \definame { 0{} m } {
6144   \stex_keys_set:nn{defname}{#1}
6145   \__stex_comps_do_defref:nn{#2}{
6146     \l_stex_key_pre_tl\l_stex_current_display_tl\l_stex_key_post_tl
6147   }
6148 }
6149 \protected\def\definames{\definame[post=s]}
6150 \stex_deactivate_macro:Nn \definame {definition-environments}
6151
6152
6153 \NewDocumentCommand \Definame { 0{} m } {
6154   \stex_keys_set:nn{defname}{#1}
6155   \__stex_comps_do_defref:nn{#2}{
6156     \l_stex_key_pre_tl \exp_args:NNe \use:nn \__stex_comps_uppercase:n \l_stex_current_disp
6157   }
6158 }

```

```

6159 \protected\def\Definames{\Definame[post=s]}
6160 \stex_deactivate_macro:Nn \Definame {definition-environments}
6161
6162
6163 \NewDocumentCommand \defnotation{ m } {
6164   \_stex_next_symbol:n { \def\comp{\_defcomp}}#1
6165 }
6166 \stex_deactivate_macro:Nn \defnotation {definition-environments}

```

(End of definition for \definiendum and others. These functions are documented on page 152.)

sfunction

Does the actual referencing:

```

6167 \cs_new_protected:Nn \_stex_comps_do_ref:nNn {
6168   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
6169   \bool_if:NTF \l_stex_allow_semantic_bool{
6170     \tl_set_eq:NN \l_stex_current_symbol_uri \l_stex_get_symbol_uri
6171     \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_current_symbol_uri
6172       %\str_set:Ne \l_stex_current_symbol_name_str { \stex_symbol_uri_name:N \l_stex_current
6173       %\str_if_in:NnT \l_stex_current_symbol_name_str / {
6174       % \str_set:Ne \l_stex_current_symbol_name_str {
6175       %   \exp_after:wN \_stex_comps_slash:w \l_stex_current_symbol_name_str
6176       %   /\_stex_args_end:
6177       % }
6178       %}
6179       \tl_clear:N \l_stex_current_term_tl
6180       \def\comp{\_comp}
6181       \let\compemph@uri#2
6182       #3{\_comp{#1}}
6183   }{
6184     \msg_error:nxxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}
6185   }
6186   \group_end:
6187 }

\definame et al:

6188 \cs_new_protected:Nn \_stex_comps_do_defref:nn {
6189   \stex_if_html_backend:T{\relax\ifvmode\indent\fi}
6190   \group_begin:
6191   \stex_get_symbol:n{#1}
6192   \bool_if:NTF \l_stex_allow_semantic_bool{
6193     \tl_set:Nn \l_stex_current_full_tl { \stex_use_symbol_uri:N \l_stex_get_symbol_uri }
6194     \tl_set:Nn \l_stex_current_display_tl {
6195       \stex_symbol_uri_name:N \l_stex_get_symbol_uri
6196     }
6197     %\str_if_in:NnT \l_stex_get_svariable_str / {
6198     % \str_set:Ne \l_stex_get_variable_str {
6199     %   \exp_after:wN \_stex_comps_slash:w \l_stex_get_variable_str
6200     %   /\_stex_args_end:
6201     % }
6202     %}
6203     \exp_args:Ne \stex_ref_new_sym_target:n \l_stex_current_full_tl
6204     \_do_comp:nnNn {definiendum}{\l_stex_current_full_tl}\defemph@uri{#2}
6205   }{
6206     \msg_error:nxxx{stex}{error/notallowed}{#1}{\l_stex_current_full_tl}

```

```

6207 }
6208 \group_end:
6209 }
6210
6211 \cs_new:Npn \__stex_comps_slash:w #1/#2/#3\_stex_args_end: {
6212   \tl_if_empty:nTF{#3}{
6213     #2
6214   }{
6215     \__stex_comps_slash:w #2 / #3 \_stex_args_end:
6216   }
6217 }

```

\foldexpr

```

6218 \stex_if_html_backend:TF {
6219   \NewDocumentCommand \foldexpr { 0{ } mm } {
6220     \stex_annotate:nn{
6221       data-ftml-fold-expression={\exp_args:Ne \str_if_eq:nnT{#1}{show}{show}}
6222     }{
6223       \stex_annotate:nn{
6224         data-ftml-fold-short={}
6225       }{#2}
6226       #3
6227     }
6228   }
6229 }{
6230   \NewDocumentCommand \foldexpr { 0{ } mm } {
6231     \exp_args:Ne \str_if_eq:nnTF{#1}{hide}{
6232       \underbrace{...}{#2}
6233     }{
6234       \underbrace{#3}{#2}
6235     }
6236   }
6237 }

```

(End of definition for \foldexpr. This function is documented on page 152.)
sfunction

32.5 Checking

```

\stex_if_check_terms_p:
\stex_if_check_terms:TF
6238 <@@=stex_check>
6239 \stex_if_html_backend:TF {
6240   \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
6241     \prg_return_false:
6242   }
6243 }{
6244   \stex_get_env:Nn\__stex_check_env_str{STEX_CHECKTERMS}
6245   \str_if_empty:NF\__stex_check_env_str{
6246     \exp_args:No \str_if_eq:nnF \__stex_check_env_str{false}{
6247       \bool_set_true:N \c_stex_check_terms_bool
6248     }
6249   }
6250   \bool_if:NTF \c_stex_check_terms_bool {

```

```

6251     \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
6252       \prg_return_true:
6253     }
6254   }{
6255     \prg_new_conditional:Nnn \stex_if_check_terms: {p, T, F, TF} {
6256       \prg_return_false:
6257     }
6258   }
6259 }

```

(End of definition for `\stex_if_check_terms:TF`. This function is documented on page 152.)

sfunction

`\stex_check_term:nn`

```

6260 \stex_if_check_terms:TF{
6261   \cs_new_protected:Nn \stex_check_term:nn {
6262     \marginpar {\tiny Checking~#1:~
6263       \group_begin:
6264         $#2$
6265       \group_end:
6266     }
6267   }
6268 }{
6269   \cs_new_protected:Nn \stex_check_term:nn {}
6270 }

```

(End of definition for `\stex_check_term:nn`. This function is documented on page 152.)

sfunction

`\_stex_check_terms:`

```

6271 \stex_if_check_terms:TF{
6272   \cs_new_protected:Nn \_stex_check_terms: {
6273     \stex_debug:nn{check_terms}{Checking~type...}
6274     \tl_if_empty:NF \l_stex_key_type_tl {
6275       \stex_check_term:nn{type}\l_stex_key_type_tl
6276     }
6277     \stex_debug:nn{check_terms}{Checking~definiens...}
6278     \tl_if_empty:NF \l_stex_key_def_tl {
6279       \stex_check_term:nn{definiens}\l_stex_key_def_tl
6280     }
6281     \stex_debug:nn{check_terms}{Checking~return...}
6282     \tl_if_empty:NF \l_stex_key_return_tl {
6283       \stex_check_term:nn{return}\l_stex_key_return_tl!}
6284     }
6285     \stex_debug:nn{check_terms}{Checking~argument~types...}
6286     \group_begin:\l_stex_key_argtypes_clist\group_end:
6287     \tl_if_empty:NF \l_stex_key_argtypes_clist {
6288       \stex_check_term:nn{argument~types}\l_stex_key_argtypes_clist}
6289     }
6290   }
6291 }{
6292   \cs_new_protected:Nn \_stex_check_terms: {}
6293 }
6294

```

(End of definition for \stex_check_terms:. This function is documented on page 152.)
sfunction

Chapter 33

Notations

```
6295 <@@=stex_notations>

      keys:
6296 \stex_keys_define:nnnn{notation}{
6297   \str_clear:N \l_stex_key_variant_str
6298   \str_clear:N \l_stex_key_prec_str
6299   \str_clear:N \l_stex_key_op_tl
6300   \str_clear:N \l_stex_key_intent_str
6301   \clist_clear:N \l_stex_key_intent_args_clist
6302 }{
6303   variant      .str_set_x:N = \l_stex_key_variant_str ,
6304   prec         .str_set_x:N = \l_stex_key_prec_str ,
6305   op           .tl_set:N     = \l_stex_key_op_tl ,
6306   intent       .str_set:N     = \l_stex_key_intent_str ,
6307   argnames     .clist_set:N   = \l_stex_key_intent_args_clist ,
6308   unknown      .code:n       = {
6309     \str_if_empty:NTF \l_keys_key_str {
6310       \str_set:Nc \l_stex_key_variant_str {\l_keys_key_tl}
6311     }{
6312       \str_set_eq:NN \l_stex_key_variant_str \l_keys_key_str
6313     }
6314   }
6315 }{style}
6316
6317 \stex_keys_define:nnnn{symdef}{ }{ }{decl,notation}
6318
6319 \stex_keys_define:nnnn{vardef}{
6320   \bool_set_false:N \l__stex_notations_bind_bool
6321 }{
6322   bind .bool_set:N = \l__stex_notations_bind_bool
6323 }{symdef}
6324
\notation

6325 \stex_new_stylable_cmd:nnnn {notation} { s m O{ } m } {
6326   \stex_keys_set:nn{notation}{#3}
6327   \stex_get_symbol:n{#2}
6328   \stex_notation_parse:n{#4}
6329   \stex_if_check_terms:T{ \_stex_notation_check: }
```

```

6330 \_stex_notation_add:
6331 \stex_if_do_html:T {
6332   \def\comp{\_comp}
6333   \_stex_notation_do_html:n{\stex_use_symbol_uri:N \l_stex_get_symbol_uri}
6334 }
6335 \IfBooleanTF#1{
6336   \_stex_notation_set_default:n{
6337     \l_stex_get_symbol_uri
6338   }
6339 }{}
6340 \stex_if_smsmode:F{
6341   \group_begin:
6342   \__stex_notations_styledefs:
6343   \stex_style_apply:
6344   \group_end:
6345 }
6346 \stex_smsmode_do:
6347 }{}
6348
6349 \stex_deactivate_macro:Nn \notation {module~environments}
6350 \stex_every_module:n {\stex_reactivate_macro:N \notation}
6351 \stex_sms_allow_escape:N \notation
6352
6353 \cs_new_protected:Nn \__stex_notations_styledefs: {
6354   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
6355   \str_set:Nn \thisdeclname {\stex_symbol_uri_name:N \l_stex_get_symbol_uri}
6356   \tl_set:Ne \thisdecluri {\stex_use_symbol_uri:N \l_stex_get_symbol_uri}
6357   \def\thisnotation{
6358     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}}{
6359     \_stex_notation_make_args:
6360   }
6361 }
6362 }

```

(End of definition for \notation. This function is documented on page 153.)

sfunction

\varnotation

```

6363 \stex_new_stylable_cmd:nnnn {\varnotation} { s m O{} m} {
6364   \stex_keys_set:nn{notation}{#3}
6365   \stex_get_var:n{#2}
6366   \str_set_eq:NN \l_stex_key_name_str \l_stex_get_variable_str
6367   \stex_notation_parse:n{#4}
6368   \stex_if_check_terms:T{ \_stex_notation_check: }
6369   \_stex_var_notation_macro:
6370   \IfBooleanTF#1{
6371     \_stex_notation_set_default:n{\l_stex_get_variable_str}
6372   }{}
6373   \group_begin:
6374   \tl_set_eq:NN \thisvarname \l_stex_get_variable_str
6375   \tl_clear:N \thisstyle
6376   \str_set_eq:NN\thisnotationvariant\l_stex_key_variant_str
6377   \def\thisnotation{
6378     \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}}{

```

```

6379     \_stex_notation_make_args:
6380   }
6381 }
6382 \stex_style_apply:
6383 \group_end:
6384 }{}

```

(End of definition for \varnotation. This function is documented on page 153.)

sfunction

\stex_notation_parse:n

```

6385 \cs_new_protected:Nn \stex_notation_parse:n {
6386   \tl_if_empty:NF \l_stex_key_op_tl {
6387     \tl_set:Nc \l_stex_key_op_tl { \exp_not:N\maincomp {
6388       \exp_args:No \exp_not:n \l_stex_key_op_tl
6389     } }
6390   }
6391   \seq_clear:N \l__stex_notations_precs_seq
6392   \tl_clear:N \l_stex_notation_args_tl
6393   \int_compare:nNnTF \l_stex_get_symbol_arity_int = 0 {
6394     \__stex_notations_const_precs:
6395     \tl_if_empty:NT \l_stex_key_op_tl {
6396       \tl_set:Nn \l_stex_key_op_tl { \maincomp{#1} }
6397     }
6398   }{
6399     \__stex_notations_fun_precs:
6400     \__stex_notations_do_missing_args:n{#1}
6401     \tl_if_empty:NT \l_stex_key_op_tl {
6402       \hbox_set:Nn \l_tmpa_box {
6403         \tl_clear:N \l_stex_current_full_tl
6404         \tl_clear:N \l_stex_current_display_tl
6405         \cs_set:Npn \l_tmpa_cs ##1 ##2 ##3 ##4 ##5 ##6 ##7 ##8 ##9 { #1 }
6406         \cs_set:Npn \maincomp ##1 {
6407           \tl_gset:Nn \l_stex_key_op_tl { \maincomp{##1} }
6408           ##1
6409         }
6410         \cs_set:Npn \argsep ##1 ##2 {##1 ##2}
6411         \cs_set:Npn \argmap ##1 ##2 ##3 {##1 ##3}
6412         \cs_set:Npn \argarraymap ##1 ##2 ##3 ##4 {
6413           ##1 ##2
6414         }
6415         \stex_suppress_html:n{ $\l_tmpa_cs abcdefghj$ }
6416       }
6417     }
6418   }
6419   \exp_args:NNe
6420   \tl_set:Nn \l_stex_notation_macrocode_cs {
6421     \STEXInternalNotation
6422     { \l_stex_key_variant_str }
6423     { \l__stex_notations_opprec_tl }
6424     { \l_stex_key_intent_str }
6425     { \l_stex_notation_args_tl }
6426     {
6427       \int_compare:nNnTF \l_stex_get_symbol_arity_int = 0

```



```

6428         { \exp_not:n { \maincomp{ #1 } } }
6429         { \exp_not:n { #1 } \l__stex_notations_missing_tl }
6430     }
6431 }
6432 \stex_debug:nn{notation}{Notation:~\meaning\l_stex_notation_macrocode_cs}
6433 }

precedences:
6434 \cs_new_protected:Nn \__stex_notations_const_precs: {
6435     \str_if_empty:NTF \l_stex_key_prec_str {
6436         \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6437     }{
6438         \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
6439             \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6440         }{
6441             \tl_set_eq:NN \l__stex_notations_opprec_tl \l_stex_key_prec_str
6442         }
6443     }
6444 }
6445
6446 \cs_new_protected:Nn \__stex_notations_fun_precs: {
6447     \str_if_empty:NTF \l_stex_key_prec_str {
6448         \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6449     }{
6450         \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
6451             \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6452         }{
6453             \tl_set_eq:NN \l__stex_notations_opprec_tl \l_stex_key_prec_str
6454         }
6455     }
6456     \str_if_empty:NTF \l_stex_key_prec_str {
6457         \tl_set:Nn \l__stex_notations_opprec_tl { 0 }
6458         \int_step_inline:nn \l_stex_get_symbol_arity_int {
6459             \seq_put_right:Nn \l__stex_notations_precs_seq {0}
6460         }
6461     }{
6462         \str_if_eq:onTF \l_stex_key_prec_str {nobrackets}{
6463             \stex_debug:nn{notation}{No~brackets}
6464             \tl_set:No \l__stex_notations_opprec_tl { \neginfprec }
6465             \int_step_inline:nn \l_stex_get_symbol_arity_int {
6466                 \exp_args:NNo \seq_put_right:Nn \l__stex_notations_precs_seq \infprec
6467             }
6468         } \__stex_notations_parse_precs:
6469     }
6470     \__stex_notations_do_argnames:
6471 }
6472
6473 \cs_new_protected:Nn \__stex_notations_parse_precs: {
6474     \stex_debug:nn{notation}{parsing~precedence~\l_stex_key_prec_str}
6475     \seq_set_split:NnV \l__stex_notations_seq ; \l_stex_key_prec_str
6476     \seq_pop_left:NNTF \l__stex_notations_seq \l__stex_notations_str {
6477         \tl_set_eq:NN \l__stex_notations_opprec_tl \l__stex_notations_str
6478         \seq_pop_left:NNT \l__stex_notations_seq \l__stex_notations_str {
6479             \exp_args:NNo \seq_set_split:NnV \l__stex_notations_seq
6480                 {\tl_to_str:n{x}} \l__stex_notations_str

```

```

6481     }
6482   }{
6483     \tl_set:No \l__stex_notations_opprec_tl { 0 }
6484   }
6485   \int_step_inline:nn \l_stex_get_symbol_arity_int {
6486     \seq_pop_left:NNTF \l__stex_notations_seq \l__stex_notations_str {
6487       \seq_put_right:No \l__stex_notations_precs_seq \l__stex_notations_str
6488     }{
6489       \seq_put_right:No \l__stex_notations_precs_seq \l__stex_notations_opprec_tl
6490     }
6491   }
6492 }

```

Experimental; named arguments:

```

6493 \cs_new_protected:Nn \__stex_notations_do_argnames: {
6494   \tl_clear:N \l_stex_notation_args_tl
6495   \stex_map_args:N \__stex_notations_do_argname:nn
6496 }
6497
6498 \cs_new_protected:Nn \__stex_notations_do_argname:nn {
6499   \clist_if_empty:NNTF \l_stex_key_intent_args_clist {
6500     \tl_put_right:Ne \l_stex_notation_args_tl {
6501       #1#2{\seq_item:Nn \l__stex_notations_precs_seq #1}{
6502         \str_if_empty:NF \l_stex_key_intent_str {#1}
6503       }
6504     }
6505   }{
6506     \tl_put_right:Ne \l_stex_notation_args_tl {
6507       #1#2{\seq_item:Nn \l__stex_notations_precs_seq #1}
6508       {\c_dollar_str\clist_item:Nn \l_stex_key_intent_args_clist 1}
6509     }
6510     \clist_pop:NN \l_stex_key_intent_args_clist \l_tmpa_tl
6511   }
6512 }

```

inserts hidden arguments that don't occur in the body of the notation:

```

6513 \cs_new:Nn \__stex_notations_do_missing_args:n {
6514   \str_set:Nn \l__stex_notations_missing_str {#1}
6515   \exp_args:NNe \seq_set_split:NnV \l_tmpa_seq {\c_hash_str\c_hash_str\c_hash_str\c_hash_str}
6516   \str_clear:N \l__stex_notations_missing_str
6517   \seq_map_inline:Nn \l_tmpa_seq {
6518     \tl_put_right:Nn \l__stex_notations_missing_str { ##1 }
6519   }
6520   \tl_clear:N \l__stex_notations_missing_tl
6521   \stex_map_args:N \__stex_notations_add_missing_args:nn
6522 }
6523
6524 \cs_new:Nn \__stex_notations_add_missing_args:nn {
6525   % TODO this skips arguments if e.g. ##1 occurs rather than #1!!
6526   \exp_args:NNe \str_if_in:NnF \l__stex_notations_missing_str {\c_hash_str\c_hash_str#1}{
6527     \tl_put_right:Nn \l__stex_notations_missing_tl{\STEXinvisible{## #1}}
6528   }
6529 }

```

(End of definition for `\stex_notation_parse:n`. This function is documented on page 153.)

sfunction

_stex_notation_add:

```

6530 \cs_new_protected:Nn \_stex_notation_add: {
6531   \stex_add_notation:ooeoo
6532   {\l_stex_get_symbol_uri}
6533   \l_stex_key_variant_str
6534   {\int_use:N \l_stex_get_symbol_arity_int}
6535   \l_stex_notation_macrocode_cs
6536   \l_stex_key_op_tl
6537 }

```

(End of definition for _stex_notation_add:. This function is documented on page 153.)

sfunction

\stex_add_notation:nnnnn

\stex_add_notation:ooeoo

```

6538 \cs_new:Nn \__stex_notations_macro:nn {
6539   l_stex_notation_ #1?#2 _cs
6540 }
6541 \cs_new:Nn \__stex_notations_op_macro:nn {
6542   l_stex_notation_ #1?#2 _op_cs
6543 }
6544
6545 \cs_new_protected:Nn \stex_add_notation:nnnnn {
6546   \exp_args:Nne \stex_debug:nn{notations}{Adding~notation:^^J
6547     #1~\c_hash_str#2~#3^^J
6548     \tl_to_str:n{#4}^^J\tl_to_str:n{#5}^^J
6549     to~\stex_use_module_uri:N \l_stex_current_module_uri
6550   }
6551   \prop_gput:cen{ \_stex_notations_macro:N \l_stex_current_module_uri }
6552   {\stex_use_symbol_uri:n{#1}?#2}{\{#1\}{#2\}{#3\}{#4\}{#5}}
6553   \stex_do_up_to_module:n {
6554     \__stex_notations_set_macro:nnnnn{#1}{#2}{#3}{#4}{#5}
6555     %\__stex_notations_activate_not:nn{#1}{#2}
6556   }
6557 }
6558 \cs_generate_variant:Nn \stex_add_notation:nnnnn {ooeoo}
6559
6560 \cs_new_protected:Nn \_stex_activate_notations: {
6561   \stex_debug:nn{activating}{All~notations:~\cs_meaning:c{ \_stex_notations_macro:N \l_stex
6562   \prop_map_inline:cn{ \_stex_notations_macro:N \l_stex_current_module_uri }{
6563     \__stex_notations_set_macro:nnnnn ##2
6564   }
6565 }
6566
6567 \cs_new_protected:Nn \__stex_notations_activate_not:nn {
6568   \bool_set_false:N \l_tmpa_bool
6569   \stex_debug:nn{notations}{activating~\stex_use_symbol_uri:n{#1}?#2}
6570   \prop_map_inline:cn{ \_stex_notations_macro:N \l_stex_current_module_uri }{
6571     %\stex_debug:nn{notations}{##1?}
6572     \exp_args:Ne \str_if_eq:nnT{\stex_use_symbol_uri:n{#1}?#2}{##1}{
6573       \prop_map_break:n{
6574         %\stex_debug:nn{notations}{Found:~\tl_to_str:n{##2}}
6575         \bool_set_true:N \l_tmpa_bool

```

```

6576         \_stex_notations_set_macro:nnnnn ##2
6577     }
6578 }
6579 }
6580 \bool_if:NF \l_tmpa_bool {
6581     \errmessage{Woop}
6582 }
6583 }
6584
6585 \cs_new_protected:Nn \_stex_notations_set_macro:nnnnn {
6586     \tl_set:cn {\_stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{#2}}{#4}
6587     \cs_if_exist:cF{\_stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{
6588         \tl_set:cn {\_stex_notations_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{#4}
6589     }
6590     \tl_if_empty:nF{#5}{
6591         \tl_set:cn{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{#2}}{#5}
6592         \cs_if_exist:cF{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{
6593             \tl_set:cn{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:n{#1}}{}}{#5}
6594         }
6595     }
6596 }

```

(End of definition for `\stex_add_notation:nnnnn`. This function is documented on page 153.)

sfunction

_stex_map_args:N
_stex_map_notation_args:N

```

6597 \cs_new:Nn \_stex_map_args:N {
6598     \tl_if_empty:NF \l_stex_get_symbol_args_tl {
6599         \exp_after:wN \_stex_notations_map_args_i:w \exp_after:wN
6600         #1 \l_stex_get_symbol_args_tl \_stex_notations_args_end:
6601     }
6602 }
6603 \cs_new:Npn \_stex_notations_map_args_i:w #1 #2 #3 #4 \_stex_notations_args_end: {
6604     #1 {#2} {#3}
6605     \tl_if_empty:NF{#4}{
6606         \_stex_notations_map_args_i:w #1 #4 \_stex_notations_args_end:
6607     }
6608 }
6609
6610 \cs_new:Nn \_stex_map_notation_args:N {
6611     \tl_if_empty:NF \l_stex_notation_args_tl {
6612         \exp_after:wN \_stex_notations_map_args_ii:w \exp_after:wN
6613         #1 \l_stex_notation_args_tl \_stex_notations_args_end:
6614     }
6615 }
6616 \cs_new:Npn \_stex_notations_map_args_ii:w #1 #2 #3 #4 #5 #6 \_stex_notations_args_end: {
6617     #1 {#2} {#3} {#4} {#5}
6618     \tl_if_empty:NF{#6}{
6619         \_stex_notations_map_args_ii:w #1 #6 \_stex_notations_args_end:
6620     }
6621 }

```

(End of definition for `\_stex_map_args:N` and `\_stex_map_notation_args:N`. These functions are documented on page 153.)

sfunction

_stex_var_notation_macro:

```

6622 \cs_new_protected:Nn \_stex_var_notation_macro: {
6623   \tl_set_eq:cN {\_stex_notations_macro:nn{\l_stex_key_name_str}{\l_stex_key_variant_str}}
6624   \cs_if_exist:cF {\_stex_notations_macro:nn{\l_stex_key_name_str}{}}{
6625     \tl_set_eq:cN{\_stex_notations_macro:nn{\l_stex_key_name_str}{}}
6626     \l_stex_notation_macrocode_cs
6627   }
6628   \tl_if_empty:NF \l_stex_key_op_tl {
6629     \tl_set_eq:cN {\_stex_notations_op_macro:nn{\l_stex_key_name_str}{\l_stex_key_variant_
6630     \cs_if_exist:cF{\_stex_notations_op_macro:nn{\l_stex_key_name_str}{}}{
6631       \cs_set_eq:cN{\_stex_notations_op_macro:nn{\l_stex_key_name_str}{}}
6632       \l_stex_key_op_tl
6633     }
6634   }
6635 }

```

(End of definition for _stex_var_notation_macro:. This function is documented on page 153.)

sfunction

\setnotation

_stex_notation_set_default:n

```

6636 \cs_new_protected:Nn \_stex_notation_set_default:n{
6637   \stex_if_in_module:TF{
6638     \stex_add_notation:ooeo{#1}{
6639       {\int_use:N \l_stex_get_symbol_arity_int}
6640       \l_stex_notation_macrocode_cs
6641       \l_stex_key_op_tl
6642     }{
6643       \cs_set_eq:cN {\_stex_notations_macro:nn {\stex_use_symbol_uri:N \l_stex_get_symbol_ur
6644       \l_stex_notation_macrocode_cs
6645       \tl_if_empty:NF \l_stex_key_op_tl {
6646         \cs_set_eq:cN{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol
6647         \l_stex_key_op_tl
6648       }
6649     }
6650 }
6651
6652 \cs_new_protected:Npn \setnotation #1 #2 {
6653   \stex_get_symbol:n{#1}
6654   \cs_if_exist:cTF{\_stex_notations_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol_uri
6655   \tl_set_eq:Nc \l_stex_notation_macrocode_cs {\_stex_notations_macro:nn{\stex_use_symbol
6656   \cs_if_exist:cTF{\_stex_notations_op_macro:nn{\stex_use_symbol_uri:N \l_stex_get_symbol
6657     \tl_set_eq:Nc \l_stex_key_op_tl {\_stex_notations_op_macro:nn{\stex_use_symbol_uri:N
6658   }{
6659     \tl_clear:N \l_stex_key_op_tl
6660   }
6661   \_stex_notation_set_default:n{
6662     \l_stex_get_symbol_uri
6663   }
6664 }{
6665   \msg_error:nnxx{stex}{error/unknownnotation}{#2}{
6666     \stex_use_symbol_uri:N \l_stex_get_symbol_uri
6667   }
6668 }
6669 }

```

(End of definition for `\setnotation` and `\_stex_notation_set_default:n`. These functions are documented on page 153.)

sfunction

`\stex_iterate_notations:nn`

```

6670 \cs_new_protected:Nn \stex_iterate_notations:nn {
6671   \seq_clear:N \l__stex_notations_mods_seq
6672   \stex_pseudogroup_with:nn{\__stex_notations_not_cs:nnnnn}{
6673     \cs_set:Npn \__stex_notations_not_cs:nnnnn
6674       ##1 ##2 ##3 ##4 ##5 { #2 }
6675     \clist_map_function:nN {#1} \__stex_notations_it_not_i:n
6676   }
6677 }
6678
6679 \cs_new_protected:Nn \__stex_notations_it_not_i:n {
6680   \seq_if_in:NnF \l__stex_notations_mods_seq {#1} {
6681     \seq_put_left:Nn \l__stex_notations_mods_seq {#1}
6682     \prop_map_inline:cn{\_stex_notations_macro:n{#1}}{
6683       \__stex_notations_not_cs:nnnnn ##2
6684     }
6685     \prop_map_inline:cn{\_stex_morphisms_macro:n{#1}}{
6686       \__stex_notations_it_not_check:nnnn ##2
6687     }
6688   }
6689 }
6690 \cs_new_protected:Nn \__stex_notations_it_not_check:nnnn {
6691   \tl_if_empty:nT{#1}{
6692     \__stex_notations_it_not_i:n {#2}
6693   }
6694 }

```

(End of definition for `\stex_iterate_notations:nn`. This function is documented on page 154.)

sfunction

`\stex_use_notation:nnTF`

`\stex_use_op_notation:nnTF`

```

6695 \cs_new_protected:Npn \stex_use_notation:nnTF #1 #2 #3 #4 {
6696   \cs_if_exist:cTF{\_stex_notations_macro:nn{#1}{#2}}{
6697     \cs_set_eq:Nc \l_stex_notation_cs {\_stex_notations_macro:nn{#1}{#2}}
6698     #3
6699   }{#4}
6700 }
6701 \cs_new_protected:Npn \stex_use_op_notation:nnTF #1 #2 #3 #4 {
6702   \cs_if_exist:cTF{\_stex_notations_op_macro:nn{#1}{#2}}{
6703     \cs_set_eq:Nc \l_stex_notation_cs {\_stex_notations_op_macro:nn{#1}{#2}}
6704     #3
6705   }{#4}
6706 }

```

(End of definition for `\stex_use_notation:nnTF` and `\stex_use_op_notation:nnTF`. These functions are documented on page 154.)

sfunction

`\_stex_notation_check:`

```

6707 \cs_new_protected:Nn \_stex_notation_check: {

```

```

6708 \stex_check_term:nn{notation}{
6709   \cs_set:Npn \comp ##1 {##1}
6710   \stex_debug:nn{check}{Checking~notation:~\cs_meaning:N \l_stex_notation_macrocode_cs ^^
6711   \exp_args:Nne \use:nn{\l_stex_notation_macrocode_cs}{}}{
6712     \stex_notation_make_args:
6713   }
6714 }
6715 }

```

(End of definition for `\_stex_notation_check:`. This function is documented on page 154.)

sfunction

`\_stex_notation_do_html:n`

```

6716 \cs_new_protected:Nn \_stex_notation_do_html:n {
6717   \stex_annotate_invisible:n{\_stex_annotate_force_break:n{\hbox{\stex_annotate:nn {
6718     data-ftml-notation={#1},
6719     data-ftml-notationfragment={\l_stex_key_variant_str},
6720     data-ftml-precedence={\l__stex_notations_opprec_tl},
6721     data-ftml-argprec={\seq_use:Nn \l__stex_notations_precs_seq ,}
6722   }}{
6723     \cs_set_protected:Npn \argsep ##1 ##2 {
6724       \stex_annotate:nn{data-ftml-argsep={}}{
6725         ##1 \tl_if_empty:nTF{##2}{\!\,}{##2}
6726       }
6727     }
6728     \cs_set_protected:Npn \argmap ##1 ##2 ##3 {
6729       \cs_set:Npn \__stex_notations_map_cs: #####1 { ##2 }
6730       \stex_annotate:nn{data-ftml-argmap={}}{
6731         \__stex_notations_map_cs:{##1} \stex_annotate:nn{data-ftml-argmap-sep={}}{##3}
6732       }
6733     }
6734     \cs_set_protected:Npn \maincomp {
6735       \do_comp:nnNn {maincomp}{}\compemph@uri
6736     }
6737     \stex_debug:nn{notation}{Doing~notation~HTML:\meaning\l_stex_get_symbol_args_tl}
6738     $
6739     \tl_clear:N \l_stex_current_full_tl
6740     \stex_annotate:nn{data-ftml-notationcomp={}}{
6741       \exp_args:Nne \use:nn {
6742         \l_stex_notation_macrocode_cs {}
6743       }{
6744         \stex_map_args:N \__stex_notations_make_arg_html:nn
6745       }
6746     }
6747     $
6748     \tl_if_empty:NF \l_stex_key_op_tl {
6749       $
6750       \tl_clear:N \l_stex_current_full_tl
6751       \stex_annotate:nn{data-ftml-notationopcomp={}}{
6752         \stex_term_oms:nn{\l_stex_key_variant_str}{\l_stex_key_op_tl}
6753       }
6754       $
6755     }
6756   }}

```

```

6757   }}
6758 }
6759
6760 \cs_new:Nn \__stex_notations_make_arg_html:nn {
6761   {\stex_annotate:nn{data-ftml-argnum=#1}{x}}
6762 }

```

(End of definition for `\_stex_notation_do_html:n`. This function is documented on page 154.)

sfunction

`\_stex_notation_make_args:`

```

6763 \cs_new:Nn \_stex_notation_make_args: {
6764   \_stex_map_notation_args:N \_stex_notations_make_arg:nnnn
6765 }
6766
6767 \cs_new:Nn \__stex_notations_make_arg:nnnn {
6768   \str_case:nnF #2 {
6769     a {{
6770       a\c_math_subscript_token{#1,1},
6771       a\c_math_subscript_token{#1,2}
6772     }}
6773     B {{
6774       B\c_math_subscript_token{#1,1},
6775       B\c_math_subscript_token{#1,2}
6776     }}
6777   }{{
6778     \_stex_term_arg:nnnn{#1}{#2}{#3}{#4}
6779     {{#2}\c_math_subscript_token{#1}}
6780   }}
6781 }

```

(End of definition for `\_stex_notation_make_args:.` This function is documented on page 154.)

sfunction

`\stex_do_default_notation:`

`\stex_do_default_notation_op:`

```

6782 \cs_new_protected:Nn \stex_do_default_notation: {
6783   \stex_do_default_notation_op:
6784   \tl_if_empty:NTF \l_stex_current_args_tl {
6785     \tl_clear:N \l__stex_notations_args_tl
6786   }\_stex_notations_default_args:
6787   \tl_set:Ne \l_stex_default_notation {\STEXInternalNotation{}{0}{}}{\l__stex_notations_args
6788     \exp_args:No \exp_not:n \l_stex_default_notation
6789   }}
6790 }
6791
6792 \cs_new_protected:Nn \stex_do_default_notation_op: {
6793   \_stex_notations_make_name:
6794   \tl_set:Ne \l_stex_default_notation {\exp_not:N \maincomp{ \exp_not:N \mathrm {\l__stex_n
6795 }
6796
6797 \cs_new_protected:Nn \__stex_notations_default_args: {
6798   \_stex_notations_make_name:
6799   \tl_set_eq:NN \l_stex_get_symbol_args_tl \l_stex_current_args_tl
6800   \tl_set:Ne \l__stex_notations_args_tl {

```



```

6801 \stex_map_args:N \__stex_notations_augment_arg:nn
6802 }
6803 \tl_put_right:Nn \l_stex_default_notation {\comp{}}
6804 \seq_clear:N \l_tmpa_seq
6805 \int_step_inline:nn \l_stex_current_arity_str {
6806   \seq_put_right:Nn \l_tmpa_seq {#### #1}
6807 }
6808 \tl_put_right:Ne \l_stex_default_notation {
6809   \seq_use:Nn \l_tmpa_seq {\mathpunct{\comp{,}}}}
6810 }
6811 \tl_put_right:Nn \l_stex_default_notation {\comp{}}
6812 }
6813
6814 \cs_new:Nn \__stex_notations_augment_arg:nn {
6815   #1#2{0}{}}
6816 }
6817
6818 \cs_new_protected:Nn \__stex_notations_make_name: {
6819   \exp_args:NNNe \seq_set_split:Nnn \l_tmpa_seq / \l_stex_current_display_tl
6820   \seq_pop_right:NN \l_tmpa_seq \l__stex_notations_name_str
6821 }

```

(End of definition for `\stex_do_default_notation:` and `\stex_do_default_notation_op:`. These functions are documented on page 154.)

sfunction

`\STEXInternalNotation`

```

6822 \cs_new_protected:Npn \STEXInternalNotation #1 #2 #3 #4 #5 #6 {
6823   \__stex_notations_process:nnnnnn{#1}{#2}{#3}{#4}{#5}{
6824     \l__stex_notations_code_tl
6825     #6
6826   }
6827 }
6828
6829 \cs_new_protected:Npn \__stex_notations_process:nnnnnn #1 #2 #3 #4 {
6830   \tl_if_empty:nTF{#4}{
6831     \__stex_notations_simple:nnnnn{#1}{#2}{#3}
6832   }{
6833     \__stex_notations_complex:nnnnnn{#1}{#2}{#3}{#4}
6834   }
6835 }
6836
6837 \cs_new_protected:Nn \__stex_notations_simple:nnnnn {
6838   \stex_debug:nn{Notation~code}{\tl_to_str:n{#4}}
6839   \tl_set:Nn \l__stex_notations_code_tl {
6840     \cs_set:Npn \l__stex_notations_cs {
6841       \stex_maybe_brackets:nn{#2}{
6842         \stex_term_oms_or_omv:nn{#1}{#4}
6843       }
6844     }
6845     \l__stex_notations_cs
6846   }
6847   #5
6848 }

```

```

6849
6850 \cs_new_protected:Nn \__stex_notations_complex:nnnnnn {
6851   \stex_debug:nn{Notation~code}{\tl_to_str:n{#5}}
6852   \int_zero:N \l_tmpa_int
6853   \tl_set:Nn \l__stex_notations_pre_tl {\cs_set_eq:NN \stex_term_oma_or_omb:nn \stex_term_
6854   \tl_set:Nn \l__stex_notations_code_tl {
6855     \cs_generate_from_arg_count:NNnn \l__stex_notations_cs \cs_set:Npn \l_tmpa_int
6856     {
6857       \stex_maybe_brackets:nn{#2}{
6858         \stex_term_oma_or_omb:nn{#1}{
6859           \bool_set_false:N \l_stex_brackets_dones_bool
6860           #5
6861         }
6862       }
6863     }
6864     \l__stex_notations_cs
6865   }
6866   \tl_set:Nn \l__stex_notations_after_tl{
6867     \exp_args:NNo
6868     \tl_put_left:Nn \l__stex_notations_code_tl \l__stex_notations_pre_tl
6869     \tl_put_left:Ne \l__stex_notations_code_tl {
6870       \int_set:Nn \l_tmpa_int {\int_use:N \l_tmpa_int}
6871     }
6872     #6
6873   }
6874   \__stex_notations_parse_args:nnnnw #4 \__stex_notations_args_end:
6875 }
6876
6877 \cs_new_protected:Npn \__stex_notations_parse_args:nnnnw #1 #2 #3 #4 #5 \__stex_notations_a
6878   \tl_if_empty:nTF{#5}{
6879     \__stex_notations_add_last:nnnnn{#1}{#2}{#3}{#4}
6880   }{
6881     \__stex_notations_add_next:nnnnnn{#1}{#2}{#3}{#4}{#5}
6882   }
6883 }
6884
6885 \cs_new_protected:Nn \__stex_notations_add_next:nnnnnn {
6886   \__stex_notations_add:nnnnn{#1}{#2}{#3}{#4}{#6}
6887   \__stex_notations_parse_args:nnnnw #5 \__stex_notations_args_end:
6888 }
6889
6890 \cs_new_protected:Nn \__stex_notations_add_last:nnnnn {
6891   \__stex_notations_add:nnnnn{#1}{#2}{#3}{#4}{#5}
6892   \stex_debug:nn{sequences}{Executing~notation:~\meaning\l__stex_notations_code_tl}
6893   \l__stex_notations_after_tl
6894 }
6895
6896 \cs_new_protected:Nn \__stex_notations_add:nnnnn {
6897   \int_incr:N \l_tmpa_int
6898   \str_case:nn{#2}{
6899     i {
6900       \tl_put_right:Nn \l__stex_notations_code_tl {
6901         {\stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{#5}}
6902       }

```

```

6903     }
6904     b {
6905       \tl_set:Nn \l__stex_notations_pre_tl {
6906         \cs_set_eq:NN \_stex_term_oma_or_omb:nn \_stex_term_omb:nn
6907       }
6908       \tl_put_right:Nn \l__stex_notations_code_tl {
6909         {\_stex_term_arg:nnnnn{#1}{#2}{#3}{#4}{#5}}
6910       }
6911     }
6912     a {
6913       \tl_put_right:Nn \l__stex_notations_code_tl {
6914         {\_stex_term_arg_aB:nnnnn{#1}{#2}{#3}{#4}{#5}}
6915       }
6916     }
6917     B {
6918       \tl_set:Nn \l__stex_notations_pre_tl {
6919         \cs_set_eq:NN \_stex_term_oma_or_omb:nn \_stex_term_omb:nn
6920       }
6921       \tl_put_right:Nn \l__stex_notations_code_tl {
6922         {\_stex_term_arg_aB:nnnnn{#1}{#2}{#3}{#4}{#5}}
6923       }
6924     }
6925   }
6926 }

```

(End of definition for `\STEXInternalNotation`. This function is documented on page 154.)

sfunction

`\argsep`

```

6927 \cs_new_protected:Npn \argsep #1 #2 {
6928   \__stex_notations_check_aB_arg:Nn\argsep{#1}
6929   \stex_pseudogroup_with:nn{\_stex_term_do_aB_clist:}{
6930     \tl_set:Nn \_stex_term_do_aB_clist: {
6931       \seq_use:Nn \l_stex_aB_args_seq {#2}
6932     }
6933     #1
6934   }
6935 }
6936
6937 \cs_new_protected:Nn \__stex_notations_check_aB_arg:Nn {
6938   \exp_args:Ne \cs_if_eq:NNF {\tl_head:n{#2}}
6939   \_stex_term_arg_aB:nnnnn {
6940     \msg_error:nnx{stex}{error/assocarg}{\tl_to_str:n{#1}}
6941   }
6942 }

```

(End of definition for `\argsep`. This function is documented on page 154.)

sfunction

`\argmap`

```

6943 \cs_new_protected:Npn \argmap #1 #2 #3 {
6944   \__stex_notations_check_aB_arg:Nn\argmap{#1}
6945   \stex_pseudogroup_with:nn{
6946     \_stex_term_do_aB_clist:

```

```

6947 \_stex_notations_map_cs:
6948 }{
6949 \cs_set:Npn \_stex_notations_map_cs: ##1 { #2 }
6950 \tl_set:Nn \_stex_term_do_aB_clist: {
6951 \seq_clear:N \l_tmpa_seq
6952 \seq_map_inline:Nn \l_stex_aB_args_seq {
6953 \tl_if_eq:nnTF{##1}{\ellipses}{
6954 \seq_put_right:Nn \l_tmpa_seq \ellipses
6955 }{
6956 \seq_put_right:Nx \l_tmpa_seq {
6957 \exp_after:wN \exp_not:n \exp_after:wN { \_stex_notations_map_cs: {##1} }
6958 }
6959 }
6960 }
6961 \seq_set_eq:NN \l_stex_aB_args_seq \l_tmpa_seq
6962 \seq_use:Nn \l_stex_aB_args_seq {#3}
6963 }
6964 #1
6965 }
6966 }

```

(End of definition for \argmap. This function is documented on page 154.)

sfunction

\argarraymap

```

6967 \int_new:N \l__stex_notations_clist_count_int
6968 \cs_new_protected:Npn \argarraymap #1 #2 #3 #4 {
6969 \_stex_notations_check_aB_arg:Nn\argarraymap{#1}
6970 \stex_pseudogroup_with:nn{
6971 \_stex_term_do_aB_clist:
6972 \_stex_notations_map_cs:
6973 }{
6974 \cs_set:Npn \_stex_notations_map_cs: ##1 { #3 }
6975 \int_set:Nn \l__stex_notations_clist_count_int {\exp_args:No\clist_count:n{\tl_to_str:n
6976 \tl_set:Nn \_stex_term_do_aB_clist: {
6977 \tl_clear:N \l_tmpa_tl
6978 \int_zero:N \l_tmpa_int
6979 \seq_map_inline:Nn \l_stex_aB_args_seq {
6980 \int_incr:N \l_tmpa_int
6981 \int_compare:nNnT \l_tmpa_int > \l__stex_notations_clist_count_int {
6982 \int_set:Nn \l_tmpa_int 1
6983 }
6984 \tl_put_right:Ne \l_tmpa_tl {
6985 \exp_after:wN \exp_not:n \exp_after:wN { \_stex_notations_map_cs: {##1} }
6986 \clist_item:nn{#4}\l_tmpa_int
6987 }
6988 }
6989 \seq_set_eq:NN \l_stex_aB_args_seq \l_tmpa_seq
6990 \begin{array}{#2}
6991 \l_tmpa_tl
6992 \end{array}
6993 }
6994 #1
6995 }
6996 }

```

(End of definition for `\argarraymap`. This function is documented on page 154.)
sfunction

33.1 Automated Bracketing

Variables for current (downwards) precedence, set of parentheses etc.:

```

6997 \int_new:N \l_stex_notation_downprec
6998 \tl_set:Nn \l__stex_notations_left_bracket_str (
6999 \tl_set:Nn \l__stex_notations_right_bracket_str )
7000 \bool_new:N \l_stex_brackets_dones_bool

\infprec
\neginfprec
7001 \tl_const:Ne \infprec {\int_use:N \c_max_int}
7002 \tl_const:Ne \neginfprec {-\int_use:N \c_max_int}
7003
7004 \int_set:Nn \l_stex_notation_downprec \infprec

```

(End of definition for `\infprec` and `\neginfprec`. These variables are documented on page 154.)

`\_stex_maybe_brackets:nn`

```

7005 \cs_new_protected:Nn \_stex_maybe_brackets:nn {
7006   \bool_if:NTF \l_stex_brackets_dones_bool {
7007     \bool_set_false:N \l_stex_brackets_dones_bool
7008     #2
7009   } {
7010     \stex_debug:nn{brackets}{#1>\int_eval:n \l_stex_notation_downprec?}
7011     \int_compare:nNnTF { #1 } > \l_stex_notation_downprec {
7012       %\bool_if:NTF \l_stex_inarray_bool { #2 }{
7013         \dobrackets {
7014           #2
7015         }
7016       %}
7017     }{
7018       #2
7019     }
7020   }
7021 }

```

(End of definition for `\_stex_maybe_brackets:nn`. This function is documented on page 154.)
sfunction

`\dobrackets`

```

7022 \cs_new_protected:Npn \dobrackets #1 {
7023   \group_begin:
7024     \bool_set_true:N \l_stex_brackets_dones_bool
7025     \mathopen{\tl_if_exist:NT\l_stex_current_symbol_uri\comp
7026       \l__stex_notations_left_bracket_str
7027     }
7028     #1
7029   \group_end:
7030   \mathclose{\tl_if_exist:NT\l_stex_current_symbol_uri\comp
7031     \l__stex_notations_right_bracket_str
7032   }
7033 }

```

(End of definition for \dobrackets. This function is documented on page 155.)

sfunction

\withbrackets

```
7034 \cs_new_protected:Npn \withbrackets #1 #2 #3 {  
7035   \stex_pseudogroup:nn{  
7036     \tl_set:Nn \l__stex_notations_left_bracket_str { #1 }  
7037     \tl_set:Nn \l__stex_notations_right_bracket_str { #2 }  
7038     #3  
7039   }{  
7040     \stex_pseudogroup_restore:N \l__stex_notations_left_bracket_str  
7041     \stex_pseudogroup_restore:N \l__stex_notations_right_bracket_str  
7042   }  
7043 }
```

(End of definition for \withbrackets. This function is documented on page 155.)

sfunction

\dowithbrackets

```
7044 \cs_new_protected:Npn \dowithbrackets #1 #2 #3 {  
7045   \withbrackets{#1}{#2}{\dobrackets{#3}}  
7046 }
```

(End of definition for \dowithbrackets. This function is documented on page 155.)

sfunction

Chapter 34

Structures

```
7047 <@@=stex_structures>

Keys:
7048 \stex_keys_define:nnnn{mathstructure}{
7049   \tl_clear:N \l_stex_current_this_tl
7050   \str_clear:N \l__stex_structures_name_str
7051 }{
7052   this .tl_set:N = \l_stex_current_this_tl ,
7053   unknown .code:n = {
7054     \str_if_empty:NTF \l_keys_key_str {
7055       \str_set:Ne \l__stex_structures_name_str {\l_keys_key_tl}
7056     }{
7057       \str_set_eq:NN \l__stex_structures_name_str \l_keys_key_str
7058     }
7059   }
7060 }{}
```

`mathstructure (env.)`

```
7061 \stex_new_stylable_env:nnnnnnn {mathstructure}{m 0{}}{
7062   \__stex_structures_begin:nn{#1}{#2}
7063   \stex_smsmode_do:
7064 }{
7065   \stex_structural_feature_module_end:
7066   \__stex_structures_do externals:
7067 }{}{}{}
7068
7069 \stex_sms_allow_env:n{mathstructure}
7070 \stex_deactivate_macro:Nn \mathstructure {module~environments}
7071 \stex_every_module:n {\stex_reactivate_macro:N \mathstructure}
```

Shared code for `mathstructure` and `extstructure`:

```
7072 \cs_new_protected:Nn \__stex_structures_begin:nn {
7073   \stex_keys_set:nn {mathstructure}{#2}
7074   \str_if_empty:NT \l__stex_structures_name_str {
7075     \str_set:Nn \l__stex_structures_name_str {#1}
7076   }
7077   \def\comp{\_comp}
7078
7079   \tl_set:Ne \l__stex_structures_struct_uri {
```

```

7080 \exp_args:No \stex_new_module_uri:n \l__stex_structures_name_str
7081 }
7082
7083 \exp_args:Nne \use:nn { \stex_add_symbol:nnnnnnN }
7084 { {#1}{\l__stex_structures_name_str}{0}{\defed}{\l__stex_structures_struct_uri}}
7085 {} \stex_invoke_structure:
7086 \str_set:Ne \l_stex_macroname_str {#1}
7087
7088 \exp_args:No \stex_structural_feature_module:nn
7089 {\l__stex_structures_name_str}{structure}
7090 }
7091
7092 \cs_new_protected:Nn \__stex_structures_doexternals: {
7093 \tl_set:Nn \l__stex_structures_replace_this_tl {####1}
7094 }
7095

```

extstructure (env.)

```

7096 \stex_new_stylable_env:nnnnnn {extstructure}{m O{} m}{
7097 \seq_clear:N \l__stex_structures_imports_seq
7098 \clist_map_inline:nn{#3}{
7099 \stex_get_mathstructure:n{##1}
7100 \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
7101 \stex_if_module_exists:nT{####1}{
7102 \seq_put_right:Nn \l__stex_structures_imports_seq{####1}
7103 }
7104 }
7105 }
7106 \__stex_structures_begin:nn{#1}{#2}
7107 \seq_map_inline:Nn \l__stex_structures_imports_seq{
7108 \stex_if_do_html:T {
7109 \hbox{\stex_annotate_invisible:nn
7110 {data-ftml-import={\stex_use_module_uri:n{##1}}}{}}
7111 }
7112 \stex_add_morphism:nonn
7113 {}{##1}{import}{}
7114 \stex_execute_in_module:e{
7115 \stex_activate_module:n {##1}
7116 }
7117 }
7118 \stex_smsmode_do:
7119 }{
7120 \stex_structural_feature_module_end:
7121 \__stex_structures_doexternals:
7122 }{}{}
7123
7124 \stex_sms_allow_env:n{extstructure}
7125 \stex_deactivate_macro:Nn \extstructure {module~environments}
7126 \stex_every_module:n {
7127 \stex_reactivate_macro:N \extstructure
7128 }
7129
7130 \stex_new_stylable_env:nnnnnn {extstructure*}{m}{
7131 \__stex_structures_new_extstruct_name:

```



```

7132 \seq_clear:N \l__stex_structures_imports_seq
7133 \stex_get_mathstructure:n{#1}
7134
7135 \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
7136   \stex_if_module_exists:nT{##1}{
7137     \seq_put_right:Nn \l__stex_structures_imports_seq{##1}
7138   }
7139 }
7140
7141 \stex_module_uri_split_name:NNN \l__stex_structures_parent_uri \l__stex_structures_last_n
7142
7143 \stex_execute_in_module:e{
7144   \__stex_structures_extend_structure:nnnn{\l__stex_structures_parent_uri}{\l__stex_structures_last_n}
7145 }
7146
7147 \exp_args:No \__stex_structures_begin:nn\l__stex_structures_exstruct_name_str{
7148
7149   \seq_map_inline:Nn \l__stex_structures_imports_seq {
7150     \stex_if_do_html:T {
7151       \stex_annotate_invisible:nn
7152       {data-ftml-import= {\stex_use_module_uri:n{##1}}}{ }
7153     }
7154     %\stex_add_morphism:nonn
7155     % {}{##1}{import}{ }
7156     %\stex_execute_in_module:e{
7157       % \stex_activate_module:n {##1}
7158       %}
7159     \stex_activate_module:n {##1}
7160   }
7161
7162   \stex_smsmode_do:
7163 }{
7164   \prop_map_inline:cn{\_stex_symbol_macro:N \l_stex_current_module_uri }{
7165     \__stex_structures_check_def:nnnnnnn ##2
7166   }
7167   \stex_structural_feature_module_end:
7168   %\exp_args:Ne \stex_do_up_to_module:n {
7169   % \stex_activate_module:n {\exp_args:No \stex_new_module_uri:n {\l__stex_structures_exst
7170   %}
7171   \__stex_structures_do externals:
7172 }{}{}{}
7173
7174 \stex_sms_allow_env:n{extstructure*}
7175 \exp_after:wN \stex_deactivate_macro:Nn
7176 \cs:w extstructure*\cs_end: {module~environments}
7177 \stex_every_module:n {
7178   \exp_after:wN \stex_reactivate_macro:N \cs:w extstructure*\cs_end:
7179 }
7180
7181 \cs_new_protected:Nn \__stex_structures_extend_structure:nnnn {
7182   \stex_debug:nn{ext}{Extending~\stex_use_module_uri:n {#1} / #2-by~\stex_use_module_uri:n
7183   \exp_args:Nne \tl_put_right:cn{\_stex_module_code_macro:n{#4}}{
7184     \stex_activate_module:n{#3}
7185   }

```

```

7186 \exp_args:Nne \prop_put:cnn{\_stex_morphisms_macro:n{#4}}{\_stex_use_module_uri:n{#3}}{
7187   {}{#3}{extstruct}}{
7188 }
7189 \exp_args:Nne \use:nn {\_stex_structures_extend_ii:nnnn}{
7190   \exp_args:Nne \use:nn \_stex_structures_extend_i:nnnnnnnnNn {
7191     \prop_item:cn{\_stex_symbol_macro:n {#1} } { #2 }
7192     } { #3 }
7193   }{#1}{#2}
7194 }
7195
7196 \cs_new_protected:Nn \_stex_structures_extend_ii:nnnn {
7197   \prop_put:cnn{\_stex_symbol_macro:n {#3} } { #4 }{#2}
7198   \tl_set:ce{#1}{
7199     \_stex_invoke_symbol:nnnnnnN {\_stex_new_symbol_uri:nn {#3}{#4}}
7200     \use_none:nn #2
7201   }
7202 }
7203
7204 \cs_new:Nn \_stex_structures_extend_i:nnnnnnnnNn {
7205   {#1}{#{#1}{#2}{#3}{#4}{#5}{#6,#9}{#7}#8}
7206 }
7207
7208 \cs_new_protected:Nn \_stex_structures_check_def:nnnnnnnn {
7209   \tl_if_empty:nT{#5}{
7210     \msg_error:nnnn{stex}{error/needsdefinien}{#2}{extstructure*}
7211   }
7212 }
7213
7214 \cs_new_protected:Nn \_stex_structures_new_extstruct_name: {
7215   \stex_do_up_to_module:n {
7216     \str_set:Ne \l__stex_structures_extname_count {\int_eval:n{\l__stex_structures_extname_
7217   }
7218   \str_set:Ne \l__stex_structures_exstruct_name_str {EXTSTRUCT_\l__stex_structures_extname_
7219 }
7220
7221 \stex_every_module:n{ \str_set:Nn \l__stex_structures_extname_count 0}

```

\this

```

7222 \bool_new:N \l_stex_in_this_bool
7223
7224 \cs_new_protected:Npn \stex_current_this: {
7225   { \bool_set_true:N \l_stex_allow_semantic_bool
7226     \tl_if_empty:NTF \l_stex_current_this_tl {} }{
7227     \tl_set:Nn \l_stex_current_full_tl {
7228       \exp_args:Ne \stex_use_symbol_uri:n {
7229         \exp_args:No \stex_new_symbol_uri:n \l__stex_structures_name_str
7230       }
7231     }
7232     \str_set_eq:NN \l_stex_current_display_tl \l__stex_structures_name_str
7233     \bool_set_true:N \l_stex_in_this_bool
7234     \maincomp{\l_stex_current_this_tl}
7235     \bool_set_false:N \l_stex_in_this_bool
7236   }
7237 }

```

```

7238 }
7239
7240 \let \this \stex_current_this:

```

(End of definition for \this. This function is documented on page 156.)

```

sfunction

```

```

\stex_get_mathstructure:n

```

```

7241 \cs_new_protected:Nn \stex_get_mathstructure:n {
7242   \stex_get_mathstructure_maybe:nTF{#1}{}{
7243     \msg_error:nnn{stex}{error/unknownstructure}{#1}
7244   }
7245 }
7246 \cs_new_protected:Npn \stex_get_mathstructure_maybe:nTF #1 #2 #3 {
7247   \tl_clear:N \l_stex_get_structure_module_uri
7248   \stex_get_symbol:nTF{#1}{
7249     \exp_args:No \tl_if_eq:NNTF \l_stex_get_symbol_invoke_cs \stex_invoke_structure: {
7250       \tl_set:Ne \l_stex_get_structure_module_uri { \exp_after:wN \__stex_structures_get:w
7251         #2
7252       }{
7253         #3
7254       }
7255     }{
7256       #3
7257     }
7258   }
7259 }
7260 \cs_new:Npn \__stex_structures_get:w #1 , #2 \stex_end: { #1 }

```

(End of definition for \stex_get_mathstructure:n. This function is documented on page 156.)

```

sfunction

```

```

\usestructure

```

```

7261 \cs_new_protected:Npn \usestructure #1 {
7262   \stex_if_smsmode:TF{
7263     \stex_get_mathstructure_maybe:nTF{ #1 }{
7264       \stex_debug:nn{usestructure}{Using~structure~#1:~\stex_use_module_uri:N \l_stex_get_str
7265       \__stex_structures_do_usestructure:
7266     }{
7267       \stex_debug:nn{usestructure}{Structure~#1~not~found~in~sms~mode}
7268     }
7269   }{
7270     \stex_get_mathstructure:n{ #1 }
7271     \stex_debug:nn{usestructure}{Using~structure~#1:~\stex_use_module_uri:N \l_stex_get_str
7272     \__stex_structures_do_usestructure:
7273   }
7274   \stex_smsmode_do:
7275 }
7276 \stex_sms_allow_escape:N \usestructure
7277
7278 \cs_new_protected:Nn \__stex_structures_do_usestructure: {
7279   \seq_clear:N \l__stex_structures_imports_seq
7280   \clist_map_inline:Nn \l_stex_get_symbol_type_tl {
7281     \stex_if_module_exists:nT{##1}{

```

```

7282     \seq_put_right:Nn \l__stex_structures_imports_seq{##1}
7283   }
7284 }
7285 \seq_map_inline:Nn \l__stex_structures_imports_seq {
7286   \stex_if_do_html:T {
7287     \hbox{\stex_annotate_invisible:nn
7288       {data-ftml-usemodule=\stex_use_module_uri:n {##1}} {}}
7289   }
7290   \stex_activate_module:n {##1}
7291 }
7292 }

```

(End of definition for \usestructure. This function is documented on page 156.)

sfunction

Chapter 35

Statements

7293 `<@@=stex_statements>`

Keys:

```
7294 \stex_keys_define:nnnn{statement}{  
7295   \str_clear:N \l_stex_key_name_str  
7296   \str_clear:N \l_stex_key_macroname_str  
7297   \clist_clear:N \l_stex_key_for_clist  
7298   \str_clear:N \l_stex_key_args_str  
7299   \tl_clear:N \l_stex_key_type_tl  
7300   \tl_clear:N \l_stex_key_def_tl  
7301   \tl_clear:N \l_stex_key_return_tl  
7302   \clist_clear:N \l_stex_key_argtypes_clist  
7303 }{  
7304   name      .str_set:N = \l_stex_key_name_str ,  
7305   for       .clist_set:N = \l_stex_key_for_clist ,  
7306   macro     .str_set:N = \l_stex_key_macroname_str ,  
7307   % start   .str_set:N = \l_stex_key_title_str , % TODO remove  
7308   type      .tl_set:N = \l_stex_key_type_tl ,  
7309   judgment  .code:n = {},  
7310   from      .code:n= {}, % TODO remove  
7311   to        .code:n={} % TODO remove  
7312 }{id,title,style,symargs}
```

`\stex_do_for_list:`

```
7313 \cs_new_protected:Npn \stex_do_for_list: {  
7314   \seq_clear:N \l_stex_fors_seq  
7315   \clist_map_inline:Nn \l_stex_key_for_clist {  
7316     \exp_args:Ne\stex_get_symbol:n{\tl_to_str:n{##1}}  
7317     \seq_put_right:No \l_stex_fors_seq  
7318     {\l_stex_get_symbol_uri}  
7319   }  
7320 }
```

(End of definition for \stex_do_for_list:. This function is documented on page 157.)

sfunction

`\stex_new_statement:nnn`

```
7321 \cs_new_protected:Nn \stex_new_statement:nnn {  
7322   \stex_new_stylable_env:nnnnnnn {#1}{0{}}{
```

```

7323 \stex_keys_set:nn{statement}{##1}
7324 #3
7325
7326 \stex_if_smsmode:F {
7327   \exp_args:Nne \begin{stex_annotate_env}{
7328     \__stex_statements_html_keyvals:nn{#1}{false}
7329   }
7330   \tl_set_eq:NN \thistitle \l_stex_key_title_tl
7331   \str_set_eq:NN \thisname \l_stex_key_name_str
7332   \clist_set_eq:NN \thisfor \l_stex_key_for_str
7333   \stex_if_html_backend:TF {
7334     \noindent
7335     \stex_annotate:nn{data-ftml-title={}, style:display={none}}{\_stex_annotate_force_b
7336     \tl_if_empty:NF \l_stex_key_title_tl{~}
7337     \l_stex_key_title_tl
7338   }}
7339 }
7340 \stex_style_apply:
7341 }
7342 \_stex_do_id:
7343 \stex_smsmode_do:
7344 }{
7345   \stex_if_smsmode:F {
7346     \stex_if_html_backend:F \stex_style_apply:
7347     \end{stex_annotate_env}
7348   }
7349 }{}{}{s}
7350 \stex_sms_allow_env:n{s#1}
7351
7352 \tl_if_empty:NF{#2}{\__stex_statements_make_macro:nnn{#1}{#2}{#3}}
7353 }
7354
7355 \cs_new_protected:Nn \__stex_statements_make_macro:nnn {
7356   \exp_after:wN \NewDocumentCommand \cs:w inline#2\cs_end: { 0{} m}{
7357     \group_begin:
7358     \stex_keys_set:nn{statement}{##1}
7359     #3
7360     \_stex_do_id:
7361     \stex_if_smsmode:F{
7362       \exp_args:Ne \stex_annotate:nn{\__stex_statements_html_keyvals:nn{#1}{true}}{
7363         \_stex_annotate_force_break:n{##2}
7364       }
7365     }
7366     \group_end:
7367     %\stex_if_smsmode:TF \stex_smsmode_do: \stex_ignore_spaces_and_pars:
7368     \stex_smsmode_do:
7369   }
7370   \exp_after:wN \stex_sms_allow_escape:N\cs:w inline#2\cs_end:
7371 }
7372
7373 \cs_new:Nn \__stex_statements_html_keyvals:nn {
7374   data-ftml-#1={},
7375   data-ftml-inline={#2},
7376   \seq_if_empty:NF \l_stex_fors_seq {,

```

```

7377     data-ftml-fors={\seq_map_indexed_function:Nn \l_stex_fors_seq \_stex_comma_sep_uri:nn }
7378   }
7379   \str_if_empty:NF \l_stex_key_id_str {,
7380     data-ftml-id={\l_stex_key_id_str}%{\stex_uri_use:N \l_stex_current_doc_uri ? \l_stex_ke
7381   }
7382   \clist_if_empty:NF \l_stex_key_style_clist {,
7383     data-ftml-styles={\l_stex_key_style_clist}
7384   }
7385 }
7386
7387 \cs_new:Nn \_stex_comma_sep_uri:nn {
7388   \int_compare:nNnF{#1}=1 {,}
7389   \stex_use_symbol_uri:n{#2}
7390 }

```

(End of definition for `\stex_new_statement:nnn`. This function is documented on page 157.)

sfunction

`\_stex_statements_setup:n` sets up a statement that may declare a new symbol with the given `role` by processing the optional arguments, calling `\stex_symdecl_do:`, etc.

```

7391 \cs_new_protected:Nn \_stex_statements_setup:n {
7392   \str_if_empty:NF \l_stex_key_macroname_str {
7393     \str_if_empty:NT \l_stex_key_name_str {
7394       \str_set_eq:NN \l_stex_key_name_str \l_stex_key_macroname_str
7395     }
7396   }
7397   \stex_do_for_list:
7398   \tl_clear:N \l_stex_current_def_uri
7399
7400   \str_if_empty:NTF \l_stex_key_name_str \_stex_statements_setup_noname: {
7401     \_stex_statements_setup_named:n{#1}
7402   }
7403 }
7404
7405 \cs_new_protected:Nn \_stex_statements_setup_named:n {
7406   \_stex_statements_force_id:
7407   \seq_put_right:Ne \l_stex_fors_seq {
7408     \l_stex_current_module_uri {\l_stex_key_name_str}
7409   }
7410   \str_set_eq:NN \l_stex_macroname_str \l_stex_key_macroname_str
7411   \str_set:Nn \l_stex_key_role_str {#1}
7412   \stex_symdecl_do:
7413   \exp_args:Nne \use:nn {\stex_add_symbol:nnnnnnN}{
7414     {\l_stex_key_macroname_str}{\l_stex_key_name_str}
7415     {\int_use:N \l_stex_get_symbol_arity_int}
7416     {\l_stex_get_symbol_args_tl}
7417     {#1}{}}{\stex_invoke_symbol:
7418   }
7419   \stex_if_do_html:T \stex_symdecl_html:
7420   \tl_set:Ne \l_stex_current_def_uri {\exp_args:No \stex_new_symbol_uri:nn \l_stex_current_
7421   \stex_debug:nn{statement}{name:~\stex_use_symbol_uri:N \l_stex_current_def_uri}
7422 }
7423
7424 \cs_new_protected:Nn \_stex_statements_setup_noname: {

```

```

7425 \stex_debug:nn{statement}{no~name}
7426 \int_compare:nNnTF {\seq_count:N \l_stex_fors_seq} = 1 {
7427   \tl_set:N \l_stex_current_def_uri {\seq_item:Nn \l_stex_fors_seq 1}
7428   \stex_debug:nn{statement}{for:~\stex_use_symbol_uri:N \l_stex_current_def_uri}
7429 }{
7430   \stex_debug:nn{statement}{no~for}
7431 }
7432 }
7433
7434 \cs_new_protected:Nn \__stex_statements_force_id: {
7435   %\str_if_empty:NT \l_stex_key_id_str {
7436   %   \stex_ref_new_id:n{ }
7437   %   \str_set_eq:NN \l_stex_key_id_str \l__stex_refs_str
7438   %}
7439 }

```

(End of definition for __stex_statements_setup:n.)

__stex_statements_setup_def: Sets up all the macros allowed in definition-like environments:

```

7440 \cs_new_protected:Nn \__stex_statements_setup_def: {
7441   \stex_if_smsmode:F{
7442     \seq_map_inline:Nn \l_stex_fors_seq {
7443       \exp_args:Ne \stex_ref_new_sym_target:n{\stex_use_symbol_uri:n{##1}}
7444     }
7445   }
7446   \stex_reactivate_macro:N \definiendum
7447   \stex_reactivate_macro:N \defnotation
7448   \stex_reactivate_macro:N \definame
7449   \stex_reactivate_macro:N \Definame
7450   \stex_reactivate_macro:N \varbind
7451   \stex_reactivate_macro:N \definiens
7452 }

```

(End of definition for __stex_statements_setup_def:.)

sdefinition (env.)

```

7453 \stex_new_statement:nnn{definition}{def}{
7454   \__stex_statements_force_id:
7455   \__stex_statements_setup:n{ }
7456   \__stex_statements_setup_def:
7457 }

```

sassertion (env.)

```

7458 \stex_new_statement:nnn{assertion}{ass}{
7459   \__stex_statements_setup:n{assertion}
7460   \stex_if_smsmode:F{
7461     \seq_map_inline:Nn \l_stex_fors_seq {
7462       \exp_args:Ne \stex_ref_new_sym_target:n{\stex_use_symbol_uri:n{##1}}
7463     }
7464   }
7465   \stex_reactivate_macro:N \varbind
7466   \stex_reactivate_macro:N \conclusion
7467   \stex_reactivate_macro:N \premise
7468   \stex_reactivate_macro:N \definiendum

```



```

7469 \stex_reactivate_macro:N \defnotation
7470 \stex_reactivate_macro:N \definame
7471 \stex_reactivate_macro:N \Definame
7472 }

```

sexample (*env.*)

```

7473 \stex_new_statement:nnn{example}{ex}{
7474   \stex_if_smsmode:F {\_stex_statements_setup:n{example}}
7475 }

```

sparagraph (*env.*)

```

7476 \stex_new_statement:nnn{paragraph}{}{
7477   \clist_if_in:NnTF \l_stex_key_style_clist {syndoc}{
7478     \_stex_statements_force_id:
7479     \_stex_statements_setup:n{
7480       \_stex_statements_setup_def:
7481     }{
7482       \_stex_statements_setup:n{
7483     }
7484 }

```

definiens

```

7485 \NewDocumentCommand \definiens { O{} m }{
7486   \group_begin:
7487   \_stex_definiens_impl:nn{#1}{#2}
7488   \group_end:
7489   \stex_smsmode_do:
7490 }
7491
7492 \cs_new_protected:Nn \_stex_definiens_impl:nn {
7493   \tl_if_empty:nF {#1} {
7494     \stex_get_symbol:n { #1 }
7495     \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
7496   }
7497   \tl_if_empty:NT \l_stex_current_def_uri {
7498     \msg_error:nn{stex}{error/definiensfor}
7499   }
7500   \stex_debug:nn{definiens}{Checking-\stex_use_symbol_uri:N \l_stex_current_def_uri }
7501
7502   \exp_args:No \_stex_add_definiens:nn \l_stex_current_def_uri {#2}
7503 }
7504
7505 \stex_deactivate_macro:Nn \definiens {definition~environments}
7506 \stex_sms_allow_escape:N \definiens

```

(End of definition for definiens. This function is documented on page 157.)

sfunction

_stex_add_definiens:nn

```

7507 \cs_new_protected:Nn \_stex_add_definiens:nn {
7508   \stex_if_do_html:TF {
7509     \stex_annotate:nn{data-ftml-definiens={\stex_use_symbol_uri:n{#1}}}{
7510       #2 %\_stex_annotate_force_break:n{ #2 }
7511     }

```

```

7512   }{ \stex_if_smsmode:F{#2} }
7513   \stex_if_in_module:T {
7514     \exp_args:Ne \str_if_eq:noT {\stex_symbol_uri_module:n{#1}}\l_stex_current_module_uri{
7515       \__stex_statements_add_definiens:n{#1}
7516     }
7517   }
7518 }
7519
7520 \cs_new_protected:Nn \__stex_statements_add_definiens:n {
7521   \stex_debug:nn{definiens}{Adding~definiens~to~\stex_use_symbol_uri:n{#1}}
7522   \exp_args:Nne \prop_gput:cne{\exp_args:Ne \stex_symbol_macro:n {\stex_symbol_uri_module:
7523     {\stex_symbol_uri_name:n {#1}} {
7524       \exp_args:Nne \use:nn { \__stex_statements_definiens_inner:nnnnnnN }{ \exp_args:Nne
7525     }
7526   }
7527
7528 \cs_new:Nn \__stex_statements_definiens_inner:nnnnnnN {
7529   {#1}{#2}{#3}{#4}{defed}{#6}{#7}{#8}
7530 }

```

(End of definition for `\stex_add_definiens:nn`. This function is documented on page 157.)

sfunction

`\varbind`

```

7531 \NewDocumentCommand \varbind {m} {
7532   \clist_map_inline:nn {#1} {
7533     \stex_get_var:n {##1}
7534     \stex_if_do_html:T {
7535       \stex_annotate_invisible:nn {data-ftml-bind=\l_stex_get_variable_str}{
7536     }
7537   }
7538 }
7539 \stex_deactivate_macro:Nn \varbind {definition~or~assertion~environments}

```

(End of definition for `\varbind`. This function is documented on page 157.)

sfunction

`\conclusion`

```

7540 \NewDocumentCommand \conclusion { 0{ } m } {
7541   \group_begin:
7542   \tl_if_empty:nF {#1} {
7543     \stex_get_symbol:n { #1 }
7544     \tl_set_eq:NN \l_stex_current_def_uri \l_stex_get_symbol_uri
7545   }
7546   \tl_if_empty:NT \l_stex_current_def_uri {
7547     \msg_error:nn{stex}{error/conclusionfor}
7548   }
7549   \stex_annotate:nn{ data-ftml-conclusion={\stex_use_symbol_uri:N \l_stex_current_def_uri}}
7550   #2 %\stex_annotate_force_break:n{ #2 }
7551 }
7552 \group_end:
7553 }
7554 \stex_deactivate_macro:Nn \conclusion {assertion~environments}

```

(End of definition for `\conclusion`. This function is documented on page 157.)
`sfunction`

`\premise`

```

7555 \NewDocumentCommand \premise {0{} m} {
7556   \tl_if_empty:nF {#1} {
7557     \stex_debug:nn{Here:}{Variable~#1}
7558     \exp_args:Nne\use:nn{\vardef}{v#1}[name=#1]{#1}
7559   }
7560   \stex_annotate:nn{data-ftml-premise={#1}}{#2}
7561 }
7562 \stex_deactivate_macro:Nn \premise {assertion~environments}

```

(End of definition for `\premise`. This function is documented on page 157.)
`sfunction`

Chapter 36

Proofs

```
7563 <@@=stex_proof>
      Keys:
7564
7565 \stex_keys_define:nnnn{ spfcommon }{
7566   \tl_clear:N \l_stex_key_for_clist
7567   \tl_clear:N \l_stex_key_term_tl
7568   \tl_clear:N \l_stex_key_method_tl
7569   \tl_clear:N \l_stex_key_just_tl
7570 }{
7571   for      .clist_set:N = \l_stex_key_for_clist ,
7572   term     .tl_set:N    = \l_stex_key_term_tl,
7573   method   .tl_set:N    = \l_stex_key_method_tl,
7574   just     .tl_set:N    = \l_stex_key_just_tl,
7575 }{id,style,title}
7576
7577 \bool_set_false:N \l_stex_key_showname_bool
7578
7579 \stex_keys_define:nnnn{ spfnamed }{
7580   \str_clear:N \l_stex_key_name_str
7581   \str_clear:N \l_stex_key_numname_str
7582   \bool_set_false:N \l_stex_key_showname_bool
7583 }{
7584   name      .str_set_x:N = \l_stex_key_name_str,
7585   numname   .str_set_x:N = \l_stex_key_numname_str,
7586   showname  .bool_set:N  = \l_stex_key_showname_bool
7587 }{spfcommon}
7588
7589 \cs_new_protected:Nn \__stex_proof_do_spf_name: {
7590   \str_if_empty:NTF \l_stex_key_numname_str {
7591     \str_if_empty:NTF \l_stex_key_name_str {
7592       \bool_set_false:N \l_stex_key_showname_bool
7593     } {
7594       \exp_args:Nne\use:nn{\vardef}{\v\l_stex_key_name_str}[name=\l_stex_key_name_str]{\l_s
7595     }
7596   }{
7597     \bool_set_false:N \l_stex_key_showname_bool
7598     \__stex_proof_number_as_string:N \l__stex_proof_counter_str
7599     \exp_args:Nne \use:nn {\vardef}{\l\l_stex_key_numname_str}[name=\l__stex_proof_counter_s
```

```

7600 }
7601 }
7602
7603 \cs_new_protected:Nn \__stex_proof_do_spf_name_i: {
7604   \str_if_empty:NTF \l__stex_proof_key_numname_str {
7605     %\str_if_empty:NF \l__stex_proof_key_name_str {
7606       \exp_args:Nne\use:nn{\vardef}{\{v\l__stex_proof_key_name_str}[name=\l__stex_proof_key_
7607       %}
7608     }{
7609       \exp_args:Nne \use:nn {\vardef}{\{\l__stex_proof_key_numname_str}[name=\l__stex_proof_co
7610     }
7611   }
7612
7613 \cs_new_protected:Nn \__stex_proof_do_spf_varname: {
7614   \bool_if:NT \l_stex_key_showname_bool {
7615     ~($\csname v\l_stex_key_name_str\endcsname$)~
7616   }
7617 }
7618
7619 \stex_keys_define:nnnn{ spftop }{
7620   \tl_set_eq:NN \l_stex_key_proofend_tl \__stex_proof_proof_box_tl
7621   \bool_set_false:N \l_stex_key_hide_bool
7622   \tl_clear:N \l_stex_key_continues_tl
7623   \tl_clear:N \l_stex_key_functions_tl
7624   \tl_clear:N \l_stex_key_from_tl
7625 }{
7626   proofend      .tl_set:N      = \l_stex_key_proofend_tl,
7627   hide          .code:n        = {
7628     \str_if_eq:eeT{#1}{true}{
7629       \bool_set_true:N \l_stex_key_hide_bool
7630     }
7631   },
7632   continues     .tl_set:N      = \l_stex_key_continues_tl, % <- unused
7633   functions     .tl_set:N      = \l_stex_key_functions_tl, % <- unused
7634   from          .tl_set:N      = \l_stex_key_from_tl % <- unused
7635 }{spfcommon}
7636
7637 \stex_keys_define:nnnn{ subproof }{ }{ }{spftop,spfnamed}
7638
7639 \bool_set_true:N \l__stex_proof_inc_counter_bool

```

For proofs, we will have to have deeply nested structures of enumerated list-like environments. However, L^AT_EX only allows `enumerate` environments up to nesting depth 4 and general list environments up to listing depth 6. This is not enough for us. Therefore we have decided to go along the route proposed by Leslie Lamport to use a single top-level list with dotted sequences of numbers to identify the position in the proof tree. Unfortunately, we could not use his `pf.sty` package directly, since it does not do automatic numbering, and we have to add keyword arguments all over the place, to accomodate semantic information.

```

7640 \intarray_new:Nn\l__stex_proof_counter_intarray{50}
7641
7642 \cs_new_protected:Npn \__stex_proof_insert_number: {
7643   \int_set:Nn \l_tmpa_int {1}
7644   \bool_while_do:nn {

```

```

7645     \int_compare_p:nNn {
7646         \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7647     } > 0
7648 }{
7649     \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int .
7650     \int_incr:N \l_tmpa_int
7651 }
7652 }
7653 \cs_new_protected:Nn \__stex_proof_number_as_string:N {
7654     \str_clear:N #1
7655     \int_set:Nn \l_tmpa_int {1}
7656     \bool_while_do:nn {
7657         \int_compare_p:nNn {
7658             \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7659         } > 0
7660     }{
7661         \str_put_right:Ne #1 {\intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int .}
7662         \int_incr:N \l_tmpa_int
7663     }
7664 }
7665
7666 \cs_new_protected:Npn \__stex_proof_inc_counter: {
7667     \int_set:Nn \l_tmpa_int {1}
7668     \bool_while_do:nn {
7669         \int_compare_p:nNn {
7670             \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7671         } > 0
7672     }{
7673         \int_incr:N \l_tmpa_int
7674     }
7675     \int_compare:nNnF \l_tmpa_int = 1 {
7676         \int_decr:N \l_tmpa_int
7677     }
7678     \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int {
7679         \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int + 1
7680     }
7681 }
7682
7683 \cs_new_protected:Npn \__stex_proof_add_counter: {
7684     \int_set:Nn \l_tmpa_int {1}
7685     \bool_while_do:nn {
7686         \int_compare_p:nNn {
7687             \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7688         } > 0
7689     }{
7690         \int_incr:N \l_tmpa_int
7691     }
7692     \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int { 1 }
7693 }
7694
7695 \cs_new_protected:Npn \__stex_proof_remove_counter: {
7696     \int_set:Nn \l_tmpa_int {1}
7697     \bool_while_do:nn {
7698         \int_compare_p:nNn {

```

```

7699     \intarray_item:Nn \l__stex_proof_counter_intarray \l_tmpa_int
7700   } > 0
7701   ){
7702     \int_incr:N \l_tmpa_int
7703   }
7704   \int_decr:N \l_tmpa_int
7705   \intarray_gset:Nnn \l__stex_proof_counter_intarray \l_tmpa_int { 0 }
7706 }

sproof (env.)

7707 \bool_set_false:N \l__stex_proof_in_spfblock_bool
7708
7709 \cs_new_protected:Nn \__stex_proof_begin_proof:nn {\par
7710   \intarray_gzero:N \l__stex_proof_counter_intarray
7711   \intarray_gset:Nnn \l__stex_proof_counter_intarray 1 1
7712   \stex_keys_set:nn{spf}top}{#1}
7713   \_stex_do_for_list:
7714   \stex_if_do_html:T {
7715     \__stex_proof_html_env_proof:
7716   }
7717   \seq_map_inline:Nn \l_stex_fors_seq {
7718     \stex_debug:nn{definiens}{Adding~definiens~to~##1}
7719     \stex_if_do_html:TF{
7720       \STEXinvisible{\hbox{\_stex_add_definiens:nn {##1}{proven}}}}
7721     }{
7722       \_stex_add_definiens:nn {##1}{\STEXinvisible{proven}}
7723     }
7724   }
7725   \stex_if_do_html:F\stex_style_apply:
7726   %\_stex_do_id:
7727   \stex_reactivate_macro:N \subproof
7728   \stex_reactivate_macro:N \spfstep
7729   \stex_reactivate_macro:N \conclude
7730   \stex_reactivate_macro:N \assumption
7731   \stex_reactivate_macro:N \eqstep
7732   \stex_reactivate_macro:N \yield
7733   \stex_reactivate_macro:N \spfescape
7734   \stex_reactivate_macro:N \spfblock
7735   \stex_reactivate_macro:N \spfjust
7736   \stex_reactivate_macro:N \spfby
7737   \stex_reactivate_macro:N \spfarg
7738   \noindent\stex_annotate:nn{data-ftml-title={}}{\_stex_annotate_force_break:n{#2}}\par
7739   \stex_if_do_html:TF{
7740     \use:c{stex_annotate_env}{data-ftml-proofbody={}}
7741     \_stex_annotate_force_break:n{}}
7742   ){
7743     \bool_if:NT \l_stex_key_hide_bool{\setbox0\vbox\bgroup}
7744   }
7745 }

7746
7747 \cs_new_protected:Nn \__stex_proof_html_env_proof: {
7748   \exp_args:Nne \use:c{stex_annotate_env}{
7749     data-ftml-proof={}}
7750   \seq_if_empty:NF \l_stex_fors_seq {,

```

```

7751     data-ftml-fors={\seq_map_indexed_function:NN \l_stex_fors_seq \_stex_comma_sep_uri:nn
7752   }
7753   \bool_if:NT \l_stex_key_hide_bool {,
7754     data-ftml-proofhide=true
7755   }
7756 }
7757 \__stex_proof_html:
7758 }
7759
7760 \cs_new_protected:Nn \__stex_proof_html_env_subproof: {
7761   \str_if_empty:NF \l_stex_key_numname_str {
7762     \__stex_proof_number_as_string:N \l__stex_proof_counter_str
7763   }
7764   \exp_args:Nne \use:c{stex_annotate_env}{
7765     data-ftml-subproof={ }
7766     \seq_if_empty:NF \l_stex_fors_seq {,
7767       data-ftml-fors={\seq_map_indexed_function:NN \l_stex_fors_seq \_stex_comma_sep_uri:nn
7768     }
7769     \bool_if:NT \l_stex_key_hide_bool {,
7770       data-ftml-proofhide=true
7771     }
7772     \str_if_empty:NTF \l_stex_key_numname_str {
7773       \str_if_empty:NF \l_stex_key_name_str {,
7774         data-ftml-stepname={\l_stex_key_name_str}
7775       }
7776     }{
7777       , data-ftml-stepname={\l__stex_proof_counter_str}
7778     }
7779   }
7780   \__stex_proof_html:
7781 }
7782
7783 \cs_new_protected:Nn \__stex_proof_html: {
7784   \stex_annotate_force_break:n{ }
7785   \tl_set:N\l_tmpa_tl {\l_stex_key_term_tl\l_stex_key_method_tl\l_stex_key_just_tl}
7786   \tl_if_empty:NF\l_tmpa_tl{
7787     \stex_annotate_invisible:n{\hbox{
7788       \tl_if_empty:NF \l_stex_key_term_tl {
7789         $\stex_annotate:nn{data-ftml-proofterm={}}{\l_stex_key_term_tl}$
7790       }
7791       \tl_if_empty:NF \l_stex_key_just_tl {
7792         $\stex_annotate:nn{data-ftml-spfjust={}}{\l_stex_key_just_tl}$
7793       }
7794       \tl_if_empty:NF \l_stex_key_method_tl {
7795         \stex_annotate:nn{data-ftml-proofmethod={}}{\l_stex_key_method_tl}
7796       }
7797     }}}
7798 }
7799 }
7800
7801 \cs_new_protected:Nn \__stex_proof_start_comment: {
7802   \bool_if:NT \l__stex_proof_in_spfblock_bool {
7803     \par
7804     \group_begin:\stexcommentfont

```



```

7805   }
7806 }
7807 \cs_new_protected:Nn \__stex_proof_end_comment: {
7808   \bool_if:NT \l__stex_proof_in_spfblock_bool {
7809     \par
7810     \group_end:
7811   }
7812 }
7813 \cs_new_protected:Npn \spfescape #1{
7814   \__stex_proof_end_comment: #1 \__stex_proof_start_comment:
7815 }
7816 \stex_deactivate_macro:Nn \spfescape {sproof~environments}
7817
7818 \stex_new_stylable_env:nnnnnnn{proof}{0}{ m}{
7819   \__stex_proof_begin_proof:nn{#1}{#2}
7820   \bool_set_true:N\l__stex_proof_in_spfblock_bool
7821   %\__stex_proof_start_list:n{}
7822   \__stex_proof_start_comment:
7823 }{
7824   \stex_if_do_html:TF{
7825     %\__stex_proof_end_list:
7826     \use:c{endstex_annotate_env}\use:c{endstex_annotate_env}
7827   }\stex_style_apply:
7828 }{
7829   \emph{\sproofautorefname :}~
7830 }{
7831   \sproofend
7832 }{s}
7833
7834 \AddToHook{env/sproof/end}{
7835   \__stex_proof_end_comment:
7836   \stex_if_do_html:F{\bool_if:NT \l_stex_key_hide_bool\egroup}
7837 }
7838
7839
7840 \stex_new_stylable_env:nnnnnnn{proof*}{0}{}{
7841   \__stex_proof_begin_proof:nn{#1}{}
7842   \bool_set_false:N\l__stex_proof_in_spfblock_bool
7843 }{
7844   \stex_if_do_html:TF{\use:c{endstex_annotate_env}\use:c{endstex_annotate_env}}{\stex_style_
7845 }{
7846   \emph{Proof:}~
7847 }{
7848   \sproofend
7849 }{s}
7850
7851 \cs_new_protected:Nn \__stex_proof_start_list:n {
7852   %\par
7853   \group_begin:\list{}{
7854     \setlength\topsep{0pt}
7855     \setlength\parsep{0pt}
7856     \setlength\rightmargin{0pt}
7857   }\item[#1]
7858 }

```

```

7859
7860 \cs_new_protected:Nn \__stex_proof_end_list: {
7861   %\par
7862   \endlist\group_end:
7863 }

```

subproof (*env*.)

```

7864 \str_set_eq:NN \subproofautorefname \spfstepautorefname
7865
7866 \cs_new_protected:Nn \__stex_proof_do_after_subproof:N {
7867   \bgroup
7868   \expandafter \__stex_proof_do_after_subproof_i:nn #1
7869   \egroup
7870   \__stex_proof_do_spf_name_i:
7871 }
7872 \cs_new_protected:Nn \__stex_proof_do_after_subproof_i:nn {
7873   \exp_args:Nno \stex_keys_set:nn{subproof}{#1}
7874   \str_gset_eq:NN \l__stex_proof_key_name_str \l_stex_key_name_str
7875   \str_gset_eq:NN \l__stex_proof_key_numname_str \l_stex_key_numname_str
7876   \bool_gset_eq:NN \l__stex_proof_key_showname_bool \l_stex_key_showname_bool
7877   \str_gset:Nn \l__stex_proof_counter_str {#2}
7878 }
7879 \cs_new_protected:Npn \__stex_proof_set_after_subproof:n {
7880   \str_if_empty:NTF \l_stex_key_numname_str {
7881     \str_if_empty:NTF \l_stex_key_name_str \use_none:n \__stex_proof_set_after_subproof_i:n
7882   }\__stex_proof_set_after_subproof_i:n
7883 }
7884 \cs_new_protected:Npn \__stex_proof_set_after_subproof_with_number:n {
7885   \str_if_empty:NTF \l_stex_key_numname_str {
7886     \str_if_empty:NTF \l_stex_key_name_str \use_none:n \__stex_proof_set_after_subproof_wit
7887   }\__stex_proof_set_after_subproof_with_number_i:n
7888 }
7889 \cs_new_protected:Nn \__stex_proof_set_after_subproof_i:n {
7890   \tl_gset:cn{subproof_arg_ \int_use:N \currentgrouplevel}{#{1}{}}
7891
7892   \__stex_proof_set_aftergroup: % \bool_if:NTF \l__stex_proof_in_spfblock_bool \__stex_proo
7893 }
7894 \cs_new_protected:Nn \__stex_proof_set_after_subproof_with_number_i:n {
7895   \__stex_proof_number_as_string:N \l__stex_proof_counter_str
7896   \tl_gset:ce{subproof_arg_ \int_use:N \currentgrouplevel}{\exp_not:n{#1}}{\l__stex_proof_
7897   \__stex_proof_set_aftergroup:
7898 }
7899 \cs_new_protected:Nn \__stex_proof_set_aftergroup: {
7900   \aftergroup \__stex_proof_do_after_subproof:N
7901   \expandafter \aftergroup \csname subproof_arg_ \int_use:N \currentgrouplevel\endcsname
7902 }
7903
7904 \stex_new_stylable_env:nnnnnn{subproof}{s O{} m}{
7905   \stex_keys_set:nn{subproof}{#2}
7906   \stex_do_for_list:
7907   \stex_if_do_html:T {
7908     \__stex_proof_html_env_subproof:
7909   }
7910   \seq_map_inline:Nn \l_stex_fors_seq {

```

```

7911 \stex_debug:nn{definiens}{Adding-definiens-to-##1}
7912 \stex_if_do_html:TF{
7913   \STEXinvisible{\hbox{\_stex_add_definiens:nn {##1}{proven}}}}
7914 }{
7915   \_stex_add_definiens:nn {##1}{\STEXinvisible{proven}}
7916 }
7917 }
7918
7919 \IfBooleanTF #1 {
7920   \bool_set_false:N \l__stex_proof_in_spfblock_bool
7921   \__stex_proof_set_after_subproof:n{#2}
7922   \stex_if_do_html:F \stex_style_apply:
7923   \str_if_empty:NF \l_stex_key_id_str {
7924     \__stex_proof_number_as_string:N \@currentlabel
7925     %\str_set:Ne \@currentHref{subproof.\@currentlabel}
7926     %\_stex_do_id:
7927   }
7928
7929
7930   \stex_annotate:nn{data-ftml-title={}}{#3}
7931 }{
7932   \bool_if:NTF \l__stex_proof_in_spfblock_bool {
7933
7934     \__stex_proof_set_after_subproof_with_number:n{#2}
7935
7936     %\str_if_empty:NF \l_stex_key_id_str {
7937       %\_stex_proof_number_as_string:N \@currentlabel
7938       %\str_set:Ne \@currentHref{subproof.\@currentlabel}
7939       %\_stex_do_id:
7940     %}
7941     \use:c{stex_annotate_env}{data-ftml-title={}} \__stex_proof_start_list:n{\__stex_proo
7942       \__stex_proof_add_counter:
7943       \stex_if_do_html:F \stex_style_apply:
7944     }{
7945       \noindent \stex_annotate:nn{data-ftml-title={}}{#3} \par
7946       \stex_if_do_html:F \stex_style_apply:
7947
7948       \__stex_proof_set_after_subproof:n{#2}
7949
7950       %\_stex_do_id:
7951     }
7952   }
7953   \stex_if_do_html:TF{
7954     \use:c{stex_annotate_env}{data-ftml-proofbody={}}
7955     \_stex_annotate_force_break:n{
7956   }{\bool_if:NT \l_stex_key_hide_bool{\setbox0\vbox\bgroup}}
7957   \__stex_proof_start_comment:
7958 }{
7959   \bool_if:NT\l__stex_proof_in_spfblock_bool {
7960     \__stex_proof_remove_counter:
7961     \__stex_proof_inc_counter:
7962   }
7963
7964   \stex_if_do_html:TF{

```

```

7965     \use:c{endstex_annotate_env} %proofbody
7966     \bool_if:NT\l__stex_proof_in_spfblock_bool \__stex_proof_end_list:
7967     \use:c{endstex_annotate_env} % subproof
7968   }{
7969     \stex_style_apply:
7970     \bool_if:NT \l__stex_proof_in_spfblock_bool \__stex_proof_end_list:
7971   }
7972 }{}{}{}
7973
7974 \AddToHook{env/subproof/before}{
7975   \__stex_proof_end_comment:
7976 }
7977
7978 \AddToHook{env/subproof/after}{
7979   \__stex_proof_start_comment:
7980 }
7981
7982 \AddToHook{env/subproof/end}{
7983   \__stex_proof_end_comment:
7984   \stex_if_do_html:F{\bool_if:NT \l_stex_key_hide_bool\egroup}
7985 }
7986
7987 \stex_deactivate_macro:Nn \subproof {sproof~environments}
7988

```

spfsketchenv (env.) TODO:

```

7989 \newenvironment{spfsketchenv}{}{}

\spfsketch
7990 \stex_new_stylable_cmd:nnnn{spfsketch}{0}{ m}{\par
7991   \begin{spfsketchenv}
7992   \stex_keys_set:nn{spftop}{#1}
7993   \_stex_do_for_list:
7994   \_stex_do_id:
7995   \par
7996   \exp_args:Nne \use:c{stex_annotate_env}{
7997     data-ftml-proofsketch={
7998       \seq_if_empty:NF \l_stex_fors_seq {
7999         \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
8000       }
8001     }
8002   }
8003   \stex_style_apply:
8004   #2\par
8005   \use:c{endstex_annotate_env}
8006 \end{spfsketchenv}
8007 }{
8008   \noindent\emph{\spfsketchenvautorefname :}~
8009 }

```

(End of definition for \spfsketch. This function is documented on page 158.)

sfunction

Keys for proof step macros:

```

8010 \stex_keys_define:nnnn { spfsteps } {

```

```

8011 %\str_clear:N \l_stex_key_name_str
8012 }{
8013 %name .str_set_x:N = \l_stex_key_name_str
8014 }{spfcommon,spfnamed}

Common setup for all the proof step macros:

8015 \cs_new_protected:Nn \__stex_proof_make_step_macro:Nnnnn {
8016 \NewDocumentCommand #1 {s O{ } +m} {
8017 \__stex_proof_end_comment:
8018 \stex_keys_set:nn{spfsteps}{##2}
8019 %\str_if_empty:NF \l_stex_key_name_str {
8020 % \exp_args:Nne\use:nn{\vardef}{\v\l_stex_key_name_str}[name=\l_stex_key_name_str]{\l_
8021 %}
8022 \__stex_proof_do_spf_name:
8023
8024 \spfstepenv
8025 \str_if_empty:NF \l_stex_key_id_str {
8026 %\__stex_proof_number_as_string:N \@currentlabel
8027 %\str_set:Ne \@currentHref{spfstep.\@currentlabel}
8028 %\__stex_do_id:
8029 }
8030
8031 \bool_if:NTF \l__stex_proof_in_spfblock_bool {
8032 \IfBooleanTF ##1 {
8033 \__stex_proof_step_html_i:nn{#2}{##3}
8034 }{
8035 \__stex_proof_step_html:nn{#2}{\__stex_proof_start_list:n{#3} ##3 \__stex_proof_end
8036 #5
8037 }
8038 \endspfstepenv
8039 \__stex_proof_start_comment:
8040 }{
8041 \__stex_proof_step_html_i:nn{#2}{##3}
8042 \endspfstepenv
8043 }
8044 }
8045 \stex_deactivate_macro:Nn #1 {sproof~environments}
8046 }

8047
8048 \cs_new_protected:Nn \__stex_proof_step_html:nn {
8049 \str_if_empty:NF \l_stex_key_numname_str {
8050 \__stex_proof_number_as_string:N \l__stex_proof_counter_str
8051 }
8052 \stex_if_do_html:TF{
8053 %\group_begin:
8054 \exp_args:Nne \use:c{stex_annotate_env}{
8055 data-ftml-spf#1={
8056 \seq_if_empty:NF \l_stex_fors_seq {
8057 \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
8058 }
8059 }
8060 \str_if_empty:NTF \l_stex_key_numname_str {
8061 \str_if_empty:NF \l_stex_key_name_str {,
8062 data-ftml-stepname={\l_stex_key_name_str}
8063 }

```

```

8064     }{
8065     , data-ftml-stepname={\l__stex_proof_counter_str}
8066     }
8067   }
8068   \__stex_proof_html:
8069   #2
8070   \par
8071   \use:c{endstex_annotate_env}
8072   %\group_end:
8073   }{ #2 }
8074 }
8075 \cs_new_protected:Nn \__stex_proof_step_html_i:nn {
8076   \stex_if_do_html:TF{
8077     \noindent
8078     %\group_begin:
8079     \exp_args:Ne \stex_annotate:nn{
8080       data-ftml-spf#1={
8081         \seq_if_empty:NF \l_stex_fors_seq {
8082           \seq_map_function:NN \l_stex_fors_seq \stex_use_symbol_uri:N
8083         }
8084       }
8085       \str_if_empty:NTF \l_stex_key_numname_str {
8086         \str_if_empty:NF \l_stex_key_name_str {,
8087           data-ftml-stepname={\l_stex_key_name_str}
8088         }
8089       }{
8090         , data-ftml-stepname={\l__stex_proof_counter_str}
8091       }
8092     }{
8093       \__stex_proof_html:
8094       #2
8095     }
8096     %\group_end:
8097   }{ #2 }
8098 }

```

spfstepenv (env.) TODO:

```

8099 \newenvironment{spfstepenv}{
8100   \str_set_eq:NN \spfstepenvautorefname \spfstepautorefname
8101 }{}

```

spfblock (env.)

```

8102 \NewDocumentEnvironment{spfblock}{}{
8103   \bool_set_false:N \l__stex_proof_in_spfblock_bool
8104 }{
8105   \aftergroup\__stex_proof_start_comment:
8106 }
8107
8108 \stex_deactivate_macro:Nn \spfblock {sproof~environments}
8109 \AddToHook{env/spfblock/before}{
8110   \__stex_proof_end_comment:
8111 }

```

\spfstep

```

8112 \__stex_proof_make_step_macro:Nnnnn \spfststep {step} {\__stex_proof_insert_number:\__stex_pr
(End of definition for \spfststep. This function is documented on page 158.)
sfunction

\assumption
8113 \__stex_proof_make_step_macro:Nnnnn \assumption {assumption} {\__stex_proof_insert_number:\
(End of definition for \assumption. This function is documented on page 158.)
sfunction

\conclude
8114 \__stex_proof_make_step_macro:Nnnnn \conclude {conclusion} {\$ \Rightarrow$} {} {}
(End of definition for \conclude. This function is documented on page 158.)
sfunction

\eqstep
8115 \NewDocumentCommand \eqstep {s m}{
8116 \__stex_proof_end_comment:
8117 \spfststepenv
8118
8119 \bool_if:NTF \l__stex_proof_in_spfblock_bool {
8120 \IfBooleanTF #1 {
8121 \__stex_proof_step_html_i:nn{eqstep}{\$= #2$}
8122 }{
8123 \__stex_proof_step_html:nn{eqstep}{\__stex_proof_start_list:n{\$= $} $#2$ \__stex_proof
8124 }
8125 \endspfststepenv
8126 \__stex_proof_start_comment:
8127 }{
8128 \__stex_proof_step_html_i:nn{eqstep}{\$= #2$}
8129 \endspfststepenv
8130 }
8131 }
8132 \stex_deactivate_macro:Nn \eqstep {sproof~environments}
(End of definition for \eqstep. This function is documented on page 158.)
sfunction

\yield
8133 \NewDocumentCommand \yield {+m}{
8134 \stex_annotate:nn{data-ftml-proofterm={}}{ #1 }
8135 }
8136 \stex_deactivate_macro:Nn \yield {sproof~environments}
(End of definition for \yield. This function is documented on page 158.)
sfunction

\spfjust
8137 \newcommand\spfjust[1]{
8138 \stex_annotate:nn{data-ftml-spfjust={}}{ #1 }
8139 }
8140 \stex_deactivate_macro:Nn \spfjust {sproof~environments}

```

(End of definition for `\spfjust`. This function is documented on page 158.)
sfunction

`\spfby`

```

8141 \NewDocumentCommand\spfby{s}{
8142   \IfBooleanTF #1 {
8143     \stex_annotate_invisible:nn{data-ftml-proofmethod={}}
8144   }{
8145     \stex_annotate:nn{data-ftml-proofmethod={}}
8146   }
8147 }
8148 \stex_deactivate_macro:Nn \spfby {sproof-environments}

```

(End of definition for `\spfby`. This function is documented on page 158.)
sfunction

`\spfarg`

```

8149 \NewDocumentCommand\spfarg{s O{}}{
8150   \IfBooleanTF #1 {
8151     \stex_annotate_invisible:nn{data-ftml-spfarg={#2}}
8152   }{
8153     \stex_annotate:nn{data-ftml-spfarg={#2}}
8154   }
8155 }
8156 \stex_deactivate_macro:Nn \spfarg {sproof-environments}

```

(End of definition for `\spfarg`. This function is documented on page 159.)
sfunction

`\sproofend`

```

8157 \tl_set:Nn \__stex_proof_proof_box_tl {
8158   \ltx@ifpackageloaded{amssymb}{\square$}{
8159     \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
8160   }
8161 }
8162
8163 \tl_set:Nn \sproofend {
8164   \tl_if_empty:NF \l_stex_key_proofend_tl {
8165     \hfil\hfill\hfill\l_stex_key_proofend_tl\par\smallskip
8166   }
8167 }

```

(End of definition for `\sproofend`. This function is documented on page 159.)
sfunction

`\stexcommentfont`

```

8168 \cs_new_protected:Npn \stexcommentfont {
8169   \small\itshape
8170 }

```

(End of definition for `\stexcommentfont`. This function is documented on page 159.)
sfunction

Chapter 37

Metatheory

```
8171 <@@=stex_meta>

      Setup:
8172 \group_begin:
8173 \cs_set:Npn \__stex_modules_persist_module: {}
8174 \cs_set:Npn \stex_check_term:nn #1 #2 {}
8175 \cs_set:Npn \stex_sref_do_aux:n #1 { #1 }
8176 \bool_set_false:N \c_stex_check_terms_bool
8177 \bool_set_false:N \l__stex_annotate_do_output_bool
8178 \stex_set_meta_archive:
8179 \tl_set:Nx \l_stex_current_document_uri {
8180   {\tl_to_str:n{FTML/meta}}{}
8181   {\tl_to_str:n{Metatheory}}{\tl_to_str:n{en}}{}
8182 }
8183 \stex_module_setup_top_nosig:n{Metatheory}
8184 \g_stex_every_module_tl

8185
8186 \symdef{typeof}[name=type~of,args=1]{\maincomp{\mathrm{tp}}\dobrackets{#1}}
8187
8188 \symdef{commented}[args=2]{#2 \; (#1)}
8189 \notation{commented}[inv]{#2}
8190
8191 \symdef{of~type}[args=ii,invisible]{#1}
8192 \notation{of~type}[colon]{#1 \mathbin{\comp{:}} #2}
8193
8194 \symdef{apply}[args=ia,prec=0;\infpref x\infpref]{#1\mathopen{\comp{}} #2 \mathclose{\comp{}}}
8195 \notation{apply}[lambda]{#1\; \argsep{#2}{\;}}
8196 \notation{apply}[infixop]{\argsep{#2}{\mathbin{#1}}\STEXinvisible{#1}}
8197 \notation{apply}[infixrel]{\argsep{#2}{\mathrel{#1}}\STEXinvisible{#1}}
8198
8199 \symdef{autoprove}[name=prove automatically]{\mathrm{automatically\ proved}}
8200
8201 % structures
8202 \symdef{module~type}[args=i,op=\mathtt{MOD}]
8203   {\mathopen{\comp{\mathtt{MOD}}}{}#1\mathclose{\comp{}}}
8204 \symdef{module~type~merge}[args=a,op=\oplus]
8205   {\argsep{#1}{\mathbin{\comp{\oplus}}}}
8206 \symdef{anonymous~record}[args=a]
```

```

8207   {\mathopen{\comp{[[]}\#1\mathclose{\comp{[]}}}}
8208 \symdef{record~field}[args=2]{\#1\comp{.}\#2}
8209 \symdecl*{record~type}
8210
8211 \symdecl{mathstruct}[name=mathematical~structure,args=a] % TODO
8212 \notation{mathstruct}[angle,prec=nobrackets]
8213   {\mathopen{\comp{\langle} \#1 \mathclose{\comp{\rangle}}}
8214 \notation{mathstruct}[parens,prec=nobrackets]
8215   {\mathopen{\comp{() \#1 \mathclose{\comp{}}}
8216
8217 % sequences
8218 \symdef{ellipses}[ldots]{\ldots}
8219 \symdef{sequence~expression}[comma,args=a]{\#1}
8220 \symdef{sequence~type}[args=1]{\#1^{\comp{ast}}}
8221 \symdef{ranged~sequence~type}[args=ia]{\#1\c_math_subscript_token{\dobrackets{\#2}}}
8222 \symdef{sequence~range}[inv,args=a]{\#1}
8223 \symdef{sequence~map}[args=ai]{
8224   \comp{\mathrm{map}}\mathopen{\comp{()}\#1\mathpunct{\comp{,}}}
8225   \#2\mathclose{\comp{()}}
8226 }
8227 \symdef{fold~right}[args=aibbi]{
8228   \maincomp{\mathrm{fold}}\dobrackets{\#2\mathpunct{\comp{;}}\#1\mathpunct{\comp{;}}
8229   \dobrackets{\#3,\#4} \mathrel{\comp{\mapsto}} \#5
8230 }
8231 }
8232
8233 \symdef{fold}[args=abbi]{
8234   \maincomp{\mathrm{fold}}\dobrackets{\#1\mathpunct{\comp{;}}
8235   \dobrackets{\#2,\#3} \mathrel{\comp{\mapsto}} \#4
8236 }
8237 }
8238
8239 \symdecl{bind}[args=Bi,assoc=pre]
8240 \notation{bind}[defun,prec=nobrackets,op=(\cdot)\;\to\;\cdot]
8241   {\mathopen{\comp{() \#1 \mathclose{\comp{()}\mathbin{\comp{\to}}}} \#2}
8242 \notation{bind}[forall]{\comp{\forall} \#1.\;\#2}
8243 \notation{bind}[Pi]{\mathop{\comp{\prod}}\c_math_subscript_token{\#1}\#2}
8244
8245 \symdef{implicit~bind}[args=Bi,assoc=pre]{\mathopen{\comp{\}} \#1 \mathclose{\comp{\}}\c_math_
8246 \symdef{apply~implicit}[args=ia,prec=nobrackets]{
8247   \underbrace{\#1}_{\#2}
8248 }
8249
8250 \symdecl{arrow}[args=ai,assoc=pre]
8251 \notation{arrow}[single]{
8252   \argsep{\#1}{\mathbin{\comp{\times}}}
8253   \mathrel{\maincomp{\to}} \#2
8254 }
8255 \notation{arrow}[double]{
8256   \argsep{\#1}{\mathbin{\comp{\times}}}
8257   \mathrel{\maincomp{\Rrightarrow}} \#2
8258 }
8259
8260 \symdef{intsecttp}[intersection type,args=a,sqcap,prec=0;-10]{ \argsep{\#1}{\maincomp{\mathr

```

```

8261
8262 \symdef{bindin}[args=Bi]{ \mathop{\maincomp{\amalg}}_{\#1}\#2 }
8263
8264 \symdecl*{integer~literal}
8265 \notation{integer~literal}{\mathbb Z}
8266
8267 \symdecl*{ordinal}
8268 \notation{ordinal}{\mathtt{Ord}}
8269
8270 % propositions
8271 \symdef{prop}[name=proposition]{\mathtt{Prop}}
8272 \symdef{judgment~holds}[args=i,role=judgment]{\comp\vdash\;#1}
8273
8274 % any object
8275 \symdef{object}{\mathtt{Obj}}
8276
8277 \symdef{letin}[name=let in,args=Bi,role=let]{\maincomp{\mathrm{let}}\ #1\ \comp{\mathrm{in}}}
8278
8279 % TODO DELETE
8280 \symdef{aseqdots}[args=a,prec=nobrackets]
8281   {\#1\comp{\, \ldots}}\%{\##1\comp,##2}
8282 \symdef{aseqfromto}[args=ai,prec=nobrackets]
8283   {\#1\comp{\, \ldots,}\#2}\%{\##1\comp,##2}
8284 \symdef{aseqfromtovia}[args=aai,prec=nobrackets]
8285   {\#1\comp{\, \ldots,}\#2\comp{\, \ldots,}\#3}\%{\##1\comp,##2}

Closing:
8286 \global \let \l_stex_metatheory_uri \l_stex_current_module_uri
8287 \global \let \c_stex_default_metatheory \l_stex_metatheory_uri
8288 \stex_close_module:
8289 \group_end:

```

Chapter 38

Others

```
8290 <@@=stex_others>
8291
8292 \cs_new_protected:Npn \MSC #1 {}
8293
8294 \_stex_persist_read_now:
8295 \cs_new_protected:Nn \__stex_others_newlabel:n {
8296   \tl_gset:cn{__stex_sref_aux_sym_ \tl_to_str:n{#1}}{X}
8297   \exp_args:Nc\__stex_others_old_newlabel:{\tl_to_str:n{#1}}
8298 }
8299 \AtBeginDocument{
8300   \iow_now:Nn \@auxout {
8301     \ExplSyntaxOn
8302     \let\__stex_others_old_newlabel:\newlabel
8303     \let\newlabel\__stex_others_newlabel:n
8304     \ExplSyntaxOff
8305   }
8306 }

Inference Rules
8307 \NewDocumentCommand \FTMLrule {m m}{
8308   \tl_if_empty:nTF{#2}{
8309     \seq_clear:N \l_tmpa_seq
8310   }{
8311     \seq_set_split:Nnn \l_tmpa_seq , {#2}
8312   }
8313   \int_zero:N \l_tmpa_int
8314   \stex_annotate_invisible:n{\hbox{
8315     $
8316     \stex_annotate:nn{data-ftml-inferencerule={#1}}{
8317       \_stex_annotate_force_break:n{~
8318         \seq_if_empty:NF \l_tmpa_seq {
8319           \seq_map_inline:Nn \l_tmpa_seq {
8320             \int_incr:N \l_tmpa_int
8321             \stex_annotate:nn{
8322               data-ftml-argmode=i,
8323               data-ftml-arg={\int_use:N \l_tmpa_int}
8324             }{ ##1 }
8325           }
8326         }
8327       }
8328     }
8329   }
8330 }
```

```

8327         ~}
8328     }$
8329 }}
8330 }
8331 \stex_deactivate_macro:Nn \FTMLrule {module~environments}
8332 \stex_every_module:n{\stex_reactivate_macro:N \FTMLrule}

```

Chapter 39

Additional Packages

39.1 Implementation: The notesslides Package

39.1.1 Class and Package Options

We define some Package Options and switches for the `notesslides` class and activate them by passing them on to `beamer.cls` and `omdoc.cls` and the `notesslides` package. We pass the `nontheorem` option to the `statements` package when we are not in notes mode, since the `beamer` package has its own (overlay-aware) theorem environments.

```
8333 \*cls)
8334 \@@=notesslides)
8335 \ProvidesExplClass{notesslides}{2026/06/24}{4.1.0}{notesslides Class}
8336 \RequirePackage{13keys2e}
8337
8338 \str_const:Nn \c__notesslides_class_str {article}
8339
8340 \keys_define:nn{notesslides / cls}{
8341   class .str_set_x:N = \c__notesslides_class_str,
8342   notes .bool_set:N = \c__notesslides_notes_bool ,
8343   slides .code:n      = { \bool_set_false:N \c__notesslides_notes_bool },
8344   %docopt .str_set_x:N = \c__notesslides_docopt_str,
8345   unknown .code:n      = {
8346     \PassOptionsToClass{\CurrentOption}{beamer}
8347     \PassOptionsToClass{\CurrentOption}{\c__notesslides_class_str}
8348     \PassOptionsToPackage{\CurrentOption}{notesslides}
8349     \PassOptionsToPackage{\CurrentOption}{stex}
8350   }
8351 }
8352 \ProcessKeysOptions{ notesslides / cls }
8353
8354 \RequirePackage{stex}
8355 \stex_if_html_backend:T {
8356   \bool_set_true:N \c__notesslides_notes_bool
8357 }
8358
8359 \bool_if:NTF \c__notesslides_notes_bool {
8360   \PassOptionsToPackage{notes=true}{notesslides}
8361   \message{notesslides.cls:~Formatting~document~in~notes~mode}
```

```

8362 }{
8363   \PassOptionsToPackage{notes=false}{notesslides}
8364   \message{notesslides.cls:~Formatting~document~in~slides~mode}
8365 }
8366
8367 \bool_if:NTF \c_notesslides_notes_bool {
8368   \LoadClass{\c_notesslides_class_str}
8369 }{
8370   \LoadClass[10pt,notheorems,xcolor={dvipsnames,svgnames}]{beamer}
8371   %\newcounter{Item}
8372   %\newcounter{paragraph}
8373   %\newcounter{subparagraph}
8374   %\newcounter{Hfootnote}
8375 }
8376 \RequirePackage{notesslides}
8377 </cls>

```

now we do the same for the notesslides package.

```

8378 (*package)
8379 \ProvidesExplPackage{notesslides}{2026/06/24}{4.1.0}{notesslides Package}
8380 \RequirePackage{l3keys2e}
8381
8382 \keys_define:nn{notesslides / pkg}{
8383   notes .bool_set:N = \c_notesslides_notes_bool ,
8384   slides .code:n = { \bool_set_false:N \c_notesslides_notes_bool },
8385   sectocframes .bool_set:N = \c_notesslides_sectocframes_bool ,
8386   topsect .str_set_x:N = \c_notesslides_topsect_str,
8387   unknown .code:n = {
8388     \PassOptionsToPackage{\CurrentOption}{stex}
8389     \PassOptionsToPackage{\CurrentOption}{tikzinput}
8390   }
8391 }
8392 \ProcessKeysOptions{ notesslides / pkg }
8393
8394 \RequirePackage{stex}
8395 \stex_if_html_backend:T {
8396   \bool_set_true:N \c_notesslides_notes_bool
8397 }
8398
8399 \cs_set:Npn \sectiontitleemph #1 {
8400   \textbf{\Large #1}
8401 }
8402
8403 \newif\ifnotes
8404 \bool_if:NTF \c_notesslides_notes_bool {
8405   \notesttrue
8406   \PassOptionsToPackage{dvipsnames,svgnames}{xcolor}
8407   \RequirePackage[noamsthm,hyperref]{beamerarticle}
8408   \RequirePackage{mdframed}
8409   \str_if_empty:NTF \c_notesslides_topsect_str{
8410     %\setsectionlevel{section}
8411   } {
8412     \exp_args:No \setsectionlevel \c_notesslides_topsect_str
8413   }

```

```

8414 }{
8415   \notesfalse
8416
8417   \cs_new_protected:Nn \__notesslides_do_sectocframes: {
8418     \cs_set_protected:Nn \__notesslides_do_label:n {
8419       \str_case:nnF{##1}{
8420         {part} {
8421           \tl_set:Nx\l__notesslides_num{\thepart}
8422           \tl_set:cx{@ @ label}{
8423             \cs_if_exist:NTF\parttitlename{\exp_not:N\parttitlename}{\exp_not:N\partname}{\exp_not:N\partname}
8424           }
8425         {chapter} {
8426           \tl_set:Nx\l__notesslides_num{\thechapter}
8427           \tl_set:cx{@ @ label}{
8428             \cs_if_exist:NTF\chaptertitlename{\exp_not:N\chaptertitlename}{\exp_not:N\chaptername}{\exp_not:N\chaptername}
8429           }
8430         {section} {
8431           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8432           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8433         }
8434         {subsection} {
8435           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8436           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8437         }
8438         {subsubsection} {
8439           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8440           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8441         }
8442         {paragraph} {
8443           \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection}
8444           \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8445         }
8446       }{
8447         \tl_set:Nx\l__notesslides_num{\cs_if_exist:NT\thechapter{\thechapter.}\thesection.\l__notesslides_num}
8448         \tl_set:cx{@ @ label}{\l__notesslides_num\quad}
8449       }
8450     }
8451     \cs_set_protected:Nn \_sfragment_do_level:nn {
8452       \tl_if_exist:cT{c@##1}{\stepcounter{##1}}
8453       \addcontentsline{toc}{##1}{\protect\numberline{\use:c{the##1}}{##2}}
8454       \__notesslides_do_label:n{##1}
8455       \pdfbookmark[\int_use:N \l_stex_current_section_level_int]{\l__notesslides_num\ ##2}{\l__notesslides_num}
8456       \begin{frame}[noframenumbering]
8457         \vfill\centering
8458         \sectiontitleemph{
8459           \use:c{@ @ label} ##2
8460         }
8461       \end{frame}
8462       \int_incr:N \l_stex_current_section_level_int
8463       \str_set:Nn \l_stex_current_section_level_str{##1}
8464     }
8465   }
8466
8467   \cs_set_protected:Nn \_stex_titlefragment: {

```



```

8468     \begin{frame}[noframenumbering]\maketitle\end{frame}
8469     \let\maketitle\relax
8470 }
8471
8472 \AtBeginDocument{
8473   \str_if_empty:NTF \c_notesslides_topsect_str {
8474     \setsectionlevel{section}
8475   } {
8476     \exp_args:No \setsectionlevel \c_notesslides_topsect_str
8477     \exp_args:No \str_if_eq:nnTF \c_notesslides_topsect_str {chapter} {
8478       \__notesslides_define_chapter:
8479     }{
8480       \exp_args:No \str_if_eq:nnT \c_notesslides_topsect_str {part} {
8481         \__notesslides_define_chapter:
8482         \__notesslides_define_part:
8483       }
8484     }
8485   }
8486 }
8487
8488 \bool_if:NT \c_notesslides_sectocframes_bool {
8489   \__notesslides_do_sectocframes:
8490 }
8491 }
8492
8493 \cs_new_protected:Nn \__notesslides_define_chapter: {
8494   \cs_if_exist:NF \chaptername {
8495     \cs_set_protected:Npn \chaptername {Chapter}
8496   }
8497   \cs_if_exist:NF \chapter {
8498     \cs_set_protected:Npn \chapter {INVALID}
8499   }
8500   \cs_if_exist:NF \c@chapter {
8501     \newcounter{chapter}\counterwithin*{section}{chapter}
8502   }
8503 }
8504
8505 \cs_new_protected:Nn \__notesslides_define_part: {
8506   \cs_if_exist:NF \partname {
8507     \cs_set_protected:Npn \partname {Part}
8508   }
8509   \cs_if_exist:NF \part {
8510     \cs_set_protected:Npn \part {INVALID}
8511   }
8512   \cs_if_exist:NF \c@part {
8513     \newcounter{part}\counterwithin*{chapter}{part}
8514   }
8515 }

```

`\prematurestop` We initialize `\afterprematurestop`, and provide `\prematurestop@endsfragment` which looks up `\sfragment@level` and recursively ends enough `{sfragment}s`.

```

8516 \def \c__notesslides_document_str{document}
8517 \newcommand\afterprematurestop{}
8518 \def\prematurestop@endsfragment{

```

```

8519 \unless\ifx\@currenvir\c__notesslides_document_str
8520 \expandafter\expandafter\expandafter\end\expandafter\expandafter\expandafter{\expandafter
8521 \expandafter\prematurestop@endsfragment
8522 \fi
8523 }
8524 \providecommand\prematurestop{
8525 \stex_if_html_backend:F{
8526 \message{Stopping~sTeX~processing~prematurely}
8527 \prematurestop@endsfragment
8528 \afterprematurestop
8529 \end{document}}
8530 }
8531 }

```

(End of definition for \prematurestop. This function is documented on page 111.)

39.1.2 Notes and Slides

For the notes case, we also provide the \usetheme macro that would otherwise come from the the `beamer` class.

```

8532 \bool_if:NT \c_notesslides_notes_bool {
8533 \renewcommand\usetheme[2][\usepackage[#1]{beamertheme#2}]
8534 }
8535 \NewDocumentCommand \libusetheme {0{} m} {
8536 \libusepackage[#1]{beamertheme#2}
8537 }

```

We define the sizes of slides in the notes. Somehow, we cannot get by with the same here.

```

8538 \newlength{\slidewidth}\setlength{\slidewidth}{13.5cm}
8539 \newlength{\slideheight}\setlength{\slideheight}{9cm}

```

We first set up the slide boxes in `notes` mode. We set up sizes and provide a box register for the frames and a counter for the slides.

```

8540 \ifnotes
8541
8542 \newlength{\slideframewidth}
8543 \setlength{\slideframewidth}{1.5pt}

```

frame (env.) We first define the keys.

```

8544 \cs_new_protected:Nn \__notesslides_do_yes_param:Nn {
8545 \exp_args:Nx \str_if_eq:nnTF { \str_uppercase:n{ #2 } }{ yes }{
8546 \bool_set_true:N #1
8547 }{
8548 \bool_set_false:N #1
8549 }
8550 }
8551
8552 \stex_keys_define:nnnn{notesslides / frame}{
8553 \str_clear:N \l__notesslides_frame_label_str
8554 \bool_set_true:N \l__notesslides_frame_allowframebreaks_bool
8555 \bool_set_true:N \l__notesslides_frame_allowdisplaybreaks_bool
8556 \bool_set_true:N \l__notesslides_frame_fragile_bool
8557 \bool_set_true:N \l__notesslides_frame_shrink_bool

```

```

8558 \bool_set_true:N \l__notesslides_frame_squeeze_bool
8559 \bool_set_true:N \l__notesslides_frame_t_bool
8560 }{
8561   label .str_set_x:N = \l__notesslides_frame_label_str,
8562   allowframebreaks .code:n = {
8563     \__notesslides_do_yes_param:Nn \l__notesslides_frame_allowframebreaks_bool { #1 }
8564   },
8565   allowdisplaybreaks .code:n = {
8566     \__notesslides_do_yes_param:Nn \l__notesslides_frame_allowdisplaybreaks_bool { #1 }
8567   },
8568   fragile .code:n = {
8569     \__notesslides_do_yes_param:Nn \l__notesslides_frame_fragile_bool { #1 }
8570   },
8571   shrink .code:n = {
8572     \__notesslides_do_yes_param:Nn \l__notesslides_frame_shrink_bool { #1 }
8573   },
8574   squeeze .code:n = {
8575     \__notesslides_do_yes_param:Nn \l__notesslides_frame_squeeze_bool { #1 }
8576   },
8577   t .code:n = {
8578     \__notesslides_do_yes_param:Nn \l__notesslides_frame_t_bool { #1 }
8579   },
8580   unknown .code:n = {}
8581 }{}

```

We redefine the `itemize` environment so that it looks more like the one in `beamer`.

```

8582 \cs_new_protected:Nn \__notesslides_setup_itemize: {
8583   \def\itemize@level{outer}
8584   \def\itemize@outer{outer}
8585   \def\itemize@inner{inner}
8586   %\newcommand\metakeys@show@keys[2]{\marginnote{\scriptsize ##2}}
8587   \renewenvironment{itemize}{
8588     \ifx\itemize@level\itemize@outer
8589       \def\itemize@label{$\rhd$}
8590     \fi
8591     \ifx\itemize@level\itemize@inner
8592       \def\itemize@label{$\scriptstyle\rhd$}
8593     \fi
8594     \begin{list}
8595     {\itemize@label}
8596     {\setlength{\labelsep}{.3em}
8597      \setlength{\labelwidth}{.5em}
8598      \setlength{\leftmargin}{1.5em}
8599     }
8600     \edef\itemize@level{\itemize@inner}
8601   }{
8602     \end{list}
8603   }
8604 }

```

We create the box with the `mdframed` environment from the `equinymous` package.

```

8605 \stex_if_html_backend:TF {
8606   %\stex_css_literal:n{
8607   % .stex-frame {

```

```

8608 % background-color:#fff;
8609 % margin:5px ~ 0;
8610 % border:2px ~ solid ~ #000;
8611 % border-radius:5px;
8612 % padding:5px;
8613 % padding-right:10px;
8614 % width: var(--rustex-curr-width);
8615 % display:block;
8616 % }
8617 %}

8618
8619 \cs_new_protected:Nn \__notesslides_frame_box_begin: {
8620   \vbox\bgroup
8621   \begin{stex_annotate_env}{data-ftml-slide={}}%{class:stex-frame={}}
8622     \mdf@patchamsthm\notesslidesfont
8623 }
8624 \cs_new_protected:Nn \__notesslides_frame_box_end: {
8625   %^A \notesslides@slidelabel
8626   \medskip\par\hbox to \textwidth{\tiny\notesslidesfooter}
8627   \end{stex_annotate_env}\egroup
8628 }
8629 }{
8630   \cs_new_protected:Nn \__notesslides_frame_box_begin: {
8631     \begin{mdframed}[
8632       linewidth=\slideframewidth,
8633       skipabove=1ex,
8634       skipbelow=1ex,
8635       userdefinedwidth=\slidewidth,
8636       align=center,
8637       %usetwoside=false
8638     ]\notesslidesfont
8639   }
8640   \cs_new_protected:Nn \__notesslides_frame_box_end: {
8641     \medskip\par\noindent\tiny\notesslidesfooter%^A\notesslides@slidelabel
8642     \end{mdframed}
8643   }
8644 }

```

We define the environment, read them, and construct the slide number and label.

```

8645 \renewenvironment{frame}[1][]{
8646   \stex_keys_set:nn{notesslides / frame}{#1}
8647   \stepcounter{framenum}
8648   \renewcommand\newpage{\addtocounter{framenum}{1}}
8649   \def\@currentlabel{\theframenum}
8650   \str_if_empty:NF \l__notesslides_frame_label_str {
8651     \label{\l__notesslides_frame_label_str}
8652   }
8653   \__notesslides_setup_itemize:
8654   \__notesslides_frame_box_begin:
8655 }{
8656   \__notesslides_frame_box_end:
8657 }

```

Now, we need to redefine the frametitle (we are still in course notes mode).

```

8658 \renewcommand{\frametitle}[1]{
8659   \stexdoctitle { #1 }
8660   \stex_if_html_backend:TF{
8661     \notesslideslidesemph{\noindent\stex_annotate:nn{data-ftml-title={}}{#1}}\par
8662   }{
8663     \notesslideslidesemph{#1}
8664   }
8665   \medskip
8666 }

```

\pause

(End of definition for \pause. This function is documented on page ??.)

```

8668 \renewenvironment{columns}[1][ ]{
8669 \par\noindent
8670 \begin{minipage}
8671 \slidewidth\centering\leavevmode
8672 % \stex_if_html_backend:T{
8673 % \cs_if_exist:NT \rustex_if:T {
8674 % \rustex_if:T {\par
8675 % \rustex_direct_HTML:n{<table><tr><td>}
8676 % }
8677 % }
8678 % }
8679 }{
8680 % \stex_if_html_backend:T{
8681 % \cs_if_exist:NT \rustex_if:T {
8682 % \rustex_if:T {\par
8683 % \rustex_direct_HTML:n{</td></tr></table>}
8684 % }
8685 % }
8686 % }
8687 \end{minipage}\par\noindent
8688 }
8689 \newsavebox\columnbox
8690 \renewenvironment<>{column}[2][ ]{
8691 \begin{lrbox}{\columnbox}
8692 % \stex_if_html_backend:T{
8693 % \cs_if_exist:NT \rustex_if:T {
8694 % \rustex_if:T {\par
8695 % \rustex_direct_HTML:n{</td><td>}
8696 % }
8697 % }
8698 % }
8699 \begin{minipage}{#2}
8700 }{
8701 \end{minipage}
8702 % \stex_if_html_backend:T{
8703 % \cs_if_exist:NT \rustex_if:T {

```

```

8704 % \rustex_if:T {\par
8705 % \rustex_direct_HTML:n{</td><td>}
8706 % }
8707 % }
8708 % }
8709 \end{lrbox}\usebox\columnbox
8710 }
8711 \fi

```

39.1.3 Environment and Macro Patches

The `note` environment is used to leave out text in the `slides` mode. It does not have a counterpart in OMDoc. So for course notes, we define the `note` environment to be a no-operation otherwise we declare the `note` environment to produce no output.

```

8712 \cs_if_exist:cTF{note}{
8713   \bool_if:NTF \c_notesslides_notes_bool {
8714     \renewenvironment{note}{\ignorespaces}{ }
8715   }{
8716     \renewenvironment{note}{\setbox \l_tmpa_box\vbox\bgroup}{\egroup}
8717   }
8718 }{
8719   \bool_if:NTF \c_notesslides_notes_bool {
8720     \newenvironment{note}{\ignorespaces}{ }
8721   }{
8722     \newenvironment{note}{\setbox \l_tmpa_box\vbox\bgroup}{\egroup}
8723   }
8724 }

```

For other environments we introduce variants prefixed with `n`, which are excluded in `slides` mode.

```

8725 \cs_new_protected:Nn \__notesslides_notes_env:nnnn {
8726   \bool_if:NTF \c_notesslides_notes_bool {
8727     \newenvironment{#1}#2{#3}{#4}
8728   }{
8729     \newenvironment{#1}#2{
8730       \cs_set:Npn \__notesslides_eat: ####1 \end ####2 {
8731         \str_if_eq:nnTF{#1}{####2}{
8732           \end{#1}
8733         }{
8734           \__notesslides_eat:
8735         }
8736       }
8737       \__notesslides_eat:
8738       %\setbox\l_tmpa_box\vbox\bgroup#3
8739     }{
8740       %#4\egroup
8741     }
8742   }
8743 }
8744
8745 \__notesslides_notes_env:nnnn{nparagraph}{[1] []}{\begin{sparagraph}[#1]}{\end{sparagraph}}
8746 \__notesslides_notes_env:nnnn{nfragment}{[2] []}{\begin{sfragment}[#1]{#2}}{\end{sfragment}}
8747 \__notesslides_notes_env:nnnn{ndefinition}{[1] []}{\begin{sdefinition}[#1]}{\end{sdefinition}}

```

```

8748 \_notesslides_notes_env:nnnn{nassertion}{[1] []}{\begin{sassertion}[#1]}\end{sassertion}}
8749 \_notesslides_notes_env:nnnn{nproof}{[2] []}{\begin{sproof}[#1]{#2}}\end{sproof}}
8750 \_notesslides_notes_env:nnnn{nexample}{[1] []}{\begin{sexample}[#1]}\end{sexample}}
8751
8752 \RequirePackage{graphicx}
8753
8754 \NewDocumentCommand\frameimage{s O{} m}{
8755   \IfBooleanTF #1 {
8756     \begin{frame}[plain]
8757   }{
8758     \begin{frame}
8759   }
8760   \bool_if:NTF \c_notesslides_notes_bool {
8761     \slidewidth=\dimexpr\slidewidth-(2\slideframewidth)\relax
8762   }{
8763     \slidewidth=\textwidth\relax
8764   }
8765   \def\Gin@ewidth{}\setkeys{Gin}{#2}
8766   \tl_if_empty:NTF \Gin@ewidth {
8767     \mhgraphics[width=\slidewidth,#2]{#3}
8768   }{
8769     \mhgraphics[#2]{#3}
8770   }
8771   \end{frame}
8772 }

```

hacking inputref:

\inputref*

```

8773 \cs_set_eq:NN\_notesslides_inputref:\inputref
8774 \cs_set_protected:Npn\inputref{\@ifstar\ninputref\_notesslides_inputref:}
8775 \bool_if:NTF \c_notesslides_notes_bool {
8776   \newcommand\ninputref[2] []{
8777     \_notesslides_inputref:[#1]{#2}
8778   }
8779 }{
8780   \newcommand\ninputref[2] []{}
8781 }

```

(End of definition for \inputref*. This function is documented on page 110.)

39.1.4 Styling Across Notes/Slides

```

8782 \def\notesslidesitlemph#1{
8783   {\Large\bf\sf#1}
8784   \vskip0.1\baselineskip
8785   \leaders\vrule width \textwidth
8786   \vskip0.4pt%
8787   \nointerlineskip
8788 }
8789
8790 \def\notesslidesfooter{}
8791
8792 \let\notesslidesfont\sffamily

```

39.1.5 Beamer Compatibility

All of this should be removed and made part of a template

```

8793
8794 \stex_if_do_html:TF{
8795   \NewDocumentEnvironment{slideshow}{}{
8796     \begin{stex_annotate_env}{data-ftml-slideshow={}}
8797     \cs_set_protected:Npn \nextslide ##1 {
8798       \begin{stex_annotate_env}{data-ftml-slideshow-slide={}} ##1
8799       \end{stex_annotate_env}
8800     }
8801     \cs_set_protected:Npn \lastslide ##1 {
8802       \begin{stex_annotate_env}{data-ftml-slideshow-slide={}} ##1
8803       \end{stex_annotate_env}
8804     }
8805   }{
8806     \end{stex_annotate_env}
8807   }
8808 }{
8809   \int_new:N \l__notesslides_slideshow_counter_int
8810   \NewDocumentEnvironment{slideshow}{}{
8811     \int_zero:N \l__notesslides_slideshow_counter_int
8812     \cs_set_protected:Npn \nextslide ##1 {
8813       \int_incr:N \l__notesslides_slideshow_counter_int
8814       \only<\int_use:N \l__notesslides_slideshow_counter_int>{##1}
8815     }
8816     \cs_set_protected:Npn \lastslide ##1 {
8817       \int_incr:N \l__notesslides_slideshow_counter_int
8818       \only<\int_use:N \l__notesslides_slideshow_counter_int ->{##1}
8819     }
8820   }{
8821
8822   }
8823 }
8824
8825 \bool_if:NT \c_notesslides_notes_bool {
8826   \def\author{\@dblarg\ns@author}
8827   \long\def\ns@author[#1]#2{%
8828     \tl_if_empty:nTF{#1}{
8829       \def\beamer@shortauthor{#2}
8830     }{
8831       \def\beamer@shortauthor{#1}
8832     }
8833     \def\@author{#2}
8834   }
8835   \def\title{\@dblarg\ns@title}
8836   \long\def\ns@title[#1]#2{%
8837     \tl_if_empty:nTF{#1}{
8838       \def\beamer@shorttitle{#2}
8839     }{
8840       \def\beamer@shorttitle{#1}
8841     }
8842     \def\@title{#2}
8843     \stexdoctitle{#2}

```



```

8844 }
8845 \def\insertshortauthor{
8846   \hbox\bgroup\def\{\}\cs_if_exist:NT\beamer@shortauthor\beamer@shortauthor\egroup
8847 }
8848 \def\insertshorttitle{
8849   \hbox\bgroup\def\{\}\cs_if_exist:NT\beamer@shorttitle\beamer@shorttitle\egroup
8850 }
8851 \stex_if_html_backend:TF{
8852   \def\insertframenumbers{\stex_annotate:nn{data-ftml-slide-number={}}{}}
8853 }{
8854   \def\insertframenumbers{\@arabic\c@framenumbers}
8855 }
8856 \def\insertshortdate{\today}
8857 }

```

39.1.6 TODO Excursions

`\excursion` The excursion macros are very simple, we define a new internal macro `\excursionref` and use it in `\excursion`, which is just an `\inputref` that checks if the new macro is defined before formatting the file in the argument.

```

8858 \gdef\printexcursions{}
8859
8860
8861
8862 \stex_if_html_backend:TF{
8863   \newcommand\excursionref[4][\% repos, label, path, text
8864     \begin{sparagraph}[title=Excursion]
8865       #4~
8866       \stex_in_archive:nn{#1}{
8867         \stex_document_uri_from_archive_file:Nn \l__notesslides_uri {#3}
8868         \exp_args:Ne\href{\stex_use_document_uri:N\l__notesslides_uri}{here}
8869       }
8870     \end{sparagraph}
8871   }
8872   \def\excursion{\excursionref}
8873 }{
8874   \newcommand\excursionref[4][\% repos, label, path, text
8875     \bool_if:NT \c_notesslides_notes_bool {
8876       \begin{sparagraph}[title=Excursion]
8877         #4~\sref[fallback=the appendix]{#2}.
8878       \end{sparagraph}
8879     }
8880   }
8881   \newcommand\activate@excursion[2][\%
8882     \tl_gput_right:Nn\printexcursions{\inputref[#1]{#2}}
8883   }
8884   \newcommand\excursion[4][\% repos, label, path, text
8885     \bool_if:NT \c_notesslides_notes_bool {
8886       \activate@excursion[#1]{#3}
8887       \excursionref[#1]{#2}{#3}{#4}
8888     }
8889   }
8890

```

8891 }

(End of definition for \excursion. This function is documented on page 112.)

\excursiongroup

```

8892 \keys_define:nn{notesslides / excursiongroup }{
8893   id          .str_set_x:N = \l__notesslides_excursion_id_str,
8894   intro       .tl_set:N    = \l__notesslides_excursion_intro_tl,
8895   archive     .str_set_x:N = \l__notesslides_excursion_mhrepos_str
8896 }
8897 \cs_new_protected:Nn \l__notesslides_excursion_args:n {
8898   \tl_clear:N \l__notesslides_excursion_intro_tl
8899   \str_clear:N \l__notesslides_excursion_id_str
8900   \str_clear:N \l__notesslides_excursion_mhrepos_str
8901   \keys_set:nn {notesslides / excursiongroup }{ #1 }
8902 }
8903
8904
8905 \stex_if_html_backend:TF{
8906   \newcommand\excursiongroup[1][]{ }
8907 }{
8908   \newcommand\excursiongroup[1][]{
8909     \l__notesslides_excursion_args:n{ #1 }
8910     \tl_if_empty:NF\printexcursions
8911     {\IfInputref{}{\begin{note}
8912       \begin{sfragment}{Excursions}% TODO pass on id
8913       \ifdefempty\l__notesslides_excursion_intro_tl{\{
8914         \exp_args:NNe \use:nn \inputref{[\l__notesslides_excursion_mhrepos_str]{
8915           \l__notesslides_excursion_intro_tl
8916         }}
8917       }
8918       \printexcursions%
8919       \end{sfragment}
8920       \end{note}}}
8921     }
8922   }
8923
8924   \ifcsname beameritemnestingprefix\endcsname\else\def\beameritemnestingprefix{}\fi

```

(End of definition for \excursiongroup. This function is documented on page 112.)

```

8925 \prop_new:N \g__notesslides_variables_prop
8926 \cs_set_protected:Npn \setSGvar #1 #2 {
8927   \prop_gput:Nnn \g__notesslides_variables_prop {#1}{#2}
8928 }
8929 \cs_set_protected:Npn \useSGvar #1 {
8930   \prop_item:Nn \g__notesslides_variables_prop {#1}
8931 }
8932 \cs_set_protected:Npn \ifSGvar #1 #2 #3 {
8933   \prop_get:NnNF \g__notesslides_variables_prop {#1} \l__notesslides_tmp {
8934     \PackageError{document-structure}
8935     {The sTeX Global variable #1 is undefined}
8936     {set it with \protect\setSGvar}\TODO better error
8937   }
8938   \tl_if_eq:NnT \l__notesslides_tmp {#2}{ #3 }

```

```

8939 }
8940
8941
8942 </package>

```

39.2 Implementation: The problem Package

39.2.1 Package Options

The first step is to declare (a few) package options that handle whether certain information is printed or not. They all come with their own conditionals that are set by the options.

```

8943 <*package>
8944 <@@=problems>
8945 \ProvidesExplPackage{problem}{2026/06/24}{4.1.0}{Semantic Markup for Problems}
8946 \RequirePackage{l3keys2e}
8947
8948 \keys_define:nn { problem / pkg }{
8949   notes      .default:n      = { true },
8950   notes      .bool_set:N     = \c__problems_notes_bool,
8951   gnotes     .default:n      = { true },
8952   gnotes     .bool_set:N     = \c__problems_gnotes_bool,
8953   hints      .default:n      = { true },
8954   hints      .bool_set:N     = \c__problems_hints_bool,
8955   solutions  .default:n      = { true },
8956   solutions  .bool_set:N     = \c__problems_solutions_bool,
8957   pts        .default:n      = { true },
8958   pts        .bool_set:N     = \c__problems_pts_bool,
8959   min        .default:n      = { true },
8960   min        .bool_set:N     = \c__problems_min_bool,
8961   %boxed     .default:n      = { true },
8962   %boxed     .bool_set:N     = \c__problems_boxed_bool,
8963   test       .default:n      = { true },
8964   test       .bool_set:N     = \c__problems_test_bool,
8965   unknown    .code:n         = {
8966     \PassOptionsToPackage{\CurrentOption}{stex}
8967   }
8968 }
8969 \newif\ifsolutions
8970
8971 \ProcessKeysOptions{ problem / pkg }
8972 \bool_if:NTF \c__problems_solutions_bool {
8973   \solutionstrue
8974 }{
8975   \solutionsfalse
8976 }
8977 \newif\ifintest
8978 \bool_if:NTF \c__problems_test_bool {
8979   \intesttrue
8980 }{
8981   \intestfalse
8982 }
8983

```

```

8984 \RequirePackage{stex}
8985
8986 \ifintest
8987   \AtBeginDocument{
8988     \def\symrefemph#1{#1}
8989     \def\compemph#1{#1}
8990     \def\varemp#1{#1}
8991   }
8992 \fi
8993
8994 \newcommand\precondition[2]{
8995   \stex_get_symbol:n{#2}
8996   \tl_if_empty:NF \l_stex_get_symbol_uri {
8997     \str_case:nnTF {#1}{
8998       {remember}{}
8999       {understand}{}
9000       {analyze}{}
9001       {evaluate}{}
9002       {apply}{}
9003       {create}{}
9004     }{
9005       \ifstexhtml\else\bool_if:NT\c_stex_metadata_bool {
9006         \marginpar{\tiny \textbf{Precondition:}}~#1~
9007         \exp_args:Ne\symrefemph@uri{\stex_symbol_uri_name:N \l_stex_get_symbol_uri}{\l_stex
9008       }
9009     }\fi
9010     \stex_annotate_invisible:nn{
9011       data-ftml-preconditionsymbol={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
9012       data-ftml-preconditiondimension={#1}
9013     }{}
9014     }{\errmessage{Unknown~cognitive~dimension~#1}}
9015   }
9016 }
9017 \newcommand\objective[2]{
9018   \stex_get_symbol:n{#2}
9019   \tl_if_empty:NF \l_stex_get_symbol_uri {
9020     \str_case:nnTF {#1}{
9021       {remember}{}
9022       {understand}{}
9023       {analyze}{}
9024       {evaluate}{}
9025       {apply}{}
9026       {create}{}
9027     }{
9028       \ifstexhtml\else\bool_if:NT\c_stex_metadata_bool {
9029         \marginpar{\tiny \textbf{Objective:}}~#1~
9030         \exp_args:Ne\symrefemph@uri{\stex_symbol_uri_name:N \l_stex_get_symbol_uri}{\l_stex
9031       }
9032     }\fi
9033     \stex_annotate_invisible:nn{
9034       data-ftml-objectivesymbol={\stex_use_symbol_uri:N \l_stex_get_symbol_uri},
9035       data-ftml-objectivedimension={#1}
9036     }{}
9037     }{\errmessage{Unknown~cognitive~dimension~#1}}

```

```

9038 }
9039 }

```

\problem@kw@* For multilinguality, we define internal macros for keywords that can be specialized in *.ldf files.

```

9040 \AddToHook{begindocument}{
9041   \ExplSyntaxOn\makeatletter
9042   \input{problem-english.ldf}
9043   \ltx@ifpackageloaded{babel}{
9044     \clist_set:Nx \l_tmpa_clist {\exp_args:No \tl_to_str:n \bbl@loaded}
9045     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{ngerman}}{
9046       \input{problem-ngerman.ldf}
9047     }
9048     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{finnish}}{
9049       \input{problem-finnish.ldf}
9050     }
9051     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{french}}{
9052       \input{problem-french.ldf}
9053     }
9054     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{russian}}{
9055       \input{problem-russian.ldf}
9056     }
9057   }{}
9058   \makeatother\ExplSyntaxOff
9059 }

```

(End of definition for \problem@kw@*. This function is documented on page ??.)

39.2.2 Problems and Solutions

We now prepare the KeyVal support for problems. The key macros just set appropriate internal macros.

```

9060 \bool_new:N \l_stex_key_autogradable_bool
9061 \stex_keys_define:nnnn{ problem }{
9062   \tl_clear:N \l_stex_key_pts_tl
9063   \tl_set:Nn \l_stex_key_min_tl 0
9064   \str_clear:N \l_stex_key_name_str
9065   \str_clear:N \l_stex_key_mhrepos_str
9066   \bool_set_false:N \l_stex_key_autogradable_bool
9067 }{
9068   pts .tl_set:N = \l_stex_key_pts_tl,
9069   min .tl_set:N = \l_stex_key_min_tl,
9070   name .str_set:N = \l_stex_key_name_str,
9071   autogradable .bool_set:N = \l_stex_key_autogradable_bool,
9072   archive .code:n = {},
9073   %archive .str_set:N = \l_stex_key_mhrepos_str,
9074   %creators .code:n = {}
9075   %imports .tl_set:N = \l__problems_prob_imports_tl,
9076   %refnum .int_set:N = \l__problems_prob_refnum_int,
9077 }{id,title,style,uses}

```

Then we set up a counter for problems.

\numberproblemsin

```

9078 \newcounter{sproblem}[section]
9079 \newcommand\numberproblemsin[1]{
9080   \@addtoreset{sproblem}{#1}
9081   \def\thesproblem{\arabic{#1}.\arabic{sproblem}}
9082 }
9083 \numberproblemsin{section}
9084 %\def\theplainsproblem{\arabic{sproblem}}
9085 %\def\thesproblem{\thesection.\theplainsproblem}

```

(End of definition for \numberproblemsin. This function is documented on page ??.)

sproblem (env.)

```

9086 \cs_new:Nn \__problems_activate_macros: {
9087   \stex_reactivate_macro:N \solution
9088   \stex_reactivate_macro:N \mcb
9089   \stex_reactivate_macro:N \scb
9090   \stex_reactivate_macro:N \fillinsol
9091   \stex_reactivate_macro:N \hint
9092   \stex_reactivate_macro:N \exnote
9093   \stex_reactivate_macro:N \gnote
9094 }
9095
9096 \fp_new:N \g_problem_total_pts_fp
9097 \fp_new:N \g_problem_total_min_fp
9098
9099 \bool_new:N \l__problems_in_problem_bool
9100 \bool_new:N \l__problems_has_pts_bool
9101 \bool_new:N \l__problems_has_min_bool
9102 \bool_set_false:N \l__problems_in_problem_bool
9103 \stex_new_stylable_env:nnnnnnn {problem} {0{}}{
9104   \bool_if:NT \l__problems_in_problem_bool {
9105     \msg_error:nn{stex}{error/nestedproblem}
9106   }
9107   \cs_if_exist:NTF \l_problem_inputproblem_keys_tl {
9108     \tl_put_left:Nn \l_problem_inputproblem_keys_tl {#1,}
9109     \exp_args:Nno \stex_keys_set:nn{problem}{
9110       \l_problem_inputproblem_keys_tl
9111     }
9112   }{
9113     \stex_keys_set:nn{problem}{#1}
9114   }
9115   \refstepcounter{sproblem}
9116
9117   \stex_if_do_html:T {
9118     \str_if_empty:NT \l_stex_key_name_str {
9119       \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
9120       \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
9121     }
9122     \exp_args:Nne \begin{stex_annotate_env} {
9123       data-ftml-problem={\l_stex_key_name_str},
9124       %data-ftml-language={ \l_stex_current_language_str},
9125       data-ftml-autogradable={\bool_if:NTF \l_stex_key_autogradable_bool {true}{false}}
9126       \tl_if_empty:NF \l_stex_key_pts_tl{,data-ftml-problempoints={\l_stex_key_pts_tl}}

```

```

9127     }
9128     \noindent \stex_annotate:nn{data-ftml-title={}}{\_stex_annotate_force_break:n{\l_stex_k
9129     \tl_if_empty:NF \l_stex_key_title_tl {
9130         \exp_args:No \stexdoctitle \l_stex_key_title_tl
9131     }
9132     \_stex_annotate_force_break:n{}}
9133 }
9134 \tl_set_eq:NN \thistitle \l_stex_key_title_tl
9135 \tl_if_empty:NT \l_stex_key_pts_tl { \tl_set:Nn \l_stex_key_pts_tl{0} }
9136
9137
9138 \bool_set_true:N \l__problems_in_problem_bool
9139 \tl_set_eq:NN \l__problems_pts_tl \l_stex_key_pts_tl
9140 \tl_set_eq:NN \l__problems_min_tl \l_stex_key_min_tl
9141 \tl_if_eq:NnTF \l__problems_pts_tl {0}
9142     {\bool_set_false:N \l__problems_has_pts_bool}
9143     {\bool_set_true:N \l__problems_has_pts_bool}
9144 \tl_if_eq:NnTF \l__problems_min_tl {0}
9145     {\bool_set_false:N \l__problems_has_min_bool}
9146     {\bool_set_true:N \l__problems_has_min_bool}
9147 \int_gzero:N \g__problems_subproblem_int
9148
9149 \stex_if_do_html:F\stex_style_apply:
9150 \_stex_do_id:
9151 \__problems_activate_macros:
9152 }{
9153 \fp_gadd:Nn \g_problem_total_pts_fp { \l__problems_pts_tl}
9154 \fp_gadd:Nn \g_problem_total_min_fp { \l__problems_min_tl}
9155 \__problems_record_problem:
9156 \stex_if_do_html:F\stex_style_apply:
9157 \stex_if_do_html:T{ \end{stex_annotate_env} }
9158 }{
9159 \par\noindent\problemheader
9160 \stex_if_do_html:F{
9161     \bool_if:NT \c__problems_pts_bool {
9162         \tl_if_eq:NnF \l__problems_pts_tl {0}{
9163             \marginpar{\l__problems_pts_tl}{~\problem@kw@pts\smallskip}
9164         }
9165     }
9166     \bool_if:NT \c__problems_min_bool {
9167         \tl_if_eq:NnF \l__problems_min_tl {0} {
9168             \marginpar{\l__problems_min_tl}{~\problem@kw@minutes\smallskip}
9169         }
9170     }
9171 }
9172 \par
9173 \stex_ignore_spaces_and_pars:
9174 }{
9175 \par\bigskip
9176 % \bool_if:NT \c__problems_test_bool \pagebreak
9177 }{s}
9178
9179 \tl_set:Nn \problemheader {
9180     \stex_if_do_html:TF{

```

```

9181 \tl_if_empty:NF \thistitle {
9182 \stex_annotate:nn{data-ftml-title={}}{\textbf{\thistitle}}
9183 }
9184 }{
9185 \textbf{\sproblemautorefname{~}\thesproblem
9186 \tl_if_empty:NF \thistitle {
9187 {~}(\thistitle)
9188 }
9189 }
9190 }
9191 }
9192
9193 \cs_new_protected:Nn \__problems_record_problem: {
9194 \exp_args:Nne \iow_now:Nn \auxout {
9195 \problem@restore {\thesproblem}{\l__problems_pts_tl}{\l__problems_min_tl}
9196 }
9197 }
9198
9199 \cs_new_protected:Npn \problem@restore #1 #2 #3 {}

```

subproblem (env.)

```

9200 \int_new:N \g__problems_subproblem_int
9201
9202 \stex_new_stylable_env:nnnnnnn {subproblem} {0{}}{
9203 \stex_keys_set:nn{problem}{#1}
9204 \bool_if:NF \l__problems_in_problem_bool{
9205 \ifstexhtml\else
9206 \par{\bfseries WARNING~subproblem~to~be~used~in~some~problem}\par
9207 \fi
9208 \__problems_activate_macros:
9209 \bool_set_true:N \l__problems_in_problem_bool
9210 \tl_set:Nn \l__problems_pts_tl{0}
9211 \tl_set:Nn \l__problems_min_tl{0}
9212 }
9213 \str_if_empty:NT \l_stex_key_name_str {
9214 \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
9215 \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
9216 }
9217 \stex_if_do_html:T {
9218 \str_if_empty:NT \l_stex_key_name_str {
9219 \stex_file_split_off_lang:NN \l__problems_path_seq \g_stex_current_file
9220 \seq_get_right:NN \l__problems_path_seq \l_stex_key_name_str
9221 }
9222 \exp_args:Nne \begin{stex_annotate_env} {
9223 data-ftml-subproblem={\l_stex_key_name_str},
9224 %data-ftml-language={ \l_stex_current_language_str},
9225 data-ftml-autogradable={\bool_if:NTF \l_stex_key_autogradable_bool {true}{false}}
9226 \tl_if_empty:NF \l_stex_key_pts_tl{,data-ftml-problempoints={\l_stex_key_pts_tl}}
9227 }
9228 \noindent \stex_annotate:nn{data-ftml-title={}}{\stex_annotate_force_break:n{\l_stex_k
9229 \tl_if_empty:NF \l_stex_key_title_tl {
9230 \exp_args:No \stexdoctitle \l_stex_key_title_tl
9231 }
9232 \tl_if_empty:NF \l_stex_key_title_tl {

```



```

9233     \exp_args:No \stexdoctitle \l_stex_key_title_tl
9234   }
9235   \_stex_annotate_force_break:n{}
9236 }
9237 \tl_if_empty:NT \l_stex_key_pts_tl { \tl_set:Nn \l_stex_key_pts_tl{0} }
9238 \int_gincr:N \g__problems_subproblem_int
9239 \bool_if:NF \l__problems_has_pts_bool {
9240   \tl_gset:Ne \l__problems_pts_tl {\fp_to_decimal:n {\l__problems_pts_tl + \l_stex_key_pt
9241 }
9242 \bool_if:NF \l__problems_has_min_bool {
9243   \tl_gset:Ne \l__problems_min_tl {\fp_to_decimal:n {\l__problems_min_tl + \l_stex_key_mi
9244 }
9245 \stex_if_smsmode:F {\stex_if_do_html:F \stex_style_apply: }
9246 }{
9247   \stex_if_do_html:TF{ \end{stex_annotate_env} }{\stex_if_smsmode:F\stex_style_apply:}
9248 }{
9249   \begin{list}{}{
9250     \setlength\topsep{0pt}
9251     \setlength\parsep{0pt}
9252     \setlength\rightmargin{0pt}
9253   } \item[\int_use:N \g__problems_subproblem_int .]
9254   \bool_if:NT \c__problems_pts_bool {
9255     \bool_if:NF \l__problems_has_pts_bool {
9256       \marginpar{\smallskip\l_stex_key_pts_tl{}}~\problem@kw@pts}
9257     }
9258   }
9259   \bool_if:NT \c__problems_min_bool {
9260     \bool_if:NF \l__problems_has_min_bool{
9261       \marginpar{\smallskip\l_stex_key_min_tl{}}~\problem@kw@minutes}
9262     }
9263   }
9264   \ignorespaces
9265 }{
9266   \end{list}
9267 }{}

```

\includeproblem

```

9268 \stex_keys_define:nnnn{ includeproblem }{
9269   \str_clear:N \l_stex_key_mhrepos_str
9270 }{
9271   archive .str_set:N = \l_stex_key_mhrepos_str,
9272   unknown .code:n = {}
9273 }{}
9274
9275 \NewDocumentCommand\includeproblem{0{ } m}{
9276   \group_begin:
9277   \tl_set:Nn \l_problem_inputproblem_keys_tl {#1}
9278   \stex_keys_set:nn{includeproblem}{#1}
9279   \exp_args:Nno \use:nn{\inputref[]\l_stex_key_mhrepos_str}{#2}
9280   \group_end:
9281 }
9282

```

(End of definition for \includeproblem. This function is documented on page 117.)

`solution (env.)`

```

9283 \int_new:N \g_problem_id_counter
9284 \dim_new:N \l_stex_key_testspace_dim
9285 \stex_keys_define:nnnn{ solution }{
9286   \str_clear:N \l_stex_key_answerclass_str
9287   \dim_zero:N \l_stex_key_testspace_dim
9288 }{
9289   testspace .dim_set:N = \l_stex_key_testspace_dim,
9290   answerclass .str_set:N = \l_stex_key_answerclass_str
9291 }{id,title,style}
9292
9293 \cs_new_protected:Nn \__problems_solution_start:n {
9294   \stex_keys_set:nn{ solution }{#1}
9295   \str_if_empty:NT \l_stex_key_id_str {
9296     \int_gincr:N \g_problem_id_counter
9297     \str_set:Nx \l_stex_key_id_str {
9298       SOLUTION_\int_use:N \g_problem_id_counter
9299     }
9300   }
9301   \stex_if_do_html:TF{
9302     \begin{stex_annotate_env}{
9303       data-ftml-solution=\l_stex_key_id_str,
9304       data-ftml-answerclass={\l_stex_key_answerclass_str}
9305     }
9306   }
9307 }
9308
9309 \stex_new_stylable_env:nnnnnnn { solution }{ 0{ } }{
9310   \stex_if_do_html:TF{
9311     \__problems_solution_start:n{#1}
9312   }{
9313     \ifsolutions
9314       \__problems_solution_start:n{#1}
9315       \stex_style_apply:
9316     \else
9317       \stex_keys_set:nn{ solution }{#1}
9318       \testspace{\l_stex_key_testspace_dim}
9319       \setbox\l_tmpa_box\vbox\bgroup
9320       \fi
9321   }
9322 }{
9323   \stex_if_do_html:TF{
9324     \end{stex_annotate_env}
9325   }{
9326     \ifsolutions
9327       \stex_style_apply:
9328       \stex_if_do_html:TF{
9329         \end{stex_annotate_env}
9330       }
9331     \else
9332       \egroup
9333     \fi
9334   }
9335 }{

```

```

9336 \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9337 \noindent\emph{\problem@kw@solution\tl_if_empty:NF \l_stex_key_title_tl{
9338   {~}\l_stex_key_title_tl \str_if_empty:NF \l_stex_key_answerclass_str {
9339     (Answer Class: \l_stex_key_answerclass_str)
9340   }
9341 } :~}
9342 }{
9343 \par\rule[.3em]{\linewidth}{0.4pt}\newline
9344 }{
9345
9346 \stex_deactivate_macro:Nn \solution {sproblem-environments}

```

`\startsolutions`
`\stopsolutions`

```

9347 \cs_new_protected:Npn \startsolutions{
9348   \global\solutionstrue
9349 }
9350 \cs_new_protected:Npn \stopsolutions{
9351   \global\solutionsfalse
9352 }

```

(End of definition for `\startsolutions` and `\stopsolutions`. These functions are documented on page 114.)

`hint (env.)`

```

9353
9354 \stex_keys_define:nnnn{ problemenv }{{}}{id,title,style}
9355
9356 \cs_new_protected:Nn \__problems_hint_start:n {
9357   \stex_keys_set:nn{ problemenv }{#1}
9358   \str_if_empty:NT \l_stex_key_id_str {
9359     \int_gincr:N \g_problem_id_counter
9360     \str_set:Nx \l_stex_key_id_str {
9361       HINT_\int_use:N \g_problem_id_counter
9362     }
9363   }
9364   \stex_if_do_html:TF{
9365     \begin{stex_annotate_env}{
9366       data-ftml-problemhint=\l_stex_key_id_str
9367     }
9368   }
9369 }
9370
9371 \stex_new_stylable_env:nnnnnnn { hint }{ 0{} }{
9372   \stex_if_do_html:TF{
9373     \__problems_hint_start:n{#1}
9374   }{
9375     \bool_if:NTF \c__problems_hints_bool {
9376       \__problems_hint_start:n{#1}
9377       \stex_style_apply:
9378     }{
9379       \setbox\l_tmpa_box\vbox\bgroup
9380     }
9381   }
9382 }{

```

```

9383 \stex_if_do_html:TF{
9384   \end{stex_annotate_env}
9385 }{
9386   \bool_if:NTF \c__problems_hints_bool {
9387     \stex_style_apply:
9388     \stex_if_do_html:T{
9389       \end{stex_annotate_env}
9390     }
9391   }{
9392     \egroup
9393   }
9394 }
9395 }{
9396   \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9397   \noindent\emph{\problem@kw@hint\tl_if_empty:NF \l_stex_key_title_tl{
9398     {~}\l_stex_key_title_tl
9399   } :~}
9400 }{
9401   \par\rule[.3em]{\linewidth}{0.4pt}\newline
9402 }{}
9403 \stex_deactivate_macro:Nn \hint {sproblem-environments}

```

exnote (env.)

```

9404 \cs_new_protected:Nn \__problems_exnote_start:n {
9405   \stex_keys_set:nn{ problemenv }{#1}
9406   \str_if_empty:NT \l_stex_key_id_str {
9407     \int_gincr:N \g_problem_id_counter
9408     \str_set:Nx \l_stex_key_id_str {
9409       EXNOTE_\int_use:N \g_problem_id_counter
9410     }
9411   }
9412   \stex_if_do_html:T{
9413     \begin{stex_annotate_env}{
9414       data-ftml-problemnote=\l_stex_key_id_str
9415     }
9416   }
9417 }
9418
9419 \stex_new_stylable_env:nnnnnn { exnote }{ 0{} }{
9420   \stex_if_do_html:TF{
9421     \__problems_exnote_start:n{#1}
9422   }{
9423     \bool_if:NTF \c__problems_notes_bool {
9424       \__problems_exnote_start:n{#1}
9425       \stex_style_apply:
9426     }{
9427       \setbox\l_tmpa_box\vbox\bgroup
9428     }
9429   }
9430 }{
9431   \stex_if_do_html:TF{
9432     \end{stex_annotate_env}
9433   }{
9434     \bool_if:NTF \c__problems_notes_bool {

```

```

9435     \stex_style_apply:
9436     \stex_if_do_html:TF{
9437         \end{stex_annotate_env}
9438     }
9439 }{
9440     \egroup
9441 }
9442 }
9443 }{
9444     \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9445     \noindent\emph{\problem@kw@note\tl_if_empty:NF \l_stex_key_title_tl{
9446         {~}\l_stex_key_title_tl
9447     } :~}
9448 }{
9449     \par\rule[.3em]{\linewidth}{0.4pt}\newline
9450 }{}}
9451 \stex_deactivate_macro:Nn \exnote {sproblem~environments}

```

gnote (env.)

```

9452 \int_new:N \l__problems_anscls_int
9453
9454 \cs_new_protected:Nn \__problems_gnote_start:n {
9455     \stex_keys_set:nn{ problemenv }{#1}
9456     \str_if_empty:NT \l_stex_key_id_str {
9457         \int_gincr:N \g_problem_id_counter
9458         \str_set:Nx \l_stex_key_id_str {
9459             GNOTE_\int_use:N \g_problem_id_counter
9460         }
9461     }
9462
9463     \stex_if_do_html:TF{
9464         \begin{stex_annotate_env}{
9465             data-ftml-problemgnote=\l_stex_key_id_str
9466         }
9467     }
9468     \stex_style_apply:
9469 }
9470
9471 \stex_new_stylable_env:nnnnnn { gnote }{ 0{} }{
9472     \stex_if_do_html:TF{
9473         \__problems_gnote_start:n{#1}
9474     }{
9475         \bool_if:NTF \c__problems_gnotes_bool {
9476             \__problems_gnote_start:n{#1}
9477         }{
9478             \setbox\l_tmpa_box\vbox\bgroup
9479         }
9480     }
9481     \stex_reactivate_macro:N \anscls
9482 }{
9483     \stex_if_do_html:TF{
9484         \end{stex_annotate_env}
9485     }{
9486         \bool_if:NTF \c__problems_gnotes_bool {

```

```

9487     \stex_style_apply:
9488     \stex_if_do_html:T{
9489         \end{stex_annotate_env}
9490     }
9491 }{
9492     \egroup
9493 }
9494 }
9495 }{
9496     \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
9497     \noindent\emph{\problem@kw@grading\str_if_empty:NF \l_stex_key_title_tl{
9498         {~}\l_stex_key_title_tl
9499     } :~}
9500 }{
9501     \par\rule[.3em]{\linewidth}{0.4pt}\newline
9502 }{}
9503 \stex_deactivate_macro:Nn \gnote {sproblem-environments}
9504
9505
9506 \stex_keys_define:nnnn{ anscls }{
9507     \str_clear:N \l_stex_key_pts_str
9508     \tl_clear:N \l_stex_key_feedback_tl
9509 }{
9510     pts      .str_set:N = \l_stex_key_pts_str,
9511     feedback .tl_set:N = \l_stex_key_feedback_tl,
9512     update   .code:n    = {}
9513 }{id}
9514 \newcommand \anscls [2] [] {
9515     \stex_keys_set:nn{ anscls }{#1}
9516     \str_if_empty:NT \l_stex_key_id_str {
9517         \int_incr:N \l__problems_anscls_int
9518         \str_set:Nx \l_stex_key_id_str {
9519             AC\int_use:N \l__problems_anscls_int
9520         }
9521     }
9522     \stex_if_do_html:TF{
9523         \begin{list}{}{
9524             \setlength\topsep{0pt}
9525             \setlength\parsep{0pt}
9526             \setlength\rightmargin{0pt}
9527         }\item[] \exp_args:Ne \stex_annotate:nn{
9528             data-ftml-answerclass={\l_stex_key_id_str}
9529             \str_if_empty:NF \l_stex_key_pts_str{
9530                 ,data-ftml-answerclass-pts={\l_stex_key_pts_str}
9531             }
9532         }{
9533             #2~
9534             \tl_if_empty:NF \l_stex_key_feedback_tl{
9535                 \stex_annotate:nn{
9536                     data-ftml-answerclass-feedback={}
9537                 }{\l_stex_key_feedback_tl}
9538             }
9539         }
9540     } \end{list}

```

```

9541 }{
9542   \begin{list}{}{
9543     \setlength\topsep{0pt}
9544     \setlength\parsep{0pt}
9545     \setlength\rightmargin{0pt}
9546   }\item[\l_stex_key_id_str] #2
9547     \str_if_empty:NF \l_stex_key_pts_str {\par
9548       ~ \problem@kw@points :~\l_stex_key_pts_str
9549     }
9550     \tl_if_empty:NF \l_stex_key_feedback_tl {\par
9551       ~ \problem@kw@feedback :~\l_stex_key_feedback_tl
9552     }
9553   \end{list}
9554 }
9555 }
9556 \stex_deactivate_macro:Nn \anscls {gnote-environments}

```

The margin pars are reader-visible, so we need to translate

```

9557 \def\pts#1{
9558   \bool_if:NT \c__problems_pts_bool {
9559     \stex_annotate:nn{data-ftml-problempoints={#1}}{\marginpar{#1~\problem@kw@pts}}
9560   }\hbox_unpack:N\c_empty_box
9561 }
9562 \def\min#1{
9563   \bool_if:NT \c__problems_min_bool {
9564     \stex_annotate:nn{data-ftml-problemminutes={}}{\marginpar{#1~\problem@kw@minutes}}
9565   }\hbox_unpack:N\c_empty_box
9566 }

```

mcb (*env.*)

```

9567 \stex_new_stylable_env:nnnnnnn{mcb}{0}{0}{
9568   \stex_keys_set:nn{style}{#1}
9569   \cs_set:Nn \__problems_mccline:n{
9570     \begin{list}{}{
9571       \setlength\topsep{0pt}
9572       \setlength\parsep{0pt}
9573       \setlength\rightmargin{0pt}
9574     }\item[\problem_mcc_box_tl] ##1 \end{list}
9575   }
9576
9577   \stex_if_do_html:T{
9578     \tl_set:Nn\problem_mcc_box_tl{}
9579     \__problems_maybe_inline:n{
9580       \exp_args:Nne \begin{stex_annotate_env}{
9581         data-ftml-multiple-choice-block={}
9582         \clist_if_empty:NF \l_stex_key_style_clist {,
9583           data-ftml-styles={\l_stex_key_style_clist}
9584         }
9585       }
9586     }
9587     \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
9588       \cs_set:Nn \__problems_mccline:n {##1}
9589     }{
9590       \clist_if_in:NnT \l_stex_key_style_clist {dropdown} {

```

```

9591     \cs_set:Nn \__problems_mccline:n {##1}
9592   }
9593 }
9594 }
9595 \stex_deactivate_macro:Nn \mcb {sproblem~environments}
9596 \stex_deactivate_macro:Nn \scb {sproblem~environments}
9597 \stex_deactivate_macro:Nn \solution {sproblem~environments}
9598 \stex_deactivate_macro:Nn \hint {sproblem~environments}
9599 \stex_deactivate_macro:Nn \exnote {sproblem~environments}
9600 \stex_deactivate_macro:Nn \gnote {sproblem~environments}
9601 \stex_reactivate_macro:N \mcc
9602 \stex_if_do_html:F \stex_style_apply:
9603 }{
9604   \stex_if_do_html:TF{
9605     \__problems_maybe_inline:n{\end{stex_annotate_env}}
9606   }\stex_style_apply:
9607 }{\par}{\par}
9608
9609 \stexstylemcb[inline]{
9610   {\Large[]\cs_set:Nn \__problems_mccline:n{\problem_mcc_box_tl{~} #1}
9611   }{\Large[]}}
9612
9613 \stexstylemcb[dropdown]{
9614   {\Large[]\cs_set:Nn \__problems_mccline:n{\problem_mcc_box_tl{~} #1}
9615   }{\Large[]}}
9616
9617 \stex_deactivate_macro:Nn \mcb {sproblem~environments}

```

we define the keys for the mcc macro

```

9618 \cs_new_protected:Nn \__problems_do_yes_param:Nn {
9619   \exp_args:Nx \str_if_eq:nnTF { \str_lowercase:n{ #2 } }{ yes }{
9620     \bool_set_true:N #1
9621   }{
9622     \bool_set_false:N #1
9623   }
9624 }
9625 \stex_keys_define:nnnn{mcc}{
9626   \tl_clear:N \l_stex_key_feedback_tl
9627   \bool_set_false:N \l_stex_key_T_bool
9628   \tl_clear:N \l_stex_key_Ttext_tl
9629   \tl_clear:N \l_stex_key_Ftext_tl
9630 }{
9631   feedback .tl_set:N      = \l_stex_key_feedback_tl ,
9632   T         .code:n       = {\bool_set_true:N \l_stex_key_T_bool} ,
9633   F         .code:n       = {\bool_set_false:N \l_stex_key_T_bool} ,
9634   Ttext     .tl_set:N     = \l_stex_key_Ttext_tl ,
9635   Ftext     .tl_set:N     = \l_stex_key_Ftext_tl ,
9636 }{id}
9637
\mcc

```

```

9638
9639 \cs_set:Nn \__problems_maybe_newline: { \\\ }
9640

```



```

9641 \stex_if_html_backend:TF{
9642   \tl_set:Nn \problem_mcc_box_tl {
9643     \ltx@ifpackageloaded{amssymb}{\square$}{
9644       \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
9645     }
9646   }
9647
9648   \newcommand\mcc[2][]{
9649     \stex_keys_set:nn{mcc}{#1}
9650     \__problems_mccline:n{
9651       \stex_if_do_html:T{
9652         \stex_annotate:nn{data-ftml-problem-choice={
9653           \bool_if:NTF \l_stex_key_T_bool {true}{false}
9654         }}{
9655           #2~
9656           \stex_annotate:nn{data-ftml-problem-choice-verdict={}}{
9657             \bool_if:NTF \l_stex_key_T_bool {
9658               \tl_if_empty:NTF \l_stex_key_Ttext_tl \problem@kw@correct \l_stex_key_Ttext_tl
9659             }{
9660               \tl_if_empty:NTF \l_stex_key_Ftext_tl \problem@kw@wrong \l_stex_key_Ftext_tl
9661             }
9662           }
9663           \tl_if_empty:NF \l_stex_key_feedback_tl {
9664             \stex_annotate:nn{data-ftml-problem-choice-feedback={}}{
9665               \l_stex_key_feedback_tl
9666             }
9667           }
9668         }
9669       }
9670     }
9671   }
9672 }{
9673   \tl_set:Nn \problem_mcc_box_default_tl {
9674     \ltx@ifpackageloaded{amssymb}{\square$}{
9675       \hbox{\vrule\vbox{\hrule width 6 pt\vskip 6pt\hrule}\vrule}
9676     }
9677   }
9678   \tl_set:Nn \problem_mcc_box_tl {
9679     \ifsolutions
9680       \bool_if:NTF \l_stex_key_T_bool {
9681         \makebox[0pt][l]{\problem_mcc_box_default_tl}
9682         \hspace{0.1em}$\cs_if_exist:NTF\checkmark{
9683           \raisebox{.15ex}{\checkmark}
9684         } X$
9685       }{\problem_mcc_box_default_tl}
9686     \else
9687       \problem_mcc_box_default_tl
9688     \fi
9689   }
9690   \newcommand\mcc[2][]{
9691     \stex_keys_set:nn{mcc}{#1}
9692     \ifsolutions
9693       \tl_set:Nx \l_tmpa_tl{
9694         \bool_if:NTF \l_stex_key_T_bool {

```

```

9695     \tl_if_empty:NF \l_stex_key_Ttext_tl {\exp_not:N \l_stex_key_Ttext_tl}
9696   }{
9697     \tl_if_empty:NF \l_stex_key_Ftext_tl {\exp_not:N \l_stex_key_Ftext_tl}
9698   }
9699   \tl_if_empty:NF \l_stex_key_feedback_tl {
9700     \exp_not:n{\emph{\l_stex_key_feedback_tl}}
9701   }
9702 }
9703 \fi
9704
9705 \__problems_mccline:n{ #2
9706   \ifsolutions
9707     \tl_if_empty:NF \l_tmpa_tl { \footnote{\l_tmpa_tl} }
9708   \fi
9709 }
9710 }
9711 }
9712 \stex_deactivate_macro:Nn \mcc {mcb~environments}

```

(End of definition for \mcc. This function is documented on page 115.)

scb (env.)

```

9713
9714 \cs_new_protected:Nn \__problems_maybe_inline:n {
9715   \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
9716     \let\__problems_oldpar:\stex_par:
9717     \cs_set:Nn\stex_par:{~}
9718     \cs_set:Nn\__problems_maybe_newline:{}
9719     #1
9720     \let\stex_par:\__problems_oldpar:
9721   }{
9722     \clist_if_in:NnTF \l_stex_key_style_clist {dropdown} {
9723       \let\__problems_oldpar:\stex_par:
9724       \cs_set:Nn\stex_par:{~}
9725       \cs_set:Nn\__problems_maybe_newline:{}
9726       #1
9727       \let\stex_par:\__problems_oldpar:
9728     }{\par #1}
9729   }
9730 }
9731
9732 \stex_new_stylable_env:nnnnnnn{scb}{0}{}{
9733   \stex_keys_set:nn{style}{#1}
9734   \cs_set:Nn \__problems_sccline:n{
9735     \begin{list}{}{
9736       \setlength\topsep{0pt}
9737       \setlength\parsep{0pt}
9738       \setlength\rightmargin{0pt}
9739     }\item[\problem_scc_box_tl] ##1 \end{list}
9740   }
9741
9742   \stex_if_do_html:T{
9743     \__problems_maybe_inline:n{
9744       \exp_args:Ne\stex_annotate_env{

```

```

9745     data-ftml-single-choice-block={}
9746     \clist_if_empty:NF \l_stex_key_style_clist {,
9747       data-ftml-styles={\l_stex_key_style_clist}
9748     }
9749   }
9750 }
9751 \tl_set:Nn\problem_scc_box_tl{}
9752 \clist_if_in:NnTF \l_stex_key_style_clist {inline} {
9753   \cs_set:Nn \__problems_sccline:n {##1}
9754 }{
9755   \clist_if_in:NnT \l_stex_key_style_clist {dropdown} {
9756     \cs_set:Nn \__problems_sccline:n {##1}
9757   }
9758 }
9759 }
9760 \stex_deactivate_macro:Nn \mcb {sproblem~environments}
9761 \stex_deactivate_macro:Nn \scb {sproblem~environments}
9762 \stex_deactivate_macro:Nn \solution {sproblem~environments}
9763 \stex_deactivate_macro:Nn \hint {sproblem~environments}
9764 \stex_deactivate_macro:Nn \exnote {sproblem~environments}
9765 \stex_deactivate_macro:Nn \gnote {sproblem~environments}
9766 \stex_reactivate_macro:N \scc
9767 \stex_if_do_html:F \stex_style_apply:
9768 }{
9769   \stex_if_do_html:TF{
9770     \__problems_maybe_inline:n { \endstex_annotate_env }
9771   }\stex_style_apply:
9772 }{\par}{}{}
9773
9774 \stexstylescb[inline]{
9775   {\Large[]\cs_set:Nn \__problems_sccline:n{\problem_scc_box_tl{~} #1}
9776   }{\{ \Large[]}}
9777
9778 \stexstylescb[dropdown]{
9779   {\Large[]\cs_set:Nn \__problems_sccline:n{\problem_scc_box_tl{~} #1}
9780   }{\{ \Large[]}}
9781
9782 \stex_deactivate_macro:Nn \scb {sproblem~environments}

```

\scc

```

9783
9784 \stex_if_html_backend:TF{
9785   \tl_set:Nn\problem_scc_box_tl{${\bigcirc}}
9786   \newcommand\scc[2][]{
9787     \stex_keys_set:nn{mcc}{#1}
9788     \__problems_sccline:n{
9789       \stex_if_do_html:T{
9790         \stex_annotate:nn{data-ftml-problem-choice={
9791           \bool_if:NTF \l_stex_key_T_bool {true}{false}
9792         }}{
9793           #2~
9794           \stex_annotate:nn{data-ftml-problem-choice-verdict={}}{
9795             \bool_if:NTF \l_stex_key_T_bool {
9796               \tl_if_empty:NTF \l_stex_key_Ttext_tl \problem@kw@correct \l_stex_key_Ttext_t

```

```

9797         }{
9798         \tl_if_empty:NTF \l_stex_key_Ftext_tl \problem@kw@wrong \l_stex_key_Ftext_tl
9799         }
9800     }
9801     \tl_if_empty:NF \l_stex_key_feedback_tl {
9802         \stex_annotate:nn{data-ftml-problem-choice-feedback={}}{
9803             \l_stex_key_feedback_tl
9804         }
9805     }
9806 }
9807 }
9808 }
9809 }
9810 }{
9811     \tl_set:Nn\problem_scc_box_default_tl{${\bigcirc}$}
9812     \tl_set:Nn\problem_scc_box_tl{
9813         \ifsolutions
9814             \bool_if:NTF \l_stex_key_T_bool {
9815                 \makebox[Opt][l]{\problem_scc_box_default_tl}
9816                 \hspace{0.1em}${\cs_if_exist:NTF\checkmark{
9817                     \raisebox{.15ex}{\checkmark}
9818                 }} X$
9819             }{\problem_scc_box_default_tl}
9820         \else
9821             \problem_scc_box_default_tl
9822         \fi
9823     }
9824     \newcommand\scc[2][]{
9825         \stex_keys_set:nn{mcc}{#1}
9826         \ifsolutions
9827             \tl_set:Nx \l_tmpa_tl{
9828                 \bool_if:NTF \l_stex_key_T_bool {
9829                     \tl_if_empty:NF \l_stex_key_Ttext_tl {\exp_not:N \l_stex_key_Ttext_tl}
9830                 }{
9831                     \tl_if_empty:NF \l_stex_key_Ftext_tl {\exp_not:N \l_stex_key_Ftext_tl}
9832                 }
9833                 \tl_if_empty:NF \l_stex_key_feedback_tl {
9834                     \exp_not:n{ \\emph{\l_stex_key_feedback_tl} }
9835                 }
9836             }
9837         \fi
9838
9839         \__problems_sccline:n{ #2
9840             \ifsolutions
9841                 \tl_if_empty:NF \l_tmpa_tl { \footnote{\l_tmpa_tl} }
9842             \fi
9843         }
9844     }
9845 }
9846
9847 \stex_deactivate_macro:Nn \scc {scb-environments}
9848
9849
9850 \newcommand\yesTnoF{

```

```

9851 \begin{scb}[style=inline]
9852 \scc[T]{yes}~\scc[F]{no}
9853 \end{scb}
9854 }
9855 \newcommand\yesFnoT{
9856 \begin{scb}[style=inline]
9857 \scc[F]{yes}~\scc[T]{no}
9858 \end{scb}
9859 }
9860 \newcommand\trueTfalseF{
9861 \begin{scb}[style=inline]
9862 \scc[T]{true}~\scc[F]{false}
9863 \end{scb}
9864 }
9865 \newcommand\trueFfalseT{
9866 \begin{scb}[style=inline]
9867 \scc[F]{true}~\scc[T]{false}
9868 \end{scb}
9869 }

```

(End of definition for \scc. This function is documented on page ??.)

\fillinsol

```

9870 \stex_keys_define:nnnn{fillinsol}{
9871 \tl_clear:N \l__problems_fillin_solution_tl
9872 \dim_zero:N \l_stex_key_testspace_dim
9873 }{
9874 \testspace .dim_set:N = \l_stex_key_testspace_dim,
9875 \exact .code:n = {\__problems_parse_fillin_arg:nnnn{exact}#1 },
9876 \numrange .code:n = {\__problems_parse_fillin_arg:nnnn{numrange}#1 },
9877 \regex .code:n = {\__problems_parse_fillin_arg:nnnn{regex}#1 }
9878 }{}
9879
9880 \tl_const:Nn \c__problems_true_tl{T}
9881 \tl_const:Nn \c__problems_false_tl{F}
9882
9883 \msg_set:nnn{problem}{error/neither-true-nor-false}{
9884 \Fillinsol~verdict~#1~is~neither~T~nor~F!
9885 }
9886
9887 \stex_if_html_backend:TF{
9888 \cs_new:Nn \__problems_parse_fillin_arg:nnnn {
9889 \tl_set:Nn \l_tmpa_tl{#3}
9890 \tl_if_eq:NNF \l_tmpa_tl\c__problems_true_tl{
9891 \tl_if_eq:NNF \l_tmpa_tl\c__problems_false_tl {
9892 \msg_error:nnn{problem}{error/neither-true-nor-false}{#3}
9893 }
9894 }
9895 \tl_put_right:Ne \l__problems_fillin_solution_tl {
9896 \stex_annotate:nn{
9897 data-ftml-fillin-case={#1},
9898 data-ftml-fillin-case-value={\tl_to_str:n{#2}},
9899 data-ftml-fillin-case-verdict={
9900 \tl_if_eq:NNTF\l_tmpa_tl\c__problems_true_tl{true}{false}

```

```

9901         },
9902     }{\stex_annotate_force_break:n{\exp_not:n{#4}}}}
9903 }
9904 }
9905 }{
9906     \cs_new:Nn \__problems_parse_fillin_arg:nnnn {
9907         \tl_put_right:Nn \l__problems_fillin_solution_tl {
9908             #1 & \tl_to_str:n{#2} & #3 & #4 \crr
9909         }
9910     }
9911 }
9912
9913
9914 \newcommand\fillinsol[2][{}{
9915     \stex_if_do_html:F \quad
9916     \mode_if_math:TF{
9917         \hbox{\__problems_fillinsol:nn{#1}{#2}}
9918     }{
9919         \__problems_fillinsol:nn{#1}{#2}
9920     }
9921     \stex_if_do_html:F \quad
9922 }
9923
9924 \stex_if_html_backend:TF{
9925     \cs_new_protected:Nn \__problems_fillinsol:nn {
9926         \stex_keys_set:nn{fillinsol}{#1}
9927         \tl_if_empty:nF{#2}{
9928             \__problems_parse_fillin_arg:nnnn{exact}{#2}{T}{}
9929         }
9930         \hbox_set:Nn \l_tmpa_box{\fbox{\huge{\texttt{\tl_to_str:n{#2}}}\hskip\l_stex_key_testsp
9931         \exp_args:Ne \stex_annotate:nn{
9932             data-ftml-fillinsol={},
9933             data-ftml-fillinsol-width={\dim_to_decimal:n{1.5 \box_wd:N \l_tmpa_box }}
9934         }{
9935             \stex_annotate_force_break:n{
9936                 \l__problems_fillin_solution_tl
9937                 #2
9938             }
9939         }
9940     }
9941 }{
9942     \cs_new_protected:Nn \__problems_fillinsol:nn {
9943         \stex_keys_set:nn{fillinsol}{#1}
9944         \ifsolutions
9945
9946         \cs_if_exist:NT\textcolor{\textcolor{red}}{\fbox{\texttt{\tl_to_str:n{#2}}}}
9947         \tl_if_empty:NF \l__problems_fillin_solution_tl {
9948             \footnote{
9949                 \halign{ ~~~~\hfil & ~~~~\hfil & ~~~~\hfil & ~~~~\hfil \cr
9950                     \textbf{type}&\textbf{case}&\textbf{verdict}&\textbf{feedback}\cr
9951                     \tl_if_empty:nF{#2}{
9952                         exact & #2 & T & \cr
9953                     }
9954                     \l__problems_fillin_solution_tl

```

```

9955     }
9956   }
9957 }
9958 \else
9959   \fbox{\dim_compare:nNnTF\l_stex_key_testspace_dim={0pt}{
9960     \phantom{\huge\tl_to_str:n{#2}}}}
9961   }{
9962     \hspace{\l_stex_key_testspace_dim}\vphantom{\huge{A \tl_to_str:n{#2}}}}
9963   }}
9964 \fi
9965 }
9966 }
9967
9968 \stex_deactivate_macro:Nn \fillinsol {sproblem-environments}

```

(End of definition for `\fillinsol`. This function is documented on page 117.)

`\testemptypage`

```

9969 \newcommand\testemptypage[1][\%
9970 \bool_if:NT \c__problems_test_bool {\ \vfill\begin{center}\hwexam@kw@testemptypage\end{center}}
9971 }

```

(End of definition for `\testemptypage`. This function is documented on page ??.)

`\testspace`

```

9972 \newcommand\testspace[1]{\bool_if:NT \c__problems_test_bool {\vspace*{#1}}}
9973 \newcommand\testsmallspace{\testspace{1cm}}
9974 \newcommand\testmedspace{\testspace{2cm}}
9975 \newcommand\testbigspace{\testspace{3cm}}

```

(End of definition for `\testspace`. This function is documented on page ??.)

`\testnewpage`

```

9976 \newcommand\testnewpage{\bool_if:NT \c__problems_test_bool {\newpage}}

```

(End of definition for `\testnewpage`. This function is documented on page ??.)

`\testnewpage`

```

9977 \newcommand{\testnewpageInProblem}%
9978 {\ifintest\vfill\begin{center}\emph{continued-on-next-page}\end{center}\testnewpage\fi}%

```

(End of definition for `\testnewpage`. This function is documented on page ??.)

```

9979 \end{package}

```

39.3 Implementation: The hwexam Package

39.3.1 Package Options

The first step is to declare (a few) package options that handle whether certain information is printed or not. Some come with their own conditionals that are set by the options, the rest is just passed on to the `problems` package.

```

9980 \*package
9981 \ProvidesExplPackage{hwexam}{2026/06/24}{4.1.0}{homework assignments and exams}

```

```

9982 \RequirePackage{l3keys2e}
9983
9984 \keys_define:nn {hwexam / pkg}{
9985   multiple .default:n = { false },
9986   multiple .bool_set:N = \c_hwexam_multiple_bool,
9987   qrcode .default:n = { false },
9988   qrcode .bool_set:N = \c_hwexam_qrcode_bool,
9989   unknown .code:n = {
9990     \PassOptionsToPackage{\CurrentOption}{problem}
9991   }
9992 }
9993 \ProcessKeysOptions{ hwexam /pkg }
9994 \RequirePackage{problem}

```

\hwexam_kw_* For multilinguality, we define internal macros for keywords that can be specialized in *.ldf files.

```

9995 \AddToHook{begindocument}{
9996   \ExplSyntaxOn\makeatletter
9997   \input{hwexam-english.ldf}
9998   \ltx@ifpackageloaded{babel}{
9999     \clist_set:Nx \l_tmpa_clist {\exp_args:No \tl_to_str:n \bbl@loaded}
10000     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{ngerman}}{
10001       \input{hwexam-ngerman.ldf}
10002     }
10003     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{finnish}}{
10004       \input{hwexam-finnish.ldf}
10005     }
10006     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{french}}{
10007       \input{hwexam-french.ldf}
10008     }
10009     \exp_args:NNx \clist_if_in:NnT \l_tmpa_clist {\detokenize{russian}}{
10010       \input{hwexam-russian.ldf}
10011     }
10012   }{ }
10013   \makeatother\ExplSyntaxOff
10014 }

```

(End of definition for \hwexam_kw_*. This function is documented on page ??.)

39.3.2 QR Codes

```

10015 \group_begin:
10016   \escapechar=-1
10017   \xdef\__qr_backslash{\string\}
10018 \group_end:
10019
10020 \bool_if:NT \c_hwexam_qrcode_bool {
10021   \RequirePackage{qrcode}
10022   \RequirePackage{marginnote}
10023   \str_new:N \g__qr_json_str
10024   \bool_new:N \g__qr_in_problems_json_bool
10025   \bool_set_false:N \g__qr_in_problems_json_bool
10026   \bool_new:N \g__qr_in_subproblems_json_bool
10027   \bool_set_false:N \g__qr_in_subproblems_json_bool

```



```

10028 \bool_new:N \g_@@_qr_in_anscls_json_bool
10029 \bool_set_false:N \g_@@_qr_in_anscls_json_bool
10030 \bool_if:NTF \c__problems_gnotes_bool {
10031   \gdef \qrjson { \str_gput_right:Nx \g_@@_qr_json_str}
10032 }{
10033   \gdef \qrjson #1 {}
10034 }
10035 \qrjson{[]}
10036 \AtEndDocument{\qrjson{}}
10037 \def\_@@_qr_escape_char:n #1 {
10038   #1\exp_args:Nno\use:nn{\if_charcode:w#1}\_@@_qr_backslash#1\fi
10039 }
10040
10041 \gdef\_@@_qr_escape:n #1{\exp_args:Ne\str_map_function:nN{\tl_to_str:n{#1}}\_@@_qr_escape_
10042 \gdef \_@@_qr_escape:o #1{\exp_args:No\_@@_qr_escape:n#1}
10043
10044 \def\qrschema{T0:D0:\examnumber}
10045
10046 \bool_if:NTF \c__problems_test_bool {
10047   \def\doproblemqr{
10048     \ifstexhtml\else{
10049       \reversemarginpar\marginnote{\qrcline[height=1.5cm]{\qrschema}}
10050       \normalmarginpar
10051     }\fi
10052   }
10053   \def\insertexamnumber{
10054     \ifstexhtml\else
10055       \tl_if_exist:NTF \examnumber {
10056         {\Large\bfseries ID:~\examnumber}
10057       }{
10058         {\color{red}\Large\bfseries!!!~ WARNING:~NO~{\string\examnumber}~SET~!!!}\gdef\examnum
10059       }
10060       \global\def\insertexamnumber{}
10061     \fi
10062   }
10063 }{
10064   \def\doproblemqr{}
10065   \def\insertexamnumber{}
10066 }
10067
10068 \stexstyleproblem[noqr]{
10069   \par\noindent\problemheader \xdef\grid{\thesproblem}
10070   \bool_if:NT \c__problems_pts_bool {
10071     \tl_if_eq:NnF \l__problems_pts_tl {0}{
10072       \marginpar{\l__problems_pts_tl}{~\problem@kw@points\smallskip}
10073     }
10074   }
10075   \bool_if:NT \c__problems_min_bool {
10076     \tl_if_eq:NnF \l__problems_min_tl {0}{
10077       \marginpar{\l__problems_min_tl}{~\problem@kw@minutes\smallskip}
10078     }
10079   }
10080
10081   \bool_if:NTF \g_@@_qr_in_problems_json_bool {

```

```

10082 \qrjson{,}
10083 }{
10084 \bool_gset_true:N \g_@@_qr_in_problems_json_bool
10085 }
10086
10087 \qrjson {
10088 \c_left_brace_str
10089 "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl
10090 "number": "\thesproblem"
10091 }
10092
10093 \tl_if_eq:NnF \l__problems_pts_tl {0}{
10094 \qrjson {
10095 , "pts": \l__problems_pts_tl
10096 }
10097 }
10098 \par
10099 \stex_ignore_spaces_and_pars:
10100 }{
10101 \bool_if:NT \g_@@_qr_in_subproblems_json_bool {
10102 \qrjson {}}
10103 }
10104 \bool_gset_false:N \g_@@_qr_in_subproblems_json_bool
10105 \qrjson {\c_right_brace_str}
10106 \par\bigskip
10107 }
10108
10109 \stexstyleproblem{
10110 \par\noindent\problemheader \xdef\grid{\thesproblem}
10111 \doproblemqr
10112 \bool_if:NT \c__problems_pts_bool {
10113 \tl_if_eq:NnF \l__problems_pts_tl {0}{
10114 \marginpar{\l__problems_pts_tl}{~\problem@kw@points\smallskip}
10115 }
10116 }
10117 \bool_if:NT \c__problems_min_bool {
10118 \tl_if_eq:NnF \l__problems_min_tl {0} {
10119 \marginpar{\l__problems_min_tl}{~\problem@kw@minutes\smallskip}
10120 }
10121 }
10122
10123 \bool_if:NTF \g_@@_qr_in_problems_json_bool {
10124 \qrjson{,}
10125 }{
10126 \bool_gset_true:N \g_@@_qr_in_problems_json_bool
10127 }
10128
10129 \qrjson {
10130 \c_left_brace_str
10131 "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl
10132 "number": "\thesproblem"
10133 }
10134
10135 \tl_if_eq:NnF \l__problems_pts_tl {0}{

```

```

10136 \qrjson {
10137   , "pts": \l__problems_pts_tl
10138 }
10139 }
10140 \par
10141 \stex_ignore_spaces_and_pars:
10142 }{
10143   \bool_if:NT \g_@@_qr_in_subproblems_json_bool {
10144     \qrjson {}
10145   }
10146   \bool_gset_false:N \g_@@_qr_in_subproblems_json_bool
10147   \qrjson {\c_right_brace_str}
10148   \par\bigskip
10149 }
10150
10151 \stexstylesubproblem[noqr]{
10152   \begin{list}{}{
10153     \setlength\topsep{0pt}
10154     \setlength\parsep{0pt}
10155     \setlength\rightmargin{0pt}
10156   } \item[\int_use:N \g__problems_subproblem_int .]
10157   \xdef\grid{\thesproblem.\int_use:N \g__problems_subproblem_int}
10158   \bool_if:NT \c__problems_pts_bool {
10159     \bool_if:NF \l__problems_has_pts_bool {
10160       \marginpar{\smallskip\l_stex_key_pts_tl}{~\problem@kw@points}
10161     }
10162   }
10163   \bool_if:NT \c__problems_min_bool {
10164     \bool_if:NF \l__problems_has_min_bool{
10165       \marginpar{\smallskip\l_stex_key_min_tl}{~\problem@kw@minutes}
10166     }
10167   }
10168   \bool_if:NTF \g_@@_qr_in_subproblems_json_bool {
10169     \qrjson {,}
10170   }{
10171     \qrjson {
10172       , "subproblems": [
10173       }
10174       \bool_gset_true:N \g_@@_qr_in_subproblems_json_bool
10175     }
10176     \qrjson {
10177       \c_left_brace_str
10178       "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl
10179       "number": "\thesproblem.\int_use:N \g__problems_subproblem_int"
10180     }
10181     \bool_if:NF \l__problems_has_pts_bool {
10182       \qrjson {
10183         , "pts": \l_stex_key_pts_tl
10184       }
10185     }
10186   }{
10187     \qrjson {\c_right_brace_str}
10188   \end{list}
10189 }

```

```

10190
10191 \stexstylesubproblem{
10192   \begin{list}{}{
10193     \setlength\topsep{0pt}
10194     \setlength\parsep{0pt}
10195     \setlength\rightmargin{0pt}
10196   }\item[\int_use:N \g__problems_subproblem_int .]
10197   \xdef\grid{\thesproblem.\int_use:N \g__problems_subproblem_int}
10198   \doproblemqr
10199   \bool_if:NT \c__problems_pts_bool {
10200     \bool_if:NF \l__problems_has_pts_bool {
10201       \marginpar{\smallskip\l_stex_key_pts_tl}{~\problem@kw@points}
10202     }
10203   }
10204   \bool_if:NT \c__problems_min_bool {
10205     \bool_if:NF \l__problems_has_min_bool{
10206       \marginpar{\smallskip\l_stex_key_min_tl}{~\problem@kw@minutes}
10207     }
10208   }
10209   \bool_if:NTF \g_@@_qr_in_subproblems_json_bool {
10210     \qrjson {,}
10211   }{
10212     \qrjson {
10213       , "subproblems": [
10214     }
10215     \bool_gset_true:N \g_@@_qr_in_subproblems_json_bool
10216   }
10217   \qrjson {
10218     \c_left_brace_str
10219     "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl"
10220     "number": "\thesproblem.\int_use:N \g__problems_subproblem_int"
10221   }
10222   \bool_if:NF \l__problems_has_pts_bool {
10223     \qrjson {
10224       , "pts": \l_stex_key_pts_tl
10225     }
10226   }
10227 }{
10228   \qrjson {\c_right_brace_str}
10229   \end{list}
10230 }
10231
10232 \stexstylegnote{
10233   \par\smallskip\rule[.3em]{\linewidth}{0.4pt}\newline\smallskip
10234   \noindent\emph{\problem@kw@grading\str_if_empty:NF \l_stex_key_title_tl{
10235     {~}\l_stex_key_title_tl
10236   } :~}
10237   \qrjson {
10238     , "gnote": \c_left_brace_str
10239     "id": "\_@@_qr_escape:o\l_stex_key_id_str", "title": "\_@@_qr_escape:o\l_stex_key_title_tl"
10240     "anscls": [
10241   }
10242 }{
10243   \qrjson {

```

```

10244 ]\c_right_brace_str
10245 }
10246 \par\rule[.3em]{\linewidth}{0.4pt}\newline
10247 }
10248
10249 \renewcommand \anscls [2] [] {
10250   \stex_keys_set:nn{ anscls }{#1}
10251   \str_if_empty:NT \l_stex_key_id_str {
10252     \int_incr:N \l__problems_anscls_int
10253     \str_set:Nx \l_stex_key_id_str {
10254       AC\int_use:N \l__problems_anscls_int
10255     }
10256   }
10257   \bool_if:NTF \g_@@_qr_in_anscls_json_bool {
10258     \qrjson{,}
10259   }{
10260     \bool_set_true:N \g_@@_qr_in_anscls_json_bool
10261   }
10262   \qrjson{
10263     \c_left_brace_str
10264     "id": "\_@@_qr_escape:o\l_stex_key_id_str", "description": "\_@@_qr_escape:n{#2}",
10265     "feedback": "\_@@_qr_escape:o\l_stex_key_feedback_tl",
10266     "pts": "\l_stex_key_pts_str"
10267   \c_right_brace_str
10268   }
10269   \begin{list}{}{
10270     \setlength\topsep{Opt}
10271     \setlength\parsep{Opt}
10272     \setlength\rightmargin{Opt}
10273   }\item[\l_stex_key_id_str]
10274     \stex_if_do_html:TF{
10275       \exp_args:Ne \stex_annotate:nn{
10276         data-ftml-answerclass={\l_stex_key_id_str}
10277         \str_if_empty:NF \l_stex_key_pts_str{
10278           ,data-ftml-answerclass-pts={\l_stex_key_pts_str}
10279         }
10280       }{
10281         #2
10282         \tl_if_empty:NF \l_stex_key_feedback_tl{
10283           \stex_annotate_invisible:nn{
10284             data-ftml-answerclass-feedback={true}
10285           }{\l_stex_key_feedback_tl}
10286         }
10287       }
10288     }{#2}
10289     \str_if_empty:NF \l_stex_key_pts_str {\par
10290       ~ \problem@kw@points :~\l_stex_key_pts_str
10291     }
10292     \str_if_empty:NF \l_stex_key_feedback_tl {\par
10293       ~ \problem@kw@feedback :~\l_stex_key_feedback_tl
10294     }
10295   \end{list}
10296 }
10297 \stex_deactivate_macro:Nn \anscls {gnote-environments}

```

```

10298
10299 \AtEndDocument{
10300   \message{^^J^^J\g_@@_qr_json_str^^J^^J}
10301   \bool_if:NT \c__problems_gnotes_bool {
10302     \iow_new:N \c_@@_qr_json_iow
10303     \iow_open:Nn \c_@@_qr_json_iow {\jobname-vollkorn.json}
10304     \iow_now:Nx \c_@@_qr_json_iow {\g_@@_qr_json_str}
10305     \iow_close:N \c_@@_qr_json_iow
10306   }
10307 }
10308
10309 \renewcommand\testemptypage[1][\%
10310   \bool_if:NT \c__problems_test_bool {\
10311     \xdef\grid{P\thepage}
10312     \doprobblemqr
10313     \vfill\begin{center}\hwexam@kw@testemptypage\end{center}\eject
10314   }
10315 }
10316 }

```

39.3.3 Assignments

Then we set up a counter for problems and make the problem counter inherited from `problem.sty` depend on it. Furthermore, we specialize the `\prob@label` macro to take the assignment counter into account.

`assignment (env.)`

```

10317 \stex_keys_define:nnnn{ assignment }{
10318   \tl_clear:N \l_stex_key_number_tl
10319   \tl_clear:N \l_stex_key_given_tl
10320   \tl_clear:N \l_stex_key_due_tl
10321 }{
10322   number .tl_set:N      = \l_stex_key_number_tl,
10323   given .tl_set:N       = \l_stex_key_given_tl,
10324   due .tl_set:N         = \l_stex_key_due_tl,
10325   unknown .code:n = {}
10326 }{id,title,style}
10327
10328 \newcounter{assignment}
10329 \stex_new_stytable_env:nnnnnn {assignment}{0}{}{
10330   \cs_if_exist:NTF \l_hwexam_includeassignment_keys_tl {
10331     \tl_put_left:Nn \l_hwexam_includeassignment_keys_tl {\#1,}
10332     \exp_args:Nno \stex_keys_set:nn{assignment}{
10333       \l_hwexam_includeassignment_keys_tl
10334     }
10335   }{
10336     \stex_keys_set:nn{assignment}{\#1}
10337   }
10338   \tl_if_empty:NF \l_stex_key_number_tl {
10339     \global\setcounter{assignment}{\int_eval:n{\l_stex_key_number_tl-1}}
10340   }
10341   \global\refstepcounter{assignment}
10342   \setcounter{sproblem}{0}
10343   \def\thesproblem{\theassignment.\arabic{sproblem}}

```

```

10344 \stex_if_do_html:T{
10345   \tl_if_empty:NF \l_stex_key_title_tl {
10346     \stexdoctitle \l_stex_key_title_tl
10347   }
10348 }
10349 \stex_style_apply:
10350 \_stex_do_id:
10351 }{
10352 \stex_style_apply:
10353 }{
10354 \par\begin{center}
10355 \textbf{\Large\assignmentautorefname~\theassignment
10356 \tl_if_empty:NF \l_stex_key_title_tl {
10357   {~}---\l_stex_key_title_tl
10358 }
10359 }\par\smallskip
10360 \textbf{
10361 \tl_if_empty:NF \l_stex_key_given_tl {
10362   \hwexam@kw@given :~\l_stex_key_given_tl\quad
10363 }
10364 \tl_if_empty:NF \l_stex_key_due_tl {
10365   \hwexam@kw@due :~\l_stex_key_due_tl\quad
10366 }
10367 }
10368 \end{center}
10369 \par\bigskip
10370 }{
10371 \par\pagebreak
10372 }{}

```

\includeassignment

```

10373 \NewDocumentCommand\includeassignment{0{} m}{
10374   \group_begin:
10375   \tl_set:Nn \l_hwexam_includeassignment_keys_tl {#1}
10376   \stex_keys_set:nn{includeproblem}{#1}
10377   \exp_args:Nno \use:nn{\inputref[]\l_stex_key_mhrepos_str}{#2}
10378   \group_end:
10379 }

```

(End of definition for \includeassignment. This function is documented on page 76.)

Restoring information about problems:

```

10380 \prop_new:N \c_@@_problems_prop
10381 \tl_set:Nn \c_@@_total_mins_tl {0pt}
10382 \tl_set:Nn \c_@@_total_pts_tl {0pt}
10383 \int_new:N \c_@@_total_problems_int
10384 \cs_set_protected:Npn \problem@restore #1 #2 #3 {
10385   \int_gincr:N \c_@@_total_problems_int
10386   \prop_gput:Nnn \c_@@_problems_prop {#1}{#2}{#3}
10387   \tl_gset:Ne \c_@@_total_pts_tl { \dim_eval:n { \c_@@_total_pts_tl + #2pt }}
10388   \tl_gset:Ne \c_@@_total_mins_tl { \dim_eval:n { \c_@@_total_mins_tl + #2pt }}
10389 }

```

\correction@table This macro generates the correction table

```

10390 \newcommand\correction@table{

```

```

10391 \int_compare:nNnT \c_@@_total_problems_int = 0 {
10392   \int_incr:N \c_@@_total_problems_int
10393   \prop_put:Nnn \c_@@_problems_prop {~}{~}{~}}
10394 }
10395 \tl_clear:N \l_tmpa_tl
10396 \tl_clear:N \l_tmpb_tl
10397 \tl_clear:N \l_tmpc_tl
10398 \prop_map_inline:Nn \c_@@_problems_prop {
10399   \tl_put_right:Nn \l_tmpa_tl { ##1 & }
10400   \tl_put_right:Nx \l_tmpb_tl { \use_i:nn ##2 & }
10401   \tl_put_right:Nn \l_tmpc_tl { & }
10402 }
10403 \resizebox{\textwidth}{!}{%
10404 \exp_args:Nne \begin{tabular}{|l|*{\int_use:N \c_@@_total_problems_int}{c|}c|l|}\hline
10405 &\exp_args:Ne \multicolumn{\int_eval:n{ \c_@@_total_problems_int + 1}}{c|}
10406 {\footnotesize\hwexam@kw@forgrading} &\\ \hline
10407 \hwexam@kw@probs & \l_tmpa_tl \hwexam@kw@sum & \hwexam@kw@grade\\ \hline
10408 \hwexam@kw@pts & \l_tmpb_tl \dim_to_decimal:n\c_@@_total_pts_tl & \\ \hline
10409 \hwexam@kw@reached & \l_tmpc_tl & \[.7cm]\hline
10410 \end{tabular}}

```

(End of definition for \correction@table. This function is documented on page ??.)

\testheading

```

10411 \def\hwexamheader{\input{hwexam-default.header}}
10412
10413 \def\hwexamminutes{
10414   \tl_if_empty:NTF \hwexam@duration {
10415     {\hwexam@min}~\hwexam@minutes@kw
10416   }{
10417     \hwexam@duration
10418   }
10419 }
10420
10421 \stex_keys_define:nnnn{ hwexam / testheading }{
10422   \tl_clear:N \hwexam@min
10423   \tl_clear:N \hwexam@duration
10424   \tl_clear:N \hwexam@reqpts
10425   \tl_clear:N \hwexam@tools
10426 }{
10427   min .tl_set:N = \hwexam@min,
10428   duration .tl_set:N = \hwexam@duration,
10429   reqpts .tl_set:N = \hwexam@reqpts,
10430   tools .tl_set:N = \hwexam@tools
10431 }{}
10432
10433 \newenvironment{testheading}[1][]{
10434   \stex_keys_set:nn { hwexam / testheading}{#1}
10435
10436   \tl_set:Nx \hwexam@totalpts {\dim_to_decimal:n \c_@@_total_pts_tl}
10437   \tl_set:Nx \hwexam@totalmin {\dim_to_decimal:n \c_@@_total_mins_tl}
10438   \tl_set:Nx \hwexam@checktime {\dim_to_decimal:n { \hwexam@min pt - \hwexam@totalmin pt }}
10439
10440   \newif\if@bonuspoints

```



```

10441 \tl_if_empty:NTF \hwexam@reqpts {
10442   \@bonuspointsfalse
10443 }{
10444   \tl_set:Nx \hwexam@bonuspts {
10445     \dim_to_decimal:n{\hwexam@totalpts pt - \hwexam@reqpts pt}
10446   }
10447   \@bonuspointstrue
10448 }
10449
10450 \makeatletter\hwexamheader\makeatother
10451 }{
10452   \newpage
10453 }

```

(End of definition for \testheading. This function is documented on page ??.)

```

examdata
quizdata
homeworkdata
10454 \bool_new:N \g_@@_allow_data_bool
10455 \bool_gset_true:N \g_@@_allow_data_bool
10456 \AtBeginDocument{
10457   \bool_gset_false:N \g_@@_allow_data_bool
10458 }
10459
10460 \msg_set:nnn{stex}{hwexam/doubledata}{
10461   Exam/Homework/Quiz~Data~already~set
10462 }
10463
10464 \msg_set:nnn{stex}{hwexam/nodate}{
10465   Exam/Homework/Quiz~Data~missing~date~value
10466 }
10467
10468 \msg_set:nnn{stex}{hwexam/nocourse}{
10469   Exam/Homework/Quiz~Data~missing~course~value
10470 }
10471
10472 \stex_keys_define:nnnn{ hwexam / commondata }{
10473   % https://docs.rs/dateparser/latest/dateparser/#accepted-date-formats
10474   \str_clear:N \l_@@_key_exam_date_str
10475   \str_clear:N \l_@@_key_exam_course_id_str
10476   \str_clear:N \l_@@_key_exam_term_str
10477   \int_set:Nn \l_@@_key_exam_num_int {1}
10478 }{
10479   date      .str_set:N = \l_@@_key_exam_date_str,
10480   course    .str_set:N = \l_@@_key_exam_course_id_str,
10481   term      .str_set:N = \l_@@_key_exam_term_str,
10482   num       .int_set:N = \l_@@_key_exam_num_int
10483 }{}
10484
10485 \stex_keys_define:nnnn{ hwexam / examdata }{
10486   \bool_set_false:N \l_@@_key_exam_retake_bool
10487 }{
10488   retake    .bool_set:N = \l_@@_key_exam_retake_bool
10489 }{hwexam/commondata}
10490

```

```

10491 \cs_set_protected:Nn \_@@_do_metadata:nnnn {
10492   \bool_if:NF \g_@@_allow_data_bool {
10493     \msg_fatal:nn{stex}{hwexam/doubledata}
10494   }
10495   \bool_gset_false:N \g_@@_allow_data_bool
10496   \stex_keys_set:nn { hwexam / #1}{#2}
10497   \str_set:Nn \g_stex_document_kind_str{#4}
10498   \str_if_empty:NT\l_@@_key_exam_date_str{
10499     \str_set:Ne \l_@@_key_exam_date_str \today
10500     \%msg_fatal:nn{stex}{hwexam/nodate}
10501   }
10502   \str_if_empty:NT\l_@@_key_exam_course_id_str{
10503     \msg_fatal:nn{stex}{hwexam/nocourse}
10504   }
10505   \cs_if_exist:NT \date {
10506     \exp_args:No \date \l_@@_key_exam_date_str
10507   }
10508
10509   \tl_set:Ne \g_stex_document_kind_parameters_tl{
10510     ,data-ftml-document-kind-date={\l_@@_key_exam_date_str}
10511     ,data-ftml-document-kind-num=\int_use:N \l_@@_key_exam_num_int
10512     ,data-ftml-document-kind-course=\l_@@_key_exam_course_id_str
10513     \str_if_empty:NF \l_@@_key_exam_term_str{
10514       ,data-ftml-document-kind-term=\l_@@_key_exam_term_str
10515     }
10516     #3
10517   }
10518 }
10519
10520 \newcommand\examdata[1]{
10521   \_@@_do_metadata:nnnn{examdata}{#1}{
10522     ,data-ftml-document-kind-retake=\bool_if:NTF \l_@@_key_exam_retake_bool{true}{false}
10523   }{exam}
10524 }
10525
10526 \newcommand\homeworkdata[1]{
10527   \_@@_do_metadata:nnnn{commondata}{#1}{}{homework}
10528 }
10529
10530 \newcommand\quizdata[1]{
10531   \_@@_do_metadata:nnnn{commondata}{#1}{}{quiz}
10532 }
10533
10534 \end{package}

```

(End of definition for examdata, quizdata, and homeworkdata. These functions are documented on page ??.)

39.3.4 Leftovers

at some point, we may want to reactivate the logos font, then we use

```

here we define the logos that characterize the assignment
\font\bierfont=../assignments/bierglas

```

```

\font\denkerfont=../assignments/denker
\font\uhrfont=../assignments/uhr
\font\warnschildfont=../assignments/achtung

\newcommand\bierglas{{\bierfont\char65}}
\newcommand\denker{{\denkerfont\char65}}
\newcommand\uhr{{\uhrfont\char65}}
\newcommand\warnschild{{\warnschildfont\char 65}}
\newcommand\hardA{\warnschild}
\newcommand\longA{\uhr}
\newcommand\thinkA{\denker}
\newcommand\discussA{\bierglas}

```

39.4 Tikzinput Implementation

```

10535 <@@=tikzinput>
10536 <*package>
10537
10538 %%%%%%%%%%% tikzinput.dtx %%%%%%%%%%%
10539
10540 \ProvidesExplPackage{tikzinput}{2025/11/11}{4.0.0}{tikzinput package}
10541 \RequirePackage{l3keys2e}
10542
10543 \keys_define:nn { tikzinput } {
10544   image .bool_set:N = \c_tikzinput_image_bool,
10545   image .default:n = false ,
10546   unknown .code:n = {}
10547 }
10548
10549 \ProcessKeysOptions { tikzinput }
10550
10551 \bool_if:NTF \c_tikzinput_image_bool {
10552   \RequirePackage{graphicx}
10553
10554   \providecommand\usetikzlibrary[]{}
10555   \newcommand\tikzinput[2] [] {\includegraphics[#1]{#2}}
10556 }{
10557   \RequirePackage{tikz}
10558   \RequirePackage{standalone}
10559
10560   \newcommand \tikzinput [2] [] {
10561     \setkeys{Gin}{#1}
10562     \ifx \Gin@ewidth \Gin@exclamation
10563       \ifx \Gin@eheight \Gin@exclamation
10564         \input { #2 }
10565       \else
10566         \resizebox{!}{ \Gin@eheight }{
10567           \input { #2 }
10568         }
10569       \fi
10570     \else

```

```

10571     \ifx \Gin@eheight \Gin@exclamation
10572         \resizebox{ \Gin@ewidth }{!}{
10573             \input { #2 }
10574         }
10575     \else
10576         \resizebox{ \Gin@ewidth }{ \Gin@eheight }{
10577             \input { #2 }
10578         }
10579     \fi
10580 \fi
10581 }
10582 }
10583
10584 \newcommand \ctikzinput [2] [] {
10585     \begin{center}
10586         \tikzinput [#1] {#2}
10587     \end{center}
10588 }
10589 \end{package}

```

Index

The italic numbers denote the pages where the corresponding entry is described, numbers underlined point to the definition, all others indicate the places where it is used.

Symbols	
\!	6725
\\$	2177, 2180, 2184
* commands:	
*:n	145
\,	6725
\:	617
\;	8188, 8195, 8240, 8242, 8272
@@ commands:	
\g_@@_allow_data_bool	10454, 10455, 10457, 10492, 10495
_@@_do_metadata:nnnn	10491, 10521, 10527, 10531
\l_@@_key_exam_course_id_str	10475, 10480, 10502, 10512
\l_@@_key_exam_date_str	10474, 10479, 10498, 10499, 10506, 10510
\l_@@_key_exam_num_int	10477, 10482, 10511
\l_@@_key_exam_retake_bool	10486, 10488, 10522
\l_@@_key_exam_term_str	10476, 10481, 10513, 10514
\c_@@_problems_prop	10380, 10386, 10393, 10398
_@@_qr_escape:n	10041, 10042, 10089, 10131, 10178, 10219, 10239, 10264, 10265
_@@_qr_escape_char:n	10037, 10041
\g_@@_qr_in_anscls_json_bool	10028, 10029, 10257, 10260
\g_@@_qr_in_problems_json_bool	10024, 10025, 10081, 10084, 10123, 10126
\g_@@_qr_in_subproblems_json_-bool	10026, 10027, 10101, 10104, 10143, 10146, 10168, 10174, 10209, 10215
\c_@@_qr_json_iow	10302, 10303, 10304, 10305
\g_@@_qr_json_str	10023, 10031, 10300, 10304
\c_@@_total_mins_tl	10381, 10388, 10437
\c_@@_total_problems_int	10383, 10385, 10391, 10392, 10404, 10405
c@@_total_pts_tl	10382, 10387, 10408, 10436
\\	88, 115, 842, 1464, 8423, 8428, 8846, 8849, 9639, 9700, 9834, 10017, 10406, 10407, 10408, 10409
\{	8245
\}	8245
\sqcup	19, 79, 619, 756, 8199, 8277, 8455, 9970, 10310
_	618
_@@_qr_backslash	10017, 10038
_comp	3909, 4667, 5965, 6180, 6332, 7077
_customthiscomp	5027, 5028, 5040, 5045, 5048, 5053
_defcomp	5998, 6164
_thiscomp	4796, 4797, 5969
_varcomp	4354, 4513, 5295, 5316, 5866, 5987, 6095, 6109, 6123
\l	2176, 2179, 2183
A	
\activateexcursion	64, 112
\addbibresource	82, 141, 2193
\addcontentsline	8453
\addmhbibresource	82, 141, 2186
\addtocounter	8648
\AddToHook	1793, 7834, 7974, 7978, 7982, 8109, 9040, 9995
\aftergroup	125, 126, 2942, 2964, 3013, 4630, 7900, 7901, 8105
\afterprematurestop	63, 111, 8517, 8528
\amalg	8262
\anscls	9481, 9514, 9556, 10249, 10297
\apply	90
\arabic	9081, 9084, 10343
\arg	40, 95, 5416
\argarraymap	94, 154, 6412, 6967
\argmap	94, 154, 297, 6411, 6728, 6943
\argsep	36, 94, 154, 5799, 6410, 6723, 6927, 8195, 8196, 8197, 8205, 8252, 8256, 8260
\assign	58, 146, 3334, 3593
assignment (env.)	76, 119, 10317
\assignmentautorefname	10355
\assignMorphism	146, 3335, 3698
\assumption	158, 7730, 8113
\ast	8220

`\AtBeginDocument` 69, 77,
 806, 814, 1483, 1510, 1691, 1716,
 1797, 2960, 8299, 8472, 8987, 10456
`\AtEndDocument` . . . 627, 1798, 10036, 10299
`\AtEndOfPackageFile` 2229, 2245, 2260
`\author` 8826
`\autoref` 86, 1863, 1879

B

`\backmatter` 210, 1758, 1759, 1761
`\baselineskip` 8784
`\beameritemnestingprefix` 8924
`\begin` 98, 128, 139, 142–144,
 157, 1470, 1582, 1645, 2088, 2243,
 2258, 2278, 2622, 2780, 2798, 2813,
 3089, 6990, 7327, 7991, 8456, 8468,
 8594, 8621, 8631, 8670, 8691, 8699,
 8745, 8746, 8747, 8748, 8749, 8750,
 8756, 8758, 8796, 8798, 8802, 8864,
 8876, 8911, 8912, 9122, 9222, 9249,
 9302, 9365, 9413, 9464, 9523, 9542,
 9570, 9580, 9735, 9851, 9856, 9861,
 9866, 9970, 9978, 10152, 10192,
 10269, 10313, 10354, 10404, 10585
`\begingroup` 46, 2096, 2601
`\bf` 8783
`\bfseries` 9206, 10056, 10058
`\bgroup` . . 1900, 1998, 4397, 4570, 7743,
 7867, 7956, 8620, 8716, 8722, 8738,
 8846, 8849, 9319, 9379, 9427, 9478
`\bigcirc` 9785, 9811
`\bigskip` 9175, 10106, 10148, 10369
`blindfragment (env.)` 84, 139, 1628
 bool commands:
 `\bool_gset_eq:NN` 3145, 7876
 `\bool_gset_false:N`
 10104, 10146, 10457, 10495
 `\bool_gset_true:N`
 . . 10084, 10126, 10174, 10215, 10455
 `\bool_if:NTF` . . . 473, 581, 600, 601,
 607, 696, 724, 978, 987, 989, 1083,
 1626, 1743, 2403, 2455, 2683, 2998,
 3206, 3346, 3378, 3742, 4412, 4637,
 5290, 5314, 5595, 5605, 5611, 5621,
 5625, 5645, 5659, 5663, 5677, 5706,
 5713, 5727, 6169, 6192, 6250, 6580,
 7006, 7012, 7614, 7743, 7753, 7769,
 7802, 7808, 7836, 7892, 7932, 7956,
 7959, 7966, 7970, 7984, 8031, 8119,
 8359, 8367, 8404, 8488, 8532, 8713,
 8719, 8726, 8760, 8775, 8825, 8875,
 8885, 8972, 8978, 9005, 9028, 9104,
 9125, 9161, 9166, 9176, 9204, 9225,
 9239, 9242, 9254, 9255, 9259, 9260,

 9375, 9386, 9423, 9434, 9475, 9486,
 9558, 9563, 9653, 9657, 9680, 9694,
 9791, 9795, 9814, 9828, 9970, 9972,
 9976, 10020, 10030, 10046, 10070,
 10075, 10081, 10101, 10112, 10117,
 10123, 10143, 10158, 10159, 10163,
 10164, 10168, 10181, 10199, 10200,
 10204, 10205, 10209, 10222, 10257,
 10301, 10310, 10492, 10522, 10551
`\bool_if_exist:NTF` 339, 353
`\bool_lazy_any:nTF` 4243, 4312
`\bool_lazy_any_p:n` 285
`\bool_new:N` 27, 685, 1312,
 1606, 2394, 2680, 3040, 4331, 4658,
 5592, 6006, 7000, 7222, 9060, 9099,
 9100, 9101, 10024, 10026, 10028, 10454
`\bool_not_p:n` 284, 288
`\bool_set_false:N` 455,
 582, 693, 701, 975, 998, 1314, 1621,
 2408, 3047, 3359, 3715, 4661, 6022,
 6320, 6568, 6859, 7007, 7235, 7577,
 7582, 7592, 7597, 7621, 7707, 7842,
 7920, 8103, 8176, 8177, 8343, 8384,
 8548, 9066, 9102, 9142, 9145, 9622,
 9627, 9633, 10025, 10027, 10029, 10486
`\bool_set_true:N` 341, 355,
 463, 571, 577, 690, 705, 972, 1352,
 1611, 2406, 2723, 3043, 3343, 3727,
 3737, 4060, 4417, 4659, 4728, 4998,
 5030, 5055, 5100, 5101, 5401, 5410,
 5525, 5626, 5632, 5650, 5668, 5701,
 5824, 5914, 5947, 5972, 5973, 6011,
 6247, 6575, 7024, 7225, 7233, 7629,
 7639, 7820, 8356, 8396, 8546, 8554,
 8555, 8556, 8557, 8558, 8559, 9138,
 9143, 9146, 9209, 9620, 9632, 10260
`\bool_while_do:Nn` 973
`\bool_while_do:nn`
 283, 2209, 7644, 7656, 7668, 7685, 7697
`\l_tmpa_bool` 3715,
 3727, 3737, 3742, 6568, 6575, 6580

box commands:

`\box_wd:N` 9933
`\c_empty_box`
 1530, 1540, 3935, 9560, 9565
`\l_tmpa_box`
 . . . 2722, 6402, 8716, 8722, 8738,
 9319, 9379, 9427, 9478, 9930, 9933
 boxed 113

C

`\catcode` 19, 20, 46,
 79, 88, 617, 618, 619, 756, 757, 842,

1999, 2175, 2176, 2177, 2178, 2179,
2180, 2182, 2183, 2184, 2514, 2515
\cdot 8240
\centering 8457, 8671
\chapter 26, 84, 8497, 8498
\chaptername 8428, 8494, 8495
\chaptertitlename 8428
\checkmark 9682, 9683, 9816, 9817
\child 125
\circ 47
\clearpage 1752, 1765, 1776, 1787
clist commands:
\clist_clear:N ... 125, 399, 3783,
4729, 4879, 4979, 6301, 7297, 7302
\clist_count:N 4858, 4869
\clist_count:n 4962, 6975
\clist_get:NN 508, 546
\clist_if_empty:NTF
..... 454, 504, 542, 4076, 4857,
4904, 5235, 6499, 7382, 9582, 9746
\clist_if_in:NnTF
.. 104, 107, 131, 7477, 9045, 9048,
9051, 9054, 9587, 9590, 9715, 9722,
9752, 9755, 10000, 10003, 10006, 10009
\clist_if_in:nnTF 4988
\clist_item:Nn 5186, 6508
\clist_item:nn 6986
\clist_map_function:NN ... 4730, 4907
\clist_map_function:nN
..... 3511, 3577, 4161, 6675
\clist_map_inline:Nn
.. 126, 135, 457, 3246, 4078, 4435,
4594, 5236, 7100, 7135, 7280, 7315
\clist_map_inline:nn
363, 412, 5750, 5853, 5882, 7098, 7532
\clist_pop:NN 6510
\clist_put_right:Nn
... 127, 4743, 4747, 4886, 5006, 5008
\clist_set:Nn 42, 123, 4499, 9044, 9999
\clist_set_eq:NN 121, 7332
\l_tmpa_clist 9044, 9045, 9048, 9051,
9054, 9999, 10000, 10003, 10006, 10009
\clstininputmhlisting 85, 141, 2245
\cmhgraphics 85, 141, 2229
\cmhtikzinput 121, 141, 2260
\color 10058
\columnbox 8689, 8691, 8709
\comp 31–
34, 40, 92, 95, 105, 151, 3909, 4354,
4513, 4678, 4679, 4797, 4845, 4853,
5036, 5037, 5047, 5732, 5866, 5957,
6009, 6010, 6095, 6109, 6123, 6164,
6180, 6182, 6332, 6709, 6803, 6809,
6811, 7025, 7030, 7077, 8192, 8194,
8203, 8205, 8207, 8208, 8213, 8215,
8220, 8224, 8225, 8228, 8229, 8234,
8235, 8241, 8242, 8243, 8245, 8252,
8256, 8272, 8277, 8281, 8283, 8285
\comp-macro 151
\compemph 105, 151, 5957, 8989
\conclude 158, 7729, 8114
\conclusion .. 44, 101, 103, 157, 7466, 7540
\copymod 55–57, 145, 3521, 3523, 3525
copymodule (env.) 145, 3461
\copymodule 3496, 3498
\counterwithin 8501, 8513
\cr 9949, 9950, 9952
\crrcr 9908
cs commands:
\cs:w 483, 486, 517, 520,
620, 2459, 2460, 2461, 2467, 2471,
2477, 2736, 7176, 7178, 7356, 7370
\cs_argument_spec:N 4195
\cs_end: 483, 486, 517, 520,
620, 2459, 2460, 2461, 2469, 2473,
2477, 2736, 7176, 7178, 7356, 7370
\cs_generate_from_arg_count:NNnn
..... 4069,
4426, 4585, 4762, 5534, 5570, 6855
\cs_generate_variant:Nn 56,
146, 176, 248, 331, 732, 1330, 2553,
2559, 3037, 4635, 4657, 5529, 6558
\cs_if_eq:NNTF .. 65, 73, 802, 810,
1200, 1205, 1688, 2766, 2780, 2783,
2798, 2801, 4197, 5783, 5785, 6938
\cs_if_exist:NTF
675, 1563, 1567, 1610, 1614, 1630,
1633, 1662, 1666, 1700, 1706, 1717,
1745, 1758, 1773, 1784, 1832, 1833,
1862, 1920, 1974, 2054, 2055, 2057,
2152, 3937, 4192, 4699, 5265, 5268,
5276, 5279, 6587, 6592, 6624, 6630,
6654, 6656, 6696, 6702, 8423, 8428,
8431, 8435, 8439, 8443, 8447, 8494,
8497, 8500, 8506, 8509, 8512, 8673,
8681, 8693, 8703, 8712, 8846, 8849,
9107, 9682, 9816, 9946, 10330, 10505
\cs_meaning:N . 2458, 4103, 6561, 6710
\cs_new:Nn
.. 26, 34, 205, 306, 649, 763, 771,
888, 895, 1098, 1105, 1117, 1120,
1123, 1126, 1137, 1140, 1143, 1146,
1149, 1155, 1161, 1238, 1241, 1244,
1251, 1254, 1257, 1260, 1264, 1285,
1288, 1299, 1307, 1407, 1410, 1413,
1421, 1424, 1427, 1430, 1433, 1436,
1440, 1443, 1446, 1979, 1991, 2281,
2284, 2287, 2290, 2293, 2296, 2299,

2302, 2305, 2485, 2490, 2495, 3174,
 3178, 3973, 4136, 4772, 4915, 4922,
 4925, 4935, 5445, 5542, 5729, 6047,
 6072, 6513, 6524, 6538, 6541, 6597,
 6610, 6760, 6763, 6767, 6814, 7204,
 7373, 7387, 7528, 9086, 9888, 9906
 \cs_new:Npn 726,
 727, 735, 736, 881, 884, 891, 1006,
 1152, 1158, 1291, 1987, 2020, 3322,
 5778, 6051, 6211, 6603, 6616, 7260
 \cs_new_nopar:Nn 359, 374
 \cs_new_protected:Nn
 3, 4, 42, 45, 53, 67,
 73, 77, 87, 88, 97, 98, 103, 113, 141,
 153, 169, 177, 192, 208, 215, 245,
 249, 313, 324, 333, 346, 482, 495,
 503, 516, 541, 556, 599, 611, 625,
 630, 654, 699, 729, 740, 741, 779,
 825, 834, 898, 920, 928, 944, 957,
 968, 985, 995, 1010, 1164, 1170,
 1188, 1199, 1218, 1271, 1275, 1313,
 1332, 1359, 1372, 1384, 1392, 1495,
 1504, 1579, 1587, 1591, 1604, 1628,
 1643, 1698, 1744, 1811, 1830, 1850,
 1861, 1872, 1899, 1909, 1918, 1951,
 1993, 2009, 2045, 2071, 2085, 2095,
 2162, 2171, 2197, 2333, 2370, 2379,
 2396, 2402, 2411, 2426, 2438, 2454,
 2464, 2502, 2521, 2525, 2533, 2547,
 2550, 2554, 2572, 2576, 2610, 2620,
 2633, 2643, 2648, 2653, 2667, 2673,
 2688, 2720, 2731, 2740, 2751, 2758,
 2765, 2771, 2793, 2810, 2818, 2827,
 2836, 2845, 2868, 2883, 2898, 2925,
 2944, 2970, 2976, 2996, 3017, 3032,
 3051, 3102, 3112, 3121, 3131, 3139,
 3152, 3182, 3203, 3228, 3236, 3244,
 3254, 3279, 3294, 3299, 3304, 3310,
 3314, 3341, 3364, 3381, 3401, 3405,
 3415, 3435, 3437, 3442, 3485, 3552,
 3605, 3621, 3649, 3661, 3680, 3749,
 3812, 3843, 3932, 3940, 3953, 3967,
 3977, 3988, 4039, 4086, 4102, 4109,
 4115, 4139, 4156, 4165, 4177, 4184,
 4214, 4229, 4242, 4253, 4255, 4267,
 4308, 4332, 4369, 4382, 4395, 4451,
 4459, 4465, 4475, 4527, 4541, 4554,
 4617, 4636, 4665, 4674, 4692, 4696,
 4701, 4720, 4741, 4760, 4774, 4790,
 4795, 4801, 4855, 4878, 4885, 4889,
 4953, 4978, 5004, 5026, 5044, 5052,
 5062, 5080, 5092, 5117, 5127, 5174,
 5182, 5206, 5220, 5234, 5242, 5251,
 5261, 5289, 5322, 5341, 5352, 5399,
 5407, 5461, 5522, 5530, 5546, 5552,
 5589, 5594, 5605, 5610, 5621, 5624,
 5641, 5644, 5659, 5662, 5677, 5681,
 5691, 5694, 5705, 5711, 5727, 5739,
 5743, 5803, 5814, 5846, 5878, 5931,
 6008, 6167, 6188, 6261, 6269, 6272,
 6292, 6353, 6385, 6434, 6446, 6473,
 6493, 6498, 6530, 6545, 6560, 6567,
 6585, 6622, 6636, 6670, 6679, 6690,
 6707, 6716, 6782, 6792, 6797, 6818,
 6837, 6850, 6885, 6890, 6896, 6937,
 7005, 7072, 7092, 7181, 7196, 7208,
 7214, 7241, 7278, 7321, 7355, 7391,
 7405, 7424, 7434, 7440, 7492, 7507,
 7520, 7589, 7603, 7613, 7653, 7709,
 7747, 7760, 7783, 7801, 7807, 7851,
 7860, 7866, 7872, 7889, 7894, 7899,
 8015, 8048, 8075, 8295, 8417, 8493,
 8505, 8544, 8582, 8619, 8624, 8630,
 8640, 8725, 8897, 9193, 9293, 9356,
 9404, 9454, 9618, 9714, 9925, 9942
 \cs_new_protected:Npn
 16, 148, 303, 327,
 422, 587, 709, 713, 716, 720, 721,
 753, 1267, 1400, 1404, 1450, 1456,
 1491, 1561, 1660, 1676, 2022, 2053,
 2366, 2606, 2742, 2850, 2916, 3042,
 3046, 3214, 3224, 3286, 3326, 3427,
 3669, 4190, 4210, 4604, 4654, 4707,
 4727, 4782, 4787, 4822, 4844, 4849,
 4942, 4961, 4974, 5345, 5377, 5439,
 5757, 5935, 5959, 5963, 5965, 5969,
 5981, 5985, 5987, 5990, 5994, 5998,
 6001, 6005, 6652, 6695, 6701, 6822,
 6829, 6877, 6927, 6943, 6968, 7022,
 7034, 7044, 7224, 7246, 7261, 7313,
 7642, 7666, 7683, 7695, 7813, 7879,
 7884, 8168, 8292, 9199, 9347, 9350
 \cs_set:Nn 9569, 9588, 9591,
 9610, 9614, 9639, 9717, 9718, 9724,
 9725, 9734, 9753, 9756, 9775, 9779
 \cs_set:Npe 4119
 \cs_set:Npn 487,
 524, 530, 614, 615, 900, 913, 919,
 2698, 3344, 3358, 3471, 3476, 3537,
 3542, 4070, 4141, 4143, 4146, 4159,
 4216, 4427, 4477, 4586, 4762, 4981,
 4987, 5032, 5056, 5535, 5571, 5880,
 5937, 6405, 6406, 6410, 6411, 6412,
 6673, 6709, 6729, 6840, 6855, 6949,
 6974, 8173, 8174, 8175, 8399, 8730
 \cs_set_eq:NN 54, 74, 315, 319, 1512,
 1518, 1649, 1901, 2000, 3337, 3338,
 3339, 4193, 4668, 4670, 4723, 5296,

5298, 5416, 5419, 5447, 5448, 5608, 5737, 5848, 5851, 6631, 6643, 6646, 6697, 6703, 6853, 6906, 6919, 8773	definiens 157, <u>7485</u>
<code>\cs_set_protected:Nn</code> 1040, 8418, 8451, 8467, 10491	<code>\defnotation</code> 33, 39, 102, 152, <u>6127</u> , 7447, 7469
<code>\cs_set_protected:Npn</code> 69, 1528, 1538, 3933, 3934, 5053, 5957, 6723, 6728, 6734, 8495, 8498, 8507, 8510, 8774, 8797, 8801, 8812, 8816, 8926, 8929, 8932, 10384	<code>\detokenize</code> 9045, 9048, 9051, 9054, 10000, 10003, 10006, 10009
<code>\cs_set_protected:Npx</code> 5028	dim commands:
<code>\cs_undefine:N</code> 351, 1071, 1834	<code>\dim_compare:nNnTF</code> 9959
<code>\l_tmpa_cs</code> 5880, 5898, 5907, 5937, 5942, 6405, 6415	<code>\dim_eval:n</code> 10387, 10388
<code>\csname</code> ... 349, 663, 2173, 2174, 2175, 2176, 2177, 2182, 2183, 2184, 2587, 3502, 3504, 3568, 3570, 7615, 7901	<code>\dim_new:N</code> 9284
<code>\ctikzinput</code> 121, 10584	<code>\dim_to_decimal:n</code> 9933, 10408, 10436, 10437, 10438, 10445
<code>\currentgrouplevel</code> 325, 328, 334, 335, 337, 339, 341, 347, 349, 351, 353, 355, 7890, 7896, 7901	<code>\dim_zero:N</code> 9287, 9872
<code>\CurrentOption</code> 10, 8346, 8347, 8348, 8349, 8388, 8389, 8966, 9990	<code>\dimexpr</code> 8761
<code>\Currentsectionlevel</code> 84, 139, <u>1528</u>	do commands:
<code>\currentsectionlevel</code> 84, 139, <u>1528</u>	<code>_do_comp:nNn</code> 151
	<code>_do_comp:nnNn</code> 151, 5966, 5988, 5999, <u>6006</u> , 6008, 6204, 6735
	<code>\dobracket</code> 94
	<code>\dobrackets</code> ... 155, 7013, <u>7022</u> , 7045, 8186, 8221, 8228, 8229, 8234, 8235
	<code>\document-URI</code> 125, <u>144</u>
	<code>\doproblemqr</code> 10047, 10064, 10111, 10198, 10312
	<code>\dowithbrackets</code> 155, <u>7044</u>
	<code>\due</code> 76, 119
	<code>\duration</code> 77, 119
	E
<code>\date</code> 10505, 10506	<code>\edef</code> 2175, 2176, 2177, 4797, 8600
<code>\DeclareOption</code> 10	<code>\egroup</code> .. 1904, 2003, 4442, 4601, 7836, 7869, 7984, 8627, 8716, 8722, 8740, 8846, 8849, 9332, 9392, 9440, 9492
<code>\def</code> 93, 669, 673, 676, 678, 1493, 2510, 2536, 3852, 3909, 4354, 4361, 4513, 4520, 4678, 4679, 5035, 5040, 5048, 5057, 5866, 5980, 6010, 6070, 6087, 6095, 6109, 6123, 6149, 6159, 6164, 6180, 6332, 6357, 6377, 7077, 8516, 8518, 8583, 8584, 8585, 8589, 8592, 8649, 8765, 8782, 8790, 8826, 8827, 8829, 8831, 8833, 8835, 8836, 8838, 8840, 8842, 8845, 8846, 8848, 8849, 8852, 8854, 8856, 8872, 8924, 8988, 8989, 8990, 9081, 9084, 9085, 9557, 9562, 10037, 10044, 10047, 10053, 10060, 10064, 10065, 10343, 10411, 10413	<code>\eject</code> 9970, 10313
<code>\defemph</code> 105, 151, <u>5990</u>	<code>\ellipses</code> 98, 4742, 4744, 5854, 5892, 6953, 6954
<code>\Definame</code> 102, 152, <u>6127</u> , 7449, 7471	<code>\else</code> . 662, 674, 1469, 2089, 2111, 2126, 2754, 2761, 5995, 8924, 9005, 9028, 9205, 9316, 9331, 9686, 9820, 9958, 10048, 10054, 10565, 10570, 10575
<code>\definame</code> 23, 33, 39, 101, 102, 105, 152, 306, <u>6127</u> , 7448, 7470	<code>\emph</code> 17, 18, <u>104</u> , 1471, 6005, 7829, 7846, 8008, 9337, 9397, 9445, 9497, 9700, 9834, 9978, 10234
<code>\Definames</code> 6159	<code>\end</code> 128, <u>144</u> , 1474, 1588, 1657, 2088, 2243, 2258, 2278, 2637, 2783, 2801, 2821, 3147, 6992, 7347, 8006, 8461, 8468, 8520, 8529, 8602, 8627, 8642, 8687, 8701, 8709, 8730, 8732, 8745, 8746, 8747, 8748, 8749, 8750, 8771, 8799, 8803, 8806, 8870, 8878, 8919, 8920, 9157, 9247, 9266, 9324, 9329, 9384, 9389, 9432, 9437, 9484, 9489, 9540, 9553, 9574, 9605, 9739, 9853, 9858, 9863,
<code>\definames</code> 6149	
<code>\definiendum</code> 23, 33, 39, 101, 102, 105, 152, <u>6127</u> , 7446, 7468	
<code>\definiens</code> 41, 50, 58, 101–103, 7451, 7485, 7505, 7506	

9868, 9970, 9978, 10188, 10229, 10295, 10313, 10368, 10410, 10587	<code>\envname</code> 144
<code>\endbackmatter</code> 1760, 1762	<code>\eq</code> 31
<code>\endcsname</code> 349,	<code>\eqstep</code> 158, 7731, <u>8115</u>
662, 663, 2173, 2174, 2175, 2176, 2177, 2182, 2183, 2184, 2587, 3502, 3504, 3568, 3570, 7615, 7901, 8924	<code>\equal</code> 30
<code>\endfrontmatter</code> 1747, 1748	<code>\errmessage</code> 6581, 9014, 9037
<code>\endgroup</code> 48, 50, 2103, 2603	<code>\escapechar</code> 88, 842, 10016
<code>\endinput</code> 1913, 2012	<code>\everyeof</code> 2732
<code>\endlist</code> 7862	<code>\examdata</code> 10520
<code>\endspfststepenv</code> 8038, 8042, 8125, 8129	<code>examdata</code> <u>10454</u>
endstex commands:	<code>\examnumber</code> 10044, 10055, 10056, 10058
<code>\endstex_annotate_env</code> 2364, 9770	<code>\excursion</code> 64, 112, <u>8858</u>
<code>\ensuremath</code> 5627, 5633, 5948	<code>\excursiongroup</code> 64, 112, <u>8892</u>
environments:	<code>\excursionref</code> 64, 112, 8863, 8872, 8874, 8887
<code>assignment</code> 76, 119, <u>10317</u>	<code>exnote (env.)</code> <u>1</u> , <u>9404</u>
<code>blindfragment</code> 84, 139, <u>1628</u>	<code>\exnote</code> 9092, 9451, 9599, 9764
<code>copymodule</code> 145, <u>3461</u>	exp commands:
<code>exnote</code> <u>1</u> , <u>9404</u>	<code>\exp_after:wN</code>
<code>extstructure</code> 98, 156, <u>7096</u>	 158, 483, 486, 517, 520, 882,
<code>extstructure*</code> 98	885, 892, 1006, 1030, 1121, 1138,
<code>frame</code> <u>1</u> , <u>1</u> , 8544	1141, 1144, 1147, 1150, 1153, 1159,
<code>gnote</code> <u>1</u> , 9452	1239, 1252, 1255, 1261, 1268, 1293,
<code>hint</code> <u>1</u> , <u>9353</u>	1295, 1408, 1428, 1431, 1434, 1444,
<code>interpretmodule</code> 145, <u>3526</u>	1543, 1964, 2285, 2291, 2297, 2303,
<code>mathstructure</code> 98, 156, <u>7061</u>	2397, 2398, 2459, 2460, 2461, 2467,
<code>mcb</code> <u>1</u> , <u>9567</u>	2471, 2475, 2476, 2536, 2734, 2753,
<code>nassertion</code> <u>1</u> , <u>1</u>	2754, 2755, 2760, 2761, 2762, 3197,
<code>ndefinition</code> <u>1</u> , <u>1</u>	3264, 3265, 3266, 3502, 3504, 3568,
<code>nexample</code> <u>1</u> , <u>1</u>	3570, 3615, 3616, 3617, 3653, 4073,
<code>note</code> <u>1</u> , <u>1</u>	4430, 4589, 5119, 5196, 5197, 5202,
<code>nparagraph</code> <u>1</u> , <u>1</u> , <u>1</u> , <u>1</u>	5360, 5370, 5372, 5415, 5548, 5555,
<code>nsproof</code> <u>1</u> , <u>1</u>	5556, 5557, 5565, 5566, 5567, 5580,
<code>problem</code> <u>1</u>	5762, 5764, 5869, 5870, 5871, 5884,
<code>sassertion</code> 101, 157, <u>7458</u>	6119, 6175, 6199, 6599, 6612, 6957,
<code>scb</code> 9713	6985, 7175, 7178, 7250, 7356, 7370
<code>sdefinition</code> 101, 157, <u>7453</u>	<code>\exp_args:Ne</code> 58,
<code>sexample</code> 101, 157, <u>7473</u>	63, 69, 77, 81, 157, 265, 266, 274,
<code>sfragment</code> 84, 139, <u>1547</u>	806, 814, 818, 903, 962, 1451, 1814,
<code>smodule</code> 88, 142, <u>2309</u>	1835, 1837, 1853, 1854, 1969, 2050,
<code>solution</code> <u>1</u> , 9283	2075, 2334, 2743, 2861, 3188, 3196,
<code>sparagraph</code> 101, 157, <u>7476</u>	3197, 3247, 3683, 3895, 3964, 4041,
<code>spfblock</code> 158, <u>8102</u>	4130, 4195, 4197, 4274, 4397, 4555,
<code>spfsketchenv</code> 158, <u>7989</u>	4796, 4823, 4827, 4906, 4917, 4988,
<code>spfststepenv</code> 158, <u>8099</u>	5027, 5045, 5186, 5464, 5682, 5783,
<code>sproblem</code> <u>9086</u>	5785, 5793, 5799, 6063, 6080, 6203,
<code>sproof</code> 103, 158, <u>7707</u>	6221, 6231, 6572, 6938, 7168, 7228,
<code>stex_annotate_env</code> 128, <u>709</u>	7316, 7362, 7443, 7462, 7514, 7522,
<code>stex_env_node</code> 128, <u>709</u>	7524, 8079, 8297, 8868, 9007, 9030,
<code>subproblem</code> <u>9200</u>	9527, 9744, 9931, 10041, 10275, 10405
<code>subproof</code> 158, <u>7864</u>	<code>\exp_args:NNe</code> 89, 474, 477,
<code>testheading</code> 77, 119	879, 889, 896, 1903, 2002, 2433,
<code>titlefragment</code> 139, <u>1606</u>	2537, 3114, 3626, 4236, 4281, 4293,
	5008, 5132, 5149, 5244, 5263, 5496,
	5499, 5769, 5820, 5910, 6083, 6156,
	6419, 6515, 6526, 7190, 8914, 9194

<code>\exp_args:Nne</code>	304,	<code>\exp_args:Nx</code>	8545, 9619
1065, 1342, 1484, 1596, 1815, 2144,		<code>\exp_not:N</code>	94, 257, 308,
2165, 2622, 2901, 3033, 3062, 3289,		310, 913, 1080, 1081, 1086, 1087,	
3853, 3886, 3954, 4129, 4362, 4761,		1118, 1127, 1128, 1133, 1245, 1246,	
5047, 5064, 5119, 5190, 5192, 5245,		1414, 1415, 1820, 1905, 2004, 2481,	
5533, 5569, 5579, 6358, 6378, 6546,		2732, 2752, 2760, 3063, 3690, 4619,	
6711, 6741, 7083, 7183, 7186, 7189,		4620, 4623, 4627, 4631, 4688, 4764,	
7327, 7413, 7522, 7524, 7558, 7594,		4767, 4861, 4937, 5011, 5017, 5037,	
7599, 7606, 7609, 7748, 7764, 7996,		5070, 5102, 5105, 5109, 5134, 5137,	
8020, 8054, 9122, 9222, 9580, 10404		5143, 5152, 5157, 5166, 5201, 5424,	
<code>\exp_args:NNne</code>	4283, 6819	5430, 5543, 5821, 5826, 5905, 5911,	
<code>\exp_args:NNne</code> 4370, 4528, 5475, 5481		5915, 5917, 5918, 5940, 6387, 6794,	
<code>\exp_args:NNno</code>		8423, 8428, 9695, 9697, 9829, 9831	
. 1017, 2884, 3636, 4370, 4528		<code>\exp_not:n</code>	257, 308, 914,
<code>\exp_args:NNno</code>	143	1596, 1823, 1824, 1936, 1942, 1981,	
<code>\exp_args:NNno</code>	3207, 3209, 5949	1983, 2004, 2475, 2486, 2541, 3914,	
<code>\exp_args:NNNx</code>	1017	3961, 4122, 4376, 4377, 4378, 4388,	
<code>\exp_args:NNo</code>		4389, 4390, 4534, 4535, 4536, 4547,	
13, 42, 63, 83, 104, 107, 127, 131,		4548, 4549, 4621, 4624, 4628, 4630,	
156, 222, 348, 750, 800, 1020, 1342,		4632, 4772, 4862, 4900, 4931, 4936,	
1876, 1968, 2213, 2224, 2429, 2901,		4938, 5010, 5012, 5014, 5018, 5021,	
2933, 3633, 3639, 3643, 5272, 5273,		5031, 5103, 5106, 5110, 5135, 5138,	
5283, 5284, 5897, 6466, 6479, 6867		5140, 5144, 5146, 5151, 5155, 5164,	
<code>\exp_args:Nno</code>		5201, 5556, 5572, 5577, 5582, 5584,	
. . . 335, 337, 372, 1911, 1946, 2511,		5806, 5827, 5864, 5869, 5907, 5919,	
3191, 4068, 4236, 4281, 4293, 4425,		5920, 5942, 6388, 6428, 6429, 6788,	
4574, 4584, 4606, 4784, 4788, 4997,		6957, 6985, 7896, 9700, 9834, 9902	
7873, 9109, 9279, 10038, 10332, 10377		<code>\expandafter</code>	48,
<code>\exp_args:NNx</code>		663, 2173, 2174, 2175, 2176, 2177,	
. . . . 90, 93, 426, 439, 9045, 9048,		2587, 5995, 7868, 7901, 8520, 8521	
9051, 9054, 10000, 10003, 10006, 10009		<code>\ExplSyntaxOff</code> 2611, 2659, 8304, 9058, 10013	
<code>\exp_args:No</code>		<code>\ExplSyntaxOn</code>	
. . . 48, 257, 308, 348, 570, 576, 657,		. . . 143, 2607, 2658, 8301, 9041, 9996	
668, 785, 914, 1029, 1521, 1596,		<code>\extref</code>	87, 2043
1615, 1875, 1883, 1884, 1910, 1921,		<code>extstructure</code> (env.)	98, 156, 7096
1924, 1934, 1936, 1942, 1943, 1944,		<code>\extstructure</code>	7125, 7127
1953, 1967, 1974, 1983, 2004, 2010,		<code>extstructure*</code> (env.)	98
2038, 2039, 2274, 2350, 2383, 2443,			
2541, 2573, 2859, 2863, 3109, 3113,			
3194, 3274, 3407, 3428, 3443, 3451,			
3650, 3656, 3657, 3700, 3882, 3945,			
3961, 4121, 4126, 4234, 4283, 4376,			
4377, 4378, 4388, 4389, 4390, 4534,			
4535, 4536, 4547, 4548, 4549, 4621,			
4624, 4628, 4655, 4748, 4862, 4866,			
4873, 4900, 4936, 4938, 5010, 5012,			
5018, 5103, 5135, 5138, 5144, 5146,			
5151, 5155, 5164, 5187, 5201, 5262,			
5473, 5501, 5572, 5577, 5582, 5584,			
5783, 5785, 5827, 5907, 5920, 5942,			
6246, 6388, 6788, 6975, 7080, 7088,			
7144, 7147, 7169, 7229, 7249, 7420,			
7502, 8412, 8476, 8477, 8480, 9044,			
9130, 9230, 9233, 9999, 10042, 10506			

F

<code>\fbox</code>	9930, 9946, 9959
<code>\fi</code>	665, 680, 1475, 2091,
	2115, 2126, 2755, 2762, 4666, 4697,
	5294, 5995, 6168, 6189, 8522, 8590,
	8593, 8711, 8924, 8992, 9009, 9032,
	9207, 9320, 9333, 9688, 9703, 9708,
	9822, 9837, 9842, 9964, 9978, 10038,
	10051, 10061, 10569, 10579, 10580
<code>fiboxed</code>	60, 108
file commands:	
<code>\file_if_exist:nTF</code>	612, 631, 997
<code>\fillinsol</code>	117, 9090, 9870
<code>\fn</code>	47
<code>\foldexpr</code>	152, 6218
<code>\foo</code>	16, 47, 90

<code>\foiname</code>	16	<code>\hbox</code> 1506, 2947, 3020, 3596, 3611, 3906,	
<code>\footnote</code>	9707, 9841, 9948	3936, 4041, 4397, 4570, 6717, 7109,	
<code>\footnotesize</code>	10406	7287, 7720, 7787, 7913, 8159, 8314,	
<code>\foral</code>	40	8626, 8846, 8849, 9644, 9675, 9917	
<code>\forall</code>	40, 8242	hbox commands:	
fp commands:		<code>\hbox_set:Nn</code>	6402, 9930
<code>\fp_gadd:Nn</code>	9153, 9154	<code>\hbox_unpack:N</code>	
<code>\fp_new:N</code>	9096, 9097		1530, 1540, 3935, 9560, 9565
<code>\fp_to_decimal:n</code>	9240, 9243	<code>\HCode</code>	675
<code>frame (env.)</code>	1, 1, 8544	<code>\hfil</code>	8165, 9949
<code>\frameimage</code>	62, 111, 8754	<code>\hfill</code>	8165
<code>frameimages</code>	60, 108	<code>hint (env.)</code>	1, 9353
<code>\frametitle</code>	8658	<code>\hint</code>	9091, 9403, 9598, 9763
<code>\frontmatter</code>	210, 1745, 1746, 1749	<code>hints</code>	76, 113, 118
<code>\FTMLrule</code>	8307, 8331, 8332	<code>\hline</code> ..	10404, 10406, 10407, 10408, 10409
<code>\fun</code>	39	<code>\homeworkdata</code>	10526
<code>\funspace</code>	34	<code>homeworkdata</code>	10454
G			
<code>\gdef</code>	1497, 1514, 1519, 8858,	<code>\href</code>	1974, 2057, 8868
	10031, 10033, 10041, 10042, 10058	<code>\hrule</code>	8159, 9644, 9675
<code>\given</code>	76, 119	<code>\hskip</code>	9930
<code>\global</code>	1493, 2174, 8286,	<code>\hspace</code>	9682, 9816, 9962
	8287, 9348, 9351, 10060, 10339, 10341	<code>\HTML</code>	16, 25
<code>gnote (env.)</code>	1, 9452	<code>\huge</code>	9930, 9960, 9962
<code>\gnote</code>	9093, 9503, 9600, 9765	hwexam commands:	
<code>gnotes</code>	76, 113, 118	<code>\l_hwexam_includeassignment_</code>	
group commands:		<code>keys_tl</code> ..	10330, 10331, 10333, 10375
<code>\group_begin:</code>		<code>\hwexam_kw_*</code>	9995
.....	18, 78, 87, 613, 755, 841,	<code>\c_hwexam_multiple_bool</code>	9986
	2513, 2629, 2693, 2702, 2710, 2951,	<code>\c_hwexam_qrcode_bool</code> ..	9988, 10020
	2987, 3024, 3060, 3095, 3813, 3844,	<code>\hwexamheader</code>	10411, 10450
	3918, 4357, 4516, 4638, 4749, 4753,	<code>\hwexamminutes</code>	10413
	5029, 5054, 5183, 5291, 5306, 5524,	<code>\hyperlink</code>	2055
	5567, 5695, 5740, 5815, 5863, 5879,		
	5971, 6041, 6058, 6075, 6089, 6101,	I	
	6115, 6190, 6263, 6286, 6341, 6373,	<code>\if</code>	2753, 2760
	7023, 7357, 7486, 7541, 7804, 7853,	if commands:	
	8053, 8078, 8172, 9276, 10015, 10374	<code>\if_charcode:w</code>	10038
<code>\group_end:</code>	82, 85,	<code>\IfBooleanTF</code>	
	93, 621, 822, 845, 2527, 2638, 2706,		3509, 3575, 3798, 5452, 6335, 6370,
	2713, 2716, 2956, 2993, 3029, 3062,		7919, 8032, 8120, 8142, 8150, 8755
	3148, 3821, 3858, 3923, 4367, 4524,	<code>\ifcsname</code>	662, 8924
	4699, 4712, 4738, 4750, 4754, 4784,	<code>\ifdefempty</code>	8913
	4945, 5001, 5038, 5059, 5065, 5149,	<code>\IfFileExists</code>	6, 21, 1884, 1953,
	5202, 5320, 5357, 5368, 5384, 5395,		2072, 2212, 2223, 2904, 2909, 2918
	5403, 5436, 5527, 5548, 5580, 5585,	<code>\IfInputref</code>	27, 85, 140, 2119, 8911
	5703, 5740, 5820, 5871, 5910, 5975,	<code>\ifinputref</code> ..	27, 85, 140, 2109, 2118, 2126
	6186, 6208, 6265, 6286, 6344, 6383,	<code>\ifintest</code>	8977, 8986, 9978
	7029, 7366, 7488, 7552, 7810, 7862,	<code>\ifmmode</code>	5995
	8072, 8096, 8289, 9280, 10018, 10378	<code>\ifnotes</code>	61, 109, 8403, 8540
<code>\group_insert_after:N</code>	340, 354	<code>\ifnum</code>	2086
H			
<code>\halign</code>	9949	<code>\ifSGvar</code>	63, 112, 8932
		<code>\ifsolutions</code> 114, 8969, 9313, 9326, 9679,	
		9692, 9706, 9813, 9826, 9840, 9944	

<code>\ifstexhtml</code>	27, 83, 128, <u>685</u> ,	<code>\int_incr:N</code>	
	1469, 9005, 9028, 9205, 10048, 10054		... 1564, 1568, 1600, 1631, 1634,
<code>\ifvmode</code>	4666, 4697, 5294, 6168, 6189		1638, 1663, 1667, 1703, 1709, 4030,
<code>\ifx</code>	2173,		4031, 4032, 4033, 5758, 6897, 6980,
	8519, 8588, 8591, 10562, 10563, 10571		7650, 7662, 7673, 7690, 7702, 8320,
<code>\ignorespaces</code>	2942, 2964,		8462, 8813, 8817, 9517, 10252, 10392
	3013, 4367, 4524, 8714, 8720, 9264	<code>\int_new:N</code>	323, 1526,
<code>image</code>	120		2015, 3986, 4182, 5405, 5742, 6967,
<code>\importmodule</code>	35, 43, 48, 96,		6997, 8809, 9200, 9283, 9452, 10383
	97, 124, 128, 143, 145, 223, <u>2961</u> , 3331	<code>\int_set:Nn</code>	1678, 1679, 1680, 1681,
<code>\includeassignment</code>	76, 10373		1682, 1683, 1685, 3418, 3901, 3995,
<code>\includegraphics</code>	85, 141, 2241, 10555		3999, 4003, 4007, 4011, 4015, 4019,
<code>\includeproblem</code>	75, 117, <u>9268</u>		4023, 4027, 4218, 4258, 4319, 4347,
<code>\indent</code>	4666, 4697, 5294, 6168, 6189		4479, 5700, 6870, 6975, 6982, 7004,
<code>\infpref</code>	92, 93, 154, 6466, <u>7001</u> , 8194		7643, 7655, 7667, 7684, 7696, 10477
<code>\inline</code>	157	<code>\int_set_eq:NN</code>	325
<code>\inline*</code>	102	<code>\int_step_function:nN</code>	4765, 5536
<code>\inlineass</code>	44, 48, 51, 102, 157	<code>\int_step_inline:nn</code>	
<code>\inlinedef</code>	44, 102, 157		... 3979, 6458, 6465, 6485, 6805
<code>\inlineex</code>	102, 157	<code>\int_use:N</code>	1582, 1645, 1689, 1692,
<code>\input</code> 26, 82, 125, 143, 182, 44, 246, 684,			1713, 3957, 4374, 4386, 4532, 4545,
	2136, 9042, 9046, 9049, 9052, 9055,		4608, 5428, 5464, 5805, 5823, 5913,
	9997, 10001, 10004, 10007, 10010,		6534, 6639, 6870, 7001, 7002, 7415,
	10411, 10564, 10567, 10573, 10577		7890, 7896, 7901, 8323, 8455, 8814,
<code>\inputassignment</code>	119		8818, 9253, 9298, 9361, 9409, 9459,
<code>\inputref</code>	26,		9519, 10156, 10157, 10179, 10196,
	27, 84, 85, 104, 124, 140, <u>2061</u> ,		10197, 10220, 10254, 10404, 10511
	8773, 8774, 8882, 8914, 9279, 10377	<code>\int_zero:N</code>	3989,
<code>\inputref*</code>	61, 110, <u>8773</u>		3990, 5749, 6852, 6978, 8313, 8811
<code>\inputreffalse</code>	2114, 2118	<code>\c_max_int</code>	7001, 7002
<code>\inputreftrue</code>	2097, 2112	<code>\l_tmpa_int</code>	
<code>\insertexnumber</code>	10053, 10060, 10065		6852, 6855, 6870, 6897, 6978, 6980,
<code>\insertframenumber</code>	8852, 8854		6981, 6982, 6986, 7643, 7646, 7649,
<code>\insertshortauthor</code>	8845		7650, 7655, 7658, 7661, 7662, 7667,
<code>\insertshortdate</code>	8856		7670, 7673, 7675, 7676, 7678, 7679,
<code>\insertshorttitle</code>	8848		7684, 7687, 7690, 7692, 7696, 7699,
<code>\inset</code>	39		7702, 7704, 7705, 8313, 8320, 8323
int commands:		intarray commands:	
<code>\int_case:nn</code>	1629	<code>\intarray_gset:Nnn</code>	
<code>\int_case:nnTF</code>	1562, 1661, 1699, 1718		... 7678, 7692, 7705, 7711
<code>\int_compare:nNnTF</code>		<code>\intarray_gzero:N</code>	7710
	328, 1599, 1637, 2142,	<code>\intarray_item:Nn</code>	7646, 7649,
	2164, 3627, 4858, 4869, 4962, 5386,		7658, 7661, 7670, 7679, 7687, 7699
	5782, 5792, 5798, 5857, 6393, 6427,	<code>\intarray_new:Nn</code>	7640
	6981, 7011, 7388, 7426, 7675, 10391	<code>\interpretmod</code>	145, 3588, 3590, 3592
<code>\int_compare_p:nNn</code>		<code>interpretmodule (env.)</code>	145, <u>3526</u>
	7645, 7657, 7669, 7686, 7698	<code>\interpretmodule</code>	3562, 3564
<code>\int_decr:N</code>	7676, 7704	<code>\intestfalse</code>	8981
<code>\int_eval:n</code>	347, 349, 351,	<code>\intesttrue</code>	8979
	5478, 5484, 7010, 7216, 10339, 10405	ior commands:	
<code>\int_gincr:N</code>	5463,	<code>\ior_close:N</code>	638, 644, 846, 1028
	9238, 9296, 9359, 9407, 9457, 10385	<code>\ior_map_inline:Nn</code>	1016
<code>\int_gset:Nn</code>	5428	<code>\ior_new:N</code>	1002
<code>\int_gzero:N</code>	5413, 9147	<code>\ior_open:Nn</code>	632, 640, 1011

\ior_open:NnTF	840	\libinpu	19, 82, 140, 2129, 2199
\ior_str_get:NN	843	\libusepackage ...	82, 83, 140, <u>2140</u> , 8536
\ior_str_map_inline:Nn	634, 641	\libusetheme	61, 8535
\g_tmpa_ior	632,	\libusetikzlibrary	82, 121, 140, <u>2151</u>
634, 638, 640, 641, 644, 840, 843, 846		\linewidth ...	9336, 9343, 9396, 9401,
iow commands:		9444, 9449, 9496, 9501, 10233, 10246	
\iow_close:N ...	627, 637, 1798, 10305	\list	7853
\iow_new:N	585, 1796, 10302	\LoadClass	16, 8368, 8370
\iow_now:Nn	635,	\loadsms	127, <u>586</u> , 587
642, 730, 1816, 1819, 8300, 9194, 10304		\long	8827, 8836
\iow_open:Nn ...	626, 633, 1797, 10303	\lstinputlisting	85, <u>141</u> , 2257
\g_tmpa_iow	633, 635, 637	\lstinputmhlisting	85, <u>141</u> , <u>2245</u>
\isassociative	<u>43</u>		
\iscommutative	<u>43</u>	M	
\item	7857, 9253, 9527,	\macro	123, 124, 126, 135, <u>144</u>
9546, 9574, 9739, 10156, 10196, 10273		\macroname	31, 127
\itshape	8169	\magma	<u>46</u>
J		\maincomp	31–34, 47, 92, 100,
\jobname	79, 235, 604, 626, 631,	105, 4449, 4679, 4698, 4796, 4797,	
632, 633, 640, 645, 1797, 1799, 10303		5027, 5035, 5040, 5045, 5048, 5057,	
\join	<u>56</u>	5402, 5980, 6387, 6396, 6406, 6407,	
K		6428, 6734, 6794, 7234, 8186, 8228,	
keys commands:		8234, 8253, 8257, 8260, 8262, 8277	
\l_keys_choice_tl	3772	\mainmatter	1773, 1774, 1784, 1785
\keys_define:n	29, 372,	\makeatletter	2656, 9041, 9996, 10450
8340, 8382, 8892, 8948, 9984, 10543		\makeatother	2657, 9058, 10013, 10450
\l_keys_key_str 6309, 6312, 7054, 7057		\makebox	9681, 9815
\l_keys_key_tl ... 13, 750, 6310, 7055		\maketitle	
\keys_set:n	126, 376, 8901	139, 1518, 1519, 1610, 1619, 8468, 8469	
keyval commands:		\mapsto	8229, 8235
\keyval_parse:NNn	4881	\marginnote	8586, 10049
L		\marginpar	152, 6262,
\label	86, 1814, 1835, 1837, 8651	9006, 9029, 9163, 9168, 9256, 9261,	
\labelsep	8596	9559, 9564, 10072, 10077, 10114,	
\labelwidth	8597	10119, 10160, 10165, 10201, 10206	
\langle	8213	\mathbb	8265
\Large	8400,	\mathbin	
8783, 9610, 9611, 9614, 9615, 9775,		32, 8192, 8196, 8205, 8241, 8252, 8256	
9776, 9779, 9780, 10056, 10058, 10355		\mathcal	5905, 5907, 5940, 5942
\lastslide	8801, 8816	\mathclose	32, 7030, 8194, 8203,
\LaTeX	<u>23</u>	8207, 8213, 8215, 8225, 8241, 8245	
\latex	<u>23</u>	\mathhub	80, <u>132</u> , 32, <u>837</u>
\ldots	8218, 8281, 8283, 8285	\mathop	32, 8243, 8262
\leaders	8785	\mathopen	32, 7025, 8194, 8203,
\leavevmode	8671	8207, 8213, 8215, 8224, 8241, 8245	
\leftmargin	8598	\mathord	32
\let	1746, 1747, 1748, 1749,	\mathpunct 32, 5732, 6809, 8224, 8228, 8234	
1759, 1760, 1761, 1762, 2098, 2174,		\mathrel	31,
2695, 2703, 2711, 2733, 2738, 3007,		32, 8197, 8229, 8235, 8253, 8257, 8260	
3936, 4677, 4698, 6046, 6069, 6086,		\mathrm	6794,
6181, 7240, 8286, 8287, 8302, 8303,		8186, 8199, 8224, 8228, 8234, 8277	
8469, 8792, 9716, 9720, 9723, 9727		mathstructure (env.)	98, <u>156</u> , <u>7061</u>

mcb (env.)	1, 9567	\neginfpref	37, 92, 93, 154, 4710, 5380, 5391,
\mcb	9088, 9595, 9617, 9760	\newcommand	483,
\mcc	70, 115, 9601, <u>9638</u>		517, 520, 2031, 2120, 2125, 2129,
\meaning	1962,		2140, 2151, 2186, 2252, 2258, 2268,
	2587, 3154, 3157, 3160, 4709, 5200,		2278, 8137, 8517, 8586, 8667, 8776,
	5563, 5575, 5576, 6432, 6737, 6892		8780, 8863, 8874, 8881, 8884, 8906,
\medskip	2103, 8626, 8641, 8665		8908, 8994, 9017, 9079, 9514, 9648,
\meet	56		9690, 9786, 9824, 9850, 9855, 9860,
\message	28, 1865,		9865, 9914, 9969, 9972, 9973, 9974,
	2759, 3099, 8361, 8364, 8526, 10300		9975, 9976, 9977, 10390, 10520,
\mhframeimage	62, 111		10526, 10530, 10555, 10560, 10584
\mhgraphics	85, 141, <u>2229</u> , 8767, 8769	\newcounter	8371, 8372,
\mhinput	85, 140, <u>2106</u>		8373, 8374, 8501, 8513, 9078, 10328
\mhtikzinput	121, 141, <u>2260</u>	\NewDocumentCommand	
\min	77, 119		127, 486, 1468, 1829, 1840,
\min	9562		2043, 2048, 2061, 2106, 2595, 4445,
min	76, 113, 118		4448, 5305, 5451, 6040, 6057, 6074,
\mmlarg	129, 712		6088, 6100, 6114, 6137, 6143, 6153,
\mmlintent	129, 712		6163, 6219, 6230, 7356, 7485, 7531,
\mname	89, 98		7540, 7555, 8016, 8115, 8133, 8141,
mode commands:			8149, 8307, 8535, 8754, 9275, 10373
\mode_if_math:TF		\NewDocumentEnvironment	127
	3611, 3936, 3937, 4694, 4699, 9916	\NewDocumentEnvironment	
\mode_if_vertical:TF	1530, 1540, 3935		523, 1553, 1608, 1652, 8102, 8795, 8810
\mode_leave_vertical:	3268	\newenvironment	1770, 1781, 7989,
\MSC	8292		8099, 8720, 8722, 8727, 8729, 10433
msg commands:		\newif	663, 687, 2118, 8403, 8969, 8977, 10440
\msg_error:nn		\newlabel	8302, 8303
	3067, 3260, 5707, 7498, 7547, 9105	\newlength	8538, 8539, 8542
\msg_error:nnn		\newline	9336, 9343, 9396, 9401,
	58, 469, 795, 2153, 2199,		9444, 9449, 9496, 9501, 10233, 10246
	2583, 2830, 2935, 3713, 3743, 4186,	\newpage	8648, 9976, 10452
	4455, 5170, 5248, 6940, 7243, 9892	\newsavebox	8689
\msg_error:nnnn	70, 2134, 2191,	\newseq	148, <u>4448</u>
	3317, 3393, 4035, 4646, 5302, 5318,	\newvar	148, <u>4445</u>
	5489, 5516, 6184, 6206, 6665, 7210	nexample (env.)	1, 1
\msg_fatal:nn	947, 10493, 10500, 10503	\nextslide	8797, 8812
\msg_fatal:nnnn	951, 2148, 2167	\ninputref	8774, 8776, 8780
\msg_none:nn	117	\nobreak	8165
\msg_redirect_module:nnn	132	\noexpand	2753
\msg_redirect_name:nnn	136	\noindent	1583, 2074, 2087, 7334,
\msg_set:nnn			7738, 7945, 8008, 8077, 8641, 8661,
	114, 9883, 10460, 10464, 10468		8669, 8687, 9128, 9159, 9228, 9337,
\msg_warning:nn	859		9397, 9445, 9497, 10069, 10110, 10234
\msg_warning:nnnn	651	\nointerlineskip	8787
\mult	37, 93, 94	\normalmarginpar	10050
\multicolumn	10405	\notation	31, 32, 37, 90, 92, 97,
\multiple	76, 118		143, 153, 3330, <u>6325</u> , 8189, 8192,
			8195, 8196, 8197, 8212, 8214, 8240,
			8242, 8243, 8251, 8255, 8265, 8268
nassertion (env.)	1, 1	note (env.)	1, 1
\Nat	34	notes	60, 76, 108, 113, 118
ndefinition (env.)	1, 1		

\notesfalse	8415	__notesslides_inputref:	
notesslides commands:			8773, 8774, 8777
\c_notesslides_notes_bool		__notesslides_notes_env:nnnn	
8342, 8343, 8356, 8359, 8367, 8383,			8725,
8384, 8396, 8404, 8532, 8713, 8719,		8745, 8746, 8747, 8748, 8749, 8750	
8726, 8760, 8775, 8825, 8875, 8885		\l__notesslides_num	
\c_notesslides_sectocframes_bool			8421, 8423, 8426,
.....	8385, 8488	8428, 8431, 8432, 8435, 8436, 8439,	
\c_notesslides_topsect_str	8386,	8440, 8443, 8444, 8447, 8448, 8455	
8409, 8412, 8473, 8476, 8477, 8480		__notesslides_setup_itemize: ...	
notesslides internal commands:			8582, 8653
\c_notesslides_class_str		\l__notesslides_slideshow-	
.....	8338, 8341, 8347, 8368	counter_int	
__notesslides_define_chapter: ..		..	8809, 8811, 8813, 8814, 8817, 8818
.....	8478, 8481, 8493	\l__notesslides_tmp	8933, 8938
__notesslides_define_part:		\l__notesslides_uri	8867, 8868
.....	8482, 8505	\g__notesslides_variables_prop ..	
__notesslides_do_label:n	8418, 8454		8925, 8927, 8930, 8933
__notesslides_do_sectocframes: .		\notesslidesfont	8622, 8638, 8792
.....	8417, 8489	\notesslidesfooter	8626, 8641, 8790
__notesslides_do_yes_param:Nn ..		\notesslidestitleemph ..	8661, 8663, 8782
.....	8544,	\notesttrue	8405
8563, 8566, 8569, 8572, 8575, 8578		nparagraph (env.)	<u>1</u> , <u>1</u> , <u>1</u> , <u>1</u>
\c_notesslides_dicopt_str ...	8344	nsproof (env.)	<u>1</u> , <u>1</u>
\c_notesslides_document_str ...		\null	8165
.....	8516, 8519	\number	76, 119
__notesslides_eat: .	8730, 8734, 8737	\numberline	8453
__notesslides_excursion_args:n .		\numberproblemsin	<u>9078</u>
.....	8897, 8909		
\l__notesslides_excursion_id_str			
.....	8893, 8899		
\l__notesslides_excursion_intro-			
tl	8894, 8898, 8913, 8915		
\l__notesslides_excursion-			
mhrepos_str	8895, 8900, 8914		
\l__notesslides_frame_allowdisplaybreaks-			
bool	8555, 8566		
\l__notesslides_frame_allowframebreaks-			
bool	8554, 8563		
__notesslides_frame_box_begin: .			
.....	8619, 8630, 8654		
__notesslides_frame_box_end: ...			
.....	8624, 8640, 8656		
\l__notesslides_frame_fragile-			
bool	8556, 8569		
\l__notesslides_frame_label_str .			
.....	8553, 8561, 8650, 8651		
\l__notesslides_frame_shrink-			
bool	8557, 8572		
\l__notesslides_frame_squeeze-			
bool	8558, 8575		
\l__notesslides_frame_t_bool ...			
.....	8559, 8578		

O

\objective	9017
\OMDoc	25
\omdoc	16
\only	8814, 8818
\oplus	8204, 8205

P

\PackageError	8934
\pagebreak	9176, 10371
\pagenumbering ...	1754, 1767, 1778, 1789
\par ..	47, 1581, 1585, 2087, 7709, 7738,
	7803, 7809, 7852, 7861, 7941, 7945,
	7990, 7995, 8004, 8070, 8165, 8626,
	8641, 8661, 8669, 8674, 8682, 8687,
	8694, 8704, 9159, 9172, 9175, 9206,
	9336, 9343, 9396, 9401, 9444, 9449,
	9496, 9501, 9547, 9550, 9607, 9728,
	9772, 10069, 10098, 10106, 10110,
	10140, 10148, 10233, 10246, 10289,
	10292, 10354, 10359, 10369, 10371
\paragraph	26, 84
\parent	125
\parsep	7855, 9251, 9525,
	9544, 9572, 9737, 10154, 10194, 10271

<code>\part</code>	26, 84, 8509, 8510	<code>\problem_scc_box_tl</code>	
<code>\partname</code>	8423, 8506, 8507	..	9739, 9751, 9775, 9779, 9785, 9812
<code>\parttitlename</code>	8423	<code>\g_problem_total_min_fp</code> ..	9097, 9154
<code>\PassOptionsToClass</code>	8346, 8347	<code>\g_problem_total_pts_fp</code> ..	9096, 9153
<code>\PassOptionsToPackage</code>		<code>\problemheader</code> ..	9159, 9179, 10069, 10110
.....	10, 8348, 8349, 8360,	problems internal commands:	
8363, 8388, 8389, 8406, 8966, 9990		<code>__problems_activate_macros:</code> ..	
<code>\pause</code>	8667		9086, 9151, 9208
<code>\PDF</code>	16, 25	<code>\l__problems_anscls_int</code>	
<code>\pdfbookmark</code>	8455		9452, 9517, 9519, 10252, 10254
peek commands:		<code>\c__problems_boxed_bool</code>	8962
<code>\peek_charcode:NTF</code>		<code>__problems_do_yes_param:Nn</code> ..	9618
1393, 3662, 4703, 4716, 4776, 4778,		<code>__problems_exnote_start:n</code>	
4805, 4811, 4954, 4965, 5328, 5335			9404, 9421, 9424
<code>\peek_charcode_remove:NTF</code>		<code>\c__problems_false_tl</code> ..	9881, 9891
... 4702, 4775, 4810, 5063, 5324,		<code>\l__problems_fillin_solution_tl</code> .	
5326, 5333, 5343, 5346, 5547, 5590		..	9871, 9895, 9907, 9936, 9947, 9954
<code>\phantom</code>	9960	<code>__problems_fillinsol:n</code>	
<code>\plus</code>	35–38, 92–94		9917, 9919, 9925, 9942
<code>\precondition</code>	8994	<code>__problems_gnote_start:n</code>	
<code>\prematurestop</code>	63, 111, 8516		9454, 9473, 9476
<code>\premise</code>	103, 157, 7467, 7555	<code>\c__problems_gnotes_bool</code>	
prg commands:			8952, 9475, 9486, 10030, 10301
<code>\prg_new_conditional:Nnn</code>		<code>\l__problems_has_min_bool</code> .	9101,
. 57, 62, 263, 272, 695, 2560, 2564,		9145, 9146, 9242, 9260, 10164, 10205	
2568, 2682, 4125, 4128, 4303, 5495,		<code>\l__problems_has_pts_bool</code>	
5781, 5791, 5797, 6240, 6251, 6255			9100, 9142, 9143,
<code>\prg_new_protected_conditional:Nnn</code>		9239, 9255, 10159, 10181, 10200, 10222	
.....	279	<code>__problems_hint_start:n</code>	
<code>\prg_return_false:</code>	60,		9356, 9373, 9376
65, 264, 267, 273, 275, 295, 299,		<code>\c__problems_hints_bool</code>	
697, 2562, 2566, 2570, 2683, 4126,			8954, 9375, 9386
4137, 5513, 5517, 5787, 5788, 5789,		<code>\l__problems_in_problem_bool</code> ..	
5794, 5795, 5800, 5801, 6241, 6256		..	9099, 9102, 9104, 9138, 9204, 9209
<code>\prg_return_true:</code>		<code>__problems_maybe_inline:n</code>	
..... 60, 65, 265, 267, 275, 299,			9579, 9605, 9714, 9743, 9770
697, 2562, 2566, 2570, 2683, 4126,		<code>__problems_maybe_newline:</code>	
4137, 5511, 5787, 5794, 5800, 6252			9639, 9718, 9725
<code>\printexcursion</code>	64, 112	<code>__problems_mccline:n</code> ..	9569,
<code>\printexcursions</code> .	8858, 8882, 8910, 8918	9588, 9591, 9610, 9614, 9650, 9705	
<code>problem (env.)</code>	1	<code>\c__problems_min_bool</code> 8960, 9166,	
problem commands:		9259, 9563, 10075, 10117, 10163, 10204	
<code>\g_problem_id_counter</code>		<code>\l__problems_min_tl</code>	
..... 9283, 9296, 9298,		9140, 9144, 9154, 9167, 9168, 9195,	
9359, 9361, 9407, 9409, 9457, 9459		9211, 9243, 10076, 10077, 10118, 10119	
<code>\l_problem_inputproblem_keys_tl</code> .		<code>\c__problems_notes_bool</code>	
..... 9107, 9108, 9110, 9277			8950, 9423, 9434
<code>\problem_mcc_box_default_tl</code>		<code>__problems_oldpar:</code>	
..... 9673, 9681, 9685, 9687			9716, 9720, 9723, 9727
<code>\problem_mcc_box_tl</code>		<code>__problems_parse_fillin_-</code>	
.. 9574, 9578, 9610, 9614, 9642, 9678		arg:nnnn	
<code>\problem_scc_box_default_tl</code>		.. 9875, 9876, 9877, 9888, 9906, 9928	
..... 9811, 9815, 9819, 9821		<code>\l__problems_path_seq</code>	
		..	9119, 9120, 9214, 9215, 9219, 9220

```

\l_problems_prob_imports_tl . 9075
\l_problems_prob_refnum_int . 9076
\c_problems_pts_bool 8958, 9161,
    9254, 9558, 10070, 10112, 10158, 10199
\l_problems_pts_tl . . . . .
    9139, 9141, 9153, 9162, 9163, 9195,
    9210, 9240, 10071, 10072, 10093,
    10095, 10113, 10114, 10135, 10137
\__problems_record_problem: . . . .
    . . . . . 9155, 9193
\__problems_sccline:n . . . . 9734,
    9753, 9756, 9775, 9779, 9788, 9839
\__problems_solution_start:n . . .
    . . . . . 9293, 9311, 9314
\c_problems_solutions_bool . . . .
    . . . . . 8956, 8972
\g_problems_subproblem_int . . . .
    . . . 9147, 9200, 9238, 9253, 10156,
    10157, 10179, 10196, 10197, 10220
\c_problems_test_bool 8964, 8978,
    9176, 9970, 9972, 9976, 10046, 10310
\c_problems_true_tl 9880, 9890, 9900
\ProcessKeysOptions . . . . .
    . . . 43, 8352, 8392, 8971, 9993, 10549
\ProcessOptions . . . . . 11
\prod . . . . . 8243
prop commands:
    \prop_clear:N . . . . . 5175
    \prop_const_from_keyval:Nn . 426, 439
    \prop_gclear:N . . . . . 2387, 2388,
        2389, 2447, 2448, 2449, 2512, 5411
    \prop_get:NnNTF . . . . . 465, 5496, 8933
    \prop_gput:Nnn . . . . .
        . . . 2537, 3033, 4094, 5440, 5475,
        5481, 5499, 6551, 7522, 8927, 10386
    \prop_gset_from_keyval:Nn . . . . .
        . . . . . 2505, 2506, 5424
    \prop_if_exist:NTF 929, 945, 1172, 5423
    \prop_if_in:NnTF . . . . .
        222, 1203, 1208, 5129, 5207, 5221, 5243
    \prop_item:Nn . . . . . 3290, 4129,
        5120, 5167, 5246, 7191, 7524, 8930
    \prop_map_break:n . . . . . 3360,
        4144, 4147, 4256, 4467, 4470, 6573
    \prop_map_function:NN . . . . . 2479
    \prop_map_inline:Nn . . . . .
        2507, 3122, 3368, 4104, 4150, 4168,
        4171, 4275, 4288, 4294, 4460, 4993,
        6562, 6570, 6682, 6685, 7164, 10398
    \prop_new:N . . . . . 4330, 8925, 10380
    \prop_put:Nnn . . . . . 4371, 4529,
        5209, 5223, 5252, 7186, 7197, 10393
    \prop_remove:Nn . . . . . 3756
    \prop_set_eq:NN . . . . . 1074
    \prop_to_keyval:N . . . . .
        . . . 2459, 2460, 2461, 2467, 2471, 5425
    \protect . . . . . 8453, 8936
    \protected . . . 1519, 6070, 6087, 6149, 6159
    \providecommand . . . 2236, 2243, 8524, 10554
    \ProvidesExplClass . . . . . 5, 8335
    \ProvidesExplPackage . . . . .
        . . . . . 20, 8379, 8945, 9981, 10540
    \pts . . . . . 9557
    pts . . . . . 76, 113, 118

Q
    \qrcode . . . . . 10049
    \qrid . . . 10069, 10110, 10157, 10197, 10311
    \qrjson . . . . . 10031, 10033, 10035,
        10036, 10082, 10087, 10094, 10102,
        10105, 10124, 10129, 10136, 10144,
        10147, 10169, 10171, 10176, 10182,
        10187, 10210, 10212, 10217, 10223,
        10228, 10237, 10243, 10258, 10262
    \qrschema . . . . . 10044, 10049
    \quad . . . . . 8432, 8436, 8440,
        8444, 8448, 9915, 9921, 10362, 10365
    quark commands:
        \q_no_value . . . . . 1200, 1205
        \quark_new:N . . . . . 2519, 2729
    quark internal commands:
        \q_stex_smsmode_break . . . . .
            . . . . . 2729, 2732, 2766
    \quizdata . . . . . 10530
    quizdata . . . . . 10454

R
    \raisebox . . . . . 9683, 9817
    \rangle . . . . . 8213
    \realize . . . . . 57, 58
    \ref . . . . . 86, 1868
    \refstepcounter . . . . . 9115, 10341
    \relax . . . . . 19, 20, 46,
        617, 618, 619, 620, 756, 757, 1748,
        1749, 1761, 1762, 1999, 2073, 2087,
        2173, 2514, 2515, 2736, 2738, 2752,
        2753, 2760, 3936, 4666, 4697, 4698,
        5294, 6168, 6189, 8469, 8761, 8763
    \renamedecr . . . . . 58, 146, 3336, 3672
    \renewcommand 8533, 8648, 8658, 10249, 10309
    \renewenvironment . . . . .
        . . . 8587, 8645, 8668, 8690, 8714, 8716
    \reqpts . . . . . 77, 119
    \requiremodule . . . 95, 96, 145, 3009, 3332
    \RequirePackage . . . . .
        . . . 4, 13, 19, 24, 25, 474, 477, 8336,
        8354, 8376, 8380, 8394, 8407, 8408,

```

8752, 8946, 8984, 9982, 9994, 10021,
 10022, 10541, 10552, 10557, 10558
`\resizebox` ... 10403, 10566, 10572, 10576
`\reversemarginpar` 10049
`\rhd` 8589, 8592
`\Rightarrow` 8114, 8257
`\rightmargin` 7856, 9252, 9526,
 9545, 9573, 9738, 10155, 10195, 10272
`\rule` 9336, 9343, 9396, 9401,
 9444, 9449, 9496, 9501, 10233, 10246
 rustex commands:
`\rustex_direct_HTML:n`
 8675, 8683, 8695, 8705
`\rustex_if:TF` 8673, 8674,
 8681, 8682, 8693, 8694, 8703, 8704
`\rustexBREAK` 2660

S

`sassertion (env.)` 101, 157, 7458
`scb (env.)` 9713
`\scb` 9089, 9596, 9761, 9782
`\scc` 9766, 9783
`\scriptsize` 8586
`\scriptstyle` 8592
`sdefinition (env.)` 101, 157, 7453
`\section` 26, 84, 139
`\sectiontitleemph` 8399, 8458
`sectocframes` 60, 108
 seq commands:
`\seq_clear:N` 142, 180, 456,
 861, 864, 2099, 2201, 2696, 3103,
 3104, 3105, 3106, 3108, 3245, 3342,
 4157, 4891, 5176, 5177, 5185, 5748,
 5816, 5881, 5890, 6391, 6671, 6804,
 6951, 7097, 7132, 7279, 7314, 8309
`\seq_count:N` .. 2142, 2164, 3627, 7426
`\seq_gclear:N` 2689, 2690, 5412
`\seq_get_right:NN` ... 9120, 9215, 9220
`\seq_gpush:Nn` 64, 801
`\seq_gput_right:Nn` .. 2654, 2674, 5441
`\seq_gset_eq:NN`
 244, 252, 3141, 3142, 3143, 3144
`\seq_gset_split:Nnn` 5430
`\seq_if_empty:NTF` 178,
 196, 264, 273, 299, 930, 946, 1193,
 1278, 1319, 2133, 2190, 4233, 7376,
 7750, 7766, 7998, 8056, 8081, 8318
`\seq_if_empty_p:N` 286, 287, 2209
`\seq_if_exist:NTF` 5429
`\seq_if_in:NnTF` 63, 800, 2213, 2224,
 2577, 2692, 2811, 2819, 3123, 3316,
 3366, 4166, 5005, 5132, 5263, 6680
`\seq_item:Nn` 265, 266,
 274, 2143, 2165, 5497, 6501, 6507, 7427

`\seq_map_break:` 960, 1092, 4144
`\seq_map_break:n` 3230,
 3238, 3416, 3444, 3452, 4147, 4256
`\seq_map_function:NN` 182, 186, 2136,
 2193, 3171, 4903, 7999, 8057, 8082
`\seq_map_indexed_function:NN` ...
 7377, 7751, 7767
`\seq_map_inline:Nn`
 866, 958, 1060, 1064,
 1225, 2204, 3163, 3184, 3216, 3305,
 3385, 3389, 3446, 3454, 4149, 4272,
 5891, 6517, 6952, 6979, 7107, 7149,
 7285, 7442, 7461, 7717, 7910, 8319
`\seq_new:N`
 24, 761, 2308, 2651, 2671, 2686
`\seq_pop:NN` 179
`\seq_pop_left:NN`
 163, 292, 293, 1064, 1226,
 1228, 2217, 3629, 3631, 3638, 3642
`\seq_pop_left:NNTF`
 1019, 6476, 6478, 6486
`\seq_pop_right:NN`
 . 197, 210, 212, 217, 219, 221, 979,
 1190, 1192, 1204, 1277, 1317, 1320,
 2205, 2885, 4232, 4269, 4894, 6820
`\seq_push:Nn` 185, 862, 2694, 3124, 3690
`\seq_put_left:Nn`
 164, 183, 2452, 4167, 5208, 5222, 6681
`\seq_put_right:Nn` 200, 213, 223, 226,
 235, 467, 873, 1209, 2214, 2218,
 2225, 2392, 2579, 3114, 3133, 3248,
 3265, 3367, 3616, 5244, 5804, 5821,
 5855, 5862, 5887, 5893, 5897, 5911,
 6459, 6466, 6487, 6489, 6806, 6954,
 6956, 7102, 7137, 7282, 7317, 7407
`\seq_reverse:N` 4895
`\seq_set_eq:NN`
 188, 209, 216, 234, 280,
 281, 1063, 1189, 1224, 5902, 6961, 6989
`\seq_set_split:Nnn`
 143, 211, 218, 865, 970, 971, 1018,
 1061, 1191, 1223, 1276, 1316, 2202,
 2203, 2884, 3626, 3636, 4231, 4268,
 4893, 6475, 6479, 6515, 6819, 8311
`\seq_use:Nn` 206,
 213, 226, 472, 475, 478, 879, 952,
 1021, 1215, 1282, 2132, 2189, 2207,
 2886, 3634, 3640, 3644, 4237, 4270,
 5431, 5731, 6721, 6809, 6931, 6962
`\l_tmpa_seq`
 456, 467, 472, 475, 478, 860,
 862, 865, 866, 970, 974, 1061, 1062,
 1064, 1191, 1192, 1193, 1204, 1209,
 1215, 1223, 1225, 1316, 1317, 1319,

1320, 1373, 1378, 2884, 2885, 2886,
 4268, 4269, 4270, 4891, 4893, 4894,
 4895, 4903, 6515, 6517, 6804, 6806,
 6809, 6819, 6820, 6951, 6954, 6956,
 6961, 6989, 8309, 8311, 8318, 8319
 \l_tmpb_seq 1063,
 1064, 1065, 1224, 1226, 1228, 1229
 \seqmap 98, 151, 297, 5793, 5935
 \setbox 144, 1900, 1998, 7743, 7956, 8716,
 8722, 8738, 9319, 9379, 9427, 9478
 \setcounter 10339, 10342
 \setkeys ... 2240, 2256, 2273, 8765, 10561
 \setlength 7854, 7855, 7856, 8538, 8539,
 8543, 8596, 8597, 8598, 9250, 9251,
 9252, 9524, 9525, 9526, 9543, 9544,
 9545, 9571, 9572, 9573, 9736, 9737,
 9738, 10153, 10154, 10155, 10193,
 10194, 10195, 10270, 10271, 10272
 \setlicensing 62, 110
 \setmetatheory 143, 2595, 2662
 \setnotation 32, 93, 153, 6636
 \setsectionlevel 26,
 84, 139, 1676, 8410, 8412, 8474, 8476
 \setSGvar 63, 112, 8926, 8936
 \setslidelogo 62, 110
 \setsource 62, 110
 sexample (env.) 101, 157, 7473
 \sf 8783
 \sffamily 8792
 sfragment (env.) 84, 139, 1547
 sfragment commands:
 _sfragment_do_level:nn
 1563, 1567, 1571, 1572,
 1573, 1574, 1575, 1579, 1591, 8451
 _sfragment_end: 1558, 1587, 1604, 1626
 \shorttitle 1614, 1615
 \skipfragment 84, 139, 1660
 \slideframewidth . 8542, 8543, 8632, 8761
 \slideheight 8539
 slides 60, 108
 \slidewidth
 .. 8538, 8635, 8671, 8761, 8763, 8767
 \smacro 94, 95
 \small 8169
 \smallskip 8165, 9163, 9168,
 9256, 9261, 9336, 9396, 9444, 9496,
 10072, 10077, 10114, 10119, 10160,
 10165, 10201, 10206, 10233, 10359
 smodule (env.) 88, 142, 2309
 \smodule 3333
 \Sn 19, 91, 6086
 \sn 19, 23, 91, 152, 1463, 6047
 \Sn\Sns 152, 6047
 \Sns 19, 91, 6087
 \sns 19, 91, 152, 6047
 \solution 68
 solution (env.) 1, 9283
 \solution 9087, 9346, 9597, 9762
 solutions 76, 113, 118
 \solutionsfalse 8975, 9351
 \solutionstrue 8973, 9348
 \source 124
 sparagraph (env.) 101, 157, 7476
 \spfarg 159, 7737, 8149
 spfblock (env.) 158, 8102
 \spfblock 7734, 8108
 \spfby 158, 7736, 8141
 \spfescape 7733, 7813, 7816
 \spfjust 158, 7735, 8137
 \spfsketch 158, 7990
 spfsketchenv (env.) 158, 7989
 \spfsketchenvautorefname 8008
 \spfstep 158, 7728, 8112
 \spfstepautorefname 7864, 8100
 spfstepenv (env.) 158, 8099
 \spfstepenv 8024, 8117
 \spfstepenvautorefname 8100
 sproblem (env.) 9086
 \sproblemautorefname 9185
 sproof (env.) 103, 158, 7707
 \sproofautorefname 7829
 \sproofend 159, 7831, 7848, 8157
 \sqcap 8260
 \square 8158, 9643, 9674
 \sr 19, 23, 90, 152, 6040
 \sref 86, 87, 101, 1840, 8877
 \sreflabel 86, 88, 1811
 \srefsetin 86, 2031
 \srefsym 91, 2048
 \srefsymuri 91, 2050, 2053
 \startsolutions 114, 9347
 \stepcounter 1662, 1666, 1670,
 1671, 1672, 1673, 1674, 8452, 8647
 \sTeX 14, 16, 83
 \stex 16, 23, 83
 stex commands:
 \l_stex_aB_args_seq 296,
 5731, 5748, 5804, 5816, 5821, 5855,
 5862, 5881, 5887, 5891, 5902, 5911,
 6931, 6952, 6961, 6962, 6979, 6989
 \stex_activate_module:N
 143, 2418, 2436, 2572, 2572, 2833, 2840
 \stex_activate_module:n 143, 2404,
 2407, 2572, 2573, 2576, 2978, 5186,
 7115, 7157, 7159, 7169, 7184, 7290
 _stex_activate_notations: 2586, 6560
 _stex_activate_symbols: . 2585, 4102

<code>\stex_add_definiens:nn</code>	<code>\stex_apply_patch_begin:n</code>
..... 157, 3338, 7502,	 525, 541, 3472, 3538
<u>7507</u> , 7507, 7720, 7722, 7913, 7915	<code>\stex_apply_patch_end:n</code>
<code>\stex_add_morphism:nnnn</code> 145, 2980,	 531, 556, 3477, 3543
<u>3032</u> , 3032, 3037, 3167, 7112, 7154	<code>\stex_archive_base:N</code>
<code>\stex_add_notation:nnnnn</code>	 132, <u>884</u> , 884, 1079
153, 3196, 6531, <u>6538</u> , 6545, 6558, 6638	<code>\stex_archive_base:n</code>
<code>\stex_add_symbol:nnnnnnN</code> .. 147, <u>4086</u>	132, <u>884</u> , 888, 1118, 1127, 1245, 1414
<code>\stex_add_symbol:nnnnnnnN</code>	<code>\stex_archive_id:N</code> 132, <u>881</u> ,
..... 148, 3074, 3220,	881, 903, 1072, 1076, 1079, 1080,
3225, 3886, 3954, 4086, 7083, 7413	1085, 1086, 1174, 1362, 2034, 2203
<code>\l_stex_all_module_seq</code>	<code>\stex_archive_path:N</code>
5185	 132, <u>891</u> , 891, 1100, 1222, 2202
<code>\l_stex_all_modules_seq</code>	<code>\stex_archive_path:n</code>
..... 142, 2099, <u>2308</u> , 2392,	 132, <u>891</u> , 895, 1110
2452, 2577, 2579, 2696, 4149, 4272	<code>\stex_args_end:</code> 5415,
<code>\l_stex_allow_semantic_bool</code>	5439, 5442, 6176, 6200, 6211, 6215
..... 151, 4637, 4658,	<code>\stex_assign_do:n</code>
4659, 4661, 4728, 4998, 5030, 5055,	 3283, 3596, 3598, 3605, 3657
5100, 5290, 5314, 5401, 5410, 5525,	<code>\l_stex_assoc_args_count</code>
5626, 5632, 5650, 5668, 5701, 5824,	 3986, 3990, 4032, 4033
5914, 5947, 5972, 6169, 6192, 7225	<code>\l_stex_brackets_dones_bool</code>
<code>\stex_annotate:nn</code>	 6859, 7000, 7006, 7007, 7024
128, 129, <u>709</u> , 714, 717, 1531, 1541,	<code>\stex_check_term:nn</code>
1853, 1969, 2121, 2122, 3268, 3270,	152, 2698, 3607, <u>6260</u> , 6261, 6269,
3611, 4041, 4062, 4065, 4072, 4077,	6275, 6279, 6283, 6288, 6708, 8174
4079, 4419, 4422, 4429, 4433, 4436,	<code>\stex_check_terms:</code> ... 152, 3885,
4573, 4578, 4581, 4588, 4592, 4595,	3949, 4342, 4507, <u>6271</u> , 6272, 6292
4731, 4917, 4926, 4931, 5161, 5682,	<code>\c_stex_check_terms_bool</code>
5715, 5830, 6013, 6220, 6223, 6717,	 36, 6247, 6250, 8176
6724, 6730, 6731, 6740, 6751, 6761,	<code>\stex_close_module:</code>
7335, 7362, 7509, 7549, 7560, 7738,	... 142, 2362, <u>2454</u> , 2454, 2635, 8288
7789, 7792, 7795, 7930, 7945, 8079,	<code>\stex_comma_sep_uri:nn</code>
8134, 8138, 8145, 8153, 8316, 8321,	 7377, 7387, 7751, 7767
8661, 8852, 9128, 9182, 9228, 9527,	<code>\stex_css_link:n</code>
9535, 9559, 9564, 9652, 9656, 9664,	129, <u>712</u>
9790, 9794, 9802, 9896, 9931, 10275	<code>\stex_css_literal:n</code>
<code>\stex_annotate_env</code>	 129, 81, <u>712</u> , 818, 1722, 8606
2334, 9744	<code>\l_stex_current_*</code>
<code>\stex_annotate_force_break:n</code> ...	149, 150
. 129, <u>712</u> , 1584, 1646, 3269, 3610,	<code>\l_stex_current_archive</code>
4059, 4077, 4416, 4434, 4571, 4593,	 132, 133, 902, 903, 912, 923,
4930, 5066, 5155, 5687, 5714, 5718,	<u>1060</u> , 1099, 1100, 1172, 1174, 1221,
5730, 5835, 6717, 7335, 7363, 7510,	1222, 1360, 1362, 2198, 2202, 2203
7550, 7738, 7741, 7784, 7955, 8317,	<code>\l_stex_current_args_tl</code>
9128, 9132, 9228, 9235, 9902, 9935	.. 4681, 5414, 5415, 5418, 6784, 6799
<code>\stex_annotate_invisible:n</code> . 129,	<code>\l_stex_current_arity_str</code>
<u>709</u> , 710, 1656, 2342, 2628, 3094,	 4680, 4748, 4763,
3596, 4040, 5453, 6717, 7787, 8314	4765, 5386, 5535, 5536, 5571, 6805
<code>\stex_annotate_invisible:nn</code>	<code>\l_stex_current_def_uri</code>
..... 129, <u>709</u> , 1506, 1701,	3257, 3259, 3262, 3266, 3268, 3274,
1707, 1713, 2947, 2983, 3020, 3609,	7398, 7420, 7421, 7427, 7428, 7495,
3683, 3723, 3733, 4397, 4555, 5631,	7497, 7500, 7502, 7544, 7546, 7549
5649, 5667, 5946, 7109, 7151, 7287,	<code>\l_stex_current_display_tl</code>
7535, 8143, 8151, 9010, 9033, 10283	 149, 4641,
	5293, 5309, 5312, 5698, 5818, 6062,

6066, 6079, 6083, 6094, 6108, 6122,	<code>\l_stex_current_section_level_-</code>
6146, 6156, 6194, 6404, 6819, 7232	<code>str</code> 139, 1526, 1533, 1543, 1602, 8463
<code>\l_stex_current_doc_uri</code> 7380	<code>\l_stex_current_symbol_args_tl</code> . .
<code>\l_stex_current_document_uri</code> . . .	 149, 150, 273
. 125, 135, 144, 253, 254,	<code>\l_stex_current_symbol_arity_int</code>
259, 1159, 1221, 1295, 1315, 1334,	 149, 150, 273
1338, 1343, 1345, 1452, 2430, 8179	<code>\l_stex_current_symbol_invoke_cs</code> 149
<code>\l_stex_current_domain_str</code> . . . 3751	<code>\l_stex_current_symbol_name_str</code> .
<code>\l_stex_current_domain_uri</code>	 6172, 6173, 6174, 6175
. . . 3039, 3052, 3056, 3058, 3063,	<code>\l_stex_current_symbol_return_tl</code>
3071, 3091, 3109, 3113, 3140, 3169,	 149, 150, 273
3194, 3489, 3514, 3556, 3580, 3700	<code>\l_stex_current_symbol_type_tl</code> . .
<code>\g_stex_current_file</code>	 149, 150, 273
. . 125, 244, 252, 257, 1102, 1112,	<code>\l_stex_current_symbol_uri</code>
1374, 2427, 2901, 9119, 9214, 9219	 149, 150,
<code>\l_stex_current_full_tl</code> 149, 4640,	273, 295, 3814, 3827, 3828, 3833,
4649, 4688, 4693, 4708, 4721, 4733,	3845, 3881, 3896, 3905, 3910, 3913,
4861, 4944, 5070, 5105, 5109, 5157,	3919, 3944, 3965, 4639, 4640, 4642,
5292, 5302, 5308, 5311, 5318, 5353,	5696, 6170, 6171, 6172, 7025, 7030
5354, 5365, 5378, 5379, 5408, 5489,	<code>\l_stex_current_term_tl</code> 4663, 4684,
5516, 5684, 5697, 5817, 5867, 5915,	4937, 4943, 5152, 5315, 5597, 5613,
5916, 6015, 6018, 6061, 6078, 6093,	5627, 5633, 5648, 5651, 5666, 5669,
6107, 6121, 6171, 6184, 6193, 6203,	5699, 5825, 5826, 5827, 5917, 6179
6204, 6206, 6403, 6739, 6750, 7227	<code>\stex_current_this:</code> . 4677, 7224, 7240
<code>\l_stex_current_language_str</code> . . .	<code>\l_stex_current_this_tl</code>
. 125, 127, 254, 258, 453,	 7049, 7052, 7226, 7234
458, 459, 1195, 1201, 1206, 1235,	<code>\l_stex_current_type_tl</code>
2336, 2870, 2872, 2888, 9124, 9224	 4683, 4730, 4803, 4858,
<code>\l_stex_current_module_str</code>	4866, 4869, 4873, 5186, 5187, 5262
. 2634, 3099, 3716, 3725, 3735	<code>\stex_deactivate_macro:Nn</code>
<code>\l_stex_current_module_uri</code>	 123, 67, 67, 2618, 2959,
142, 1293, 1422, 2100, 2308, 2335,	2966, 3014, 3327, 3328, 3329, 3330,
2356, 2391, 2423, 2440, 2451, 2457,	3331, 3332, 3333, 3496, 3502, 3521,
2458, 2459, 2460, 2461, 2466, 2468,	3562, 3568, 3588, 3603, 3678, 3747,
2472, 2477, 2479, 2555, 2556, 2561,	3808, 3839, 3928, 6141, 6150, 6160,
2581, 2582, 2587, 2588, 2590, 2612,	6166, 6349, 7070, 7125, 7175, 7505,
2697, 3033, 4087, 4094, 4103, 4104,	7539, 7554, 7562, 7816, 7987, 8045,
4116, 4121, 6549, 6551, 6561, 6562,	8108, 8132, 8136, 8140, 8148, 8156,
6570, 7164, 7408, 7420, 7514, 8286	8331, 9346, 9403, 9451, 9503, 9556,
<code>\l_stex_current_ns_uri</code> 2322	9595, 9596, 9597, 9598, 9599, 9600,
<code>\l_stex_current_rec_tl</code>	9617, 9712, 9760, 9761, 9762, 9763,
. 5091, 5108, 5137, 5138, 5155	9764, 9765, 9782, 9847, 9968, 10297
<code>\l_stex_current_redo_tl</code>	<code>\stex_debug:nn</code> 124, 103,
4664, 4669, 4676, 4686, 4687, 4690,	103, 133, 137, 166, 173, 237, 472,
4862, 4900, 4936, 5095, 5297, 5567	547, 616, 847, 852, 880, 922, 931,
<code>\l_stex_current_return_tl</code>	949, 996, 1033, 1067, 1075, 1096,
. . 4682, 4698, 5355, 5367, 5387, 5572	1166, 1237, 1315, 1353, 1366, 1381,
<code>\l_stex_current_section_level_-</code>	1390, 1498, 1851, 1877, 1879, 1888,
<code>int</code> 139, 1526, 1562,	1892, 1930, 1936, 1942, 1956, 1959,
1564, 1568, 1582, 1599, 1600, 1629,	1962, 1995, 1997, 2101, 2132, 2189,
1631, 1634, 1637, 1638, 1645, 1661,	2207, 2211, 2222, 2375, 2382, 2384,
1663, 1667, 1678, 1679, 1680, 1681,	2412, 2415, 2417, 2420, 2432, 2440,
1682, 1683, 1685, 1689, 1692, 1699,	2442, 2444, 2456, 2504, 2556, 2578,
1703, 1709, 1713, 1718, 8455, 8462	2587, 2592, 2596, 2598, 2699, 2752,

2773, 2777, 2795, 2812, 2820, 2902,
 2917, 2919, 2945, 3018, 3053, 3071,
 3125, 3153, 3190, 3205, 3262, 3281,
 3382, 3438, 3606, 3625, 3628, 3632,
 3652, 3656, 3681, 3701, 3717, 3721,
 3731, 3752, 3754, 3827, 4087, 4103,
 4116, 4230, 4271, 4273, 4292, 4309,
 4311, 4348, 4476, 4693, 4709, 4802,
 5046, 5075, 5093, 5128, 5184, 5200,
 5323, 5342, 5354, 5356, 5359, 5365,
 5378, 5400, 5408, 5523, 5563, 5574,
 5744, 5759, 5761, 5768, 5771, 6273,
 6277, 6281, 6285, 6432, 6463, 6474,
 6546, 6561, 6569, 6571, 6574, 6710,
 6737, 6838, 6851, 6892, 7010, 7182,
 7264, 7267, 7271, 7421, 7425, 7428,
 7430, 7500, 7521, 7557, 7718, 7911
 \c_stex_debug_clist 30,
 42, 104, 107, 121, 125, 127, 131, 135
 \c_stex_default_metatheory
 2098, 2695, 8287
 \l_stex_default_notation
 ... 4723, 5368, 5373, 5393, 5851,
 6787, 6788, 6794, 6803, 6808, 6811
 _stex_definiens_impl:nn
 3339, 7487, 7492
 \stex_do_default_notation:
 ... 154, 4722, 5366, 5850, 6782, 6782
 \stex_do_default_notation_op: ...
 154, 5390, 6782, 6783, 6792
 _stex_do_deprecation:n 649, 649, 3968
 _stex_do_for_list: ... 157, 3337,
 7313, 7313, 7397, 7713, 7906, 7993
 _stex_do_id: 654, 654,
 1556, 1623, 7342, 7360, 7726, 7926,
 7939, 7950, 7994, 8028, 9150, 10350
 \stex_do_up_to_module:n
 142, 2550, 2550,
 2553, 2557, 4097, 6553, 7168, 7215
 \g_stex_document_kind_parameters_-
 tl 1480, 1486, 10509
 \g_stex_document_kind_str
 14, 1479, 1481, 1485, 10497
 \stex_document_uri:n 134
 \stex_document_uri_archive:N ...
 134, 1137, 1137, 1338
 \stex_document_uri_element:N ...
 135, 1149, 1149
 \stex_document_uri_from_archive_-
 file:Nn 135, 1164,
 1164, 1876, 1968, 2063, 2108, 8867
 \stex_document_uri_language:N ...
 135, 254, 1146, 1146, 1235
 \stex_document_uri_name:N
 135, 1143, 1143, 1343, 1345
 \stex_document_uri_path:N
 135, 1140, 1140, 1334
 \stex_document_uri_with_language:Nn
 135, 1152, 1152, 2429
 _stex_eat_exclamation_point: ...
 150, 3974, 5357, 5368, 5589, 5589, 5590
 _stex_end: 414, 422
 \stex_end: 7250, 7260
 \stex_every_module:n
 142, 2546, 2547,
 2619, 2967, 3015, 3497, 3503, 3522,
 3563, 3569, 3589, 3809, 3840, 3929,
 6350, 7071, 7126, 7177, 7221, 8332
 \g_stex_every_module_tl
 2373, 2546, 2548, 8184
 \l_stex_every_symbol_tl 4618, 4620,
 4621, 4627, 4628, 4631, 4660, 4686
 \stex_execute_in_module:n
 142, 2397, 2554, 2554,
 2559, 2977, 3912, 7114, 7143, 7156
 \l_stex_feature_name_str .. 3086,
 3168, 3204, 3205, 3211, 3394, 3693
 \stex_file_in_smsmode:Nn
 144, 2433, 2685, 2688, 2933
 \stex_file_resolve:Nn 124, 147, 153,
 169, 176, 233, 243, 251, 867, 869, 1373
 \stex_file_set:Nn
 124, 141, 141, 146, 160, 171, 860, 1165
 \stex_file_split_off_ext:NN ...
 124, 208, 208
 \stex_file_split_off_lang:NN ...
 124, 215, 215, 2427, 9119, 9214, 9219
 \stex_file_use:N ... 124, 166, 173,
 205, 205, 237, 840, 860, 862, 870,
 874, 962, 996, 997, 999, 1012, 1065,
 1102, 1112, 1166, 1181, 1374, 1378,
 1799, 2210, 2221, 2432, 2434, 2901
 \l_stex_fors_seq
 157, 3245, 3248, 7314, 7317, 7376,
 7377, 7407, 7426, 7427, 7442, 7461,
 7717, 7750, 7751, 7766, 7767, 7910,
 7998, 7999, 8056, 8057, 8081, 8082
 \stex_get_env:Nn
 124, 86, 87, 98, 119, 229,
 231, 239, 241, 568, 574, 667, 838, 6244
 \stex_get_in_morphism:n 145, 3247,
 3256, 3381, 3381, 3594, 3650, 3673
 \stex_get_mathstructure:n
 156, 3055, 7099, 7133, 7241, 7241, 7270
 \stex_get_mathstructure_maybe:nTF
 7242, 7246, 7263

\l_stex_get_structure_module_uri	\stex_has_definiens:n	4128
..... 156, 3056,	\stex_has_definiens:NTF	147, <u>4125</u>
7141, 7144, 7247, 7250, 7264, 7271	\stex_has_definiens:nTF	
\l_stex_get_svariable_str	 147, <u>4125</u> , 4126	
.... 6197	\stex_has_definiens_p:N	147, <u>4125</u>
\stex_get_symbol:n	\stex_has_definiens_p:n	147, <u>4125</u>
..... 148, 2049, <u>4182</u> , 4184,	\c_stex_home_file	125, <u>238</u> , 840, 860
4605, 6043, 6060, 6077, 6191, 6327,	\stex_html_literal:n	
6653, 7316, 7494, 7543, 8995, 9018	... 27, 35, 764, 772, 1689, 1692, 2075	
\stex_get_symbol:nTF	\stex_html_node:nnn	
.. 148, 1462,	 128, 129, <u>709</u> , 1484, 1583	
<u>4182</u> , 4185, 4190, 4210, 4211, 7248	\stex_if_check_terms:	6240, 6251, 6255
\l_stex_get_symbol_args_tl	\stex_if_check_terms:TF	
..... 148, 3419, 3958,	 152, 3830, 4342, 4351, 4507,	
3978, 3980, 3981, 4219, 4259, 4320,	4510, <u>6238</u> , 6260, 6271, 6329, 6368	
4375, 4387, 4480, 4533, 4546, 4609,	\stex_if_check_terms_p:	152, <u>6238</u>
5418, 6598, 6600, 6737, 6799, 7416	\stex_if_do_html:	 695
\l_stex_get_symbol_arity_int	\stex_if_do_html:TF	128, <u>695</u> , 704,
.... 148, 3418, 3901, 3957, 3979,	1505, 1529, 1539, 1687, 2354, 2364,	
3989, 3995, 3999, 4003, 4007, 4011,	2621, 2636, 2946, 2982, 3019, 3088,	
4015, 4019, 4023, 4027, 4030, 4031,	3146, 3263, 3595, 3608, 3682, 3832,	
4032, 4033, 4070, 4182, 4218, 4258,	3897, 3908, 3951, 4345, 4353, 4396,	
4319, 4347, 4374, 4386, 4427, 4479,	4506, 4512, 5712, 6331, 7108, 7150,	
4532, 4545, 4586, 4608, 6393, 6427,	7286, 7419, 7508, 7534, 7714, 7719,	
6458, 6465, 6485, 6534, 6639, 7415	7725, 7739, 7824, 7836, 7844, 7907,	
\l_stex_get_symbol_def_tl	7912, 7922, 7943, 7946, 7953, 7964,	
..... 148, 3295,	7984, 8052, 8076, 8794, 9117, 9149,	
3420, 4220, 4260, 4321, 4481, 4610	9156, 9157, 9160, 9180, 9217, 9245,	
\l_stex_get_symbol_invoke_cs	9247, 9301, 9310, 9323, 9328, 9364,	
..... 148, 3423,	9372, 9383, 9388, 9412, 9420, 9431,	
4223, 4263, 4324, 4484, 4613, 7249	9436, 9463, 9472, 9483, 9488, 9522,	
\l_stex_get_symbol_macro_str	9577, 9602, 9604, 9651, 9742, 9767,	
. 3417	9769, 9789, 9915, 9921, 10274, 10344	
\l_stex_get_symbol_name_str	\stex_if_do_html_p:	128, <u>695</u>
.. 3288	\stex_if_file_absolute:N	263, 272
\l_stex_get_symbol_return_tl	\stex_if_file_absolute:NTF	
..... 148, 3422, 4222,	 125, <u>262</u> , 868	
4262, 4323, 4349, 4483, 4498, 4612	\stex_if_file_absolute_p:N	125, <u>262</u>
\l_stex_get_symbol_type_tl	\stex_if_file_starts_with:NN	125, 279
..... 148, 3421, 4221, 4261, 4322,	\stex_if_file_starts_with:NNTF	..
4482, 4611, 7100, 7135, 7250, 7280	 125, 279, 1062	
\l_stex_get_symbol_uri	\stex_if_html_backend	 518
. 148, 1463,	\stex_if_html_backend:TF	128,
2050, 3249, 3257, 3280, 3287, 3383,	179, 2, 529, <u>685</u> , 688, 712, 725, 739,	
3388, 3392, 3408, 3429, 3606, 3609,	1482, 1578, 1642, 1655, 1697, 1966,	
3617, 3681, 3684, 3691, 3693, 3828,	2068, 2119, 2332, 4666, 4697, 5294,	
3905, 4191, 4207, 4217, 4257, 4318,	5593, 5609, 5623, 5643, 5661, 5680,	
4607, 6061, 6063, 6078, 6080, 6170,	5710, 6012, 6168, 6189, 6218, 6239,	
6193, 6195, 6333, 6337, 6355, 6356,	7333, 7346, 8355, 8395, 8525, 8605,	
6532, 6643, 6646, 6654, 6655, 6656,	8660, 8672, 8680, 8692, 8702, 8851,	
6657, 6662, 6666, 7318, 7495, 7544,	8862, 8905, 9641, 9784, 9887, 9924	
8996, 9007, 9011, 9019, 9030, 9034	\stex_if_html_backend_p:	128, <u>685</u>
\stex_get_var:n	\stex_if_in_module:	2560
..... 149, <u>4451</u> ,		
4451, 6091, 6103, 6117, 6365, 7533		
\l_stex_get_variable_str		
.... 4452, 4454, 4478, 6093, 6094,		
6105, 6107, 6108, 6119, 6121, 6122,		
6198, 6199, 6366, 6371, 6374, 7535		
\stex_has_definiens:N		
..... 4125		

<code>\stex_if_in_module:TF</code>	<code>_stex_invoke_symbol:nnnnnnN</code> ...
143, 1292, 2367, 2554, <u>2560</u> , 6637, 7513	 149, 4120, 4198, 4215, 4216,
<code>\stex_if_in_module_p:</code> 143, <u>2560</u>	<u>4636</u> , 4636, 4655, 5211, 5225, 7199
<code>\stex_if_module_exists:N</code> 2564	<code>\stex_invoke_text_symbol:</code>
<code>\stex_if_module_exists:n</code> 2568	 149, 3893, <u>4696</u> , 4696
<code>\stex_if_module_exists:NTF</code>	<code>_stex_invoke_variable:nnnnnn</code> ...
..... 143, 1350, 2383,	 150, <u>5289</u>
2416, 2443, <u>2564</u> , 2828, 2837, 2934	<code>_stex_invoke_variable:nnnnnnN</code> ..
<code>\stex_if_module_exists:nTF</code>	 4384, 4543, 5289, 5784
..... 143, <u>2564</u> , 7101, 7136, 7281	<code>_stex_is_sequentialized:n</code>
<code>\stex_if_module_exists_p:N</code> 143, <u>2564</u>	 5739, 5822, 5895, 5912
<code>\stex_if_module_exists_p:n</code> 143, <u>2564</u>	<code>\stex_iterate_break:</code>
<code>\stex_if_smsmode:</code> 2682	 148, <u>4139</u> , 4140, 4143
<code>\stex_if_smsmode:TF</code>	<code>\stex_iterate_break:n</code>
. 144, 655, 1496, 1812, 1831, 2355,	 148, 3358, 3722,
2363, <u>2680</u> , 2900, 2950, 2986, 3023,	3732, 3750, <u>4139</u> , 4140, 4146, 4317
3275, 3464, 3479, 3487, 3512, 3530,	<code>\stex_iterate_morphisms:nn</code>
3546, 3554, 3578, 3804, 3835, 3917,	 145, <u>3341</u> , 3341, 3700, 3716
6340, 7262, 7326, 7345, 7361, 7367,	<code>\stex_iterate_notations:nn</code>
7441, 7460, 7474, 7512, 9245, 9247	 154, 3194, 5262, <u>6670</u> , 6670
<code>\stex_if_smsmode_p:</code> 144, <u>2680</u>	<code>\stex_iterate_symbols:n</code>
<code>\stex_ignore_spaces_and_pars:</code> ...	 148, <u>4139</u> , 4139, 4310
..... 123, <u>45</u> , 45, 48,	<code>\stex_iterate_symbols:nn</code>
2611, 2613, 7367, 9173, 10099, 10141	... 148, 3113, 3753, <u>4156</u> , 4156, 5187
<code>\l_stex_import_uri</code> 2597,	<code>\l_stex_key_*</code> 147
2598, 2600, 2939, 2940, 2948, 2952,	<code>\l_stex_key_answerclass_str</code>
2953, 2971, 2972, 2978, 2981, 2984,	 9286, 9290, 9304, 9338, 9339
2989, 2990, 2997, 3000, 3010, 3011,	<code>\l_stex_key_archive_str</code> 379,
3021, 3025, 3026, 3059, 3061, 3063	382, 1875, 1928, 1936, 1942, 1943, 1967
<code>\stex_in_archive:nn</code>	<code>\l_stex_key_args_str</code>
.. 132, 898, 898, 1875, 1943, 1967,	 3764, 3769, 3778, 3818,
2038, 2062, 2107, 2130, 2158, 2187,	3849, 3982, 3991, 3996, 4000, 4004,
2231, 2247, 2262, 2274, 2859, 8866	4008, 4012, 4016, 4020, 4024, 4028,
<code>\g_stex_in_comp_bool</code>	4043, 4399, 4493, 4494, 4557, 7298
..... 5706, 6006, 6011, 6022	<code>\l_stex_key_argtypes_clist</code>
<code>\stex_in_invisible_html_bool</code> ...	3783, 3787, 4076, 4078, 4432, 4435,
..... 4060, 4417, 5592, 5625	4591, 4594, 6286, 6287, 6288, 7302
<code>\l_stex_in_meta_bool</code>	<code>\l_stex_key_assoc_str</code> 3767, 3772,
..... 2394, 2403, 2406, 2408	4050, 4051, 4403, 4404, 4561, 4562
<code>\l_stex_in_this_bool</code>	<code>\l_stex_key_autogradable_bool</code> ...
..... 5101, 5595, 5605,	 9060, 9066, 9071, 9125, 9225
5611, 5621, 5645, 5659, 5663, 5677,	<code>\l_stex_key_continues_tl</code> . 7622, 7632
5713, 5727, 5973, 7222, 7233, 7235	<code>\l_stex_key_def_tl</code> 3780, 3790, 3817,
<code>\l_stex_inparray_bool</code> 7012	3848, 3863, 3867, 3890, 3959, 4064,
<code>\stex_input_with_hooks:Nn</code> .. 125,	4065, 4376, 4388, 4421, 4422, 4534,
<u>245</u> , 245, 248, 2082, 2102, 2110, 2113	4547, 4580, 4581, 6278, 6279, 7300
<code>\stex_invoke_sequence:</code>	<code>\stex_key_def_tl</code> 152
... 150, 4537, 4550, <u>4701</u> , 4701, 5786	<code>\l_stex_key_deprecate_str</code>
<code>\stex_invoke_structure:</code>	 405, 407, 650, 651
..... 150, <u>4790</u> , 4790, 7085, 7249	<code>\l_stex_key_due_tl</code>
<code>\stex_invoke_symbol:</code> .. 149, 3082,	 10320, 10324, 10364, 10365
3962, 4379, 4391, <u>4692</u> , 4692, 7417	<code>\l_stex_key_fallback_tl</code>
<code>_stex_invoke_symbol:NnnnnnN</code> ...	 1803, 1806, 1857, 1889, 1960
..... 149, 4606, <u>4636</u> , 4654, 4657	

\l_stex_key_feedback_tl	\l_stex_key_number_tl
..... 9508, 9511, 9534, 9537,	 10318, 10322, 10338, 10339
9550, 9551, 9626, 9631, 9663, 9665,	\l_stex_key_numname_str
9699, 9700, 9801, 9803, 9833, 9834,	7581, 7585, 7590, 7599, 7761, 7772,
10265, 10282, 10285, 10292, 10293	7875, 7880, 7885, 8049, 8060, 8085
\l_stex_key_file_str	\l_stex_key_op_tl
..... 380, 383, 1843, 1876,	... 3902, 4500, 4502, 6299, 6305,
1883, 1929, 1936, 1942, 1944, 1968	6386, 6387, 6388, 6395, 6396, 6401,
\l_stex_key_for_clist	6407, 6536, 6628, 6629, 6632, 6641,
157, 3246, 7297, 7305, 7315, 7566, 7571	6645, 6647, 6657, 6659, 6748, 6752
\l_stex_key_for_str	\l_stex_key_post_tl
7332	... 1802, 1863, 1868, 6027, 6031,
\l_stex_key_from_tl	6066, 6083, 6105, 6119, 6146, 6156
7624, 7634	\l_stex_key_pre_tl
\l_stex_key_Ftext_tl	... 1801, 1863, 1868, 6026, 6030,
.. 9629, 9635, 9660, 9697, 9798, 9831	6066, 6083, 6105, 6119, 6146, 6156
\l_stex_key_functions_tl . 7623, 7633	\l_stex_key_prec_str
\l_stex_key_given_tl	3904,
..... 10319, 10323, 10361, 10362	6298, 6304, 6435, 6438, 6441, 6447,
\l_stex_key_hide_bool 7621, 7629,	6450, 6453, 6456, 6462, 6474, 6475
7743, 7753, 7769, 7836, 7956, 7984	\l_stex_key_proofend_tl
\l_stex_key_id_str . 387, 389, 656,	 7620, 7626, 8164, 8165
657, 7379, 7380, 7435, 7437, 7923,	\l_stex_key_pts_str
7936, 8025, 9295, 9297, 9303, 9358,	9507, 9510, 9529, 9530, 9547, 9548,
9360, 9366, 9406, 9408, 9414, 9456,	10266, 10277, 10278, 10289, 10290
9458, 9465, 9516, 9518, 9528, 9546,	\l_stex_key_pts_tl
10089, 10131, 10178, 10219, 10239,	9062,
10251, 10253, 10264, 10273, 10276	9068, 9126, 9135, 9139, 9226, 9237,
\l_stex_key_intent_args_clist ...	9240, 9256, 10160, 10183, 10201, 10224
..... 6301, 6307, 6499, 6508, 6510	\l_stex_key_reorder_str 3766, 3770,
\l_stex_key_intent_str	4053, 4054, 4409, 4410, 4567, 4568
..... 3903, 6300, 6306, 6424, 6502	\l_stex_key_return_tl
\l_stex_key_just_tl	3781, 3786, 3961, 4067, 4070, 4349,
..... 7569, 7574, 7785, 7791, 7792	4378, 4390, 4424, 4427, 4498, 4536,
\l_stex_key_macroname_str	4549, 4583, 4586, 6282, 6283, 7301
.. 7296, 7306, 7392, 7394, 7410, 7414	\stex_key_return_tl
\l_stex_key_method_tl	152
..... 7568, 7573, 7785, 7794, 7795	\l_stex_key_role_str
\l_stex_key_mhrepos_str	 3765, 3773, 3880, 4056,
. 9065, 9073, 9269, 9271, 9279, 10377	4057, 4406, 4407, 4564, 4565, 7411
\l_stex_key_min_tl	\l_stex_key_root_tl
9063,	 6028, 6032, 6036, 6038
9069, 9140, 9243, 9261, 10165, 10206	\l_stex_key_short_tl
\l_stex_key_name_str	.. 1548, 1550, 1593, 1596, 1613, 1615
3777,	\l_stex_key_showname_bool . 7577,
3785, 3815, 3846, 3861, 3865, 3875,	7582, 7586, 7592, 7597, 7614, 7876
3876, 3878, 3882, 3888, 3920, 3941,	\l_stex_key_sig_str .. 2312, 2327,
3942, 3945, 3956, 3968, 4042, 4337,	2337, 2371, 2412, 2430, 2432, 2434
4338, 4348, 4355, 4358, 4371, 4373,	\l_stex_key_style_clist
4385, 4398, 4490, 4491, 4514, 4517,	399, 401, 504, 508, 542, 546, 7382,
4529, 4531, 4544, 4556, 6366, 6623,	7383, 7477, 9582, 9583, 9587, 9590,
6624, 6625, 6629, 6630, 6631, 7295,	9715, 9722, 9746, 9747, 9752, 9755
7304, 7331, 7393, 7394, 7400, 7408,	\l_stex_key_T_bool
7414, 7420, 7580, 7584, 7591, 7594,	... 9627, 9632, 9633, 9653, 9657,
7615, 7773, 7774, 7874, 7881, 7886,	9680, 9694, 9791, 9795, 9814, 9828
8011, 8013, 8019, 8020, 8061, 8062,	\l_stex_key_term_tl
8086, 8087, 9064, 9070, 9118, 9120,	 7567, 7572, 7785, 7788, 7789
9123, 9213, 9215, 9218, 9220, 9223	

<code>\l_stex_key_testspace_dim</code>	<code>\c_stex_language_abbrevs_prop</code> ...
..... 9284, 9287, 9289,	 127 , 426
9318, 9872, 9874, 9930, 9959, 9962	<code>\c_stex_languages_clist</code> . 31, 454, 457
<code>\l_stex_key_title_str</code>	<code>\c_stex_languages_prop</code>
..... 7307	 127 , 222, 426 , 465, 1203, 1208
<code>\l_stex_key_title_tl</code>	<code>\g_stex_last_feature_str</code>
..... 393, 395, 1926, 1936,	2634
1982, 1983, 2348, 7330, 7336, 7337,	<code>\l_stex_macroname_str</code>
9128, 9129, 9130, 9134, 9228, 9229,	147 , 2624, 2625, 3794, 3799, 3801,
9230, 9232, 9233, 9337, 9338, 9397,	3825, 3874, 3887, 3955, 4044, 4045,
9398, 9445, 9446, 9497, 9498, 10089,	4336, 4372, 4383, 4400, 4401, 4489,
10131, 10178, 10219, 10234, 10235,	4530, 4542, 4558, 4559, 7086, 7410
10239, 10345, 10346, 10356, 10357	<code>\c_stex_main_archive</code> 133 , 1060 , 2034
<code>\l_stex_key_Ttext_tl</code>	<code>\c_stex_main_document_uri</code>
.. 9628, 9634, 9658, 9695, 9796, 9829	 135 , 1221 , 1878
<code>\l_stex_key_type_tl</code>	<code>\c_stex_main_file</code>
3779, 3789, 3816, 3847, 3862, 3866,	 124 , 228 , 244, 1224, 1231
4061, 4062, 4377, 4389, 4418, 4419,	<code>_stex_map_args:N</code>
4577, 4578, 6274, 6275, 7299, 7308	 153 , 4071, 4428, 4587, 5420,
<code>\stex_key_type_tl</code>	6495, 6521, 6597 , 6597, 6744, 6801
152	<code>_stex_map_notation_args:N</code>
<code>\l_stex_key_variant_str</code>	 153 , 6597 , 6610, 6764
..... 3851, 4360, 4519,	<code>\c_stex_mathhub<id>_archive</code> .. 132
6297, 6303, 6310, 6312, 6354, 6376,	<code>\c_stex_mathhub_file</code>
6422, 6533, 6623, 6629, 6719, 6752	962
<code>\l_stex_key_wikidata_str</code>	<code>\c_stex_mathhub_files</code>
..... 3782, 3788, 4047, 4048	... 132 , 837 , 930, 946, 952, 958, 1060
<code>\l_stex_keys_cls_id</code> 6, 13, 36, 43, 44,	<code>\c_stex_mathhub_main_archive</code> ...
46, 81, 743, 750, 773, 780, 781, 783, 818	 1070, 1071
<code>\l_stex_keys_counter_id</code>	<code>_stex_maybe_brackets:nn</code>
..... 7, 10, 28, 37, 61, 62, 63, 64,	 154 , 4710,
744, 747, 765, 774, 798, 799, 800, 801	5380, 5391, 6841, 6857, 7005 , 7005
<code>\l_stex_keys_counter_parent</code>	<code>\c_stex_metadata_bool</code> 37, 9005, 9028
..... 8, 11, 29, 48, 50, 51, 52, 53,	<code>\stex_metagroup_do_in:n</code>
54, 55, 56, 58, 745, 748, 766, 785,	 125 , 126 , 327 ,
787, 788, 789, 790, 791, 792, 793, 795	327, 331, 348, 2551, 3264, 3615, 3689
<code>\stex_keys_define:nnnn</code> 126 ,	<code>\stex_metagroup_new:</code>
5, 359 , 359, 378, 386, 392, 398,	126 , 323 , 324, 2380, 2413, 2439, 3098
404, 410, 742, 1547, 1800, 1810,	<code>\l_stex_metatheory_uri</code> 2098, 2309,
2311, 3763, 3776, 3860, 6025, 6035,	2316, 2318, 2338, 2339, 2374, 2375,
6127, 6132, 6296, 6317, 6319, 7048,	2398, 2600, 2602, 2695, 8286, 8287
7294, 7565, 7579, 7619, 7637, 8010,	<code>_stex_module_code_macro:N</code>
8552, 9061, 9268, 9285, 9354, 9506,	2302, 2386, 2446, 2458, 2477, 2511,
9625, 9870, 10317, 10421, 10472, 10485	2555, 2565, 2582, 2587, 2588, 2612
<code>\stex_keys_set:nn</code>	<code>_stex_module_code_macro:n</code>
126 , 17, 374 , 374, 754, 1554, 1609,	 2305, 2569, 7183
1841, 1946, 2347, 3797, 3824, 3872,	<code>\stex_module_setup:n</code>
3873, 4335, 4488, 6042, 6059, 6076,	 142 , 2352, 2366 , 2366, 2630
6090, 6102, 6116, 6138, 6144, 6154,	<code>_stex_module_setup_top_nosig:n</code> .
6326, 6364, 7073, 7323, 7358, 7712,	 2372, 2379, 8183
7873, 7905, 7992, 8018, 8646, 9109,	<code>\stex_module_setup_top_nosig:n</code> ..
9113, 9203, 9278, 9294, 9317, 9357,	 142 , 2379
9405, 9455, 9515, 9568, 9649, 9691,	<code>\stex_module_uri:n</code>
9733, 9787, 9825, 9926, 9943, 10250,	135
10332, 10336, 10376, 10434, 10496	<code>\stex_module_uri_archive:N</code>
<code>\stex_kpsewhich:Nn</code> 123 , 77 , 77, 89, 99	 135 , 1251 , 1251, 2852

<code>\stex_module_uri_as_qm:n</code>	<code>\c_stex_no_archive_str</code>
..... 136, <u>1285</u> , 1285, 4273, 4274	 133, <u>1051</u> , 1177, 1377, 2855
<code>\stex_module_uri_name:N</code> 136, <u>1260</u> ,	<code>\c_stex_no_archive_uri</code> ... 1053, 1059
1260, 2335, 2854, 2953, 2990, 3026	<code>\c_stex_no_frontmatter_bool</code> 39, 1743
<code>\stex_module_uri_name:n</code>	<code>\stex_notation_add:</code>
..... 136, <u>1260</u> , 1264, 4281, 4284	... 153, 3831, 3907, 6330, <u>6530</u> , 6530
<code>\stex_module_uri_path:N</code>	<code>\l_stex_notation_args_tl</code>
..... 135, <u>1254</u> , 1254, 2853	 6392, 6425,
<code>\stex_module_uri_path:n</code>	6494, 6500, 6506, 6611, 6613, 6710
..... 135, <u>1254</u> , 1257, 4293	<code>\stex_notation_check:</code> . 154, 3830,
<code>\stex_module_uri_split_name:NNN</code> .	4351, 4510, 6329, 6368, <u>6707</u> , 6707
..... 136, <u>1267</u> , 1267, 7141	<code>\l_stex_notation_cs</code>
<code>\stex_module_uri_split_name:NNn</code> .	 154, 4709, 4712, 4723,
..... 136, <u>1267</u> , 1271	4750, 4754, 4762, 4769, 4784, 4945,
<code>\l_stex_morphism_assigns_seq</code> 3106,	5357, 5361, 5382, 5848, 6697, 6703
3143, 3160, 3216, 3265, 3316, 3616	<code>\stex_notation_do_html:n</code>
<code>\l_stex_morphism_morphisms_seq</code> ..	 154, 3833,
..... 3105, 3133, 3144	3910, 4355, 4514, 6333, <u>6716</u> , 6716
<code>\l_stex_morphism_renames_seq</code> 3104,	<code>\l_stex_notation_downprec</code>
3142, 3157, 3171, 3184, 3389, 3690	 5700, 6997, 7004, 7010, 7011
<code>\l_stex_morphism_symbols_prop</code> ...	<code>\l_stex_notation_macrocode_cs</code> ...
..... 3290, 3756	... 153, 3853, 4362, 4521, 6358,
<code>\l_stex_morphism_symbols_seq</code> ...	6378, 6420, 6432, 6535, 6623, 6626,
..... 3103, 3114, 3141,	6640, 6644, 6655, 6710, 6711, 6742
3154, 3163, 3305, 3385, 3446, 3454	<code>\stex_notation_make_args:</code>
<code>\stex_morphisms_macro:N</code> .. 2290,	 154, 3854,
2387, 2447, 2459, 2468, 2505, 3033	4363, 6359, 6379, 6712, <u>6763</u> , 6763
<code>\stex_morphisms_macro:n</code>	<code>\stex_notation_parse:n</code>
.. 2293, 3122, 3368, 4168, 6685, 7186	 153, 3829, 3906, 4350,
<code>\stex_new_document_element_uri:n</code>	4501, 4504, 6328, 6367, <u>6385</u> , 6385
..... 135, <u>1158</u> , 1158, 1813	<code>\stex_notation_set_default:n</code> ...
<code>\stex_new_module_uri:n</code>	... 153, 6336, 6371, <u>6636</u> , 6636, 6661
..... 136, <u>1291</u> , 1291,	<code>\stex_notations_macro:N</code>
2381, 2414, 2441, 7080, 7144, 7169	... 2296, 2389, 2449, 2461, 2479,
<code>\stex_new_statement:nnn</code>	2512, 2538, 6551, 6561, 6562, 6570
157, <u>7321</u> , 7321, 7453, 7458, 7473, 7476	<code>\stex_notations_macro:n</code> . 2299, 6682
<code>\stex_new_stylable_cmd:nnnn</code> 127,	<code>\stex_par:</code>
<u>482</u> , 482, 2938, 2961, 3009, 3508,	.. 9716, 9717, 9720, 9723, 9724, 9727
3574, 3593, 3672, 3698, 3796, 3823,	<code>\stex_persist:n</code>
3871, 4334, 4487, 6325, 6363, 7990	127,
<code>\stex_new_stylable_env:nnnnnnn</code> ..	184, <u>586</u> , 614, 615, 726, 727, 729,
.. 127, <u>516</u> , 516, 2346, 3461, 3470,	732, 735, 736, 1035, 1078, 1085, 2465
3526, 3536, 7061, 7096, 7130, 7322,	<code>\c_stex_persist_force_bool</code> .. 35, 581
7818, 7840, 7904, 9103, 9202, 9309,	<code>\c_stex_persist_mode_bool</code>
9371, 9419, 9471, 9567, 9732, 10329	 33, 571, 582, 600
<code>\stex_new_symbol_uri:n</code>	<code>\stex_persist_read_now:</code>
136, <u>1421</u> , 1421, 3197, 3882, 3945, 7229	 599, 1084, 8294
<code>\stex_new_symbol_uri:nn</code>	<code>\c_stex_persist_write_mode_bool</code> .
..... 136, <u>1424</u> , 1424, 3115,	... 34, 577, 601, 607, 724, 1083, 2455
4121, 4257, 4318, 5194, 7199, 7420	<code>\stex_pseudogroup:nn</code> 125, 126, 250,
<code>\stex_next_symbol:n</code>	<u>303</u> , 303, 700, 899, 2580, 5409, 7035
149, <u>4615</u> , 4617, 4635, 5009, 5150, 6164	<code>\stex_pseudogroup_restore:N</code>
<code>\c_stex_no_archive</code>	 126, 258,
133, <u>1051</u>	259, <u>306</u> , 306, 912, 2590, 7040, 7041

<code>\stex_pseudogroup_with:nn</code>	<code>\g_stex_sms_import_code</code>
..... 126, 313, 313, 4140, 4158,	 144, 2679, 2691, 2708, 2999
4215, 6009, 6672, 6929, 6945, 6970	
<code>\c_stex_pwd_file</code>	<code>\stex_smsmode_do:</code> 144, 2360, 2604,
..... 124, 228, 870, 1062, 1063, 1799	2615, 2733, 2735, 2738, 2738, 2963,
<code>\stex_reactivate_macro:N</code>	3492, 3518, 3559, 3585, 3600, 3675,
..... 123, 67, 73, 2619, 2960,	3805, 3836, 3925, 6346, 7063, 7118,
2967, 3006, 3015, 3334, 3335, 3336,	7162, 7274, 7343, 7367, 7368, 7489
3498, 3504, 3523, 3564, 3570, 3590,	<code>\stex_source_path:n</code>
3809, 3840, 3929, 6350, 7071, 7127,	 133, 1098, 1098, 1107, 1883,
7178, 7446, 7447, 7448, 7449, 7450,	1944, 2039, 2072, 2082, 2101, 2102,
7451, 7465, 7466, 7467, 7468, 7469,	2110, 2113, 2238, 2254, 2271, 2861
7470, 7471, 7727, 7728, 7729, 7730,	<code>\stex_source_path:nn</code>
7731, 7732, 7733, 7734, 7735, 7736,	... 133, 1098, 1105, 2233, 2249, 2264
7737, 8332, 9087, 9088, 9089, 9090,	<code>_stex_sref_do_aux:n</code>
9091, 9092, 9093, 9481, 9601, 9766	8175
<code>\stex_ref_new_doc_target:n</code>	<code>\stex_str_if_ends_with:nn</code> .. 123, 62
..... 657, 1811, 1811, 1829	<code>\stex_str_if_ends_with:nnTF</code>
<code>_stex_ref_new_id:n</code>	 123, 62,
7436	588, 591, 3707, 3730, 4274, 4281, 4293
<code>\stex_ref_new_sym_target:n</code>	<code>\stex_str_if_ends_with:p:nn</code>
..... 1830, 1830, 6203, 7443, 7462	 123, 62, 4246, 4315
<code>\stex_ref_new_symbol:n</code>	<code>\stex_str_if_starts_with:nn</code> 123, 57
..... 2045, 2508, 3895, 3964	<code>\stex_str_if_starts_with:nnTF</code> ...
<code>\l_stex_relative_import_bool</code> ...	. 123, 57, 413, 1029, 1911, 4823, 4827
..... 1312, 1314, 1352, 2998	<code>\stex_str_if_starts_with:p:nn</code> ...
<code>_stex_renamedec1_do:nn</code>	 123, 57
..... 3670, 3674, 3680	<code>\l_stex_struct_this_tl</code>
<code>\stex_require_module:N</code> . 145, 2602,	4809, 5011, 5012, 5015, 5096, 5099,
2827, 2827, 2940, 2972, 3011, 3061	5112, 5128, 5134, 5135, 5141, 5974
<code>\stex_require_module_noerr:N</code> ...	<code>\stex_structural_feature_-</code>
..... 145, 2827, 2836, 2847	module:nn ... 143, 2620, 2620, 7088
<code>\stex_require_module_noerr:n</code> ...	<code>\stex_structural_feature_module_-</code>
..... 145, 2827, 2845, 3000	end: 143, 2620, 2633, 7065, 7120, 7167
<code>_stex_return_args:nn</code>	<code>\stex_structural_feature_-</code>
..... 3973, 4071, 4428, 4587	morphism:nnnnn
<code>\l_stex_return_notation_tl</code>	145,
..... 4675, 4845, 4853, 5017,	3039, 3046, 3462, 3510, 3527, 3576
5018, 5113, 5143, 5144, 5554, 5560	<code>\stex_structural_feature_-</code>
<code>\stex_set_language:n</code>	morphism_check_total:
2325	 3304, 3529, 3545, 3583
<code>_stex_set_meta_archive:</code> .. 928, 8178	<code>\stex_structural_feature_-</code>
<code>\stex_sms_allow:N</code>	morphism_end: . 145, 3039, 3139,
144, 2641,	3467, 3482, 3517, 3533, 3549, 3584
2643, 2656, 2657, 2658, 2659, 2660	<code>\stex_structural_feature_-</code>
<code>\stex_sms_allow_env:n</code>	morphism_with_macros:nnnnn ..
144, 2651, 2653, 2661, 3500, 3506,	145, 3039, 3042, 3474, 3509, 3540, 3575
3566, 3572, 7069, 7124, 7174, 7350	<code>\stex_style_apply:</code>
<code>\stex_sms_allow_escape:N</code>	 127, 3, 482, 487, 524,
144, 2646, 2648, 2662, 2663, 2968,	530, 740, 2358, 2363, 2955, 2992,
3525, 3592, 3602, 3677, 3746, 3810,	3028, 3465, 3471, 3476, 3480, 3490,
3841, 3930, 6351, 7276, 7370, 7506	3515, 3531, 3537, 3542, 3547, 3557,
<code>\stex_sms_allow_import:Nn</code>	3581, 3820, 3857, 3922, 4366, 4523,
..... 144, 2664, 2667, 3005	6343, 6382, 7340, 7346, 7725, 7827,
<code>\stex_sms_allow_import_*</code>	7844, 7922, 7943, 7946, 7969, 8003,
144	9149, 9156, 9245, 9247, 9315, 9327,
<code>\stex_sms_allow_import_env:nn</code> ...	
..... 144, 2671, 2673, 2678	

9377, 9387, 9425, 9435, 9468, 9487,
 9602, 9606, 9767, 9771, 10349, 10352
 \stex_suppress_html:n
 128, 699, 699, 2721, 3073, 6415
 \stex_symbol_macro:N
 ... 2284, 2388, 2448, 2460, 2472,
 2506, 2507, 4094, 4103, 4104, 7164
 \stex_symbol_macro:n
 ... 2287, 4130, 4150, 4171, 4275,
 4288, 4294, 7191, 7197, 7522, 7524
 \stex_symbol_uri:n 136
 \stex_symbol_uri_archive:N
 136, 1427, 1427
 \stex_symbol_uri_module:N
 137, 1433, 1433
 \stex_symbol_uri_module:n
 137, 1433, 1436, 4130, 7514, 7522, 7524
 \stex_symbol_uri_name:N
 137, 1443, 1443, 3693, 4642, 6063,
 6080, 6172, 6195, 6355, 9007, 9030
 \stex_symbol_uri_name:n
 137, 1443, 1446, 3188, 4132, 7523, 7524
 \stex_symbol_uri_path:N
 136, 1430, 1430
 \stex_symdecl_do:
 147, 258, 335, 3884,
 3948, 3967, 3967, 4341, 4496, 7412
 \stex_symdecl_html:
 ... 147, 3898, 3951, 4039, 4039, 7419
 \stex_term_arg:nnn
 ... 151, 5526, 5702, 5705, 5711, 5727
 \stex_term_arg:nnnnn
 151, 5694, 5694, 5746,
 5805, 5823, 5913, 6778, 6901, 6909
 \stex_term_arg_aB:nnnnn
 ... 151, 5729, 5743, 6914, 6922, 6939
 \stex_term_do_aB_clist: ... 296,
 5737, 5753, 5807, 5836, 5865, 5923,
 6929, 6930, 6946, 6950, 6971, 6976
 \stex_term_oma:nn
 ... 150, 5419, 5643, 5644, 5659, 6853
 \stex_term_oma_or_omb:nn
 6853, 6858, 6906, 6919
 \stex_term_omb:nn ... 150, 5447,
 5448, 5661, 5662, 5677, 6906, 6919
 \stex_term_oms:nn
 150, 4668, 4670, 5593, 5594,
 5605, 5608, 6044, 6067, 6084, 6752
 \stex_term_oms_or_omv:nn
 4668, 4670, 4698,
 4711, 4863, 4901, 4940, 4996, 5296,
 5298, 5381, 5392, 5402, 5608, 6842
 \stex_term_omv:nn
 150, 5296, 5298, 5316,
 5609, 5610, 5621, 6096, 6110, 6124
 \stex_titlefragment: 8467
 \stex_undefine:N
 . 123, 53, 53, 56, 310, 320, 2100, 2697
 \stex_uri_from_archive_file:Nn . 135
 \stex_uri_from_pair:Nn
 136, 1392, 1392, 2318
 \stex_uri_from_pair:Nnn
 136, 1312, 1313, 1330, 1401, 1405,
 2597, 2939, 2971, 2997, 3010, 3059
 \stex_uri_resolve:Nn 2322
 \stex_uri_use:N 7380
 \stex_use_archive_uri:n
 134, 1117, 1117
 \stex_use_document_uri:N
 134, 1120, 1120, 1237, 1315, 1452,
 1814, 1815, 1817, 1822, 1877, 1882,
 1969, 2077, 2101, 2432, 2699, 8868
 \stex_use_document_uri:n
 134, 1120, 1123
 \stex_use_module_uri:N 135,
 1238, 1238, 1353, 1366, 1381, 1390,
 2339, 2356, 2375, 2382, 2415, 2440,
 2442, 2457, 2504, 2556, 2598, 2830,
 2935, 2948, 2952, 2984, 2989, 3021,
 3025, 3071, 3091, 3489, 3514, 3556,
 3580, 4087, 4116, 6549, 7264, 7271
 \stex_use_module_uri:n 135,
 1238, 1241, 2578, 2583, 2592, 3034,
 4917, 7110, 7152, 7182, 7186, 7288
 \stex_use_notation:nn 154
 \stex_use_notation:nnTF
 154, 4721, 4944, 5353, 5847, 6695, 6695
 \stex_use_op_notation:nnTF
 154, 4708, 5379, 6695, 6701
 \c_stex_use_sref_bool 27
 \stex_use_symbol_uri:N
 136, 1407, 1407, 2050, 3262, 3268,
 3606, 3609, 3681, 3684, 3814, 3827,
 3833, 3845, 3896, 3910, 3913, 3919,
 3965, 4640, 6061, 6078, 6171, 6193,
 6333, 6356, 6643, 6646, 6654, 6655,
 6656, 6657, 6666, 7421, 7428, 7500,
 7549, 7999, 8057, 8082, 9011, 9034
 \stex_use_symbol_uri:n
 136, 1407, 1410, 2539, 3317,
 4647, 5265, 5266, 5268, 5269, 5276,
 5277, 5279, 5280, 6552, 6569, 6572,
 6586, 6587, 6588, 6591, 6592, 6593,
 7228, 7389, 7443, 7462, 7509, 7521
 \stex_var_notation_macro:
 ... 153, 4352, 4511, 6369, 6622, 6622
 \stex_variable:nnnnnnN . 4332, 4477

\l_stex_variables_prop	__stex_expr_add_prop_arg:nnw ...
..... 148, 4330, 4371, 4460, 4529	 5415, 5439, 5442
\stex_with_file_hooks:Nnn	\l__stex_expr_after_tl
..... 125, 246, 249, 249, 2700	.. 5196, 5200, 5201, 5202, 5273, 5284
stex internal commands:	__stex_expr_annotate:nnn
\l_stex_annotate_do_output_bool	 5598, 5614,
... 685, 690, 693, 696, 701, 705, 8177	5630, 5647, 5665, 5681, 5691, 5945
__stex_annotate_env_str ... 667, 668	__stex_expr_arg:n
__stex_check_env_str 6244, 6245, 6246	5416, 5451
__stex_comps_do_defref:nn	\l__stex_expr_arg_counter_int ...
..... 6139, 6145, 6155, 6188	 5405, 5413, 5428, 5463, 5464
__stex_comps_do_ref:nNn .. 6044,	__stex_expr_arg_do:nnn
6065, 6082, 6092, 6104, 6118, 6167	 5465, 5471, 5487, 5522, 5529
__stex_comps_slash:w	__stex_expr_arg_inner:nn
..... 6175, 6199, 6211, 6215	 5454, 5457, 5461, 5467
__stex_comps_split_slash:n	\l__stex_expr_assigned_seq
..... 6047, 6063, 6080	 5005, 5132, 5177, 5244
__stex_comps_split_slash_i:nw ..	__stex_expr_assoc_make_seq:nnn ..
..... 6048, 6051, 6053	 5819, 5846, 5932
__stex_comps_uppercase:n	__stex_expr_assoc_seq:nnnnnnn ..
..... 6072, 6083, 6119, 6156	 5763, 5814
__stex_debug:nn	__stex_expr_check:n
105, 108, 113	5495
\l__stex_debug_cl	__stex_expr_check:nTF ... 5464, 5470
121, 123, 126	__stex_expr_check_b:nn .. 5420, 5445
__stex_debug_env_str . 119, 120, 123	__stex_expr_check_comp:
__stex_doc_check_topsect:	... 5596, 5605, 5612, 5621, 5646,
..... 1698, 1704, 1710, 1716	5659, 5664, 5677, 5705, 5719, 5727
__stex_doc_do_section:n	\l__stex_expr_clist .. 4729, 4736,
..... 1555, 1561, 1565, 1569, 1622	4743, 4747, 4979, 5000, 5006, 5008
\g_stex_doc_ftml_link_text_tl ..	\l__stex_expr_comp_cs
..... 1457, 1460, 1472	 5032, 5037, 5056, 5058
__stex_doc_maketitle: ... 1518, 1523	\l__stex_expr_count_int
__stex_doc_orig_backmatter	.. 5742, 5749, 5758, 5805, 5823, 5913
..... 1759, 1764, 1782	\l__stex_expr_cs
__stex_doc_orig_endbackmatter 1760	.. 4981, 4987, 4994, 5848, 5851, 5871
__stex_doc_orig_endfrontmatter .	__stex_expr_current_type:
..... 1747, 1756	 4935, 5071, 5158
\l__stex_doc_orig_frontmatter ...	\l__stex_expr_current_type_tl ...
..... 1746, 1751, 1771	 4860, 4897, 4938, 4943
__stex_doc_set_title:n .. 1495, 1512	__stex_expr_custom:n 5333, 5343, 5407
__stex_doc_skip_section:	\l__stex_expr_customs_prop
..... 1643, 1649, 1653	 5411, 5423, 5424,
__stex_doc_skip_section_i:	5425, 5440, 5475, 5481, 5496, 5499
..... 1628, 1631, 1634, 1644, 1649	\l__stex_expr_customs_seq
__stex_doc_title_html:	.. 5412, 5429, 5430, 5431, 5441, 5497
..... 1500, 1504, 1515	__stex_expr_do_aB_clist:
\g_stex_doc_title_tl	 5729, 5737, 5807, 5865
..... 1477, 1492, 1499, 1506, 1511	__stex_expr_do_ab_next:nn
\l__stex_doc_titlefragment_bool .	 5419, 5421, 5447, 5448
..... 1606, 1611, 1621, 1626	__stex_expr_do_all:w ... 4778, 4787
__stex_doc_x_matter: ... 1744, 1793	__stex_expr_do_assign:nn 4881, 4885
__stex_expr_aB_arg:nnnnn 5751, 5757	__stex_expr_do_assign_list:n ...
__stex_expr_aB_simple_arg:nnnnn	 4856, 4878
..... 5772, 5803	__stex_expr_do_decl:nnnnnnnnn ..
	 5192, 5220

__stex_expr_do_decl_nomacro:nnnnnnnn	__stex_expr_maybe_notation:w ...
..... 5190, 5206	 4812, 4814, 4942
__stex_expr_do_first_arg:n	__stex_expr_maybe_return:n
..... 4765, 4772	 5360, 5371, 5530
__stex_expr_do_first_next:	__stex_expr_merge:nw ... 4806, 4822
..... 4767, 4774	\l__stex_expr_more_nextsymbol_tl
__stex_expr_do_headterm:nn	 5131, 5133, 5164
..... 5600, 5616, 5624, 5641, 5922	__stex_expr_notation:w
__stex_expr_do_one:w ... 4776, 4782	 4788, 5335, 5336, 5345, 5388
__stex_expr_do_seqmap:nnnnnn ...	\l__stex_expr_old_seq
..... 5769, 5878	 5890, 5893, 5897, 5902
__stex_expr_end:	__stex_expr_op_custom:n
.. 4824, 4828, 4844, 4849, 5765, 5778	 5326, 5343, 5399
\l__stex_expr_field_name_str ...	__stex_expr_op_notation:w
..... 5118, 5122, 5123, 5161	 5328, 5329, 5347, 5377
\l__stex_expr_fields_clist 4857,	__stex_expr_present: ... 4953, 5085
4879, 4886, 4904, 4907, 5235, 5236	__stex_expr_present:nn
\l__stex_expr_first_args_tl	 4957, 4963, 4968, 4975, 4978
..... 4764, 4784, 4788	__stex_expr_present_entry:nn ...
__stex_expr_full_notation:n ...	 4983, 4989, 5004
..... 5349, 5352	__stex_expr_present_i:w
__stex_expr_get_field_name:n ...	 4949, 4955, 4961
..... 5117, 5130	__stex_expr_present_ii:nw 4966, 4974
__stex_expr_get_index_notation:n	\l__stex_expr_prop
..... 4715, 4720, 4783	5120, 5129, 5167, 5175, 5197, 5207,
__stex_expr_gobble:nnnnnnnn ...	5209, 5221, 5223, 5243, 5246, 5252
..... 5765, 5778	__stex_expr_prop_do_decls:
\l__stex_expr_iarg_tl 5858, 5860, 5871	 5179, 5182
__stex_expr_invoke:nnnnN 4644, 4665	__stex_expr_prop_do_notations: .
__stex_expr_invoke_field:n	 5199, 5261
..... 5087, 5127	__stex_expr_range:w ... 4716, 4727
__stex_expr_invoke_maybe_	\l__stex_expr_redo_tl
field:nn	.. 5151, 5178, 5201, 5255, 5272, 5283
5076, 5080	\l__stex_expr_reset_tl
__stex_expr_invoke_this:n 4817, 5062	 4615, 4619, 4623, 4624, 4630
__stex_expr_is_argsep:n 5797	__stex_expr_ret_cs:
__stex_expr_is_seqmap:n 5791	 5534, 5539, 5570, 5575, 5580
__stex_expr_is_seqmap:nTF ... 5767	__stex_expr_return: 5549, 5552
__stex_expr_is_varseq:n 5781	__stex_expr_return_arg:n 5536, 5542
__stex_expr_is_varseq:nTF 5760, 5883	\l__stex_expr_return_args_tl ...
__stex_expr_make_mod:n .. 4903, 4915	.. 5532, 5543, 5548, 5557, 5577, 5582
__stex_expr_make_oml:n .. 4907, 4922	__stex_expr_return_next: 5537, 5546
__stex_expr_make_oml:nn . 4923, 4925	\l__stex_expr_return_this_tl ...
__stex_expr_make_prop:	 5531, 5548,
..... 4947, 5081, 5174	5553, 5557, 5563, 5566, 5576, 5584
__stex_expr_make_prop_assign: ..	\l__stex_expr_seq
..... 4948, 5084, 5234	 5176, 5208, 5222, 5263
__stex_expr_make_prop_assign:nn	__stex_expr_seq_arg:n ... 4730, 4741
..... 5237, 5242	__stex_expr_seq_first: .. 4704, 4760
__stex_expr_make_prop_assign_	__stex_expr_seq_op:w ... 4703, 4707
replace:nnnn	\l__stex_expr_set_comp_tl
5245, 5251	 4791, 4846, 4850, 5010, 5146
__stex_expr_make_type:n	__stex_expr_set_custom_comp:n ..
..... 4866, 4871, 4873, 4889	 4851, 5026, 5044
__stex_expr_math:	
4694, 5322	


```

\__stex_expr_set_custom_i:nn ...
..... 5047, 5052
\__stex_expr_set_customcomp: ...
..... 4824, 4849
\__stex_expr_set_this:n .. 5082, 5092
\__stex_expr_set_thiscomp: 4791, 4795
\__stex_expr_set_thisnotation: ..
..... 4828, 4844
\__stex_expr_setup:nnnnn .....
..... 4667, 4674, 5295
\__stex_expr_struct_top:n . 4792,
4801, 4825, 4829, 4832, 4835, 4837
\__stex_expr_struct_type:n 4808, 4855
\__stex_expr_text: ..... 4694, 5341
\__stex_expr_varseq_in_map:nnnnnnnn
..... 5885, 5931
\__stex_groups_do: .... 340, 346, 354
\__stex_groups_do_in:n .... 329, 333
\l__stex_groups_lvl_int 323, 325, 328
\l__stex_import_archive_str ....
..... 2852, 2855, 2859, 2927
\__stex_import_check_file:nnn ...
..... 2871,
2872, 2873, 2887, 2888, 2889, 2916
\l__stex_import_doc_uri .. 2926, 2933
\l__stex_import_file_str .....
..... 2839, 2869,
2899, 2901, 2904, 2909, 2920, 2933
\__stex_import_import_module:nn .
..... 2962, 2970, 3007
\__stex_import_import_module_i:n
..... 2973, 2976
\__stex_import_import_module_-
presms:nn ..... 2996, 3007
\l__stex_import_language_str ...
..... 2870, 2874, 2890, 2930
\__stex_import_load_check:n ....
..... 2857, 2863, 2868
\__stex_import_load_check_i:n ...
..... 2876, 2883
\__stex_import_load_check_ii: ...
..... 2880, 2898
\__stex_import_load_file: .....
..... 2905, 2910, 2925
\__stex_import_load_module:NTF ..
..... 2829, 2838, 2850
\l__stex_import_name_str .....
.. 2854, 2871, 2872, 2873, 2885, 2929
\l__stex_import_path_str .....
.. 2853, 2856, 2861, 2884, 2886, 2928
\l__stex_import_pre_str .....
..... 2856, 2860, 2917, 2918, 2920
\__stex_import_requiremodule: ...
..... 3012, 3017

\l__stex_import_uri .....
..... 2846, 2847, 2851, 2934, 2935
\__stex_import_usemodule: 2941, 2944
\l__stex_inputs_gin_repo_str ...
..... 2266, 2269, 2274
\l__stex_inputs_id_seq .....
..... 2203, 2204, 2209, 2217
\l__stex_inputs_libinput_files_-
seq ..... 2132, 2133, 2136,
2142, 2143, 2164, 2165, 2189, 2190,
2193, 2201, 2213, 2214, 2224, 2225
\l__stex_inputs_path_seq .....
.. 2202, 2205, 2207, 2210, 2218, 2221
\l__stex_inputs_path_str .....
2210, 2211, 2212, 2213, 2214, 2217,
2218, 2221, 2222, 2223, 2224, 2225
\__stex_inputs_ref:n 2064, 2085, 2095
\__stex_inputs_ref_i:n .....
..... 2071, 2087, 2090
\c__stex_inputs_ref_post_str ...
..... 2070, 2078
\c__stex_inputs_ref_pre_str ....
..... 2069, 2076
\l__stex_inputs_tmp_str .. 2143, 2145
\__stex_inputs_up_archive:nn ...
.. 2131, 2141, 2163, 2188, 2197, 2197
\l__stex_inputs_uri .. 2063, 2077,
2082, 2101, 2102, 2108, 2110, 2113
\__stex_inputs_usetikzlibrary:n .
..... 2156, 2158, 2162
\__stex_inputs_usetikzlibrary_-
i:nn ..... 2165, 2171
\__stex_keys_split_at_bracket:w .
..... 414, 422
\l__stex_keys_tl .....
..... 364, 365, 366, 368, 371, 372
\l__stex_lang_turkish_bool .....
..... 455, 463, 473
\__stex_mathhub: ..... 1006, 1030
\l__stex_mathhub_base_str .....
.. 1014, 1024, 1029, 1030, 1032, 1036
\l__stex_mathhub_bool .....
.... 972, 973, 975, 978, 987, 989, 998
\__stex_mathhub_check_manifest: .
..... 977, 985
\__stex_mathhub_check_manifest:n
..... 986, 988, 990, 995
\l__stex_mathhub_cs .....
.... 900, 903, 905, 909, 913, 914, 919
\__stex_mathhub_do_manifest:nn ..
..... 932, 950, 957, 1041
\l__stex_mathhub_file_str 1012, 1032
\__stex_mathhub_find_manifest:n 962

```

```

\__stex_mathhub_find_manifest:nn
..... 959, 968, 1065
\l__stex_mathhub_id_str .....
..... 1013, 1023, 1032, 1036
\l__stex_mathhub_key 1019, 1020, 1022
\c__stex_mathhub_manifest_ior ...
..... 1002, 1011, 1016, 1028
\l__stex_mathhub_manifest_str ...
..... 960, 963, 969, 999, 1011, 1066
\__stex_mathhub_parse_manifest:n
..... 964, 1010, 1069
\__stex_mathhub_replace_https:w .
..... 1006, 1030
\__stex_mathhub_require:n .. 921, 944
\__stex_mathhub_restore:nn .....
..... 1036, 1040, 1079
\l__stex_mathhub_seq 971, 974, 979,
996, 997, 999, 1012, 1018, 1019, 1021
\__stex_mathhub_set_current:n ...
..... 908, 920
\l__stex_mathhub_str .....
..... 838, 839, 843, 844,
849, 855, 858, 865, 867, 868, 869, 874
\l__stex_mathhub_tl ..... 979
\l__stex_mathhub_val 1021, 1023, 1024
\__stex_modules_export:n . 2608, 2610
\__stex_modules_html_annots: ...
..... 2333, 2354
\__stex_modules_load_meta: .....
..... 2376, 2394, 2396
\__stex_modules_load_meta_i:n ...
..... 2398, 2402
\__stex_modules_load_sig: 2421, 2426
\__stex_modules_macro_short:nnn .
..... 2281, 2285, 2288,
2291, 2294, 2297, 2300, 2303, 2306
\__stex_modules_nested:n .....
..... 2367, 2438, 2438
\__stex_modules_persist_module: .
..... 2455, 2464, 8173
\__stex_modules_persist_notst_-
i:nn ..... 2480, 2485
\__stex_modules_restore_module:nnnn
..... 2466, 2502
\__stex_modules_restore_notst:n ..
..... 2516, 2521
\__stex_modules_restore_notst_i:n
..... 2522, 2525, 2544
\__stex_modules_restore_notst_-
ii:nnnn ..... 2529, 2533
\l__stex_modules_sig 2428, 2432, 2433
\l__stex_modules_sigfile .....
..... 2427, 2432, 2434

\__stex_modules_stringify_-
uri:nnn ..... 2490, 2503
\__stex_modules_stringify_-
uri:nnnn ..... 2495, 2540
\l__stex_modules_tl . 2534, 2536, 2541
\__stex_modules_top:n ... 2367, 2370
\__stex_modules_top_sig:n .....
..... 2372, 2411, 2411
\l__stex_modules_uri . 2381, 2382,
2383, 2386, 2387, 2388, 2389, 2391,
2392, 2414, 2415, 2416, 2418, 2423,
2436, 2441, 2442, 2443, 2446, 2447,
2448, 2449, 2451, 2452, 2503, 2504,
2505, 2506, 2507, 2511, 2512, 2538
\__stex_morphisms_add_definiens:nn
..... 3274, 3279, 3338
\__stex_morphisms_add_symbol:nnnnnnnnN
..... 3191, 3207, 3209, 3214
\__stex_morphisms_add_symbol_-
i:nnnnnnnnN ..... 3221, 3224
\l__stex_morphisms_ass_tl .....
.. 3624, 3640, 3644, 3655, 3656, 3657
\__stex_morphisms_begin_copy:Nnnn
..... 3462, 3474, 3485
\__stex_morphisms_begin_interpret:Nnnn
..... 3527, 3540, 3552
\__stex_morphisms_break: . 3282, 3286
\__stex_morphisms_check_name:nnnn
..... 3402, 3405
\l__stex_morphisms_continue_bool
..... 3343, 3346, 3359, 3378
\__stex_morphisms_cs:nnnn 3344, 3369
\__stex_morphisms_definiens_-
impl:nn ..... 3254, 3339
\__stex_morphisms_do:n .....
..... 3109, 3121, 3135
\__stex_morphisms_do_decls: ....
..... 3107, 3112
\l__stex_morphisms_do_dones_seq .
..... 3108, 3123, 3124
\__stex_morphisms_do_elaboration:
..... 3149, 3152
\__stex_morphisms_do_for_list: ..
..... 3244, 3337
\__stex_morphisms_do_morph:nnnn .
..... 3126, 3131
\__stex_morphisms_do_morph_-
assign:nnn ..... 3705, 3708, 3749
\__stex_morphisms_do_parsed_-
assign: ..... 3646, 3649
\__stex_morphisms_do_parsed_-
newname: ..... 3653, 3661
\__stex_morphisms_do_parsed_-
newname:w ..... 3663, 3665, 3669

```

<code>__stex_morphisms_elab:nn</code>	3164, 3182	<code>__stex_morphisms_set_definiens_</code>	
<code>__stex_morphisms_elab_check_</code>		<code> macros_i:nnnnnnn</code>	 3289, 3294
<code> assign:nnnnn</code>	 3217, 3228	<code>__stex_morphisms_setup:</code>	.. 3096, 3102
<code>__stex_morphisms_elab_check_</code>		<code>__stex_morphisms_split_qm:w</code>	.. 3322
<code> rename:nnn</code>	 3185, 3236	<code>\l__stex_morphisms_tmp</code>	 3183,
<code>__stex_morphisms_elab_i:nnn</code>	...		3187, 3190, 3191, 3197, 3204, 3239
	 3188, 3203	<code>\l__stex_morphisms_tmp_b</code>	
<code>__stex_morphisms_end:</code>	... 3653, 3669		 3215, 3219, 3231
<code>\l__stex_morphisms_feature_str</code>	..	<code>\l__stex_morphisms_todo_tl</code>	
	 3085, 3170		 3348, 3352, 3365, 3378
<code>__stex_morphisms_get_check:nn</code>	..	<code>__stex_morphisms_total_check:nn</code>	
	 3386, 3401, 3447, 3455		 3306, 3310
<code>\l__stex_morphisms_get_str</code>		<code>__stex_morphisms_total_check:nnnnnnnN</code>	
	 3384, 3407,		 3311, 3314
	3428, 3438, 3443, 3445, 3451, 3453	<code>\l__stex_morphisms_with_macros_</code>	
<code>__stex_morphisms_gobble:nnnnnnN</code>		<code> bool</code>	... 3040, 3043, 3047, 3145, 3206
	 3431, 3435	<code>__stex_notations_activate_</code>	
<code>__stex_morphisms_iterate:nn</code>	...	<code> not:nn</code>	 6555, 6567
	 3349, 3353, 3362, 3364	<code>__stex_notations_add:nnnnn</code>	
<code>__stex_morphisms_macro:nnnnnnnN</code>			 6886, 6891, 6896
	 3411, 3427	<code>__stex_notations_add_last:nnnnn</code>	
<code>\l__stex_morphisms_mods_seq</code>			 6879, 6890
	 3342, 3366, 3367	<code>__stex_notations_add_missing_</code>	
<code>__stex_morphisms_morphism:nnnnn</code>		<code> args:nn</code>	 6521, 6524
	 3044, 3048, 3051	<code>__stex_notations_add_next:nnnnnnn</code>	
<code>\l__stex_morphisms_morphism_dom_</code>			 6881, 6885
<code> str</code>	... 3699, 3712, 3724, 3734, 3751	<code>\l__stex_notations_after_tl</code>	
<code>\l__stex_morphisms_name_str</code>			 6866, 6893
	 3622, 3631, 3632, 3633,	<code>__stex_notations_args_end:</code>	
	3637, 3638, 3639, 3650, 3652, 3656		 6600, 6603, 6606,
<code>\l__stex_morphisms_newname_str</code>	..		6613, 6616, 6619, 6874, 6877, 6887
	.. 3623, 3642, 3643, 3651, 3652, 3653	<code>\l__stex_notations_args_tl</code>	
<code>\l__stex_morphisms_next_tl</code>			 6785, 6787, 6800
	 3629, 3634, 3636	<code>__stex_notations_augment_arg:nn</code>	
<code>__stex_morphisms_parse_assign:n</code>			 6801, 6814
	 3511, 3577, 3621	<code>\l__stex_notations_bind_bool</code>	...
<code>__stex_morphisms_reactivate:</code>	...		 6320, 6322
	 3097, 3326	<code>__stex_notations_check_aB_</code>	
<code>__stex_morphisms_rename:n</code>	3171, 3174	<code> arg:Nn</code>	 6928, 6937, 6944, 6969
<code>__stex_morphisms_rename:nn</code>		<code>\l__stex_notations_clist_count_</code>	
	 3175, 3178	<code> int</code>	 6967, 6975, 6981
<code>__stex_morphisms_rename_all:</code>	.. 3299	<code>\l__stex_notations_code_tl</code>	
<code>__stex_morphisms_renamed_</code>			 6824, 6839, 6854, 6868,
<code> check:nn</code>	 3390, 3437		6869, 6892, 6900, 6908, 6913, 6921
<code>__stex_morphisms_renamed_</code>		<code>__stex_notations_complex:nnnnnnn</code>	
<code> check:nnnnnn</code>	 3439, 3442		 6833, 6850
<code>\l__stex_morphisms_seq</code>		<code>__stex_notations_const_precs:</code>	..
	 3626, 3627, 3629, 3631,		 6394, 6434
	3634, 3636, 3638, 3640, 3642, 3644	<code>\l__stex_notations_cs</code>	
<code>__stex_morphisms_set:nnnnnnN</code>	...		 6840, 6845, 6855, 6864
	 3409, 3415, 3430	<code>__stex_notations_default_args:</code>	..
<code>__stex_morphisms_set_definiens_</code>			 6786, 6797
<code> macros:</code>	 3282, 3286	<code>__stex_notations_do_argname:nn</code>	..
			 6495, 6498

<code>__stex_notations_do_argnames: ..</code>	<code>__stex_notations_process:nnnnnn</code>
..... 6470, 6493	 6823, 6829
<code>__stex_notations_do_missing_-</code>	<code>\l__stex_notations_right_-</code>
<code>args:n</code>	<code>bracket_str .</code>
..... 6400, 6513	 6999, 7031, 7037, 7041
<code>__stex_notations_fun_precs: ..</code>	<code>\l__stex_notations_seq</code>
..... 6399, 6446	 6475, 6476, 6478, 6479, 6486
<code>__stex_notations_it_not_-</code>	<code>__stex_notations_set_macro:nnnnn</code>
<code>check:nnnn</code>	 6554, 6563, 6576, 6585
..... 6686, 6690	<code>__stex_notations_simple:nnnnn ..</code>
<code>__stex_notations_it_not_i:n ..</code>	 6831, 6837
..... 6675, 6679, 6692	<code>\l__stex_notations_str</code>
<code>\l__stex_notations_left_bracket_-</code>	 6476, 6477, 6478, 6480, 6486, 6487
<code>str</code>	<code>__stex_notations_styledefs: ..</code>
..... 6998, 7026, 7036, 7040	 6342, 6353
<code>__stex_notations_macro:nn</code>	<code>__stex_others_newlabel:n</code>
..... 6538, 6586, 6587, 6588, 6623, 6624,	 8295, 8303
..... 6625, 6643, 6654, 6655, 6696, 6697	<code>__stex_others_old_newlabel: ..</code>
<code>__stex_notations_make_arg:nnnn .</code>	 8297, 8302
..... 6764, 6767	<code>__stex_path_:</code>
<code>__stex_notations_make_arg_-</code>	 148, 158
<code>html:nn</code>	<code>\l__stex_path_a_seq ..</code>
..... 6744, 6760	 280, 286, 292
<code>__stex_notations_make_name: ..</code>	<code>\l__stex_path_a_tl</code>
..... 6793, 6798, 6818	 292, 294
<code>__stex_notations_map_args_i:w ..</code>	<code>\l__stex_path_b_seq</code>
..... 6599, 6603, 6606	 281, 287, 293, 299
<code>__stex_notations_map_args_ii:w .</code>	<code>\l__stex_path_b_tl</code>
..... 6612, 6616, 6619	 293, 294
<code>__stex_notations_map_cs:</code>	<code>\l__stex_path_can_seq</code>
..... 6729, 6731,	 180, 183, 188, 196, 197, 200
..... 6947, 6949, 6957, 6972, 6974, 6985	<code>\l__stex_path_can_str</code>
<code>\l__stex_notations_missing_str ..</code>	 179, 181, 185, 197
..... 6514, 6515, 6516, 6518, 6526	<code>__stex_path_canonicalize:N</code>
<code>\l__stex_notations_missing_tl ..</code>	 161, 172, 177
..... 6429, 6520, 6527	<code>__stex_path_dodots:n .</code>
<code>\l__stex_notations_mods_seq</code>	 182, 186, 192
..... 6671, 6680, 6681	<code>\l__stex_path_return_tl</code>
<code>\l__stex_notations_name_str</code>	 282, 288, 295, 298, 300
..... 6794, 6820	<code>\l__stex_path_seq</code>
<code>__stex_notations_not_cs:nnnnn ..</code>	 213, 218, 219, 221, 223, 226, 251, 252
..... 6672, 6673, 6683	<code>\l__stex_path_str</code>
<code>__stex_notations_op_macro:nn ..</code>	 149, 150, 151, 154,
..... 6541, 6591, 6592, 6593, 6629, 6630,	 156, 157, 158, 160, 163, 170, 171,
..... 6631, 6646, 6656, 6657, 6702, 6703	 210, 211, 212, 217, 218, 219, 221,
<code>\l__stex_notations_opprec_tl</code>	 222, 223, 229, 231, 233, 239, 241, 243
..... 6423,	<code>\l__stex_path_win_drive</code>
..... 6436, 6439, 6441, 6448, 6451, 6453,	 150, 155, 162, 164
..... 6457, 6464, 6477, 6483, 6489, 6720	<code>__stex_path_win_take:w</code>
<code>__stex_notations_parse_args:nnnnw</code>	 148, 158
..... 6874, 6877, 6887	<code>__stex_persist_env_str</code>
<code>__stex_notations_parse_precs: ..</code>	 568, 569, 570, 574, 575, 576
..... 6468, 6473	<code>__stex_persist_load_file:n</code>
<code>\l__stex_notations_pre_tl</code>	 589, 592, 594, 604, 611, 645
..... 6853, 6868, 6905, 6918	<code>__stex_persist_read_and_write: .</code>
<code>\l__stex_notations_precs_seq ..</code>	 602, 630
..... 6391, 6459,	<code>\c__stex_persist_sms_iow</code>
..... 6466, 6487, 6489, 6501, 6507, 6721	 585, 626, 627, 642, 730
	<code>__stex_persist_write_only:</code>
	 607, 625, 639, 646
	<code>__stex_proof_add_counter: 7683, 7942</code>
	<code>__stex_proof_begin_proof:nn ..</code>
	 7709, 7819, 7841

<code>\l_stex_proof_counter_intarray .</code>	<code>_stex_proof_set_after_subproof_-</code>
..... 7640, 7646,	with_number:n 7884, 7934
7649, 7658, 7661, 7670, 7678, 7679,	<code>_stex_proof_set_after_subproof_-</code>
7687, 7692, 7699, 7705, 7710, 7711	with_number_i:n . 7886, 7887, 7894
<code>\l_stex_proof_counter_str</code>	<code>_stex_proof_set_aftergroup: . . .</code>
... 7598, 7599, 7609, 7762, 7777,	 7892, 7897, 7899
7877, 7895, 7896, 8050, 8065, 8090	<code>_stex_proof_set_aftergroup_-</code>
<code>_stex_proof_do_after_subproof:N</code>	double: 7892
..... 7866, 7900	<code>_stex_proof_start_comment: . . .</code>
<code>_stex_proof_do_after_subproof_-</code>	 7801, 7814,
i:nn 7868, 7872	7822, 7957, 7979, 8039, 8105, 8126
<code>_stex_proof_do_spf_name: 7589, 8022</code>	<code>_stex_proof_start_list:n</code>
<code>_stex_proof_do_spf_name_i: . . .</code>	 7821, 7851, 7941, 8035, 8123
..... 7603, 7870	<code>_stex_proof_step_html:nn</code>
<code>_stex_proof_do_spf_varname: . . .</code>	 8035, 8048, 8123
..... 7613, 7941, 8112, 8113	<code>_stex_proof_step_html_i:nn . . .</code>
<code>_stex_proof_end_comment:</code>	 8033, 8041, 8075, 8121, 8128
..... 7807, 7814,	<code>_stex_refs_check_i:nnnn 1901, 1909</code>
7835, 7975, 7983, 8017, 8110, 8116	<code>_stex_refs_check_in:nnnn 2000, 2009</code>
<code>_stex_proof_end_list:</code>	<code>\l_stex_refs_default_archive . . .</code>
.. 7825, 7860, 7966, 7970, 8035, 8123	 1928, 2028, 2034, 2036, 2038
<code>_stex_proof_html:</code>	<code>\l_stex_refs_default_file</code>
..... 7757, 7780, 7783, 8068, 8093	.. 1923, 1927, 1931, 1933, 2027, 2039
<code>_stex_proof_html_env_proof: . . .</code>	<code>\l_stex_refs_default_relpath . . .</code>
..... 7715, 7747	 1929, 2029, 2032, 2039
<code>_stex_proof_html_env_subproof:</code>	<code>\l_stex_refs_default_title</code>
..... 7760, 7908	 1926, 2026, 2041
<code>\l_stex_proof_in_spfblock_bool .</code>	<code>\l_stex_refs_file</code>
..... 7707, 7802,	... 1883, 1884, 1902, 1927, 1936,
7808, 7820, 7842, 7892, 7920, 7932,	1944, 1953, 1956, 1994, 1997, 2001
7959, 7966, 7970, 8031, 8103, 8119	<code>_stex_refs_find: 1885, 1899</code>
<code>_stex_proof_inc_counter:</code>	<code>_stex_refs_find_in: . . . 1954, 1993</code>
..... 7666, 7961, 8112, 8113	<code>\l_stex_refs_in</code>
<code>\l_stex_proof_inc_counter_bool 7639</code>	.. 1874, 1919, 1942, 1946, 1968, 1969
<code>_stex_proof_insert_number: . . .</code>	<code>_stex_refs_in: 1937, 1947, 1951</code>
..... 7642, 7941, 8112, 8113	<code>_stex_refs_in_text:nn . . 1964, 1979</code>
<code>\l_stex_proof_key_name_str</code>	<code>_stex_refs_in_text:w . . . 1980, 1987</code>
..... 7605, 7606, 7874	<code>\c_stex_refs_iow</code>
<code>\l_stex_proof_key_numname_str . .</code>	 1796, 1797, 1798, 1819
..... 7604, 7609, 7875	<code>\c_stex_refs_iow_str</code>
<code>\l_stex_proof_key_showname_bool</code>	 1799, 1931, 1933, 1994
..... 7876	<code>\l_stex_refs_key 1873, 1910</code>
<code>_stex_proof_make_step_macro:Nnnnn</code>	<code>_stex_refs_local:n 1844, 1850, 1880</code>
..... 8015, 8112, 8113, 8114	<code>_stex_refs_local_ref:n</code>
<code>_stex_proof_number_as_string:N</code>	 1854, 1861, 1921, 1924, 1934
..... 7598, 7653,	<code>_stex_refs_maybe_in: . . . 1893, 1918</code>
7762, 7895, 7924, 7937, 8026, 8050	<code>\l_stex_refs_new_tl</code>
<code>_stex_proof_proof_box_tl 7620, 8157</code>	 1813, 1814, 1815, 1817, 1822
<code>_stex_proof_remove_counter: . . .</code>	<code>_stex_refs_remote:nn . . . 1846, 1872</code>
..... 7695, 7960	<code>_stex_refs_stop: 1980, 1987</code>
<code>_stex_proof_set_after_subproof:n</code>	<code>\l_stex_refs_str 7437</code>
..... 7879, 7921, 7948	<code>\l_stex_refs_tmp 1842, 1887,</code>
<code>_stex_proof_set_after_subproof_-</code>	1891, 1905, 1912, 1952, 1958, 1962,
i:n 7881, 7882, 7889	1963, 1964, 1970, 1974, 2004, 2011

\l__stex_refs_unnamed_counter_- int 2015	__stex_smsmode_start_smsmode:n 2705, 2712, 2731
__stex_refs_uppercase:n . 1988, 1991	__stex_statements_add_definiens:n 7515, 7520
\l__stex_refs_uri 1876, 1877, 1878, 1882, 1891, 1892, 1911, 1920, 1921, 1924, 1934, 1969, 1974, 1997, 2010	__stex_statements_definiens_- inner:nnnnnnnN 7524, 7528
__stex_seqs_add: 4508, 4527	__stex_statements_force_id: 7406, 7434, 7454, 7478
\l__stex_seqs_args_tl ... 4587, 4589	__stex_statements_html_keyvals:nn 7328, 7362, 7373
\l__stex_seqs_cs 4585, 4589	__stex_statements_make_macro:nnn 7352, 7355
__stex_seqs_html: 4506, 4554	__stex_statements_setup:n <u>7391</u> , 7391, 7455, 7459, 7474, 7479, 7482
__stex_seqs_macro: 4509, 4541	__stex_statements_setup_def: 7440, 7440, 7456, 7480
\l__stex_seqs_range_clist 4499, 4535, 4548, 4574	__stex_statements_setup_named:n 7401, 7405
\g__stex_smsmode_allowed_escape_- tl 2646, 2649, 2776	__stex_statements_setup_noname: 7400, 7424
\g__stex_smsmode_allowed_import_- env_seq 2671, 2674, 2799, 2802	__stex_structures_begin:nn 7062, 7072, 7106, 7147
\g__stex_smsmode_allowed_import_- tl 2664, 2668, 2794	__stex_structures_check_- def:nnnnnnnn 7165, 7208
\g__stex_smsmode_allowed_tl 2641, 2644, 2772	__stex_structures_do_externals: 7066, 7092, 7121, 7171
\g__stex_smsmode_allowedenvs_seq 2651, 2654, 2781, 2784	__stex_structures_do_usestructure: 7265, 7272, 7278
\g__stex_smsmode_bool 2680, 2683, 2723	\l__stex_structures_exstruct_- name_str ... 7144, 7147, 7169, 7218
__stex_smsmode_check_begin:Nn 2781, 2799, 2810	__stex_structures_extend_- i:nnnnnnnn 7190, 7204
__stex_smsmode_check_cs:N 2745, 2751	__stex_structures_extend_- ii:nnnn 7189, 7196
__stex_smsmode_check_cs:NNn 2744, 2758	__stex_structures_extend_- structure:nnnn 7144, 7181
__stex_smsmode_check_end:Nn 2784, 2802, 2818	\l__stex_structures_extname_- count 7216, 7218, 7221
\l__stex_smsmode_cycles_seq 2686, 2692, 2694	__stex_structures_get:w . 7250, 7260
__stex_smsmode_do:w 2740, 2742, 2744, 2747, 2755, 2774, 2786, 2804, 2815, 2821, 2823	\l__stex_structures_imports_seq 7097, 7102, 7107, 7132, 7137, 7149, 7279, 7282, 7285
__stex_smsmode_do_aux:N 2744, 2754, 2765	\l__stex_structures_last_name_- str 7141, 7144
__stex_smsmode_do_aux_curr:N 2703, 2711, 2767	\l__stex_structures_name_str 7050, 7055, 7057, 7074, 7075, 7080, 7084, 7089, 7229, 7232
__stex_smsmode_do_aux_imports:N 2703, 2793	__stex_structures_new_extstruct_- name: 7131, 7214
__stex_smsmode_do_aux_normal:N 2711, 2771	\l__stex_structures_parent_uri 7141, 7144
\g__stex_smsmode_import_setup_tl 2665, 2669, 2675, 2704	\l__stex_structures_replace_- this_tl 7093
\l__stex_smsmode_importmodules_- seq 2689	
__stex_smsmode_in_smsmode:n 2701, 2720	
\l__stex_smsmode_sigmodules_seq 2690	
__stex_smsmode_smsmode_do: 2733, 2740	

\l__stex_structures_struct_uri ..	__stex_syms_style:
..... 7079, 7084	3804, 3812
__stex_styles_apply_patch:n 488, 503	__stex_syms_sym_cs:nnnnnnnnN ...
\g__stex_styles_counters_seq ...	.. 4140, 4141, 4151, 4158, 4159, 4172
..... 24, 63, 64, 761, 800, 801	__stex_syms_sym_from_str_i:nnnn
__stex_styles_css_patch:nnnn ...	 4242, 4276, 4289, 4295
..... 16, 97, 521, 753, 834	__stex_syms_sym_i_finish:nnnnnnN
\l__stex_styles_first_str	 4248, 4255
..... 60, 62, 81, 797, 799, 818	__stex_syms_sym_i_gobble:nnnnnn
__stex_styles_patch:nnn ... 484, 495	 4250, 4253
__stex_styles_patch:nnnn	__stex_syms_top:n
..... 4, 88, 518, 741, 825	 3803, 3826, 3940, 3940
__stex_styles_patch_html:	__stex_syms_uri_match:n
..... 34, 74, 77, 771, 811, 814	4303
__stex_styles_patch_html_c: ...	__stex_uris_absolute:N
..... 26, 66, 69, 763, 803, 806	 1324, 1355, 1359
__stex_styles_patch_i:nnn	\l__stex_uris_archive 1173, 1177, 1180
..... 21, 42, 758, 779	__stex_uris_as_qm:nnn ... 1286, 1288
__stex_syms_activate_i:nnnnnnnn	__stex_uris_doc:nnnnn
..... 4098, 4111, 4115	 1121, 1124, 1126
__stex_syms_add_decl: ... 3950, 3953	__stex_uris_doc_from_archive_-
\l__stex_syms_args_tl ... 4071, 4073	file:NN ... 1167, 1170, 1229, 1231
\l__stex_syms_cs	__stex_uris_end: ... 1397, 1400, 1404
.. 4069, 4073, 4193, 4195, 4197, 4225	\l__stex_uris_file .. 1165, 1166, 1167
__stex_syms_def_style: .. 3835, 3843	__stex_uris_first_three:nnnn ...
__stex_syms_do_args: ... 3970, 3977	 1434, 1437, 1440
__stex_syms_get_from_one_-	\l__stex_uris_lang_str 1183,
string:n 4234, 4308	1192, 1194, 1195, 1200, 1201, 1203,
__stex_syms_get_symbol_from_cs:	1204, 1205, 1206, 1208, 1209, 1210
..... 4199, 4214	__stex_uris_maybe_relative:N ...
__stex_syms_get_symbol_from_-	 1324, 1332
modules:nn 4236, 4267	__stex_uris_module:nnn
__stex_syms_get_symbol_from_-	 1239, 1242, 1244
string:n ... 4200, 4202, 4205, 4229	\l__stex_uris_name_str
__stex_syms_has_definiens:nnnnnnN	... 1182, 1190, 1191, 1194, 1215,
..... 4129, 4136	1317, 1342, 1348, 1364, 1379, 1388
__stex_syms_it_decl_check:nnnn .	__stex_uris_new_mod:nnnnnn
..... 4169, 4177	 1295, 1299
__stex_syms_it_decl_i:n	__stex_uris_new_nested_mod:nnnn
..... 4161, 4165, 4179	 1293, 1307
__stex_syms_maybe_activate:nnnnnnnn	__stex_uris_pair_in_archive:Nn .
..... 4105, 4109	 1327, 1384
\l__stex_syms_mods_seq	__stex_uris_pair_no_archive:N ..
..... 4157, 4166, 4167	 1368, 1372
\l__stex_syms_name 4232, 4238	__stex_uris_parent:NNnnn
\l__stex_syms_name_str	 1268, 1272, 1275
..... 4269, 4281, 4283	__stex_uris_parse_i:Nw .. 1394, 1400
__stex_syms_parse_arity: 3969, 3988	__stex_uris_parse_ii:Nw . 1396, 1404
\l__stex_syms_path_str	\l__stex_uris_path_seq
..... 4270, 4280, 4287, 4292, 4293	 1181, 1189, 1190
\l__stex_syms_seq	\l__stex_uris_path_str
..... 4231, 4232, 4233, 4237	.. 1318, 1320, 1323, 1363, 1374, 1387
__stex_syms_set_textsymdecl_-	__stex_uris_split_file:N 1171, 1188
macro:nnn 3913, 3932	__stex_uris_split_file_i: 1196, 1199
	__stex_uris_split_file_ii: .. 1218

<code>__stex_uris_symbol:nnnn</code>	<code>\stexstylenotation</code>	106
..... 1408, 1411, 1413	<code>\stexstyleparagraph</code>	106, 127
<code>__stex_uris_with_elem:nnnnnn</code> ...	<code>\stexstyleproblem</code>	10068, 10109
..... 1159, 1161	<code>\stexstyleproof</code>	106
<code>__stex_uris_with_language:nnnnnn</code>	<code>\stexstylerealization</code>	106
..... 1153, 1155	<code>\stexstylerealize</code>	106
<code>__stex_vars_add:</code>	<code>\stexstylerenamedekl</code>	106
<code>\l__stex_vars_args_tl</code> ...	<code>\stexstylerequiremodule</code>	106
<code>\l__stex_vars_bind_bool</code> ..	<code>\stexstylescb</code>	9774, 9778
<code>__stex_vars_check_var:nnnnnnnnN</code>	<code>\stexstylespfsketch</code>	106
..... 4461, 4465	<code>\stexstylesubproblem</code>	10151, 10191
<code>\l__stex_vars_cs</code>	<code>\stexstylesubproof</code>	106
<code>__stex_vars_get_var:n</code> ...	<code>\stexstylesymdecl</code>	106
<code>__stex_vars_html:</code>	<code>\stexstylesymdef</code>	106
<code>__stex_vars_macro:</code>	<code>\stexstyletextsymbdecl</code>	106
<code>__stex_vars_set_vars:nnnnnnN</code> ...	<code>\stexstyleusemodule</code>	106
..... 4467, 4470, 4475	<code>\stexstylevardef</code>	106
<code>stex_annotate_env (env.)</code>	<code>\stexstylevarnotation</code>	106
<code>stex_env_node (env.)</code>	<code>\stexstylevarseq</code>	106
<code>\stexcommentfont</code>	<code>\stopsolutions</code>	114, 9347
<code>\stexdoctitle</code>	str commands:	
138, 1477, 1580, 1592, 1618, 2350,	<code>\c_ampersand_str</code> .	1129, 1131, 1132,
8659, 8843, 9130, 9230, 9233, 10346	1134, 1247, 1249, 1416, 1418, 1419	
<code>\STEXexport</code> ...	<code>\c_backslash_str</code>	156, 1866
<code>\STEXftmlink</code>	<code>\c_colon_str</code>	1018
<code>\stexhtmlfalse</code>	<code>\c_dollar_str</code>	6508
<code>\stexhtmltrue</code>	<code>\c_hash_str</code>	6515, 6526, 6547
<code>\STEXInternalNewSRefLabel</code> 1815, 1817, 2022	<code>\c_left_brace_str</code>	10088,
<code>\STEXInternalNotation</code> 154, 6421, 6787, 6822	10130, 10177, 10218, 10238, 10263	
<code>\STEXInternalSRefLabel</code>	<code>\c_percent_str</code>	89, 90, 239
..... 1820, 1901, 2000, 2020	<code>\c_right_brace_str</code>	10105,
<code>\STEXinvisible</code> ...	10147, 10187, 10228, 10244, 10267	
7720, 7722, 7913, 7915, 8196, 8197	<code>\str_case:Nn</code>	1022
<code>\STEXlinkftml</code>	<code>\str_case:nn</code>	5446, 6898
<code>\STEXRestoreNotsEnd</code> ...	<code>\str_case:nnTF</code> 48, 785, 1677, 3992,	
<code>\STEXsetlinkftml</code>	5473, 5501, 6768, 8419, 8997, 9020	
<code>\stexstyle<environmentname></code>	<code>\str_clear:N</code>	
<code>\stexstyle<macroname></code>	... 6, 7, 8, 91, 155, 379, 380, 387,	
<code>\stexstyleassertion</code>	405, 743, 744, 745, 849, 969, 1318,	
<code>\stexstyleassign</code>	2269, 2312, 2869, 3622, 3623, 3699,	
<code>\stexstyleassignMorphism</code>	3764, 3765, 3766, 3767, 3777, 3778,	
<code>\stexstylecopymod</code>	3782, 3799, 3861, 3903, 3904, 4452,	
<code>\stexstylecopymodule</code>	6297, 6298, 6299, 6300, 6516, 7050,	
<code>\stexstyledefinition</code>	7295, 7296, 7298, 7580, 7581, 7654,	
<code>\stexstyleexample</code>	8011, 8553, 8899, 8900, 9064, 9065,	
<code>\stexstyleextstructure</code>	9269, 9286, 9507, 10474, 10475, 10476	
<code>\stexstylegnote</code>	<code>\str_const:Nn</code>	1051, 8338
<code>\stexstyleimportmodule</code>	<code>\str_count:n</code>	59, 64
<code>\stexstyleinterpretmod</code>	<code>\str_gput_right:Nn</code>	10031
<code>\stexstyleinterpretmodule</code>	<code>\str_gset:Nn</code>	7877
<code>\stexstylemathstructure</code>	<code>\str_gset_eq:NN</code>	844, 7874, 7875
<code>\stexstylemcb</code>	<code>\str_if_empty:NTF</code>	
<code>\stexstyleMMTinclude</code>	 43, 61, 120, 162, 181,	
<code>\stexstylemodule</code>	458, 569, 575, 650, 656, 780, 798,	

837, 839, 858, 960, 963, 1066, 1323,	2232, 2237, 2248, 2253, 2263, 2266,
1345, 1843, 2337, 2371, 2624, 2839,	2270, 2356, 2357, 2860, 2874, 2886,
2899, 3637, 3651, 3712, 3875, 3941,	2890, 2920, 2952, 2953, 2989, 2990,
4044, 4047, 4050, 4053, 4056, 4280,	3025, 3026, 3069, 3085, 3287, 3288,
4287, 4337, 4400, 4403, 4406, 4409,	3384, 3417, 3445, 3453, 3633, 3639,
4454, 4490, 4493, 4558, 4561, 4564,	3643, 3751, 3772, 3801, 3825, 3874,
4567, 5122, 6245, 6309, 6435, 6447,	3876, 3878, 3880, 3942, 3996, 4000,
6456, 6502, 7054, 7074, 7379, 7392,	4004, 4008, 4012, 4016, 4020, 4024,
7393, 7400, 7435, 7590, 7591, 7604,	4028, 4270, 4336, 4338, 4478, 4489,
7605, 7761, 7772, 7773, 7880, 7881,	4491, 4494, 4680, 5118, 5123, 6172,
7885, 7886, 7923, 7936, 8019, 8025,	6174, 6198, 6310, 6355, 6514, 7055,
8049, 8060, 8061, 8085, 8086, 8409,	7075, 7086, 7216, 7218, 7221, 7411,
8473, 8650, 9118, 9213, 9218, 9295,	7925, 7938, 8027, 8463, 9297, 9360,
9338, 9358, 9406, 9456, 9497, 9516,	9408, 9458, 9518, 10253, 10497, 10499
9529, 9547, 10234, 10251, 10277,	<code>\str_set_eq:NN</code> 855,
10289, 10292, 10498, 10502, 10513	1023, 1024, 1177, 1194, 1195, 1201,
<code>\str_if_empty:nTF</code> 89, 142, 193, 496, 826, 3686, 4110	1206, 1891, 1927, 1928, 1929, 2856,
<code>\str_if_eq:NNTF</code> 294, 1933, 1994, 2855	2870, 3086, 3828, 3851, 4360, 4519,
<code>\str_if_eq:nnTF</code> 58,	6312, 6354, 6366, 6376, 7057, 7232,
63, 90, 157, 194, 195, 266, 462,	7331, 7394, 7410, 7437, 7864, 8100
570, 576, 668, 1034, 1301, 1342,	<code>\str_uppercase:n</code> 1543, 8545
1910, 2010, 2901, 3195, 3347, 3407,	<code>\l_tmpa_str</code> 461, 465, 466, 467, 469, 1004, 1006,
3428, 3443, 3451, 3704, 3720, 4283,	1222, 1223, 3069, 3071, 3086, 3090
4304, 4466, 4469, 4748, 6221, 6231,	<code>\string</code> 1879, 10017, 10058
6246, 6438, 6450, 6462, 6572, 7514,	<code>\subparagraph</code> 26, 84
7628, 8477, 8480, 8545, 8731, 9619	<code>subproblem (env.)</code> 9200
<code>\str_if_eq:p:nn</code> 4244, 4245, 4313, 4314	<code>subproof (env.)</code> 158, 7864
<code>\str_if_in:NnTF</code> 6173, 6197, 6526	<code>\subproof</code> 7727, 7987
<code>\str_item:Nn</code> 157, 3982	<code>\subproofautorefname</code> 7864
<code>\str_item:nn</code> 266	<code>\subsection</code> 26, 84, 139
<code>\str_lowercase:n</code> 9619	<code>\subsubsection</code> 26, 84
<code>\str_map_break:</code> 3993	<code>\svar</code> 97,
<code>\str_map_break:n</code> . 3994, 3998, 4002,	150, 3974, 5305, 5905, 5907, 5940, 5942
4006, 4010, 4014, 4018, 4022, 4026	<code>\symbol</code> 91
<code>\str_map_function:nN</code> 10041	<code>\symdecl</code> 22, 30, 31, 36, 41, 89–91, 101,
<code>\str_map_inline:Nn</code> 3991	106, 143, 147, 149, 258, 3327, 3794,
<code>\str_new:N</code> 453,	8209, 8211, 8239, 8250, 8264, 8267
1479, 2027, 2028, 2029, 3794, 10023	<code>\symdef</code> 31–
<code>\str_put_left:Nn</code> 62, 799	33, 37, 90, 97, 143, 147, 149, 258,
<code>\str_put_right:Nn</code> 7661	3329, 3823, 8186, 8188, 8191, 8194,
<code>\str_range:Nnn</code> 2145	8199, 8202, 8204, 8206, 8208, 8218,
<code>\str_range:nnn</code> 59, 64	8219, 8220, 8221, 8222, 8223, 8227,
<code>\str_replace_all:Nnn</code> 156	8233, 8245, 8246, 8260, 8262, 8271,
<code>\str_set:Nn</code> 13, 14, 44, 46,	8272, 8275, 8277, 8280, 8282, 8284
50, 51, 52, 53, 54, 55, 56, 60, 83, 94,	<code>\Symname</code> 91, 102, 152, 6047
149, 150, 151, 154, 170, 254, 459,	<code>\symname</code> 18,
461, 750, 781, 783, 787, 788, 789,	19, 23, 56, 91, 92, 102, 104, 152, 6047
790, 791, 792, 793, 797, 879, 999,	<code>\symref</code> 18, 19,
1004, 1012, 1013, 1014, 1020, 1021,	23, 42, 90, 91, 95, 102, 104, 152, 6040
1030, 1173, 1210, 1215, 1222, 1235,	<code>\symrefemph</code> 104, 105, 151, 6001, 8988
1481, 1527, 1799, 1873, 1882, 1883,	<code>\symuse</code> .. 42, 91, 149, 4574, 4604, 4898,
1912, 1944, 2032, 2034, 2036, 2039,	4905, 4916, 4999, 5065, 5153, 5154,
2041, 2069, 2070, 2143, 2210, 2221,	5904, 5918, 5936, 5939, 5949, 5953

sys commands:	
\sys_get_shell:nnN	80
\sys_if_platform_windows:TF	86, 147, 228, 238, 262
T	
\target	124
\test	76, 118
test	113
\testbigspace	9975
\testemptypage	75, 118
\testemptypage	9969, 10309
testheading (env.)	77, 119
\testheading	10411
\testmedspace	9974
\testnewpage	75, 118
\testnewpage	9976, 9977
\testnewpageInProblem	9977
\testsmallspace . 75, 75, 75, 118, 118, 118	
\testsmallspace	9973
\testspace	75, 118
\testspace	9318, 9972
\TeX	23
\tex	23
TeX and L ^A T _E X 2 _ε commands:	
\@	2175, 2178, 2182
\@addtoreset	9080
\@arabic	8854
\@author	8833
\@auxout	1816, 8300, 9194
\@bonuspointsfalse	10442
\@bonuspointstrue	10447
\@currentHref . 1823, 7925, 7938, 8027	
\@currentlabel	7924,
7925, 7937, 7938, 8026, 8027, 8649	
\@currentlabelname	1824
\@currenvir	8519, 8520
\@dblarg	8826, 8835
\@gobble	48
\@ifnextchar	47
\@ifstar	8774
\@listdepth	2086
\@mainmatterfalse	1753, 1766
\@mainmattertrue	1777, 1788
\@notprerr	1688
\@onlypreamble	1688
\@rustexfalse	664
\@title	1520, 1521, 8842
\activate@excursion	8881, 8886
\bbl@loaded	9044, 9999
\beamer@shortauthor . 8829, 8831, 8846	
\beamer@shorttitle . 8838, 8840, 8849	
\c@chapter	8500
\c@framenumber	8854
\c@part	8512
\compemph@uri 105, 151, 5957, 6181, 6735	
\correction@table	10390
\defemph@uri 105, 151, 5990, 6204	
\define@key	2230, 2246, 2261
\Gin@eheight 10563, 10566, 10571, 10576	
\Gin@ewidth	
.... 8765, 8766, 10562, 10572, 10576	
\Gin@exclamation . 10562, 10563, 10571	
\Gin@mhrepos	
.. 2232, 2237, 2241, 2263, 2270, 2275	
\hwexam@bonuspts	10444
\hwexam@checktime	10438
\hwexam@duration	
.... 10414, 10417, 10423, 10428	
\hwexam@kw@due	10365
\hwexam@kw@forgrading	10406
\hwexam@kw@given	10362
\hwexam@kw@grade	10407
\hwexam@kw@probs	10407
\hwexam@kw@pts	10408
\hwexam@kw@reached	10409
\hwexam@kw@sum	10407
\hwexam@kw@testemptypage 9970, 10313	
\hwexam@min 10415, 10422, 10427, 10438	
\hwexam@minutes@kw	10415
\hwexam@reqpts	
.... 10424, 10429, 10441, 10445	
\hwexam@tools	10425, 10430
\hwexam@totalmin	10437, 10438
\hwexam@totalpts	10436, 10445
\if@bonuspoints	10440
\if@rustex	672
\itemize@inner	8585, 8591, 8600
\itemize@label	8589, 8592, 8595
\itemize@level 8583, 8588, 8591, 8600	
\itemize@outer	8584, 8588
\lst@mhrepos	2248, 2253, 2257
\ltx@ifpackageloaded . 2229, 2245,	
2260, 8158, 9043, 9643, 9674, 9998	
\mdf@patchamsthm	8622
\metakeys@show@keys	8586
\notesslides@slidelabel .. 8625, 8641	
\ns@author	8826, 8827
\ns@title	8835, 8836
\pgf@temp	2172, 2173, 2174
\pgfkeys@spdef	2172
\pgfutil@empty	2174
\pgfutil@InputIfFileExists ... 2181	
\prematurestop@endsfragment	
.... 8518, 8521, 8527	
\problem@kw@*	9040
\problem@kw@correct	9658, 9796
\problem@kw@feedback ... 9551, 10293	

<code>\problem@kw@grading</code>	9497, 10234	<code>\thismodulename</code> ..	2357, 2953, 2990, 3026
<code>\problem@kw@hint</code>	9397	<code>\thismoduleuri</code>	2356, 2952,
<code>\problem@kw@minutes</code>	9168,		2989, 3025, 3489, 3514, 3556, 3580
	9261, 9564, 10077, 10119, 10165, 10206	<code>\thisname</code>	7331
<code>\problem@kw@note</code>	9445	<code>\thisnotation</code>	3852, 4361, 4520, 6357, 6377
<code>\problem@kw@points</code>	9548,	<code>\thisnotationvariant</code>	
	10072, 10114, 10160, 10201, 10290		3851, 4360, 4519, 6354, 6376
<code>\problem@kw@pts</code>	9163, 9256, 9559	<code>\thisstyle</code>	505, 508,
<code>\problem@kw@solution</code>	9337		509, 510, 543, 546, 547, 548, 549,
<code>\problem@kw@wrong</code>	9660, 9798		557, 560, 561, 2954, 2991, 3027,
<code>\problem@restore</code>	9195, 9199, 10384		3819, 3850, 3921, 4359, 4518, 6375
<code>\stex@backend</code>	128, 662	<code>\thistitle</code>	105, 2348, 2349, 2350,
<code>\symrefemph@uri</code>	104, 105,		7330, 9134, 9181, 9182, 9186, 9187
	151, 6001, 6044, 6067, 6084, 9007, 9030	<code>\thistype</code>	3816, 3847
<code>\varemp@uri</code>		<code>\thisvarname</code>	4358, 4517, 6374
	105, 151, 5981, 6092, 6106, 6120	<code>\throwaway</code>	144
tex commands:		<code>\tikzinput</code>	
<code>\tex_undefined:D</code>	54		121, 141, 2275, 10555, 10560, 10586
<code>\texname</code>	23	tikzinput commands:	
<code>\text</code>	19	<code>\c_tikzinput_image_bool</code>	
<code>\textbf</code>	18, 5995, 8400, 9006,		38, 10544, 10551
	9029, 9182, 9185, 9950, 10355, 10360	<code>\times</code>	8252, 8256
<code>\textcolor</code>	9946	<code>\tiny</code>	152, 6262, 8626, 8641, 9006, 9029
<code>\textsymdecl</code>	23, 90, 147, 149, 3328, 3860	<code>\title</code>	76, 119
<code>\texttt</code>	9930, 9946	<code>\title</code>	1612, 8835
<code>\textwarning</code>	63, 111	<code>titlefragment (env.)</code>	139, 1606
<code>\textwidth</code>	8626, 8763, 8785, 10403	tl commands:	
<code>\the</code>	334, 335, 337,	<code>\tl_clear:N</code>	
	339, 341, 353, 355, 2175, 2176, 2177		282, 393, 505, 543, 1280, 1548,
<code>\theassignment</code>	10343, 10355		1801, 1802, 1803, 1842, 1952, 2316,
<code>\thechapter</code>	1567, 1633, 1666, 1706, 1717,		2691, 2954, 2991, 3027, 3052, 3183,
	8426, 8431, 8435, 8439, 8443, 8447		3215, 3365, 3383, 3624, 3779, 3780,
<code>\theframenumber</code>	8649		3781, 3819, 3850, 3862, 3863, 3902,
<code>\thepage</code>	10311		3921, 3978, 4191, 4359, 4502, 4518,
<code>\theparagraph</code>	8443, 8447		4675, 4684, 5096, 5131, 5178, 5315,
<code>\thepart</code>	1563, 1630, 1662, 1700, 8421		5387, 5532, 5696, 5697, 5698, 5699,
<code>\theplainsproblem</code>	9084, 9085		6026, 6027, 6028, 6036, 6179, 6375,
<code>\thesection</code>			6392, 6403, 6404, 6494, 6520, 6659,
	8431, 8435, 8439, 8443, 8447, 9085		6739, 6750, 6785, 6977, 7049, 7247,
<code>\thesproblem</code>	9081, 9085, 9185,		7299, 7300, 7301, 7398, 7566, 7567,
	9195, 10069, 10090, 10110, 10132,		7568, 7569, 7622, 7623, 7624, 8898,
	10157, 10179, 10197, 10220, 10343		9062, 9508, 9626, 9628, 9629, 9871,
<code>\thesubparagraph</code>	8447		10318, 10319, 10320, 10395, 10396,
<code>\thesubsection</code>	8435, 8439, 8443, 8447		10397, 10422, 10423, 10424, 10425
<code>\thesubsubsection</code>	8439, 8443, 8447	<code>\tl_const:Nn</code>	
<code>\this</code>	50–52, 98, 100, 156, 274, 4677,		1053, 7001, 7002, 9880, 9881
	5015, 5102, 5103, 5112, 5141, 7222	<code>\tl_count:N</code>	5478, 5484
<code>\thisarchive</code>	2988	<code>\tl_count:n</code>	5782, 5792, 5798
<code>\thisargs</code>	3818, 3849	<code>\tl_gclear:N</code>	2386, 2446
<code>\thiscopyname</code>	3488, 3513, 3555, 3579	<code>\tl_gput_left:Nn</code>	365, 367, 368
<code>\thisdeclname</code>	3815, 3846, 3920, 6355	<code>\tl_gput_right:Nn</code>	
<code>\thisdecluri</code>	3814, 3845, 3919, 6356		335, 2548, 2555, 2612, 2644,
<code>\thisdefiniens</code>	3817, 3848		2649, 2668, 2669, 2675, 2999, 8882
<code>\thisfor</code>	7332		

\tl_gset:Nn	257, 337, 360, 361,	\tl_item:nn	5785
	933, 1032, 1457, 1460, 1492, 1499,	\tl_map_inline:nn	314, 318
	2023, 2511, 4619, 4623, 6407, 7890,	\tl_new:N	1477, 1480, 2026,
	7896, 8296, 9240, 9243, 10387, 10388		2309, 2546, 2641, 2646, 2664, 2665,
\tl_gset_eq:NN			2679, 3039, 4615, 4663, 4664, 5091
	81, 1080, 1081, 1086, 1087, 2634, 3140	\tl_put_left:Nn	
\tl_head:N	4197, 4796, 5027, 5045		4687, 5566, 6868, 6869, 9108, 10331
\tl_head:n	5783, 5793, 5799, 6938	\tl_put_right:Nn	
\tl_if_empty:NTF	298, 557,		1084, 3348, 3352, 3980,
	1463, 1511, 1520, 1593, 1613, 1857,		3981, 4669, 4686, 5095, 5201, 5255,
	1887, 1889, 1919, 1923, 1958, 1960,		5272, 5273, 5283, 5284, 5297, 5543,
	1982, 2338, 2349, 2374, 3058, 3187,		6500, 6506, 6518, 6527, 6803, 6808,
	3219, 3259, 3388, 3392, 3655, 3890,		6811, 6900, 6908, 6913, 6921, 6984,
	3959, 4061, 4064, 4067, 4207, 4418,		7183, 9895, 9907, 10399, 10400, 10401
	4421, 4424, 4432, 4500, 4577, 4580,	\tl_set:Nn	90,
	4583, 4591, 5355, 5367, 5414, 5498,		91, 93, 94, 295, 308, 492, 497, 499,
	5554, 5597, 5613, 5648, 5666, 5825,		536, 537, 827, 828, 830, 831, 1043,
	6015, 6274, 6278, 6282, 6287, 6386,		1179, 1279, 1282, 1333, 1336, 1361,
	6395, 6401, 6598, 6611, 6628, 6645,		1376, 1385, 1602, 1751, 1756, 1764,
	6748, 6784, 7226, 7336, 7497, 7546,		1813, 1874, 1905, 1963, 2004, 2011,
	7786, 7788, 7791, 7794, 8164, 8766,		2381, 2414, 2428, 2441, 2503, 2534,
	8910, 8996, 9019, 9126, 9129, 9135,		2581, 2846, 2852, 2853, 2854, 2926,
	9181, 9186, 9226, 9229, 9232, 9237,		2988, 3063, 3204, 3231, 3239, 3280,
	9337, 9397, 9445, 9534, 9550, 9658,		3295, 3408, 3419, 3420, 3421, 3422,
	9660, 9663, 9695, 9697, 9699, 9707,		3423, 3429, 3488, 3489, 3513, 3514,
	9796, 9798, 9801, 9829, 9831, 9833,		3555, 3556, 3579, 3580, 3634, 3640,
	9841, 9947, 10282, 10338, 10345,		3644, 3814, 3845, 3881, 3919, 3944,
	10356, 10361, 10364, 10414, 10441		4071, 4217, 4219, 4220, 4221, 4222,
\tl_if_empty:nTF . . .	265, 274, 362,		4223, 4257, 4259, 4260, 4261, 4262,
	901, 1100, 1102, 1106, 1110, 1112,		4263, 4318, 4320, 4321, 4322, 4323,
	1128, 1133, 1162, 1246, 1304, 1322,		4324, 4383, 4428, 4480, 4481, 4482,
	1415, 1472, 1981, 2033, 2155, 2315,		4483, 4484, 4542, 4587, 4618, 4620,
	2743, 3034, 3054, 3066, 3132, 3255,		4627, 4639, 4640, 4641, 4660, 4676,
	3315, 3692, 4096, 4137, 4178, 4195,		4681, 4682, 4683, 4688, 4764, 4791,
	4831, 4834, 4880, 4890, 4980, 4982,		4809, 4845, 4846, 4850, 4853, 4860,
	5066, 5083, 5094, 5155, 5189, 5253,		4861, 4897, 4937, 5011, 5017, 5070,
	5274, 5307, 5442, 5462, 5745, 6052,		5099, 5102, 5105, 5108, 5109, 5133,
	6212, 6590, 6605, 6618, 6691, 6725,		5134, 5137, 5143, 5152, 5157, 5196,
	6830, 6878, 7209, 7352, 7493, 7542,		5197, 5201, 5254, 5256, 5264, 5266,
	7556, 8308, 8828, 8837, 9927, 9951		5269, 5275, 5277, 5280, 5292, 5293,
\tl_if_empty_p:N	288		5308, 5309, 5311, 5312, 5478, 5484,
\tl_if_eq:NNTF . .	974, 1878, 4796,		5497, 5503, 5506, 5509, 5531, 5553,
	5027, 5045, 7249, 9890, 9891, 9900		5817, 5818, 5826, 5858, 5860, 5867,
\tl_if_eq:NnTF	8938,		5903, 5915, 5917, 5938, 5970, 6061,
	9141, 9144, 9162, 9167, 10071,		6062, 6078, 6079, 6093, 6094, 6107,
	10076, 10093, 10113, 10118, 10135		6108, 6121, 6122, 6171, 6193, 6194,
\tl_if_eq:nnTF . .	2526, 3229, 3237,		6356, 6387, 6396, 6420, 6436, 6439,
	4742, 5793, 5799, 5854, 5892, 6953		6448, 6451, 6457, 6464, 6483, 6586,
\tl_if_exist:NTF	307,		6588, 6591, 6593, 6787, 6794, 6800,
	334, 347, 509, 548, 560, 671, 902,		6839, 6853, 6854, 6866, 6905, 6918,
	1042, 1095, 1099, 1109, 1221, 1360,		6930, 6950, 6976, 6998, 6999, 7036,
	1535, 1545, 1852, 2198, 2561, 2565,		7037, 7079, 7093, 7198, 7227, 7250,
	2569, 2582, 7025, 7030, 8452, 10055		7420, 7427, 7785, 8157, 8163, 8179,
\tl_if_in:NnTF	2772, 2776, 2794		8421, 8422, 8426, 8427, 8431, 8432,

8435, 8436, 8439, 8440, 8443, 8444, 8447, 8448, 9063, 9135, 9179, 9210, 9211, 9237, 9277, 9578, 9642, 9673, 9678, 9693, 9751, 9785, 9811, 9812, 9827, 9889, 10375, 10381, 10382, 10436, 10437, 10438, 10444, 10509	<code>\today</code> 8856, 10499
<code>\tl_set_eq:NN</code> 68, 253, 364, 366, 371, 923, 1059, 1070, 1072, 1234, 1351, 1926, 2348, 2391, 2423, 2451, 2600, 2851, 3056, 3257, 3815, 3816, 3817, 3818, 3846, 3847, 3848, 3849, 3905, 3920, 4349, 4358, 4498, 4517, 4943, 5015, 5112, 5141, 5418, 5807, 5865, 6170, 6374, 6441, 6453, 6477, 6623, 6625, 6629, 6655, 6657, 6799, 7330, 7495, 7544, 7620, 9134, 9139, 9140	<code>\TODO</code> 4881, 8936
<code>\tl_set_rescan:Nnn</code> 466	token commands:
<code>\tl_tail:n</code> 2743, 4906, 5769	<code>\c_math_subscript_token</code> 43, 89, 4449, 5970, 6770, 6771, 6774, 6775, 6779, 8221, 8243, 8245
<code>\tl_to_str:n</code> 68, 74, 104, 107, 128, 131, 143, 166, 173, 235, 315, 319, 320, 426, 439, 860, 934, 935, 1018, 1044, 1045, 1055, 1118, 1127, 1129, 1131, 1132, 1134, 1245, 1247, 1249, 1304, 1310, 1316, 1386, 1414, 1416, 1418, 1419, 1422, 1498, 1832, 1833, 1834, 1835, 1837, 2023, 2054, 2153, 2491, 2492, 2493, 2496, 2497, 2498, 2499, 2539, 2540, 2556, 2654, 2674, 2759, 2773, 2777, 2795, 2811, 2819, 3125, 3247, 3281, 3438, 3606, 3626, 3656, 4089, 4090, 4091, 4092, 4117, 4292, 4803, 4823, 4827, 4988, 5046, 5075, 5093, 5128, 5132, 5208, 5222, 5244, 5263, 5523, 5744, 5759, 6480, 6548, 6574, 6838, 6851, 6940, 6975, 7316, 8180, 8181, 8296, 8297, 9044, 9898, 9908, 9930, 9946, 9960, 9962, 9999, 10041	<code>topsect</code> 60
<code>\tl_trim_spaces:N</code> 84	<code>\topsep</code> 7854, 9250, 9524, 9543, 9571, 9736, 10153, 10193, 10270
<code>\l_tmpa_tl</code> 80, 81, 83, 1064, 1226, 1228, 1336, 1350, 1351, 2205, 2510, 2511, 4894, 5264, 5272, 5273, 5275, 5283, 5284, 5465, 5471, 5476, 5478, 5482, 5484, 5487, 5496, 5498, 5503, 5506, 5509, 5903, 5920, 5938, 5949, 6510, 6977, 6984, 6991, 7785, 7786, 9693, 9707, 9827, 9841, 9889, 9890, 9891, 9900, 10395, 10399, 10407	<code>\trueFfalseT</code> 9865
<code>\l_tmpb_tl</code> 1333, 1341, 1345, 5465, 5471, 5473, 5487, 5497, 5501, 10396, 10400, 10408	<code>\trueFfalseF</code> 9860
tmpc commands:	<code>\type</code> 76, 119
<code>\l_tmpc_tl</code> 10397, 10401, 10409	
<code>\to</code> 8240, 8241, 8253	
	U
	<code>\underbrace</code> 6232, 6234, 8247
	<code>\univ</code> 56
	<code>\unless</code> 8519
	<code>\uppercase</code> 149, 1991, 6072
	<code>\url</code> 1451
	use commands:
	<code>\use:N</code> 375, 506, 510, 512, 544, 549, 551, 558, 561, 563, 889, 896, 1033, 1594, 1596, 1853, 1854, 1902, 2001, 2588, 3892, 7740, 7748, 7764, 7826, 7844, 7941, 7954, 7965, 7967, 7996, 8005, 8054, 8071, 8453, 8459
	<code>\use:n</code> 329, 2144, 2229, 2245, 2260, 4068, 4425, 4584, 4606, 4997, 5119, 5533, 5569
	<code>\use:nn</code> 93, 304, 474, 477, 889, 896, 1596, 1903, 2002, 2537, 3062, 3191, 3289, 3853, 3886, 3954, 4129, 4362, 4761, 4784, 4788, 5064, 5149, 5245, 5579, 5769, 5820, 5910, 6083, 6156, 6358, 6378, 6711, 6741, 7083, 7189, 7190, 7413, 7524, 7558, 7594, 7599, 7606, 7609, 8020, 8914, 9279, 10038, 10377
	<code>\use_i:nn</code> 3191, 5119, 10400
	<code>\use_i:nnn</code> 882, 1252
	<code>\use_i:nnnn</code> 1428
	<code>\use_i:nnnnn</code> 1138
	<code>\use_ii:nn</code> 3179, 3197, 5166
	<code>\use_ii:nnn</code> 885, 889, 1255, 1258
	<code>\use_ii:nnnn</code> 1431
	<code>\use_ii:nnnnn</code> 1141
	<code>\use_iii:nnn</code> 892, 896, 1261, 1265
	<code>\use_iii:nnnnn</code> 1144
	<code>\use_iv:nnnn</code> 1444, 1447
	<code>\use_iv:nnnnn</code> 1147
	<code>\use_none:n</code> 7881, 7886
	<code>\use_none:nn</code> 7200
	<code>\use_v:nnnnn</code> 1150
	<code>\usebox</code> 8709

<code>\usemodule</code>	16, 21, 22, 25, 35, 43, 95, 96, 98, 124, 128, 145, 416, 423, 2938	<code>\vfill</code>	8457, 9970, 9978, 10313
<code>\usepackage</code>	7, 22, 2144, 8533	<code>\vM</code>	46
<code>\useSGvar</code>	63, 112, 8929	<code>\vMs</code>	48
<code>\usestructure</code>	98, 156, 7261	<code>\vop</code>	39, 41
<code>\usetheme</code>	8533	<code>\vphantom</code>	9962
<code>\usetikzlibrary</code>	82, 140, 2152, 10554	<code>\vrule</code>	8159, 8785, 9644, 9675
V			
<code>\varbind</code>	41, 97, 102, 157, 7450, 7465, 7531	<code>\vskip</code>	2087, 2088, 8159, 8784, 8786, 9644, 9675
<code>\vardef</code>	33, 41, 97, 98, 148, 4331, 4446, 7558, 7594, 7599, 7606, 7609, 8020	<code>\vspace</code>	9972
<code>\varemp</code>	105, 151, 5981, 8990	W	
<code>\Varname</code>	152, 6088	<code>\wff</code>	19
<code>\varname</code>	152, 6088	<code>\withbrackets</code>	94, 155, 7034, 7045
<code>\varnotation</code>	97, 153, 6363	X	
<code>\varref</code>	152, 6088	<code>\xdef</code>	10017, 10069, 10110, 10157, 10197, 10311
<code>\varseq</code>	38, 98, 149, 150, 4449, 4487	<code>\xspace</code>	139, 1535, 1545, 3936, 3937, 4698, 4699
<code>\vbox</code>	144, 1900, 1998, 2074, 2087, 7743, 7956, 8159, 8620, 8716, 8722, 8738, 9319, 9379, 9427, 9478, 9644, 9675	Y	
vbox commands:		<code>\yesFnoT</code>	9855
<code>\vbox_set:Nn</code>	2722	<code>\yesTnoF</code>	9850
<code>\vdash</code>	8272	<code>\yield</code>	158, 7732, 8133